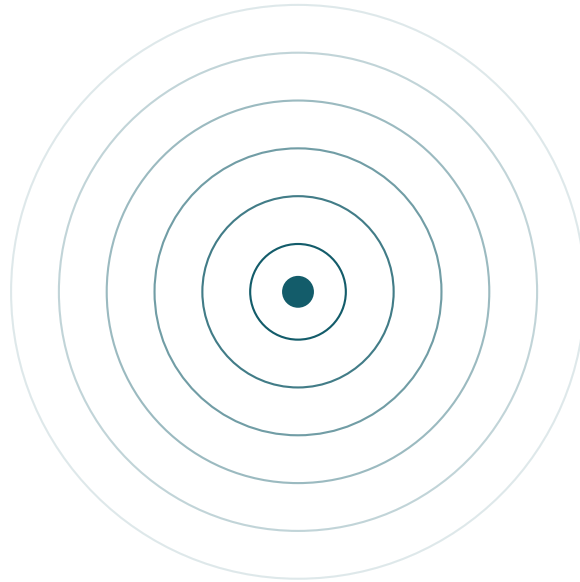


From deciding what to build to a running, observable production system



The Complete Software Engineering Guide

FIRST EDITION

The Complete Software Engineering Guide

*From Deciding What to Build
to a Running Production System*

Product Discovery · Project Planning · Writing Good Code · Building Real Applications

FIRST EDITION

The Complete Software Engineering Guide

From Deciding What to Build to a Running Production System

First Edition, 2026. Independent digital edition.

Copyright © 2026. All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means without prior written permission.

The programs in this book are included for their instructional value. They have been written with care, but no warranty, express or implied, is offered with respect to them, and no liability is accepted for incidental or consequential damages arising from their use.

ISBN: not assigned.

Typeset in Libertinus Serif, Fira Sans, and Menlo with $\LaTeX$. Source: Markdown, converted with pandoc.

*For every developer who has wondered
what actually happens after they press Enter.*

Contents

Preface	xxiii
How Software Gets Built	1
The One-Sentence Version	1
Layer 1: The Loop	2
Layer 2: The Pieces.	5
Layer 3: The People	7
The Same Map, for Any Application	9
How to Use the Map	10
How This Book Walks the Map.	10
How Web Applications Work	13
Two Sides of Every App	15
One App, Two Places.	15
The Restaurant Picture	15
Client and Server	16
The Request and Response Loop	16
Why Split an App at All?	17
Try It	17
Summary, Key Terms, and Review Questions	17
The Front End: What Runs on Your Screen	19
The Server Sends Instructions, Not a Picture	19
Three Languages, Three Jobs	19
One Page, Every Screen	21
Front-End Frameworks.	21
Mobile Apps Are Front Ends Too	21
What the Front End Can't Do.	21
Try It	22
Summary, Key Terms, and Review Questions	22

The Back End: What Runs Where You Can't See	25
A Computer That Never Sleeps	25
Back-End Frameworks	25
Business Logic: The House Rules	26
The Database: Where Your Data Lives.	27
How the Server Remembers You	27
APIs: The Contract Between the Two Sides	27
Try It	28
Summary, Key Terms, and Review Questions	28
Following One Tap from Start to Finish	31
The Button, Now for Real	31
The Whole Trip	32
Common Stacks	33
Try It	34
Summary, Key Terms, and Review Questions	34
Security, Speed, Failure, and Cost	37
Security: What Can Go Wrong	37
Speed: What Makes an App Feel Slow.	38
Failure: What Breaks, and How to Tell Where	38
Cost: What It Costs to Run	38
Where to Go from Here	41
What to Learn Next	41
What to Skip for Now	41
Part I • Product Discovery	43
Product Discovery	45
Why Discovery Exists: The Cost of Building the Wrong Thing	45
The Four Risks Every Idea Must Survive	46
Understanding the Problem and the User	46
Validating That It's Worth Solving	47
Shaping and Testing the Solution Cheaply.	47
Prioritization: Deciding What to Pursue.	48
A Worked Example: Discovery for a Notes Service.	49
Common Traps	49
The Handoff to Project Planning	50
A Discovery Checklist	50

Part II • Project Planning	53
Project Planning Guide	55
What This Guide Is	55
How to Read This Guide	55
Section 1 — Problem Statement & Goals	56
Section 2 — Architecture & Design	58
Section 3 — Developer Guide	62
Section 4 — Operational Guide	66
Section 5 — Delivery & Changelog	69
A Worked Example: Planning the Notes API	71
How This Guide Evolves	74
Part III • Writing Good Code	75
The Craft of Good Code	77
What “Good Code” Actually Means	77
Naming	78
Functions	78
Modules, Boundaries, and Design	79
Comments	79
Error Handling	80
Code Smells	80
Refactoring	81
Strategic vs Tactical Programming	81
A Practitioner’s Habits	81
Engineering Practices & Maturity	83
Overview	83
The Complete Roadmap at a Glance	84
Stage 1 — Project Foundation	84
Stage 2 — Early Development	85
Stage 3 — Growing Codebase	87
Stage 4 — Production Readiness	89
Stage 5 — Scale and Reliability	90
Stage 6 — Advanced Engineering Excellence	92
Full Quality Framework Reference	93
Priority Reference Matrix	97
Gaps and Domain-Specific Considerations	98

Part IV • Building Real Applications 101**Introduction to Part IV 103**

About This Part	103
How to Read This Part	106
A Note on Examples, Tools, and the Running Project	107

STAGE 1 • Creating a Running Program**1 Command → Process 113**

1.1 The Shell and What It Does with a Command	114
1.2 Executable Resolution via \$PATH	115
1.3 How the Operating System Starts a New Program: fork and exec	116
1.4 The Process Control Block: What the OS Remembers About Your Program	119
1.5 Memory Layout: Stack, Heap, and Why They Exist	121
1.6 File Descriptors: stdin, stdout, and stderr	124

2 Runtime Initialization 131

2.1 What the Python Interpreter Is and Why It Exists	132
2.2 Loading Source Code into Memory	133
2.3 Compilation to Bytecode: What .pyc Files Really Are	135
2.4 Importing Dependencies: What import Actually Does.	137
2.5 Framework Initialization: How Flask Starts Up	139
2.6 Global State Setup: What Happens Before the First Request	141

3 Creating Network Capability 149

3.1 What a Socket Is and Why It Exists	150
3.2 Creating a Socket: The socket() Call.	151
3.3 Binding to an Address: bind() and localhost:5000	152
3.4 Entering the Listening State: listen()	155
3.5 How the Kernel Registers Port-to-Process Mappings.	156
3.6 The Connection Queue: What Happens Before accept()	157

4 Defining Request Handling Paths 165

4.1 What a Route Is and What It Represents.	166
4.2 Defining Routes in Flask: @app.route()	167
4.3 HTTP Method Mapping: GET, POST, PUT, DELETE	169
4.4 How Flask Builds Its Internal Route Table	171
4.5 Binding Handler Functions to Routes	173
4.6 Defining Middleware Chains	175
4.7 Testing Locally: curl, Postman, and the Browser	178

5	Internal Request Execution	187
5.1	Accepting a Connection: <code>accept()</code>	188
5.2	Reading the Request: <code>recv()</code>	189
5.3	Parsing the HTTP Protocol: Turning Bytes into Structure	190
5.4	Extracting Metadata: Path, Method, Headers, Query Parameters, and Body	192
5.5	Route Matching and Handler Lookup	194
5.6	Middleware Execution: Authentication, Logging, Validation	196
5.7	Business Logic Execution.	197
5.8	Data Access: Database Calls During a Request.	199
5.9	Response Construction: Status Code, Headers, Body	200
5.10	Serialization: Turning Python Objects into JSON	202
5.11	Sending the Response: <code>send()</code>	204
6	Local Dependencies and Resources	209
6.1	Runtime Dependencies: What Python Itself Depends On.	210
6.2	Library Dependencies: <code>pip</code> and the Package Ecosystem	212
6.3	Virtual Environments: Isolating Your Project's Dependencies.	213
6.4	Local Databases and the Filesystem	217
6.5	Environment Variables and the <code>.env</code> File	224
 STAGE 2 • Breaking the Local Illusion		
7	Environment Drift	237
7.1	The Illusion of “It Works on My Machine”	238
7.2	Operating System Differences: macOS, Windows, and Linux	239
7.3	Runtime Version Mismatches: Python 3.9 vs 3.11	242
7.4	Missing System Libraries: <code>libc</code> , <code>OpenSSL</code> , and Silent Failures	244
7.5	File System Differences: Paths, Permissions, and Line Endings	247
8	Hidden Dependencies	253
8.1	The Difference Between Global and Local Installs	254
8.2	<code>PATH</code> Dependencies: Tools Your Code Assumes Are Present	256
8.3	External Binaries: System Programs Called from Python	259
8.4	Implicit Assumptions: What Your Code Relies On Without Knowing It	261
9	Configuration Coupling	267
9.1	Hardcoded Secrets	268
9.2	Local Database References	270
9.3	Debug Configurations	272
9.4	Environment-Specific Behavior Buried in Code	275
10	Networking Limitations	283
10.1	The Loopback Address: Why <code>127.0.0.1</code> Is Invisible to Others	283

10.2	No Public IP: What Your Machine Looks Like from the Internet	285
10.3	No DNS Mapping: Why <code>localhost</code> Is Meaningless Outside Your Machine	287
10.4	No Firewall Rules: Why Local Security Is Not Real Security	288
11	Reliability and Security Gaps	293
11.1	No Process Restart: What Happens When Your Program Crashes.	294
11.2	No Monitoring: Running Blind Without Visibility	295
11.3	No HTTPS: Why HTTP Is Not Safe Enough	297
11.4	No Authentication or Security Layers	299
 STAGE 3 • Making the Program Portable		
12	Dependency Declaration	307
12.1	<code>requirements.txt</code> : The Manifest of What Your Program Needs.	308
12.2	Version Locking: Why Exact Versions Matter	310
12.3	How <code>pip</code> Resolves and Installs Dependencies	311
12.4	Installing from <code>requirements.txt</code>	313
12.5	The Complete <code>requirements.txt</code> for the Notes API	314
12.6	<code>pyproject.toml</code> : The Modern Alternative	316
13	Configuration Externalization	321
13.1	Identifying What Needs to Be Configurable	322
13.2	Moving Values into Environment Variables	325
13.3	Runtime Config Loading with <code>os.getenv</code>	326
13.4	Validation of Required Configuration on Startup.	327
13.5	Switching from SQLite to PostgreSQL Through Configuration	329
13.6	The Full Updated <code>app.py</code> with Complete Configuration Externalization	332
14	Runtime Contract Definition	341
14.1	Required Python Version	342
14.2	Required Operating System Features	343
14.3	Required Ports	346
14.4	Required Environment Variables	347
15	Structuring the Application	355
15.1	The Problem with a Single File	356
15.2	The Standard Python Web Application Directory Structure	360
15.3	What a Python Package Is	361
15.4	The Application Factory Pattern	362
15.5	Building the Restructured Application	364
15.6	The Entry Point: <code>run.py</code>	379
15.7	The Updated Project Structure	380
15.8	The Startup Sequence in the Restructured Application	381

15.9	Verifying the Restructured Application	383
15.10	The Standardized Startup Command	385
16	Logging and Error Handling	389
16.1	What Is Wrong with <code>print()</code>	390
16.2	Python's logging Module: The Architecture	392
16.3	Configuring Logging for the Notes API	394
16.4	Request Logging Middleware	400
16.5	Error Handling: Catching Failures Before They Reach the Client	402
16.6	Integrating Everything into the Application Factory	408
16.7	Adding Logging to Route Handlers	411
16.8	What the Log Output Looks Like	413
16.9	Logging Best Practices	415
16.10	Debugging: From a Failure to Its Cause	417
16.11	The Updated <code>.env.example</code>	421
 STAGE 4 • Build and Packaging		
17	Build Process	427
17.1	What a “Build” Means for a Python Web Application	428
17.2	Setting Up the Test Infrastructure.	429
17.3	Writing the Test Infrastructure	431
17.4	Code Quality Tools.	443
17.5	The Makefile — Automating the Build	445
17.6	Configuration Files for Code Quality Tools	449
17.7	What Test Coverage Means.	451
17.8	A Test Suite You Can Trust	453
17.9	Working with Git: Branches, Pull Requests, and Review	462
17.10	Continuous Integration: The Build Running Automatically.	466
17.11	The Complete Updated Project Structure	469
18	Containerization	473
18.1	What a Container Is	474
18.2	Docker: The Container Toolchain.	476
18.3	The Dockerfile	476
18.4	How Docker Layers Work	482
18.5	Building and Running the Image	483
18.6	Docker Compose: Running Multiple Containers Together	486
18.7	Image Tagging and Versioning	490
18.8	Integrating Docker into the Build Process	491
18.9	Adding Docker Tests to CI	493
18.10	The Complete Updated Project Structure	494

19	Artifact Storage	497
19.1	What a Container Registry Is	498
19.2	Registry Options	499
19.3	Pushing to GitHub Container Registry	501
19.4	Integrating Registry Push into CI	502
19.5	The Tagging Strategy.	505
19.6	Beyond Docker Images: Other Artifacts	507
19.7	Image Lifecycle Management	509
19.8	The Complete Artifact Flow	511
19.9	The Updated Makefile	512
 STAGE 5 • Infrastructure and Environment		
20	Server Fundamentals	517
20.1	What a Production Server Is	517
20.2	Physical Servers vs Virtual Machines vs Cloud Instances.	519
20.3	Choosing a Cloud Provider	521
20.4	Understanding Server Resources	522
20.5	The Linux Distribution Choice	524
20.6	Sizing the Notes API Server.	525
20.7	Managed Services vs Self-Managed Servers	525
20.8	What the Production Server Must Provide.	527
20.9	The Architecture of the Complete System	528
21	System Setup	533
21.1	First Contact: SSH	533
21.2	Initial System Update.	535
21.3	Creating a Non-Root User	536
21.4	Securing SSH	537
21.5	Configuring the Firewall	539
21.6	Installing Docker Engine	541
21.7	Setting Up Automatic Security Updates	544
21.8	Creating the Application Directory Structure	545
21.9	Configuring System Limits	547
21.10	Configuring the Server Timezone and Locale	549
21.11	Installing Useful Server Tools.	549
21.12	Verifying the Setup.	551
21.13	The SSH Config File: Simplifying Connections.	554
22	Network Configuration	557
22.1	The Network Layers	557
22.2	Virtual Private Cloud (VPC)	559

22.3	The Public IP Address	560
22.4	Cloud-Level Firewalls	561
22.5	Verifying Network Connectivity	562
22.6	Docker Networking Inside the Container	564
22.7	Network Monitoring and Diagnostics	565
22.8	Network Security Best Practices	566

STAGE 6 • Deployment

23	Code Transfer	571
23.1	Why Pull Images, Not Copy Files	571
23.2	How <code>docker pull</code> Works	572
23.3	Authenticating the Server to the Registry	573
23.4	The Deployment Script	574
23.5	Zero-Downtime Deployment	579
23.6	Automating Deployment from CI	583
23.7	Rollback	585
24	Process Startup in Production	587
24.1	What WSGI Is	588
24.2	How Gunicorn Works	589
24.3	Configuring Gunicorn	591
24.4	Signals and Graceful Shutdown	596
24.5	PID 1 and Signal Handling in Containers	598
24.6	Docker Restart Policies	600
24.7	<code>systemd</code> Integration (an Alternative)	601
24.8	The Complete Startup Sequence	603
25	Process Management	607
25.1	What a Process Supervisor Is	608
25.2	<code>systemd</code> : The Linux Process Manager	609
25.3	Managing Services with <code>systemctl</code>	611
25.4	Logs with <code>journalctl</code>	613
25.5	Managing Multiple Processes on One Server	615
25.6	Process Crash Investigation	616
25.7	Process Resource Monitoring	618
25.8	The Relationship Between Docker and <code>systemd</code>	618
25.9	Beyond the Always-On Service: Scheduled and Triggered Work	619
25.10	A Process Management Cheat Sheet	623

STAGE 7 • Making It Reachable

26	Network Stack Flow	629
26.1	The Layered Network Model	629
26.2	The Journey of a Single Packet	630
26.3	The Server's Network Stack	632
26.4	TCP States	633
26.5	The Kernel Receive Path in Detail.	634
26.6	Nginx's Role in the Network Stack	635
26.7	The Complete Network Path (End to End).	636
26.8	Where Network Failures Occur	638
27	Domain Names and DNS	641
27.1	How DNS Works: The Resolution Chain	641
27.2	DNS Record Types	643
27.3	Registering a Domain	645
27.4	Creating DNS Records for the Notes API	646
27.5	TTL: Time to Live	648
27.6	Verifying DNS Configuration	649
27.7	DNS Propagation.	650
27.8	DNSSEC: DNS Security Extensions	651
28	Traffic Routing Layer	653
28.1	Why Unicorn Cannot Face the Internet Directly	654
28.2	Installing Nginx	655
28.3	Nginx Configuration Structure	655
28.4	Initial Nginx Configuration (HTTP only)	656
28.5	TLS Certificates with Let's Encrypt	657
28.6	The Complete Production Nginx Configuration	660
28.7	Understanding X-Forwarded-For	665
28.8	Nginx as a Load Balancer.	665

STAGE 8 • Request Flow: External to Program

29	Client-to-Server Journey	671
29.1	The Request: POST /notes/	671
29.2	Phase 1: DNS Resolution (Client)	671
29.3	Phase 2: TCP Connection (Client → Server).	672
29.4	Phase 3: TLS Handshake (Client ↔ Nginx)	673
29.5	Phase 4: HTTP Request (Client → Nginx).	674
29.6	Phase 5: Nginx → Gunicorn Proxy	675
29.7	Phase 6: Flask Request Processing.	676
29.8	Phase 7: Database Query	677
29.9	Phase 8: Response Construction	678
29.10	Phase 9: Response Path (Gunicorn → Client)	679
29.11	The Complete Timeline.	680
29.12	Log Output for This Request	680
30	Kernel to Application Handoff	683
30.1	User Space vs Kernel Space	683
30.2	The Accept System Call	684
30.3	Reading Data: <code>recv()</code> and <code>read()</code>	686
30.4	From Bytes to Request Object.	688
30.5	The Buffer Chain.	690
30.6	System Call Tracing	691

STAGE 9 • Inside the Running Program

31	Input Handling	697
31.1	Why Input Is Untrusted	697
31.2	The Four Categories of Validation.	698
31.3	A Reusable Validation Framework	700
31.4	URL Parameters and Query Strings	706
31.5	SQL Injection and Parameterized Queries	708
31.6	Input Handling for Headers.	709
31.7	Testing Input Validation	710
32	Routing	715
32.1	Registration: What <code>@app.route()</code> Does	715
32.2	The URL Map and Rule Objects	718
32.3	URL Matching: How a Request Finds Its Handler	719
32.4	URL Variable Converters	721
32.5	Routing Priority and Ambiguity	723
32.6	Blueprints and the URL Map	725

32.7	URL Building with <code>url_for()</code>	726
32.8	Inspecting Routes	728
32.9	How Flask Calls the Handler	729
32.10	Routing in the Notes API	730
33	Logic Execution	733
33.1	What Is Business Logic?	734
33.2	The Flask Request Context	735
33.3	Database Connection Management	738
33.4	Executing a Database Query	740
33.5	Transactions	742
33.6	The Complete Handler: <code>create_note</code>	744
33.7	Reading a Note: <code>get_note</code>	747
33.8	Listing Notes with Filtering	748
33.9	Updating and Deleting Notes	749
33.10	Migrations as a System	751
33.11	Growing the Schema: Tags	763
33.12	Querying Across Tables	768
33.13	Transactions in Depth: When Requests Collide	773
33.14	Raw SQL, Query Builders, and ORMs	778
33.15	Error Handling During Logic Execution	780
33.16	Structuring Logic for Testability	783
34	Response Handling	789
34.1	From Handler Return Value to HTTP Response	789
34.2	HTTP Status Codes	792
34.3	Response Headers	795
34.4	JSON Serialization	797
34.5	Error Response Structure	800
34.6	Content Negotiation	802
34.7	After-Request Hooks	803
34.8	The Complete Response Journey	805
34.9	Pagination for List Endpoints	807
34.10	The API as a Contract	810

STAGE 10 • Production Reality

35	Observability	821
35.1	The Three Pillars of Observability	821
35.2	Structured Logging in Depth	822
35.3	Health Checks	826
35.4	Metrics Collection	829
35.5	Log Aggregation	834
35.6	Alerting	836
35.7	The Nginx Access Log	839
35.8	The Observability Stack for the Notes API.	840
36	Scaling and Load Handling	843
36.1	Understanding Capacity	843
36.2	Identifying Bottlenecks.	846
36.3	Vertical Scaling	850
36.4	Horizontal Scaling	850
36.5	Connection Pooling	853
36.6	Caching	856
36.7	Rate Limiting	860
36.8	Gunicorn Worker Types for Higher Concurrency	862
36.9	Capacity Planning Summary	865
37	Reliability	869
37.1	Availability and the “Nines”	869
37.2	Failure Modes by Layer.	870
37.3	Graceful Degradation	873
37.4	Database Backups	875
37.5	Calling Another Service	880
37.6	Retry Logic and Exponential Backoff	890
37.7	Circuit Breaker Pattern.	893
37.8	Graceful Shutdown.	896
37.9	Runbooks	897
37.10	Postmortems.	899
38	Security	903
38.1	Threat Modeling	903
38.2	Transport Security: HTTPS Everywhere	905
38.3	Authentication: Who Is Asking?	905
38.4	Authorization: What May They Do?	919
38.5	Secrets Management	922
38.6	OWASP Top 10	925

38.7	Security Headers Deep Dive	930
38.8	Input Sanitization vs Validation.	932
38.9	Security Audit Checklist	932
39	Local vs Production	937
39.1	The Differences That Matter	937
39.2	The Database Difference	941
39.3	The Environment Variable Difference	946
39.4	The Network Difference	948
39.5	The Performance Difference	950
39.6	The Concurrency Difference	951
39.7	What Docker Guarantees (and What It Doesn't)	952
39.8	The Staging Environment	953
39.9	Simulating Production Locally	954
39.10	Environment Parity Principles	957
39.11	Delivering and Tracking the System Over Time	958
 STAGE 11 · Beyond a Single Service		
40	Distributed Systems	965
40.1	Why Distribute — and What It Costs	966
40.2	The Network Changes Everything	966
40.3	Spreading Data: Replication and Partitioning	967
40.4	Consistency and the CAP Trade-off	967
40.5	Transactions Across Services: the Saga	968
40.6	Idempotency Becomes Mandatory	968
40.7	Coordination: Consensus, Quorums, Leaders	969
41	Event-Driven and Asynchronous Architecture	971
41.1	Two Ways for Systems to Talk	972
41.2	The Message Queue	972
41.3	Delivery Semantics, and Why Idempotency Returns	973
41.4	Queues vs Pub/Sub, and What an “Event” Is	973
41.5	Building It: A Job Queue for the Notes API	974
41.6	Event Sourcing, CQRS, and Streaming.	989
41.7	Backpressure and the Real Costs	989
42	The Data and Analytics Plane	993
42.1	Two Workloads That Pull in Opposite Directions	993
42.2	Where Analytical Data Lives	994
42.3	Getting Data There: Pipelines.	995
42.4	Modeling Data for Analysis.	995

42.5	Trust: Quality, Lineage, and Governance	996
43	Cloud-Native and Orchestration	999
43.1	From One Server to a Fleet	1000
43.2	Kubernetes: Desired State and the Control Loop	1000
43.3	Infrastructure as Code	1001
43.4	The Other Direction: Serverless and Managed Services	1002
43.5	The Trade-off: Power vs Complexity	1002

THE PROJECT

44	Notes for Many Users	1005
44.1	How to Work Through This Project	1005
44.2	Milestone 1: Accounts and Ownership	1006
44.3	Milestone 2: Sharing, Search, and the Trash	1007
44.4	Milestone 3: Reminders.	1009
44.5	Milestone 4: A Suite and a Contract You Can Trust	1009
44.6	Milestone 5: Production	1010
44.7	Stretch Goals	1011
44.8	Comparing Your Solution with the Reference	1011

CLOSING

The Same Process in a Real Team	1015
The Team	1015
The First Version, as a Team	1016
Where Hand-offs Break	1018
A Change to a Live System	1019
The Rituals, and What They Are For	1020
The Same Map, for Any Application	1021
Where Specialists Sit	1023
Using the Map from Now On	1023
Where to Go Next	1024

APPENDICES

Appendix A — Python Environment Setup	1027
A.1 Installing Python.	1027
A.2 Virtual Environments	1029
A.3 pip: Installing Packages.	1030
A.4 The Project Structure.	1031
A.5 Common pip Issues and Fixes.	1032
Appendix B — Glossary	1035
Appendix C — Command Cheat Sheet	1043
C.1 Python and pip.	1043
C.2 pytest	1044
C.3 Docker.	1045
C.4 Docker Compose	1046
C.5 Gunicorn	1047
C.6 systemd	1047
C.7 Nginx	1048
C.8 PostgreSQL	1049
C.9 SSH and Server Management	1050
C.10 DNS Debugging.	1051
C.11 curl: Testing the API.	1051
Appendix D — Further Reading	1053
D.1 Python and Flask.	1053
D.2 Databases and SQL.	1053
D.3 Networking and HTTP.	1054
D.4 Production Operations	1055
D.5 Security	1055
D.6 Docker and Containers.	1056
D.7 Performance	1056
D.8 Command Line and Linux	1056
D.9 Online Resources.	1057
Appendix E — Walking Into an Unfamiliar System	1059
Appendix F — The Mental-Model Spine	1061
Appendix G — Python Essentials Used in This Book	1065
G.1 Modules, Packages, and Running Code	1065
G.2 Functions: Arguments and Unpacking.	1066
G.3 Decorators.	1067
G.4 Type Hints.	1067

G.5 Strings.	1068
G.6 Comprehensions and Built-ins	1068
G.7 Exceptions.	1069
G.8 Context Managers: with	1069
G.9 Classes, Briefly.	1070
G.10 Generators and yield.	1070
G.11 The Standard Library the Book Leans On.	1071
G.12 Async, Briefly.	1071

Preface

How this book is organized, and how to read it

Building software is a lifecycle, and each part of it is easiest to understand when it sits in its proper place. This book follows that lifecycle from end to end: from the moment an idea appears in someone's head to the moment it runs in production. It concentrates on *the craft of building software*. It shows where teams, roles and their rituals fit into the process, but how to run a team, and how to navigate a career, are left to other books.

The shape of the book

The book opens with a map and closes by returning to it. **How Software Gets Built**, the opening chapter, lays out the whole process before any of its parts: the loop every piece of software goes around, the pieces a running application is made of, and the people who do each part. Between the map and the closing chapter are four parts, one for each phase of the lifecycle. Each part stands on its own, and they are ordered the way software actually comes into being.

Part I • Product Discovery. Deciding *what* is worth building, before any code. It is the most commonly skipped phase, and the one that stops you building the wrong thing well.

Part II • Project Planning. Turning a chosen problem into a plan and a set of living documents: problem statement, architecture, developer and operational guides.

Part III • Writing Good Code. Two chapters: the craft of code that is correct, clear, and changeable, and the engineering practices a whole codebase needs, sequenced as a maturity model so you know the order in which to adopt them.

Part IV • Building Real Applications. A 43-chapter narrative, followed by a project, that takes a single Notes API from a command in a terminal to a containerized, HTTPS-served, monitored production service, and then on to the distributed systems beyond it. It is divided into eleven *stages*, from Stage 1, a program running on one laptop, to Stage 11, systems beyond a single service.

The closing chapter, **The Same Process in a Real Team**, replays the whole journey as a team makes it (who does each step, and what they hand to each other) and lays the same map over other kinds of software. Every part, and every stage of Part IV, opens by marking where you are on the map.

Two ways to read it

To learn, read the opening chapter first, then Part IV straight through; it builds the mental model of how a real system works. Then read Part III for the craft, and keep Part II for when you start something of your own. **To build something now**, work through Parts I and II to decide what to build and to frame it, use Part III as your checklist while you write it, and use Part IV as the reference for taking it all the way to production.

Exercises, a project, and a companion repository

Every chapter of Part IV ends with four exercises: *predict* what will happen, *break and fix* something on purpose, *extend* the code, and *explain* an idea in your own words. Part IV then closes with a project, *Notes for Many Users*, in five milestones, which you plan and build yourself. The companion repository holds the Notes API's code, a solution to every exercise (one file per chapter, in `exercises/`), and a complete, tested reference solution to the project, with one Git tag per milestone, so that you can compare your work at each step. Try each exercise, and each milestone, before you look.

Conventions used in this book

Source code appears in shaded listings with a colored rule on the left. Numbered listings carry a caption:

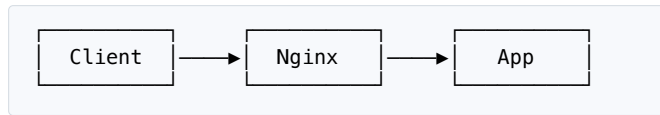
Listing 0.1 A minimal Python program

```
def greet(name: str) -> str:
    return f"Hello, {name}!"
```

Commands you type and the output they produce appear in plain framed blocks:

```
$ python app.py
* Running on http://127.0.0.1:5000
```

Diagrams are drawn in text, so they stay exact and diff-able in the source:



Note: asides, warnings and version-specific details appear in callouts like this one.

Each chapter of Part IV opens with what it will teach you and closes with its key takeaways.

Versions, tools, and a book that will age

Software tooling changes faster than books do. The principles here (how a process binds a port, why a transaction must commit before a response is sent, why a deployment needs a rollback) will hold for decades. The specifics will not: version numbers, command-line flags, package names, configuration syntax, default settings, cloud providers' products and prices, and even what counts as current best practice. Everything version-specific in this book was written and checked against the versions listed below. When you follow along later, expect some details to differ, and treat the official documentation of each tool, not this book, as the authority for version-specific facts. When a command or setting fails, check the tool's current documentation before assuming the concept is wrong. Prices, free-tier limits, and product names quoted from vendors were correct at the time of writing and are the details most likely to have changed.

Python	3.11	Ubuntu Server	22.04 LTS
Flask / Werkzeug	3.0.2 / 3.0.1	Docker Engine	25
Gunicorn	22.0	Docker Compose	v2
psycpg2-binary	2.9.9	Nginx	1.18 (Ubuntu 22.04 package)
PostgreSQL	15–16	Certbot	1.21 (apt) or current (snap)
pytest	8.1	GitHub Actions	checkout v4, setup-python v5
bcrypt / requests	4.1.2 / 2.31.0	Mailpit	current image
prometheus-flask-exporter	0.23	openapi-spec-validator	0.7

Pinned versions in the listings are the ones the examples were written against, not a recommendation to stay on them. Updating dependencies, and checking them with a vulnerability scanner as Chapter 17 shows, is part of the job. The companion repository does exactly that:

when the scanner reports a known vulnerability in a version this book pins, the repository moves to a fixed release, so its versions may be newer than the ones in the listings.

How Software Gets Built

A map of the whole process, before any of its parts

If you have worked on software for a while, you have probably met its pieces one at a time: a ticket here, a deployment there, a meeting about “the API contract,” an alert in the middle of the night, a design review. Each one arrives on its own, with its own vocabulary, and nobody explains where it fits. The result is a pile of disconnected facts. You can follow the instructions for each one, but you cannot see the whole.

This chapter gives you the whole first. It is a map of how any piece of software goes from an idea to a working system that people use — and what happens after that. It uses almost no technical vocabulary, because you are not expected to know any yet. Everything in the rest of the book is a closer look at one region of this map, and every part and stage of the book opens by showing you where on the map you are.

The map has three layers:

1. **The loop** — the steps software goes through, in *time*.
2. **The pieces** — what a running application is made of, in *space*.
3. **The people** — who does which part, and what they hand to each other.

Learn these three, and every topic you meet later — in this book or at work — has a place to go.

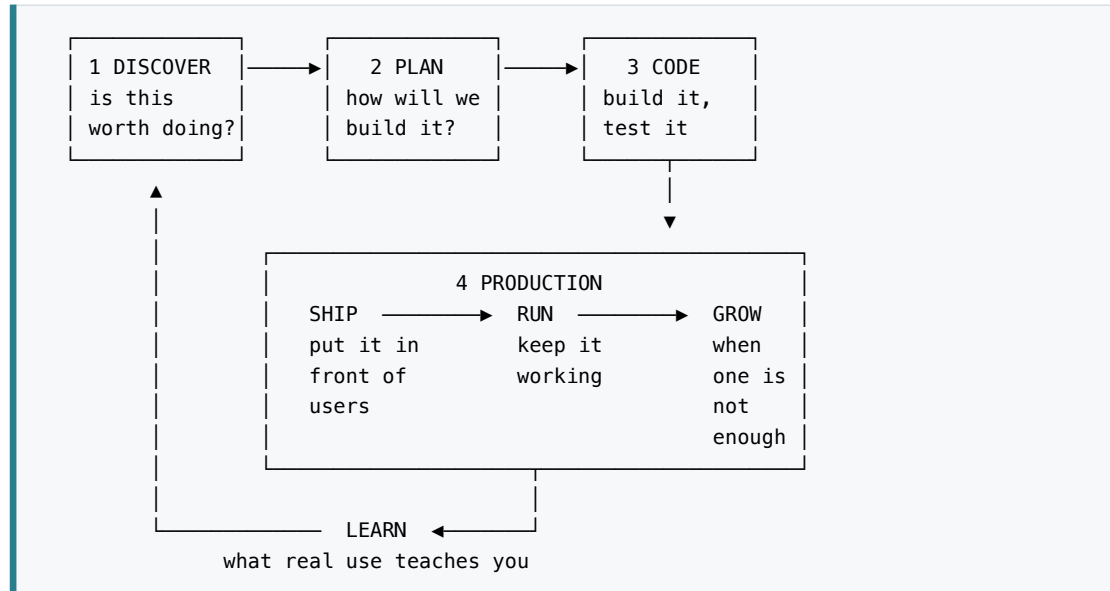
The One-Sentence Version

Software is built in a loop: someone decides what is worth building, plans how to build it, builds it, puts it in front of real users, and keeps it working — and what they learn from real use decides what to build next.

That is the whole process. Everything else is detail about one of those steps. A company with a thousand engineers and a student with a weekend project are going around the same loop. The difference is how many people work on each step, and how carefully they hand work

from one step to the next.

Layer 1: The Loop



The loop has four phases. The first three happen before anyone uses the software; the fourth is where it meets the real world. For each step below you will find the question it answers, what comes into it, what comes out, what goes wrong when it is skipped, and where this book covers it.

Step 1 — Discover: is this worth building?

Every piece of software starts as someone’s idea or request: “customers keep asking for this,” “our team wastes hours every week on that.” Discovery is the work of checking the idea *before* anyone builds it: who actually has this problem, how badly, what they do about it today, and whether a solution would really be used. It is mostly talking to people, watching them work, and testing cheap sketches — not writing code.

- **Comes in:** an idea, a request, a complaint, a number that looks wrong.
- **Goes out:** a clearly described problem, who has it, and what “solved” will look like.
- **When it is skipped:** the team builds the wrong thing, and builds it well. This is the most expensive mistake in software, because every later step multiplies it.

- **In this book:** Part I, [Product Discovery](#).

Step 2 — Plan: how will we build it?

Now the question changes from *what* to *how*. Planning turns the problem into things engineers can act on: what the system must do (the **requirements**), what it deliberately will not do (the **scope**), how it will be shaped (the **design**: which pieces, how they talk to each other, where the information lives), how it will be operated once it is live, and in what order the work will happen. The main output is written down — usually as a **design document** — because a plan that lives in one person’s head is re-argued every week and lost when that person leaves.

- **Comes in:** the problem, and what “solved” means.
- **Goes out:** a plan — requirements, a design, a list of pieces of work, and the important decisions with their reasons.
- **When it is skipped:** every engineer builds a slightly different idea, the pieces do not fit, and the big decisions get made by accident.
- **In this book:** Part II, [Project Planning](#).

Step 3 — Code: build it

Engineers turn the plan into working code. This is the step most people picture when they think of software engineering — but notice that it is only one of the six steps: discover, plan, code, ship, run and grow. Good teams write code in small pieces. Each piece is checked by automated **tests** (small programs that check the real program behaves as intended) and read by another engineer before it is accepted. Accepted code is kept in one shared place, the **code repository**, which holds every version of the code and is the single truth of what the software is.

- **Comes in:** the plan, broken into small pieces of work.
- **Goes out:** working, tested, reviewed code in the shared repository.
- **When it is done badly:** code that works today but that nobody can safely change tomorrow.
- **In this book:** Part III, [Writing Good Code](#), for the craft of writing code; and Stages 1 and 9 of Part IV for what the code actually does when it runs.

Step 4 — Production: make it real

Production is the name engineers give to the real world: the place where the software runs for real users, with real data, and where mistakes cost something. Code sitting in a repository helps nobody. Production has three parts.

Ship — put it in front of users. The code has to leave the engineer’s laptop and run on a computer that is always on and reachable from anywhere. That means packaging it so that it runs the same way everywhere, getting a machine to run it on, putting it there, keeping it running, and connecting it to the internet under a name people can find. Done well, shipping is automatic and boring: a change is approved, and minutes later it is live.

Run — keep it working. Once people depend on the software, it has to keep working when nobody is watching. And it will be tested by things you never saw on your laptop: machines fail, traffic suddenly doubles, someone tries to break in, a disk fills up. Running a system means being able to *see* what it is doing, being told when something goes wrong, recovering quickly, protecting it from attackers, and coping with more users.

Grow — when one is not enough. Some systems eventually outgrow a single program on a single machine: too many users, too much data, too many engineers changing one codebase. Then the work is split across many programs and many machines that must cooperate. This is powerful and expensive, and the most common mistake is doing it too early.

- **Comes in:** reviewed code.
- **Goes out:** a system real people use — and a steady stream of evidence about how it behaves.
- **When it is done badly:** “it worked on my machine”; failures nobody notices until users complain; the same failure happening again and again.
- **In this book:** Part IV — Stages 2 to 7 for Ship, Stages 8 and 10 for Run, Stage 11 for Grow.

The arrow back: Learn

Learning is what turns the line into a loop. Production produces evidence: how people really use the software, what they ignore, what breaks, what is slow, what they ask for next. That evidence goes back to Discover, and the loop starts again — for a new feature, a fix, or an improvement.

This is the most important thing to understand about real engineering work: **the loop never finishes.** The first trip around produces a first version. Every trip after that changes a system that already exists and already has users. That is where most engineers spend almost all of their working lives — not building new systems, but changing live ones without breaking them.

Three sizes of the same loop

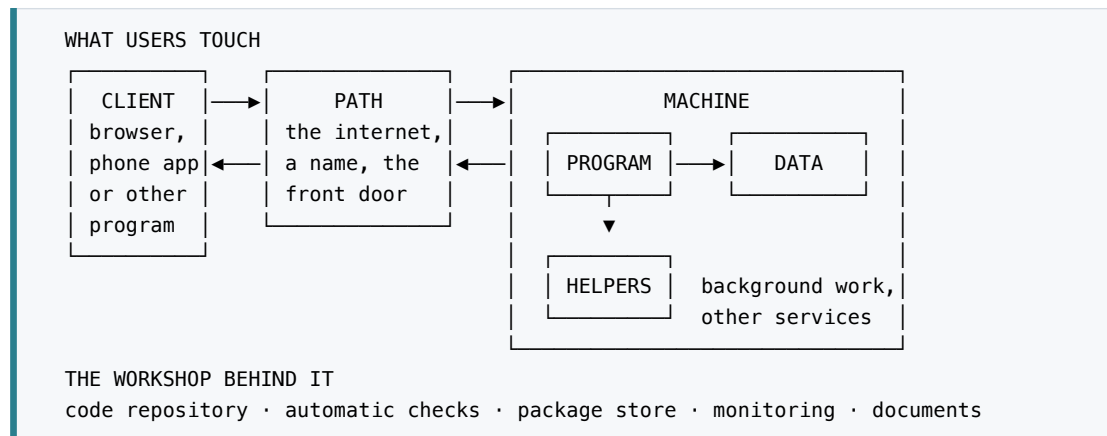
The loop is not only a months-long cycle. The same shape repeats at three sizes, one inside the other:

Size	How long one trip takes	Example
The big loop	weeks to months	a new product or a major feature: discover, plan, build, launch, learn
The change loop	hours to days	one small change: a piece of work, the code, a review, automatic checks, release, watch it
The inner loop	seconds to minutes	one engineer at a keyboard: change the code, run it, look at the result, change it again

Healthy teams make the two smaller loops fast and safe, so that they can go around the big loop often. A large part of this book is about exactly that: making the change loop fast and safe enough that shipping stops being frightening.

Layer 2: The Pieces

The loop is about *time*. The second layer is about *space*: what a working application is actually made of once it is running. When you use almost any app — send a message, book a ticket, save a note — the same handful of pieces is involved.



- **The client** is whatever the user holds: a web browser, a phone app, or another program. It shows things and sends requests. It keeps as little as possible, because it lives on a device you do not control.
- **The path** is everything between the client and your machine: the internet, the name users type (which has to be turned into a numeric address), and a **front door** on your side that receives every visitor first, checks them, and passes them in.

- **The machine** is a computer that is always on — usually rented from a cloud provider — with an operating system that runs your program and connects it to the network.
- **The program** is your code, running. It receives a request (“save this note”), checks it, applies the rules of your application, and sends back an answer. A program that does this for other programs, out of sight on a server, is called a **backend**, and the set of requests it accepts is its **API** (application programming interface).
- **The data** lives in a **database**, so that it survives when the program restarts or the machine is replaced. Almost every hard problem in real systems eventually turns out to be a problem about data.
- **The helpers** do work the program should not do itself. Slow jobs that should not make the user wait — sending an email, building a report — go to **background workers**. Work that another company does better — taking payments, delivering email — goes to **other services**, which your program calls over the internet.

The second group is **the workshop**. Users never see it, but it is what makes the loop work:

- the **code repository**, where all the code lives with its full history;
- the **automatic checks**, which test every change before it is accepted;
- the **package store**, which keeps the finished, runnable package of every version;
- **monitoring**, the numbers and records that show what the running system is doing, and the alarms that go off when something is wrong;
- the **documents**: the plan, the decisions and their reasons, and step-by-step instructions for handling failures.

One request, end to end

To see how the pieces fit together, follow one action. You tap *Save* on a note in an app:

1. **Client.** The app packages “save this note” as a request and sends it toward the name of the service.
2. **Path.** The name is turned into a numeric address; the request crosses the internet and reaches the front door, which checks it and passes it to the program.
3. **Program.** The program checks that the request is valid and that you are allowed to make it, applies the application’s rules, and asks the database to store the note.
4. **Data.** The database stores the note permanently and confirms.
5. **Back.** The program builds an answer — “saved; here is its number” — and it travels back along the same path to the app, which shows it to you.
6. **Workshop.** Meanwhile, monitoring records one more successful request, and how long it

took.

The whole trip usually takes a fraction of a second. Stages 8 and 9 of Part IV follow exactly this trip, in full detail.

Two coordinates for everything

Put the first two layers together and you get a grid. Any topic, tool, or task in software has two coordinates: **when** it happens (a step of the loop) and **where** it lives (a piece of the system, or the workshop).

What you hear	When (step)	Where (piece)
“Write the design document first.”	Plan	the workshop: documents
“The checks failed on my change.”	Code, in the change loop	the workshop: automatic checks
“We’re releasing the new version.”	Ship	from the package store to the machine
“Users can’t reach the site.”	Run	the path, the machine, or the program
“Saving is slow since yesterday.”	Run	the program or the data
“We need to split this into several services.”	Grow	the program and the machines

Notice how often the “where” for a problem in Run is *several* pieces. That is normal. The first job when something goes wrong is always to find *which* piece is failing — and you can only do that if you know what the pieces are.

Layer 3: The People

On a small project one person can do everything. That is how this book works: you do every step yourself, so that you see all of them. In a company, the steps and the pieces are divided between people with different jobs. Titles vary from company to company, but the division is remarkably consistent:

Role	Works mostly on (steps)	Works mostly on (pieces)	Hands to others
Product manager	Discover, Learn; priorities in Plan	—	the problem, why it matters, what comes first
Designer	Discover, Plan	the client: what users see and do	screens and flows
Tech lead or architect	Plan, and the whole loop	how all the pieces connect	the design and the key decisions
Frontend engineer	Code	the client, in the browser	the website
Mobile engineer	Code, Ship	the client, on phones	the phone apps
Backend engineer	Code, Ship, Run	the program, the data, the helpers	the API: what the program accepts and answers
QA or test engineer	Code, Ship	the automatic checks	confidence that it works
Platform or DevOps engineer	Ship	the workshop, the machines, the path	a safe, automatic way to ship
Site reliability engineer (SRE)	Run	monitoring, and the whole running system	reliability; leading the response when things break
Data engineer	Code, Learn	the data, and copies of it for analysis	reports and numbers people trust
Security engineer	every step	every piece	reviews, rules, protection
Engineering manager	the loop as a way of working	—	the people, and a team that works well

Two things are worth noticing.

First, the rows do not disappear when a team is small. In a startup, three engineers cover all of them; in a large company, each row is a whole team. Someone is always doing each job — often without the title.

Second, **people coordinate mostly through artifacts, not conversation**. Each step hands the next one something concrete: a written problem, a design, a reviewed change, a versioned package, a dashboard, a **postmortem** (a written account of a failure: what happened, why, and what will change). When those things are clear, dozens of people can work in parallel without constantly talking to each other. When they are vague, everything turns into a meeting.

From → to	What is handed over
Discover → Plan	the problem statement: who has what problem, and what “solved” means
Plan → Code	the design, broken into small pieces of work, and the agreed API
Engineer → engineer	a proposed change, reviewed before it is accepted
Code → Ship	reviewed code that has passed every check, turned into a versioned package
Ship → Run	a running system, plus written instructions for when it fails
Run → Learn → Discover	measurements, failure reports, user feedback
Frontend ↔ backend	the API contract: exactly which requests the program accepts and what it answers

A great deal of engineering practice exists purely to make one of these hand-offs safe. When you meet a practice and wonder why anyone bothers, ask which hand-off it protects. The closing chapter of this book, [The Same Process in a Real Team](#), returns to this with a whole team.

The Same Map, for Any Application

This book builds one kind of software: a backend service — a program that other programs talk to over the internet. But the map is not specific to it. The loop is the same for every kind of software; what changes is which pieces exist, and what Ship and Run look like.

Kind of software	What changes on the map
A website	The client is code your server sends to each browser, so you can update every user at once.
A phone app	The client lives on users' phones and ships through app stores. Old versions stay in use for months, so the backend must keep answering them.
A data pipeline	Nobody sends it requests; it runs on a schedule. "Working" means the data it produces is fresh and correct.
Installed software, or a library	Ship means publishing a release. You do not run it — your users do — so Run becomes bug reports and support.
A large system of many services	Grow has happened: many programs, and many teams, each going around its own loop.

Every one of them has a Discover, a Plan, Code, a way to Ship, something to keep running, and a way to Learn. The closing chapter lays them side by side.

How to Use the Map

Whenever you meet something new — in this book, at work, in a meeting, in a job description — ask four questions:

1. **When?** Which step of the loop does it belong to?
2. **Where?** Which piece of the system, or which part of the workshop?
3. **Who?** Which role owns it, and what do they hand to whom?
4. **Why?** What goes wrong without it?

If you can answer all four, the thing has a place in your head and is no longer an isolated fact. If you cannot, the question you could not answer is exactly what to find out next.

How This Book Walks the Map

The book goes around the loop once, from start to finish, for one small application — a service that stores notes, called the **Notes API** — and you do every step yourself. That is deliberate. Specialists usually see only their own region of the map, and the fastest way to see all of it is to walk all of it once.

Part or stage	Step	Piece	The question it answers
Part I – Product Discovery	Discover	–	Should we build this, and what exactly?
Part II – Project Planning	Plan	documents	How will we build it and run it?
Part III – Writing Good Code	Code	the program	How do we write code others can safely change?
Stage 1 – Creating a Running Program	Code	the program, on your laptop	What is a program, really, when it runs?
Stage 2 – Breaking the Local Illusion	Code → Ship	the gap between laptop and server	Why is “it works on my machine” not enough?
Stage 3 – Making the Program Portable	Ship	the program	How is it made to run anywhere?
Stage 4 – Build and Packaging	Ship	the workshop	How is it checked and packaged?
Stage 5 – Infrastructure and Environment	Ship	the machine	What will it run on?
Stage 6 – Deployment	Ship	the program, on the machine	How does it get there, and stay running?
Stage 7 – Making It Reachable	Ship	the path	How do users reach it?
Stage 8 – Request Flow	Run	the path, end to end	What happens to one request on its way in?
Stage 9 – Inside the Running Program	Code, seen again	the program and its data	What does the program do with the request?
Stage 10 – Production Reality	Run	the whole system	How does it keep working in the real world?
Stage 11 – Beyond a Single Service	Grow	many programs and machines	What changes when one is not enough?
Chapter 44 – the project	the whole loop, five times	the whole system	Can you do all of this yourself?
The Same Process in a Real Team	the whole loop	the whole system	How does a team do all of this, for any application?

Most of the book’s pages are spent in the backend engineer’s region of the map — the

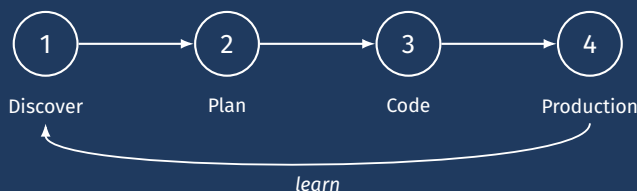
program, its data, and everything between it and its users — because that is where Code, Ship and Run meet. The book does not teach frontend or mobile code, or the day-to-day mechanics of running a team; they have their places on the map, and the closing chapter shows where, but they belong to other books.

Every part of the book opens with the loop, marked with where you are, and every stage of Part IV opens with its place on the loop and the piece it works on. When you feel lost in detail, come back to this chapter. The detail always belongs somewhere on the map.

BEFORE PART I

How Web Applications Work

What a web application is, before we build one: the two sides, what runs on the screen, what runs where you cannot see, and one tap followed from start to finish.



Two Sides of Every App

This chapter gives you the one picture the rest of the book hangs on. Every app you use has two sides: one in your hand, and one far away. The two sides talk by passing messages. By the end of the chapter you will know what each side is called and how their conversation works.

The problem. I tap a button, and something happens far away.

The question. What is on the other end of my tap, and how do the two ends talk?

One App, Two Places

You open a food delivery app, pick a pizza, and tap *Order*. A few seconds later the screen says “Order placed.” In between, the restaurant received your order, a driver was found, and your card was charged. None of that happened on your phone. Your phone doesn’t know which drivers are free, and it can’t charge a card by itself.

So the app you “use” is really two programs. One runs on your phone. The other runs on the delivery company’s computers, often hundreds of kilometres away. Together they make one **web application**: a program split between your device and a computer on the internet, with the two halves talking over the network. Email, maps, banking, social media, and online shopping all work this way.

The Restaurant Picture

A restaurant has two areas. The *front of house* is the dining room: tables, menus, waiters. Everything there is designed for you. The *back of house* is the kitchen. You never go there, but that is where the food, the recipes, and the cooks are.

Apps are built the same way, and developers even use the same words:

- The **front end** is the part you see and touch: screens, buttons, text boxes. It runs on your device.

- The **back end** is the part you never see: the program that keeps the data, applies the rules, and does the real work. It runs on the company’s computers.

A waiter carries orders to the kitchen and plates back to the table. In an app, messages sent over the internet do the carrying.

Note: You will also see “frontend”, “client side”, “backend”, and “server side”. They mean the same two things.

Client and Server

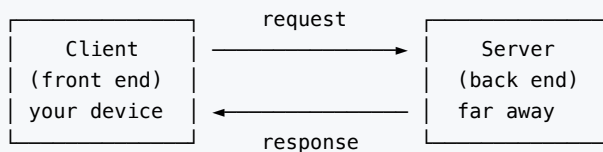
Two more words describe the *roles* in the conversation:

- The **client** is the program that asks. Your phone app is a client. So is a web browser.
- The **server** is the program that answers. It waits for questions, day and night, from many clients at once. People also say “server” for the computer it runs on.

One kitchen feeds every table, and one back end serves millions of phones. That is why shared data lives on the server: it is the one place every client can reach. Your friend sees your new photo because the photo is stored on the server, not on your phone.

The Request and Response Loop

Client and server talk in a strict pattern. The client sends a **request**: a message saying what it wants. The server sends back a **response**: a message with the answer. One request plus its response is a **round trip**.



Three rules hold for almost every round trip:

1. *The client speaks first.* The server answers questions; it doesn’t start conversations.
2. *Every request gets one response*, even if the answer is “no” or “something broke”.
3. *Each round trip stands alone.* The server treats every request as a fresh message. (Chapter 3 shows how it still remembers who you are.)

Here is the pizza order as a round trip:

- Request: “Place an order: one margherita, to 12 Park Road, paid with the saved card.”
- Response: “Done. Order number 5531. The driver arrives in 25 minutes.”

Then the app shows the result on your screen. That is the whole shape of a web application: *tap, request, work, response, show*. The rest of this book fills in each step.

Why Split an App at All?

Why not put everything on the phone? Four reasons:

- *Sharing*. Data that many people need must live in one place everyone can reach.
- *Trust*. You control what runs on your own phone. A rule like “charge the card only after the restaurant accepts” must run where customers can’t tamper with it.
- *Power*. A server can search millions of restaurants in a blink. A phone has a small battery.
- *Updates*. Fix a bug on the server once, and every user gets the fix at the same moment.

The price of the split: the two sides must be able to reach each other. No network, no app. You have seen this price as “No internet connection.”

Try It

You can watch round trips happen. Your browser has **developer tools**: a built-in panel that shows what a web page is doing behind the scenes.

1. Open any news website in Chrome, Edge, or Firefox on a computer.
2. Open the developer tools: press F12 (Windows, Linux) or Cmd+Option+I (Mac), or right-click the page and choose *Inspect*.
3. Click the *Network* tab, then reload the page.

Expected result: The panel fills with a long list, often over a hundred lines. Each line is one round trip. The *Status* column shows numbers such as 200, which means “OK”, and the bar at the bottom counts the requests. One page, loaded by one click, needed all of them.

Summary, Key Terms, and Review Questions

Summary

- A web application is split in two: a front end on your device and a back end on the company's computers.
- The client asks and the server answers. One server answers many clients.
- Client and server talk in round trips: one request, one response.
- Apps are split for sharing, trust, power, and easy updates. The cost is that both sides need the network.

Key Terms

Term	Meaning
Web application	A program split between your device and a server
Front end	The part you see and touch
Back end	The hidden part that keeps data and does the work
Client	The program that asks
Server	The program that answers
Request	The message the client sends
Response	The message the server sends back
Round trip	One request plus its response
Developer tools	The browser panel that shows what a page is doing

Review Questions

1. Using the restaurant picture, explain the front end and the back end.
2. Why does your friend's phone show a photo you uploaded from your phone?
3. Walk through “tap, request, work, response, show” for sending a chat message.
4. Give two reasons a rule like “charge the card” runs on the server, not the phone.

You understand this when you can pick any app on your phone and say which of its jobs happen on your device and which happen far away.

The Front End: What Runs on Your Screen

This chapter looks at the side of the app you can see: who draws the screen, the three languages every web page is made of, what a front-end framework adds, and why a phone app is a front end too.

The problem. The same app looks right on my laptop and my phone, but who draws it?

The question. How does an app become the picture on my screen, and how does that picture fit every size of screen?

The Server Sends Instructions, Not a Picture

When you open a website, the server does not send a finished picture of the page. It sends *instructions*, and your device draws the page itself.

Think of flat-pack furniture. The shop doesn't deliver a built bookcase. It sends the parts and an instruction sheet, and you assemble it at home. A good sheet even says: "If your wall is narrow, stack the shelves in one column."

The program that does the assembling is the **browser**: Chrome, Safari, Firefox, or Edge. It downloads the instructions, follows them, and draws the result. That is why the same app looks right on a laptop and a phone: each device builds the page to fit its own screen.

Three Languages, Three Jobs

The instructions come in three languages:

Language	Its job	In the furniture picture
HTML	What is on the page: text, buttons, images	The list of parts
CSS	How it looks: colours, fonts, layout	The paint and arrangement
JavaScript	What happens when you touch it	The drawers that slide

Here is a tiny but complete page that uses all three:

```

<!DOCTYPE html>
<html>
<head>
  <style>
    h1 { font-family: sans-serif; }
    button { background: royalblue; color: white; }
  </style>
</head>
<body>
  <h1>My Notes</h1>
  <button id="save">Save</button>
  <p id="status"></p>
  <script>
    const button = document.getElementById("save");
    button.onclick = function () {
      document.getElementById("status").textContent = "Saved!";
    };
  </script>
</body>
</html>

```

The CSS inside `<style>` makes the heading plain and the button blue. The HTML inside `<body>` lists a heading, a button, and an empty paragraph. The JavaScript inside `<script>` says: when someone clicks the button, put *Saved!* in the empty paragraph. Open this file in a browser and click *Save*, and *Saved!* appears under the button.

Note: This button is a fake. It changed the page, but nothing left your computer, so nothing was saved. A real Save button must send a request to a back end. Chapter 4 shows this same button doing exactly that.

One Page, Every Screen

A phone screen is narrow; a laptop screen is wide. Designers don't build two websites. They write CSS rules that change the layout to fit the space: "on a wide screen, show the menu on the left; on a narrow one, hide it behind a button." This is **responsive design**: the "stack the shelves if the wall is narrow" line of the instruction sheet.

Front-End Frameworks

The example page has one button. An online shop has thousands, and changing one number can mean five places on the screen must change with it. Doing that by hand in JavaScript is slow and error-prone.

A **front-end framework** is ready-made code that does this bookkeeping. React, Vue, Angular, and Svelte are well-known ones. The developer describes what the screen should look like for the current data ("the cart shows these three items"), and the framework updates the page whenever the data changes.

Frameworks build screens from **components**: small, reusable pieces such as a product card or a like button. Like building bricks, one component can appear hundreds of times on a page.

Note: A framework helps the developer; it is not a new language for the browser. A React app still reaches your browser as HTML, CSS, and JavaScript, the only three languages a browser draws.

Mobile Apps Are Front Ends Too

Apps you install from an app store are usually **native apps**: programs written for one kind of phone, in languages like Swift (iPhone) or Kotlin (Android). They draw their own screens without a browser.

The tools differ, but the role is the same. A native app shows screens, reacts to taps, and asks a back end for anything it doesn't have. Often one back end serves the website, the iPhone app, and the Android app together.

What the Front End Can't Do

Everything in the front end travels to the user's device, where the user can read it and change it. So the front end can't keep secrets, and it can't make final decisions. It can say "this coupon looks valid", but only the back end can accept it.

Warning: Never put a password, a secret key, or a price the customer must not change in front-end code. The back end must check everything the front end sends.

Try It

See the instructions behind a real page, and change them.

1. Open any website on a computer. Right-click a heading and choose *Inspect*. The *Elements* tab shows the page's HTML, with your heading highlighted.
2. Double-click the heading's text in that panel, type something new, and press Enter. Then reload the page.
3. Click the phone-and-tablet icon at the top of the developer tools (the device toolbar) and pick a phone.

Expected result: In step 2 the heading changes to your text, then goes back when you reload. You changed only your own copy of the instructions; the server never knew. In step 3 the page rearranges itself for a narrow screen: responsive design at work.

Summary, Key Terms, and Review Questions

Summary

- The server sends instructions; the browser follows them and draws the page.
- HTML says what is on the page, CSS says how it looks, and JavaScript says what it does.
- Responsive design lets one page fit any screen.
- Front-end frameworks keep large screens up to date, built from reusable components.
- Native apps are front ends too, often sharing one back end with the website.
- The front end can't keep secrets or make final decisions.

Key Terms

Term	Meaning
Browser	The program that downloads and draws web pages
HTML	The language for what is on a page
CSS	The language for how a page looks
JavaScript	The language for what a page does
Responsive design	One layout that adapts to any screen size
Front-end framework	Ready-made code that keeps a screen in step with its data
Component	A small, reusable piece of a screen
Native app	An app written for one kind of phone

Review Questions

1. Why can the same website look right on a laptop and a phone?
2. For a *Like* button, what does the HTML do, what does the CSS do, and what does the JavaScript do?
3. What problem does a front-end framework solve?
4. In what way is an iPhone app the same thing as a website?
5. Why can't a shop let the front end decide the final price?

You understand this when you can look at any screen in any app and say what the front end drew by itself and what it must have asked the back end for.

The Back End: What Runs Where You Can't See

This chapter walks into the kitchen: what the back end does with each request, where your data ends up, how the server remembers you, and how the two sides agree on what they may ask each other.

The problem. Where does my data go when I press Save?

The question. What happens on the far side of the app, and how does it keep my data and remember me?

A Computer That Never Sleeps

A back end runs on a server: usually a computer with no screen in a *data centre*, a warehouse full of such computers. Most companies rent these machines from a cloud company.

The back-end program *listens*: it waits for a request, answers it, and waits again. For each request it does three jobs, like a kitchen handling an order slip:

1. *Routing*: send the order to the right station. Which piece of code handles “save a note”?
2. *Business logic*: follow the recipe and the house rules. Is this allowed? Does it make sense?
3. *Storing and fetching*: get ingredients from the store room. Read or write the database.

Then it builds a response and sends it back.

Back-End Frameworks

Every back end needs the same plumbing: receive requests, find the right code, send responses. A **back-end framework** provides it, so developers write only what is special to their app. Here is a complete, tiny back end written with FastAPI, a Python framework:

```

from fastapi import FastAPI

app = FastAPI()
notes = [] # the app's memory, for now

@app.post("/notes")
def save_note(note: dict):
    notes.append(note["text"])
    return {"saved": True, "count": len(notes)}

@app.get("/notes")
def list_notes():
    return {"notes": notes}

```

The two functions are ordinary Python. The line above each, starting with @, is a **route**: it tells the framework, “when a request to save a note (POST /notes) arrives, run this function.” A function that answers a request is a **handler**. Whatever it returns becomes the response.

You don't need to run this; it is here to show how small a handler can be. Saved as `app.py` and started with `uvicorn app:app` (a program that runs FastAPI apps), it listens on your own computer, called `localhost`. `curl`, a terminal program that sends one request and prints the response, can then play the client:

```

curl -X POST http://localhost:8000/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "Buy milk"}'

```

```

{"saved":true,"count":1}

```

Business Logic: The House Rules

The **business logic** is the set of rules that makes an app *this* app:

- A cinema can't sell the same seat twice.
- A bank can't let you send more money than you have.
- Only the person who wrote a post can edit it.
- The shop, not your browser, adds up the price of your cart.

These rules live on the back end because it is the one place the company fully controls.

The Database: Where Your Data Lives

Our tiny back end keeps notes in a Python list. Stop the server, start it again, and ask for the list:

```
curl http://localhost:8000/notes
```

```
{"notes": []}
```

The milk is gone: a list in memory vanishes when the program stops. Real apps use a **database**: a program built to keep data safely on disk, find any piece of it quickly, and let many users read and write at once. Picture a huge filing room with a fast, careful clerk. PostgreSQL, MySQL, and MongoDB are well-known ones.

So here is the answer to this chapter's problem. When you press Save, your data travels to the back end, which checks it against the rules and hands it to the database. Your phone may keep a copy for speed, but the real copy lives there.

How the Server Remembers You

The server greets every request like a stranger. So how does it know this request comes from *you*, who logged in ten minutes ago?

Think of a cloakroom. You hand over your coat and get a numbered ticket. Later you show the ticket, not your face, and get your coat back.

When you log in, the back end checks your password and writes down a **session**: a record saying “ticket 7f3a9c belongs to Priya, logged in at 9:14.” It gives the ticket to your browser as a **cookie**: a small piece of text the browser stores and sends back with every later request to that site. Mobile apps use a similar ticket, usually called a token.

Warning: Whoever holds your ticket *is* you, as far as the server knows. Never paste cookie values anywhere, and log out on shared computers: that tears up the ticket.

APIs: The Contract Between the Two Sides

The front end and the back end are often built by different people. They need an agreement: which requests the back end accepts, what each must contain, and what comes back. That agreement is the **API** (Application Programming Interface). Each thing it offers, such as “save a note” at `POST /notes`, is an **endpoint**. The data usually travels as **JSON**: plain text that looks

almost exactly like a Python dictionary, such as `{"text": "Buy milk"}`.

The API is the restaurant's menu. As long as the menu stays the same, the kitchen can buy new stoves and the dining room can be repainted without breaking each other. And a website, an iPhone app, and an Android app can share one back end: they all order from the same menu.

Try It

Look at the cloakroom tickets your browser holds for a site.

1. Open a private (incognito) window and log in to a site you use, such as web email.
2. Open the developer tools. In Chrome or Edge, go to *Application*, then *Cookies*; in Firefox, *Storage*, then *Cookies*. Click the site's name.
3. Right-click the site's name, choose *Clear* (Chrome, Edge) or *Delete All* (Firefox), and reload the page.

Expected result: In step 2 you see several cookies, each a name and a value; the login one usually has a long, random value. In step 3 you are suddenly logged out: you threw away your tickets, so the server no longer knows you.

Summary, Key Terms, and Review Questions

Summary

- A back end listens for requests on a server. For each one it routes, applies business logic, and reads or writes the database.
- Back-end frameworks provide the plumbing; developers write handlers.
- Data in memory vanishes; a database keeps it safe and shared.
- A session and a cookie let the server know who you are on every request.
- The API is the contract between the two sides, usually spoken in JSON.

Key Terms

Term	Meaning
Back-end framework	Ready-made plumbing for answering requests
Route	A rule linking a kind of request to its handler
Handler	A function that answers one kind of request
Business logic	The app's own rules
Database	A program that stores data safely and shares it
Session	The server's record of who is logged in
Cookie	A small piece of text the browser keeps and sends back
API	The agreement on what the front end may ask the back end
Endpoint	One thing an API offers, such as <code>POST /notes</code>
JSON	A plain-text data format, like a Python dictionary

Review Questions

1. What three jobs does a back end do for every request?
2. In the tiny FastAPI app, what is the route and what is the handler for saving a note?
3. Why did the note disappear when the server restarted, and what fixes that?
4. Explain sessions and cookies using the cloakroom picture.
5. How can a company rebuild its back end without breaking the apps that use it?

You understand this when you can answer “where does my data go when I press Save?” for any app, naming the handler, the rules it checks, and the database, and explain how the server knows it was you.

Following One Tap from Start to Finish

This chapter puts the two sides together. You will follow one tap on a Save button to the server and back, then meet the common combinations of tools real apps are built from.

The problem. I know the two sides now, but not how one tap travels between them.

The question. What happens, in order, between tapping Save and seeing “Saved!”?

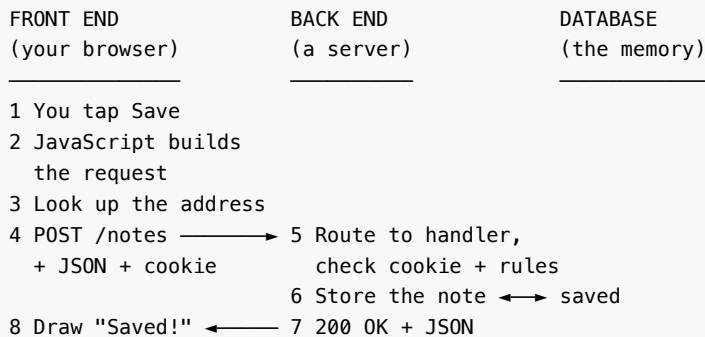
The Button, Now for Real

In Chapter 2, the Save button only changed the page. Here is the same button’s JavaScript, now talking to the back end from Chapter 3:

```
button.onclick = async function () {
  const response = await fetch("/notes", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({text: "Buy milk"}),
  });
  const data = await response.json();
  document.getElementById("status").textContent =
    "Saved! Notes so far: " + data.count;
};
```

fetch sends a request; await means “wait here for the response.” The last lines put the result on the page: *Saved! Notes so far: 1*. Everything in this chapter happens inside that one await.

The Whole Trip



1. *The tap.* The browser notices the click and runs the button’s JavaScript.
2. *The request.* JavaScript builds a request: an action (POST, meaning “create”), an address, and a JSON body. The browser attaches the site’s cookie by itself.
3. *The address.* Every request goes to a URL, such as `https://notes.example.com/notes`. The middle part, `notes.example.com`, is a **domain name**: a human-friendly name for the server. Computers really find each other by numbers called **IP addresses**, so the browser asks **DNS**, the internet’s phone book, to turn the name into a number. The `https` at the front means **HTTPS**: the request travels sealed, so nobody on the way can read it.
4. *The journey.* The sealed request hops across the internet to the server.
5. *The back end.* The framework matches `POST /notes` to its route and runs the handler, which looks up the cookie to learn who you are and applies the rules.
6. *The database.* The handler asks the database to store the note and waits for “OK”.
7. *The response.* The back end sends back a **status code**, a number that says how it went, plus JSON. Here is that response exactly as it arrives, shown by `curl -i` (the `-i` means “include the headers”):

```
curl -i -X POST http://localhost:8000/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "Buy milk"}'
```

```

HTTP/1.1 200 OK
date: Sat, 26 Sep 2026 14:31:05 GMT
server: uvicorn
content-length: 24
content-type: application/json

{"saved":true,"count":1}

```

The first line is the status. Then come headers, notes about the message. After the blank line comes the body: the handler’s JSON.

8. *The drawing.* JavaScript reads the JSON and changes one paragraph. The browser redraws only that part; the page doesn’t reload.

The whole trip usually takes a tenth to a third of a second. You feel it as “instant”.

Tip: Status codes come in families. Numbers starting with 2 mean success, with 4 mean “your request was wrong” (404 is “not found”), and with 5 mean “the server broke”. The first digit tells you which side to blame.

Common Stacks

Every app picks a front end, a back end, and a database. That combination is its **stack**. The pieces mix freely, because they only talk through requests and responses:

Stack	Front end	Back end	Database
Python	React or plain HTML	FastAPI or Django	PostgreSQL
JavaScript everywhere	React	Node.js with Express	MongoDB
Classic “LAMP”	HTML made by PHP	PHP on Linux	MySQL
Ruby on Rails	Rails pages	Ruby on Rails	PostgreSQL
Mobile	Swift or Kotlin app	Any of the above	Any

A developer who works on both sides is called a **full-stack developer**. The tools change every few years; the trip you just followed does not.

Try It

Catch a real request in the act.

1. Open a site with a search box that suggests results as you type, such as an online shop.
2. Open the developer tools, click *Network*, and choose the *Fetch/XHR* filter. This hides images and files and shows only requests made by JavaScript. Clear the list with the clear button.
3. Type one letter into the search box. Click the new request that appears.

Expected result: A request appears as you type, with status **200**. *Headers* shows its URL and method, *Response* (or *Preview*) shows JSON holding the suggestions, and *Timing* shows how long the server took. Match each piece to the eight steps above.

Summary, Key Terms, and Review Questions

Summary

- A tap runs JavaScript, which builds a request and sends it to a URL.
- DNS turns the domain name into an IP address; HTTPS seals the request for the journey.
- The back end routes the request, checks who you are and the rules, and uses the database.
- The response carries a status code and JSON; JavaScript redraws part of the page.
- A stack is one app's choice of front end, back end, and database.

Key Terms

Term	Meaning
URL	The full address a request is sent to
Domain name	A human-friendly name for a server
IP address	The number computers use to find each other
DNS	The internet's phone book, from names to numbers
HTTPS	The sealed, encrypted way requests travel
Status code	A number saying how a request went
Stack	An app's front end, back end, and database
Full-stack developer	Someone who works on both sides

Review Questions

1. List the eight steps between tapping Save and seeing "Saved!".
2. What does DNS do, and why is it needed?
3. What are the three parts of the response shown by `curl -i`?
4. What would a status code of 500 tell you, and what would 404 tell you?
5. Why can a React front end work with a Django back end?

You understand this when you can draw the whole trip from memory for any button in any app, and point to the step where each piece of this book fits.

Security, Speed, Failure, and Cost

Every web application, from a to-do list to a bank, faces the same four questions: what can go wrong, what makes it slow, how it breaks, and what it costs. This chapter answers each one using the picture you now have of the two sides and the trip between them.

The problem. My app works on my screen. But could someone break in, could it feel slow, could it fall over, and who pays for it?

The question. What can go wrong with a web application, and what keeps each kind of trouble small?

Security: What Can Go Wrong

The front end lives on the user's device, so the user controls it. Anyone can skip your page and send requests with a tool like `curl`. So the back end must treat every request as a stranger at the door and check three things: *who are you* (the session), *are you allowed* (the rules), and *does this make sense* (a negative price? a 10 MB name?).

The other dangers follow the trip:

- *On the device:* secrets in front-end code are not hidden. Keep them on the back end.
- *On the way:* a stolen cookie is a stolen login. HTTPS seals every request, and sessions that expire limit how long a stolen ticket works.
- *In the database:* a leak exposes everything stored. Keep only what you need, and never store passwords as plain text. Store them **hashed**: scrambled one way, so that even a leak doesn't reveal them.

What limits the damage: give each part only the access it needs, so one mistake can't open every door.

Speed: What Makes an App Feel Slow

Time goes to three places on the trip:

- *Distance*. Every round trip takes time; twenty trips take twenty times as long.
- *Size*. Big images and heavy JavaScript download slowly, especially on a phone.
- *Work*. A back end that searches a huge database carelessly can take seconds.

The fixes match. A **cache** keeps a copy of an answer close by, so the next request travels less far, or not at all. A **CDN** (content delivery network) keeps copies of files on servers near the users. Shrink images and code. And *feel* faster: show the new note at once, while the request is still travelling.

Failure: What Breaks, and How to Tell Where

When something goes wrong, first find which step of the trip failed. The developer tools tell you most of it:

What you see	Where it likely failed	Where to look
A button does nothing	Front end: a JavaScript error	The <i>Console</i> tab
"Can't reach this site"	The network or DNS	Try another site
An endless spinner	Back end slow or down	<i>Network</i> tab
A status starting with 4	Your request (401, 404)	<i>Network</i> tab
A status starting with 5	The back end broke	The server's logs

A client gives up after a **timeout**, a set time such as thirty seconds. On the back end, developers read **logs**: lines the server writes about every request, such as "POST /notes HTTP/1.1" 200 OK from Chapter 3's back end.

Cost: What It Costs to Run

The two sides cost very different amounts:

- *The front end* is cheap to serve: its files are sent once and cached, and the drawing happens on the user's device.
- *The back end* runs day and night, and that is where the bills are: servers, the database (often the biggest item), data sent to users, and paid services such as email.

Costs grow with users, though free tiers let a small app run almost free. The largest cost is usually people, who build, update, and protect both sides.

Where to Go from Here

You now know what happens between tapping a button and seeing the result. Here is what to learn next, and what to leave alone for now.

What to Learn Next

Each step of the trip has its own book in this series:

- The journey between the two sides: *Networking from Zero*.
- The contract and the back end: *APIs to FastAPI*.
- Where the data lives: *Databases from Zero*.
- The tools every developer uses: *Linux and Git from Zero*.
- Getting a back end onto real servers: *Infrastructure Engineering from Zero*.
- Putting it all together: *The Complete Software Engineering Guide*.

For the front end, learn plain HTML, CSS, and JavaScript first, and pick a framework only after that. MDN Web Docs is a free, reliable reference for all three.

What to Skip for Now

- *Choosing the “best” framework*. They all do the jobs in Chapters 2 and 3. Learn one well.
- *Scaling to millions of users*. Microservices and Kubernetes solve problems you don’t have yet.
- *Other ways to talk*. WebSockets, GraphQL, and server-side rendering are variations on the same trip. Meet them when a project needs them.

Whatever you learn next, keep the trip in mind: tap, request, work, response, show. Every new tool is a better way to do one of those steps.

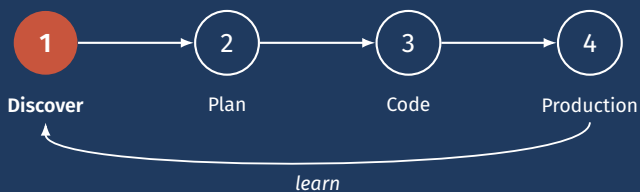


PART I

Product Discovery

Deciding what is worth building, before you build it.

YOU ARE HERE



Product Discovery

Deciding what is worth building — before you build it

The most expensive mistake in software is not bad code. It is building the wrong thing *well* — pouring months of careful engineering into a product, feature, or system that nobody actually needs. Slow or buggy code can be fixed; a beautifully built solution to a problem no one has is simply waste.

Product discovery is the work that happens *before* planning and code: figuring out what is worth building, for whom, and why — and validating that cheaply, before committing to expensive engineering. It is the most commonly skipped phase of building software, and the one that decides whether everything downstream even matters. You can plan, code, and operate flawlessly and still fail completely if discovery was skipped.

This part is about answering one question well: **should we build this at all — and if so, what exactly?**

Why Discovery Exists: The Cost of Building the Wrong Thing

Engineering effort is expensive and largely irreversible: once a team has spent three months building something, the sunk cost makes it hard to admit it was the wrong thing. Discovery exists because the cost of *finding out you were wrong* should be paid in days of conversation and cheap experiments, not months of code.

Two ideas frame the whole discipline:

- **Outputs are not outcomes.** An *output* is something you ship (a feature, a screen, an API). An *outcome* is a change in the world (a user succeeds at something, a metric moves, a problem goes away). Shipping is easy; creating an outcome is the actual job. A team can ship constantly and create no value at all.
- **The build trap.** Most teams measure themselves by how much they ship, and slide into being a “feature factory” — busy, productive-looking, and disconnected from whether any of

it matters. Discovery is the discipline that keeps effort tied to outcomes instead of motion. The mindset shift discovery asks for: fall in love with the **problem**, not your solution. Your first idea is a guess. Discovery is how you find out — cheaply — whether it’s a good one.

The Four Risks Every Idea Must Survive

A useful way to think about discovery (from Marty Cagan) is that every idea carries four risks, and discovery is the work of *retiring* them before you build:

- **Value risk** — will anyone actually want this? Will they use it, switch to it, pay for it? This is the biggest risk and the most often ignored, because it’s the easiest to assume away.
- **Usability risk** — can users figure out how to use it? A valuable thing nobody can operate fails just as surely as one nobody wants.
- **Feasibility risk** — can *we* actually build it, with our time, technology, data, and skills? (This is the one engineers naturally focus on — and often the *least* risky of the four.)
- **Business viability risk** — does it work for *our* business? Legal, financial, brand, sales, support, and ethical constraints can all kill an otherwise good idea.

The trap is to spend all your energy on feasibility (“can we build it?”) while assuming value (“they’ll want it”). Discovery deliberately attacks the *value* risk first, because it’s both the largest and the cheapest to test before any code exists.

Understanding the Problem and the User

You cannot validate a solution to a problem you don’t understand. Discovery starts with the people you’re building for.

Talk to users — but listen, don’t pitch

The single highest-leverage discovery activity is talking to real users — and the single most common way to do it badly is to pitch your idea and collect compliments. People are polite; they will tell you your idea is “interesting,” and you will learn nothing.

The discipline (captured well in Rob Fitzpatrick’s *The Mom Test*): **ask about their life and their past behavior, not your idea**. Good questions are concrete and backward-looking — “Walk me through the last time you ran into this” — because what people *actually did* is data, while what they *say they would do* is a guess. Avoid leading questions, avoid hypotheticals, and treat compliments as a warning that you’ve started pitching.

Find the problem beneath the solution

Users almost always arrive with a *solution*, not a problem: “I need a faster horse,” “just add an export button.” Your job is to dig past the requested solution to the underlying need — the “five whys” is a blunt but effective tool here. The export button might really be “I need to share this with my boss, who lives in spreadsheets,” which a dozen better solutions could serve. Solving the stated solution literally is how you build features no one uses.

Jobs To Be Done

A sharp lens for “what is the user really trying to do” is **Jobs To Be Done**: people “hire” a product to make progress in a particular situation. They don’t buy a drill because they want a drill; they hire it to get a hole — and really, to hang a picture. Focusing on the *job* (the progress someone is trying to make) rather than the demographic or the feature keeps you anchored to the outcome the user cares about.

Validating That It’s Worth Solving

Understanding a problem isn’t enough — plenty of real problems aren’t worth solving. A problem is worth building for when it is **real, frequent, painful, and something people will change behavior (or pay) to fix**. A rare, mild annoyance is not a business.

Two practical tools:

- **The Riskiest Assumption Test.** List the assumptions your idea depends on, find the one that — if false — kills the whole thing, and test *that* first, as cheaply as possible. Usually it’s a value assumption (“people will pay for this”), not a feasibility one. Spending your first effort anywhere else is motion, not progress.
- **Demand before build.** You can measure demand before writing the product: a landing page that describes the offer and counts sign-ups, a “fake door” (a button for a feature that doesn’t exist yet, measuring who clicks), or a pre-sell. Real behavior — a click, an email, a pre-order — is far stronger evidence than any survey.

Shaping and Testing the Solution Cheaply

Once the problem is validated, discovery shifts to the solution — and the goal is to learn the most while building the least.

Define success as an outcome, not an output

Before designing anything, state what success *is* — and state it as an outcome. “Ship a recommendations feature” is an output and tells you nothing about whether it worked. “Increase the share of users who find a relevant item in their first session” is an outcome you can actually test a solution against. Outcome-framing is what keeps the rest of discovery honest.

Prototype and test before building

You can test a solution long before you build it, with experiments that cost hours instead of months:

- **Prototypes** — paper sketches or clickable mockups put in front of real users to test usability and desirability.
- **Wizard of Oz** — the interface looks real, but a human does the work behind the curtain, so you test the experience without building the engine.
- **Concierge** — you deliver the outcome manually for a few real users (no product at all) to learn whether it’s wanted and what it really takes.
- **Fake door / landing page** — measure interest in a thing that doesn’t exist yet.

Each retires a specific risk (usability, value, feasibility) for a fraction of the cost of building.

The MVP, correctly understood

The **Minimum Viable Product** is the most misunderstood term in software. It is **not** a low-quality first version of your product. It is the *smallest thing that lets you test your core hypothesis with real users* — an experiment, not a deliverable. The “viable” means *viable as a test of the idea*, not *viable as a product to live with*. If you can learn what you need from a landing page or a concierge run, that is your MVP, and you’ve saved months.

Prioritization: Deciding What to Pursue

Discovery surfaces more opportunities than you can pursue, so you need a way to choose. A few frameworks, used as thinking aids rather than oracles:

- **Value vs effort** — the simplest: plot ideas by expected value against cost, and start in the high-value / low-effort corner. Crude, but it filters out a lot.
- **RICE** — score by $\text{Reach} \times \text{Impact} \times \text{Confidence} \div \text{Effort}$, to compare opportunities on a consistent (if rough) basis.

- **MoSCoW** — sort into Must-have / Should-have / Could-have / Won't-have to force explicit trade-offs and an explicit “not now.”
- **Kano model** — distinguishes *basic* expectations (their absence angers, their presence is unnoticed), *performance* features (more is better), and *delighters* (unexpected joys). Useful for deciding where effort actually changes how users feel.
- **Opportunity Solution Trees** (Teresa Torres, *Continuous Discovery Habits*) — map a desired **outcome** to the **opportunities** (user needs / pain points) that could move it, then to candidate **solutions**, then to **experiments**. It keeps a whole team's ideas tied back to the outcome instead of to a feature wish-list.

The point of any of these is not the number they produce; it's that they force the conversation and make the reasoning behind a choice explicit.

A Worked Example: Discovery for a Notes Service

Suppose a team wants to build a small notes service — the same Notes API that Part IV of this book takes all the way to production. Discovery for it might look like this:

- **Problem, not solution.** Interviews with a dozen developers reveal the real pain is not “I need a notes app” but “the snippets and decisions I jot down during the day are scattered across chat, sticky notes, and scratch files, and I cannot find them a week later.”
- **Riskiest assumption.** *Value risk*: will people actually send notes to an API from their own tools, or will they keep using whatever is already open on their screen?
- **Smallest test.** Before writing a server, give five developers a two-line shell alias that appends a note to a shared file, and watch for two weeks: do they use it daily, and do they come back to search it?
- **Outcome.** Success is “developers retrieve a note they saved at least once a week,” not “we shipped create, read, update, and delete endpoints.”
- **Decision.** If usage stalls after the first days, stop — the evidence says the problem is not painful enough. If it holds, the validated problem, the target users, and the success measure become the input to project planning.

Notice that nothing here required the API to exist. Discovery retires the biggest risk with the cheapest possible experiment, and only then does building begin.

Common Traps

Discovery fails in predictable ways. Watch for:

- **Solution-first thinking** — falling in love with an answer and reverse-justifying the problem.
- **The feature factory** — measuring success by shipping volume, never by outcomes.
- **Confirmation bias** — running “tests” designed to hear yes; talking only to people who already agree; counting compliments as validation.
- **Building because you can** — technical excitement (“we could use this new model / framework”) masquerading as user value.
- **Skipping discovery under deadline pressure** — the most expensive shortcut there is, because it trades a few days of cheap learning for months of possibly wasted building.
- **Mistaking activity for progress** — interviews, decks, and experiments that never actually retire a real risk.

The antidote to all of them is the same: tie every step to a real, falsifiable question about value, and be willing to hear “no.”

The Handoff to Project Planning

Discovery is done when you can answer, with evidence rather than hope: *who* this is for, *what* problem it solves, *why* it’s worth solving, what *outcome* defines success, and which *riskiest assumptions* you’ve retired. At that point the question flips from “should we build this?” to “what exactly are we building, and how will we document and plan it?”

That is precisely where **Project Planning** (Part II) picks up: it turns a validated, well-understood problem into a problem statement, architecture, and the set of documents a real project keeps. Discovery decides *whether and what*; planning decides *exactly what and how*. Skip discovery and you plan beautifully for the wrong thing; skip planning and you build the right thing chaotically.

A Discovery Checklist

Before committing engineering effort, you should be able to answer:

- **Problem** — what real, frequent, painful problem am I solving, and for whom? (Stated as a problem, not a solution.)
- **Evidence** — have I talked to real users about their actual behavior, not pitched my idea?
- **Value** — is there evidence people will *change behavior* (use, switch, pay) to get this?
- **Outcome** — what observable outcome defines success, and how will I measure it?
- **Riskiest assumption** — what single assumption, if wrong, kills this — and have I tested it

cheaply?

- **Smallest test** — what is the least I can build or fake to learn whether this works?
- **Decision** — am I prepared to *not build it* if the evidence says no?

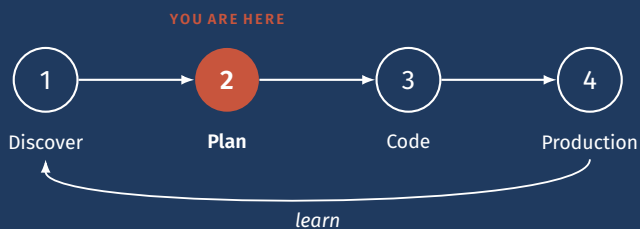
If you can answer these honestly, you've done the work that makes everything downstream worth doing.



PART II

Project Planning

Turning a chosen problem into a plan, and the living documents a well-run project keeps.



Project Planning Guide

A structured documentation template for new projects. Technology-agnostic: independent of any specific framework or language.

What This Guide Is

This is a structured documentation template to complete before — or at the very start of — any new project. Every section a project needs — across problem framing, architecture, code craft, operations, and delivery — is defined here at the conceptual level, ready to be filled in with project-specific detail.

The Guide is **technology-agnostic**. It does not prescribe which language to write in, which database to use, or which platform to deploy on. It defines *what topics must be documented*, *what each section should contain*, and *what a well-completed entry looks like*.

The structure mirrors what real project documentation actually contains — the sections any project's docs would cover are the sections this Guide prompts you to complete.

How to Read This Guide

For each subsection:

- **One-line definition** — what this section *is*
- **Why it matters** — what the project loses if this section is skipped or left vague
- **What this topic covers** — the specific questions and details to address
- **You understand this when** — a completeness check: the standard a well-filled section should meet

This is not a tutorial. It is a documentation prompt. The entries you write for each section are the deliverable; this Guide is the structure that tells you what to write.

Section 1 — Problem Statement & Goals

1.1 Background & Context

The history and surrounding conditions that gave rise to the project — the situation, team, system, or workflow this work plugs into.

Why it matters: Without context, a project's purpose is ambiguous. Engineers without context build technically correct systems that solve the wrong problem.

What this topic covers:

- The triggering event or pain point that initiated this project
- Upstream and downstream systems
- The team, process, or workflow this fits into
- Previous attempts and why they didn't work
- The history that informs current constraints

You understand this when you can explain in two paragraphs why this project exists *now*, what alternatives were considered, and what would happen if it weren't done.

1.2 Core Problem

The specific, concrete problem this project solves — narrow enough to be solvable, broad enough to matter.

Why it matters: A vague problem statement allows scope creep and produces a system that satisfies no one. A precise problem statement is the foundation for every later decision.

What this topic covers:

- The current pain, stated in 1–2 sentences
- How often the problem occurs
- Who is affected and what it costs them
- The root cause where known
- How to distinguish the problem from any proposed solution

You understand this when you can describe the problem in 30 seconds and a stranger understands what's broken without needing technical context.

1.3 Goals & Success Criteria

What “done” looks like — observable, ideally measurable conditions that indicate the problem is solved.

Why it matters: Goals like “improve performance” never tell you when to stop. Goals like “reduce p99 latency from 800 ms to 200 ms” do.

What this topic covers:

- Each goal as a specific outcome
- Each goal paired with an observable success condition
- The difference between vague aspirations and measurable criteria
- Functional vs non-functional goals
- Goals measurable before, during, and after the work

You understand this when every goal has a clear, testable success condition that even an outsider could verify.

1.4 Out of Scope

What this project explicitly does **not** address — the boundaries that protect against scope creep.

Why it matters: Without explicit scope boundaries, every stakeholder assumes their pet feature is in scope. Listing what’s out makes it possible to ship.

What this topic covers:

- Specific features or capabilities not being built
- The reason each thing is excluded
- Items deferred to a future phase, with the reason
- The distinction between “not yet” and “never”
- How to negotiate scope when new requests arrive

You understand this when the team can answer “is X in scope?” without re-litigating the question every meeting.

1.5 Stakeholders & Users

Who is affected by this project — users, beneficiaries, owners, and decision-makers.

Why it matters: Different roles need different things. Confusing “user” with “decision-maker” or “owner” with “beneficiary” causes communication failures and missed requirements.

What this topic covers:

- Primary users who interact day-to-day
- Beneficiaries who consume output indirectly
- Owners and maintainers responsible for the running system

- Stakeholders with input on priorities or compliance
- How to talk to each group about the work

You understand this when you can name every role, what they care about, how often they're consulted, and who has final say.

1.6 Constraints & Assumptions

Hard constraints (non-negotiable) and assumptions (things assumed true, that would change the design if false).

Why it matters: Implicit constraints become bugs. Implicit assumptions become outages. Both must be made explicit before they bite.

What this topic covers:

- Technical constraints (platforms, languages, integrations)
- Legal and compliance constraints
- Budget and timeline constraints
- Team and capability constraints
- Assumptions about user behavior, data, and environment
- Distinguishing constraints (hard) from assumptions (probabilistic)

You understand this when an outside engineer reading the constraints list can predict where the design is going to bend before they see it.

Section 2 — Architecture & Design

2.1 System Overview

A narrative of three to five steps describing what happens from trigger to output, in plain language, including how the system runs as a process.

Why it matters: Without an overview, every other architecture decision is unmoored. The overview is the elevator pitch for the whole system.

What this topic covers:

- Entry points — what initiates the system (user action, cron, event, API call)
- Process model — how the application exists as a running process (single process, worker pool, scheduled job) and what supervises it
- Core processing — what the system does with its input
- Output — what the system produces and where it goes

- Runtime lifecycle — startup → ready → handle → shutdown

You understand this when you can sketch the system for someone in five sentences, with no implementation jargon — the details belong in the sections that follow.

2.2 Architecture Diagram

A visual of the system showing components, data flow, external systems, storage, and APIs.

Why it matters: A diagram often communicates faster than prose. A team without diagrams reinvents the architecture in every conversation.

What this topic covers:

- Diagram types — flowchart, data-flow, system context, sequence, deployment
- What to show — components, arrows, external systems, storage, APIs
- The C4 model (context, container, component, code) and when each level fits
- ASCII diagrams as a viable medium when there's "no time for diagrams"
- The discipline of keeping diagrams current

You understand this when you can produce a diagram for a system you just understood, at the right level of detail for the audience.

2.3 Component Breakdown

Each major component, its responsibility, its inputs and outputs, its dependencies.

Why it matters: A system is its components. Without explicit responsibility boundaries, ownership is unclear and changes cascade unpredictably.

What this topic covers:

- One responsibility per component
- Inputs each component accepts
- Outputs each component produces
- Dependencies between components
- Interfaces — what's exposed, what's internal
- How to recognize when one component is doing too much (or too little)

You understand this when you can tell whether a change should live in component A or component B, and defend the choice.

2.4 Data Flow

The journey data takes through the system, from source to final destination, including the end-to-end request lifecycle.

Why it matters: Bugs in distributed systems are usually data-flow bugs — data that didn't arrive, arrived in the wrong order, or arrived in the wrong form. Tracing data flow is one of the most effective debugging techniques.

What this topic covers:

- Source — where data originates
- Ingestion — how data enters the system
- Processing — transformations, validations, enrichments
- Storage — intermediate and final results
- Output — final form and destination
- Error paths — what happens to data that fails validation or processing
- The request lifecycle in detail — client → network → kernel → application → database → response → back

You understand this when you can trace any input through the system and predict every transformation it undergoes.

2.5 External Integrations

Every external system, API, or service the project depends on — and the contract with each one.

Why it matters: External integrations are where systems meet. A bad integration contract is harder to fix than internal bugs because it requires both sides to change.

What this topic covers:

- Each integration's purpose
- Authentication method (API key, OAuth, mTLS, service account)
- Failure modes and how the system degrades when the integration is unavailable
- Contract dimensions — shape (REST/GraphQL/gRPC), versioning, identity, authorization, failure semantics
- Idempotency for safely retryable operations
- Rate limits imposed by external services
- Webhook handling for incoming events

You understand this when for every external dependency, you can answer “what happens

when this is unavailable?” and “how do we handle version skew safely?”

2.6 Data Storage & Schema

Every database, file store, and cache used — including the schema design and storage choice rationale.

Why it matters: Schema choices outlive everything else. The application can be rewritten; the database structure usually cannot.

What this topic covers:

- Storage type (relational, document, key-value, graph, columnar, time-series, search, object/blob)
- Schema design — entities, attributes, relationships
- Normalization (1NF–BCNF) and when to denormalize
- Key strategies (auto-increment, UUID, ULID; surrogate vs natural)
- Indexing strategy
- Transactions and consistency model
- Retention policy, archival, deletion behavior
- Schema evolution patterns (expand/contract, dual-write, backfills)

You understand this when for every entity in the system, you can defend the choice of storage type, the schema design, and the indexing strategy.

2.7 Key Design Decisions

The significant decisions made during architecture and design — and crucially, *why*.

Why it matters: Decisions made today are forgotten in six months. Without a written record, every architectural debate gets re-litigated.

What this topic covers:

- The decision itself
- Alternatives considered
- Rationale — why this option won
- Trade-offs accepted
- Date and owner of the decision
- ADR (Architecture Decision Record) format and lifecycle
- When a decision deserves an ADR and when it doesn’t

You understand this when you can read an ADR and reconstruct the discussion that produced

it, including the rejected alternatives.

2.8 Security Considerations

How the system protects against misuse, attack, and accidental exposure.

Why it matters: Security cannot be bolted on at the end. It must be designed in. A breach costs more than a hundred features.

What this topic covers:

- Authentication — how identity is verified (sessions, tokens, OAuth, mTLS)
- Authorization — who can do what (RBAC, ABAC, ownership-based)
- Secrets management — where credentials live, how they rotate
- Data sensitivity — classifying data (PII, financial, public) and handling each appropriately
- Transport security — HTTPS everywhere, TLS configuration
- Input validation and injection prevention (SQL, command, XSS, SSRF)
- Defense in depth — multiple layers, no single point of failure
- Logging policy — what to log, what *not* to log
- OWASP Top 10 awareness

You understand this when you can threat-model a new feature in your head and identify the top three risks before implementation begins.

Section 3 — Developer Guide

3.1 Prerequisites

Everything that must exist on a machine before the project can run.

Why it matters: Undocumented prerequisites are a leading cause of “works on my machine” failures.

What this topic covers:

- Runtime — language and version
- Package manager
- External tools and system libraries
- Credentials required (env vars, API keys, certificates)
- System requirements (OS, RAM, disk, network)
- How to verify each prerequisite is present

You understand this when a colleague can install every documented prerequisite on a fresh

machine without asking anyone for help.

3.2 Repository Structure

A map of the codebase — where things live, the conventions followed.

Why it matters: Developers should be able to find code by following the structure, not by searching.

What this topic covers:

- Top-level directories and what each contains
- Application code organization
- Where configuration, tests, scripts, documentation, and fixtures live
- A single, explicit composition point where the application is assembled (in Flask, the application factory)
- Naming conventions for files and directories

You understand this when you can predict where to add a new feature file before opening the project.

3.3 Environment Setup

The step-by-step process for getting the project running locally.

Why it matters: The setup process is the first impression. Bad setup means lost developers.

What this topic covers:

- Cloning the repository
- Creating and activating an isolated environment
- Installing dependencies
- Copying and configuring the environment file
- Running migrations or seed scripts
- Verifying setup with a smoke test
- Common setup failures and their fixes

You understand this when someone new can go from `git clone` to a running local instance in under 15 minutes following only your documented steps.

3.4 Running Locally

How to start the application, in development mode and in production-like mode.

Why it matters: Running locally is where most code is written and debugged. Subtle differences between local and production cause bugs that only surface late.

What this topic covers:

- Start command for development mode
- Start command for production-like mode
- How to switch environments (env var, config flag)
- Hot reload behavior
- Ports and URLs to access the running service
- A sample invocation with expected output
- How to verify the service is healthy

You understand this when you can run the project in both dev and prod modes locally and know why each mode exists.

3.5 Configuration Reference

Every configuration variable the project supports — type, default, description.

Why it matters: Undocumented config is invisible config. Engineers who don't know an option exists won't use it correctly.

What this topic covers:

- Each variable's name, type, default
- Required vs optional vs derived
- What each variable controls
- Format expected (URL, integer, boolean string)
- Validation that runs at startup
- The `.env.example` pattern for documenting variables

You understand this when you can read the configuration reference and predict the system's behavior in a new environment.

3.6 Testing

The test strategy — what's tested, how, and at what level.

Why it matters: Untested code is owned by no one. Tests document intent, catch regressions, and enable safe change.

What this topic covers:

- The test pyramid — unit, integration, contract, end-to-end

- How to run the full test suite
- How to run a single test or test file
- How to add a new test
- Test doubles — mock, stub, fake, spy — when each fits
- Test fixtures and data management
- Coverage as a signal (its uses and limits)
- Mocking strategy for external services
- Flaky tests and what to do about them

You understand this when you can write a test that catches a real regression and explain why each level of the pyramid is needed.

3.7 Code Conventions

The agreed-upon style and design principles the codebase follows.

Why it matters: Consistency reduces cognitive load. Code that all looks the same is code that’s easier to read. Code-craft principles are also the foundation under every other layer — design, testing, deployment, operations.

What this topic covers:

- Naming — files, functions, variables, constants
- Linting and formatting tools
- Comment style — when and how to comment
- Module design — deep modules, information hiding, coupling vs cohesion
- Error handling — how errors are raised, caught, logged; where to handle them
- Function design — size, parameters, return shapes, side effects
- Code smells — long methods, large classes, primitive obsession, feature envy
- Refactoring as a discipline — small steps, under test
- The cost of complexity over time
- Commit message format

You understand this when you can read existing code and tell which conventions are followed, then write new code that fits in seamlessly.

3.8 Common Debugging Scenarios

The recurring problems developers hit, and how to resolve them.

Why it matters: A debugging guide saves hours of repeated investigation. The “five-minute

fix you’ve forgotten how to do” is exactly what this section captures.

What this topic covers:

- Per-layer diagnosis — where to look first (DNS, network, app, database)
- Tools for each layer (strace, dig, curl, log search, journalctl, jq, database stats)
- Reproducing production issues locally
- Reading logs effectively — filtering, correlating with request IDs
- Using a debugger — breakpoints, stepping, watching state
- Common failure symptoms and their typical causes
- When to escalate vs when to dig deeper

You understand this when, the moment something breaks, you can identify which layer it’s in within two minutes and have a diagnostic command ready.

Section 4 — Operational Guide

4.1 Deployment

How the application gets from source to running production — including build, packaging, infrastructure, and the path traffic takes to reach it.

Why it matters: Deployment is where code becomes a service. A team that can’t deploy reliably can’t ship.

What this topic covers:

- Build process — turning source into a deployable artifact
- Packaging strategy (containers, wheels, OS packages)
- Artifact storage (registries, versioning by git hash)
- Infrastructure — servers, VMs, containers, managed services
- The path from artifact to running process (pull, supervise, start)
- Networking — DNS, reverse proxy, TLS termination, firewalls
- Deployment strategies — recreate, rolling, blue-green, canary
- Rollback procedure
- How configuration and secrets are injected at runtime

You understand this when you can trace a code change from `git push` all the way to user traffic, naming every system it passes through.

4.2 Scheduling & Triggers

How the system handles work that isn't direct user requests — scheduled jobs, event-triggered tasks, webhooks.

Why it matters: Most production systems have hidden work running in the background. Missed runs are silent failures.

What this topic covers:

- Trigger types — cron, event, webhook, manual
- Cron expressions and the timezone trap (cron uses the server's local time — schedule in UTC)
- Modern alternatives to cron (systemd timers, cloud schedulers)
- Event-triggered work — webhooks, message queues
- Overlap handling — what if one run is still active when the next fires
- Idempotency for scheduled jobs — essential; jobs must be safely re-runnable
- Durability — what if the scheduler or job crashes mid-execution
- Monitoring missed runs (not just failed runs)

You understand this when for every scheduled job in the system, you can describe what triggers it, what happens if it fails, and how you'd notice if it stopped running.

4.3 Monitoring & Health Checks

How you know the system is alive and behaving correctly.

Why it matters: A system you can't observe is a system you can't operate. Problems must surface before users notice.

What this topic covers:

- The three pillars — logs, metrics, traces
- Structured logging (JSON) and request ID correlation
- Health check endpoints — deep vs shallow
- The four golden signals — latency, traffic, errors, saturation
- Metrics — counters, gauges, histograms
- Log aggregation
- Alerting on symptoms (not causes), and the alert-fatigue trap
- SLIs, SLOs, error budgets

You understand this when you can answer “is the system healthy right now?” with a single command, and you trust the answer.

4.4 Common Failure Modes

The known ways the system can fail in production, and how to diagnose each one.

Why it matters: Engineers diagnose faster when failures are familiar. A documented failure mode is a failure that's already half-solved.

What this topic covers:

- Per-layer failures — server (OOM, disk full), reverse proxy (cert expiry, misconfig), container (crash loop, unhealthy), application (worker timeout, queue saturation), database (connection refused, slow queries)
- Symptoms — what the user sees
- Detection — what to look for in logs/metrics
- Diagnosis commands — what to run to confirm
- Recovery steps
- Prevention measures for the future

You understand this when you can pattern-match an alert to a likely failure mode in under a minute and start diagnosis with the right command.

4.5 Restart & Recovery Procedures

The step-by-step procedure for restoring a service after a failure.

Why it matters: Under stress, fresh thinking fails. A documented runbook is the difference between five-minute and five-hour recovery.

What this topic covers:

- Assess — how to determine the scope of the failure
- Stop — how to cleanly stop a running process if needed
- Fix — what commonly needs to be resolved before restart
- Restart — the exact command
- Verify — how to confirm the system is healthy
- Process supervisors and restart policies
- Graceful shutdown — draining in-flight requests, signal handling
- The PID 1 signal-handling gotcha in containers
- Restoring from backup during an incident (strategy and restore testing live in 4.6)
- Postmortem documentation and blameless culture

You understand this when you can recover the service from a cold start without consulting documentation — but you wrote the runbook anyway.

4.6 Data Management

How data is retained, archived, deleted, and protected over time.

Why it matters: Real systems accumulate data that's expensive to keep and legally fraught to delete badly. Both retention and deletion are increasingly regulated.

What this topic covers:

- Retention policy — how long different data lives
- Storage tiers — hot, warm, cold, archival
- Archival procedures — moving old data without losing it
- Cleanup jobs — safe deletion with dry-runs and audit logs
- PII handling — minimization, pseudonymization, what to log and what not to
- Backup strategy — frequency, storage, verification
- Restore testing — an untested backup is a hope, not a backup
- Compliance-driven retention (GDPR right-to-delete, legal hold)
- The difference between soft delete, hard delete, and cryptographic deletion
- Local vs production parity in data behavior

You understand this when for every category of data in the system, you can describe how long it lives, how it's backed up, how it would be deleted, and how a data-subject deletion request would be handled.

Section 5 — Delivery & Changelog

5.1 Delivery Phases

The discrete milestones the project is broken into — each with a clear goal and definition of done.

Why it matters: Without phases, projects either drift indefinitely or get rushed at the end. Phases create natural checkpoints.

What this topic covers:

- Each phase's goal
- Scope included in each phase
- Status (completed / in progress / planned)
- Target or actual completion date
- The distinction between phases (stable milestones) and tasks (individual work items)
- How to handle scope additions during a phase

You understand this when you can describe the project’s phases to a stakeholder and predict when each one delivers value.

5.2 Current State

A snapshot of what’s live in production right now.

Why it matters: Without a current-state record, no one knows what version is deployed or what works. New team members can’t get up to speed.

What this topic covers:

- What features are live in production
- Current version (release tag or git hash)
- Date of most recent change to production
- Known limitations of the current state
- Active workarounds in place
- The relationship between current state and the roadmap

You understand this when any team member can answer “what’s in production right now?” — and the answer matches what the deployment system reports.

5.3 Changelog

A running record of significant changes — what was changed, when, and by whom.

Why it matters: Without a changelog, releases are opaque. Users can’t tell what’s new; engineers can’t tell what’s broken.

What this topic covers:

- Date of each significant change
- Version number (semantic versioning — major.minor.patch)
- Author
- Category of change — added, changed, deprecated, removed, fixed, security
- Conventions — Keep a Changelog, Conventional Commits
- The difference between deploys (technical) and releases (user-visible)
- Release notes for stakeholders vs developer changelog

You understand this when you can read the changelog and reconstruct the project’s history at a glance.

5.4 Known Issues & Tech Debt

The acknowledged problems that haven't been fixed yet — and the trade-offs made deliberately.

Why it matters: Tech debt nobody tracks compounds silently. Acknowledged debt can be paid down strategically.

What this topic covers:

- Each known issue with impact — who or what is affected
- The current workaround
- The proper fix
- Each piece of tech debt with: why it was done this way, what it would take to clean up
- The technical-debt quadrant (Martin Fowler) — deliberate vs inadvertent, prudent vs reckless
- How to prioritize debt against new feature work

You understand this when you can defend the prioritization between paying down a piece of debt and shipping a new feature.

5.5 Future Roadmap

What comes next — features and improvements planned, prioritized.

Why it matters: A roadmap aligns the team and stakeholders on direction. Without one, every priority debate happens in isolation.

What this topic covers:

- Each upcoming feature or improvement
- Priority (High / Medium / Low)
- Rationale for each priority
- Dependencies between roadmap items
- Time horizon (next quarter, next year, eventually)
- How the roadmap is revised when reality intervenes

You understand this when you can explain to a new team member what's coming in the next quarter and why.

A Worked Example: Planning the Notes API

The template is easier to use once you have seen it filled in. Here is a compressed plan for the notes service whose discovery was sketched in Part I — the same Notes API that Part IV

builds, chapter by chapter. A real plan would say more in each section; the point here is what *one good line* per section looks like.

Section 1 — Problem Statement & Goals

- **Background & context:** discovery showed that developers lose the snippets and decisions they jot down during the day; a two-week trial with a shared file confirmed they would use a simple capture tool daily.
- **Core problem:** capturing a note must take one command from wherever the developer already is, and finding it again must be just as quick.
- **Goals & success criteria:** a note can be created, listed, read, updated, and deleted over HTTPS; success is “each pilot user retrieves a saved note at least once a week” after one month.
- **Out of scope (v1):** user accounts and sharing, a web or mobile front end, full-text search ranking, attachments.
- **Stakeholders & users:** the twelve pilot developers (users); the one engineer building it (owner); the team lead who pays the hosting bill.
- **Constraints & assumptions:** one engineer, six weeks, hosting under \$25 a month; assumes a single region and a few requests per second at most.

Section 2 — Architecture & Design

- **System overview:** a small REST API — Python and Flask — in a Docker container on one Linux VM, behind Nginx for TLS, with a managed PostgreSQL database.
- **Architecture diagram:** client → DNS → cloud firewall → Nginx (TLS) → Gunicorn in Docker → managed PostgreSQL over a private network.
- **Component breakdown:** application factory, configuration module, database access, route handlers, validation, error handlers, logging.
- **Data flow:** a POST /notes/ is validated, inserted, committed, and returned as JSON; every request is logged with its status and duration.
- **External integrations:** none in v1 beyond the database — which is still a contract, with its own connection string, credentials, and failure modes.
- **Data storage & schema:** one table, notes(id, content, created_at, updated_at); schema changes go through versioned migrations.
- **Key design decisions:** SQLite for local development and PostgreSQL in production, recorded as an ADR — and later superseded by “PostgreSQL everywhere” when the difference hid a bug.
- **Security considerations:** HTTPS only; all input validated; secrets in environment files,

never in the repository; no authentication in v1 is an *accepted, written-down* risk: the pilot’s URL is shared only with the pilot users, the data is low-stakes, and authentication is first on the roadmap.

Section 3 — Developer Guide

- **Prerequisites:** Python 3.11, Docker, git.
- **Repository structure:** `app/` (the package), `tests/`, `migrations/`, `Dockerfile`, `Makefile`, `requirements*.txt`.
- **Environment setup:** create a virtual environment, `pip install -r requirements-dev.txt`, copy `.env.example` to `.env` and fill in `SECRET_KEY`.
- **Running locally:** `python run.py`, then `curl http://localhost:5000/health`.
- **Configuration reference:** `DATABASE_URL` and `SECRET_KEY` (required); `FLASK_DEBUG`, `PORT`, `HOST`, `LOG_LEVEL`, `LOG_FORMAT` (optional, with defaults).
- **Testing:** `make build` runs lint, type checks, the test suite, and a dependency audit; CI runs the same on every push.
- **Code conventions:** formatted by black, linted by ruff; one error-response shape for the whole API.
- **Common debugging scenarios:** “port already in use”, a missing environment variable at startup, a 415 because a client forgot `Content-Type: application/json`.

Section 4 — Operational Guide

- **Deployment:** CI builds and smoke-tests an image tagged by commit, pushes it to the registry, and runs `deploy.sh sha-<commit>` on the server, which rolls back automatically if the health check fails.
- **Scheduling & triggers:** none in v1; a nightly cleanup job would be the first.
- **Monitoring & health checks:** `/health` checks the database and is watched by an external uptime monitor; structured JSON logs; alerts on downtime and error rate.
- **Common failure modes:** database unreachable (503 from `/health`), expired TLS certificate, disk full from logs, a crash-looping container after a bad deploy.
- **Restart & recovery:** redeploy the last good tag; the service-down runbook covers the first ten minutes of any outage.
- **Data management:** managed-database backups kept 7 days, plus an off-server copy before every schema change; a monthly restore test.

Section 5 — Delivery & Changelog

- **Delivery phases:** (1) runs locally, (2) portable and configurable, (3) packaged as an image,

(4) deployed to a server, (5) reachable over HTTPS, (6) observable and hardened — each a working system before the next begins.

- **Current state:** v1.2.0 in production (image sha-a1b2c3d), deployed on the date in the changelog.
- **Changelog:** kept in `CHANGELOG.md` under *Added* / *Changed* / *Fixed* / *Security*, one entry per release.
- **Known issues & tech debt:** no authentication (accepted for the pilot); one server is a single point of failure; the list endpoint needs pagination before the data grows.
- **Future roadmap:** High — authentication, before anyone outside the pilot gets the URL; Medium — pagination and search; Low — a small web front end.

Filled in like this, the plan fits on three pages, and every line is something a newcomer, an operator, or a future you will need to know.

How This Guide Evolves

This Guide is a living project artifact. Complete each section before or during early development, and return to update it as the project evolves — when architecture decisions are made, when scope changes, when new failure modes are discovered.

Over time, the **Why it matters** and **What this topic covers** prompts recede into the background and the sections fill with real project content. The Guide is alive. Each decision made, each component added, and each incident resolved is an input to it.



PART III

Writing Good Code

The craft of code that is correct, clear, and changeable, and the practices that keep a codebase good.



The Craft of Good Code

Code that *works* is the floor, not the goal. Almost any approach produces code that passes its tests today. **Good** code is code that the next person — usually you, six months from now — can read, understand, and change *safely*. That is a higher and more durable bar, and it is what this part of the book is about.

It has two chapters, because “good code” means two things at once:

- **The Craft of Good Code.** How to write each individual piece well: naming, functions, modules, error handling, comments, refactoring. This is the *how* of turning intent into code a human can follow.
- **Engineering Practices & Maturity.** What a whole *codebase* must have to be considered good — logging, testing, validation, security, reliability — and, crucially, in what *order* to adopt each practice as a project grows. This is the practical checklist behind the question “is this codebase actually good?”

The two reinforce each other: the craft makes each line good; the practices make the whole system good.

What “Good Code” Actually Means

There is one enemy beneath every rule that follows, and it has a name: **complexity** — anything about the structure of the code that makes it hard to understand or change. Complexity is not “lots of code” or “advanced code”; a hundred-line file can be complex and a hundred-thousand-line system can be simple. What matters is how hard it is to change *safely*.

John Ousterhout’s *A Philosophy of Software Design* names its three symptoms, and every technique in this part is a tactic against one of them:

- **High cognitive load** — how much you must hold in your head to make a correct change. The more context a single edit requires, the worse the code.
- **Change amplification** — how many places one logical change forces you to touch. Good

structure keeps one change *one* change.

- **Unknown unknowns** — when it is not even obvious *which* code must change, or what else a change will affect. This is the worst of the three, because you cannot plan around what you cannot see.

A useful reframing: **code is read far more often than it is written, and changed far more often than you expect.** Optimize for the reader and the future editor, not for the few minutes it takes to type it.

Naming

Names are the cheapest and highest-leverage design decision you make. A good name tells the reader *what a thing is*, *what it does*, and *what it costs* — so they never have to open the body to find out.

- **Reveal intent.** `invoice_due_date_utc` beats `d`; `is_eligible_for_refund` beats `flag`.
- **Avoid noise words** — `data`, `info`, `tmp`, `manager`, `helper`, `util`. They carry no meaning; a name that could apply to anything describes nothing.
- **Be consistent.** One concept, one word, everywhere. Don't call it `user` here and `account` there if it's the same thing.
- **Length should match scope.** A loop index `i` is fine; a one-letter module-level constant is not.

A name you struggle to find is usually a sign the thing is doing too much. The naming difficulty is the design telling you to split it.

Functions

- **Small, and one thing.** A function should do a single thing at a single level of abstraction. If you need the word “and” to describe it, it's probably two functions.
- **Few parameters.** Many parameters are hard to read and easy to mis-order. Prefer passing an object; be suspicious of boolean flag parameters (they usually mean the function does two things — split it).
- **Command/Query Separation.** A function should either *do* something (a command — it has a side effect and returns nothing meaningful) or *answer* something (a query — it returns a value and changes nothing), but not both. Functions that quietly do both surprise their callers.

- **Make side effects obvious.** Hidden state changes are where bugs hide. A function that mutates something outside itself should say so in its name and its docs.

Modules, Boundaries, and Design

Above the function sits the module (a class, a file, a package, a service). Good module design is mostly one idea — **information hiding** — expressed several ways.

- **Information hiding.** A module exposes a small interface and hides its implementation behind it. The secret it keeps is exactly the complexity its users never have to think about. Expose little; hide much.
- **Deep vs shallow modules.** A *deep* module offers a simple interface over substantial hidden functionality — it earns its place. A *shallow* module has an interface nearly as complex as its body (a thin wrapper, a class whose every method you must understand to call any) — it barely pays for the cost of existing. Prefer deep modules.
- **Coupling and cohesion.** Aim for **high cohesion** (each unit does one well-defined job) and **low coupling** (units interact through the narrowest possible interface). This single sentence is the north star of structural design.
- **Separation of concerns.** Keep distinct responsibilities in distinct places — business logic, data access, and transport (HTTP/CLI) in separate layers. When a request handler also queries the database and sends email, every future change has to understand all three.
- **The SOLID principles**, briefly, each guarding against one failure: Single Responsibility (a unit has one reason to change); Open/Closed (extend without modifying); Liskov Substitution (subtypes honor their base’s contract); Interface Segregation (small, focused interfaces); Dependency Inversion (depend on abstractions, not concretions — your logic talks to a `UserRepository` interface, not to PostgreSQL directly).
- **Avoid premature complexity.** Microservices, event sourcing, CQRS, and elaborate abstractions are powerful and frequently catastrophic when adopted too early. Choose the simplest design that solves today’s problem while keeping tomorrow’s options open.

Comments

Good code explains *what* it does through naming and structure. Comments are for the **why** — the non-obvious trade-off, the external constraint, the subtle bug that forced this approach.

- A comment that restates the code (`i = i + 1 # add one to i`) is noise.
- A comment that has drifted out of sync with the code is worse than none — it actively lies.

If you change the code, change the comment.

- The best “comment” is often a better name or a smaller function that makes the comment unnecessary.

Error Handling

Decide an error-handling philosophy and apply it consistently — inconsistency here produces the hardest bugs to find.

- **Expected vs exceptional.** Outcomes that are a normal part of the job (invalid input, a missing record) are best handled with ordinary return values and clear responses. Genuine surprises (a dropped connection) are exceptions, and should be allowed to propagate.
- **Handle errors where you can act.** If a function can’t do anything useful about an error, it should not catch it — let it propagate to a boundary that can log it and turn it into a sane response. Catching to do nothing hides the problem without solving it.
- **Never swallow silently.** `except Exception: pass` is how a system acquires invisible failures. Catch *narrowly*, or not at all.
- **Design errors out of existence.** The best error handling is the error you made impossible — an idempotent “delete” that treats “already gone” as success removes an entire error path. (Whether that is right depends on what callers need to know; the Notes API in Part IV deliberately keeps a 404 so clients can tell “deleted now” from “never existed”.)

Code Smells

A **code smell** (the term comes from Kent Beck and Martin Fowler’s *Refactoring*) is a recognizable surface pattern that hints at deeper trouble. A smell is not a bug; it’s a prompt to look closer. The ones worth memorizing:

- **Long method / large class (god object)** — a unit that has absorbed too many responsibilities.
- **Long parameter list** — usually a missing object, or a function doing too much.
- **Primitive obsession** — passing raw strings and dicts where a named type belongs.
- **Feature envy** — a function more interested in another module’s data than its own.
- **Shotgun surgery** — one logical change that forces edits scattered across many files.
- **Duplicated code** — the same knowledge expressed in more than one place.
- **Dead code** — commented-out blocks and unused paths that confuse what actually runs. Delete aggressively; version control remembers.

Refactoring

Refactoring is changing the *structure* of code without changing its *behavior* — and it is a continuous discipline, not a one-time project.

- Work in **small, safe steps**, each leaving the code working, ideally under tests: small change → run tests → commit. This rhythm is what makes restructuring safe instead of scary.
- Follow the **boy-scout rule**: leave each file a little cleaner than you found it. Continuous small improvements compound; the “big rewrite” rarely arrives and rarely survives contact with reality.

Strategic vs Tactical Programming

Every change is an opportunity to invest or to borrow.

- **Tactical programming** optimizes for *making this work now*. It is faster per change and leaves a little complexity behind each time — and that complexity compounds until the codebase fights you.
- **Strategic programming** treats clean design as an investment that keeps *every future change* cheap. It costs a little more today and pays back continuously.

Good code is the result of many small strategic investments. You don’t need a quarter to “clean up the codebase”; you need the habit of leaving good structure behind on every change.

A Practitioner’s Habits

The things experienced engineers do almost without thinking:

- Name things so the body rarely needs reading.
- Keep functions small and honest about their side effects.
- Hide complexity behind deep, narrow interfaces.
- Make the easy case obvious and the dangerous case hard.
- Handle errors at a boundary; never swallow them.
- Delete dead code on sight.
- Refactor in small steps, under tests, as you go.
- Reach for the simplest thing that works, and add complexity only when reality demands it.

Engineering Practices & Maturity

The previous chapter is about each *piece* of code. This chapter is about the whole *codebase*: the practices a project should have to be considered good, sequenced by the category of risk each one manages. No codebase has all of this on day one — the goal is to be deliberate about what you have and what you’re deferring, and to adopt each practice in the right order, before its absence causes an incident.

Overview

This chapter serves two purposes:

1. A **maturity model** that sequences the adoption of engineering practices across the lifecycle of a software project.
2. A **comprehensive reference** for every major dimension of codebase quality.

Each stage in the maturity model manages a different *category of risk*. Moving to the next stage before the current one is solid doesn’t skip work — it defers it under worse conditions. **The order matters as much as the content.**

The Complete Roadmap at a Glance

Stage	Primary Risk Being Managed	Key Deliverables
1. Foundation	Irreversible early mistakes	Version control, secrets hygiene, linting, README
2. Early Development	Silent failures, no feedback loops	Unit tests, logging, CI, input validation
3. Growing Codebase	Inability to change safely	Architecture layers, integration tests, code review, ADRs
4. Production Readiness	Inability to operate	Metrics/alerting, deployment automation, rollback, runbooks
5. Scale & Reliability	Systemic and cascading failures	IaC, circuit breakers, tracing, verified backups
6. Excellence	Long-term sustainability	Postmortems, performance testing, compliance, bus factor

Stage 1 — Project Foundation

Day 0. Before a single line of business logic is written.

The decisions made here are the hardest to undo later. This stage is about establishing the structural skeleton that everything else will grow on.

Stage 1: Non-Negotiables

Version Control with a Branching Strategy

Initialize a Git repository before writing any code. Decide immediately on a branching convention — trunk-based development (recommended for small teams) or Gitflow. This decision shapes how you integrate, review, and release for the entire life of the project. Changing it later is painful.

Dependency and Environment Management

Define your runtime explicitly: a `package.json`, `pyproject.toml`, `go.mod`, or equivalent. Pin versions. Use a virtual environment or container from the start. The “it works on my machine” problem is born here if you ignore it.

Project README

Write it on day one, even if it's three lines. It should answer: what is this, how do I run it, and how do I run the tests. Update it as the project evolves. This is the front door to your codebase.

.gitignore and Secrets Hygiene

A `.gitignore` that excludes build artifacts, local config, and `.env` files should exist before the first commit. An `.env.example` with placeholder values documents what configuration the project needs without leaking real values. A secret committed to version control must be treated as leaked: rotate it immediately. Rewriting history afterwards is not enough, because every clone, fork, and CI cache may already hold a copy.

Linter and Formatter

Configure both before writing any code. A linter (Ruff, ESLint, golangci-lint) catches likely bugs and bad patterns; a formatter settles style. Zero-configuration formatters (Black or ruff format, Prettier, gofmt) are preferable because they eliminate style decisions entirely. Enforcing style from the start means you never have a “style cleanup” PR that pollutes your git history.

Stage 1: Nice to Have

- A basic Makefile or task runner (just, invoke) with commands for run, test, lint so common operations are documented as code.
- A brief CONTRIBUTING.md stub, even if it's just the branching strategy.

Stage 1: Why Not Later?

Version control, secrets hygiene, and formatting enforced *after* code is already written require retroactive cleanup that almost never happens completely. These are the only practices where “later” genuinely means “never.”

Stage 2 — Early Development

The first few weeks. You're writing real features and the codebase is taking shape.

You now have business logic worth protecting and interfaces worth stabilizing. This stage builds the feedback loops and structural habits that will carry the project forward.

Stage 2: Non-Negotiables

Unit Tests for Business Logic

Write tests alongside every non-trivial function. You don't need 100% coverage — you need coverage of critical paths, edge cases, and anything that's broken once before. The discipline of writing tests early forces better design: tightly coupled, side-effectful code is hard to test, so tests push you toward cleaner architecture.

Basic Logging

Introduce structured logging from the start. Every significant operation, error, and state transition should emit a log entry. Use levels correctly:

Level	Purpose
DEBUG	Development noise, verbose detail
INFO	Normal operations, milestones
WARNING	Recoverable anomalies
ERROR	Failures requiring attention

Retrofitting logging into existing code is tedious and leaves gaps.

Basic CI Pipeline

Connect your repository to a CI service (GitHub Actions, GitLab CI, CircleCI). At minimum, CI should: install dependencies, run the linter, and run the test suite on every push and pull request. This is the first automated quality gate. Without it, broken code regularly reaches shared branches.

Explicit Error Handling

Decide on an error-handling philosophy and apply it consistently. Never swallow exceptions silently. Errors should either be handled gracefully (with logging and a meaningful response) or propagate loudly so they surface in tests and monitoring. Silent failures are the hardest bugs to diagnose.

Input Validation at Boundaries

Any data entering your system from outside — HTTP requests, file uploads, CLI arguments, environment variables — should be validated at the point of entry. Do not let unvalidated data propagate deep into business logic. Libraries like Pydantic, Zod, or Joi make this structured

and cheap.

Stage 2: Nice to Have

- A `CHANGELOG.md` started early, even informally.
- Pre-commit hooks for linting and formatting (faster feedback than waiting for CI).
- Basic health check endpoint if the project is a service.

Stage 2: Dependencies on Stage 1

- CI depends on version control.
- Testing depends on having a structured project layout.
- Both depend on dependency management being correctly configured.

Stage 2: Why Not Later?

Tests and logging written after the fact tend to be superficial — they cover the happy path because the developer already knows the code works. Written alongside the code, they reveal design problems while they're still cheap to fix. CI without tests is useless; tests without CI are run inconsistently.

Stage 3 — Growing Codebase

The project has real users or real collaborators. The codebase is large enough that no one person holds it all in their head.

The primary risk at this stage shifts from “will it work?” to “will we be able to change it safely?” The practices here protect the ability to move quickly without breaking things.

Stage 3: Non-Negotiables

Architecture with Clear Separation of Concerns

Business logic, data access, and transport (HTTP handlers, CLI commands) should live in distinct layers with clear interfaces between them. If your HTTP handler is also querying the database and sending emails, you have a maintenance problem that compounds with every feature added. Introduce this structure before the codebase is too large to refactor easily.

Integration Tests

Unit tests verify logic in isolation. Integration tests verify that components actually work together — that your code and your database produce the right results, that your service and its dependencies interact correctly. Add them as you build each significant integration.

Secrets Manager or Vault

If you're still loading secrets from `.env` files passed around manually, now is the time to move to a proper secrets management solution (AWS Secrets Manager, HashiCorp Vault, Doppler). As teams grow, sharing secrets over Slack or email becomes a serious security liability.

Dependency Vulnerability Scanning

Enable automated scanning (GitHub Dependabot, Snyk, `pip-audit`) on your repository. At this stage you likely have enough dependencies that manually auditing them is impractical. This runs passively and alerts you to known CVEs.

Code Review Process

Establish a pull request process with required reviews before merging. Document what reviewers are expected to check. Code review is the primary mechanism for knowledge sharing, catching design problems, and maintaining quality standards as a team grows. Without it, the codebase quickly diverges in style and quality.

Architecture Decision Records (ADRs)

When you make a significant architectural decision — choosing a database, selecting a message queue, deciding on an auth approach — write a short ADR. Future maintainers (including yourself) will otherwise spend hours reverse-engineering *why* the code looks the way it does.

Stage 3: Nice to Have

- Database migration tooling (Flyway, Alembic, `golang-migrate`) with versioned, reversible migrations.
- API documentation (OpenAPI/Swagger) if you have external or cross-team consumers.
- A formal `CONTRIBUTING.md` that formalizes PR conventions, commit message format, and review expectations.
- Performance benchmarks for critical paths, even informal ones, to establish a baseline.

Stage 3: Dependencies on Stage 2

- Integration tests depend on having a test infrastructure already established.

- A code review process depends on CI — reviewers should not be manually checking things the machine can check.

Stage 3: Why Not Earlier?

Formal architecture layers are premature in the first days of a project when the domain isn't understood yet. Introduced too early, they add ceremony without value. By this stage, the domain is understood and the cost of architectural debt is becoming visible.

Stage 4 — Production Readiness

You're about to deploy to real users, or you're already there and gaps are becoming painful.

The primary risk shifts again: from “can we change it safely?” to “can we operate it?” This stage is about making the system observable, deployable, and recoverable.

Stage 4: Non-Negotiables

Structured Metrics and Alerting

Logs tell you what happened; metrics tell you the shape of your system's behavior over time. Instrument key operations: request rates, error rates, latency percentiles, queue depths. Connect metrics to an alerting system (PagerDuty, Opsgenie, or even a simple email alert) so you find out about problems before your users do.

Health Check and Readiness Endpoints

Every service must expose health endpoints. Load balancers and orchestration systems (Kubernetes) use these to route traffic. Without them, you may be serving requests to a broken instance with no visibility into it.

Automated, Repeatable Deployment

Deploying to production should be a single command or pipeline trigger, never a manual runbook. Every manual deployment step is a potential source of human error and an undocumented process. The deployment pipeline is part of the product.

Rollback Strategy

Every deployment must have a defined, tested rollback path. Blue-green deployments keep the previous version hot. At minimum, know the exact steps to revert a bad deployment before

you deploy. **A rollback strategy you haven't practiced is not a rollback strategy.**

End-to-End Tests for Critical Paths

A small suite of E2E tests covering your most important user journeys — login, checkout, core workflow — gives you confidence that a deployment hasn't broken the product from the user's perspective. These are slow and should be few, but they catch failure modes nothing else will.

Runbooks for Common Incidents

Before going to production, write a runbook for each foreseeable incident: service is down, database is full, a background job is stuck, an external dependency is unavailable. The act of writing them surfaces gaps in your system design. They save hours during actual incidents.

Stage 4: Nice to Have

- Feature flags for decoupling deployment from release.
- Staging environment that closely mirrors production.
- Basic load testing before major releases.
- Audit logging for user-facing actions that touch sensitive data.

Stage 4: Dependencies on Stage 3

- Automated deployment depends on a CI pipeline.
- E2E tests depend on an established test infrastructure.
- Metrics alerting depends on knowing what “normal” looks like. Start with simple static thresholds (error rate, health check, disk space) at launch, and tune them once real traffic shows the baseline.

Stage 4: Why Not Earlier?

Runbooks, rollback strategies, and production metrics are irrelevant before you have a production environment. Introducing E2E tests too early, when the UI and API are changing daily, produces a brittle suite that becomes a maintenance burden rather than a safety net.

Stage 5 — Scale and Reliability

The product is established. Failures are costly. The team is larger. The system has real operational complexity.

At this stage, the risks are systemic: cascading failures, knowledge silos, environments that drift, and infrastructure that was clicked together in a console two years ago by someone who has since left.

Stage 5: Non-Negotiables

Infrastructure as Code

All cloud infrastructure — servers, databases, queues, DNS, IAM roles — should be defined in version-controlled code (Terraform, Pulumi, CloudFormation). Undocumented infrastructure is invisible technical debt. When something breaks in an environment you can't reproduce, you have a serious problem.

Graceful Degradation and Circuit Breakers

Design explicitly for partial failure. When a non-critical dependency is down, the core product should continue functioning. Circuit breakers (`resilience4j`, `pybreaker`) stop requests to a known-failing service before they exhaust your connection pool and bring down everything else. This is what separates “that service was slow for 5 minutes” from a full outage.

Idempotency for Critical Operations

Any operation that can be retried — payment processing, email sending, job execution — must be idempotent: running it twice should produce the same result as running it once. Without this, retries produce duplicate charges, duplicate emails, and corrupted state.

Timeouts, Retries, and Backoff Everywhere

Every outbound call should have an explicit timeout. Retries should use exponential backoff with jitter to prevent thundering herds. Without these, a single slow dependency can exhaust your threads, block your event loop, and cascade into a full outage.

Distributed Tracing

When a request crosses multiple services, you need the ability to follow it end-to-end. OpenTelemetry is the current standard. Without tracing, debugging latency or correctness issues across service boundaries is largely guesswork.

Disaster Recovery — Verified Backups

Having backups is not enough. You must regularly test your ability to *restore* from them and measure your actual RTO (recovery time objective) and RPO (recovery point objective). Many teams discover their backups are corrupted or incomplete during an actual disaster.

Stage 5: Nice to Have

- Chaos engineering practices (controlled fault injection to find reliability gaps before they find you).
- SLOs (Service Level Objectives) formally defined and tracked.
- Contract tests between services to catch API incompatibilities before deployment.
- Capacity planning reviews — understanding what breaks first when load doubles.

Stage 5: Dependencies on Stage 4

- Circuit breakers and graceful degradation require observability to know when to open and close.
- IaC is most tractable when introduced before infrastructure proliferates; at this stage it may require an audit and cleanup first.
- Distributed tracing is only valuable once you have multiple services.

Stage 6 — Advanced Engineering Excellence

A mature, large, or high-stakes system. Operational discipline is institutionalized. You are optimizing for long-term sustainability.

These practices have high value but require foundational maturity to implement well. Applied too early, they add overhead without payoff.

Stage 6: Practices

Blameless Postmortems as Institutional Practice

Every significant incident produces a written postmortem: timeline, root cause, contributing factors, and concrete action items. The goal is systemic improvement, not blame. Over time, a postmortem corpus becomes invaluable institutional knowledge that outlasts any individual engineer.

Performance Testing as a First-Class Practice

Automated load tests run in CI or pre-release pipelines. Benchmarks for critical paths are tracked over time so regressions are caught before they reach production. Latency budgets are defined and enforced.

Data Retention and Lifecycle Policies

Define what data gets deleted or archived, and when. Databases that grow without bound degrade in performance and increase cost in ways that are often invisible until they become incidents. Compliance requirements (GDPR right to erasure, HIPAA retention periods) add urgency for regulated domains.

API Versioning and Backward Compatibility Strategy

If external consumers depend on your interfaces, establish a formal versioning policy, deprecation timelines, and compatibility guarantees. Schema changes without versioning discipline are a primary source of silent breakage for API consumers.

Compliance and Auditability

In regulated domains (finance, healthcare, legal), audit logs — recording who did what and when — are distinct from application logs and are a legal requirement. Access controls, data residency, and retention policies also become non-negotiable at this stage.

Developer Onboarding Time as a Metric

Track how long it takes a new engineer to make their first meaningful contribution. This single metric reflects the aggregate quality of your documentation, tooling, architecture, and development environment. Optimizing it forces improvements across many dimensions simultaneously.

Knowledge Sharing and Bus Factor Reduction

Formal practices for reducing knowledge silos: pair programming rotations, internal tech talks, documentation sprints, rotating on-call assignments. A system that only one person understands is a liability, not an asset.

Full Quality Framework Reference

Observability and Diagnostics

Logging (*Essential*) — Structured logs (JSON, key-value pairs) are far more useful than plain strings. Log at appropriate levels and never log sensitive data. Without logging, you're flying blind in production — you can't reconstruct what happened before a failure.

Common mistake: Logging too much noise (every loop iteration) or too little (only errors). Both defeat the purpose.

Metrics (Important) — Counters, gauges, and histograms tell you the shape of your system’s behavior over time — request rates, latency percentiles, queue depths, error rates. Tools like Prometheus + Grafana or Datadog make this concrete. Logs tell you *what* happened; metrics tell you *how often* and *how slow*.

Distributed Tracing (Advanced) — When a request touches multiple services, tracing (OpenTelemetry, Jaeger) lets you follow it end-to-end and pinpoint exactly which component is slow or failing. Without it, debugging across service boundaries is guesswork.

Health Checks and Readiness Probes (Essential) — Every service should expose a `/health` or `/ready` endpoint. Orchestration systems need these to route traffic correctly. Without them, you may be serving traffic to a broken instance.

Testing

Unit Tests (Essential) — Test individual functions and classes in isolation. They run fast, catch regressions early, and force better design. Aim for meaningful coverage of business logic — 100% coverage is a vanity metric; covering critical paths is the goal.

Integration Tests (Essential) — Test that components work together correctly. Unit tests can pass while integration tests fail catastrophically.

End-to-End (E2E) Tests (Important) — Simulate real user workflows through the full stack. Slower and more brittle, but they catch failure modes nothing else will.

Contract Tests (Advanced) — If you own multiple services, contract tests (Pact) verify that a consumer and provider agree on an API’s shape without requiring both to be running simultaneously.

Test Pyramid Discipline (Important) — Many fast unit tests, fewer integration tests, very few E2E tests. Inverting this produces a fragile, slow test suite that developers start skipping.

Common mistake: Writing tests only after bugs appear, or only testing the happy path. Never mock what you don’t own — mocking third-party libraries leads to tests that pass but code that fails.

Architecture and Design

Separation of Concerns (Essential) — Business logic, data access, and transport should be in distinct layers. When these bleed together, changing anything requires understanding everything.

Dependency Inversion (Important) — Depend on abstractions, not concrete implementations. Your business logic shouldn’t know whether it’s talking to PostgreSQL or MongoDB — it should talk to a `UserRepository` interface.

Explicit Boundaries (*Important*) — Define clear interfaces between modules or services. Everything outside a module’s public interface should be private by default.

Avoiding Premature Complexity (*Essential*) — Microservices, event sourcing, CQRS, and hexagonal architecture are powerful — and frequently catastrophic when adopted too early. Choose the simplest architecture that solves today’s problem while keeping tomorrow’s options open.

Code Quality and Maintainability

Consistent Style and Formatting (*Essential*) — Use a linter and formatter enforced automatically. Consistency makes diffs cleaner and reviews faster.

Meaningful Naming (*Essential*) — Variable names like `d`, `tmp`, or `data` are a form of technical debt. Names should convey intent: `invoice_due_date_utc`, not `d`. This is the single highest-ROI maintainability practice.

Small, Focused Functions and Modules (*Important*) — Functions that do one thing are easier to test, understand, and reuse. A 500-line function is almost always a maintenance liability.

No Dead Code (*Important*) — Commented-out code, unused variables, deprecated paths never deleted — these create confusion about what actually runs. Delete aggressively; version control preserves history.

Dependency Management (*Essential*) — Pin dependency versions. Regularly audit and update them. Every library you add is a liability you inherit.

Common mistake: Treating `npm install` as free. Every dependency brings its own bugs, licenses, and security surface.

Documentation

README as the Front Door (*Essential*) — A new developer should be able to clone the repo and get it running from the README alone.

Architecture Decision Records (ADRs) (*Important*) — When you make a significant design decision, write it down. Future maintainers will otherwise spend hours reverse-engineering *why* the code looks the way it does.

Inline Comments for Why, Not What (*Important*) — Code explains *what* through good naming. Comments explain *why* — the non-obvious trade-off, the external constraint, the bug that forced this approach.

API Documentation (*Important*) — Any interface consumed by others needs documented inputs, outputs, error codes, and examples. OpenAPI/Swagger is the standard for HTTP APIs.

Security

Secrets Management (*Essential*) — No credentials, API keys, or passwords in source code. Ever. Use environment variables or a secrets manager. One leaked credential in a public repo can compromise an entire system.

Input Validation and Sanitization (*Essential*) — Never trust input from users, external APIs, or other internal services. Validate at system boundaries. This is the first defense against injection attacks and data corruption.

Principle of Least Privilege (*Important*) — Services, users, and processes should have only the permissions they need. Blast radius from a compromise is proportional to the permissions granted.

Dependency Vulnerability Scanning (*Important*) — Tools like Snyk, `npm audit`, or GitHub Dependabot scan your dependency tree for known CVEs. Passive, low-cost, and high-value.

Authentication and Authorization Hygiene (*Essential*) — Use proven libraries and protocols. Never roll your own cryptography. Separate authentication (who are you?) from authorization (what are you allowed to do?).

Common mistake: Implementing authorization checks inconsistently across endpoints.

Reliability and Error Handling

Explicit Error Handling (*Essential*) — Every error path should be intentional. Swallowing exceptions with empty catch blocks produces silent failures that are nearly impossible to diagnose.

Graceful Degradation (*Important*) — If a non-critical dependency fails, the core user experience should continue. Design for partial failure.

Idempotency (*Important*) — Operations that can be retried safely should produce the same result no matter how many times they're called.

Timeouts and Retries with Backoff (*Important*) — Every outbound call should have a timeout. Retries should use exponential backoff with jitter to avoid thundering herd problems.

CI/CD and Deployment

Continuous Integration (*Essential*) — Every push should automatically run linting, tests, and security scans. Broken builds should block merges.

Repeatable, Automated Deployment (*Essential*) — Deploying should be a single command or button push. Manual deployments are slow, error-prone, and undocumented.

Environment Parity (*Important*) — Dev, staging, and production environments should be

as similar as possible. Containerization (Docker) and IaC (Terraform) are the primary tools.

Feature Flags (*Advanced*) — Decouple deployment from release. Deploy code hidden behind a flag; roll back instantly by toggling it rather than reverting a deployment.

Rollback Strategy (*Important*) — Every deployment should have a defined rollback path. Blue-green deployments and canary releases make this safer.

Developer Experience

Fast Local Setup (*Essential*) — A new developer should be productive within hours, not days. Docker Compose for local dependencies, seed data scripts, and `.env.example` files all matter here.

Fast Feedback Loops (*Important*) — Tests that take 20 minutes to run will be skipped. Invest in parallelization, caching, and incremental builds.

Useful Pre-Commit Hooks (*Important*) — Run formatting, linting, and fast tests before a commit is made. Tools: husky (JS), pre-commit (Python).

Clear Contribution Guidelines (*Important*) — A `CONTRIBUTING.md` that explains branching strategy, PR conventions, commit message format, and review expectations.

Operational Excellence and Long-Term Scalability

Infrastructure as Code (*Important*) — Infrastructure should be defined in version-controlled code. Undocumented cloud resources are invisible technical debt.

Runbooks (*Important*) — For each common operational scenario, write a step-by-step guide for the on-call engineer. The act of writing them also surfaces gaps in your system design.

Capacity Planning Awareness (*Advanced*) — Understand the growth limits of your system: what breaks first if traffic doubles? You don't need to solve these problems in advance, but you should know where the cliff is.

Data Retention and Cleanup Policies (*Important*) — Define what gets deleted or archived, and when. Databases that grow without bound eventually cause performance degradation and unexpected costs.

Postmortems and Incident Reviews (*Important*) — When things go wrong, write a blameless postmortem. Document the timeline, root cause, contributing factors, and action items.

Priority Reference Matrix

Area	Essential	Important	Advanced
Observability	Logging, Health Checks	Metrics	Distributed Tracing
Testing	Unit, Integration	E2E, Test Pyramid	Contract Tests
Architecture	Separation of Concerns, Avoiding Premature Complexity	Dependency Inversion, Explicit Boundaries	—
Code Quality	Linting and Formatting, Naming, Dependency Management	Small Functions, No Dead Code	—
Documentation	README	ADRs, Inline Why, API Documentation	—
Security	Secrets, Input Validation, AuthN/AuthZ	Least Privilege, Vulnerability Scanning	—
Reliability	Error Handling	Graceful Degradation, Idempotency, Timeouts	—
CI/CD	CI Pipeline, Automated Deployment	Environment Parity, Rollback	Feature Flags
Developer Experience	Fast Setup	Fast Feedback, Pre-commit Hooks, Contribution Guidelines	—
Operations	—	IaC, Runbooks, Data Retention, Postmortems	Capacity Planning

Gaps and Domain-Specific Considerations

The main framework covers the vast majority of what matters in most codebases. The following areas are either domain-specific or become critical only at scale.

Data Integrity. Database migrations (versioned, reversible, never destructive), transaction boundaries, and consistency guarantees. These cause some of the worst production incidents and are underrepresented in typical quality checklists.

Versioning and Backward Compatibility. API versioning strategy and how you evolve schemas without breaking consumers. Critical if anything external depends on your interfaces.

Compliance and Auditability. Audit logs — recording who did what, when — are distinct from application logs. Relevant if your domain involves user data, finance, or regulated indus-

tries.

Performance Testing. Load testing and benchmarking as a sustained practice, not just functional correctness. A system can pass all tests and still fall over under real traffic.

Human and Team Factors. Code review culture, pair programming norms, and how knowledge is shared across a team. Tools don't help if the team practices aren't there.

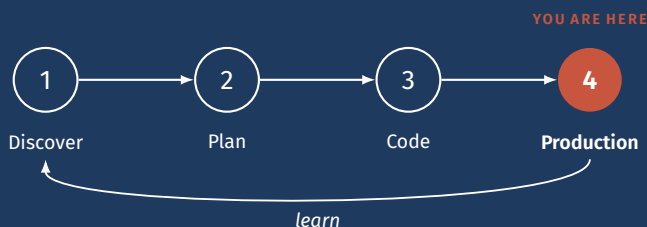
Disaster Recovery. Backup strategy, RTO/RPO definitions, and actually testing your ability to restore from backups. Most teams have backups; far fewer have *verified* backups.

IV

PART IV

Building Real Applications

From writing code to a running production system: one Notes API, followed from a command in a terminal to a monitored production service, and beyond.



Introduction to Part IV

About This Part

What This Part Is

This part exists to solve a specific problem.

When you start learning to program, you write code on your own machine. You save a file, run it, and watch it do something. For a while, that is enough. You learn how to write functions, how to work with data, how to build things that solve small problems. It feels good. It feels like you are making progress.

Then one day, someone asks: “Great — but how do we put it on a server?”

And you realize that you do not know.

Not because you are not capable. Not because the question is unanswerable. But because the path from “running on my machine” to “running in production” was never described to you in full. You have seen pieces of it — a tutorial about Flask here, a guide about Docker there, a blog post about deploying to a cloud service somewhere else. But the pieces do not connect. You do not understand how a program starts, how it listens for connections, how a request travels across the internet, how it reaches your code, or what your code actually does with it at runtime.

That gap — between writing code and running a real system — is what this part closes.

The Problem with Most Explanations

Most explanations of backend systems make one of two mistakes.

The first mistake is starting too late. They begin at deployment and work backwards. They show you how to set up a server, configure a reverse proxy, write a Dockerfile — but they skip over the question of what the program actually is before all of that infrastructure surrounds it. This produces a developer who can deploy things but does not understand what they are

deploying.

The second mistake is starting too early. They begin deep inside the application — routing, middleware, database connections — but skip over the question of how the program exists as a running process, how it acquires network capability, and what actually happens when a request arrives at the machine. This produces a developer who can write code but does not understand how that code connects to the outside world.

This part makes neither mistake. It starts at the very beginning — the command that starts your program — and follows every layer of the system outward, in the same order that a careful engineer would reason about it.

The Progression That Matters

The journey this part takes is not arbitrary. It mirrors the actual sequence in which a real system is built and understood.

First, you will understand what happens when you run a program. Not in a vague sense — concretely. What the operating system does, what memory gets allocated, what the Python interpreter does before your first line of code runs.

Second, you will understand how that program becomes capable of receiving network connections. How a socket is created, how a port is bound, how the kernel connects incoming traffic to your process.

Third, you will understand how request-handling behavior is defined. Where routes come from, how Flask knows which function to call, what middleware does and when it runs.

Fourth, you will confront the limitations of the local machine. What “works on my machine” actually hides. Why your program is fragile, environment-dependent, and unready for anyone else to run.

Fifth, you will learn to remove those limitations — to make the program portable, then packageable.

Sixth, you will place the program into a real environment: a server with an operating system, a public IP address, a firewall, and real constraints.

Seventh, you will make that program reachable from the outside world — through DNS, through a reverse proxy, through HTTPS.

Eighth, you will trace the complete journey of a real request: from a user’s browser, through DNS resolution, across the internet, into the server, through the kernel, into your application’s memory, through your code, and back out as a response.

Ninth, you will zoom into the internal execution of that request — the exact code path the program follows, the decisions it makes, the data it accesses, the response it constructs.

And finally, you will deal with production reality: the monitoring, scaling, reliability, and

security concerns that separate a program that merely runs from a system that actually operates.

That is the progression of this part. It is deliberate, and it is the right order.

Who This Part Is For

This part is for developers who are starting from scratch.

You do not need prior experience with servers, networking, or deployment. You need only to know that Python is a programming language and that a terminal is a place where you type commands. Everything else is explained from first principles.

That said, this part does not talk down to you. It explains beginner concepts with full precision and technical accuracy. The goal is not to simplify — it is to make complexity approachable by showing it in the right order and with the right examples. By the time you finish, you will understand things that many working developers find opaque.

If you have some Python experience already — you have written scripts, played with Flask, deployed a small project — you will find this part gives you the underlying mental model that experience alone does not provide. You will stop wondering why things work and start understanding how.

The Running Example: A Notes API

Throughout this part, a single application evolves from the very first chapter to the very last. It is called the **Notes API**.

The Notes API is a simple web service that allows users to create, list, read, update, and delete short text notes. It starts as an extremely small Flask application — barely twenty lines of code — and grows with each new concept introduced.

By the time you reach the final chapter, the Notes API will be:

- Running as a containerized process on a remote Linux server
- Accessible through a real domain name over HTTPS
- Restarted automatically if it crashes, and started again when the server reboots
- Sitting behind Nginx, which terminates TLS and forwards traffic
- Writing structured logs that can be searched and monitored
- Health-checked, rate-limited, and secured

Every piece of infrastructure introduced in this part is connected to the same application. Where a chapter needs an example the Notes API cannot provide — a native library, a message queue, a data warehouse — it says so. This means that when Chapter 26 explains how packets arrive at a network interface, you can trace that packet all the way to the Notes API code you

wrote in Chapter 4. The system is always whole. Only your understanding of it grows.

A Note on Python

Every code example in this part is written in Python. Python is used for two reasons.

First, it is approachable. The syntax does not get in the way of understanding the concepts. A beginner can read Python code and focus on what it is doing without fighting with types, brackets, or ceremony.

Second, it is real. Python powers a significant portion of the world's production web services. Everything this part shows you is the way things are actually done in professional engineering environments, not a simplified toy version designed only for teaching.

The primary web framework used is **Flask**. Flask is a small, explicit framework — it does only what you ask it to do, and it makes the underlying mechanics visible. This makes it ideal for a book whose purpose is to show how things work. Where the more modern **FastAPI** framework offers advantages (and it does), those alternatives are noted.

The target Python version is **3.11 or later**. All code targets this version.

How to Read This Part

The Recommended Path: Linear

If you are new to backend systems, read this part from beginning to end, in order.

This is not a reference manual. It is a progression. Each chapter assumes you have read the ones before it. Terms are defined once, at the moment they first become necessary, and then used freely afterward. Concepts are introduced at the layer of the system where they belong, not before.

If you skip chapters, you will encounter terms you do not understand and concepts that seem to arrive from nowhere. If you read in order, each chapter will feel like a natural next step, because it is.

The progression is structured so that by the time a new layer of the system is introduced, all the layers beneath it are already understood. Networking is introduced after you understand processes. Deployment is introduced after you understand packaging. Production concerns are introduced after you understand what the running system actually is. Nothing appears before it is needed.

How to Use the Code Examples

Code examples in this part are real programs, not pseudocode. Where a listing is only a sketch — a fragment of a larger file, or a hypothetical scenario — the text says so.

You will need:

- Python 3.11 or later installed on your machine
- `pip`, the Python package manager (included with Python)
- A terminal (on macOS or Linux: Terminal.app, iTerm2, or any terminal emulator; on Windows: PowerShell or Windows Terminal with WSL recommended)

Instructions for setting up your machine, including installing Python, creating virtual environments, and installing Flask, are in → **Appendix A — Python Environment Setup**.

Code examples are presented in blocks like this:

```
# A minimal Python program
print("Hello from the Notes API")
```

Every listing is numbered by chapter. “Listing 4.2” means the second code example in Chapter 4. When a later chapter refers back to an earlier listing, it will say so explicitly.

Conventions Used in This Part

The following typographic conventions are used throughout:

- `monospace text` indicates code, commands, file names, and terminal output.
- **Bold text** introduces a new term for the first time.
- *Italic text* is used for emphasis and for referring to named concepts.
- → Chapter N indicates a forward reference — something explained in a later chapter.
- ← Chapter N indicates a backward reference — something established in an earlier chapter.
- Boxes marked **Note**: contain supplementary information that is useful but not part of the main flow.
- Boxes marked **Warning**: flag something that is easy to get wrong and worth paying attention to.

A Note on Examples, Tools, and the Running Project

The Notes API

The application that runs through this entire book is a Notes API: a web service that lets users create, read, update, and delete short text notes. It is small by design. The point is not to impress you with the application — it is to keep the application simple enough that you can

focus entirely on the infrastructure and systems around it.

The Notes API begins in Chapter 1 as a raw Python program that prints a message. By Chapter 3 it can listen for network connections. By Chapter 4 it is a Flask application with real routes. By Chapter 6 it stores notes in a SQLite database. By Chapter 13 it uses PostgreSQL, driven by environment variables. By Chapter 18 it lives inside a Docker container. By Chapter 25 it is supervised and restarted automatically on a remote server. By Chapter 28 it is behind Nginx with HTTPS. By Chapter 35 it emits structured logs and metrics.

At each stage, only the concept being taught changes. The application itself remains recognizable. This is intentional: you should be able to look at the Chapter 35 version and trace every line of it back to the chapter where it was first introduced.

Tools Used

The following tools appear in this part. You do not need to install all of them before you start. Each is introduced at the chapter where it becomes relevant.

Tool	Version	First Used	Purpose
Python	3.11+	Chapter 1	Programming language and runtime
Flask	3.x	Chapter 4	Web framework
pip	Included with Python	Chapter 6	Package installer
venv	Included with Python	Chapter 6	Virtual environment manager
SQLite	Included with Python	Chapter 6	Local database
python-dotenv	Latest stable	Chapter 6	Loading <code>.env</code> files
Git	2.x+	Chapter 6	Version control
PostgreSQL	Latest stable	Chapter 13	Production-grade database
pytest	Latest stable	Chapter 17	Test runner
Docker	Latest stable	Chapter 18	Container runtime
Gunicorn	Latest stable	Chapter 18	Production WSGI server
GitHub Container Registry	—	Chapter 19	Container image registry
SSH	Included with macOS/Linux	Chapter 21	Remote server access
systemd	—	Chapter 24	Service manager (Linux)
Nginx	Latest stable	Chapter 28	Reverse proxy

Tool	Version	First Used	Purpose
Certbot	Latest stable	Chapter 28	TLS certificate management

Operating System Notes

This part uses Linux as the primary target for server-side examples. Linux is what the vast majority of production servers run. Command examples use the bash shell.

For local development, macOS and Linux behave nearly identically. Windows users can follow along by using **Windows Subsystem for Linux (WSL2)**, which provides a full Linux environment inside Windows. Where behavior differs meaningfully between operating systems, a note is included.

A Note on Simplification

Every technical book makes choices about what to include and what to defer. Where this part simplifies — for example, by using SQLite before PostgreSQL, or by deploying to a single VM before discussing Kubernetes — it does so consciously and says so. The goal is always to introduce the simplest version of something that is still real and accurate, then graduate to more production-appropriate versions as the book progresses.

Where something is simplified, the text says so, and a later chapter fills in what is missing.

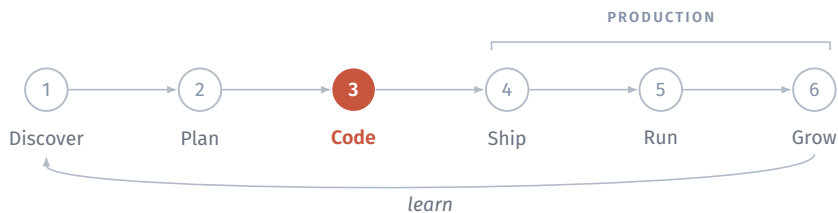
STAGE 1

1

Creating a Running Program

What is the system on the developer's machine?

YOU ARE HERE



The piece: The program, on your own machine

The whole map: *How Software Gets Built*, page 1

Command → Process

How Your Program Starts

When you open a terminal and type `python app.py`, something happens. The program runs. Output appears. Maybe a server starts listening. It all seems immediate and automatic — you type, things happen.

But a great deal happens between the moment you press Enter and the moment your Python code begins executing. The operating system receives your command, locates the Python interpreter, creates a new running process, allocates memory, opens file handles, and transfers control to Python — which then has its own startup work to do before a single line of your code is read.

Understanding this sequence is not an academic exercise. It is the foundation of everything that follows in this book. Every server, every deployment, every production failure ultimately traces back to the fact that software runs as a process on an operating system. If you do not understand what a process is, how it starts, and what resources it owns, then concepts like “the server is running” or “the process crashed” remain permanently vague.

This chapter traces exactly what happens from the moment you type `python app.py` to the moment the Python interpreter is ready to begin reading your code. Each step is concrete, ordered, and connected to the next.

In this chapter

By the end of this chapter, you will understand:

- What the shell is and how it interprets your command
- How the operating system finds the Python executable
- How a new process is created using `fork` and `exec`
- What a Process Control Block is and why the OS maintains one
- How memory is laid out when a program starts
- What file descriptors are and why `stdin`, `stdout`, and `stderr` exist

1.1 The Shell and What It Does with a Command

Before your Python program starts, something else runs first: the **shell**.

A **shell** is a program that reads text commands from a terminal and interprets them. When you open a terminal on your computer, you are interacting with a shell. On macOS and most Linux systems, the default shell is `bash` or `zsh`. On Windows with WSL, it is typically `bash`. The shell is itself a running program — a process that sits in a loop, waiting for you to type something, and then acting on what you type.

When you type:

```
python app.py
```

and press Enter, here is what the shell does:

Step 1: Read the line. The shell reads the string `python app.py` from the terminal.

Step 2: Split into tokens. The shell splits the line into tokens — distinct pieces separated by spaces. The first token is `python`. Everything after it is an argument. So the shell identifies:

- Command: `python`
- Arguments: `app.py`

Step 3: Check for built-in commands. The shell first checks whether `python` is a built-in shell command. Built-in commands (like `cd`, `echo`, and `exit`) are handled directly by the shell without starting a new process. `python` is not a built-in, so the shell moves on.

Step 4: Look up the executable. The shell needs to find the actual file on disk that the word `python` refers to. This is done by searching a list of directories called `$PATH`. More on this in Section 1.2.

Step 5: Create a new process. Once the shell finds the Python executable, it asks the operating system to start a new process running that executable. This is done using two system calls: `fork` and `exec`. More on these in Section 1.3.

Step 6: Wait (or not). By default, the shell waits for the new process to finish before showing you the prompt again. If you add `&` to the end of the command (`python app.py &`), the shell starts the process and returns immediately without waiting. When you start a Flask server, the shell appears to “hang” — that is actually it waiting for the server process to finish, which only happens when you press `Ctrl+C`.

This sequence is the same regardless of what program you are running. Whether you type `python app.py`, `ls`, `git status`, or `npm install`, the shell does the same thing: parse, locate, `fork`, `exec`, `wait`.

1.2 Executable Resolution via \$PATH

The shell now knows you want to run `python`. But where is the Python program stored on disk?

The answer is: somewhere in a directory listed in an environment variable called **\$PATH**.

What \$PATH is

\$PATH is a list of directory paths, separated by colons (:), that the shell searches when you type a command. It looks like this:

```
/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

This means: when you type a command, look for it first in `/usr/local/bin`, then in `/usr/bin`, then in `/bin`, and so on, stopping at the first match.

To see your current **\$PATH**, type this in your terminal:

```
echo $PATH
```

You will see something like:

```
/home/yourname/.local/bin:/usr/local/bin:/usr/bin:/bin
```

How the shell searches \$PATH

When you type `python`, the shell takes the word `python` and tries to find a file with that exact name in each directory listed in **\$PATH**, in order, left to right.

For each directory in **\$PATH**, it checks: “Does a file named `python` exist here, and is it executable?” If yes, that is the program to run. If no, move to the next directory.

You can ask the shell to show you which `python` it will find by using the `which` command:

Note: On current macOS, Debian, and Ubuntu systems the interpreter is installed as `python3`, and there is no plain `python` command until you create a virtual environment (→ [Chapter 6](#)). If `python` is not found on your machine, type `python3` wherever this book says `python` – for example, `python3 app.py`.

```
which python
```

Output:

```
/usr/bin/python3
```

or, if you are using a virtual environment (we cover this in → [Chapter 6 — Local Dependencies and Resources](#)):

```
/home/yourname/notes-api/venv/bin/python
```

This is a critical detail. When you install Python multiple times — once from your operating system’s package manager, once from python.org, once inside a virtual environment — your machine may have several Python executables at different paths. The one that runs when you type `python` is whichever one appears first in `$PATH`. This is the source of a class of bugs that we explore in → [Chapter 7 — Environment Drift](#) and → [Chapter 8 — Hidden Dependencies](#).

What happens if Python is not found

If the shell searches every directory in `$PATH` and finds no file named `python`, it gives you an error:

```
bash: python: command not found
```

This does not mean Python is not installed. It means Python is not installed anywhere that `$PATH` knows to look. This happens frequently on fresh Linux servers, where you may need to install Python explicitly or use `python3` instead of `python`. We will deal with this in → [Chapter 21 — System Setup](#).

The full path alternative

You can bypass `$PATH` entirely by typing the full path to the executable:

```
/usr/bin/python3 app.py
```

This tells the shell exactly where to find Python, without searching. This is how systemd service files (→ [Chapter 25 — Process Management](#)) specify which Python to use, because in a non-interactive system environment, `$PATH` may not be configured the way you expect.

1.3 How the Operating System Starts a New Program: `fork` and `exec`

The shell has found the Python executable. Now it needs to start a new process running that executable. This is done in two steps: **fork** and **exec**.

These are **system calls** — requests that a program makes to the operating system kernel. The kernel is the core of the operating system; it controls hardware, manages memory, and arbitrates access to resources. Programs do not directly access hardware or create processes — they ask the kernel to do it on their behalf through system calls.

What fork does

fork creates an exact copy of the current process.

When the shell calls fork, the operating system creates a new process that is an identical copy of the shell process. This new process — called the **child process** — gets its own:

- Memory space (initially a copy of the shell's memory)
- File descriptors
- Program counter (pointing to the same position in the shell's code)

The original shell is the **parent process**. After fork, there are now two processes: the parent (still the shell) and the child (a new copy of the shell).

Both processes continue executing from the same point in the code — right after the fork call returns. The only difference is the return value:

- In the parent process, fork returns the **process ID (PID)** of the child
- In the child process, fork returns **0**

This return value is how each process knows which role it is playing. The shell's code, simplified, looks like this:

```
# This is a conceptual illustration, not actual shell code.
# Real shells are written in C, not Python. This shows the logic.

pid = fork()

if pid == 0:
    # We are the child process
    # Replace ourselves with the Python executable
    exec("python", ["python", "app.py"])
else:
    # We are the parent (the shell)
    # Wait for the child to finish
    wait(pid)
```

What exec does

After `fork`, the child process is still running shell code. That is not what we want. We want it to run Python.

exec replaces the current process's program with a different program. When the child process calls `exec("python", ["python", "app.py"])`, the operating system:

1. Loads the Python executable from disk into memory, replacing the existing program code
2. Sets up the arguments and the environment it was handed (inside Python, `app.py` becomes `sys.argv[0]` — the name of the script being run)
3. Resets the program counter to the entry point of the Python executable
4. Begins executing Python

The child process is now a Python process. It has the same PID as before — only the program code changed. The shell, still running as the parent, waits for this Python process to exit.

Why fork + exec instead of just launching directly?

You might wonder: why not just launch a new process directly without copying first? The answer is setup and inheritance.

By forking first, the child process starts with a copy of everything the shell has, and much of it survives the `exec` that follows:

- Open file descriptors (which is how the child inherits `stdin`, `stdout`, and `stderr`)
- The current working directory
- The user and group it runs as

Environment variables work slightly differently. `exec` throws away the child's old memory, so the environment cannot simply “survive” it; instead, the shell passes its **exported** variables explicitly as an argument to the `execve()` system call, and the kernel places them in the new program's memory. That is how `$PATH`, `$HOME`, and any variable you `export` reach Python's `os.environ`, and why a variable you set without `export` does not.

The split between `fork` (copy the process) and `exec` (replace the program) is what lets the shell adjust the child in between — redirecting its output to a file, for example — before the new program starts.

Viewing the process in action

After you run `python app.py` in one terminal, open a second terminal and run:

```
ps aux | grep python
```

You will see something like this (columns trimmed; the full output also shows memory, terminal, and start time, and usually a line for the `grep` command itself):

```
yourname 12345 0.1 0.2 python app.py
```

The number 12345 is the **PID** (Process Identifier) of your Python process. Every running process on the system has a unique PID assigned by the kernel at the moment `fork` was called.

1.4 The Process Control Block: What the OS Remembers About Your Program

The operating system does not just start your process and forget about it. It maintains a detailed record of every running process. This record is called the **Process Control Block**, often abbreviated **PCB**.

The PCB is a data structure stored in kernel memory. It contains everything the operating system needs to manage, suspend, resume, and eventually destroy the process. Think of it as the OS's official file on your program.

The PCB contains, among other things:

Field	What it stores
PID	The unique process identifier
PPID	The PID of the parent process (the shell that started Python)
State	Whether the process is running, sleeping, waiting, or stopped
Program Counter	Where in the code the process is currently executing
CPU Registers	The current values of all CPU registers
Memory Map	The memory addresses allocated to this process
File Descriptor Table	Open files and network connections
Environment Variables	The variables inherited from the parent
User ID	Which user this process is running as
Priority / Scheduling Info	How much CPU time the OS gives this process
Signal Handlers	How the process responds to signals like Ctrl+C

Why the PCB matters

The PCB is what makes **multitasking** possible. Your computer runs many processes simultaneously — your terminal, your text editor, your browser, your Python server, and dozens of system processes. The CPU, however, can only execute one thing at a time (on a single core).

The operating system switches between processes rapidly — often thousands of times per second — giving each process a slice of CPU time. This is called **context switching**. When the OS switches away from your Python process to give CPU time to something else, it saves the CPU's state — the program counter and the register values — *into* the PCB, so it can resume exactly where it left off when Python gets CPU time again.

This is why a Flask server can handle multiple requests: not because it is doing two things at literally the same instant, but because the OS rapidly switches between them (and because, for I/O-bound tasks like waiting for a database query, the process can yield the CPU voluntarily).

Viewing the PCB in practice

You can see a simplified view of process information on any Linux or macOS system:

```
ps -p $(pgrep python) -o pid,ppid,stat,user,command
```

Output:

PID	PPID	STAT	USER	COMMAND
12345	9876	S	yourname	python app.py

- PID: the Python process
- PPID: the parent PID — this is the shell that started it
- STAT: S means sleeping (waiting for I/O or a timer), which is normal for a server waiting for connections
- USER: the user account this process belongs to

On Linux, you can see the full details of a process's state through the `/proc` filesystem:

```
cat /proc/12345/status
```

This prints dozens of fields from the PCB directly. It is how monitoring tools, debuggers, and system administrators inspect running processes.

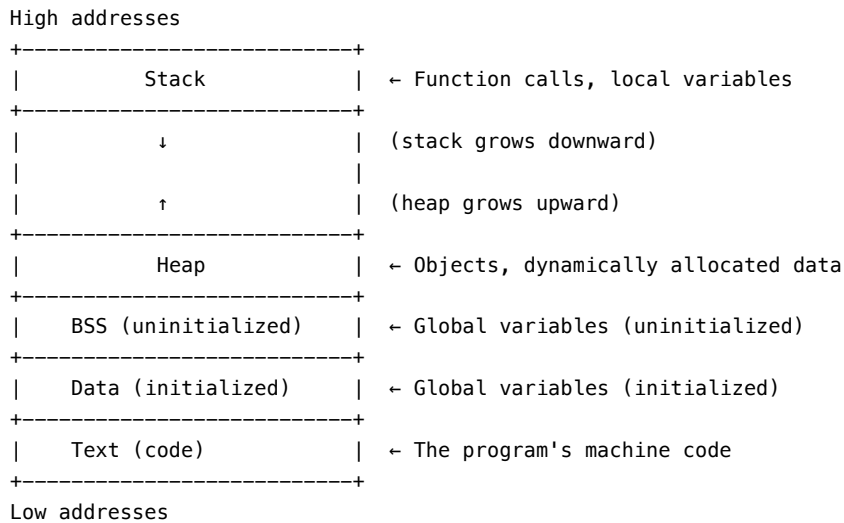
1.5 Memory Layout: Stack, Heap, and Why They Exist

When the operating system creates a process, it allocates a region of memory for that process to use. This memory is divided into distinct sections, each with a specific purpose. Understanding this layout matters because it explains how Python manages variables, function calls, and objects — and why programs can run out of memory in specific ways.

The virtual address space

Every process gets its own **virtual address space** — a private, isolated range of memory addresses that the process can use. The operating system and the CPU's memory management unit (MMU) ensure that one process cannot read or write another process's memory directly. This isolation is fundamental to security and stability: your Python program cannot accidentally (or deliberately) corrupt another process's data.

The virtual address space looks like this:



The text segment (code)

The **text segment** (also called the **code segment**) contains the actual executable machine code of the program. After `exec` loads Python from disk, the Python interpreter's machine code lives here. This segment is read-only — the program cannot modify its own code while running. (This prevents accidental or malicious self-modification.)

The data and BSS segments

These two segments hold the **global variables of the program's machine code** — for a Python process, that means the C globals of the interpreter itself.

- **Initialized data segment:** holds C global variables that have an initial value
- **BSS segment:** holds C global variables that have no initial value yet; the OS fills them with zeros

Your own Python module-level variables do *not* live here. A line such as `MAX_CONNECTIONS = 100` at the top of your module creates an integer object on the heap and stores a reference to it in the module's namespace (itself a dictionary on the heap). Everything your Python code creates is a heap object.

The heap

The **heap** is where **dynamically allocated objects** live. In Python, when you create any object — a string, a list, a dictionary, a Flask request object, a database connection — that object is allocated on the heap. The heap grows upward from low addresses as more objects are created.

Python manages heap allocation automatically. You do not manually allocate and free memory in Python (as you would in C or C++). CPython counts the references to every object and frees it the moment the count drops to zero; a separate **garbage collector** runs periodically to clean up groups of objects that refer only to each other (reference cycles).

This is why, when you load a large file into memory or create thousands of objects in a loop without releasing references, your program's memory usage grows: the heap is expanding.

The stack

The **stack** is where **function calls and local variables** live. It grows downward from high addresses.

Every time a function is called, the program pushes a **stack frame** onto the stack. A stack frame contains:

- The local variables of that function
- The function's parameters
- The return address (where execution should resume when the function returns)

When the function returns, its stack frame is popped off the stack and that memory is immediately available for reuse.

This is why local variables disappear when a function ends — they existed only in a stack frame that has since been removed.

CPython adds one twist: the frames of your *Python* functions (with their local variables) are interpreter objects, but every nested Python call still consumes space on the process's real C stack, because the interpreter calls itself to run it. That is what connects Python recursion to the machine stack.

The stack has a fixed maximum size (typically 8 MB on Linux). If a program makes too many nested function calls — a common bug with infinite recursion — the stack fills up and the program crashes with a **stack overflow**. Python protects you from this: it counts the depth of nested calls and raises a `RecursionError` *before* the real stack can overflow.

Listing 1.1 Infinite recursion hits Python's recursion limit

```
def count_down(n):
    # This function calls itself infinitely if you forget the base case
    print(n)
    count_down(n - 1) # No base case - recursion never stops

count_down(1000)
# Python will raise: RecursionError: maximum recursion depth exceeded
```

Python limits recursion depth to 1000 by default (you can check with `sys.getrecursionlimit()`). The limit exists specifically to turn a would-be stack overflow — which would kill the process — into an ordinary exception you can see and handle.

Memory and the Notes API

When you eventually start the Notes API server, here is what happens in memory:

- The Python interpreter's code sits in the text segment
- The Flask framework objects and route table sit on the heap, created during initialization
- When a request arrives, a stack frame is created for the handler function
- The request data (parsed JSON, headers, URL parameters) is allocated on the heap
- When the handler function returns, its stack frame is destroyed; heap objects it created are freed as soon as nothing references them any more

1.6 File Descriptors: stdin, stdout, and stderr

The process has been created. Memory has been allocated. There is one more set of resources the OS sets up automatically before your program starts: **file descriptors**.

What a file descriptor is

A **file descriptor** is a small integer that represents an open connection to an I/O resource. I/O resources include:

- Files on disk
- Network sockets (connections to other computers)
- Pipes (channels between processes)
- Terminals (the screen and keyboard)

When a process wants to read from or write to any of these resources, it refers to them by file descriptor number, not by name. The operating system maintains a **file descriptor table** for each process — a list that maps descriptor numbers to actual I/O resources.

When you open a file in Python:

Listing 1.2 Opening a file and observing its file descriptor

```
f = open("notes.txt", "w")
print(f.fileno()) # Prints the file descriptor number, e.g., 3
```

The `fileno()` method shows you the underlying file descriptor number. File descriptor 3 means: “the kernel’s record of this open connection is at index 3 in this process’s descriptor table.”

The three standard file descriptors

Every new process starts with three file descriptors already open, inherited from the parent (the shell):

Number	Name	Direction	Default connection
0	stdin	Input	Keyboard (terminal)
1	stdout	Output	Screen (terminal)
2	stderr	Output (errors)	Screen (terminal)

These are called the **standard streams**. They exist by convention — every Unix-like operating system guarantees that a new process will have these three descriptors ready to use.

stdin (file descriptor 0) is where input comes from. When you use Python’s `input()` function, it reads from stdin. When you pipe data into a program (`echo "hello" | python app.py`), that data arrives through stdin.

stdout (file descriptor 1) is where normal output goes. When you call `print("hello")` in Python, Python writes the string to file descriptor 1, which is connected to the terminal. The text appears on your screen.

stderr (file descriptor 2) is where error output goes. Python writes tracebacks and interpreter errors to stderr, not stdout. This distinction matters: stdout is for data, stderr is for diagnostics. In production, these are typically written to separate log streams so that error messages do not pollute the data output. When Flask logs a request or an error, it writes to stderr by default.

Why the distinction between stdout and stderr matters

Consider this scenario: your program is writing JSON data to stdout, and you want to capture that data with shell redirection:

```
python app.py > output.json
```

This redirects stdout to a file. If errors were mixed into stdout, your JSON file would be corrupted with error messages. Because errors go to stderr, the redirection above captures only the clean JSON output. Error messages still appear on the terminal because stderr is untouched.

You can see this behavior explicitly:

Listing 1.3 Writing to stdout and stderr explicitly

```
import sys

print("This is normal output")           # Goes to stdout (fd 1)
print("This is an error", file=sys.stderr) # Goes to stderr (fd 2)
```

Run this, then run it with redirection:

```
python listing_1_3.py > output.txt
# "This is an error" still appears on the terminal
# output.txt contains only "This is normal output"
```

File descriptors for network connections

When your Flask server later accepts a network connection from a client (→ [Chapter 3 — Creating Network Capability](#), → [Chapter 5 — Internal Request Execution](#)), that connection will also be represented as a file descriptor. The kernel assigns the next available number (typically 4 or 5, since 0, 1, 2 are taken, and 3 might be used by Python internally).

This is the elegant unification that Unix provides: files, terminals, and network sockets all look the same to a program — they are all just integers that you can `read()` and `write()`. Flask’s ability to receive an HTTP request and read its contents uses the same underlying mechanism as reading a file.

Listing 1.4 Inspecting the current process's open file descriptors

```

import os

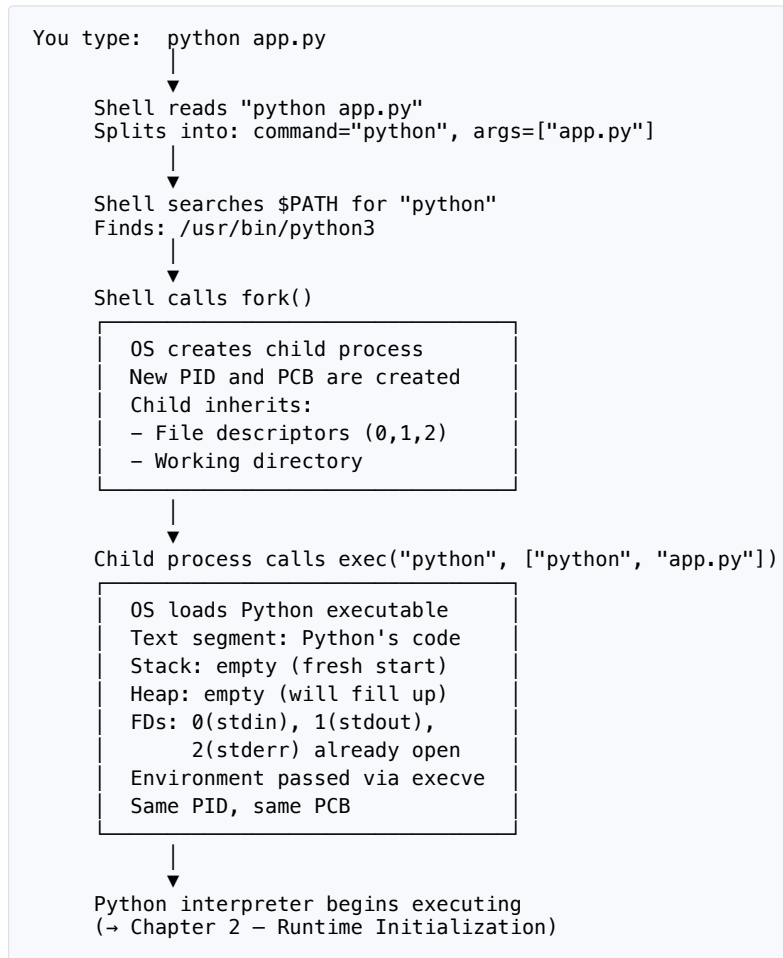
# List all open file descriptors for this process
fd_dir = f"/proc/{os.getpid()}/fd"
try:
    for fd in os.listdir(fd_dir):
        try:
            target = os.readlink(f"{fd_dir}/{fd}")
            print(f"fd {fd} -> {target}")
        except OSError:
            pass
except FileNotFoundError:
    # /proc is Linux-specific; this won't work on macOS
    print("Run this on Linux to see file descriptors")
    print(f"Current process PID: {os.getpid()}")

```

On Linux, this will show you stdin, stdout, and stderr mapped to /dev/pts/0 (the terminal), and possibly additional file descriptors used by Python internally.

Putting It Together: The Complete Startup Sequence

Let us trace the full sequence from command to running process one more time, now with all the pieces connected:



This sequence — shell parse, `PATH` lookup, fork (new PID and PCB), `exec`, memory layout, file descriptor inheritance — is the foundation. Every Flask server, every containerized application, every systemd-managed service begins with exactly these steps.

Key Takeaways

- The **shell** is a program that reads commands from the terminal, locates the corresponding executable, and asks the OS to run it.
- The **`$PATH`** variable is a list of directories the shell searches when you type a command. The first match wins. Multiple Python installations can coexist; which one runs depends entirely on `$PATH`.

- **fork** creates an exact copy of the current process. **exec** replaces the child process's code with the program you actually want to run. Together, they start every new process on a Unix-like system.
- The **Process Control Block (PCB)** is the OS's record for each running process. It stores the PID, memory map, file descriptors, CPU state, and everything else needed to manage the process.
- A process's **memory** is divided into the text segment (code), data segments (the program's C globals), heap (dynamic objects — which includes every Python object, even module-level variables), and stack (function calls). The heap grows when you create objects; the stack grows when you call functions.
- **File descriptors** are integers that represent open I/O connections. Every process starts with three: `stdin` (0), `stdout` (1), and `stderr` (2). Network connections are also file descriptors — sockets are treated the same as files.

Exercises

1. **Predict.** Run `which -a python3` and write down which executable a plain `python3` will start. Now create a new directory containing an executable script called `python3` that prints `fake` (remember `chmod +x`), and put that directory first in your `PATH`. What does `python3 --version` print now, and why?
2. **Break and fix.** Start a long-running server, `python3 -m http.server 8000`, in one terminal, find its PID with `ps`, and kill it with `kill -9`. What exit status does the shell report? Then start it again and stop it with `kill (SIGTERM)` instead. What is the difference?
3. **Extend.** Write a ten-line Python program that prints its own PID, its parent's PID and its open file descriptors (`os.listdir('/proc/self/fd')` on Linux, or `lsof -p` on macOS). Run it from the shell, then from inside another Python program with `subprocess.run`. Which numbers change?
4. **Explain.** In two or three sentences, explain to a colleague why a network connection and a file are both “file descriptors”, and what that makes possible.

Solutions and hints are in the companion repository, in `exercises/chapter-01.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

The operating system has created the Python process and set up its memory and file descriptors. Now Python itself has work to do. Before any of your code runs, the interpreter must

start up, load your source file, compile it to bytecode, import your dependencies, and initialize the Flask framework.

→ [Chapter 2](#) — Runtime Initialization: Python Wakes Up

Runtime Initialization

Python Wakes Up

In the previous chapter, the operating system did its work: it parsed the command, located the Python executable, created a new process through `fork` and `exec`, allocated memory, and set up file descriptors. The process exists. It has a PID. It has memory. It has `stdin`, `stdout`, and `stderr` open.

But the Python interpreter itself has not done anything yet.

`exec` placed the Python interpreter's machine code into the process's text segment and jumped to its entry point — the first instruction of the Python interpreter. From that moment, Python begins its own initialization sequence. This sequence happens entirely before your first line of `app.py` is read.

This matters because a great deal of the system's behavior is determined during this initialization phase. The import system is set up here. The standard library becomes available here. Flask's routing table — the data structure that connects URL paths to handler functions — is populated here, before any request ever arrives. If anything goes wrong during initialization (a missing dependency, an import error, a bad configuration value), the program fails here, at startup, before it ever serves a single request.

Understanding initialization also explains why production servers are often structured the way they are. Web servers like Gunicorn (which we introduce in → [Chapter 24](#) — Process Startup in Production) start multiple worker processes, and each worker imports and initializes your application. The cost of that initialization is paid once per worker, not once per request. This design is efficient only if you understand what initialization actually does.

In this chapter

By the end of this chapter, you will understand:

- What the Python interpreter is and what it does before running your code
- How Python loads your source file into memory
- What bytecode is and why Python compiles your code before running it
- What the `import` statement actually does at the machine level
- How Flask initializes itself and builds its route table
- What global state is and what it means for your application

2.1 What the Python Interpreter Is and Why It Exists

When you write `print("Hello")` in Python and run it, you are not running a program that the CPU can execute directly. The CPU executes machine code — sequences of binary instructions specific to the CPU’s architecture (x86, ARM, etc.). Python source code is not machine code. It is high-level text that humans can read.

The **Python interpreter** is the program that bridges the gap. It reads your Python source code, translates it into a compact intermediate form, and then carries out that form step by step on your behalf.

More precisely, Python uses a **virtual machine** approach:

1. Python source code is first compiled into an intermediate representation called **bytecode**
2. The bytecode is executed by the **CPython virtual machine** (or CPython VM), which is written in C
3. The CPython VM is a loop of compiled C code: it reads each bytecode instruction and runs the piece of C code that implements it. Your program is never translated into machine code of its own (CPython 3.13 added an experimental JIT compiler, off by default)

This design provides portability: the same Python source code runs on any machine that has a Python interpreter installed, regardless of whether that machine uses x86, ARM, or another CPU architecture. Only the interpreter itself is compiled for a specific CPU; the bytecode it executes is the same on every machine running the same Python version.

CPython is the name for the reference implementation of Python — the one you download from python.org and the one used in this book. When we say “the Python interpreter,” we mean CPython.

The interpreter's job in brief

The Python interpreter's initialization sequence, before your code runs, involves:

1. Setting up the interpreter's own internal state
2. Initializing the memory allocator and garbage collector
3. Setting up the module system (so `import` works)
4. Importing a handful of core modules (such as `encodings` and `site`); the rest of the standard library is imported only when your code asks for it
5. Processing command-line arguments
6. Loading and compiling your source file
7. Executing your code top to bottom

Steps 1 through 5 happen invisibly every time you run any Python program. They are fast but not instantaneous. On a modern machine, a `python -c "pass"` (which runs an empty program) still takes 30–100 milliseconds. That time is the interpreter's startup cost — unavoidable, and something to be aware of when designing systems that restart frequently.

2.2 Loading Source Code into Memory

After the interpreter initializes its own internal state, it turns its attention to your file: `app.py`.

The interpreter asks the operating system to open `app.py` as a file. The OS performs a file system lookup — it traverses the directory tree, finds `app.py` in the current working directory, and opens it, returning a file descriptor. Python then reads the raw bytes of the file through that file descriptor.

At this point, Python has the contents of `app.py` in memory as a sequence of raw bytes. It decodes them as UTF-8 (the default encoding for Python source files) into text — characters that are not yet understood as a program.

The current working directory

There is a subtlety here worth noting. Python opens `app.py` relative to the **current working directory (CWD)** — the directory your terminal was in when you typed `python app.py`. This is inherited from the shell through `fork` (← [Section 1.3](#)).

If you type:

```
cd /home/yourname/notes-api
python app.py
```

Then Python looks for `app.py` in `/home/yourname/notes-api`. If you type:

```
python /home/yourname/notes-api/app.py
```

from a different directory, Python still runs `app.py`, but the CWD is whatever directory you were in. This distinction affects how your code handles relative file paths — for example, `open("notes.db")` in your code will look for `notes.db` in the CWD, which may or may not be where you expect it.

This is a source of bugs that we examine more carefully in → [Chapter 7 — Environment Drift](#) and → [Chapter 15 — Structuring the Application](#).

Parsing: from text to a syntax tree

After reading the file’s bytes, Python **parses** the source code. Parsing means reading the text and building an internal tree structure that represents the program’s grammar.

This tree is called the **Abstract Syntax Tree**, or AST. The AST is not something you normally interact with directly, but it is the form in which Python’s compiler understands your program. Every `if`, `for`, `def`, `class`, `import`, and expression becomes a node in this tree.

You can actually inspect the AST of any Python code using the built-in `ast` module:

Listing 2.1 Inspecting the Abstract Syntax Tree of a simple program

```
import ast

source_code = """
x = 10
print(x)
"""

tree = ast.parse(source_code)
print(ast.dump(tree, indent=2))
```

Running this prints the tree structure Python built from those two lines of code. It is verbose, but it shows you exactly how Python understood the program — which nodes are assignments, which are function calls, which are names.

If your source code has a **syntax error** — for example, a missing colon after a `def` statement — parsing fails here, during this phase, before any code is executed. This is why syntax errors look different from runtime errors:

Listing 2.2 A syntax error is caught during parsing, not during execution

```
def greet(name)    # Missing colon
    print(name)
```

```
File "app.py", line 1
    def greet(name)
        ^
SyntaxError: expected ':'
```

The error is found and reported before the program runs a single instruction.

2.3 Compilation to Bytecode: What .pyc Files Really Are

After parsing, Python **compiles** the AST into **bytecode**. Bytecode is a compact, lower-level set of instructions designed for the CPython virtual machine. It is not machine code — the CPU cannot execute bytecode directly. But it is much faster for the Python VM to interpret than raw source code text.

What bytecode looks like

You can inspect the bytecode of any Python function using the built-in `dis` module (short for “disassembler”):

Listing 2.3 Inspecting bytecode with the `dis` module

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

Output:

3	0 RESUME	0
4	2 LOAD_FAST	0 (a)
	4 LOAD_FAST	1 (b)
	6 BINARY_OP	0 (+)
	10 RETURN_VALUE	

Each line is one bytecode instruction:

- `LOAD_FAST 0` means “push the local variable at index 0 (which is `a`) onto the evaluation stack”
- `LOAD_FAST 1` means “push the local variable at index 1 (which is `b`) onto the evaluation stack”
- `BINARY_OP (+)` means “pop the top two items from the stack, add them, push the result”
- `RETURN_VALUE` means “pop the top of the stack and return it as this function’s result”

The CPython virtual machine executes these instructions one at a time, in order. The CPU runs C code that implements each instruction. Your Python code never runs directly on the CPU — it is always mediated by the interpreter.

The .pyc cache

Parsing and compilation take time. If Python had to parse and compile every file every time it was imported, startup would be slow for programs with many files.

To avoid this, Python **caches** the bytecode of every *imported* module in a `.pyc` file, stored in a `__pycache__` directory next to the source file. (The script you run directly — `app.py` in `python app.py` — is compiled fresh every time and never cached; only the modules it imports are.)

```
notes-api/
├── app.py
├── notes_store.py
├── __pycache__/
│   └── notes_store.cpython-311.pyc  ← cached bytecode for Python 3.11
```

When you import a module, Python checks:

1. Does a `.pyc` file exist for this source file?
2. Does the `.pyc` file record the same modification time and size as the current source file (or, optionally, the same hash of its contents)?
3. Was it compiled by the same Python version?

If all three are yes, Python skips parsing and compilation and loads the cached bytecode directly. If any check fails, Python recompiles the source file and updates the `.pyc` cache.

This is why you sometimes see a `__pycache__` directory appear after running a Python program. It is not a problem — it is a performance optimization. You can safely add `__pycache__/` to your `.gitignore` file; it will be regenerated automatically.

Note: The `.pyc` files are version-specific. A `.pyc` file compiled by Python 3.11 cannot be used by Python 3.9. The filename includes the Python version (`cpython-311`) to prevent cross-version confusion.

2.4 Importing Dependencies: What `import` Actually Does

After compiling your source file to bytecode, Python begins executing it from the top. One of the first things a typical Flask application does is import its dependencies:

Listing 2.4 The imports at the top of a typical Flask app

```
from flask import Flask
import sqlite3
import os
```

To most beginners, `import` looks like a simple instruction — “make this module available.” In reality, `import` triggers a significant chain of work.

The module system

Python’s **module system** is the mechanism that finds, loads, and caches code from other files and packages. Every `import` statement goes through this system.

When Python encounters `from flask import Flask`, it does the following:

Step 1: Check `sys.modules` (the import cache). `sys.modules` is a dictionary that maps module names to already-loaded module objects. If `flask` is already there, Python uses the cached version and does not re-read any files. This is why importing the same module in multiple files is cheap — it is loaded once and cached.

Listing 2.5 Inspecting the module cache after importing Flask

```
import sys
import flask

print("flask" in sys.modules)      # True
print(type(sys.modules["flask"]))  # <class 'module'>
```

Step 2: Find the module. If `flask` is not in `sys.modules`, Python searches for it. The search path is stored in `sys.path` — a list of directories Python checks in order. It looks like this:

Listing 2.6 Inspecting the module search path

```
import sys
for path in sys.path:
    print(path)
```

Output (varies by installation):

```
/home/yourname/notes-api
/usr/lib/python311.zip
/usr/lib/python3.11
/usr/lib/python3.11/lib-dynload
/home/yourname/notes-api/venv/lib/python3.11/site-packages
```

Python checks each directory in order:

- The first entry is usually the directory containing your script (or an empty string meaning the current directory)
- The standard library directories come next, so installed packages cannot accidentally shadow standard modules
- The `site-packages` directory comes last — this is where `pip install` puts packages. When a virtual environment is active, it is the environment’s own `site-packages` (→ [Chapter 6](#) — Local Dependencies and Resources)

Step 3: Load the module. Once found, Python reads and executes the module’s source file (or loads its `.pyc` cache). “Executes” is the key word here — when Python imports Flask, it literally runs Flask’s `__init__.py` file from top to bottom. Any code at the module level runs immediately.

Step 4: Cache the result. The loaded module object is stored in `sys.modules` under the module’s name. Future `import flask` statements in any file will use this cached object.

Step 5: Bind the name. `from flask import Flask` extracts the `Flask` attribute from the loaded module object and binds it to the name `Flask` in the current module’s namespace. After this, `Flask` refers to the `Flask` class defined inside the Flask library.

Import errors

If Python cannot find a module anywhere in `sys.path`, you get an error:

```
ModuleNotFoundError: No module named 'flask'
```

This means Flask is not installed in the Python environment currently active. It does not mean Flask does not exist. It means `pip install flask` has not been run in the right environment. We cover this in detail in → [Chapter 6 — Local Dependencies and Resources](#) and the solution to this class of problems in → [Chapter 12 — Dependency Declaration](#).

What happens during Flask's import

When Python imports Flask, Flask's code runs. Specifically, Flask's `__init__.py` and several supporting modules execute. During this execution, Flask:

- Defines the `Flask` class
- Defines the `Blueprint` class
- Sets up the Jinja2 templating system
- Sets up the Werkzeug HTTP library (which Flask uses internally for request/response handling)
- Registers default error handlers
- Sets up signal support (for request started/ended events)

None of this is your code — it is Flask setting up its own internals. But it runs during your application's startup, and any error here (for example, if Werkzeug is not installed) will prevent your application from starting.

2.5 Framework Initialization: How Flask Starts Up

After imports, your `app.py` continues executing. The next significant step is creating the Flask application object:

Listing 2.7 Creating the Flask application object

```
from flask import Flask

app = Flask(__name__)
```

This single line does a surprising amount of work.

What `Flask(__name__)` does

`Flask(__name__)` calls the `Flask` class constructor with one argument: `__name__`.

`__name__` is a special Python variable that contains the name of the current module. When the file being run directly is `app.py`, `__name__` has the value `"__main__"`. When `app.py` is

imported by another module, `__name__` has the value `"app"`. Flask uses `__name__` to:

- Locate the root directory of your application (for finding templates and static files)
- Set up proper logging with the correct module name

Inside `Flask.__init__()`, Flask does the following:

1. **Records the application root path** — the directory where `__name__`'s file lives. This becomes the base directory for finding templates, static files, and instance folders.
2. **Creates the route table** — an empty data structure (a `Map` object from Werkzeug) that will store URL patterns and their corresponding handler functions. At this point, the table is empty.
3. **Creates the error handler registry** — a dictionary mapping error codes (like 404, 500) to handler functions. Flask starts with empty registries; your code will add to them.
4. **Creates the before/after request handler lists** — lists of functions to be called before and after every request. These are empty initially.
5. **Sets up configuration** — a `Config` object with Flask's defaults (debug mode off, testing mode off, etc.).
6. **Prepares the logger** — `app.logger`, a Python `logging.Logger` for this application, is created the first time your code uses it.

After `Flask(__name__)` returns, you have an application object (`app`) that is a fully initialized Flask application with no routes and no handlers.

Routes are registered next

After the `app` object is created, your code registers routes using decorators:

Listing 2.8 Registering routes with Flask decorators

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def index():
    return jsonify({"message": "Notes API is running"})

@app.route("/notes", methods=["GET"])
def list_notes():
    return jsonify({"notes": []})
```


When Python executes the `@app.route("/")` decorator on the `index` function, Flask calls `app.add_url_rule("/", "index", index)`. This adds an entry to the route table mapping the URL pattern `"/"` and the HTTP method `GET` to the `index` function.

This happens at **import time** — the moment Python reads those lines from top to bottom during initialization, not when a request arrives. By the time Python finishes executing `app.py`, Flask's route table is fully populated and ready to handle incoming requests.

This is a crucial distinction: the route table is built once, during startup, and then consulted on every request. Building it is not cheap — it involves compiling URL patterns into a matcher structure — but it is done once. Lookups are fast.

We examine the route table in detail in → [Chapter 4 — Defining Request Handling Paths](#) and → [Chapter 32 — Routing](#).

2.6 Global State Setup: What Happens Before the First Request

As Python executes your `app.py` from top to bottom, it may encounter other forms of global initialization beyond route registration.

Module-level code

Any code at the module level — code that is not inside a function or class — runs during initialization. This includes:

Listing 2.9 Module-level code that runs at startup

```
from flask import Flask
import sqlite3
import os

app = Flask(__name__)

# This runs at startup, not per-request
DATABASE_PATH = os.getenv("DATABASE_PATH", "notes.db")
print(f"Database path: {DATABASE_PATH}")

# This function definition runs at startup (registers the route)
@app.route("/notes", methods=["GET"])
def list_notes():
    # This code runs only when a request arrives
    conn = sqlite3.connect(DATABASE_PATH)
    ...
```

In Listing 2.9:

- `DATABASE_PATH = os.getenv(...)` runs once, at startup. The environment variable is read once.
- `print(f"Database path: ...")` runs once, at startup. The output appears in the terminal when the server starts.
- The `list_notes` function body does NOT run at startup. It runs only when a request arrives at `/notes`.

This distinction matters for performance and correctness. If you put database connection setup at the module level, the connection is made once at startup. If you put it inside the handler function, a new connection is made on every request. Both patterns have valid use cases, but you must know which one you have chosen.

Database initialization at startup

A common pattern is to initialize the database schema at startup:

Listing 2.10 Early version of the Notes API with startup initialization

```
from flask import Flask, jsonify, request
import sqlite3
import os

app = Flask(__name__)
DATABASE_PATH = os.getenv("DATABASE_PATH", "notes.db")

def init_db():
    """Create the notes table if it does not already exist."""
    conn = sqlite3.connect(DATABASE_PATH)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            content TEXT NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    conn.close()

# This runs at startup, before any request is handled
init_db()
```

```

@app.route("/notes", methods=["GET"])
def list_notes():
    conn = sqlite3.connect(DATABASE_PATH)
    cursor = conn.execute("SELECT id, content, created_at FROM notes")
    notes = [
        {"id": row[0], "content": row[1], "created_at": row[2]}
        for row in cursor.fetchall()
    ]
    conn.close()
    return jsonify({"notes": notes})

@app.route("/notes", methods=["POST"])
def create_note():
    data = request.get_json()
    content = data.get("content", "")
    conn = sqlite3.connect(DATABASE_PATH)
    cursor = conn.execute(
        "INSERT INTO notes (content) VALUES (?)", (content,)
    )
    conn.commit()
    note_id = cursor.lastrowid
    conn.close()
    return jsonify({"id": note_id, "content": content}), 201

if __name__ == "__main__":
    app.run(debug=True)

```

This is the first real version of the Notes API. It is small — about 40 lines — but it is a complete, working web service. It does not validate its input yet: a request without a JSON body, or without content, is not handled gracefully. Chapters 4, 5, and 31 add that. The database is initialized when the program starts (`init_db()` runs at the module level). The routes are registered as Python reads through the decorators. By the time `app.run()` is called, the server is fully configured and ready to accept connections.

The `if __name__ == "__main__":` guard

The final two lines of Listing 2.10 use a pattern you will see in almost every Python program:

```

if __name__ == "__main__":
    app.run(debug=True)

```

This condition is `True` only when the file is run directly (`python app.py`). It is `False` when

the file is imported by another module.

Why does this matter? When you use a production server like Gunicorn (→ [Chapter 24](#)), Gunicorn imports your `app.py` file as a module rather than running it directly. It does this to access the `app` object without starting Flask’s built-in development server. The `if __name__ == "__main__":` guard ensures that `app.run()` is called only when you run the file directly — not when it is imported by Gunicorn.

Without this guard, importing `app.py` would immediately start a server, which is not what production deployment tools expect.

What `app.run(debug=True)` does

When called, `app.run()`:

1. Creates a socket and binds it to `localhost:5000` (by default)
2. Starts listening for connections
3. Enters the request-handling loop

In debug mode (`debug=True`), Flask also:

- Enables the interactive debugger (shows a detailed error page in the browser when exceptions occur)
- Enables the automatic reloader (restarts the server when you save changes to source files)

Warning: `debug=True` must never be used in production. The interactive debugger allows arbitrary Python code execution from the browser — a critical security vulnerability. We address this in → [Chapter 9](#) — Configuration Coupling and → [Chapter 38](#) — Security.

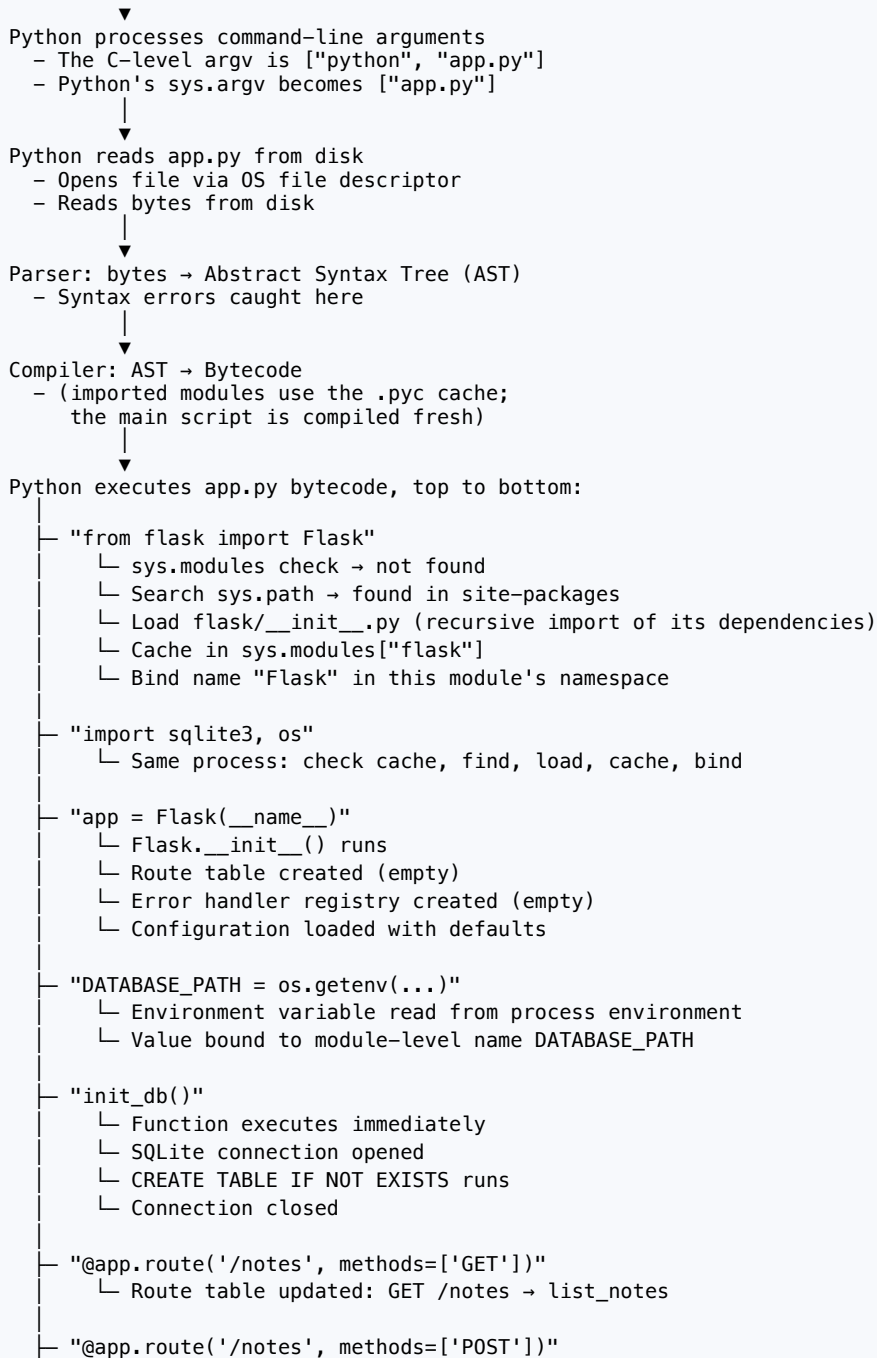
The Complete Initialization Timeline

Here is the full sequence from `exec` to “ready to accept connections,” with everything from this chapter included:

```

exec() loads Python interpreter
    ↓
Python interpreter self-initialization
  - Memory allocator setup
  - Garbage collector setup
  - Built-in types registered (int, str, list, dict...)
  - sys, builtins modules loaded
    |

```



```

|   └ Route table updated: POST /notes → create_note
└ "if __name__ == '__main__': app.run(debug=True)"
    └ Creates socket
      └ Binds to localhost:5000
        └ Begins listening for connections
          └ (→ Chapter 3 – Creating Network Capability)

```

Key Takeaways

- The **Python interpreter** (CPython) is the program that runs your Python code. It compiles source code to bytecode and then executes the bytecode on the CPython virtual machine.
- Python **reads and parses** your source file before executing it. Syntax errors are detected during parsing, before any code runs.
- Python **compiles** your code to bytecode and caches the bytecode of imported modules in `.pyc` files inside `__pycache__`/. This speeds up subsequent runs.
- The **import statement** searches `sys.path` for the module, loads and executes it, caches the result in `sys.modules`, and binds the requested names in the current namespace.
- **Flask initialization** — creating the Flask object and registering routes with decorators — happens during startup, not per request. The route table is fully built before the first connection is accepted.
- **Module-level code** (code not inside a function) runs during startup. This includes `init_db()`, reading environment variables, and registering routes.
- The `if __name__ == "__main__":` guard ensures that `app.run()` is only called when the script is run directly, not when it is imported by a production server like Gunicorn.
- `debug=True` enables the interactive debugger and auto-reloader. It must **never** be used in production.

Exercises

1. **Predict.** Add `print("module loaded")` at the top level of `app.py`, and start the application both with `python app.py` and with a WSGI server (`pip install gunicorn`, then `gunicorn -w 2 app:app`). How many times is the line printed in each case, and why?
2. **Break and fix.** Introduce a syntax error on the last line of `app.py` and run it. Does any line before it execute? Then instead make the last line raise a `NameError` (a misspelled variable at module level). What runs this time?
3. **Extend.** Use `python -X importtime -c "import flask"` to find the three slowest imports Flask triggers. What does this tell you about start-up time?

4. **Explain.** Explain why `if __name__ == "__main__":` matters for a program that a production server will import rather than run.

Solutions and hints are in the companion repository, in `exercises/chapter-02.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

Python has initialized. Flask has built its route table. The application is calling `app.run()`. The next step is everything that happens inside `app.run()` at the network level: creating a socket, binding it to a port, and entering the listening state.

→ [Chapter 3](#) — Creating Network Capability

Creating Network Capability

At the end of [Chapter 2](#), the Notes API had finished its initialization sequence. Flask’s route table was populated. The database schema was in place. The application object was ready. The last line of code that ran was `app.run(debug=True)`.

That call — `app.run()` — is where the application stops being a batch program and becomes a server.

A **server** is a program that waits for connections. Unlike a script that runs from top to bottom and exits, a server sits in a loop, continuously accepting incoming connections, handling them, and waiting for the next one. It does not exit until you tell it to.

To become a server, a program must acquire **network capability** — the ability to accept connections from other programs, whether those other programs are a browser on your laptop, a mobile app on the other side of the world, or an automated test suite running on a CI server.

Network capability does not come automatically. A Python program with no networking code is completely invisible to the network. It cannot receive requests, cannot send responses, and cannot be reached by any other machine or program. Network capability must be explicitly created, step by step, through a series of system calls that configure the operating system’s networking subsystem.

Those steps — creating a socket, binding it to an address, entering the listening state, and waiting for connections — are what this chapter explains.

In this chapter

By the end of this chapter, you will understand:

- What a socket is, and what problem it solves
- How to create a socket using Python's `socket` module
- What it means to bind a socket to an address and port
- What the listening state is and how connection queuing works
- How the kernel maps incoming connections to the correct process
- What the connection queue is and why it exists

This chapter uses Python's `socket` module directly to show you exactly what Flask does under the hood. Flask hides these calls, but they happen — and understanding them changes how you think about web servers permanently.

3.1 What a Socket Is and Why It Exists

Before a program can send or receive data over a network, it needs an **endpoint** — a specific, addressable point through which communication flows. In networking, that endpoint is called a **socket**.

A socket is an abstraction provided by the operating system. It represents one end of a communication channel. The other end is another socket — either on the same machine or on a different machine anywhere in the world.

Think of a socket like a telephone. Before you can make or receive calls, you need a phone (the socket). The phone has a number (the address and port). When someone calls you (a connection arrives), you pick up (accept the connection). Until you pick up, the call waits (the connection queue). The conversation (data exchange) flows through the phone once it is connected.

Why the operating system provides sockets

The operating system provides sockets because network communication is complex and hardware-dependent. Sending data over a network requires:

- Formatting data into packets according to protocols (TCP/IP)
- Routing packets through network interfaces and routers
- Managing transmission reliability (retransmitting lost packets)
- Handling congestion control
- Managing the kernel's network buffers

If your program had to do all of this directly, network programming would be extraordinarily difficult. The operating system's **networking stack** handles all of this complexity. Your program interacts with the networking stack through sockets — a clean, simple interface that lets you treat a network connection almost like a file.

This is the same unification we saw in [Chapter 1](#): file descriptors represent all I/O resources uniformly. Network sockets are file descriptors. The same `read()` and `write()` system calls that work on a file also work on a network connection. Sockets add a few calls of their own — `recv()` and `send()` are socket-specific versions with extra options — but underneath, a connection is just another file descriptor.

Socket types

Not all sockets are the same. The two most important types are:

SOCK_STREAM — a connection-oriented socket using the TCP protocol. TCP guarantees that data arrives in order and without loss. It requires a connection to be established before data can be sent. This is the type Flask uses. HTTP (and therefore Flask) runs over TCP.

SOCK_DGRAM — a connectionless socket using the UDP protocol. UDP sends individual packets without a connection. Packets may arrive out of order or not at all. UDP is used for things like DNS lookups, voice and video calls, and online games, where speed matters more than perfect reliability.

Flask uses TCP (SOCK_STREAM) exclusively.

3.2 Creating a Socket: The `socket()` Call

The first step in creating network capability is calling `socket()`. This is a system call that asks the kernel to create a new socket and return a file descriptor for it.

In Python, `socket()` is exposed through the `socket` module:

Listing 3.1 Creating a TCP socket

```
import socket

# socket.AF_INET      = use IPv4 addresses
# socket.SOCK_STREAM = use TCP (reliable, connection-oriented)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

print(f"Socket created. File descriptor: {server_socket.fileno()}")
# Output: Socket created. File descriptor: 3
```

The two arguments to `socket()` specify:

- **Address family (AF_INET):** We are using IPv4 addresses. IPv4 addresses look like 127.0.0.1 or 192.168.1.100. The modern alternative is AF_INET6 for IPv6, but IPv4 is what most services use today.
- **Socket type (SOCK_STREAM):** We want a TCP socket.

When this call returns, the kernel has:

1. Allocated a socket data structure in kernel memory
2. Created a file descriptor (an integer like 3) in this process's file descriptor table
3. Associated the socket data structure with that file descriptor

At this point, the socket exists but is not connected to anything. It has no address. It is not listening. It is like having a phone with no phone number.

The socket option: SO_REUSEADDR

Before binding (the next step), there is an important socket option to set:

Listing 3.2 Setting SO_REUSEADDR before binding

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Allow reuse of the address immediately after the process exits
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

When a server process exits, the kernel keeps its port in a TIME_WAIT state for a short period (60 seconds on Linux) to handle any delayed packets still in transit. During this period, if you try to restart the server, you will get:

```
OSError: [Errno 98] Address already in use
```

SO_REUSEADDR tells the kernel to allow reuse of the port immediately, even if it is in TIME_WAIT. Flask sets this option automatically, which is why you can stop and restart your development server quickly without waiting.

3.3 Binding to an Address: bind() and localhost:5000

A socket without an address is like a phone without a number. To receive connections, the socket needs to be associated with a specific **IP address** and **port number**. This association is

called **binding**.

What IP addresses and ports are

An **IP address** identifies a machine on a network. Every device connected to a network — your laptop, your phone, a server in a data center — has an IP address. IPv4 addresses are written as four numbers separated by dots: **192.168.1.100**.

A **port number** is a 16-bit integer (0 to 65535) that identifies a specific communication endpoint on a machine. A single machine can run many programs, each listening on a different port. Port numbers allow the kernel to route incoming connections to the correct process.

Some port numbers are reserved by convention:

- Port 80: HTTP
- Port 443: HTTPS
- Port 22: SSH
- Port 5432: PostgreSQL
- Port 5000: Flask’s default development port

Note: On macOS 12 and later, port 5000 is used by the AirPlay Receiver. If Flask reports “Address already in use” on a Mac, either turn off AirPlay Receiver (System Settings → General → AirDrop & Handoff) or run Flask on another port, such as **5001**.

Ports below 1024 are **privileged ports** — binding to them requires root (administrator) privileges on most Linux systems. This is why Nginx (running as root or with special capabilities) can listen on port 80 and 443, while your Flask app (running as a normal user) listens on port 5000 or 8000.

The `bind()` call

Listing 3.3 Binding the socket to an address

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Bind to localhost, port 5000
# "localhost" resolves to 127.0.0.1 — the loopback address
server_socket.bind(("localhost", 5000))

print("Socket bound to localhost:5000")
```

`bind()` takes a tuple of (host, port). After this call:

- The kernel has recorded that this socket is associated with IP address `127.0.0.1` and port `5000`
- No other socket on this machine can bind to `127.0.0.1:5000` until this one is closed
- The socket is still not listening — it cannot accept connections yet

The loopback address: `127.0.0.1`

`localhost` resolves to the IP address `127.0.0.1`. This is the **loopback address** — a special IP address that refers to the machine itself. Traffic sent to `127.0.0.1` never leaves the machine. It travels through the network stack internally and arrives back at the same machine.

This is why, when you run Flask on your laptop and open `http://localhost:5000` in your browser, the request works — the browser sends to `127.0.0.1`, the kernel routes it to the Flask process, and Flask responds. The network packet never touches your Wi-Fi card or any external network.

This is also why your server is invisible to other machines on the network. `127.0.0.1` is not a routable address. Traffic from another computer cannot reach it. We examine this limitation in detail in → [Chapter 10 — Networking Limitations](#), and we fix it in → [Chapter 24 — Process Startup in Production](#) (by binding to `0.0.0.0` instead).

Binding to `0.0.0.0`

The address `0.0.0.0` has a special meaning in the context of `bind()`. It means: “listen on all available network interfaces.” If your server has multiple IP addresses (a common scenario for cloud servers with both public and private IPs), binding to `0.0.0.0` accepts connections

arriving on any of them.

Listing 3.4 Binding to all interfaces (used in production)

```
server_socket.bind(("0.0.0.0", 5000))
```

When a server must accept connections from other machines directly, it binds to `0.0.0.0`. In the production setup this book builds, the application sits behind a reverse proxy on the same machine, so it only needs to be reachable locally — the proxy is what faces the outside world (→ [Chapter 22](#) and → [Chapter 28](#)). On your local machine, you bind to `localhost` so the server is not accidentally exposed to your local network.

Flask's `app.run()` binds to `127.0.0.1` by default. To change this:

Listing 3.5 Running Flask bound to all interfaces

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Warning: Never run Flask in debug mode (`debug=True`) bound to `0.0.0.0` on a public network. The debug server's interactive debugger would be accessible to anyone who can reach your machine — a severe security risk. In production, Gunicorn or another production-grade server handles network binding; Flask's development server is never exposed directly.

3.4 Entering the Listening State: `listen()`

The socket is now bound to an address. The next step is telling the kernel to start queuing incoming connections. This is done with `listen()`.

Listing 3.6 Entering the listening state

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind(("localhost", 5000))

# Begin listening. The argument is the backlog — see Section 3.6.
server_socket.listen(5)

print("Server is listening on localhost:5000")
```

After `listen()`:

- The kernel marks this socket as a “listening socket”
- The kernel begins accepting TCP connection requests from clients, storing them in a queue
- Your program is not yet involved in individual connections — it just said “I’m ready to receive connections”

The socket’s state has changed from “bound but passive” to “listening.” This state change is reflected in the kernel’s network tables. You can see it with the `ss` command on Linux:

```
ss -tlnp
```

Output:

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	5	127.0.0.1:5000	0.0.0.0:*	python

The LISTEN state and the `127.0.0.1:5000` address confirm that the socket is ready. No connections have arrived yet.

3.5 How the Kernel Registers Port-to-Process Mappings

When `listen()` is called, the kernel makes an internal record:

“If a TCP connection arrives destined for IP 127.0.0.1 and port 5000, deliver it to the process with PID 12345, which has the listening socket with file descriptor 3.”

This record is maintained in the kernel’s network tables. It is why multiple programs cannot listen on the same port simultaneously — by default there can only be one listening socket per (IP address, port) combination. (The `SO_REUSEPORT` option relaxes this so several processes can share a port — a feature some servers use to spread load.)

How the kernel routes an incoming connection

When a client (say, your browser) sends a connection request to `127.0.0.1:5000`, the following sequence occurs inside the kernel:

1. The packet arrives on a network interface – for 127.0.0.1, the virtual loopback
↳ interface (``lo``)
2. Kernel's TCP/IP stack processes the packet
3. TCP/IP stack reads the destination IP (127.0.0.1) and port (5000)
4. Kernel looks up (127.0.0.1, 5000) in the socket table
5. Finds: "this belongs to file descriptor 3 in process 12345"
6. Kernel completes the TCP handshake with the client
7. Kernel places the completed connection in the socket's accept queue
8. When the process calls `accept()`, the kernel delivers the connection

Your Python code is not involved in steps 1 through 7. The kernel handles all of that. Your code only participates starting at step 8, when you call `accept()`. This is the separation between kernel-space networking (which is fast, runs in privileged mode, and handles the protocol machinery) and user-space networking (your program, which works with already-established connections).

Checking the kernel's socket table

On Linux, the kernel's socket table is accessible through `/proc`:

```
cat /proc/net/tcp
```

Or more readably:

```
ss -tlnp | grep 5000
```

Both commands show you which processes own which sockets. When debugging “why isn’t my server reachable?”, these commands are the first thing to check.

On macOS:

```
lsof -i :5000
```

This lists all processes with an open socket on port 5000.

3.6 The Connection Queue: What Happens Before `accept()`

When a client connects to your server, the kernel does not immediately hand the connection to your program. Instead, it stores the connection in a **queue** — a waiting list of connections that are ready to be handled but have not yet been picked up by the program.

This queue is called the **accept queue** (or backlog queue). Its maximum size is specified by the argument to `listen()`. In Listing 3.6, `server_socket.listen(5)` creates a queue that holds up to 5 pending connections.

Why the queue exists

Consider what happens when two connections arrive at almost the same moment, but your program can only handle one at a time. Without a queue:

- Connection 1 arrives: your program handles it
- Connection 2 arrives while Connection 1 is being handled: Connection 2 is rejected because your program is busy

With a queue:

- Connection 1 arrives: kernel puts it in the queue
- Connection 2 arrives: kernel puts it in the queue too
- Your program calls `accept()`: retrieves Connection 1 from the queue, handles it
- Your program calls `accept()` again: retrieves Connection 2, handles it

The queue allows the kernel to absorb brief bursts of connections even when the program is temporarily busy.

What happens when the queue is full

If connections arrive faster than the program calls `accept()`, and the queue fills up, the kernel starts rejecting new connection attempts. From the client's perspective, the connection times out. This is why a server under heavy load that cannot keep up with incoming connections eventually starts dropping requests.

The backlog value (5 in our example) is deliberately small for a development server. Production deployments (using Unicorn or similar) set this much higher. Modern Linux kernels also have kernel-level tuning parameters (`net.core.somaxconn`, `net.ipv4.tcp_max_syn_backlog`) that cap the backlog independently of what your program requests.

The two queues in TCP

Technically, Linux maintains two queues for incoming TCP connections:

1. **SYN queue (incomplete connections)**: Holds connections where the TCP three-way handshake is still in progress (SYN received, SYN-ACK sent, waiting for ACK).

2. **Accept queue (completed connections):** Holds connections where the handshake is complete and the connection is ready for `accept()`.

The `listen()` argument controls the accept queue. This distinction matters in the context of SYN flood attacks (→ [Chapter 38 — Security](#)), where attackers send many half-open connections to fill the SYN queue.

A Raw Socket Server: Seeing the Full Picture

Before Flask hides all of this, here is a complete, raw TCP server written with Python's `socket` module directly. This is what a web server looks like at its core, without any framework:

Listing 3.7 A minimal raw TCP server showing the full socket lifecycle

```
import socket

def run_raw_server():
    # Step 1: Create the socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    # Step 2: Bind to an address
    server_socket.bind(("localhost", 5000))

    # Step 3: Start listening
    server_socket.listen(5)
    print("Listening on localhost:5000 — connect with: curl http://localhost:5000")

    # Step 4: Accept connections in a loop
    while True:
        # accept() blocks until a connection arrives
        # It returns a NEW socket for this specific connection,
        # and the address of the connecting client
        client_socket, client_address = server_socket.accept()
        print(f"Connection from {client_address}")

        # Step 5: Read the request (raw bytes)
        request_bytes = client_socket.recv(4096)
        print(f"Received {len(request_bytes)} bytes")
        print(request_bytes.decode("utf-8", errors="replace"))

        # Step 6: Send an HTTP response
        response = (
```

```

        "HTTP/1.1 200 OK\r\n"
        "Content-Type: text/plain\r\n"
        "Content-Length: 13\r\n"
        "Connection: close\r\n"
        "\r\n"
        "Hello, world!"
    )
    client_socket.sendall(response.encode("utf-8"))

    # Step 7: Close this connection's socket
    client_socket.close()
    print(f"Connection from {client_address} closed.")

if __name__ == "__main__":
    run_raw_server()

```

Run this program in one terminal:

```
python listing_3_7.py
```

Then in another terminal, send a request:

```
curl http://localhost:5000
```

You will see:

In the server terminal:

```

Listening on localhost:5000 – connect with: curl http://localhost:5000
Connection from ('127.0.0.1', 54321)
Received 78 bytes
GET / HTTP/1.1
Host: localhost:5000
User-Agent: curl/7.88.1
Accept: */*

Connection from ('127.0.0.1', 54321) closed.

```

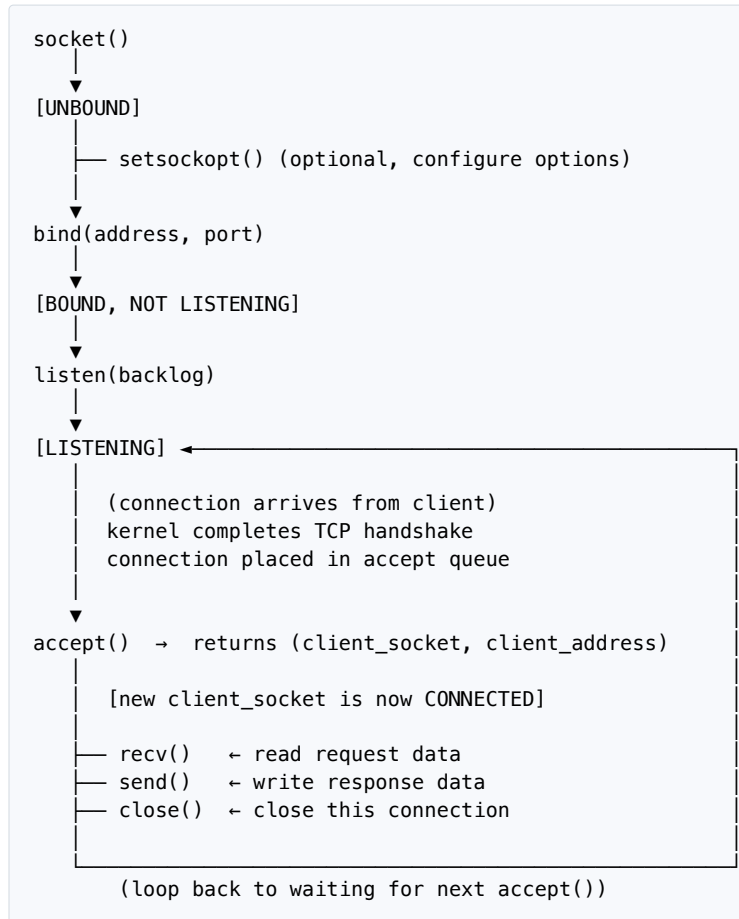
In the curl terminal:

```
Hello, world!
```

This program — 40 lines of Python — is the core of every web server. Everything Flask does is

every `curl` or browser request is a real TCP connection being accepted from that queue.

The Socket Lifecycle: Full State Diagram



Key Takeaways

- A **socket** is an OS abstraction representing one end of a network communication channel. It is represented as a file descriptor — an integer — like any other I/O resource.
- **socket(AF_INET, SOCK_STREAM)** creates a TCP socket. TCP provides reliable, ordered data delivery, which is what HTTP requires.
- **bind(address, port)** associates the socket with a specific IP address and port number.

`localhost` (127.0.0.1) means “only accept connections from this machine.” `0.0.0.0` means “accept connections on all network interfaces.”

- **listen(backlog)** tells the kernel to start queuing incoming connections. The kernel completes TCP handshakes and places ready connections in the accept queue.
- The kernel maintains a **socket table** mapping (IP, port) pairs to processes. Incoming packets are routed to the correct process using this table.
- The **accept queue** holds completed connections waiting for `accept()` to be called. If the queue fills up, new connections are rejected.
- **accept()** removes the next connection from the accept queue and returns a **new socket** specific to that connection. The listening socket remains open and ready for the next connection.
- Flask (via Werkzeug) performs all of these steps internally when you call `app.run()`. The socket lifecycle is real — it happens even though Flask hides it.

Exercises

1. **Predict.** Start the Notes API bound to `127.0.0.1:5001`. From a second machine on the same Wi-Fi, request `http://<your-laptop-ip>:5001/`. What happens, and what would you change so that it works? (Port 5001, because on macOS the AirPlay Receiver usually holds 5000.)
2. **Break and fix.** Start two copies of the application on the same port. Read the exact error message. Then find which process holds the port with `lsof -i :5000` and stop it.
3. **Extend.** Write a twenty-line TCP server with the `socket` module that accepts one connection, reads the request, and answers `HTTP/1.1 200 OK` with a short body. Call it with `curl -v`.
4. **Explain.** What is the difference between the listening socket and the socket `accept()` returns, and why does a server need both?

Solutions and hints are in the companion repository, in `exercises/chapter-03.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What’s Next

The server socket is bound and listening. Connections can arrive and be queued. But connections arriving at the right port are not enough — the server must know what to do with them. In the next chapter, we define the request-handling paths: the routes, methods, and handler functions that give each incoming request its meaning.

→ [Chapter 4](#) — Defining Request Handling Paths: The API Is Created Here

Defining Request Handling Paths

The API Is Created Here

In the previous chapter, we built a raw TCP server. It listened on a port, accepted connections, read bytes from the network, and sent bytes back. It was a complete, working network server.

But it treated every connection the same way. It had no concept of `/notes` versus `/users` versus `/login`. It could not distinguish a GET request from a POST request. It could not route a request about creating a note to a function that creates notes, or a request about listing notes to a function that lists them.

That differentiation — the ability to take an incoming HTTP request and decide which piece of code should handle it — is what makes a raw TCP server into an **API**.

An **API** (Application Programming Interface) is a contract. It declares: “If you send me a message in this specific form (this URL, this HTTP method, this data format), I will perform this specific action and return this specific kind of response.” The client — a browser, a mobile app, another service — knows exactly what to expect. The server knows exactly what to do.

This contract is created during initialization, as we saw in [Chapter 2](#). When Python reads through your `app.py` file top to bottom, it encounters decorator expressions like `@app.route("/notes", methods=["GET"])`, and Flask adds entries to its internal route table. By the time the first connection ever arrives, the route table is complete and ready.

This chapter explains every aspect of how that route table is built: what routes are, how HTTP methods work, how Flask constructs and stores the routing map, how handler functions are connected to routes, how middleware chains are defined, and how you test your routes locally before deploying anywhere.

In this chapter

By the end of this chapter, you will understand:

- What a route is and what it represents in an API
- How `@app.route()` decorators work under the hood
- What HTTP methods (GET, POST, PUT, DELETE) mean and when to use each
- How Flask builds and stores its route table (the Map object)
- How handler functions are bound to routes
- How middleware is defined in Flask and what it does
- How to test the Notes API locally using `curl`, Postman, and the browser

4.1 What a Route Is and What It Represents

A **route** is a rule that maps an incoming HTTP request — characterized by a URL path and an HTTP method — to a specific Python function.

Every time a request arrives at a Flask server, Flask must answer two questions:

1. What URL path is being requested? (e.g., `/notes`, `/notes/42`, `/users/login`)
2. What HTTP method is being used? (e.g., GET, POST, PUT, DELETE)

The combination of path and method identifies what the client wants to do. A route tells Flask: “When these two conditions match, call this Python function.”

URL paths

A **URL path** is the part of the URL after the domain name and port. Given the URL:

```
http://localhost:5000/notes/42
```

The path is `/notes/42`. It identifies a resource (note number 42) that the client wants to interact with.

Paths can be:

- **Static paths:** Exactly as written. `/notes` matches only the string `/notes`.
- **Dynamic paths:** Contain variable parts that Flask extracts. `/notes/<int:note_id>` matches `/notes/42`, `/notes/100`, `/notes/7` — any path matching the pattern — and Flask extracts the integer part as `note_id`.

HTTP methods

HTTP methods (sometimes called “HTTP verbs”) describe what the client wants to do with the resource at the given path. The main methods are:

Method	Meaning	Notes API use
GET	Retrieve a resource	GET /notes → list all notes; GET /notes/42 → get note 42
POST	Create a new resource	POST /notes → create a new note
PUT	Replace an existing resource entirely	PUT /notes/42 → replace note 42
PATCH	Update part of an existing resource	PATCH /notes/42 → update only the content field
DELETE	Remove a resource	DELETE /notes/42 → delete note 42

A GET request should never modify data. A POST request creates. A DELETE request removes. The HTTP standard defines these meanings (GET is “safe”; GET, PUT, and DELETE are “idempotent”), but nothing enforces them — following them is what makes your API predictable and interoperable with standard tools.

Routes are the API’s contract

Every route you define is a public commitment: “This path and method combination is supported.” Routes you do not define return a **404 Not Found** response automatically (Flask handles this). The set of routes in your application is the API’s contract — the specification of what the server can do.

4.2 Defining Routes in Flask: `@app.route()`

Flask uses **Python decorators** to register routes. A decorator is a function that takes a function and returns a function — often a wrapped version that adds behavior. Flask’s `@app.route()` decorator does not wrap anything: it registers the decorated function as the handler for a specific URL pattern and method combination.

Listing 4.1 The simplest Flask route

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def index():
    return "Notes API is running."
```

The `@app.route("/")` line does two things when Python reads it:

1. It calls `app.route("/")`, which returns a decorator function
2. That decorator function is applied to `index`, registering `index` as the handler for GET /

This is standard Python decorator syntax. The `@` symbol is syntactic sugar. Writing:

```
@app.route("/")
def index():
    return "Notes API is running."
```

is exactly equivalent to:

```
def index():
    return "Notes API is running."

index = app.route("/")(index)
```

Both expressions register `index` as the handler for GET / and then rebind the name `index` to the (possibly wrapped) function.

What happens inside `app.route()`

When `app.route("/")` is called, Flask calls `app.add_url_rule()` internally. Here is a simplified view of what that does:

```
# Conceptual illustration of what Flask does internally
def add_url_rule(self, rule, endpoint, view_func, methods=None):
    # 'rule' is the URL pattern: "/"
    # 'endpoint' is a name for this route (defaults to the function name)
    # 'view_func' is the handler function
    # 'methods' is the list of allowed HTTP methods

    if methods is None:
        methods = ["GET"] # Default to GET (Flask also adds HEAD and OPTIONS)

    # Add to the URL map (the route table)
    self.url_map.add(Rule(rule, endpoint=endpoint, methods=methods))

    # Associate the endpoint name with the handler function
    self.view_functions[endpoint] = view_func
```

The URL map (`self.url_map`) is a Werkzeug Map object — a compiled, efficient data structure for URL pattern matching. Adding a Rule to it registers the pattern.

The `view_functions` dictionary maps endpoint names (strings) to handler functions. The endpoint name defaults to the function's name. So `@app.route("/")` on a function named `index` creates an endpoint named "index" that maps to the `index` function.

4.3 HTTP Method Mapping: GET, POST, PUT, DELETE

By default, `@app.route()` responds only to GET requests (plus the HEAD and OPTIONS requests Flask answers automatically). To handle other methods, specify them explicitly:

Listing 4.2 Defining routes with specific HTTP methods

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/notes", methods=["GET"])
def list_notes():
    """Return all notes."""
    return jsonify({"notes": []})

@app.route("/notes", methods=["POST"])
def create_note():
    """Create a new note."""
    data = request.get_json()
```

```

    return jsonify({"message": "Note created", "data": data}), 201

@app.route("/notes/<int:note_id>", methods=["GET"])
def get_note(note_id):
    """Return a specific note by ID."""
    return jsonify({"id": note_id, "content": "Sample note"})

@app.route("/notes/<int:note_id>", methods=["PUT"])
def update_note(note_id):
    """Replace a note entirely."""
    data = request.get_json()
    return jsonify({"id": note_id, "content": data.get("content")})

@app.route("/notes/<int:note_id>", methods=["DELETE"])
def delete_note(note_id):
    """Delete a note."""
    return jsonify({"message": f"Note {note_id} deleted"}), 200

```

Notice that `/notes` and `/notes/<int:note_id>` are two different route patterns:

- `/notes` matches the collection (all notes)
- `/notes/<int:note_id>` matches a specific note by ID

And within each pattern, different methods perform different operations:

- GET `/notes` lists notes; POST `/notes` creates one
- GET `/notes/42` retrieves note 42; PUT `/notes/42` replaces it; DELETE `/notes/42` removes it

This design — paths identify resources, methods identify operations — is the foundation of REST (Representational State Transfer), the dominant API design style for web services.

What happens when a method is not allowed

If a client sends a request using a method that is not registered for that path, Flask automatically returns a 405 Method Not Allowed response. For example, if you only define GET `/notes` and a client sends DELETE `/notes`:

```

curl -X DELETE http://localhost:5000/notes
# Response: 405 Method Not Allowed

```

Flask handles this automatically — you do not need to write code for it.

Combining methods on a single function

You can register multiple methods on a single handler function. This is sometimes done when the same function handles both viewing and submitting a form (common in traditional web applications):

Listing 4.3 One handler for multiple methods

```
@app.route("/notes", methods=["GET", "POST"])
def notes():
    if request.method == "GET":
        return jsonify({"notes": []})
    elif request.method == "POST":
        data = request.get_json()
        return jsonify({"message": "Created"}), 201
```

For APIs, however, it is cleaner to have separate functions for separate operations, as in Listing 4.2. The code is easier to read and reason about.

4.4 How Flask Builds Its Internal Route Table

Flask uses Werkzeug’s URL routing system to build its route table. When you understand this system, you understand why some routes match and others do not, and how URL patterns with variables work.

The Map and Rule objects

Werkzeug’s URL routing centers on two classes:

Rule: Represents one URL pattern. A Rule stores:

- The URL pattern string (e.g., `"/notes/<int:note_id>"`)
- The allowed HTTP methods (e.g., `["GET", "PUT", "DELETE"]`)
- The endpoint name (e.g., `"get_note"`)
- Whether the URL must end with a trailing slash

Map: A collection of Rule objects. After all rules are added, the Map is “locked” (compiled into an efficient matching structure). You can inspect Flask’s URL map at runtime:

Listing 4.4 Inspecting Flask's route table

```

from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/notes", methods=["GET"])
def list_notes():
    return jsonify({"notes": []})

@app.route("/notes/<int:note_id>", methods=["GET"])
def get_note(note_id):
    return jsonify({"id": note_id})

# Print the route table
with app.app_context():
    for rule in app.url_map.iter_rules():
        methods = ", ".join(sorted(rule.methods))
        print(f"{rule.rule:30s} [{methods}] → {rule.endpoint}")

```

Output:

```

/static/<path:filename>      [GET, HEAD, OPTIONS] → static
/notes                      [GET, HEAD, OPTIONS] → list_notes
/notes/<int:note_id>         [GET, HEAD, OPTIONS] → get_note

```

The first rule, `static`, is Flask's built-in route for serving files from a `static/` folder. You will also notice `HEAD` and `OPTIONS` appear even though you did not define them. Flask adds these automatically:

- `HEAD` is like `GET` but returns only the headers, no body. Flask handles it by running the `GET` handler and stripping the response body.
- `OPTIONS` is used by browsers for CORS preflight checks. Flask handles it automatically.

URL variable converters

In the pattern `/notes/<int:note_id>`, the `<int:note_id>` part is a **variable converter**. It tells Flask:

- Extract the part of the URL at this position
- Convert it to an `int` (if it cannot be converted, return 404)
- Pass it to the handler as the `note_id` argument

Flask's built-in converters:

Converter	Matches	Example
string (default)	Any text without slashes	<name> matches "alice"
int	Non-negative integers	<int:id> matches "42"
float	Floating-point numbers	<float:value> matches "3.14"
path	Text including slashes	<path:filename> matches "dir/file.txt"
uuid	UUID strings	<uuid:token> matches a valid UUID

If the URL contains something that does not match the converter — for example, /notes/abc when the pattern is /notes/<int:note_id> — Flask returns a 404 Not Found response automatically.

Route ordering and conflicts

More specific patterns take precedence over less specific ones. For example:

```
@app.route("/notes/latest", methods=["GET"])
def get_latest_note():
    ...

@app.route("/notes/<int:note_id>", methods=["GET"])
def get_note(note_id):
    ...
```

A request for GET /notes/latest will match the first route (the static string "latest" is more specific than the <int:note_id> pattern, which also fails to match because "latest" is not an integer). A request for GET /notes/42 will match the second route.

Werkzeug builds the route table so that more specific patterns always win over less specific ones, regardless of registration order.

4.5 Binding Handler Functions to Routes

The handler function — the Python function that executes when a request matches a route — is the heart of the API. It is where the actual work happens.

Flask calls handler functions **view functions** (a term inherited from the Model-View-Controller pattern). When Flask matches a request to a route, it calls the corresponding view

function and uses its return value to construct the HTTP response.

What a handler function receives

Handler functions receive their URL variable arguments as keyword arguments:

Listing 4.5 Handler functions and their arguments

```
@app.route("/notes/<int:note_id>", methods=["GET"])
def get_note(note_id):
    # note_id is automatically extracted from the URL and passed here
    # It is already an int (Flask applied the converter)
    print(f"Fetching note with ID: {note_id}, type: {type(note_id)}")
    # Output: Fetching note with ID: 42, type: <class 'int'>
    ...
```

For request data (headers, body, query parameters), Flask provides the request object, which is a context-local proxy that automatically refers to the request currently being handled:

Listing 4.6 Accessing request data in a handler

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/notes", methods=["POST"])
def create_note():
    # Request headers
    content_type = request.headers.get("Content-Type")

    # Query string parameters: /notes?page=2
    page = request.args.get("page", default=1, type=int)

    # Request body as JSON
    data = request.get_json()

    # Request body as raw bytes
    raw_body = request.data

    return jsonify({
        "content_type": content_type,
        "page": page,
        "data": data,
    })
```

What a handler function must return

Flask handler functions must return a value that Flask can convert into an HTTP response. Flask accepts:

- A **string**: becomes the response body with a 200 status and `text/html` content type
- A **Response object**: used as-is
- A **tuple** of (body, status_code) or (body, status_code, headers)
- A **dictionary**: automatically serialized to JSON (Flask 1.1+)

The most common pattern for APIs is to use `jsonify()`, which creates a Response object with the correct `Content-Type: application/json` header:

Listing 4.7 Different return value formats

```
from flask import Flask, jsonify, Response

app = Flask(__name__)

@app.route("/a")
def returns_string():
    return "Hello" # 200 OK, Content-Type: text/html

@app.route("/b")
def returns_json():
    return jsonify({"message": "Hello"}) # 200 OK, Content-Type: application/json

@app.route("/c")
def returns_tuple():
    return jsonify({"error": "Not found"}), 404 # 404 Not Found

@app.route("/d")
def returns_dict():
    return {"message": "Hello"} # Flask auto-converts to JSON (Flask 1.1+)
```

For the Notes API, we always use `jsonify()` to be explicit about the content type.

4.6 Defining Middleware Chains

Middleware is code that runs before or after every request, regardless of which route was matched. Middleware is the right place for concerns that apply to all requests: logging, authentication, rate limiting, and request validation.

Flask provides several hooks for registering middleware functions:

@app.before_request

Functions decorated with `@app.before_request` run before the handler function. They can inspect or modify the request. If they return a value, that value becomes the response and the handler is never called:

Listing 4.8 A `before_request` middleware for request logging

```
from flask import Flask, request, jsonify, g
import time

app = Flask(__name__)

@app.before_request
def log_request_start():
    """Record when the request started."""
    g.start_time = time.time()
    print(f"→ {request.method} {request.path}")

@app.before_request
def require_json_for_post():
    """Reject POST requests that are not JSON."""
    if request.method == "POST":
        if not request.is_json:
            return jsonify({"error": "Content-Type must be application/json"}), 415
```

Two `@before_request` functions are registered here. Flask calls them in registration order. If `require_json_for_post` returns a response, Flask skips the handler and any remaining `before_request` functions; the `after_request` functions still run on that response before it is sent.

@app.after_request

Functions decorated with `@app.after_request` run after the handler returns a response. They receive the response object and must return it (possibly modified):

Listing 4.9 An `after_request` middleware for logging and headers

```

@app.after_request
def log_request_end(response):
    """Log the request duration and add a response header."""
    duration = time.time() - g.start_time
    print(f"← {response.status_code} in {duration * 1000:.1f}ms")

    # Add a custom header to every response
    response.headers["X-Response-Time-Ms"] = f"{duration * 1000:.1f}"
    return response

```

@app.teardown_request

Functions decorated with `@app.teardown_request` run after every request, even if an exception occurred. They are used for cleanup (closing database connections, releasing resources):

Listing 4.10 Teardown for database connection cleanup

```

@app.teardown_request
def close_db_connection(exception):
    """Close the database connection if one was opened."""
    db = g.pop("db", None)
    if db is not None:
        db.close()

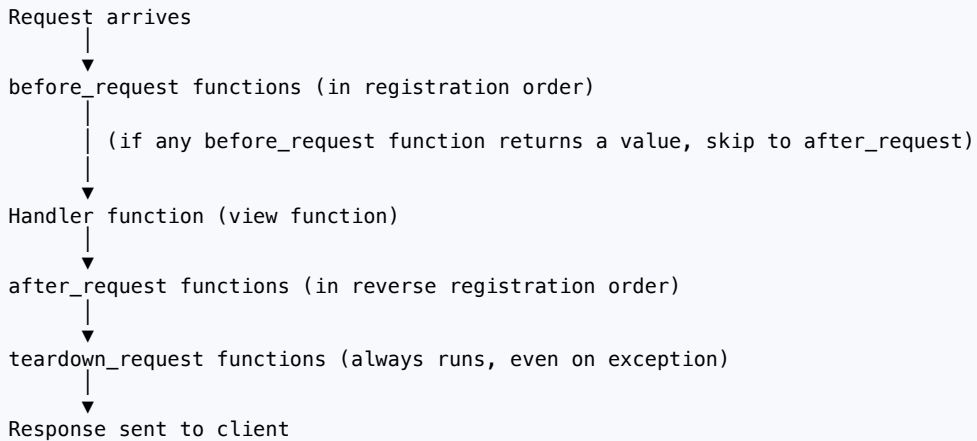
```

Flask's `g` object

You saw `g` in Listing 4.8 and 4.10. `g` is Flask's **request context global** — a storage object that lives for exactly one request. Unlike Python module-level variables (which persist across all requests), `g` is fresh for each request and discarded after the response is sent.

`g` is how you share data between `before_request`, the handler, and `after_request` without using global variables. In Listing 4.8, `g.start_time` is set in `log_request_start` and read in `log_request_end`. This is safe even with multiple simultaneous requests because each request has its own `g` object.

The middleware execution order



4.7 Testing Locally: curl, Postman, and the Browser

Before connecting any frontend, before deploying anywhere, before writing more code — test every route locally. This is the fastest feedback loop you have.

Starting the Notes API

Here is the full Notes API at this stage of the book, incorporating everything from Chapters 2 and 4:

Listing 4.11 The Notes API with full routing and middleware (app.py)

```

from flask import Flask, request, jsonify, g
import sqlite3
import os
import time

app = Flask(__name__)
DATABASE_PATH = os.getenv("DATABASE_PATH", "notes.db")

def get_db():
    """Get the database connection for this request."""
    if "db" not in g:
        g.db = sqlite3.connect(DATABASE_PATH)
        g.db.row_factory = sqlite3.Row # Return rows as dict-like objects
  
```

```

    return g.db

def init_db():
    """Create the notes table if it does not exist."""
    conn = sqlite3.connect(DATABASE_PATH)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            content TEXT NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    conn.close()

init_db() # Runs at startup – also when a server such as Gunicorn imports this file

# — Middleware —————

@app.before_request
def start_timer():
    g.start_time = time.time()

@app.before_request
def require_json_for_writes():
    if request.method in ("POST", "PUT", "PATCH") and not request.is_json:
        return jsonify({"error": "Content-Type must be application/json"}), 415

@app.after_request
def add_timing_header(response):
    duration_ms = (time.time() - g.start_time) * 1000
    response.headers["X-Response-Time-Ms"] = f"{duration_ms:.1f}"
    return response

@app.teardown_request
def close_db(exception):
    db = g.pop("db", None)
    if db is not None:
        db.close()

# — Routes —————

@app.route("/health", methods=["GET"])
def health_check():

```

```

        """A simple health check endpoint."""
        return jsonify({"status": "ok"})

@app.route("/notes", methods=["GET"])
def list_notes():
    """Return all notes."""
    db = get_db()
    cursor = db.execute("SELECT id, content, created_at FROM notes ORDER BY id DESC")
    notes = [dict(row) for row in cursor.fetchall()]
    return jsonify({"notes": notes, "count": len(notes)})

@app.route("/notes", methods=["POST"])
def create_note():
    """Create a new note."""
    data = request.get_json(silent=True) # None if the body is not valid JSON
    if not isinstance(data, dict):
        return jsonify({"error": "Request body must be a JSON object"}), 400

    content = data.get("content")
    if not isinstance(content, str) or not content.strip():
        return jsonify({"error": "content is required and cannot be empty"}), 400
    content = content.strip()

    db = get_db()
    cursor = db.execute("INSERT INTO notes (content) VALUES (?)", (content,))
    db.commit()

    return jsonify({"id": cursor.lastrowid, "content": content}), 201

@app.route("/notes/<int:note_id>", methods=["GET"])
def get_note(note_id):
    """Return a specific note by ID."""
    db = get_db()
    cursor = db.execute(
        "SELECT id, content, created_at FROM notes WHERE id = ?", (note_id,)
    )
    row = cursor.fetchone()
    if row is None:
        return jsonify({"error": f"Note {note_id} not found"}), 404
    return jsonify(dict(row))

@app.route("/notes/<int:note_id>", methods=["DELETE"])
def delete_note(note_id):
    """Delete a note by ID."""

```



```

db = get_db()
cursor = db.execute("DELETE FROM notes WHERE id = ?", (note_id,))
db.commit()
if cursor.rowcount == 0:
    return jsonify({"error": f"Note {note_id} not found"}), 404
return jsonify({"message": f"Note {note_id} deleted"})

# — Entry point —————

if __name__ == "__main__":
    app.run(debug=True, host="localhost", port=5000)

```

Save this as `app.py` in a directory called `notes-api`. Then create a virtual environment, install Flask into it, and start the server (Chapter 6 explains virtual environments in detail — on many systems a plain `pip install` outside one is refused):

```

python3 -m venv venv
source venv/bin/activate
pip install flask
python app.py

```

You should see:

```

* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use
↳ a production WSGI server instead.
* Running on http://localhost:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 123-456-789

```

Note: With debug mode on, Flask pretty-prints JSON responses over several lines. With debug mode off — as in production — it sends the same JSON compactly on one line. Either way, Flask sorts the keys alphabetically. This book shows the compact form.

Testing with curl

`curl` is a command-line tool for making HTTP requests. It is available on macOS, Linux, and Windows (via WSL or natively in recent Windows versions).

Check that the server is running:

```
curl http://localhost:5000/health
```

Expected output:

```
{"status":"ok"}
```

Create a note:

```
curl -X POST http://localhost:5000/notes \  
  -H "Content-Type: application/json" \  
  -d '{"content": "Buy groceries"}'
```

Expected output:

```
{"content":"Buy groceries","id":1}
```

List all notes:

```
curl http://localhost:5000/notes
```

Expected output:

```
{"count":1,"notes":[{"content":"Buy groceries","created_at":"2024-01-01  
↪ 12:00:00","id":1}]}
```

Get a specific note:

```
curl http://localhost:5000/notes/1
```

Try to get a note that does not exist:

```
curl http://localhost:5000/notes/999
```

Expected output:

```
{"error":"Note 999 not found"}
```

HTTP status: 404

Delete a note:

```
curl -X DELETE http://localhost:5000/notes/1
```

Expected output:

```
{"message": "Note 1 deleted"}
```

Try to POST without the correct Content-Type (middleware should reject it):

```
curl -X POST http://localhost:5000/notes \  
  -d '{"content": "test"}'  
# Missing Content-Type: application/json header
```

Expected output:

```
{"error": "Content-Type must be application/json"}
```

HTTP status: 415

Each of these tests exercises a different route and a different code path. Testing this way — with `curl` — gives you the exact HTTP request and response, including status codes and headers. It is the most direct way to verify your API behaves as expected.

Viewing headers with `curl -i`

Adding `-i` to a `curl` command shows response headers, which is important for verifying your middleware worked:

```
curl -i http://localhost:5000/health
```

Output:

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.1 Python/3.11.8
Date: Mon, 01 Jan 2024 12:00:00 GMT
Content-Type: application/json
Content-Length: 16
X-Response-Time-Ms: 0.8
Connection: close

{"status":"ok"}
```

The `X-Response-Time-Ms` header confirms that the `after_request` middleware ran and added the timing information.

Testing with Postman

Postman is a graphical tool for testing APIs. It provides a visual interface for constructing requests, viewing responses, and saving test cases.

To test the Notes API in Postman:

1. Open Postman and create a new request
2. Set the method to `POST` and the URL to `http://localhost:5000/notes`
3. Go to the “Body” tab, select “raw” and “JSON”
4. Enter `{"content": "Learn Flask"}` in the body
5. Click “Send”

Postman shows the response body, status code, response time, and headers in separate panes. It is particularly useful when:

- Constructing complex request bodies
- Testing with authentication headers
- Organizing multiple requests into a collection for repeated testing

Testing with the browser

For GET requests, the browser is the simplest tool. With the server running, open:

- `http://localhost:5000/health`
- `http://localhost:5000/notes`
- `http://localhost:5000/notes/1`

The browser will display the JSON response body. For `POST`, `PUT`, and `DELETE` requests, you

need either `curl` or Postman — the browser’s address bar only makes GET requests.

The Route Table After Listing 4.11

After running the application in Listing 4.11, Flask’s route table looks like this:

Route Pattern	Methods	Handler
/health	GET, HEAD, OPTIONS	health_check
/notes	GET, HEAD, OPTIONS	list_notes
/notes	OPTIONS, POST	create_note
/notes/<int:note_id>	GET, HEAD, OPTIONS	get_note
/notes/<int:note_id>	DELETE, OPTIONS	delete_note
/static/<path:filename>	GET, HEAD, OPTIONS	static (built-in)

When a request arrives:

1. Flask extracts the method and path
2. Flask matches against this table using Werkzeug’s URL matcher
3. Flask retrieves the handler function for the matching endpoint
4. Flask calls the handler, passing any URL variables as arguments

This table is the API. It is the contract. Everything that happens from this point forward — deployment, networking, production operations — exists to make this contract available to the outside world.

Key Takeaways

- A **route** maps a URL path and HTTP method combination to a Python handler function. Routes define the API’s contract.
- **@app.route()** is a decorator that calls Flask’s `add_url_rule()` internally, registering the handler in Flask’s URL map and view function registry.
- **HTTP methods** (GET, POST, PUT, DELETE) describe what operation the client wants to perform. Use GET to read, POST to create, PUT to replace, DELETE to remove.
- **URL variable converters** (<int:note_id>) extract and type-convert parts of the URL path, passing them to handler functions as arguments. Invalid conversions produce automatic 404 responses.
- Flask builds a **Werkzeug Map** (the route table) during initialization. It is complete before the first request arrives.
- **Middleware** (`@before_request`, `@after_request`, `@teardown_request`) runs around every request. It is the right place for cross-cutting concerns like logging, authentication,

- and resource cleanup.
- Flask’s `g` object stores request-scoped data safely, even with multiple concurrent requests.
- Test every route locally with `curl -X METHOD URL -H "Header" -d "body"` before doing anything else.

Exercises

1. **Predict.** What status code does `curl -X PATCH -H 'Content-Type: application/json' -d '{}'` `http://127.0.0.1:5000/notes/1` return, and which part of Flask produces it? Then try it without the header.
2. **Break and fix.** Register two handlers for the same method and path. What happens at start-up, and which one handles requests? Fix it so that each has a distinct route.
3. **Extend.** Add `GET /notes/count` returning `{"count": n}`. Make sure it does not collide with `GET /notes/<int:note_id>`, and prove it with `curl`.
4. **Explain.** Why is a route “part of the API’s contract”, and what goes wrong if you rename one after clients start using it?

Solutions and hints are in the companion repository, in `exercises/chapter-04.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What’s Next

The routes are defined. The handler functions are registered. Middleware is in place. The server is running and the route table is ready.

Now a request actually arrives. The next chapter traces exactly what happens from the moment `accept()` returns a connection to the moment `send()` delivers the response — the complete internal execution path of a single API request.

→ [Chapter 5 — Internal Request Execution: The Local API Flow](#)

Internal Request Execution

The Local API Flow

We have arrived at the center of Stage 1. Everything before this chapter has been preparation: starting the process, initializing the runtime, creating a socket, defining routes, and registering middleware. Now a request actually arrives.

A user opens their API client and sends `POST /notes` with a JSON body. That action generates a TCP connection, a stream of bytes, and — from the perspective of your running program — a single call to `accept()` returning a new socket. Everything that follows, from reading those bytes to sending a response, is what this chapter traces.

This is the complete execution path of a single API request. It is the heart of what a web server actually does. It is also the most important chapter in Stage 1, because it connects all the layers: the socket ([Chapter 3](#)), the route table ([Chapter 4](#)), the business logic (your code), the database (touched in [Chapter 2](#)), and the serialized response sent back through the same connection.

The request journey has eleven distinct steps, each building on the last. By the end of this chapter, you will be able to read any web request — from any framework, in any language — and know exactly what is happening at each stage.

In this chapter

By the end of this chapter, you will understand:

- How `accept()` returns a connection and what the two sockets are
- How `recv()` reads raw bytes from the network
- How HTTP is parsed: turning bytes into structured request data
- How Flask extracts path, method, headers, query parameters, and body
- How the route table resolves a request to a handler function
- How middleware runs before and after the handler
- How business logic accesses the database during a request
- How the response is constructed, serialized, and sent back through the socket

5.1 Accepting a Connection: `accept()`

The server socket has been listening since `app.run()` was called (← Chapter 3 — Creating Network Capability). When a client makes a connection, the kernel completes the TCP handshake and places the connection in the accept queue.

Your program retrieves it by calling `accept()`.

Listing 5.1 The `accept()` call, shown explicitly

```
# (Flask calls this internally; shown here for understanding)

client_socket, client_address = server_socket.accept()
# client_socket: a new socket connected to this specific client
# client_address: a tuple like ('127.0.0.1', 54321)
```

`accept()` blocks — it pauses the program until a connection is available. If no connection is in the queue, the program waits. If a connection is already in the queue (because it arrived while you were handling a previous request), `accept()` returns immediately.

After `accept()` returns:

- `client_socket` is a new file descriptor (e.g., fd 4) representing this specific connection
- `client_address` identifies the client: the IP address and port the client used on its end
- The original listening socket (e.g., fd 3) is unchanged and continues listening

The client address is a pair of numbers: `('127.0.0.1', 54321)`. The IP is the client's address (in this case, the same machine since we are running locally). The port (54321) is an **ephemeral port** — a temporary port number the client's operating system assigned to this outgoing con-

nection. The client does not listen on port 54321; it is simply the source port for tracking the connection on the client's side.

Flask's threading model for accept

Flask's development server (since Flask 1.0) is multi-threaded by default: after `accept()` returns a connection, it hands that connection to a new thread and immediately goes back to `accept()` for the next one. Several requests can therefore be in progress at once — but it is a convenience server, not tuned or hardened for real traffic.

In production, Gunicorn spawns multiple worker processes, each calling `accept()` independently. Multiple requests are handled simultaneously. We cover this in → [Chapter 24](#) — Process Startup in Production and → [Chapter 36](#) — Scaling and Load Handling.

For local development, the built-in server is fine. You are the only user.

5.2 Reading the Request: `recv()`

After `accept()` returns the client socket, the program reads the request data from it using `recv()`.

Listing 5.2 Reading raw bytes from the client socket

```
# (again, Flask handles this internally)

raw_request = client_socket.recv(4096)
# raw_request is a bytes object — raw network data
```

`recv(4096)` reads up to 4096 bytes from the socket. If the request is longer than 4096 bytes, you would need multiple `recv()` calls to read it all. Flask's underlying HTTP library (Werkzeug, using Python's `http.server` module in development) handles this correctly, reading in chunks until the full request is received.

The value returned by `recv()` is a bytes object — raw bytes, not a string. It is the exact bytes the client sent over the network.

What the raw bytes look like

To understand what you have just received, let us look at what a `POST /notes` request actually looks like in raw form:

```
POST /notes HTTP/1.1\r\n
Host: localhost:5000\r\n
Content-Type: application/json\r\n
Content-Length: 28\r\n
User-Agent: curl/7.88.1\r\n
Accept: */*\r\n
\r\n
{"content": "Buy groceries"}
```

In actual bytes, every `\r\n` is a carriage return followed by a newline (two bytes: `0x0D 0x0A`). The blank line `\r\n` after the headers separates the header section from the body. This is the HTTP/1.1 wire format — a text-based protocol defined by RFC 7230 (and successors).

Everything you need to know about the request — the method, the URL, the headers, the body — is in these bytes. The next step is extracting meaning from them.

5.3 Parsing the HTTP Protocol: Turning Bytes into Structure

The raw bytes are text following the HTTP protocol format. **Parsing** means reading that text and extracting structured data from it.

Flask uses Werkzeug for HTTP parsing. Werkzeug’s parser reads the raw bytes and builds a structured `Request` object. Here is what the parser does:

Step 1: Find the request line. The first line of an HTTP request is always the **request line**:

```
POST /notes HTTP/1.1
```

The request line contains:

- The **method**: `POST`
- The **path and query string**: `/notes` (or `/notes?page=2&sort=date`)
- The **HTTP version**: `HTTP/1.1`

The parser splits this line and stores each part separately.

Step 2: Parse the headers. Every line after the request line (up to the blank line) is a header. Headers follow the format `Name: Value`. The parser builds a dictionary of headers:

```
{
  "Host": "localhost:5000",
  "Content-Type": "application/json",
  "Content-Length": "28",
  "User-Agent": "curl/7.88.1",
  "Accept": "*/*",
}
```

Headers are case-insensitive by the HTTP standard. Content-Type, content-type, and CONTENT-TYPE all refer to the same header.

Step 3: Find the blank line. The `\r\n\r\n` sequence (a blank line) marks the end of the headers and the beginning of the body.

Step 4: Read the body. The Content-Length header tells the parser how many bytes to expect in the body. The parser reads exactly that many bytes from the socket:

```
{"content": "Buy groceries"}
```

That is 28 bytes — matching the Content-Length: 28 header.

If a request has neither a Content-Length nor a Transfer-Encoding: chunked header (the normal case for GET requests), HTTP/1.1 says it has no body, and the parser does not wait for one.

What Werkzeug builds

After parsing, Werkzeug has built a Request object. You can see this object's attributes:

Listing 5.3 Inspecting Flask's request object inside a handler

```

from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/notes", methods=["POST"])
def create_note():
    print(f"Method:           {request.method}")           # POST
    print(f"Path:           {request.path}")               # /notes
    print(f"Full URL:       {request.url}")                 # http://localhost:5000/notes
    print(f"Content-Type:   {request.content_type}")        # application/json
    print(f"Content-Length: {request.content_length}")      # 28
    print(f"User-Agent:     {request.user_agent}")          # curl/7.88.1
    print(f"Remote addr:    {request.remote_addr}")         # 127.0.0.1
    print(f"Raw body:       {request.data}")                # b'{"content": "Buy
    ↪ groceries"}'
    return jsonify({"received": True})

```

The request object provides clean, Pythonic access to every part of the HTTP request. This is the benefit of the framework layer — instead of parsing bytes manually, you work with a structured object.

5.4 Extracting Metadata: Path, Method, Headers, Query Parameters, and Body

The parsed Request object contains everything. Flask makes different parts of it accessible through different attributes and methods:

Method and path

Listing 5.4 Accessing method and path

```

request.method      # "POST", "GET", "DELETE", etc.
request.path        # "/notes" — path only, no query string
request.full_path   # "/notes?page=2" — path + query string
request.url         # "http://localhost:5000/notes?page=2" — complete URL

```

Headers

Listing 5.5 Accessing request headers

```

# Using dict-style access (case-insensitive)
content_type = request.headers.get("Content-Type")
auth_header  = request.headers.get("Authorization")

# Iterating over all headers
for name, value in request.headers:
    print(f"{name}: {value}")

```

Headers are stored in a Headers object that behaves like a dictionary but is case-insensitive.

Query parameters

Query parameters are the key-value pairs after the ? in a URL. For a request to /notes?page=2&sort=date:

Listing 5.6 Accessing query string parameters

```

page = request.args.get("page", default=1, type=int)    # 2 (as int)
sort = request.args.get("sort", default="date")        # "date"
limit = request.args.get("limit", default=20, type=int) # 20 (default, not in URL)

```

`request.args` is a `MultiDict` — a dictionary that supports multiple values per key (because ?tag=python&tag=flask is valid). `args.get()` returns the first value; `args.getlist("tag")` returns all values for a key.

Request body

Listing 5.7 Accessing the request body

```
# As raw bytes
raw_bytes = request.data

# As JSON (parses the JSON body, returns a Python dict/list/etc.)
json_data = request.get_json()
# Content-Type is not application/json → Flask aborts with 415 Unsupported Media Type
# The body is not valid JSON → Flask aborts with 400 Bad Request

# As JSON, returning None instead of aborting
json_data = request.get_json(silent=True)
# silent=True: return None on a wrong Content-Type or malformed JSON
# force=True: parse as JSON regardless of the Content-Type header

# As form data (for HTML form submissions)
form_value = request.form.get("username")

# As files (for file uploads)
uploaded_file = request.files.get("document")
```

For an API that accepts JSON (like the Notes API), `request.get_json()` is the standard way to access the request body. The Notes API uses `get_json(silent=True)` so it can return its own JSON error message instead of Flask’s default HTML error page.

5.5 Route Matching and Handler Lookup

The Request object is fully built. Now Flask must determine which handler function should process it.

Flask passes the request to Werkzeug’s URL matcher, which takes the request’s method and path and finds the matching rule in the Map.

Listing 5.8 How Flask resolves the route internally (conceptual illustration)

```
# Flask calls something like this internally:
adapter = app.url_map.bind(request.host)
endpoint, kwargs = adapter.match(request.path, method=request.method)
# endpoint = "create_note" (the function name / endpoint name)
# kwargs = {} (no URL variables in /notes)

# Then looks up the handler function:
handler_function = app.view_functions[endpoint]
# handler_function = <function create_note at 0x...>
```

For a POST /notes request:

- endpoint = "create_note"
- kwargs = {} (no URL variables)
- handler_function = the create_note Python function

For a GET /notes/42 request:

- endpoint = "get_note"
- kwargs = {"note_id": 42} (extracted from the URL, already converted to int)
- handler_function = the get_note Python function

What happens when no route matches

If no rule matches the path and method, Werkzeug raises an exception:

- `NotFound` (404): the path does not exist in the route table at all
- `MethodNotAllowed` (405): the path exists but not for this method

Flask catches these exceptions and converts them into appropriate HTTP error responses automatically. If you want to customize the 404 response (for example, to return JSON instead of HTML), you can register an error handler:

Listing 5.9 Custom error handlers

```

@app.errorhandler(404)
def not_found(error):
    return jsonify({"error": "The requested resource was not found"}), 404

@app.errorhandler(405)
def method_not_allowed(error):
    return jsonify({"error": "Method not allowed for this endpoint"}), 405

@app.errorhandler(500)
def internal_error(error):
    return jsonify({"error": "An internal server error occurred"}), 500

```

Without custom handlers, Flask returns HTML error pages, which is not appropriate for an API that should always return JSON.

5.6 Middleware Execution: Authentication, Logging, Validation

Before the handler function runs, Flask calls all registered `@before_request` functions in order. After the handler returns, Flask calls all `@after_request` functions in reverse order.

For the Notes API from Listing 4.11, the middleware chain for a `POST /notes` request looks like this:

```

Request: POST /notes

Step 1: before_request – start_timer()
    g.start_time = 1704067200.837

Step 2: before_request – require_json_for_writes()
    request.method is "POST" → check Content-Type
    request.is_json is True → pass (do not return early)

Step 3: Handler – create_note()
    [executed next – see Section 5.7]

Step 4: after_request – add_timing_header(response)
    duration = now – g.start_time = 4.2ms
    response.headers["X-Response-Time-Ms"] = "4.2"
    return response

Step 5: teardown_request – close_db(exception)

```



```
g.db exists → db.close()
g.db removed from g
```

Middleware can short-circuit the request

If a `before_request` function returns a value (any value other than `None`), Flask treats that value as the response and skips the handler entirely. The `after_request` and `teardown_request` functions still run.

This is how authentication middleware works:

Listing 5.10 Authentication middleware using `before_request`

```
import hmac

@app.before_request
def require_api_key():
    """Reject requests without a valid API key."""
    # Skip authentication for the health check endpoint
    if request.endpoint == "health_check":
        return None # None means "continue normally"

    api_key = request.headers.get("X-API-Key", "")
    expected = os.getenv("API_KEY", "")
    # compare_digest takes the same time whether the first or last character differs,
    # so an attacker cannot guess the key one character at a time (→ Chapter 38)
    if not expected or not hmac.compare_digest(api_key, expected):
        return jsonify({"error": "Unauthorized"}), 401
    # Return None implicitly – request continues to the handler
```

If the API key is missing or invalid, `require_api_key` returns a 401 response. Flask sends it immediately. The handler never runs. The `after_request` hook adds the timing header to even this error response. The `teardown` hook closes the database connection (even though it was never opened in this case — `g.pop("db", None)` returns `None` safely).

5.7 Business Logic Execution

The handler function runs. This is where your application does its actual work.

For `POST /notes`, the handler is `create_note()`:

Listing 5.11 The `create_note` handler with detailed comments

```

@app.route("/notes", methods=["POST"])
def create_note():
    # — Step 1: Parse the request body —————
    data = request.get_json(silent=True)
    # data is now: {"content": "Buy groceries"}
    # If the body was not valid JSON, data is None

    # — Step 2: Validate the input —————
    if not isinstance(data, dict):
        return jsonify({"error": "Request body must be a JSON object"}), 400

    content = data.get("content")
    if not isinstance(content, str) or not content.strip():
        return jsonify({"error": "content is required and cannot be empty"}), 400
    content = content.strip()

    if len(content) > 10000:
        return jsonify({"error": "content must not exceed 10,000 characters"}), 400

    # — Step 3: Execute business logic —————
    # (For this simple API, "business logic" is just storing the note.
    # In a more complex API, there might be permission checks, rate limiting,
    # notification triggers, audit logging, etc.)

    # — Step 4: Data access —————
    db = get_db() # Returns the request-scoped database connection from g
    cursor = db.execute(
        "INSERT INTO notes (content) VALUES (?)",
        (content,)
    )
    db.commit()
    # cursor.lastrowid is the auto-assigned ID for the new row
    note_id = cursor.lastrowid

    # — Step 5: Construct and return the response —————
    return jsonify({
        "id": note_id,
        "content": content,
    }), 201 # 201 Created – the standard status for successful resource creation

```

The handler function reads the validated input, writes a record to the database, and returns the created resource with a 201 status code.

This is pure Python. The HTTP layer is handled by Flask above this function, and the database layer is handled by SQLite below it. The handler function itself is just business logic — clean, testable, and independent of network concerns.

5.8 Data Access: Database Calls During a Request

Every real API eventually needs to read or write persistent data. In the Notes API, that means SQLite.

The request-scoped database connection

In Listing 4.11, we set up a `get_db()` function that stores the database connection in `g` (the request context object):

Listing 5.12 The `get_db()` helper (from Chapter 4's `app.py`)

```
def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DATABASE_PATH)
        g.db.row_factory = sqlite3.Row
    return g.db
```

The first time `get_db()` is called during a request, it opens a new database connection and stores it in `g.db`. Subsequent calls during the same request return the same connection. The teardown hook closes it once the response has been built.

This pattern is important for two reasons:

1. **Efficiency:** Opening a database connection has overhead (allocating memory, establishing protocol state). Doing it once per request — not once per handler call or per query — is the right balance.
2. **Consistency:** All database operations within a single request share the same connection. This means they share the same transaction. If you execute multiple `INSERT` statements and then `db.commit()`, they all commit together. If an exception occurs before the commit, none of them is saved: the uncommitted transaction is discarded when the teardown hook closes the connection (or explicitly, with `db.rollback()`). This is **atomicity** — a fundamental property of relational databases.

How SQLite processes a query

When `db.execute("INSERT INTO notes (content) VALUES (?)", (content,))` runs:

1. The `sqlite3` module hands SQLite the SQL text and the parameter (content) *separately*: SQLite compiles the statement with a `?` placeholder and then binds the value into it. The value is never parsed as SQL, which is what prevents **SQL injection** (→ [Chapter 38 – Security](#) explains this in detail)
2. SQLite receives the compiled statement and the bound value
3. SQLite parses the SQL, checks it against the schema, and executes it
4. SQLite writes the new row to the database file (`notes.db`) on disk
5. The cursor object returned contains metadata about the operation, including `cursor.lastrowid`

`db.commit()` flushes the transaction to disk. Without `commit()`, the change exists only in memory and will be lost when the connection closes.

The Row factory

The line `g.db.row_factory = sqlite3.Row` changes how SQLite returns query results. Without it, rows are returned as plain tuples: `(1, "Buy groceries", "2024-01-01 12:00:00")`. With `sqlite3.Row`, rows behave like dictionaries: `row["id"]`, `row["content"]`, `row["created_at"]`. This makes the code more readable and allows `dict(row)` to convert a row to a regular Python dictionary for JSON serialization.

5.9 Response Construction: Status Code, Headers, Body

The handler function has done its work and is ready to return a response. Flask takes the handler's return value and constructs a proper HTTP response object.

```
# The handler returns:
return jsonify({"id": 1, "content": "Buy groceries"}), 201
```

Flask decomposes this tuple:

- The first element (`jsonify(...)`) becomes the response body and sets `Content-Type: application/json`
- The second element (`201`) becomes the HTTP status code

Flask assembles the response object:

```
# What Flask builds internally (conceptual):
response = Response(
    response='{"content":"Buy groceries","id":1}\n',    # compact, keys sorted
    status=201,
    headers={
        "Content-Type": "application/json",
        "Content-Length": "35",
    }
)
```

Then `after_request` middleware runs and adds the timing header:

```
response.headers["X-Response-Time-Ms"] = "4.2"
```

The final response object has:

- Status line: HTTP/1.1 201 Created
- Headers including Content-Type, Content-Length, X-Response-Time-Ms
- Body: the JSON string

HTTP status codes

Status codes tell the client what happened:

Code	Meaning	When to use
200	OK	Successful GET, PUT, DELETE
201	Created	Successful POST that created a resource
204	No Content	Successful operation with no response body
400	Bad Request	Client sent invalid data
401	Unauthorized	Authentication required
403	Forbidden	Authenticated but not permitted
404	Not Found	Resource does not exist
405	Method Not Allowed	Method not supported for this path
415	Unsupported Media Type	Wrong Content-Type
422	Unprocessable Entity	Data was parseable but invalid
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unhandled exception in the server

Using the right status code is part of the API’s contract. A client that receives a 400 knows the problem is on its side. A client that receives a 500 knows the problem is on the server’s side. A client that receives a 404 knows the resource does not exist. These distinctions matter for debugging, monitoring, and client behavior.

5.10 Serialization: Turning Python Objects into JSON

The response body must be bytes that can be transmitted over the network. Python objects — dictionaries, lists, integers, strings — cannot be sent directly. They must be **serialized** into a text format that both sides understand.

For APIs, that format is **JSON** (JavaScript Object Notation).

`jsonify()` calls Python’s `json.dumps()` internally:

Listing 5.13 What jsonify does internally

```
import json

python_object = {"id": 1, "content": "Buy groceries"}
json_string = json.dumps(python_object)
# '{"id": 1, "content": "Buy groceries"}'

json_bytes = json_string.encode("utf-8")
# b'{"id": 1, "content": "Buy groceries"}'
```

`json.dumps()` converts a Python dictionary to a JSON string. `encode("utf-8")` converts the string to bytes. The bytes are what gets written to the socket.

What can be serialized

Python's `json` module handles:

- `dict` → JSON object `{}`
- `list`, `tuple` → JSON array `[]`
- `str` → JSON string `""`
- `int`, `float` → JSON number
- `True`, `False` → JSON `true`, `false`
- `None` → JSON `null`

What it cannot handle directly:

- `datetime` objects → Python's `json` module cannot serialize them; Flask's `jsonify()` can, but it produces an HTTP-date string such as `"Mon, 15 Jan 2024 14:00:00 GMT"`, so APIs usually convert them explicitly with `dt.isoformat()`
- `SQLite Row` objects → must be converted to `dict` with `dict(row)`
- Custom Python classes → must be converted manually or with a custom JSON encoder

This is why `dict(row)` appears in the Notes API handlers — `SQLite Row` objects must be converted to plain dictionaries before passing them to `jsonify()`.

Listing 5.14 Serializing SQLite rows correctly

```

@app.route("/notes", methods=["GET"])
def list_notes():
    db = get_db()
    cursor = db.execute("SELECT id, content, created_at FROM notes ORDER BY id DESC")

    # cursor.fetchall() returns a list of Row objects
    rows = cursor.fetchall()

    # Convert each Row to a dict for JSON serialization
    notes = [dict(row) for row in rows]

    # Now notes is a list of plain dicts – jsonify can handle this
    return jsonify({"notes": notes, "count": len(notes)})

```

5.11 Sending the Response: send()

The response object is built. The middleware has run. Now Flask sends it back to the client through the client socket.

Listing 5.15 Sending the response (conceptual, Flask does this internally)

```

response_bytes = (
    b"HTTP/1.1 201 Created\r\n"
    b"Content-Type: application/json\r\n"
    b"Content-Length: 35\r\n"
    b"X-Response-Time-Ms: 4.2\r\n"
    b"\r\n"
    b'{"content":"Buy groceries","id":1}\n'
)

client_socket.sendall(response_bytes)
client_socket.close()

```

`sendall()` is the Python socket method that ensures all bytes are sent, even if the underlying `send()` system call does not transmit everything in one shot (which can happen on a busy network or with large responses). It loops internally until all bytes are written to the socket's send buffer.

The kernel's networking stack takes those bytes from the send buffer, wraps them in TCP packets, and transmits them to the client. The client receives the packets, reassembles them into the original bytes, and passes them to the browser or API client, which parses the HTTP

response.

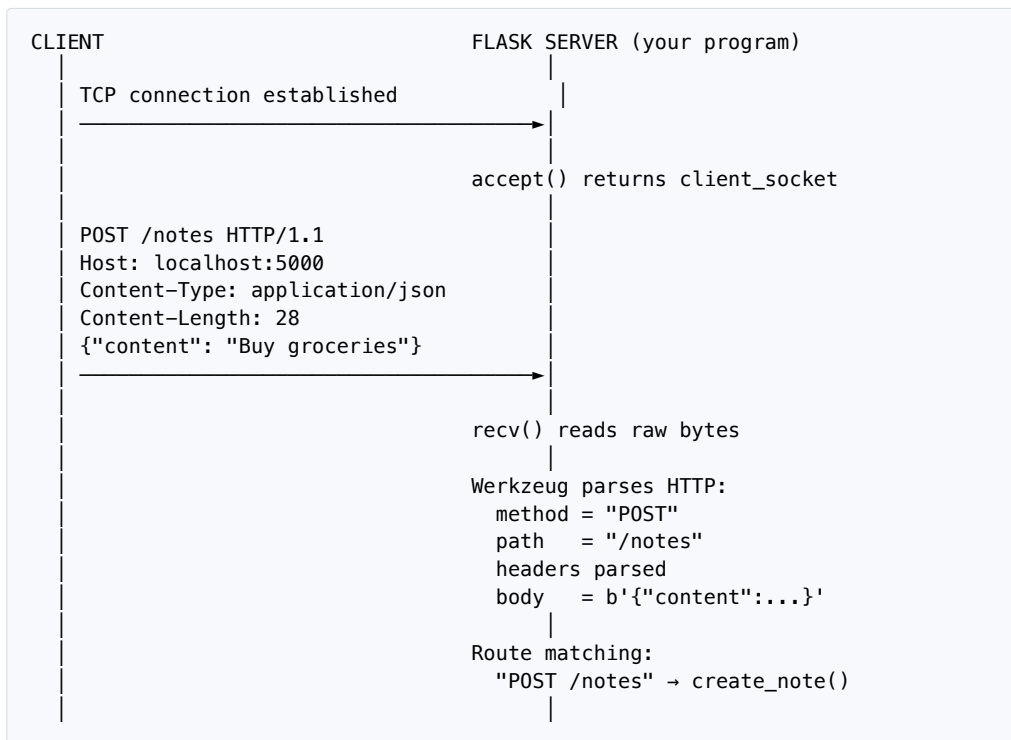
Connection keep-alive

In HTTP/1.1, connections can be reused for multiple requests (keep-alive): if the client sends another request on the same socket, the server handles it without a new connection or another trip through the accept queue. Not every server does this. Flask’s development server closes the connection after each response (it sends `Connection: close`), and so do Gunicorn’s default “sync” workers. In production, keep-alive with clients is usually handled by Nginx in front of the application (→ [Chapter 28](#)).

This is an optimization for performance — establishing a new TCP connection has a cost (the three-way handshake). Reusing a connection for multiple requests amortizes that cost.

The Complete Request Execution Sequence

Putting all eleven steps together for a `POST /notes` request with body `{"content": "Buy groceries"}`:



```

before_request[0]: start_timer()
  g.start_time = now
  |
before_request[1]: require_json_for_writes()
  is_json = True → pass
  |
Handler: create_note()
  request.get_json() → {"content": "Buy..."}
  validate: content not empty ✓
  get_db() → sqlite3 connection
  INSERT INTO notes ...
  commit()
  note_id = 1
  |
Response built:
  jsonify({"id": 1, "content": "..."})
  status = 201
  |
after_request: add_timing_header()
  duration = 4.2ms
  X-Response-Time-Ms: 4.2 added
  |
teardown_request: close_db()
  g.db.close()
  |
sendall() writes response bytes
  |
← HTTP/1.1 201 Created
  Content-Type: application/json
  X-Response-Time-Ms: 4.2
  {"content":"Buy groceries","id":1}
  |
[connection closed or reused]
  |
accept() waits for next connection

```

Key Takeaways

- **accept()** retrieves the next completed connection from the accept queue. It returns a new client socket (for communication with this specific client) and the client's address.
- **recv()** reads raw bytes from the client socket. These bytes are the HTTP request in wire format — method, path, headers, body.
- **HTTP parsing** converts the raw bytes into a structured Request object. Werkzeug handles this. The result provides clean access to method, path, headers, query parameters, and body.
- **Route matching** uses Werkzeug's Map to find the right handler function for the request's

method and path combination. If no match exists, Flask returns 404 or 405 automatically.

- **Middleware** (`before_request`) runs first. If any middleware function returns a value, the handler is skipped and that value becomes the response.
- **The handler function** does the application's real work: validate input, run business logic, access the database, and return a result.
- **Database access** during a request uses a request-scoped connection stored in `g`. The connection is opened lazily (on first use) and closed by a teardown hook once the response has been built.
- **Serialization** converts Python objects to JSON bytes using `json.dumps()`. SQLite Row objects must be converted to dicts first.
- **`sendall()`** writes the complete response — status line, headers, body — back through the client socket to the client.
- The **`after_request`** and **`teardown_request`** hooks always run, even if the handler raised an exception. Teardown hooks are the right place for cleanup.

Exercises

1. **Predict.** With `printf` and `nc`, send a request with the header `X-Note: a\r\nX-Admin: yes`, that is, a value followed by `\r\n` and a second, invented header. What does the server see? Then try to *set* such a header value in a Flask response. What happens?
2. **Break and fix.** Make a handler return a Python `set`. What error do you get, and where? Change it to return a JSON-serializable type.
3. **Extend.** Add a `before_request` hook that rejects requests whose `User-Agent` is empty, with a JSON `400`. Test it with `curl -A ""`.
4. **Explain.** Trace, in your own words, the eleven steps a request goes through between `accept()` and `sendall()`.

Solutions and hints are in the companion repository, in `exercises/chapter-05.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

The local API works. You can run it, test it, and watch it create and retrieve notes. But everything you have built so far depends on hidden assumptions about your machine: a specific Python version, specific packages installed globally, a specific file path for the database, a loopback address that is not reachable from anywhere else. Before you can move this system anywhere — another machine, another developer's laptop, a server — those assumptions must

be made visible.

→ [Chapter 6](#) — Local Dependencies and Resources

Local Dependencies and Resources

The Notes API works. You have run it, tested it, sent requests to it, and watched it store notes in a SQLite database. From your perspective, sitting at your laptop, it works perfectly.

Here is what you might not have noticed: your working API is tightly coupled to your machine.

It relies on a specific version of Python — whichever one is installed on your laptop. It relies on Flask being installed in the right place, accessible to that Python version. It relies on a SQLite database at a file path that is meaningful only on your machine. It relies on environment variables that exist only in your shell session. It relies on `pip` and `venv` (or their absence) being set up exactly the way you set them up, at some point in the past, possibly without documentation.

None of this is visible in the code. None of it is in `app.py`. It exists in the silent agreement between your code and your machine — an agreement that will break the moment the code moves to a different machine.

This chapter makes that hidden agreement visible. It names every resource the Notes API depends on, explains exactly what that dependency is, and shows how each one is managed locally. Naming these dependencies is the first step to controlling them. Controlling them is the first step to portability (→ Stage 3 — Making the Program Portable).

In this chapter

By the end of this chapter, you will understand:

- What “dependencies” means at each layer: the runtime, the libraries, the virtual environment
- How Python’s package manager (pip) installs and locates packages
- What a virtual environment is, why it exists, and how to use one
- How your program accesses the local filesystem and database
- How to **design the database schema** deliberately — entities and relationships, primary keys, normalization, audit columns, soft vs hard deletes, and designing for change
- What environment variables are, how they reach your program, and why they matter for configuration

6.1 Runtime Dependencies: What Python Itself Depends On

Before Flask. Before SQLite. Before any of your code. Your program needs the Python interpreter itself.

Python’s own dependencies

The CPython interpreter is not self-contained. It depends on:

The C standard library (libc): CPython is written in C. It links against your operating system’s C library for low-level operations: memory allocation, file access, process management, and more. On Linux, this is typically glibc. On macOS, it is Apple’s libSystem. On different systems, different versions of libc are present.

OpenSSL: CPython’s `ssl` module (used by HTTPS connections) links against OpenSSL. The version of OpenSSL on the system affects which TLS versions and cipher suites Python supports. A system with an old OpenSSL version may fail to connect to servers that require TLS 1.3, for example.

zlib: Used for compression, which underlies Python’s `gzip` and `zipimport` (.zip file imports).

SQLite’s C library: Python’s built-in `sqlite3` module links against `libsqlite3`. On most systems it is present. On a minimal Linux Docker image, it may not be.

readline or libedit: Used by the interactive Python interpreter (the REPL) for command-line editing. Not strictly required for server applications, but sometimes a missing dependency causes confusing build failures.

None of these appear in your `app.py` or in any requirements file. They are system-level dependencies — assumed to exist, never declared, and perfectly capable of being absent on a

different machine or a minimal container image.

Checking Python's dependencies

On Linux, you can see which shared libraries a program links against with `ldd`. The `python3` executable itself links only a few core libraries; most of the libraries above are loaded by Python's *extension modules* — compiled `.so` files such as `_ssl` and `_sqlite3` — the first time you import `ssl` or `sqlite3`. So check those modules:

```
ldd $(python3 -c "import _ssl; print(_ssl.__file__)")
ldd $(python3 -c "import _sqlite3; print(_sqlite3.__file__)")
```

Output (typical Ubuntu, abbreviated):

```
libssl.so.3 => /lib/x86_64-linux-gnu/libssl.so.3
libcrypto.so.3 => /lib/x86_64-linux-gnu/libcrypto.so.3
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
...
libsqlite3.so.0 => /lib/x86_64-linux-gnu/libsqlite3.so.0
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
```

Each of these `.so` files is a shared library. If one of them is missing or is the wrong version, Python still starts — but `import ssl` or `import sqlite3` fails the moment your code needs it.

On macOS, the equivalent command is `otool -L`.

Why this matters

When you deploy to a server (→ [Chapter 21](#) — System Setup), the server may be running a different Linux distribution, a different version of the same distribution, or a minimal container image with fewer libraries installed. A Flask application that runs perfectly on Ubuntu 22.04 may fail to start on Alpine Linux because Alpine uses `musl libc` instead of `glibc`, and some Python packages compiled against `glibc` will not work on `musl`.

This class of problem — where system-level libraries do not match — is one of the main reasons Docker containers (→ [Chapter 18](#) — Containerization) became so popular. A container bundles the application and all its system-level dependencies together, making the system-level environment reproducible.

6.2 Library Dependencies: pip and the Package Ecosystem

The Notes API depends on Flask. Flask itself depends on Werkzeug, Jinja2, Click, ItsDangerous, and Blinker. Each of those packages may have their own dependencies. The result is a **dependency tree** — a hierarchy of packages that your application needs to run.

pip is Python’s package installer. It downloads packages from the **Python Package Index (PyPI)** — a public registry of hundreds of thousands of Python packages — and installs them so the Python interpreter can find them.

How pip install flask works

When you run:

```
pip install flask
```

pip does the following:

1. **Queries PyPI** for the Flask package metadata
2. **Resolves the dependency tree**: Flask requires Werkzeug, Jinja2, etc. pip resolves which versions of each are compatible with each other
3. **Downloads wheel files**: Modern packages are distributed as .whl (wheel) files — pre-compiled archives that can be installed without building from source
4. **Installs packages** into the active Python environment’s `site-packages` directory
5. **Records the installation** in a file called `RECORD` inside the package’s metadata directory

After this, `import flask` works because Python’s `sys.path` includes the `site-packages` directory where Flask was installed.

Where packages are installed

Outside a virtual environment, `pip install` places packages in your Python installation’s `site-packages` directory:

- For a Python installed from python.org or Homebrew on macOS: `/Library/Frameworks/Python.framework/Versions/3.x/lib/python3.x/site-packages/` or `/opt/homebrew/lib/python3.x/site-packages/`
- For the system Python on Linux: `/usr/lib/python3/dist-packages/` (managed by the OS package manager) or `/usr/local/lib/python3.x/site-packages/`
- For a per-user install (`pip install --user`): `/home/yourname/.local/lib/python3.x/site-packages/`

Note: On current Debian, Ubuntu (23.04 and later), and Homebrew installations, a plain `pip install` outside a virtual environment is refused with error: `externally-managed-environment` (PEP 668). The operating system protects its own Python from packages installed with `pip`. The fix is the one this chapter recommends anyway: create a virtual environment (Section 6.3) and install packages there.

You can check where a package is installed:

Listing 6.1 Finding where a package is installed

```
import flask
print(flask.__file__)
# /home/yourname/.local/lib/python3.11/site-packages/flask/__init__.py
```

The path tells you which Python installation owns this Flask, and from which directory it was loaded.

The global install problem

If you install Flask globally (without a virtual environment), it is available to all Python programs on your machine that use the same Python interpreter. This sounds convenient. It is actually a problem.

Imagine two projects on your machine:

- **Project A** requires Flask 2.3
- **Project B** requires Flask 3.0

These two requirements conflict. `pip install` can only install one version of Flask at a time globally. Installing Flask 3.0 for Project B will break Project A if Flask 3.0 has incompatible changes.

This is the fundamental dependency isolation problem, and it is solved by virtual environments (Section 6.3).

6.3 Virtual Environments: Isolating Your Project's Dependencies

A **virtual environment** (`venv`) is a self-contained directory that holds a specific Python interpreter and a set of packages, isolated from the system-wide Python installation and from other virtual environments.

When a virtual environment is active, `pip install` installs packages into the virtual environment's `site-packages`, not the system-wide one. `python` refers to the interpreter inside

the virtual environment. `import flask` finds the Flask version inside the virtual environment.

Creating and activating a virtual environment

```
# Step 1: Create the virtual environment  
# "venv" is the directory name – conventional, but you can choose any name  
python3 -m venv venv  
  
# Step 2: Activate the virtual environment  
# On macOS / Linux:  
source venv/bin/activate  
  
# On Windows (PowerShell):  
# venv\Scripts\Activate.ps1
```

After activation, your terminal prompt changes to show the active environment:

```
(venv) yourname@machine:~/notes-api$
```

The `(venv)` prefix tells you the virtual environment is active. Now:

- `python` refers to `~/notes-api/venv/bin/python`
- `pip` refers to `~/notes-api/venv/bin/pip`
- Packages installed with `pip` go to `~/notes-api/venv/lib/python3.11/site-packages/`

What the venv directory contains

```

notes-api/
├── venv/
│   ├── bin/
│   │   ├── activate      ← the script you `source` to activate the venv
│   │   ├── python        ← symlink to the system Python interpreter
│   │   ├── python3       ← symlink to the same interpreter
│   │   ├── pip           ← venv-specific pip
│   │   └── flask         ← Flask's CLI entry point (added when Flask is installed)
│   ├── lib/
│   │   └── python3.11/
│   │       ├── site-packages/
│   │       │   ├── flask/      ← Flask and its files
│   │       │   ├── werkzeug/   ← Werkzeug
│   │       │   └── jinja2/     ← Jinja2
│   │       └── ...
│   └── pyvenv.cfg           ← Configuration: which Python version, home path

```

The `bin/python` is typically a symlink to the system Python interpreter — the virtual environment does not copy Python, it just uses the system one. What it isolates is the `site-packages` — each venv has its own, independent set of installed packages.

Installing Flask in the virtual environment

With the virtual environment active:

```
pip install flask
```

This installs Flask into `venv/lib/python3.11/site-packages/`. It does not touch the system-wide Python installation.

You can verify:

Listing 6.2 Verifying that Flask comes from the virtual environment

```

import flask
print(flask.__file__)
# /home/yourname/notes-api/venv/lib/python3.11/site-packages/flask/__init__.py
#                                     ^^^^ inside the venv

```

Deactivating the virtual environment

```
deactivate
```

After deactivating, `python` and `pip` refer back to the system-wide versions. Flask, if installed only in the `venv`, is no longer accessible.

Why you must always use a virtual environment

For every project, create a virtual environment. Not sometimes. Always.

- It prevents version conflicts between projects
- It makes it easy to see exactly which packages are installed for the project (`pip freeze`)
- It makes the project portable: when you move to another machine, you can recreate the exact same environment from a list of packages (→ [Chapter 12](#) — Dependency Declaration)
- It prevents accidentally breaking system tools that rely on specific package versions
- It is the industry standard: every professional Python project uses virtual environments

The only exception is when using Docker (→ [Chapter 18](#) — Containerization). Inside a Docker container, the container itself provides isolation, so a virtual environment inside a container is redundant (though not harmful).

.gitignore and the virtual environment

The `venv/` directory should **not** be committed to version control. It is large, platform-specific, and regenerable. Add it to `.gitignore`:

```
# .gitignore
venv/
__pycache__/
*.pyc
# Exclude the SQLite database from version control
*.db
# Exclude the environment variable file (it contains secrets)
.env
```

`.gitignore` does not support comments at the end of a line: in `*.db # comment`, the whole line is the pattern, and it matches nothing. Put comments on their own lines, starting with `#`.

The `venv/` directory is a local artifact, not part of the project's source code. The project's source code plus a dependency list (→ [Chapter 12](#)) is sufficient to recreate the `venv` on any machine.

6.4 Local Databases and the Filesystem

The Notes API stores notes in a SQLite database file. SQLite is a **file-based database** — all data is stored in a single file on disk. No network connection, no server process, no configuration. The database is just a file.

This is the simplest possible database setup, and it is appropriate for local development. The database file is created automatically when the application starts:

```
# From app.py (Chapter 4)
DATABASE_PATH = os.getenv("DATABASE_PATH", "notes.db")

def init_db():
    conn = sqlite3.connect(DATABASE_PATH) # Creates notes.db if it does not exist
    conn.execute("""
        CREATE TABLE IF NOT EXISTS notes (...)
    """)
    conn.commit()
    conn.close()
```

`sqlite3.connect("notes.db")` creates the file `notes.db` in the current working directory if it does not already exist. The database schema is created by the `CREATE TABLE IF NOT EXISTS` statement.

That three-column table looks too obvious to think about — and that is exactly why it is worth stopping on. The **schema** (the set of tables, columns, types, keys, and relationships) is the most consequential design decision in the whole application, because it outlives everything else. You can rewrite `app.py` over a weekend; you cannot casually restructure a table that has a million rows and live traffic. The application is a guest in the data's house. So before we move on, let us design that house on purpose.

The schema is a design decision, not a default

Every column in the notes table encodes a choice:

```
CREATE TABLE notes (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,  -- identity: how is a row uniquely
  ↪      named?
  content TEXT NOT NULL,                      -- the data, and a rule: it may
  ↪      not be missing
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- metadata about the row, not
  ↪      the note
);
```

Why a separate `id` and not just the `content`? Why is `content` `NOT NULL` but `created_at` defaulted? Why store a timestamp at all? Good schema design is the habit of being able to answer “why these columns, these types, this key” for every table you create or inherit. The rest of this section is the toolkit for answering it.

Entities, attributes, and relationships

Schema design starts before SQL, with a small model of the world. An **entity** is a kind of thing you need to remember — here, a *note*. Its **attributes** are the facts about it — its content, when it was created. A row is one specific note; the table is all notes.

Real systems have more than one entity, and entities **relate**. Imagine the Notes API grows: notes belong to *users*, and notes can be tagged. Now you have three entities and two relationships, and each relationship has a **cardinality** — how many of one side connect to the other:

- **One-to-many**: one user has many notes; each note has exactly one author. You model this by putting a `user_id` column on notes that points at a row in users — a **foreign key**.
- **Many-to-many**: a note can have many tags, and a tag can apply to many notes. You cannot express this with a column on either side; you create a third **join table**, `note_tags(note_id, tag_id)`, whose rows *are* the connections.

```
-- The Notes API, grown up: three entities, two relationships
CREATE TABLE users (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  email   TEXT NOT NULL UNIQUE
);
CREATE TABLE notes (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL REFERENCES users(id),  -- one user → many notes
  content TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```

CREATE TABLE tags (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL UNIQUE
);
CREATE TABLE note_tags (                                -- many notes ↔ many tags
  note_id INTEGER NOT NULL REFERENCES notes(id),
  tag_id INTEGER NOT NULL REFERENCES tags(id),
  PRIMARY KEY (note_id, tag_id)
);

```

Sketching entities, attributes, and cardinalities — on paper or as an ER diagram — is how you turn a vague “we need to store X” into tables that fit the domain. Get the relationships right and the queries write themselves; get them wrong and you fight the schema forever.

Choosing a primary key

Every row needs a stable, unique name: its **primary key**. There are two families, and the choice has long consequences.

A **natural key** is a real-world attribute that is already unique — a user’s email, an ISBN, a country code. A **surrogate key** is a meaningless identifier the system invents purely to name the row — our `id`. Prefer surrogate keys for anything that represents a real-world entity, because *natural facts change*: people change their email, products get renumbered, and the moment a “unique” real-world value changes, every row and foreign key that referenced it has to change too. A surrogate key never changes, because it never meant anything in the first place.

Among surrogate keys there is a further choice:

- **Auto-increment integers** (AUTOINCREMENT/SERIAL): small, fast, naturally ordered, human-readable. But they are *guessable* (exposing `/notes/41` tells the world you have at least 41 notes), and they **collide** if you ever merge two databases or shard across machines, because both start counting at 1.
- **UUIDs**: 128-bit random identifiers, globally unique with no coordination — safe to generate on the client, across shards, or before the row is even saved. The cost: they are large and random, which hurts index locality and makes inserts slower at scale.
- **ULIDs** (and UUIDv7): the modern compromise — globally unique like a UUID, but time-ordered like an auto-increment, so they index well *and* don’t leak counts.

The Notes API uses auto-increment because it is one SQLite file on one machine — the simplest thing that works. The point is to know that this is a *decision*, so that when you later expose IDs publicly or split the database, you recognize the moment to revisit it.

Normalization: every fact in exactly one place

Normalization is organizing columns so that each fact lives in one and only one place. It exists to prevent **anomalies**. Picture a denormalized `notes` table that stores the author’s details on every note:

id	content	author_name	author_email
1	Buy milk	Dana	dana@x.com
2	Call Sam	Dana	dana@x.com

The duplication is not just wasteful, it is dangerous:

- **Update anomaly:** Dana changes her email — now you must find and update *every* note she ever wrote, and if you miss one the data contradicts itself.
- **Insertion anomaly:** you cannot record a new user until they have written at least one note (there is nowhere else to put them).
- **Deletion anomaly:** delete Dana’s last note and you silently erase the fact that Dana exists.

The fix is the `users` table from earlier: store each author *once*, and have notes reference them by `user_id`. The informal ladder is worth memorizing:

- **First normal form (1NF):** values are atomic — one fact per cell. Do **not** store tags as a comma-separated string "`work,urgent`"; that is a repeating group hiding in a column. Use rows (the join table) instead.
- **Second & third normal form (2NF/3NF):** every non-key column depends on *the key, the whole key, and nothing but the key*. `author_email` depends on the user, not on the note, so it does not belong on notes.

Normalization optimizes for correct, non-contradictory writes. Occasionally you will deliberately **denormalize** — duplicate a value for read speed, like caching a `note_count` on each user to avoid counting rows on every page load. That is legitimate, but it is a trade: you have just taken personal responsibility for keeping the copy in sync, and a stale denormalized value is a bug waiting to happen. Normalize by default; denormalize consciously, with a reason.

Time and audit columns

`created_at` is not part of the note — it is **metadata about the row**: a fact about when the record came to exist. Columns like this are **audit columns**, and the common set is `created_at`, `updated_at`, and sometimes `created_by`. They cost almost nothing, and they answer “when

did this happen, and who did it?” for the rest of the table’s life. Add them by default; you will never regret having them, and you cannot reconstruct them after the fact.

Time itself is deceptively hard. SQLite’s `CURRENT_TIMESTAMP` records the current time in UTC, but stores it as plain text with no timezone marker (`2024-01-15 14:00:00`), so nothing in the value says it is UTC — and code that writes its own timestamps easily mixes in local time. That is harmless on your laptop and a latent bug in production, where a note created in Tokyo and read in New York must still refer to the same instant. The rule is **store UTC, render local**, and use a timezone-aware type in production (`TIMESTAMP WITH TIME ZONE` in PostgreSQL — → [Chapter 34](#) — Response Handling covers this when we serialize timestamps). Choosing the right time type now is free; migrating millions of ambiguous timestamps later is not.

Deleting: soft vs hard

When a user deletes note 41, what happens to the row? Two answers:

- **Hard delete:** `DELETE FROM notes WHERE id = 41`. The row is gone, the space is reclaimed, every query stays simple. But the data — and any history of it — is unrecoverable.
- **Soft delete:** add a `deleted_at` column, set it to “now,” and keep the row. Now you can undo, audit, and report on deletions. The price is that *every* query must remember `WHERE deleted_at IS NULL` (forget once and deleted notes reappear), and the user’s content still physically sits in your table, which matters for privacy and retention (→ [Chapter 37](#) — Reliability, where we cover data management).

Neither is universally correct: hard-delete low-stakes data and anything a user has a right to have truly erased; soft-delete when undo or audit history is genuinely valuable. What matters is choosing on purpose — because retrofitting soft delete onto a table after you have already hard-deleted the rows is impossible; the data is simply gone.

Schemas change: designing for evolution

The one certainty about a schema is that it will change — a new feature needs a `title`, an `is_pinned` flag, a `user_id`. The skill is changing it *without downtime and without breaking the running version*.

Changes come in two kinds. **Additive** changes are safe: adding a *nullable* column, or one with a default, leaves every existing row and the currently-deployed code working untouched. **Breaking** changes are not: renaming or dropping a column, or making an existing one `NOT NULL`, will break either old rows or old code the instant they land. The professional pattern for a breaking change is **expand/contract** (also called parallel change):

1. **Expand** — add the new column alongside the old, and have the application write to both.
2. **Migrate** — backfill the existing rows so the new column is fully populated.
3. **Contract** — switch reads to the new column, stop writing the old one, and only then drop it.

Each step is independently deployable, so the database and the application are never out of sync at any single moment — that is what lets you change schemas on a live system with zero downtime. Schema changes themselves are written as **migrations**: small, ordered, versioned SQL scripts committed to the repository and applied in sequence, so that every environment — your laptop, staging, production — reaches the same schema the same way. The golden rule is never to alter a production table by hand. (Configuration-driven databases arrive in → [Chapter 13](#) — Configuration Externalization, and connection and transaction handling in → [Chapter 33](#) — Logic Execution.)

Reading someone else’s schema

For most of your career you will *inherit* schemas far more often than you design them, and reading one fluently is how you learn an unfamiliar system fastest — often faster than reading its code. Dump the schema (`.schema` in SQLite, `\d` in `psql`) and interrogate each table with the same questions you now know to ask:

- What **entity** is this table about?
- What is the **primary key** — and is it surrogate or natural?
- Which columns are **foreign keys**? Those are the relationships; together they are the map of the domain.
- What is **NOT NULL** (required) versus nullable (optional)? The constraints tell you the business rules.
- What do the **audit columns** reveal about how rows are created and changed?

Answer those for every table and you have reconstructed the original designer’s mental model of the domain, constraints and all.

Note — when not to use a relational table at all. Relational tables fit data with clear structure and relationships, which is most business data, and certainly the Notes API. But other shapes exist: document stores for nested, schema-flexible data; key-value stores for caches and sessions; time-series databases for metrics; search indexes for full-text. The skill is recognizing when one of these genuinely earns its place — and resisting the temptation to reach for an exotic store before the relational model has actually failed you.

Where the database file lives

The database is created at the **current working directory** when the server starts. If you start the server from `~/notes-api/`, the database is at `~/notes-api/notes.db`.

```
cd ~/notes-api
python app.py
# notes.db is created at ~/notes-api/notes.db

ls -lh notes.db
# -rw-r--r-- 1 yourname staff 12K notes.db
```

After creating a few notes and stopping the server, the data persists in the file. When you restart the server, the notes are still there because the file is still there.

Filesystem dependencies

The application depends on the filesystem in several ways:

1. **The database file location:** The application assumes it can write to the current directory. On some servers, the application may run in a read-only directory, or the current directory may not be writable. This assumption is hidden.
2. **File permissions:** The application's user account must have write permission to the directory where `notes.db` lives. On a server with restricted permissions, this may not be the case.
3. **Disk space:** The database file grows as notes are added. If the disk fills up, write operations will fail.
4. **Absolute vs relative paths:** "`notes.db`" is a relative path. Its meaning depends on the current working directory when the server starts — which may differ between your laptop and a server.

These are local filesystem assumptions — perfectly reasonable for development, but fragile in deployment. We address this with configuration externalization in → [Chapter 13 — Configuration Externalization](#).

Viewing and querying the database manually

SQLite includes a command-line tool for inspecting databases directly:

```
sqlite3 notes.db
```

Inside the SQLite shell:

```
.tables
-- notes

.schema notes
-- CREATE TABLE notes (
--   id INTEGER PRIMARY KEY AUTOINCREMENT,
--   content TEXT NOT NULL,
--   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
-- );

SELECT * FROM notes;
-- 1|Buy groceries|2024-01-01 12:00:00
-- 2|Learn Flask|2024-01-01 12:05:00

.exit
```

Being able to inspect the database directly — independently of the application — is essential for debugging. If a request seems to be failing, checking the database state directly tells you whether the problem is in the database or in the code.

6.5 Environment Variables and the .env File

The Notes API has one configurable value at this stage: `DATABASE_PATH`. It is read from an environment variable:

```
DATABASE_PATH = os.getenv("DATABASE_PATH", "notes.db")
```

An **environment variable** is a named value stored in the process’s environment — the key-value pairs inherited from the parent process when the program started (← [Section 1.3](#) — how a new process receives its environment).

What environment variables are

Every process on a Unix-like system has an **environment** — a set of key-value pairs that the process can read. These are separate from the program’s variables. They are set by the shell or by the parent process before the program starts.

Common environment variables you already have:

```
echo $HOME      # /home/yourname
echo $PATH      # /usr/local/bin:/usr/bin:/bin:...
echo $USER      # yourname
echo $SHELL     # /bin/bash
```

These were set when you logged in, by the system's login scripts. Every process you run inherits them.

You can create your own:

```
export DATABASE_PATH=/var/data/notes.db
python app.py
# DATABASE_PATH is now in app.py's environment
```

Or set one only for a single command:

```
DATABASE_PATH=/tmp/test.db python app.py
# DATABASE_PATH is set only for this run; your shell environment is unchanged
```

Reading environment variables in Python

Listing 6.3 Accessing environment variables

```
import os

# Get a value; returns None if not set
value = os.environ.get("DATABASE_PATH")

# Get a value with a default
value = os.getenv("DATABASE_PATH", "notes.db")

# Get a value; raises KeyError if not set
value = os.environ["DATABASE_PATH"]

# Check if a variable is set
is_set = "DATABASE_PATH" in os.environ

# Print all environment variables
for key, val in os.environ.items():
    print(f"{key}={val}")
```

`os.environ` is a dictionary-like object that maps environment variable names to their values.

It is read from the process's environment when the Python interpreter starts. Changes made to `os.environ` at runtime affect only the current process (and any child processes it spawns) — they do not persist in the shell.

The `.env` file and `python-dotenv`

Setting environment variables by exporting them in the shell is tedious for development. You would need to remember to export them every time you open a terminal. The solution is the `.env` file.

A `.env` file is a plain text file that lists environment variables in `KEY=VALUE` format:

```
# .env
DATABASE_PATH=notes.db
FLASK_DEBUG=true
API_KEY=dev-secret-key-not-for-production
```

The `python-dotenv` library reads this file and loads its contents into `os.environ` when your application starts:

```
pip install python-dotenv
```

Listing 6.4 Loading a `.env` file with `python-dotenv`

```
from dotenv import load_dotenv
import os

# Find the nearest .env file (searching upward from this file's directory)
# and load it into os.environ
load_dotenv()

# Now these variables are in os.environ
DATABASE_PATH = os.getenv("DATABASE_PATH", "notes.db")
```

You can also use Flask's integration with `python-dotenv`. When `python-dotenv` is installed, the `flask run` command (and `app.run()` in debug mode) loads a `.env` file automatically. The explicit `load_dotenv()` call matters most for other ways of starting the app — a production server such as Gunicorn does not load `.env` files:

```
flask run
# Flask automatically loads .env before starting
```

The updated `app.py` with `python-dotenv`

Listing 6.5 Updated app.py with python-dotenv

```

from dotenv import load_dotenv
load_dotenv() # Must be called before os.getenv

from flask import Flask, request, jsonify, g
import sqlite3
import os
import time

app = Flask(__name__)
DATABASE_PATH = os.getenv("DATABASE_PATH", "notes.db")

# ... rest of the application (unchanged from Listing 4.11)

```

The `load_dotenv()` call must come before any `os.getenv()` calls, because it populates `os.environ` from the `.env` file.

Why .env files must never be committed to version control

The `.env` file often contains **secrets**: API keys, database passwords, signing tokens. If you commit the `.env` file to Git, those secrets become part of the repository history — visible to anyone with access to the repository, and very difficult to truly remove (even deleting the file does not remove it from the git history).

The `.env` file belongs in `.gitignore`. Every developer or environment creates their own `.env` file with values appropriate to their context. The `.env.example` file — a version of `.env` with placeholder values, committed to version control — documents which variables are needed without exposing actual secrets:

```

# .env.example (committed to version control)
DATABASE_PATH=notes.db
FLASK_DEBUG=false
API_KEY=your-api-key-here

```

```

# .env (NOT committed – your actual values)
DATABASE_PATH=/home/yourname/notes-api/notes.db
FLASK_DEBUG=true
API_KEY=sk_dev_abc123...

```

Every developer who clones the repository copies `.env.example` to `.env` and fills in their own values.

Why environment variables instead of a config file

You might wonder: why not just use a configuration file (a Python file or JSON file) instead of environment variables? The answer comes in three parts:

1. **Separation of code and config:** Configuration is not code. It changes per environment (development, staging, production) while the code stays the same. Environment variables make this separation explicit.
2. **Security:** Configuration injected at runtime is not part of the code. The `.env` file is only a local convenience and stays out of Git, while in production the values come from the server's environment or a secrets manager. (Environment variables are not a vault, though: anyone who can inspect the process, or `docker inspect` a container, can read them — → [Chapter 38](#).)
3. **Standard:** Environment variables are the standard mechanism for injecting configuration into processes in containerized and cloud environments. Docker, Kubernetes, AWS Elastic Beanstalk, Heroku — all of them use environment variables to configure applications. A program that reads its config from environment variables will work with all of these systems.

We explore this in depth in → [Chapter 13](#) — Configuration Externalization.

The Full Dependency Map of the Notes API

At this stage of development, the Notes API depends on the following things existing exactly right on your machine:

Notes API Dependency Map

```

Layer 1: System (silent, undeclared)
- Operating system: macOS / Linux (specific version)
- C standard library (glibc or musl)
- OpenSSL
- SQLite C library (libsqlite3)
- Disk: write access to the current directory

Layer 2: Runtime (partially declared)
- Python 3.11+ interpreter
- Path: whichever Python $PATH finds first

Layer 3: Virtual environment (locally managed)
- venv/ directory
- Contains: Flask, Werkzeug, Jinja2, Click, etc.
- Tied to: Layer 2 Python version

Layer 4: Application configuration (externalized)
- DATABASE_PATH (from .env or environment)
- FLASK_DEBUG (from .env or environment)
- API_KEY (from .env or environment)

Layer 5: Data (local file)
- notes.db (SQLite file in CWD)
- Grows as notes are created
- Not version controlled

```

That is five layers: four kinds of dependency the code needs in order to run, plus the data it works on. None of Layer 1 is written down anywhere. Layers 2 and 3 are partially managed through `pip` and `venv` but not yet documented in a `requirements.txt` file. Layer 4 exists in a `.env` file that is not committed. Layer 5 is a local file that does not exist on any other machine.

This is what “working locally” actually means: a web of assumptions about the local machine, none of which are guaranteed to hold anywhere else.

Setting Up the Notes API from Scratch

To see what is required to recreate the Notes API on a fresh machine (or a fresh clone of the repository), here is the complete setup sequence:

```

# 1. Get the code (clone the repository, or copy app.py into a new directory)
git clone https://github.com/yourname/notes-api.git && cd notes-api

# 2. Create a virtual environment
python3 -m venv venv

```

```

# 3. Activate the virtual environment
source venv/bin/activate    # macOS/Linux

# 4. Install dependencies
pip install flask python-dotenv

# 5. Create the .env file (not committed – each machine needs its own)
cat > .env << EOF
DATABASE_PATH=notes.db
FLASK_DEBUG=true
EOF

# 6. Run the server
python app.py

```

Step 4 works only because you remembered which packages to install. Nothing in the project records that list — the next person has to guess it. This is the problem that `requirements.txt` solves (→ [Chapter 12](#) — Dependency Declaration).

Key Takeaways

- Your application has **four layers of dependencies** — system libraries (silent), Python runtime, Python packages, and configuration — plus a fifth layer, its **data**. Each must be present and correct for the application to run.
- **pip install** downloads packages from PyPI and installs them in the active Python environment’s `site-packages` directory. Without a virtual environment, this installs globally and can create conflicts between projects.
- A **virtual environment** (`python3 -m venv venv`) creates an isolated Python environment for one project. Always use one. Add `venv/` to `.gitignore`.
- SQLite stores data in a **local file** (`notes.db`) in the current working directory. This is fine for development but is a machine-specific artifact — it does not belong in version control.
- **The schema is the most durable design decision in the system** — the application is a guest in the data’s house. Model **entities, attributes, and relationships** (with their cardinality) before writing SQL; the foreign keys *are* the relationships.
- Prefer **surrogate primary keys** (a meaningless id) over natural keys, because real-world facts change. Know the trade-offs between auto-increment, UUID, and ULID before you expose IDs publicly or shard.
- **Normalize so every fact lives in exactly one place** (the key, the whole key, and nothing but the key) to prevent update, insertion, and deletion anomalies. Denormalize only deliberately, accepting the duty to keep the copy in sync.
- Add **audit columns** (`created_at`, `updated_at`) by default, store time as **UTC with a**

timezone-aware type, and choose **soft vs hard delete** on purpose — these are cheap now and impossible to reconstruct later.

- Schemas always change: make **additive** changes safely, and use **expand/contract migrations** (versioned, in the repo) for breaking ones so a live system never has downtime. Reading an unfamiliar schema — keys, foreign keys, constraints, audit columns — is the fastest way to understand a system you have inherited.
- **Environment variables** are key-value pairs inherited from the parent process. They are the standard mechanism for injecting configuration into a running program without hardcoding values.
- The **.env file** (loaded by `python-dotenv`) is a convenient way to manage environment variables during local development. It must **never** be committed to version control. Use `.env.example` as a template.
- The full set of local dependencies — system libraries, Python version, installed packages, database file, environment variables — is the “working on my machine” assumption. Making these dependencies explicit is the first step toward portability.

Exercises

1. **Predict.** Deactivate your virtual environment and run `python3 -c "import flask"`. What happens, and what does that tell you about where packages live?
2. **Break and fix.** Rename your `.env` file and start the application. What fails, and how quickly? Restore it.
3. **Extend.** Sketch (on paper, as tables with keys) the schema for notes with tags and authors. Mark each primary key, each foreign key, and the cardinality of each relationship.
4. **Explain.** Explain the difference between a soft delete and a hard delete, and give one situation where each is the right choice.

Solutions and hints are in the companion repository, in `exercises/chapter-06.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What’s Next

Stage 1 is complete. You have a working local API. You understand how it starts, how it listens for connections, how it handles requests, and what it depends on.

Stage 2 is about confronting everything that makes this local setup fragile. The next five chapters examine, one by one, the ways in which “it works on my machine” can fail everywhere else.

→ Chapter 7 — Environment Drift

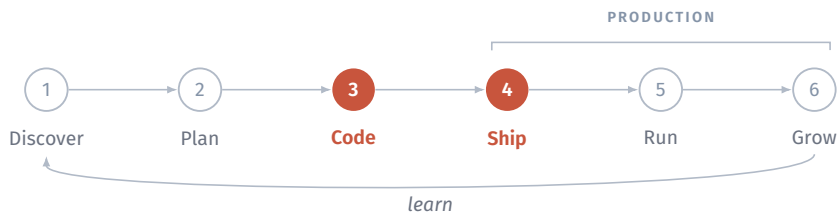
STAGE 2

2

Breaking the Local Illusion

Why is the local system not enough?

YOU ARE HERE



The piece: The gap between your laptop and a real server

The whole map: *How Software Gets Built*, page 1

Chapters 7–11

Environment Drift

Stage 1 ended with a working Notes API. The application runs, handles requests, stores data. You tested it thoroughly with `curl`, Postman, and the browser. Everything works.

Stage 2 starts with a question: what would happen if you gave your project to someone else?

Not deployed it to a server — just handed the source code to a colleague and said “run this.” What would they need to do? And would it actually work for them the way it works for you?

The honest answer, for most projects developed without careful thought about portability, is: probably not immediately. They might get it working eventually — after some frustration, some Googling, and some changes to their machine. Or they might give up.

This fragility has a name: **environment drift**. It is what happens when the execution environment of a program — the OS, the runtime, the libraries, the filesystem, the configuration — drifts away from the environment the developer had in mind when writing the code. On a developer’s machine, drift accumulates silently. The developer never notices because their machine is always the reference. The moment the code moves, the drift becomes visible.

Environment drift is not caused by bad code. It is caused by assumptions that are implicit rather than explicit. The code is not wrong. The environment is different. The fix is not to patch the code — it is to make the assumptions explicit so they can be reproduced reliably anywhere.

This chapter names the four types of environment drift that affect Python applications most commonly: OS differences, runtime version mismatches, missing system libraries, and filesystem differences. Each one is a real-world failure mode. Each one has a concrete solution, which we build in Stage 3.

In this chapter

By the end of this chapter, you will understand:

- What the operating system provides to a running program, and why OS differences matter
- How Python runtime version differences cause silent and loud failures
- How missing system libraries produce failures that look like Python errors
- How filesystem differences between operating systems create subtle bugs

7.1 The Illusion of “It Works on My Machine”

Before examining specific types of drift, it is worth understanding why the developer’s machine is a deceptive environment.

A developer’s machine is not a clean slate. It is the accumulated result of months or years of software installation, configuration changes, and forgotten one-off adjustments. The machine has:

- A specific operating system version, updated (or not updated) on some schedule
- A Python installation, possibly multiple, from various sources (system package manager, python.org, Homebrew, pyenv)
- Dozens or hundreds of system libraries, installed as dependencies of other software
- Environment variables set in shell configuration files (`.bashrc`, `.zshrc`, `.profile`) that load at login time
- User-level configurations for pip, git, SSH, and other tools
- A filesystem organized in a way that makes sense to this developer but is not reproduced anywhere else

The Notes API works in this environment. When you type `python app.py`, Python starts, finds Flask, creates the database, and listens for connections. All the implicit assumptions are satisfied — not because you made them explicit, but because they happen to be true on your machine.

The server you will eventually deploy to has none of this. It is a fresh Linux installation. It has the packages its distribution includes by default. It has no `venv`. It has no `.env` file. It has no `notes.db`. It has a different user account, a different home directory, and possibly a different Python version.

Each difference is a potential failure. Environment drift is the accumulation of all these differences.

7.2 Operating System Differences: macOS, Windows, and Linux

Most Python developers develop on macOS or Windows. Most production servers run Linux. These are not the same operating system, and the differences matter.

The file path separator

On macOS and Linux, directory separators in file paths are forward slashes (/). On Windows, the native separator is the backslash (\), although Windows APIs — and Python's `open()` — also accept forward slashes.

So a hand-built path such as `"data" + "/" + "notes.db"` usually works everywhere. The trouble with building paths from strings shows up at the edges: drive letters (`C:`), paths returned by the OS with backslashes that you then compare against strings with slashes, absolute paths (`/data` means something different on Windows), and doubled or missing separators when pieces come from configuration:

Listing 7.1 Fragile path construction with strings

```
base = os.getenv("DATA_DIR", "data")      # "data", "data/", or "C:\\notes\\data"?
path = base + "/" + "notes.db"           # may produce "data//notes.db" or mixed
↳ separators
if path == found_path:                   # "data/notes.db" != "data\\notes.db" on
↳ Windows
    ...
```

The robust approach — using `pathlib` or `os.path.join`, which handle separators and edge cases for you:

Listing 7.2 OS-independent path construction

```
from pathlib import Path

# pathlib.Path uses the correct separator for the current OS
path = Path("data") / "notes.db"
# On macOS/Linux: "data/notes.db"
# On Windows:     "data\notes.db"

# Or with os.path.join:
import os
path = os.path.join("data", "notes.db")
```

For the Notes API deployed to Linux, the path separator issue is less relevant since both devel-

opment (macOS) and production (Linux) use /. But if your team includes Windows developers, this is a real problem.

Case sensitivity

Linux file systems are **case-sensitive**. macOS file systems (APFS by default, and HFS+ before it) are **case-insensitive** unless you format them otherwise.

This means on macOS:

```
open("Notes.db") # Works even if the file is named "notes.db"
open("NOTES.DB") # Also works
```

On Linux:

```
open("Notes.db") # FileNotFoundError if the file is "notes.db"
open("NOTES.DB") # FileNotFoundError
```

If your code refers to a file with different capitalization from the name on disk — a data file, a SQL script, a template, a static asset — it works on macOS and fails on Linux. This class of bug is particularly insidious because it is invisible in development.

Listing 7.3 Capitalization bug invisible on macOS, fatal on Linux

```
# Suppose the file on disk is "schema.sql" (all lowercase)

with open("Schema.sql") as f:    # macOS: opens schema.sql without complaint
    schema = f.read()

# On Linux, this raises:
# FileNotFoundError: [Errno 2] No such file or directory: 'Schema.sql'
```

Python's own imports are protected from this: even on macOS, CPython checks that a module's file name matches the case used in the import statement, so from `Notes_model` import `create_note` fails on both systems if the file is `notes_model.py`. Every other file path your program uses gets no such protection.

Rule: Always use lowercase filenames in Python projects. It removes an entire class of OS-specific bugs.

Line endings

Text files on Windows use `\r\n` (carriage return + newline) as the line ending. On macOS and Linux, files use `\n` (newline only).

If a Python script is written on Windows with `\r\n` line endings and then run on Linux:

```
# On Linux, running a script with Windows line endings directly
./script.py
# bash: ./script.py: /usr/bin/python3^M: bad interpreter: No such file or directory
```

The `\r` character (shown by bash as `^M`) at the end of the shebang line (`#!/usr/bin/python3`) is treated as part of the interpreter path, which does not exist. (Running `python3 script.py` instead bypasses the shebang, and Python itself accepts either line ending.)

Git can be configured to handle line ending conversion automatically (`git config core.autocrlf`), but the safest approach is to configure your editor to always use Unix line endings (`\n`) for Python files.

Available system tools

macOS and Linux ship with different default tools and different versions of shared tools. For example:

- macOS ships with BSD-style `sed`, `awk`, and `grep`. Linux typically ships with GNU versions. They have different flags and behaviors.
- macOS ships with an old version of Bash (3.2, due to licensing reasons) and has used `zsh` as the default login shell since 2019. Linux distributions ship Bash 5.x.
- macOS does not have `apt` or `yum`. Linux does not have `brew`.

If your `app.py` or a helper script calls a system command using Python's `subprocess` module:

Listing 7.4 Calling a system command — risky if the tool differs by OS

```
import subprocess

result = subprocess.run(
    ["grep", "-P", "pattern", "file.txt"], # -P flag is GNU-specific
    capture_output=True,
    text=True
)
```

The `-P` flag (Perl-compatible regex) is available in GNU `grep` but not in BSD `grep` (used on

macOS). This code would work on Linux and fail on macOS with:

```
grep: invalid option -- P
```

Or vice versa — a macOS-specific flag that fails on Linux. Any call to system tools creates an OS-specific dependency.

The most reliable way to avoid this whole family of problems is to develop on the same operating system you deploy to — for a Linux server, that means a Linux machine, a Linux VM, or a Linux container (→ [Chapter 18](#) — Containerization).

7.3 Runtime Version Mismatches: Python 3.9 vs 3.11

Python is not one language — it is a language that evolves between versions. New features are added. Old features are deprecated and eventually removed. Code written for Python 3.9 may not run on Python 3.11 (rare), and code that takes advantage of Python 3.11 features will definitely not run on Python 3.9.

New syntax unavailable in older versions

Python 3.10 introduced **structural pattern matching** (the `match` statement). If your code uses it:

Listing 7.5 Python 3.10+ syntax

```
def process_event(event):
    match event["type"]:
        case "created":
            return create_handler(event)
        case "updated":
            return update_handler(event)
        case _:
            return default_handler(event)
```

On Python 3.9 or earlier, this raises:

```
SyntaxError: invalid syntax
```

Python 3.9 does not understand the `match` keyword.

Similarly, **f-string debugging** (`f"{{value=}}"`) was added in Python 3.8. **Walrus operator** (`:=`) was added in Python 3.8. **TypedDict** was added to typing in 3.8. **tomllib** (TOML parsing)

was added to the standard library in 3.11. **Exception groups** (`ExceptionGroup`) arrived in Python 3.11.

If you develop on Python 3.11 and your server runs an older Python — Debian 11 ships 3.9, Ubuntu 20.04 ships 3.8 — any code that uses 3.10 or 3.11 features will fail.

Checking the Python version

Listing 7.6 Checking the Python version at runtime

```
import sys

print(sys.version)
# 3.11.7 (main, Dec 4 2023, 18:10:11) [GCC 11.4.0]

print(sys.version_info)
# sys.version_info(major=3, minor=11, micro=7, releaselevel='final', serial=0)

# Assert a minimum version requirement
if sys.version_info < (3, 10):
    raise RuntimeError(
        f"This application requires Python 3.10 or later. "
        f"You are running Python {sys.version_info.major}.{sys.version_info.minor}."
    )
```

Adding a version check at startup gives a clear error message instead of a confusing syntax error or `AttributeError`.

Library version differences

Beyond the interpreter itself, the libraries that run on Python 3.9 versus Python 3.11 may differ in important ways.

Flask 3.0, for example, requires Python 3.8 or later. If a future version of Flask drops Python 3.8 support, users on Python 3.8 are stuck with an older Flask version. When a new dependency requires a newer Python version that the server does not have, upgrading becomes complicated.

Type annotation differences

Python's type annotation syntax has evolved significantly between versions:

```

# Python 3.8 and earlier: generic types must be imported from typing
from typing import List, Dict, Optional

def get_notes() -> List[Dict[str, str]]:
    ...

# Python 3.9+: built-in types can be used directly
def get_notes() -> list[dict[str, str]]:
    ...

# Python 3.10+: the X | Y union syntax
def find_note(note_id: int) -> dict | None:
    ...

# Equivalent that also works on Python 3.8 and 3.9:
def find_note(note_id: int) -> Optional[dict]:
    ...

```

Neither mistake is a `SyntaxError` — the code parses fine. Both fail at runtime, when Python evaluates the annotation (normally as the `def` statement runs, at import time):

```

# list[dict] on Python 3.8:
TypeError: 'type' object is not subscriptable

# dict | None on Python 3.9:
TypeError: unsupported operand type(s) for |: 'type' and 'NoneType'

```

(With `from __future__ import annotations` at the top of the module, annotations are not evaluated and neither error appears — until a library such as a validation framework evaluates them.)

7.4 Missing System Libraries: `libc`, `OpenSSL`, and Silent Failures

Some Python packages are not pure Python — they contain **compiled C extensions** that link against system libraries. When these libraries are missing or are the wrong version, the package fails to import, often with a confusing error.

What compiled extensions are

Python packages like `cryptography`, `Pillow`, `numpy`, `psycpg2`, and others include C code that is compiled into a `.so` (shared object) file on Linux or a `.dylib` on macOS. This compiled

code runs natively for performance reasons.

When you `pip install cryptography`, pip downloads a **wheel file** that contains pre-compiled `.so` files for your platform. These compiled files link against system libraries.

When there is no ready-made wheel

Many popular packages (cryptography, numpy, Pillow) publish wheels for all common platforms that *bundle* the C libraries they need, so they install cleanly almost anywhere. Trouble starts when pip cannot find a wheel for your platform — a brand-new Python version, an unusual CPU, a minimal Alpine image — or when the package’s authors do not publish one at all. Then pip downloads the source code and tries to compile it, which needs a C compiler and the *development headers* of every system library the package uses.

On a fresh minimal Ubuntu image, for example, the MySQL driver `mysqlclient` has no wheel for Linux:

```
pip install mysqlclient
```

This fails with:

```
Exception: Can not find valid pkg-config name.
Specify MYSQLCLIENT_CFLAGS and MYSQLCLIENT_LDFLAGS env vars manually
...
ERROR: Failed to build installable wheels for some pyproject.toml based projects
↳ (mysqlclient)
```

A second failure mode shows up only at runtime. Some packages load a system library when they are imported. `python-magic`, which detects file types, installs without complaint but needs `libmagic` at import time:

```
import magic
# ImportError: failed to find libmagic. Check your installation
```

The package installed, but the system library it needs is not present.

The fix is to install the system packages before installing the Python package:

```
apt-get install -y pkg-config build-essential default-libmysqlclient-dev # for
↳ mysqlclient
apt-get install -y libmagic1 # for
↳ python-magic
```

This must happen at the operating system level — `pip install` cannot do it.

The psycopg2 dependency on PostgreSQL libraries

The `psycopg2` package — the long-standing Python adapter for PostgreSQL (which we will use in → Chapter 13 — Configuration Externalization; its successor is `psycopg` version 3) — requires the PostgreSQL client library (`libpq.so`) when it is built from source:

```
pip install psycopg2
# This tries to compile C code that links against libpq

# If libpq-dev is not installed:
# Error: pg_config executable not found.
```

Solution: install the system dependency first:

```
apt-get install -y libpq-dev build-essential python3-dev # headers + C compiler
pip install psycopg2 # Now builds and installs
```

Or use `psycopg2-binary` — a wheel that includes the necessary libraries statically:

```
pip install psycopg2-binary # No system dependency needed
```

`psycopg2-binary` is convenient for development. For production, the project’s documentation recommends using the source distribution (`psycopg2`) because `psycopg2-binary` is not guaranteed to be compatible with all system configurations.

How to detect missing system libraries before deploying

```
# After installing Python packages, verify imports work:
python -c "import flask; import sqlite3; print('OK!)"

# Check that compiled extensions load:
python -c "import psycopg2; print('psycopg2 OK!)"

# List the shared libraries a compiled extension needs (psycopg2 built from source):
ldd venv/lib/python3.11/site-packages/psycopg2/_psycopg*.so
```

Running these checks in your deployment environment (before the application starts) catches library problems early with clear error messages, instead of discovering them when a user’s request hits a code path that uses the broken package.

7.5 File System Differences: Paths, Permissions, and Line Endings

Beyond file path separators and case sensitivity (covered in Section 7.2), the filesystem on a production server differs from a development machine in ways that affect application behavior.

Different home directory locations

On macOS, home directories are at `/Users/yourname`. On Linux, they are at `/home/yourname`. If your application constructs file paths relative to the home directory:

Listing 7.7 Hardcoded path to home directory — bad practice

```
import os

# This works on macOS if the user is 'developer':
config_path = "/Users/developer/notes-api/.env"

# This works on Linux:
# config_path = "/home/developer/notes-api/.env"

# Neither is portable. Use environment variables or Path.home() instead:
config_path = os.path.join(os.path.expanduser("~"), "notes-api", ".env")
# Or with pathlib:
from pathlib import Path
config_path = Path.home() / "notes-api" / ".env"
```

`os.path.expanduser("~")` expands `~` to the actual home directory, correctly on every OS.

File permissions and ownership

Linux has a strict permission model. Files and directories have an owner, a group, and permission bits for read (r), write (w), and execute (x) for the owner, the group, and others.

A common failure: the application's user account does not have write permission to the directory where it tries to create the database.

```
# On the server, the app runs as user 'appuser'
# But notes.db will be in /opt/notes-api/, owned by root

ls -la /opt/notes-api/
# drwxr-xr-x 2 root root 4096 Jan 1 12:00 .

# appuser has no write permission to this directory (others: r-x)
python app.py
# sqlite3.OperationalError: unable to open database file
```

The application fails because it cannot create the database file. The error message mentions SQLite, but the real cause is a Linux permission issue. This rarely occurs on a developer's machine, where you usually work inside your own home directory and own every file in it.

The fix is to ensure the application's user account owns the directory where it writes:

```
chown appuser:appuser /opt/notes-api/data/
chmod 755 /opt/notes-api/data/
```

Read-only filesystems

Some deployment environments use read-only root filesystems — the application code is deployed as a read-only archive, and only specific directories are writable. Docker containers, for example, can be run with `--read-only`:

```
docker run --read-only notes-api
# Any write to the filesystem will fail with:
# OSError: [Errno 30] Read-only file system
```

An application that writes its SQLite database to the current directory will fail in a read-only container. The fix is to mount a writable volume at a specific path and configure the application to write there.

Temporary files and /tmp

The `/tmp` directory is a universal location for temporary files on Unix-like systems. It is always writable. But it has two properties developers must be aware of:

1. **It is cleaned on reboot on many systems** (and often on a schedule). Anything written to `/tmp` may disappear without warning. A database in `/tmp` is a database that will be lost on the next server restart.

2. It is **shared across all users and processes**. If your application creates predictable file-names in `/tmp` (e.g., `/tmp/notes.db`), another process or another instance of your application may overwrite them. Use `tempfile.mkstemp()` or `tempfile.mkdtemp()` to create uniquely named temporary files.

Environment Drift Summary: A Checklist

Before deploying, the following differences between your development machine and the target server should be verified:

Concern	Check
OS type	macOS → Linux (different defaults, different tools)
File path separators	Use <code>pathlib.Path</code> everywhere, not string concatenation
Case sensitivity	Use lowercase filenames; every path in code matches the on-disk name exactly
Line endings	Configure your editor to use Unix line endings (<code>\n</code>)
Python version	<code>python --version</code> on both machines; ensure they match
System library availability	ldd check for compiled extensions; verify on target OS
File permissions	Application user has write access to data directories
Home directory	Do not hardcode <code>/Users/...</code> or <code>/home/...</code> paths
Temp directory assumptions	<code>/tmp</code> is ephemeral; do not store persistent data there
Filesystem mutability	Know whether the target filesystem is read-only

This checklist does not eliminate environment drift — it makes it visible. Making it visible is what Stage 3 of this book is about: converting each of these implicit assumptions into explicit, documented, reproducible specifications.

Key Takeaways

- **Environment drift** is the accumulation of differences between a developer’s machine and the target execution environment. It causes software that “works on my machine” to fail elsewhere.
- **OS differences** between macOS and Linux include: file path separators, case sensitivity, line endings, and available system tools. The safest rule: develop on Linux or use a Linux container for development.
- **Python runtime version mismatches** cause failures ranging from `SyntaxError` (new syntax on old interpreter) to subtle behavioral differences. Always know which Python version you are targeting, and verify it is installed on the target server.
- **Missing system libraries** cause Python packages with compiled C extensions to fail at import time, often with confusing error messages. These libraries must be installed at the OS level before the Python packages can work.
- **Filesystem differences** include case sensitivity, permission models, and home directory paths. Production servers often have stricter permission requirements than development machines.
- The solution to environment drift is not to fix bugs when they appear, but to declare all assumptions explicitly — which is what Chapters 12–16 build toward.

Exercises

1. **Predict.** Create files `Notes.py` and `notes.py` in the same directory on macOS, then on Linux (or in a directory *inside* a Linux container, such as `/tmp`, not a directory mounted from your Mac). What happens on each?
2. **Break and fix.** Use a Python 3.10-only syntax feature (a `match` statement) and run the file with Python 3.9 (in a container: `docker run --rm -v $PWD:/w python:3.9 python /w/file.py`). Read the error, then declare the minimum version so it fails clearly.
3. **Extend.** Write a script that prints the OS, Python version, locale, time zone and file-system case sensitivity of the machine it runs on. Run it on your laptop and in a container (from a directory inside the container, such as `/tmp`), and compare.
4. **Explain.** Why is “develop in the same kind of environment you deploy to” a better fix for drift than fixing each difference as it appears?

Solutions and hints are in the companion repository, in `exercises/chapter-07.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

Environment drift is drift you can see — visible differences in the execution environment. The next chapter addresses a subtler problem: dependencies that are hidden inside the system, not in your code, not in your `requirements.txt`, and not obviously connected to your application at all.

→ [Chapter 8 — Hidden Dependencies](#)

Hidden Dependencies

Environment drift (← Chapter 7) is largely visible. You can compare OS versions, Python versions, and installed packages. The differences are at least theoretically detectable.

Hidden dependencies are different. They are the things your program needs that are not written down anywhere and are not even obviously connected to your application. They are the software your code uses without knowing it is using it. They are the assumptions so deeply embedded that the developer who wrote the code never noticed them as assumptions at all.

A hidden dependency is not a bug in your code. It is a gap between what your code requires and what you have documented about those requirements. The code works because the developer's machine happens to have what the code needs. The gap is only discovered when the code runs in a different environment.

This chapter examines four categories of hidden dependencies that affect Python applications:

1. **Global vs local installs:** packages installed outside the virtual environment that your code depends on implicitly
2. **PATH dependencies:** command-line tools your code invokes that must exist at specific paths
3. **External binaries:** system programs called through `subprocess` or `os.system`
4. **Implicit assumptions:** things the code assumes are true about the environment without ever checking

In this chapter

By the end of this chapter, you will understand:

- What each type of hidden dependency looks like
- How each one appears in Python code
- Why each one is dangerous
- What makes a dependency “hidden” in the first place

8.1 The Difference Between Global and Local Installs

The term “hidden global dependency” describes a package that is installed globally on the developer’s machine and imported by the application, but is not listed in the application’s requirements file and is not installed in the virtual environment.

How it happens

A developer installs a useful tool globally one day:

```
pip install httpx
# Installed globally (no venv active)
```

Later, while working on the Notes API, they write code that uses it:

Listing 8.1 Using a globally installed package without declaring it

```
import httpx # This works on the developer's machine

def notify_external_service(note_id: int, content: str):
    """Send a notification when a note is created."""
    httpx.post("https://webhook.example.com/notes", json={
        "id": note_id,
        "content": content,
    })
```

This code works on the developer’s machine — but not because of the project’s virtual environment. A virtual environment created with `python3 -m venv` is isolated: when it is active, globally installed packages are *not* on `sys.path`, and `import httpx` would fail. The code works because of one of these everyday slips:

- The developer ran the server **without activating the venv** (a new terminal tab, an IDE

run configuration pointing at the system interpreter), so the global Python — which has `httpx` — ran the code.

- The venv was created with `--system-site-packages`, which deliberately lets it see global packages.
- The developer installed `httpx` into the venv while experimenting, and never added it to the project's dependency list.

The developer commits this code. `httpx` is not in `requirements.txt`, because nothing forced anyone to write it down.

A colleague clones the repository, creates a fresh virtual environment, installs from `requirements.txt`, and runs the server. When the first note is created, the application crashes:

```
ModuleNotFoundError: No module named 'httpx'
```

The package was never declared. It was never in `requirements.txt`. On the developer's machine, it happened to be available globally. Everywhere else, it does not exist.

Why this happens without the developer noticing

The developer's workflow was:

1. Install `httpx` somewhere their interpreter could see it (possibly months ago, for a different project)
2. Write code that uses `httpx`
3. Test the code (it works — `httpx` is importable in *that* interpreter)
4. Not notice that `httpx` is missing from `requirements.txt`

There is no warning. `import httpx` does not care where the package came from; it only needs the package to be somewhere on the current interpreter's `sys.path`. Nothing compares what you import with what you declared.

How to detect global dependency leaks

The safest practice is to always develop in a virtual environment with **only** what is in `requirements.txt` installed — never install packages globally.

You can also audit which packages are globally installed vs venv-installed:

```
# Activate the venv, then check what's available:
pip list # Shows packages in the active venv

# Compare to what is outside the venv:
deactivate
pip list # Shows packages in the global Python installation
```

If a package appears in the global list but not in the venv list, and your code uses it, you have a hidden global dependency.

A more systematic check: create a fresh virtual environment (a new directory) and try to run the application with only the packages from `requirements.txt`. If it fails, you have found a hidden dependency.

```
# Test for hidden dependencies by using a fresh environment
python3 -m venv /tmp/test-venv
/tmp/test-venv/bin/pip install -r requirements.txt
/tmp/test-venv/bin/python app.py
```

Any `ModuleNotFoundError` that appears in the fresh environment but not in your development venv is a hidden global dependency.

8.2 PATH Dependencies: Tools Your Code Assumes Are Present

Your Python program can invoke external command-line tools using `subprocess.run()`, `os.system()`, or `os.popen()`. When it does, it assumes that tool exists on `$PATH`. If the tool is not on `$PATH` in the deployment environment, the call fails.

A realistic example

The Notes API might need to export notes as Word documents. There is a widely used command-line tool called `pandoc` that converts between document formats, including HTML to `.docx`. A developer installs it on their macOS machine (via Homebrew):

```
brew install pandoc
```

Then writes this code:

Listing 8.2 Calling an external tool via subprocess

```

import subprocess
import tempfile
from pathlib import Path

def export_note_as_docx(note_id: int, html_content: str) -> bytes:
    """Convert note HTML content to a Word document using pandoc."""
    with tempfile.TemporaryDirectory() as tmp:          # deleted automatically
        ↪ afterwards
        input_path = Path(tmp) / "note.html"
        output_path = Path(tmp) / "note.docx"
        input_path.write_text(html_content, encoding="utf-8")

        result = subprocess.run(
            ["pandoc", str(input_path), "-o", str(output_path)],
            capture_output=True,
            text=True,
        )
        if result.returncode != 0:
            raise RuntimeError(f"Document export failed: {result.stderr}")

    return output_path.read_bytes()

```

This code works on the developer's machine because pandoc is installed and in \$PATH.

On a fresh Ubuntu server, pandoc is not installed by default. The `subprocess.run()` call fails:

```
FileNotFoundError: [Errno 2] No such file or directory: 'pandoc'
```

pandoc is a system-level dependency — not a Python package, not installable with `pip`. It must be installed with `apt-get install pandoc` (on Ubuntu). It is not in `requirements.txt`. It is not documented anywhere in the project. It is a hidden dependency.

The PATH resolution trap

When `subprocess.run(["pandoc", ...])` is called, Python's `subprocess` module uses the process's \$PATH to find pandoc. But the \$PATH in a production process launched by `systemd` or `Docker` may be different from the \$PATH in an interactive shell.

`systemd` sets a minimal \$PATH by default:

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

A developer's shell \$PATH might include:

```
/home/yourname/.local/bin:/usr/local/bin:/usr/bin:/bin:/home/yourname/go/bin:/usr/local/bin
↪ cal/go/bin
```

A tool installed at `/home/yourname/.local/bin/pandoc` is accessible from an interactive shell but not from a systemd service, because systemd does not include `~/.local/bin` in its PATH.

The safer approach

Use absolute paths when calling external tools, or verify their presence at startup:

Listing 8.3 Safer subprocess call with explicit validation

```
import subprocess
import shutil
import os

def _find_tool(name: str) -> str:
    """Find the absolute path to a required external tool."""
    # 1. Check if an explicit path is configured
    configured = os.getenv(f"{name.upper()}_PATH")
    if configured:
        if not os.path.isfile(configured):
            raise RuntimeError(f"Configured path for {name} does not exist: "
                               ↪ {configured}")
        return configured

    # 2. Search PATH
    found = shutil.which(name)
    if found is None:
        raise RuntimeError(
            f"Required tool '{name}' not found on PATH. "
            f"Install it with: apt-get install {name}"
        )
    return found

# Called at startup, before the server begins accepting requests
PANDOC_PATH = _find_tool("pandoc")

def export_note_as_docx(note_id: int, html_content: str) -> bytes:
    ...
```

```
# Use the verified absolute path
result = subprocess.run(
    [PANDOC_PATH, str(input_path), "-o", str(output_path)],
    ...
)
```

`shutil.which()` searches `$PATH` for the tool and returns its absolute path (or `None` if not found). By calling this at startup and failing fast with a clear error, you turn a hidden runtime dependency into an explicit startup check.

8.3 External Binaries: System Programs Called from Python

External binaries are programs installed at the OS level that your Python code relies on. Unlike `PATH` dependencies, which are called directly by name, external binaries may be called by libraries that use them internally — making the dependency even less visible.

Libraries that wrap external binaries

Some Python libraries are thin wrappers around external command-line tools:

imagemagick: The `wand` Python library binds to ImageMagick’s shared library (`libMagickWand`) for image processing. Importing `wand.image` works only if that library is installed on the system.

ffmpeg: Libraries like `ffmpeg-python` wrap the `ffmpeg` tool for video processing. The Python library installs fine; the binary must be separately installed.

git: The `gitpython` library wraps the `git` command. If `git` is not installed, `gitpython` fails when it tries to run `git` commands.

latex: Libraries for rendering LaTeX to PDF invoke `pdflatex` or `xelatex`. These are part of a TeX distribution that must be installed separately.

chromedriver: Web scraping libraries like Selenium use `chromedriver` (and Chrome itself) as external binaries.

Example: the `wand` library (ImageMagick wrapper):

Listing 8.4 Using a library that wraps an external binary

```

from wand.image import Image # Works on developer's Mac where ImageMagick is
→ installed

def generate_thumbnail(image_data: bytes) -> bytes:
    """Resize an image to thumbnail size."""
    with Image(blob=image_data) as img:
        img.resize(128, 128)
        return img.make_blob()

```

On Ubuntu, if ImageMagick is not installed:

```

pip install wand
python -c "from wand.image import Image"
# ImportError: MagickWand shared library not found.
# You probably had not installed ImageMagick library.

```

The Python library (wand) installed fine. The ImageMagick shared library it loads at import time is missing.

The fix:

```

apt-get install -y libmagickwand-dev # or: apt-get install -y imagemagick

```

This is a system-level operation, not a `pip install`. It must be part of your server setup (→ Chapter 21 — System Setup) or your Docker image (→ Chapter 18 — Containerization).

Finding all external binary dependencies

There is no automated way to find all external binary dependencies. The best approaches are:

1. **Read the documentation** for every library you use. Any library that wraps an external tool will say so.
2. **Test in a fresh environment** (a fresh Docker container or a new VM) with only the packages from `requirements.txt`. Any failure that mentions a missing binary reveals a hidden dependency.
3. **Instrument startup**: call `shutil.which()` for every external tool at startup and fail fast with a clear message if any are missing.

8.4 Implicit Assumptions: What Your Code Relies On Without Knowing It

The fourth category of hidden dependencies is the most subtle: things the code assumes without ever making an explicit call for them. These assumptions are baked into how the code works, but they are never expressed as requirements.

Locale and character encoding

Python 3 strings are Unicode, and Python source files default to UTF-8. But the **locale** — the system-wide setting for language, encoding, and date/number formatting — can affect how files are read and written.

On some older Linux systems, the default locale is POSIX (also called C), which does not support Unicode properly. In that locale:

Listing 8.5 Character encoding assumptions

```
with open("notes.txt", "w") as f:
    f.write("Note: café au lait ☕")
# On a system with POSIX locale, this may raise:
# UnicodeEncodeError: 'ascii' codec can't encode character '\xe9' in position 9:
# ordinal not in range(128)
```

The `open()` call without an explicit encoding argument uses the locale's default encoding, which may not be UTF-8. Since Python 3.7, a plain C/POSIX locale is automatically upgraded to UTF-8, so this error is rare on modern Python — but other non-UTF-8 locales still exist, and relying on the default is still an implicit assumption.

The fix: always specify the encoding explicitly:

Listing 8.6 Explicit encoding — no implicit locale dependency

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Note: café au lait ☕")
```

And set the system locale to UTF-8 in production (in Docker: `ENV LANG=C.UTF-8`).

Time zones

Python's `datetime.now()` returns the current time in the local timezone. If the server is in UTC and the developer's machine is in UTC-5, the same code produces different timestamps.

Listing 8.7 Timezone-dependent code

```
from datetime import datetime

created_at = datetime.now()
# On developer's machine (US Eastern, UTC-5): 2024-01-01 07:00:00
# On server (UTC):                             2024-01-01 12:00:00
```

This can cause bugs in date comparisons, scheduled tasks, and audit logs. The fix: always use UTC:

Listing 8.8 Timezone-aware datetime — no implicit timezone dependency

```
from datetime import datetime, timezone

created_at = datetime.now(tz=timezone.utc)
# Always 2024-01-01 12:00:00+00:00, regardless of system timezone
```

And set the server's timezone to UTC (in Docker: `ENV TZ=UTC`).

Available ports

The application assumes port 5000 is free. On macOS Monterey (12.x) and later, port 5000 is used by AirPlay Receiver. A developer who does not have AirPlay Receiver disabled will find that Flask fails to bind:

```
OSError: [Errno 48] Address already in use
```

This is an implicit assumption about port availability. The fix is to make the port configurable:

Listing 8.9 Making the port configurable via environment variable

```
import os

port = int(os.getenv("PORT", "5000"))
app.run(host="localhost", port=port, debug=True)
```

Now the port can be changed without modifying code:

```
PORT=8080 python app.py
```

CPU architecture and compiled packages

Some Python packages are distributed as precompiled wheels for specific CPU architectures:

- x86_64 (most Intel/AMD desktops and servers)
- arm64 / aarch64 (Apple Silicon Macs, AWS Graviton, Raspberry Pi)

A package compiled for x86_64 will not run on arm64, and vice versa. If you download a wheel on one architecture and try to run it on another:

```
ImportError: /path/to/package.cpython-311-x86_64-linux-gnu.so: cannot execute binary
↳ file: Exec format error
```

This is usually handled automatically by pip — it downloads the correct wheel for the current architecture, and it refuses to install a .whl file built for a different platform (“is not a supported wheel on this platform”). The error above appears when an already-installed environment is copied between machines — for example, a venv/ directory or a container image built on an Apple Silicon Mac and run on an x86_64 server.

The implication: install packages on (or for) the target platform — let pip run there, or build images explicitly for the server’s architecture (→ [Chapter 18](#)). Never copy compiled packages between different architectures.

File descriptor limits

The operating system limits how many file descriptors a process can have open simultaneously. On Linux, this default is typically 1024. A server that opens many database connections, log files, and network connections simultaneously may hit this limit.

Listing 8.10 Checking the file descriptor limit

```
import resource

soft_limit, hard_limit = resource.getrlimit(resource.RLIMIT_NOFILE)
print(f"Soft limit: {soft_limit}, Hard limit: {hard_limit}")
# Soft limit: 1024, Hard limit: 1048576
```

The soft limit (1024) is the enforced limit. A process that exceeds it gets `OSError: [Errno 24] Too many open files`. This limit is implicitly assumed to be sufficient. On a heavily loaded server, it may not be. We address this in → [Chapter 25 — Process Management](#).

The Hidden Dependency Audit

Before deploying the Notes API (or any application), conduct a hidden dependency audit:

Listing 8.11 A startup check that surfaces hidden dependencies explicitly

```
import shutil
import os
import sys

def check_dependencies():
    """Verify all hidden dependencies at startup. Fail fast with clear messages."""
    errors = []

    # Check Python version
    if sys.version_info < (3, 10):
        errors.append(
            f"Python 3.10+ required. Running:
            ↪ {sys.version_info.major}.{sys.version_info.minor}"
        )

    # Check required environment variables
    required_env = ["DATABASE_PATH", "API_KEY"]
    for var in required_env:
        if not os.getenv(var):
            errors.append(f"Required environment variable not set: {var}")

    # Check required external tools
    required_tools = ["pandoc"] # add any tools your app uses
    for tool in required_tools:
        if shutil.which(tool) is None:
            errors.append(f"Required external tool not found on PATH: {tool}")

    # Check writable paths
    data_dir = os.path.dirname(os.getenv("DATABASE_PATH", "notes.db"))
    if data_dir and not os.access(data_dir, os.W_OK):
        errors.append(f"Database directory is not writable: {data_dir}")

    # Check required Python packages (by import name: python-dotenv imports as
    ↪ "dotenv")
    required_packages = ["flask", "dotenv"]
    for package in required_packages:
        try:
            __import__(package)
```

```

except ImportError:
    errors.append(f"Required Python package not importable: {package}")

if errors:
    print("Startup failed – dependency check errors:")
    for error in errors:
        print(f"  x {error}")
    sys.exit(1)

print("All dependency checks passed.")

# Call this before app.run()
check_dependencies()

```

This function runs at startup and either confirms all dependencies are present or lists every missing one clearly. It turns hidden failures (crashes mid-request with cryptic errors) into explicit startup failures with actionable messages.

Key Takeaways

- **Hidden dependencies** are things your program needs that are not written down in any requirements file and may not be obviously connected to your application.
- **Global vs local installs:** packages that happen to be importable on the developer's machine — because the venv was not active, or was never cleaned — work even without being in `requirements.txt`. Test in a fresh virtual environment with only `requirements.txt` packages to find these.
- **PATH dependencies:** external tools invoked by name via `subprocess` are not Python packages and cannot be `pip` installed. They must be documented and installed separately at the OS level.
- **External binaries:** Python libraries that wrap external tools or libraries (ImageMagick, ffmpeg, pandoc) have non-Python dependencies. Read each library's documentation for system requirements.
- **Implicit assumptions:** character encoding, timezone, port availability, CPU architecture, and file descriptor limits are all assumptions embedded in the code that can silently differ between environments.
- The solution is to write an explicit **startup dependency check** that fails fast with a clear error message when any dependency is missing.

Exercises

1. **Predict.** Create a fresh virtual environment, install only `flask` and `python-dotenv`, then start the application and call every endpoint with `curl`. Did anything fail that worked in your usual environment? What does that reveal?
2. **Break and fix.** Make the application call an external program (for example `subprocess.run(["convert", ...])`) and run it where that program is not installed. Then add a start-up check that fails fast with a clear message.
3. **Extend.** Extend the start-up check so it reports *all* missing dependencies at once, not just the first.
4. **Explain.** Give three examples of implicit assumptions (not packages) that code can depend on without anyone noticing.

Solutions and hints are in the companion repository, in `exercises/chapter-08.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

We have looked at how the environment can differ from what the code expects. The next chapter looks at a specific and dangerous category of hidden coupling: configuration values — secrets, database URLs, debug flags — that are baked directly into the source code instead of externalized where they can be safely changed.

→ [Chapter 9 — Configuration Coupling](#)

Configuration Coupling

Configuration coupling is the practice of embedding configuration values directly inside source code — and it is one of the most consequential mistakes a developer can make when preparing software for production.

It does not feel like a mistake at first. In early development, hardcoding values is fast. You know what the database URL is. You know what debug mode should be. You know what the API key is. Typing the values directly into the code is simpler than setting up environment variables. The application works. You move on.

The problem emerges later, when you realize that those values — embedded in source code — need to be different in every environment the application runs in. Different database in production. Debug mode off in production. A real API key in production, not a test one. And because the values are in the source code, changing them means changing the code — which means you are shipping different source code to different environments, which means the code you tested locally is not the code that runs in production.

That last point is fundamental. **The code you tested should be the code you deploy.** Configuration coupling breaks this principle. It forces you to choose: either run the same code everywhere (with the same hardcoded values, which means your local debug key runs in production), or change the code for each environment (which means you are deploying untested variations).

Neither option is acceptable.

This chapter examines the four most common forms of configuration coupling: hardcoded secrets, local database references, debug configurations, and environment-specific behavior in code. Each one is examined in the form it typically appears, explained with concrete risks, and contrasted with the externalized version. The full externalization solution is built in → [Chapter 13 — Configuration Externalization](#).

In this chapter

By the end of this chapter, you will understand:

- Why hardcoded secrets are a security and operational disaster
- What happens when local database paths are embedded in code
- Why debug configurations left enabled in production are dangerous
- How environment-specific behavior in code creates silent differences between environments

9.1 Hardcoded Secrets

A **secret** is any value that grants access to a resource: a password, an API key, a signing token, a cryptographic key. Secrets must remain private. If a secret is exposed, the resource it protects is no longer protected.

Hardcoding secrets means writing them as string literals in source code:

Listing 9.1 Hardcoded secrets (NEVER do this)

```
from flask import Flask

app = Flask(__name__)

# Secret key for signing session cookies
app.config["SECRET_KEY"] = "my-super-secret-key-12345"

# Database password
DATABASE_URL = "postgresql://admin:password123@localhost/notesdb"

# Third-party API key
STRIPE_API_KEY = "sk_live_abc123xyz789..."

# Email service credentials
SMTP_PASSWORD = "email_password_here"
```

This is a complete example of what not to do. Every value in this listing is a secret, and every secret is exposed to anyone who can read the source code.

Why hardcoded secrets are dangerous

Version control exposure: Modern software development uses version control systems like Git. When you commit `app.py` with hardcoded secrets, those secrets become part of the repository

history. Even if you later remove them from the code and commit the change, the secrets remain visible in the git history. Anyone with access to the repository — including all current and future team members, anyone who receives a copy of the repository, and potentially the hosting service (GitHub, GitLab, Bitbucket) — can see them.

This is not theoretical. Organizations regularly discover that API keys, database credentials, and private keys were committed to repositories — sometimes years ago. The remediation is painful: change every exposed credential, audit what was accessed with the exposed credentials, notify affected parties if user data was compromised.

Repository leaks: Public repositories on GitHub are indexed by search engines and security scanning tools. There are automated tools that continuously scan public repositories for patterns that look like secrets. Within minutes of pushing a secret to a public repository, it may be found and used by malicious actors.

Deployment coupling: If the production database password is in the source code, you need to change the source code to deploy to production with a different password. This means the code running in production differs from the code running in development. If the code is different, the code was not tested in the same state it runs.

What a leaked secret can do

Consider what happens if the `STRIPE_API_KEY` above is exposed:

- An attacker can charge any amount to any credit card on file in your Stripe account
- An attacker can issue refunds, draining your account balance
- An attacker can read all your payment records
- Stripe will suspend your account when they detect the fraud
- You will need to invalidate the key, generate a new one, update all deployments, and explain to your customers why their payments failed

A leaked database password can be worse: complete read and write access to all user data.

The correct approach

Secrets belong in environment variables or a secrets management service, never in source code. The application reads them at runtime:

Listing 9.2 Secrets from environment variables

```

import os
from flask import Flask

app = Flask(__name__)

# Read from environment — never from the source file
app.config["SECRET_KEY"] = os.environ["SECRET_KEY"] # Raises KeyError if missing
DATABASE_URL = os.environ["DATABASE_URL"]           # Raises KeyError if missing
STRIPE_API_KEY = os.getenv("STRIPE_API_KEY")         # Returns None if missing

if not STRIPE_API_KEY:
    raise RuntimeError("STRIPE_API_KEY environment variable is required")

```

The production server has these environment variables set (through the server’s configuration, through a secrets manager like AWS Secrets Manager or HashiCorp Vault, or through a platform like Heroku). The development machine has them set in a `.env` file (which is in `.gitignore`). The source code contains no secrets — it only contains the code that reads them.

What to do if you accidentally commit a secret

1. **Immediately** invalidate the exposed credential. Generate a new API key, change the password. Do this before anything else.
2. Remove the secret from the code and commit the fix.
3. **Do not** assume that deleting the file from the latest commit is sufficient. The secret is in the git history. Use `git filter-repo` or `BFG Repo-Cleaner` to rewrite history, then force-push (coordinate with your team first).
4. If the repository was ever public, assume the secret was captured. Treat it as fully compromised.
5. Audit access logs to determine if the credential was used maliciously.

9.2 Local Database References

The second form of configuration coupling is embedding database connection details in code — particularly details that are specific to the local development machine.

The hardcoded local database

Listing 9.3 Hardcoded database reference (problematic)

```
import sqlite3

# Path that only exists on the developer's machine
DATABASE_PATH = "/Users/alice/projects/notes-api/notes.db"

def get_db():
    return sqlite3.connect(DATABASE_PATH)
```

/Users/alice/projects/notes-api/notes.db does not exist on any other machine. On Bob's laptop, the path would be /Users/bob/projects/notes-api/notes.db. On a Linux server, home directories are under /home/, not /Users/. On a Docker container, there is no /Users/ at all.

This code fails the moment anyone other than Alice runs it.

The relative path trap

A slightly better but still problematic approach:

Listing 9.4 Relative path — better, but still fragile

```
DATABASE_PATH = "notes.db"
```

"notes.db" is a relative path — it refers to a file in the current working directory. If you start the server from ~/notes-api/, the database is at ~/notes-api/notes.db. If a process manager like systemd starts the server with a working directory of /opt/notes-api/, the database is at /opt/notes-api/notes.db.

The behavior depends on where you start the server, which is an implicit environment assumption.

The production database URL problem

The more serious version of this problem occurs when the application switches from SQLite to a production database like PostgreSQL. PostgreSQL requires:

- A hostname (where the database server is running)
- A port (default 5432)
- A database name
- A username and password

All of these are environment-specific. In development: localhost:5432. In staging:

staging-db.internal:5432. In production: a managed database URL provided by AWS, Google Cloud, or another provider — something like `postgresql://user:password@mydb.cluster.region.rds.amazonaws.com:5432/notesdb`.

If any part of this URL is hardcoded, the application is locked to one specific database. Moving it to staging or production requires changing the code — which means re-testing and re-deploying code that differs from what was previously tested.

Listing 9.5 Hardcoded production database URL (NEVER do this)

```
DATABASE_URL = "postgresql://admin:p4ssw0rd@mydb.prod.rds.amazonaws.com/notesdb"
```

This is doubly bad: it is a secret (password), and it is a location (specific to the production environment). The code cannot run against a local database for development.

The correct approach

The database URL belongs in an environment variable. Every environment sets its own:

Listing 9.6 Database URL from environment variable

```
import os

DATABASE_URL = os.environ["DATABASE_URL"]
# Development .env: DATABASE_URL=sqlite:///notes.db
# Staging .env:    DATABASE_URL=postgresql://user:pass@staging-db:5432/notesdb
# Production env: DATABASE_URL=postgresql://user:pass@prod-db:5432/notesdb
```

The same code runs in all three environments. Only the environment variable value changes. The code you tested in development is the code that runs in production.

9.3 Debug Configurations

Debug mode is a set of development-time features that help you understand errors and iterate quickly. In Flask, `debug=True` enables:

- **The interactive debugger:** shows a full Python traceback in the browser when an exception occurs, with an interactive console that can execute arbitrary Python code
- **The automatic reloader:** restarts the server when source files change

Both features are invaluable in development. Both are catastrophic in production.

Why the interactive debugger is a critical vulnerability

When Flask’s debug mode is on and an exception occurs, the browser receives a response that looks like this:

```
Traceback (most recent call last):
  File "/path/to/venv/lib/flask/app.py", line 1489, in full_dispatch_request
    rv = self.dispatch_request()
    ...
  File "app.py", line 47, in create_note
    conn = sqlite3.connect(DATABASE_PATH)

sqlite3.OperationalError: unable to open database file

Interactive console available. Click to open.
```

The “Interactive console available” link is not a display feature — it is a live Python interpreter running in the context of the application, accessible through the browser. Anyone who can reach the application can:

- Read any file on the filesystem
- Execute arbitrary system commands
- Access any environment variable (including secrets)
- Modify data in memory
- Crash the server

```
# What an attacker could do via the interactive debugger console:
import os; os.environ.get("SECRET_KEY") # Read your secret key
import subprocess; subprocess.getoutput("cat /etc/passwd") # Read system files
import subprocess; subprocess.getoutput("rm -rf /") # Destroy the server
```

This is not a theoretical attack. An exposed Werkzeug debugger is one of the most commonly exploited misconfigurations in Python web applications. Since 2015 the console is protected by a PIN that is printed in the server’s log, but that is a thin defence: the PIN is derived from details of the machine that an attacker who can read a few files (through another bug) can recompute, and it was never meant to protect a public server. The only safe setting in production is debug mode off.

In 2015, Patreon suffered a breach that exposed source code and data for about 2.3 million accounts. The entry point was a publicly reachable development server, holding a copy of production data, with the Werkzeug debugger enabled.

Hardcoded debug mode

Listing 9.7 Debug mode hardcoded to True (never do this in production)

```
app.run(debug=True)
```

This is the single most dangerous configuration coupling. If this code is deployed to production, the application is compromised the moment it receives a request that raises an exception.

Other debug configurations

Debug mode is not the only dangerous debug configuration:

Listing 9.8 Other dangerous debug configurations

```
# Showing detailed error messages to users
app.config["PROPAGATE_EXCEPTIONS"] = True # Bypass error handlers; raw errors reach
↪ the server

# Turning off CSRF protection for easier testing
# (using Flask-WTF or similar)
app.config["WTF_CSRF_ENABLED"] = False

# Using a weak, predictable secret key
app.config["SECRET_KEY"] = "dev"

# Logging everything at DEBUG level (libraries may log request bodies,
# headers, or – with an ORM such as SQLAlchemy – every SQL query and its data)
logging.basicConfig(level=logging.DEBUG)

# Allowing all CORS origins (no restriction on who can call the API)
# (using Flask-CORS)
CORS(app, origins="*") # Fine for development, dangerous in production
```

Each of these is acceptable in a local development environment. Each is dangerous in production. The problem is when they are hardcoded into the source code rather than controlled by environment-specific configuration.

The correct approach

Debug mode and all debug-related settings should be controlled by environment variables:

Listing 9.9 Environment-controlled debug mode

```
import os

DEBUG = os.getenv("FLASK_DEBUG", "false").lower() == "true"

app.config["DEBUG"] = DEBUG
app.config["SECRET_KEY"] = os.environ["SECRET_KEY"]
app.config["WTF_CSRF_ENABLED"] = not DEBUG # CSRF on in production, optionally off
↳ in dev

if __name__ == "__main__":
    app.run(debug=DEBUG, host="localhost", port=int(os.getenv("PORT", "5000")))
```

In the development `.env`:

```
FLASK_DEBUG=true
SECRET_KEY=dev-only-secret-key-change-in-production
```

In the production environment (set by the server administrator, not in a file that could be committed):

```
FLASK_DEBUG=false
SECRET_KEY=a-long-random-cryptographically-secure-string
```

The source code is identical in both environments. The behavior differs only because the environment variables differ.

9.4 Environment-Specific Behavior Buried in Code

The fourth form of configuration coupling is the most subtle: logic in the code that branches based on which environment it is running in.

The if-else environment check

Listing 9.10 Environment-specific logic in code

```

import os

ENVIRONMENT = os.getenv("ENVIRONMENT", "development")

def send_email(to: str, subject: str, body: str):
    if ENVIRONMENT == "production":
        # Send a real email via SMTP
        send_via_smtp(to, subject, body)
    elif ENVIRONMENT == "staging":
        # Send to the staging test mailbox
        send_via_smtp("staging-test@example.com", f"[STAGING] {subject}", body)
    elif ENVIRONMENT == "development":
        # Just print to console
        print(f"[DEV EMAIL] To: {to}, Subject: {subject}\n{body}")
    else:
        raise ValueError(f"Unknown environment: {ENVIRONMENT}")

```

This code contains explicit knowledge of every environment it might run in. The number of environments may grow (adding testing, ci, demo). The code must be updated every time. The development behavior and production behavior are tangled in the same function.

This approach has a deeper problem: the behavior of the code depends on a string value ("production", "staging", "development"). If someone runs the staging environment with `ENVIRONMENT=production` (accidentally or intentionally), real emails are sent. If someone runs production with `ENVIRONMENT=development`, no emails are sent and nobody knows.

The correct approach: behavior through configuration, not environment name

Instead of branching on environment name, configure the behavior directly:

Listing 9.11 Behavior through configuration, not environment name

```

import os

# Each environment configures these variables independently:
SMTP_HOST = os.getenv("SMTP_HOST", "localhost")
SMTP_PORT = int(os.getenv("SMTP_PORT", "1025")) # 1025 = Mailpit, a local fake SMTP
↪ server
EMAIL_OVERRIDE = os.getenv("EMAIL_OVERRIDE") # If set, send all emails here

def send_email(to: str, subject: str, body: str):
    # No if-else on environment name
    # Just use the configured values
    actual_to = EMAIL_OVERRIDE or to
    actual_subject = f"[OVERRIDE] {subject}" if EMAIL_OVERRIDE else subject
    send_via_smtp(actual_to, actual_subject, body, host=SMTP_HOST, port=SMTP_PORT)

```

Development `.env`:

```

SMTP_HOST=localhost
# Mailpit, a local fake SMTP server that catches every message
SMTP_PORT=1025
# Empty: no override
EMAIL_OVERRIDE=

```

Staging `.env`:

```

SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
# All emails go to this one inbox
EMAIL_OVERRIDE=staging-test@example.com

```

Production environment:

```

SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
# Empty: real emails go to real recipients
EMAIL_OVERRIDE=

```

(Comments go on their own lines: some tools that read environment files — including the `EnvironmentFile`= setting of `systemd` used later in this book — do not support comments at the end of a line.)

The code now has no knowledge of what environment it is in. It simply reads configuration and acts accordingly. This approach scales correctly as environments are added or changed — no code changes required, only configuration changes.

Configuration files per environment

A related anti-pattern is multiple configuration files, one per environment:

Listing 9.12 Multiple config files (anti-pattern)

```
# config/development.py
DEBUG = True
DATABASE_URL = "sqlite:///dev.db"
SECRET_KEY = "dev-key"

# config/production.py
DEBUG = False
DATABASE_URL = "postgresql://prod-server/notesdb"
SECRET_KEY = "prod-key-123abc"

# app.py
import os
env = os.getenv("ENVIRONMENT", "development")
if env == "production":
    from config.production import *
elif env == "development":
    from config.development import *
```

This pattern has two problems. First, `config/production.py` contains production secrets. It will be committed to version control, exposing those secrets. Second, the code branches on environment name, recreating the if-else problem at the configuration level.

The correct pattern is a single configuration that reads its values — above all its secrets — from environment variables, with no secrets in any committed file. (Some mature projects keep per-environment configuration *classes* for non-secret defaults; that is workable as long as secrets still come from the environment and the code does not branch on the environment's name.)

Configuration Coupling Checklist

Before deploying, audit the codebase for these patterns:

Listing 9.13 Patterns to search for and eliminate

```

# BAD: Any string that looks like a secret
app.config["SECRET_KEY"] = "...                # REMOVE
API_KEY = "sk_live_..."                      # REMOVE
PASSWORD = "...                               # REMOVE

# BAD: Hardcoded database paths or URLs
DATABASE_PATH = "/Users/yourname/..."        # REMOVE
DATABASE_URL = "postgresql://user:pass@host/db" # REMOVE

# BAD: Hardcoded IP addresses or hostnames
REDIS_HOST = "192.168.1.100"                  # REMOVE
SMTP_SERVER = "mail.internal.company"         # REMOVE

# BAD: Hardcoded debug flags
app.run(debug=True)                           # REMOVE (or control via env var)
app.config["DEBUG"] = True                    # REMOVE

# BAD: Environment-name branching
if ENVIRONMENT == "production":               # REMOVE this pattern
    ...

```

You can search for these patterns automatically:

```

# Search for common secret patterns in source code
grep -r "secret" . --include="*.py" -i
grep -r "password" . --include="*.py" -i
grep -r "api_key\|apikey" . --include="*.py" -i
grep -r "debug=True" . --include="*.py"
grep -r "/Users/" . --include="*.py"

```

There are also automated tools that scan code and git history for secrets:

- **git-secrets:** prevents secrets from being committed
- **detect-secrets:** scans a repository for potential secrets
- **truffleHog:** searches git history for high-entropy strings

Key Takeaways

- **Configuration coupling** is embedding configuration values — especially secrets — directly in source code. It breaks the principle that the code you test should be the code you deploy.
- **Hardcoded secrets** (passwords, API keys, signing tokens) in source code are a critical security risk. They enter version control history, are visible to all team members, and can be accidentally exposed. Move all secrets to environment variables.
- **Local database references** (file paths, connection URLs) hardcoded in source code prevent the same codebase from running against different databases in different environments. Database URLs belong in environment variables.
- **Debug configurations** (`debug=True`, weak secret keys, CSRF disabled) must never be hardcoded. The Flask interactive debugger is a live remote code execution vulnerability when enabled in production.
- **Environment-specific behavior in code** (if-else on environment name) creates a maintenance burden and introduces the risk of wrong behavior running in the wrong environment. Prefer behavior-controlling configuration variables over environment-name branching.
- Use automated tools (`git-secrets`, `detect-secrets`) to prevent secrets from entering version control. Treat any accidentally committed secret as fully compromised.

Exercises

1. **Predict.** Search your project for secrets, in the current files and in all of history: `git grep -n -i secret`, then the same for `password` and `key`, and `git log -p | grep -i secret`. What do you find, and would it matter if the repository became public?
2. **Break and fix.** Commit a fake secret on a throwaway branch, then remove it in a new commit. Is it gone? Show where it can still be found, and describe the only real fix.
3. **Extend.** Install `detect-secrets` (or `gitleaks`) as a pre-commit hook and prove that it blocks a commit containing an obvious key.
4. **Explain.** Why is `if ENV == "production": ...` in application code worse than a configuration variable that controls the behaviour directly?

Solutions and hints are in the companion repository, in `exercises/chapter-09.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

You have now seen how configuration written into the code ties a program to one environment. The next chapter examines the networking constraints that make your local server invisible to the outside world — and explains why those constraints exist and what must change before others can reach your API.

→ [Chapter 10](#) — Networking Limitations

Networking Limitations

The previous two chapters looked at problems with environment differences and configuration. This chapter examines a different category of local limitation: the networking constraints that make your local server fundamentally unreachable from anywhere else.

When your Flask server runs on `localhost:5000`, it works from your own browser. A friend sitting at the same table — using the same Wi-Fi network — cannot reach it. A colleague on the other side of the world certainly cannot. A mobile app user would have no path to your API at all.

This is not a configuration problem. It is not a software bug. It is a fundamental property of how local network addressing works. Your server is **invisible** to the outside world, by design. The loopback interface, the absence of a public IP, the lack of DNS, and a home router that forwards no ports inward all combine to ensure that `localhost:5000` stays on your machine.

Understanding why this invisibility exists is essential. It explains what must be built to make the server reachable — and why the infrastructure in Stages 5, 6, and 7 is not optional decoration but structural necessity.

In this chapter

By the end of this chapter, you will understand:

- What the loopback address `127.0.0.1` is and why it cannot be reached from outside
- What a public IP address is and why your laptop probably does not have one
- Why `localhost` has no meaning outside your machine
- What firewall rules are and why their absence is both a convenience and a vulnerability

10.1 The Loopback Address: Why `127.0.0.1` Is Invisible to Others

In Chapter 4 (Listing 4.11), the Notes API was started with:

```
app.run(debug=True, host="localhost", port=5000)
```

`localhost` resolves to the IP address `127.0.0.1`. This is the **loopback address** — and understanding what it is explains why no one else can reach your server.

What the loopback interface is

Every computer running a TCP/IP network stack has a **loopback interface** — a virtual network interface that routes traffic back to the same machine. It has no physical network card, no cable, no radio transmitter. It is entirely software.

The loopback interface is identified by the IP address range `127.0.0.0/8` — meaning any IP address from `127.0.0.1` to `127.255.255.254`. The commonly used address is `127.0.0.1`. The corresponding hostname is `localhost`.

Traffic sent to `127.0.0.1` never leaves the machine. The kernel's network stack recognizes the address, routes the traffic internally, and delivers it back to the machine without involving any physical network hardware.

```
# Confirm that localhost resolves to 127.0.0.1
ping localhost
# PING localhost (127.0.0.1): 56 data bytes
# 64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.031 ms
```

The round-trip time is 0.031 milliseconds — essentially instantaneous, because the traffic never leaves the CPU.

Why loopback traffic cannot reach another machine

If you send a packet to `127.0.0.1`, the kernel's routing table sends it straight to the loopback interface — it can never be directed outward, and it is delivered to the local machine. There is no path from `127.0.0.1` on your laptop to `127.0.0.1` on another machine. Every machine has its own `127.0.0.1`, and they are all completely isolated.

From a colleague's machine, there is no IP address to type that reaches your `localhost:5000`. They cannot even attempt the connection, because `127.0.0.1` on their machine is their own loopback, not yours.

The practical consequence

When Flask binds to `localhost:5000`, it registers in the kernel:

“Accept connections on the loopback interface, port 5000.”

The kernel will only deliver connections that arrive on the loopback interface. Connections arriving on the machine’s physical network interface (the Wi-Fi card, the Ethernet port) are not delivered to this socket. Even if someone on your local network knows your laptop’s IP address and tries to connect to port 5000, the kernel will refuse the connection — because the socket is not listening on that interface.

```
Your laptop (192.168.1.5)
├── Loopback interface: 127.0.0.1
│   ├── Flask listens here (localhost:5000)
│   └── Connections from: ONLY this machine
└── Wi-Fi interface: 192.168.1.5
    ├── Flask does NOT listen here
    └── Connection attempts from your network: REFUSED
```

To accept connections from outside the machine, Flask must bind to the machine’s non-loopback IP address or to `0.0.0.0` (which means all interfaces). We address this in → [Chapter 24 — Process Startup in Production](#).

10.2 No Public IP: What Your Machine Looks Like from the Internet

Even if you configure Flask to bind to `0.0.0.0` (making it accessible within your local network), the internet cannot reach it. The reason is that your laptop almost certainly does not have a **public IP address**.

Public vs private IP addresses

The internet is structured around two categories of IP addresses:

Public IP addresses are globally unique. Every public IP address on the internet refers to exactly one device (or one network, via NAT). When a server at `216.58.194.46` receives a packet, that packet is delivered to that specific server and no other. Public IPs are assigned by internet service providers (ISPs) and registries (like ARIN, RIPE, APNIC). They are finite and costly.

Private IP addresses are used inside private networks (homes, offices, corporate networks). They are defined by three reserved ranges:

- `10.0.0.0/8` — addresses from `10.0.0.1` to `10.255.255.254`

- 172.16.0.0/12 — addresses from 172.16.0.1 to 172.31.255.254
- 192.168.0.0/16 — addresses from 192.168.0.1 to 192.168.255.254

Private addresses are not globally unique. Millions of devices worldwide share the address 192.168.1.100. That is fine because they are in different private networks — they never communicate with each other directly.

Your laptop's IP address — the one assigned by your home Wi-Fi router — is almost certainly a private address like 192.168.1.5. You can check:

```
# macOS
ipconfig getifaddr en0  # en0 is usually the Wi-Fi interface
# Example output: 192.168.1.5

# Linux
ip addr                  # look for your Wi-Fi or Ethernet interface (e.g. wlp2s0,
↳ enp0s3)
# Example: inet 192.168.1.5/24
```

Network Address Translation (NAT)

If your IP is private, how does your laptop browse the internet? The answer is **Network Address Translation (NAT)**.

Your home router has one public IP address (assigned by your ISP). When your laptop sends a packet to a server on the internet:

1. Your laptop sends the packet to the router with its private source IP (192.168.1.5)
2. The router replaces the source IP with its public IP (203.0.113.42) and records the translation
3. The packet reaches the internet server, which sees it as coming from 203.0.113.42
4. The server responds to 203.0.113.42
5. The router receives the response, looks up the translation, and forwards it to your laptop (192.168.1.5)

This is NAT. Your laptop can initiate outgoing connections to the internet. But inbound connections — someone else trying to reach your laptop — are blocked. The router has no way to know which device behind it should receive unsolicited inbound connections.

This is why running a server on your laptop does not expose it to the internet. NAT and private IP addressing are architectural barriers that prevent inbound connections.

Cloud servers have public IPs

A server on AWS, DigitalOcean, Google Cloud, or any other cloud provider gets a public IP address (or can be configured to get one). This is one of the key differences between a developer's laptop and a production server. The public IP is what allows traffic from anywhere on the internet to reach the server directly.

We configure public IPs and the associated routing in → [Chapter 22 — Network Configuration](#).

10.3 No DNS Mapping: Why localhost Is Meaningless Outside Your Machine

Even if your Flask server had a public IP, connecting to it as an IP address is inconvenient and fragile (IP addresses change). Real services are accessed by domain names like `api.notes.example.com`. This is what DNS provides.

What DNS is

DNS (Domain Name System) is a distributed database that maps human-readable domain names to IP addresses. When you type `api.notes.example.com` in a browser:

1. Your browser asks a **DNS resolver** (usually your ISP's or Google's `8.8.8.8`): "What is the IP address for `api.notes.example.com`?"
2. The resolver queries the global DNS hierarchy, ultimately finding the **authoritative nameserver** for `example.com`
3. That nameserver responds: "The IP address for `api.notes.example.com` is `203.0.113.42`"
4. Your browser connects to `203.0.113.42`

This lookup happens before the connection is established. Without DNS, clients would need to know the raw IP address of every server they want to reach.

localhost is a local DNS entry

`localhost` resolves to `127.0.0.1` through a special mechanism. Every machine has a **hosts file** — a local text file that maps hostnames to IP addresses, checked before DNS queries:

- On Linux/macOS: `/etc/hosts`
- On Windows: `C:\Windows\System32\drivers\etc\hosts`

```
cat /etc/hosts
# 127.0.0.1    localhost
# ::1         localhost
```

This entry exists on every machine, by convention. It means `localhost` always resolves to `127.0.0.1` — but this resolution is local. It does not exist in any global DNS system. There is no way to make your `localhost` resolvable from another machine.

Your laptop has no DNS entry

Your laptop's private IP address (`192.168.1.5`) is not in any public DNS database. No domain name points to it. Even if someone wanted to reach your Flask server by domain name, they could not — there is nothing to resolve.

When you deploy to a server with a public IP, you can create a DNS record that maps your domain to that IP:

```
api.notes.example.com. A 203.0.113.42
```

After this DNS record is created and propagated, anyone in the world can connect to `api.notes.example.com` and reach the server at `203.0.113.42`.

We create DNS records in → [Chapter 27](#) — Domain Names and DNS.

The `/etc/hosts` workaround for local development

For testing purposes, you can add entries to your local `/etc/hosts` to simulate domain names:

```
# /etc/hosts — add this line for local development
127.0.0.1    notes.test
```

After this, `http://notes.test:5000` works in your browser (use a reserved name such as `.test`; the `.local` suffix belongs to multicast DNS and causes slow lookups on macOS) — but only on your machine. Other machines do not have this entry.

This is a useful development trick for simulating domain-based routing (for example, when testing a reverse proxy configuration locally), but it is not a substitute for real DNS.

10.4 No Firewall Rules: Why Local Security Is Not Real Security

The final networking limitation of a local development setup is the absence of a properly configured firewall. On a developer's machine, the firewall is typically either disabled or configured

very permissively to allow easy access during development. This is convenient — you do not have to think about firewall rules to start your server and test it. But it creates a false sense of security.

What a firewall is

A **firewall** is a system (hardware or software) that filters network traffic based on rules. Rules specify which traffic is allowed and which is blocked, based on:

- Source IP address
- Destination IP address
- Protocol (TCP, UDP, ICMP, etc.)
- Port number
- Direction (inbound vs outbound)

A basic firewall rule for a production web server might say:

- Allow inbound TCP on port 80 (HTTP) from any source
- Allow inbound TCP on port 443 (HTTPS) from any source
- Allow inbound TCP on port 22 (SSH) from the operations team’s IP range only
- Block all other inbound traffic

Without these rules, every port that any process binds to is potentially reachable from the network.

What happens without a firewall on a developer’s machine

When you bind Flask to `0.0.0.0:5000` on your laptop (to make it accessible on your local network), you are exposing port 5000 to every device on that network. If you are on a home Wi-Fi network, this is probably fine — you trust the devices on your network. But:

- In a coffee shop, on a public Wi-Fi network, any device in the café can reach your Flask server
- If Flask is running in debug mode (`debug=True`), any device in the café has a live Python interpreter in your application
- If your application has any authentication vulnerabilities, any device in the café can exploit them

A server bound to `0.0.0.0` in debug mode on a public network is an easy target. The developer is testing in what they consider a private environment (their laptop), but the “private”

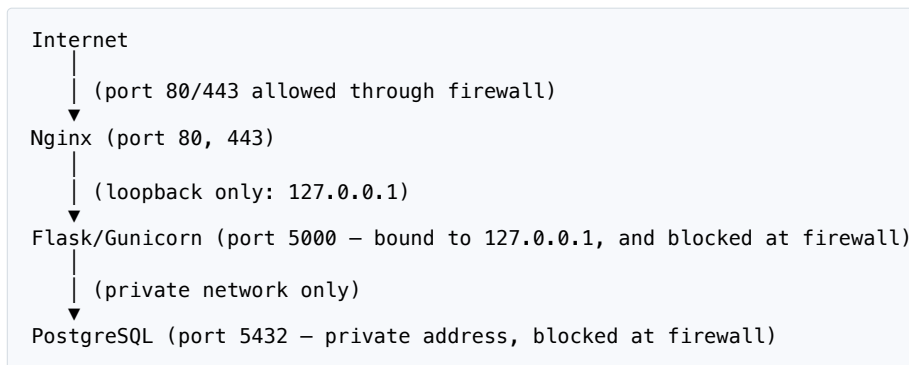
environment is actually a public coffee shop Wi-Fi. (On macOS, the built-in firewall asks for permission the first time a program listens on all interfaces — a prompt worth reading rather than clicking away.)

What firewall rules accomplish on a production server

On a production server, firewall rules are a critical security layer. A properly configured server:

- **Accepts port 80 and 443:** These are the web traffic ports. HTTP and HTTPS requests arrive here.
- **Accepts port 22 from specific IPs:** SSH is only accessible from the operations team's IPs or a VPN. Anyone else who tries to SSH is blocked before the connection even reaches the server.
- **Blocks everything else:** Port 5000 (where the Flask app might listen internally), port 5432 (PostgreSQL), port 6379 (Redis) — these are internal ports. They should not be reachable from the internet.

This is called **defense in depth**: each internal service is protected in two independent ways. First, it listens only where it must — the application on the loopback address (127.0.0.1:5000), so only processes on the same machine, such as Nginx, can reach it. Second, the firewall blocks the port anyway, in case the first layer is ever misconfigured.



On a developer's laptop, none of these rules exist. Flask on port 5000 is directly reachable (subject to the loopback/NAT constraints). The database on port 5432 is directly reachable from any process on the machine. There is no defense in depth.

Cloud security groups

Cloud providers implement these rules outside the server: **security groups** on AWS, **network security groups** on Azure, and **firewall rules** on Google Cloud and DigitalOcean. These are configured at the network level, before traffic reaches the server.

We configure these in → [Chapter 22 – Network Configuration](#).

The Networking Picture: Local vs Production

Here is the complete networking difference between a local development setup and a production server:

LOCAL DEVELOPMENT

Flask binds to: localhost:5000 (127.0.0.1 – loopback only)
 IP address: 192.168.1.5 (private – not internet-routable)
 NAT: Router translates outbound; blocks inbound
 DNS: localhost resolves locally; no public DNS entry
 Firewall: Typically disabled or permissive
 Reachable from: Only this machine (via loopback)
 Possibly local network (if bound to 0.0.0.0 and the laptop's firewall allows it)
 NOT from the internet

PRODUCTION SERVER

Flask/Gunicorn binds to: 127.0.0.1:5000 (loopback – only Nginx reaches it)
 Nginx binds to: 0.0.0.0:80 and 0.0.0.0:443 (publicly)
 IP address: 203.0.113.42 (public – internet-routable)
 NAT: Not applicable – direct internet connectivity
 DNS: api.example.com → 203.0.113.42 (global DNS)
 Firewall: Allow 80, 443 inbound; block everything else
 Reachable from: Anywhere on the internet (via api.example.com)
 Port 5000 is also blocked at the firewall

The gap between these two situations is the work of Stages 5, 6, and 7. That work cannot be skipped or compressed. Every element of the production networking setup – the public IP, the DNS entry, the firewall rules, the reverse proxy – is structurally required for the service to be reachable.

Key Takeaways

- The loopback address (**127.0.0.1**) routes traffic back to the same machine. It is not routable to or from any other machine. Binding Flask to `localhost` makes it invisible to all other machines.
- Private IP addresses (192.168.x.x, 10.x.x.x, 172.16.x.x–172.31.x.x) are not

internet-routable. Your laptop’s IP address is almost certainly private.

- **NAT (Network Address Translation)** allows outbound internet connections from private networks but blocks unsolicited inbound connections. Your laptop cannot receive direct connections from the internet.
- **DNS** maps domain names to IP addresses. `localhost` is a local-only entry in `/etc/hosts`. Your laptop has no public DNS record. Without a public DNS entry, domain-based addressing is impossible.
- **Firewalls** control which ports accept traffic from which sources. Local development typically has no effective firewall. Production servers have strict rules that allow only specific ports (80, 443) from the internet.
- The networking gap between development and production is structural, not configurational. Closing it requires a public IP, a DNS record, a reverse proxy, and firewall rules — the work of Stages 5, 6, and 7.

Exercises

1. **Predict.** Look up your laptop’s IP address (`ipconfig getifaddr en0` or `ip addr`) and your public IP (`curl ifconfig.me`). Why are they different?
2. **Break and fix.** Add `127.0.0.1 example.com` to `/etc/hosts` and open `http://example.com:5000`. What have you changed, and for whom? Remove the line.
3. **Extend.** With `nc -l 9000` running on your laptop, try to connect from your phone on mobile data (not Wi-Fi). Explain the result in terms of NAT.
4. **Explain.** List the four things the Notes API needs before a stranger on the internet can reach it (see “The Networking Picture” in this chapter), and the chapter that provides each.

Solutions and hints are in the companion repository, in `exercises/chapter-10.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What’s Next

The networking limitations are inherent and cannot be worked around locally. But even within the local environment, there are operational gaps that make the local setup unsuitable for anything resembling production: no process restart, no monitoring, no HTTPS, and no authentication. These are the reliability and security gaps that the final chapter of Stage 2 examines.

→ Chapter 11 — Reliability and Security Gaps

Reliability and Security Gaps

The previous four chapters examined what makes a local development setup fragile: environment drift, hidden dependencies, configuration coupling, and networking limitations. These are problems of portability and reachability — the system cannot survive being moved, and it cannot be reached from outside.

This final chapter of Stage 2 examines a different category of problems: operational gaps. Even if we somehow solved all the portability and reachability issues — somehow put the local server on a public IP with a domain name — it would still not be a production system. It would be a fragile, invisible, unencrypted, unauthenticated server that crashes once and never comes back.

The difference between a server that runs and a service that operates lies in four properties: it must restart when it crashes, it must be observable (you can know what it is doing), it must be encrypted (communication is private), and it must be authenticated (access is controlled). A local development setup has none of these.

This chapter explains each gap clearly. The gaps are not argued from theory — they are demonstrated with concrete failure scenarios. The solutions are not detailed here (they appear in Stages 6, 9, and 10), but after reading this chapter you will understand exactly what the solutions need to provide.

In this chapter

By the end of this chapter, you will understand:

- What happens when a process crashes and there is nothing to restart it
- Why running blind without monitoring is operationally untenable
- Why HTTP without HTTPS is not acceptable for real user data
- What authentication and authorization are, and why their absence is a security failure

11.1 No Process Restart: What Happens When Your Program Crashes

Flask’s development server runs as a single Python process. When you press Ctrl+C, it stops. But processes do not stop only on command — they also stop unexpectedly.

Ways a process can die

A Python process can terminate for many reasons beyond your control:

Unhandled exceptions at startup: An exception that propagates all the way up the call stack without being caught causes the Python interpreter to print a traceback and exit. Inside a request handler, Flask protects you — it catches the exception and turns it into a `500 Internal Server Error` response, and the process keeps running:

Listing 11.1 An unhandled exception in a handler becomes a 500, not a crash

```
@app.route("/notes/<int:note_id>")
def get_note(note_id):
    db = get_db()
    row = db.execute("SELECT * FROM notes WHERE id = ?", (note_id,)).fetchone()
    # If row is None and we don't check, the next line raises TypeError
    return jsonify({"content": row["content"]}) # TypeError if row is None
```

If `note_id` does not exist, `row` is `None`, and `None["content"]` raises `TypeError: 'NoneType' object is not subscriptable`. Flask catches it: the client gets a 500 (or, in debug mode, the interactive debugger) and the server carries on. But the same kind of mistake in code that runs at *startup* — a typo in a module-level line, a missing environment variable read at import time — has no framework to catch it, and the process exits before it serves a single request.

Memory exhaustion (OOM): If the process allocates more memory than the system has available, the Linux kernel’s **Out-Of-Memory (OOM) killer** selects a process to terminate. It chooses the process with the largest memory footprint, which is often your application. The kernel terminates it with `SIGKILL` — a signal that cannot be caught or ignored. The process is gone. No error message. No traceback. Just silence.

Signals: The operating system can send signals to processes. `SIGKILL` (kill -9) terminates a process immediately. `SIGSEGV` (segmentation fault — rare in Python, but possible with certain C extensions) terminates the process with a core dump.

Resource exhaustion: Too many open file descriptors, disk full, or network interface failure can cause a process to fail.

Hardware failures: A server running virtual machines may experience a hypervisor failure. The underlying machine may reboot for maintenance.

What happens when the process dies and nothing restarts it

On a developer's laptop, when the Python process crashes, the developer notices (the server is gone) and types `python app.py` again. This takes seconds. It is fine.

On a production server, when the process crashes, nobody notices — until a user's request fails. Depending on the time of day, this could be minutes or hours later. During that entire period:

- Every request to the API returns a “connection refused” error
- Users see errors or a non-functional application
- No alert is sent (there is no monitoring)
- No automatic recovery happens (there is no process supervisor)

The server is down. It stays down until someone manually notices, logs in, and restarts it.

This is operationally unacceptable. A service that requires human intervention to recover from every crash cannot provide reliable uptime.

The solution: process supervisors

A **process supervisor** is a system service that manages application processes: it starts them, monitors them, and restarts them when they die. On Linux, the standard process supervisor is **systemd**.

With a supervisor in place (→ [Chapter 24](#) and → [Chapter 25](#): Docker's restart policy, backed by systemd), when the application process crashes, it is detected within seconds and restarted automatically. The requests in flight at that moment fail; users do not see prolonged downtime.

In container orchestrators, Kubernetes provides similar restart behavior through its pod restart policy (→ [Chapter 43](#)).

11.2 No Monitoring: Running Blind Without Visibility

The development server prints request logs to the terminal:

```
127.0.0.1 - - [01/Jan/2024 12:00:00] "GET /notes HTTP/1.1" 200 -
127.0.0.1 - - [01/Jan/2024 12:00:05] "POST /notes HTTP/1.1" 201 -
```

When you are sitting at your laptop, watching the terminal, these logs tell you what is happening. When you close the terminal, the logs disappear. When the process restarts, the history is gone.

On a production server, nobody sits watching terminal output. You need to know what is happening without being present.

What monitoring provides

Logging is the practice of writing a persistent record of events to a durable location (a file or a log aggregation service). Unlike terminal output that disappears, logs persist and can be searched after the fact.

Metrics are numerical measurements of system behavior over time: request count, error rate, response time, memory usage, CPU usage. Metrics allow you to answer questions like “how many requests per second is the server handling?” and “has error rate increased in the last hour?”

Monitoring is the process of watching metrics and logs, detecting anomalies, and alerting humans when something requires attention.

Without these, you are running blind. Specific consequences:

You cannot detect problems before users do. If response times are increasing (a warning sign of a saturated database or memory leak), you will not know until requests start failing outright and users complain. With monitoring, you detect the trend early and intervene before users are affected.

You cannot investigate past failures. When a user reports that “the API was broken around 3pm yesterday,” you need logs from 3pm yesterday to investigate. Without persistent logs, that information is gone.

You cannot answer basic capacity questions. “Should we scale up the server before the marketing campaign launches?” requires knowing current request volume, peak load, and resource usage. Without metrics, you are guessing.

You cannot audit access. Regulatory requirements (GDPR, HIPAA, SOC 2, PCI DSS) often require audit logs of who accessed what data and when. Without structured logging, you cannot provide this.

The Flask development server’s logging limitations

Flask’s development server logs requests in **Common Log Format (CLF)**:

```
127.0.0.1 - - [01/Jan/2024 12:00:00] "GET /notes HTTP/1.1" 200 -
```

This format has several limitations for production use:

1. **Output destination:** it goes to stderr, which in development goes to the terminal and disappears when the terminal closes

2. **Format:** CLF is human-readable but hard to parse programmatically; modern log analysis tools work better with structured JSON logs
3. **Information content:** it logs the IP, method, path, and status code, but not the user, the duration, the request body, or application-specific context
4. **No log rotation:** no mechanism to prevent log files from growing indefinitely

Production logging uses a proper logging system (Python's `logging` module, configured for structured output), writes to persistent files or a log aggregation service, and includes all the context needed for investigation.

We build production-ready logging in → [Chapter 16](#) — Logging and Error Handling and → [Chapter 35](#) — Observability.

11.3 No HTTPS: Why HTTP Is Not Safe Enough

The Notes API runs over plain HTTP. Every request and response travels as unencrypted text across the network. For local development on the loopback interface, this is harmless — the traffic never leaves the machine. For a production service accessible from the internet, it is a fundamental security failure.

What HTTP looks like on the wire

When a client sends a `POST /notes` request with authentication:

```
POST /notes HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{"content": "My private note about my medical condition"}
```

Over HTTP, this is transmitted as exactly that — plain text. Any device that can intercept the network packets sees the complete request, including:

- The full content of the note
- The Authorization header containing the user's authentication token
- The fact that this user is creating notes (metadata is also sensitive)

Who can intercept HTTP traffic

On a local network: on an open Wi-Fi network (no password), any nearby device can capture everyone's traffic with a wireless card in monitor mode. On any shared network — coffee shop, hotel, conference — an attacker can also redirect other devices' traffic through their own machine (for example with ARP spoofing) and read or modify it. Protected Wi-Fi (WPA2/WPA3) encrypts each device's traffic to the access point, but the network's operator still sees everything.

At the ISP level: Internet Service Providers can and do inspect customer traffic for various purposes (content filtering, government surveillance, targeted advertising). Without HTTPS, your traffic is readable to them.

At transit routers: Traffic travels through many routers between client and server. Each router sees the packet in transit and could, in principle, read its contents.

By malicious DNS or ARP poisoning: Attacks that redirect traffic through an attacker's machine (man-in-the-middle attacks) are significantly easier against HTTP than HTTPS. HTTPS authentication (certificate verification) makes man-in-the-middle attacks much harder.

What HTTPS provides

HTTPS is HTTP running over TLS (Transport Layer Security), a cryptographic protocol that provides:

1. **Encryption:** the request and response content are encrypted between the client and server. An intercepting third party sees only encrypted bytes, not the content.
2. **Authentication:** the server presents a **TLS certificate** — a cryptographically signed document that proves the server is who it claims to be. The client's browser verifies this certificate. Without a valid certificate, the browser shows a security warning.
3. **Integrity:** TLS uses cryptographic message authentication codes (MACs) that detect if data was modified in transit. An attacker cannot inject or alter data without the client detecting it.

Without HTTPS:

- Any user credentials, session tokens, or API keys transmitted over the network are exposed
- Any data in transit (note content, personal information) is readable
- Users cannot verify that they are talking to the real server (not an impostor)
- Modern browsers show “Not Secure” for HTTP sites and may block certain features (geolocation, camera access, service workers)

Why local development uses HTTP

Setting up HTTPS requires a TLS certificate signed by a trusted Certificate Authority (CA). Certificates for localhost from public CAs are not issued. For local development, you have options:

1. **HTTP only:** acceptable on the loopback interface where no interception is possible
2. **Self-signed certificates:** create your own certificate, but browsers will show security warnings
3. **mkcert:** a tool that creates a local CA trusted by your browser, allowing localhost HTTPS without warnings:

```
# Install mkcert (macOS with Homebrew)
brew install mkcert
mkcert -install # Install local CA into system trust store
mkcert localhost 127.0.0.1 # Generate certificate
# Creates localhost.pem and localhost-key.pem
```

```
# Use the certificate with Flask's development server (development only)
from flask import Flask

app = Flask(__name__)

if __name__ == "__main__":
    app.run(ssl_context=("localhost.pem", "localhost-key.pem"))
```

For production, TLS certificates come from public CAs. **Let's Encrypt** provides free certificates, automated through the certbot tool. We set this up in → [Chapter 28 — Traffic Routing Layer](#).

11.4 No Authentication or Security Layers

The Notes API, as built in Stage 1, accepts requests from anyone. There is no concept of a user. There are no access controls. Anyone who can reach the API can create, read, and delete any note.

In a local development setup, this is fine — the API is only accessible from your own machine, so “anyone” means only you. In production, “anyone” means the entire internet.

Authentication vs authorization

Two distinct security concepts are often conflated:

Authentication is the process of verifying who someone is. “Are you who you claim to be?” The most common forms are:

- Username and password (user provides credentials, server verifies)
- API keys (client presents a key, server verifies it is valid)
- OAuth tokens (server issues a token after successful login, client presents the token)
- JWT (JSON Web Token) — a cryptographically signed token that carries claims about the user

Authorization is the process of determining what an authenticated user is allowed to do. “You are Alice. Are you allowed to delete note #42?” Authorization answers this question.

Both are required for a multi-user API. Authentication without authorization means every authenticated user can do everything. Authorization without authentication means access controls are meaningless (since there is no verified identity to apply them to).

The absence of authentication in the Notes API

The current Notes API has no authentication:

```
# This handler (from Listing 4.11) accepts requests from anyone
@app.route("/notes/<int:note_id>", methods=["DELETE"])
def delete_note(note_id):
    db = get_db()
    cursor = db.execute("DELETE FROM notes WHERE id = ?", (note_id,))
    db.commit()
    if cursor.rowcount == 0:
        return jsonify({"error": f"Note {note_id} not found"}), 404
    return jsonify({"message": f"Note {note_id} deleted"})
```

Any HTTP client — authenticated or not — can send DELETE /notes/42 and the note will be deleted. There is no check for who made the request or whether they are allowed to delete that note.

In a real Notes API with multiple users, this means:

- User A can delete User B’s notes
- An anonymous user can delete all notes with a simple loop
- An automated script can enumerate all note IDs and delete everything

What an authentication layer looks like

A minimal API key authentication approach:

Listing 11.2 Simple API key authentication middleware

```

import os
from flask import Flask, request, jsonify
import functools

app = Flask(__name__)

# In production, this comes from a database of valid API keys
# For illustration, we use a single environment variable
VALID_API_KEYS = {os.environ["API_KEY"]} # KeyError at startup if it is not set

def require_auth(f):
    """Decorator that enforces API key authentication."""
    @functools.wraps(f)
    def decorated(*args, **kwargs):
        # API key from header: Authorization: Bearer <key>
        auth_header = request.headers.get("Authorization", "")
        if not auth_header.startswith("Bearer "):
            return jsonify({"error": "Authorization header required"}), 401

        api_key = auth_header[len("Bearer "):]
        # (Chapter 38 replaces this simple set lookup with a constant-time
        ↪ comparison)
        if api_key not in VALID_API_KEYS:
            return jsonify({"error": "Invalid API key"}), 401

        return f(*args, **kwargs)
    return decorated

@app.route("/notes", methods=["GET"])
@require_auth
def list_notes():
    ...

@app.route("/notes/<int:note_id>", methods=["DELETE"])
@require_auth
def delete_note(note_id):
    ...

```

With this middleware, every request must include `Authorization: Bearer <key>`. Requests without a valid key receive a 401 Unauthorized response.

This is a minimal example. A real authentication system would:

- Store API keys in a database (not an environment variable)
- Hash keys before storing (so a database breach does not expose all keys)
- Support multiple users, each with their own keys
- Implement token expiration
- Rate-limit authentication attempts to prevent brute force

We address authentication more thoroughly in → [Chapter 38 — Security](#).

Other missing security layers

Beyond authentication, a production API needs several additional security measures:

Rate limiting: prevents a single client from overwhelming the server with requests (either accidentally or deliberately). Without rate limiting, a misbehaving client can consume all server resources, affecting every other user.

Input validation at the boundary: validating that incoming data conforms to expected types, sizes, and formats. Malicious input can cause SQL injection, buffer overflows (in C extensions), or resource exhaustion.

Content security: ensuring the application does not leak sensitive data in error messages, response headers, or log files.

CORS (Cross-Origin Resource Sharing): controlling which other websites may call your API from a user's browser. By default, browsers block a page on one site from reading responses from another site's API (the *same-origin policy*); CORS headers are how you selectively *relax* that for trusted front-ends. The danger is a careless configuration — allowing every origin while also allowing credentials — which reopens exactly the hole the browser was closing. (The related attack in which another site makes a user's browser send requests with their cookies is **CSRF**; APIs that authenticate with tokens in headers, as this one will, are not exposed to it.)

None of these are present in the local development setup. All of them are required in production.

Stage 2 Summary: The Complete Picture of Local Limitations

Having completed Stage 2, here is the full picture of what the local development setup lacks:

LOCAL DEVELOPMENT GAPS

ENVIRONMENT

- ☐ Tied to specific OS (macOS), version, file system behavior
- ☐ Tied to specific Python version (whichever happens to be first in \$PATH)
- ☐ System libraries may differ from production (libc, OpenSSL, SQLite)

DEPENDENCIES

- ☐ No requirements.txt (or incomplete one)
- ☐ Global packages may be leaking into the application
- ☐ External tools (binaries) are undocumented requirements
- ☐ Implicit assumptions (locale, timezone, ports) are undeclared

CONFIGURATION

- ☐ Secrets may be in source code
- ☐ Database path is machine-specific
- ☐ Debug mode may be hardcoded
- ☐ Environment-specific logic buried in code

NETWORKING

- ☐ Bound to localhost (invisible to others)
- ☐ Private IP (not internet-routable)
- ☐ No DNS entry
- ☐ No firewall rules

RELIABILITY

- ☐ No process restart on crash
- ☐ No persistent logging
- ☐ No metrics or monitoring
- ☐ No alerting

SECURITY

- ☐ No HTTPS (all traffic in plaintext)
- ☐ No authentication
- ☐ No rate limiting
- ☐ No input validation beyond basic checks
- ☐ No CORS configuration

This is not a list of failures. It is a list of things that are appropriate to defer during development but must be addressed before production. Stage 2 has named them all. Stages 3 through 10 address them, one by one, in the right order.

Key Takeaways

- **No process restart:** a process that crashes stays down until manually restarted. In production, a process supervisor (systemd) detects crashes and restarts automatically within seconds.
- **No monitoring:** Flask's development server logs to the terminal. In production, logs must be persistent, structured, and searchable. Metrics must be collected. Alerts must be configured to notify engineers before users notice problems.

- **No HTTPS:** HTTP traffic is unencrypted plaintext, readable by anyone who can intercept network packets. HTTPS (HTTP over TLS) encrypts all traffic, authenticates the server, and detects tampering. It is mandatory for any service handling real user data.
- **No authentication:** the Notes API accepts all requests without verifying who sent them. Production APIs require authentication (verifying identity) and authorization (verifying permission). Common mechanisms include API keys, OAuth tokens, and JWT.
- The combination of these gaps means the local development setup is not a production system. It is a development tool. Stages 3 through 10 convert it into something that can actually operate in the real world.

Exercises

1. **Predict.** While the development server runs, insert `raise RuntimeError("boom")` just above `if __name__ == "__main__":` in `app.py` and save (or kill `-9` the server). What brings it back? What would a user see meanwhile?
2. **Break and fix.** Capture your own HTTP traffic to the local API with `sudo tcpdump -i lo0 -A port 5000` while you send a note. Find the note's text in the capture. What would HTTPS change?
3. **Extend.** Write the list of alerts you would want if the Notes API were already live, before you know any tools: what should wake someone up, and what can wait until morning?
4. **Explain.** Explain, without jargon, the difference between authentication and authorization, using the Notes API as the example.

Solutions and hints are in the companion repository, in `exercises/chapter-11.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

Stage 2 is complete. You now have a precise understanding of every way the local setup is fragile, invisible, unportable, and insecure. Stage 3 begins the work of fixing these problems — starting with the most foundational: making the program's dependencies explicit.

→ Stage 3 — Making the Program Portable

→ Chapter 12 — Dependency Declaration

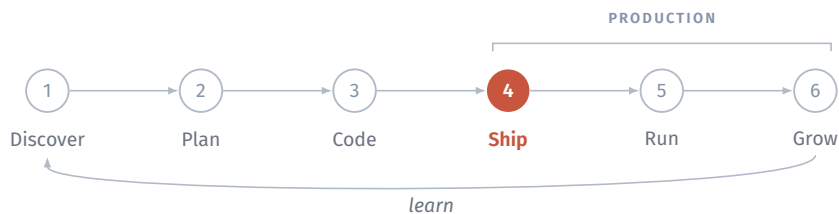
STAGE 3

3

Making the Program Portable

How is the system made portable?

YOU ARE HERE



The piece: The program, made ready to run anywhere

The whole map: *How Software Gets Built*, page 1

Chapters 12–16

Dependency Declaration

In [Chapter 6](#), we identified the layers the Notes API relies on: system libraries, the Python runtime, Python packages, and application configuration (plus its data). In [Chapter 8](#), we saw how hidden dependencies — packages installed globally or tools assumed to be present — cause failures that are impossible to diagnose without the right mental model.

The first and most important step toward portability is making the Python package dependencies explicit. Explicit means written down in a file that travels with the source code. When someone (or something — a deployment system, a Docker build, a CI pipeline) needs to recreate the application’s package environment from scratch, that file is the only instruction they need.

In Python, that file is `requirements.txt`.

`requirements.txt` is not glamorous. It is a plain text file with package names and version numbers. But it is the difference between a project that anyone can reproduce and a project that only works on the original developer’s machine. It is the foundational document of Python project portability.

This chapter explains exactly what `requirements.txt` is, how to generate it, what version locking means and why it matters, and how `pip` uses it to recreate an environment. It also introduces `pyproject.toml` — the modern alternative — and explains when each is appropriate.

In this chapter

By the end of this chapter, you will understand:

- What `requirements.txt` is and how it is structured
- How to generate it from an active virtual environment
- What version locking is, why it matters, and the difference between direct and transitive dependencies
- How `pip install -r requirements.txt` recreates an environment
- What `pip freeze` does and when to use it vs `pip list`
- The difference between `requirements.txt` and `pyproject.toml` and when to use each

12.1 `requirements.txt`: The Manifest of What Your Program Needs

A `requirements.txt` file is a plain text list of Python packages and their version requirements. Here is a minimal one for the Notes API at this stage:

```
# requirements.txt
flask==3.0.2
python-dotenv==1.0.1
```

Each line specifies a package. The `==` operator specifies the exact version to install. This file travels with the source code in the repository. When someone sets up the project for the first time:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

They get these two packages at exactly these versions. They also get Flask’s own dependencies — Werkzeug, Jinja2, and the rest — but those are *not* pinned yet, so they may be different versions from yours. Section 12.2 explains why that matters, and how to pin them too.

The structure of `requirements.txt`

Each line in `requirements.txt` is a **requirement specifier**. The syntax supports several forms:

```
# Exact version (recommended for deployment)
flask==3.0.2
```



```

# Minimum version (allows newer versions)
flask>=3.0.0

# Compatible release (allows minor updates, ~= 3.0 means >= 3.0, < 4.0)
flask~=3.0

# Range constraint
flask>=3.0.0,<4.0.0

# No version constraint (get the latest – not recommended for production)
flask

# Comments (lines starting with #)
# This is a comment

# Include another file (typically used in requirements-dev.txt to pull in the base
↪ list)
-r requirements.txt

# Install from a local path
-e ./local-package

# Install from a git repository
git+https://github.com/org/repo.git@main

```

For production deployments, exact versions (==) are strongly recommended. We explain why in Section 12.2.

What goes in requirements.txt

requirements.txt should list every package that import statements in your code rely on. These are your **direct dependencies** – the packages you explicitly use.

For the Notes API:

- flask: the web framework (direct dependency)
- python-dotenv: for loading .env files (direct dependency)

What does not go in requirements.txt:

- venv or pip themselves (these are tools, not dependencies)
- Packages that are part of Python’s standard library (sqlite3, os, json, sys – these come with Python)
- __pycache__ or .pyc files (these are build artifacts, not dependencies)

12.2 Version Locking: Why Exact Versions Matter

The `==` operator in `flask==3.0.2` specifies an exact version. This is called **version pinning** or **version locking**. It ensures that every time `pip install -r requirements.txt` runs, it installs exactly Flask 3.0.2 — not 3.0.3, not 3.1.0, not 4.0.0. Always and only 3.0.2.

Why is this important?

Packages change between versions

Flask 3.0.2 and Flask 3.0.3 are very similar — 3.0.3 fixes bugs or makes small improvements. But even a patch release might:

- Change the default behavior of a feature you rely on
- Deprecate an argument your code uses
- Fix a bug that your code accidentally depended on

Without version pinning:

```
# Without pinning, this might install Flask 3.0.2 today and a newer 3.x release
↪ tomorrow:
flask>=3.0.0
```

When a colleague sets up the project a week later, they get a different version of Flask than you used. Their environment differs from yours. The “but it worked on my machine” problem returns, but now at the package version level.

Real-world consequences of unpinned versions

Consider this (hypothetical) scenario:

- Monday: Developer A installs Flask 3.0.2, builds and tests the API, everything works
- Tuesday: a new patch release of Flask appears, fixing a security issue but also changing how `request.get_json()` treats a malformed body
- Wednesday: The CI/CD pipeline runs `pip install flask`, gets the new release, and tests the exact same code — but some tests now fail because `request.get_json()` behavior changed

Now the CI is failing but the developer’s machine passes. The versions are different. The investigation takes hours.

With pinned versions, Monday’s `flask==3.0.2` is what everyone gets. The CI and the developer’s machine behave identically.

Security updates and version pinning

There is a tension here. Pinning versions prevents automatic updates, including security updates. If a newer release fixes a security vulnerability in 3.0.2, your pinned application continues to use the vulnerable version.

The resolution is: pin versions for stability, but periodically update your pins deliberately. This is a manual process (or automated with tools like **Dependabot** or **Renovate**): update `requirements.txt`, re-run tests, verify everything works, then deploy the update.

This is better than unpinning versions because the update is a conscious decision that goes through testing. Automatic unpinning updates can introduce breaking changes at any time.

12.3 How pip Resolves and Installs Dependencies

When you run `pip install flask`, pip does not just install Flask. Flask itself has dependencies, and those dependencies have dependencies. pip resolves the entire tree.

The dependency resolution process

```
pip install flask
```

pip's internal resolver does the following:

Step 1: Fetch Flask's metadata. pip queries PyPI for Flask's metadata, which includes Flask's list of dependencies (specified in Flask's own `pyproject.toml`):

```
Flask 3.0.2 requires:
  Werkzeug >= 3.0.0
  Jinja2 >= 3.1.2
  itsdangerous >= 2.1.2
  click >= 8.1.3
  blinker >= 1.6.2
```

Step 2: Fetch each dependency's metadata. pip fetches the metadata for Werkzeug, Jinja2, etc. Each of those has its own dependencies (Jinja2 requires MarkupSafe, etc.).

Step 3: Resolve conflicts. If two packages require different versions of the same dependency, pip finds a version that satisfies both requirements. If no such version exists, pip reports a dependency conflict error.

Step 4: Download wheel files. pip downloads a compatible wheel for each package. Pure-Python packages (Flask, Jinja2, Werkzeug) have a single universal wheel (`py3-none-any`); packages with compiled code have one wheel per platform, and pip picks the one that matches

your Python version, operating system, and CPU architecture.

Step 5: Install. `pip` extracts each wheel into the virtual environment’s `site-packages`.

Direct vs transitive dependencies

Your `requirements.txt` lists your **direct dependencies** — packages you explicitly import.

But when `pip` installs Flask, it also installs Flask’s dependencies: Werkzeug, Jinja2, itsdangerous, click, blinker, and MarkupSafe. These are **transitive dependencies** — packages your code uses indirectly, through Flask.

Transitive dependencies are not listed in your `requirements.txt` (usually — see “complete lockfiles” below). They are discovered and installed automatically during the resolution process.

This creates a subtle problem: if you do not list transitive dependencies with their exact versions, they can change independently. Werkzeug 3.0.1 might be installed today and Werkzeug 3.0.3 a month from now — even though your `requirements.txt` only pins Flask.

Generating a complete lockfile with `pip freeze`

`pip freeze` prints the exact versions of every installed package — both direct and transitive:

```
pip freeze
```

Output (after `pip install flask python-dotenv`):

```
blinker==1.7.0
click==8.1.7
Flask==3.0.2
itsdangerous==2.1.2
Jinja2==3.1.3
MarkupSafe==2.1.5
python-dotenv==1.0.1
Werkzeug==3.0.1
```

Saving this output to `requirements.txt` gives you a **complete lockfile** — every package at an exact version:

```
pip freeze > requirements.txt
```

This is the most reproducible form of `requirements.txt`. Every package — direct and transitive — is pinned. Anyone who runs `pip install -r requirements.txt` gets an identical

environment.

The two-file pattern

Many projects use two files:

requirements.txt: the complete lockfile from `pip freeze`, used for deployment. Every version is pinned.

requirements.in (or `requirements-base.txt`): only the direct dependencies, without pinning:

```
# requirements.in
flask
python-dotenv
```

Tools like **pip-tools** generate `requirements.txt` from `requirements.in`:

```
pip install pip-tools
pip-compile requirements.in # Generates requirements.txt with all transitive deps
↪ pinned
```

The advantage: you maintain the short list of direct dependencies (`requirements.in`), and `pip-tools` generates the complete reproducible lockfile (`requirements.txt`). Updating a package means changing `requirements.in` and regenerating `requirements.txt` with `pip-compile`.

12.4 Installing from requirements.txt

To reproduce the environment from scratch on a new machine:

```
# 1. Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate # macOS/Linux

# 2. Install exactly what is listed
pip install -r requirements.txt
```

`pip install -r requirements.txt` reads the file and installs every listed package at the specified version. It does nothing for packages already installed at the correct version — and it does not *remove* packages that are installed but no longer listed (tools such as `pip-sync` from `pip-tools` do that).

What `-r` means

`-r` means “requirements file”. It is the flag that tells pip to read a file rather than a single package name. You can combine multiple `-r` flags:

```
pip install -r requirements.txt -r requirements-dev.txt
```

Verifying the installation

After installing, verify the environment matches expectations:

```
pip list
```

Or check specific packages:

Listing 12.1 Verifying installed versions programmatically

```
import importlib.metadata

packages = ["flask", "werkzeug", "python-dotenv"]
for pkg in packages:
    try:
        version = importlib.metadata.version(pkg)
        print(f"{pkg}: {version}")
    except importlib.metadata.PackageNotFoundError:
        print(f"{pkg}: NOT INSTALLED")
```

Output:

```
flask: 3.0.2
werkzeug: 3.0.1
python-dotenv: 1.0.1
```

12.5 The Complete `requirements.txt` for the Notes API

At this stage of the book, the Notes API uses:

```
# Direct imports in app.py:
from flask import Flask, request, jsonify, g    # → flask
from dotenv import load_dotenv                  # → python-dotenv
import sqlite3                                  # standard library – no pip package
import os                                       # standard library
import time                                     # standard library
```

The direct dependencies are Flask and python-dotenv. After installing them in a fresh virtual environment and running `pip freeze`:

```
# requirements.txt (generated by pip freeze after pip install flask python-dotenv)
blinker==1.7.0
click==8.1.7
Flask==3.0.2
itsdangerous==2.1.2
Jinja2==3.1.3
MarkupSafe==2.1.5
python-dotenv==1.0.1
Werkzeug==3.0.1
```

Save this file at the root of the project, alongside `app.py`.

The complete project structure is now:

```
notes-api/
├── app.py
├── requirements.txt
├── .env                ← NOT committed
├── .env.example        ← committed (no real values)
├── .gitignore
└── venv/               ← NOT committed
```

`.gitignore` should contain:

```
# .gitignore
venv/
__pycache__/
*.pyc
*.db
.env
```

With `requirements.txt` committed and `venv/` excluded, a collaborator can:

```
git clone https://github.com/yourname/notes-api.git
cd notes-api
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Edit .env with local values
python app.py
```

This workflow is reproducible. No hidden global dependencies. No assumptions about what the collaborator has installed.

12.6 pyproject.toml: The Modern Alternative

Python has a modern package configuration standard defined in PEP 517, PEP 518, and PEP 621, centered on a file called `pyproject.toml`. `pyproject.toml` is now the recommended way to declare Python project metadata, including dependencies.

For a simple application (as opposed to a library meant to be distributed), `requirements.txt` remains the most common and straightforward choice. But understanding `pyproject.toml` is valuable because it is increasingly seen in modern Python projects.

```
# pyproject.toml
[project]
name = "notes-api"
version = "0.1.0"
description = "A simple notes API"
requires-python = ">=3.10"

dependencies = [
    "flask>=3.0.0",
    "python-dotenv>=1.0.0",
]

[project.optional-dependencies]
dev = [
    "pytest",
    "black",
    "ruff",
]
```



```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
only-include = ["app.py"]  # the project is still a single module (Chapter 15 makes
↪ it a package)
```

With `pyproject.toml`, you install the project with:

```
pip install -e .          # Installs the project and its dependencies
pip install -e ".[dev]"   # Includes development dependencies too
```

The `-e` flag installs in **editable mode** — the source code is used directly, so changes to `app.py` take effect without reinstalling.

When to use `pyproject.toml` vs `requirements.txt`

Use case	Recommendation
Simple application (not a library)	<code>requirements.txt</code> with <code>pip freeze</code>
Application with dev/test/prod dependency groups	<code>pyproject.toml</code> + a lockfile tool (<code>pip-tools</code> , <code>Poetry</code> , <code>uv</code>)
A library that others will install	<code>pyproject.toml</code> is required
Modern project following current Python standards	<code>pyproject.toml</code> + lockfile

For the Notes API, we continue with `requirements.txt` for clarity. The concepts (exact version pinning, separating direct from transitive dependencies) are identical regardless of the file format.

The Updated Development Workflow

With `requirements.txt` in place, the correct development workflow is:

```
# Setting up (first time on a machine)
python3 -m venv venv
```

```

source venv/bin/activate
pip install -r requirements.txt

# Adding a new package
pip install gunicorn
pip freeze > requirements.txt # Update the lockfile
git add requirements.txt
git commit -m "Add gunicorn to requirements"

# After pulling changes that updated requirements.txt
pip install -r requirements.txt # Update the environment to match

# Updating a package deliberately
pip install --upgrade flask
pip freeze > requirements.txt
# Run tests to verify nothing broke
git add requirements.txt
git commit -m "Update Flask to 3.0.3"

```

The key habit: any time `requirements.txt` changes (because you added, removed, or updated a package), regenerate it with `pip freeze` and commit the result. The file should always reflect the actual installed state of the project’s virtual environment.

This works only while the virtual environment contains *nothing but* the application’s runtime packages. As soon as you install development tools into it — a test runner, a linter (→ Chapter 17) — `pip freeze` would record them as production dependencies too. From that point on, keep development tools in a separate `requirements-dev.txt`, and maintain `requirements.txt` deliberately: edit it by hand, or generate it with a lock tool such as `pip-tools` (`pip-compile`) or `uv`.

Key Takeaways

- **requirements.txt** is a text file that lists Python package dependencies and their versions. It makes the dependency environment reproducible by anyone, on any machine.
- **Version pinning** (`flask==3.0.2`) ensures that `pip install -r requirements.txt` installs the same version every time, not whatever the latest version happens to be. Use `==` for production requirements.
- Python packages have their own dependencies (**transitive dependencies**). `pip freeze` captures both direct and transitive dependencies at their exact versions. This is the most reproducible form of `requirements.txt`.
- `pip install -r requirements.txt` installs every listed package at the specified version into the active virtual environment.

- The **two-file pattern** (`requirements.in` for direct deps + `requirements.txt` for the full lockfile, generated by `pip-tools`) is a clean way to separate “what I directly need” from “the full reproducible environment”.
- `pyproject.toml` is the modern standard for project metadata and dependency declaration, especially for libraries. `requirements.txt` remains common for applications.
- Add `requirements.txt` to version control. Never add `venv/`, `__pycache__`, `.env`, or `.db` files.

Exercises

1. **Predict.** Compare `pip freeze` in your project environment with `requirements.txt`. Which packages appear in one and not the other, and why?
2. **Break and fix.** Replace `flask==3.0.2` with `flask` (unpinned), build a fresh environment, and note the version you get. Explain what could break six months from now, then pin it again.
3. **Extend.** Set up `pip-tools`: move the direct dependencies into `requirements.in`, generate `requirements.txt` with `pip-compile`, and read the comments it writes.
4. **Explain.** What is a transitive dependency, and why must it be pinned too?

Solutions and hints are in the companion repository, in `exercises/chapter-12.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What’s Next

Dependencies are declared. The next step is handling the configuration values that the application reads at runtime — moving every environment-specific value out of the source code and into a properly structured, environment-controlled configuration system.

→ [Chapter 13](#) — Configuration Externalization

Configuration Externalization

In [Chapter 9](#), we identified four dangerous forms of configuration coupling: hardcoded secrets, local database references, debug configurations, and environment-specific logic in code. In [Chapter 6](#), we introduced environment variables as the mechanism for injecting configuration. This chapter builds the complete, practical system for managing configuration correctly across all environments — not just noting that environment variables should be used, but showing exactly how to structure, validate, load, and use them in a real Flask application.

Configuration externalization means that the application’s behavior is determined by its environment, not by its source code. The same `app.py` runs in development, staging, and production. What changes is the environment’s variables. The database URL, the debug flag, the API keys, the port number — all of these come from the environment, not from the code.

This chapter also introduces PostgreSQL as the database for the Notes API from this point forward. SQLite is appropriate for local development, but a production system requires a proper database server. The code has to learn to talk to both databases once; after that, which database the application uses is purely a configuration choice — switching means changing the database URL, not the code.

In this chapter

By the end of this chapter, you will understand:

- How to identify all configuration values that need to be externalized
- How to move each value into an environment variable
- How to load configuration at runtime using `os.getenv`
- How to validate that required configuration is present at startup
- How to manage different configurations for different environments (development, staging, production)
- How to prepare the Notes API to run on SQLite or PostgreSQL, so that switching between them is a configuration change
- Why a configured dependency is an **integration contract**, and the five dimensions every contract has

13.1 Identifying What Needs to Be Configurable

The first step is auditing the application for values that are environment-specific. Ask: “If I ran this application on a different machine, or in a different context, would this value need to change?”

Go through `app.py` systematically:

Listing 13.1 Audit of the Notes API for configuration values

```
from dotenv import load_dotenv
load_dotenv()

from flask import Flask, request, jsonify, g
import sqlite3
import os
import time

app = Flask(__name__)

# — Candidate 1: Database path —————
DATABASE_PATH = os.getenv("DATABASE_PATH", "notes.db")
# ✓ Already externalized. Good. But the default "notes.db" is still SQLite-specific.
# This needs to become a full database URL when we switch to PostgreSQL.

# — Candidate 2: Debug mode —————
# Currently: hardcoded in app.run(debug=True) at the bottom of the file
```

```

# Needs to be: controlled by environment variable FLASK_DEBUG

# — Candidate 3: Port —————
# Currently: hardcoded as 5000 in app.run(port=5000)
# Needs to be: PORT environment variable (default 5000)

# — Candidate 4: Host —————
# Currently: hardcoded as "localhost"
# Needs to be: HOST environment variable (localhost in dev, 0.0.0.0 in production)

# — Candidate 5: Secret key —————
# Currently: not set (Flask's default is None – anything that signs data,
# such as sessions, would fail; the API does not use sessions yet)
# Needs to be: SECRET_KEY environment variable (required – no default)

# — Candidate 6: API key for authentication —————
# Currently: only in Listing 5.10's example middleware
# Will need: API_KEY environment variable while authentication is one shared
# key (Chapter 38 later replaces it with users and login tokens in the database)

# — Not a configuration candidate: —————
# - Route definitions (/notes, /notes/<id>) – these are code, not config
# - SQL queries – these are code, not config
# - HTTP status codes – these are code, not config

```

The complete list of configuration values for the Notes API:

Variable	Required?	Default	Description
DATABASE_URL	Yes	—	Full database connection URL
SECRET_KEY	Yes	—	Flask’s secret key for session signing
FLASK_DEBUG	No	false	Enable debug mode (dev only)
PORT	No	5000	Port to bind to
HOST	No	127.0.0.1	IP address to bind to
API_KEY	Yes, while authentication is one shared key	—	Authentication key

From “where” to “what”: configuration points at other systems

Look at two entries in that table. DATABASE_URL points at another system (a database). API_KEY is proof of identity *to* another system. Externalizing them answers *where* a dependency is and *how you authenticate* to it — but it quietly raises a much harder, longer-lived question: **what is the agreement between you and that system?**

That agreement is a **contract**, and an integration contract is harder to change than almost any code you will write, because changing it breaks *everyone on the other side* — and you cannot deploy the other side. This cuts both ways. The Notes API is a *consumer* of a contract (the database’s) and a *provider* of one (the REST interface it offers its own clients, GET /notes, POST /notes, and the rest). The same dimensions govern both directions. This chapter only names them, here at the boundary where configuration meets the systems around it; [Chapter 34](#) (section 34.10) looks at each in depth, once the API’s own requests and responses have been built.

The five dimensions of every integration

Any integration, in either direction, pins down five things. If you can answer these for a dependency, you understand it; if you cannot, you have a production surprise waiting:

1. **Shape** — the protocol and data format: REST/JSON, GraphQL, gRPC, or asynchronous events. How requests and responses are structured.
2. **Versioning** — how the contract is allowed to change over time without breaking existing callers.
3. **Identity** — who is calling: API keys, OAuth tokens, mTLS, service accounts. (API_KEY in

the table above is this dimension — a credential that proves identity to a contract.)

4. **Authorization** — what that identity is permitted to do once authenticated.
5. **Failure semantics** — what happens when it goes wrong: error formats, timeouts, retries, and crucially what a failure *means* (did my write happen, or not?).

For the configuration in this chapter, the lesson is simple: every value that points at another system — a URL, a credential — is one end of such a contract, and changing it is never *only* a configuration change.

13.2 Moving Values into Environment Variables

With the configuration inventory complete, the next step is removing all hardcoded values and replacing them with `os.getenv()` calls.

The pattern for required configuration

For values that must be present — the application cannot function without them — use `os.environ[key]`. This raises `KeyError` if the variable is not set, causing an immediate startup failure with a clear error:

Listing 13.2 Reading required configuration

```
import os

# Raises KeyError if DATABASE_URL is not set
DATABASE_URL = os.environ["DATABASE_URL"]

# Raises KeyError if SECRET_KEY is not set
SECRET_KEY = os.environ["SECRET_KEY"]
```

`os.environ["KEY"]` versus `os.getenv("KEY")`:

- `os.environ["KEY"]` raises `KeyError` if the key is missing — good for required config
- `os.getenv("KEY")` returns `None` if the key is missing — good for optional config with explicit `None` handling
- `os.getenv("KEY", "default")` returns the default if the key is missing — good for optional config with a default value

The pattern for optional configuration

For values that have a sensible default, use `os.getenv(key, default)`:

Listing 13.3 Reading optional configuration with defaults

```
DEBUG = os.getenv("FLASK_DEBUG", "false").lower() == "true"
PORT = int(os.getenv("PORT", "5000"))
HOST = os.getenv("HOST", "127.0.0.1")
```

Note the `.lower() == "true"` pattern for boolean environment variables. Environment variables are always strings. "true", "True", "TRUE", "1", "yes" might all mean “true” depending on convention. By normalizing to lowercase and comparing to "true", you accept "true" and "True" but not "1". Choose a convention and document it.

13.3 Runtime Config Loading with `os.getenv`

The configuration should be loaded at **module import time** — the top of `app.py`, before any routes or database code. This way, the application fails immediately at startup if configuration is missing, rather than failing on the first request that uses the missing value.

Listing 13.4 Complete configuration loading at the top of `app.py`

```
from dotenv import load_dotenv
load_dotenv() # Load .env file BEFORE reading os.environ

import os
import sys

# — Required configuration (fail loudly if missing) —————

def _require_env(key: str) -> str:
    """Read a required environment variable. Exit cleanly if missing."""
    value = os.getenv(key)
    if not value:
        print(f"Error: Required environment variable '{key}' is not set.",
              ↪ file=sys.stderr)
        print("Set it in your .env file or environment.", file=sys.stderr)
        sys.exit(1)
    return value

DATABASE_URL = _require_env("DATABASE_URL")
SECRET_KEY   = _require_env("SECRET_KEY")

# — Optional configuration with defaults —————
```

```
DEBUG = os.getenv("FLASK_DEBUG", "false").lower() == "true"
PORT = int(os.getenv("PORT", "5000"))
HOST = os.getenv("HOST", "127.0.0.1")
```

`_require_env()` is a small helper that reads a variable and calls `sys.exit(1)` if it is missing, printing a clear error message. This is more user-friendly than a raw `KeyError` from `os.environ["KEY"]`:

```
# Bad (confusing):
KeyError: 'DATABASE_URL'

# Good (actionable):
Error: Required environment variable 'DATABASE_URL' is not set.
Set it in your .env file or environment.
```

13.4 Validation of Required Configuration on Startup

Beyond simple presence checks, configuration values may need to be validated for correctness before the application begins accepting requests. A missing variable is one kind of failure; a malformed value is another.

Listing 13.5 Comprehensive startup configuration validation

```
import os
import sys
import re

def validate_config() -> dict:
    """
    Load and validate all configuration. Returns the config dict on success.
    Exits with a clear error message on failure.
    """
    errors = []
    config = {}

    # — DATABASE_URL —————
    db_url = os.getenv("DATABASE_URL")
    if not db_url:
        errors.append("DATABASE_URL is required")
    elif not (db_url.startswith("postgresql://") or
              db_url.startswith("sqlite:///")):
```

```

        errors.append("DATABASE_URL must start with postgresql:// or sqlite:///")
    else:
        config["DATABASE_URL"] = db_url

    # — SECRET_KEY —————
    secret_key = os.getenv("SECRET_KEY")
    if not secret_key:
        errors.append("SECRET_KEY is required")
    elif len(secret_key) < 24:
        errors.append("SECRET_KEY must be at least 24 characters long")
    else:
        config["SECRET_KEY"] = secret_key

    # — PORT —————
    port_str = os.getenv("PORT", "5000")
    try:
        port = int(port_str)
        # Ports below 1024 need root privileges (Chapter 3); the app runs as a normal
        ↪ user
        if not (1024 <= port <= 65535):
            errors.append(f"PORT must be between 1024 and 65535, got: {port}")
        else:
            config["PORT"] = port
    except ValueError:
        errors.append(f"PORT must be an integer, got: '{port_str}'")

    # — FLASK_DEBUG —————
    debug_str = os.getenv("FLASK_DEBUG", "false").lower()
    if debug_str not in ("true", "false"):
        errors.append(f"FLASK_DEBUG must be 'true' or 'false', got: '{debug_str}'")
    else:
        config["DEBUG"] = debug_str == "true"

    # — Report errors —————
    if errors:
        print("Configuration errors:", file=sys.stderr)
        for err in errors:
            print(f"  x {err}", file=sys.stderr)
        print("\nCheck your .env file or environment variables.", file=sys.stderr)
        sys.exit(1)

    return config

# Load and validate at module import time

```

```
CONFIG = validate_config()
```

With this validation, the application starts or fails immediately with a complete list of all configuration problems. Not just the first one — all of them. This saves the debugging cycle of fixing one problem, restarting, finding the next, and so on.

13.5 Switching from SQLite to PostgreSQL Through Configuration

This is where configuration externalization becomes tangible: by changing one environment variable (`DATABASE_URL`), the Notes API will switch from SQLite (local development) to PostgreSQL (production).

That takes one piece of preparation first. The two databases are not interchangeable: they use different Python drivers (`sqlite3` and `psycopg2`), different parameter placeholders (`?` and `%s`), and slightly different SQL for creating tables. So the code must learn to speak both — once, in a few small, well-contained helpers. After that, choosing a database is purely a configuration decision. Start with opening a connection, selecting the driver based on the URL:

Listing 13.6 Database abstraction layer that works with both SQLite and PostgreSQL

```
import os
from flask import g

def get_db_connection(database_url: str):
    """Open a database connection appropriate for the configured URL."""
    if database_url.startswith("sqlite:///"):
        import sqlite3
        db_path = database_url[len("sqlite:///"):]
        conn = sqlite3.connect(db_path)
        conn.row_factory = sqlite3.Row
        return conn

    elif database_url.startswith("postgresql://"):
        import psycopg2
        import psycopg2.extras
        conn = psycopg2.connect(database_url)
        conn.autocommit = False
        return conn

    else:
        raise ValueError(f"Unsupported database URL scheme: {database_url}")
```

Installing the PostgreSQL driver

Add `psycopg2-binary` to `requirements.txt`:

```
# requirements.txt (updated)
blinker==1.7.0
click==8.1.7
Flask==3.0.2
itsdangerous==2.1.2
Jinja2==3.1.3
MarkupSafe==2.1.5
psycopg2-binary==2.9.9
python-dotenv==1.0.1
Werkzeug==3.0.1
```

```
pip install psycopg2-binary
pip freeze > requirements.txt
```

`psycopg2-binary` is a prebuilt wheel with the PostgreSQL client library (`libpq`) bundled inside, so it needs no compiler and no system packages. The `psycopg2` maintainers suggest building the plain `psycopg2` package from source for demanding production use; this book uses `psycopg2-binary` throughout because it keeps the build simple, and it is widely used in production for applications of this size. (Its successor, `psycopg` version 3, offers the same choice as `psycopg[binary]`.)

Running PostgreSQL locally for development

For development, install PostgreSQL locally or use Docker to run it:

```
# Using Docker (simplest approach – no local PostgreSQL installation needed)
docker run --name notes-postgres \
  -e POSTGRES_USER=notesuser \
  -e POSTGRES_PASSWORD=notespass \
  -e POSTGRES_DB=notesdb \
  -p 5432:5432 \
  -d postgres:16-alpine
```

This starts a PostgreSQL server accessible at `localhost:5432` with:

- Username: `notesuser`
- Password: `notespass`

- Database: notesdb

Update the development `.env`:

```
# .env (development with PostgreSQL)
DATABASE_URL=postgresql://notesuser:notespass@localhost:5432/notesdb
SECRET_KEY=dev-secret-key-at-least-24-chars
FLASK_DEBUG=true
PORT=5000
HOST=127.0.0.1
```

With this `.env`, the application uses PostgreSQL locally. The same code with a different `.env` (or no `.env` and environment variables set by the server) uses a production PostgreSQL instance.

PostgreSQL-compatible schema initialization

The `init_db()` function needs a small adjustment for PostgreSQL — it uses `SERIAL` instead of SQLite's `INTEGER PRIMARY KEY AUTOINCREMENT`, and it has a real timezone-aware timestamp type, `TIMESTAMPZ` (← [Chapter 6](#): store UTC, with a type that knows it):

Listing 13.7 Database-agnostic schema initialization

```
def init_db(database_url: str):
    """Create the notes table if it does not exist."""
    conn = get_db_connection(database_url)
    cursor = conn.cursor()

    if database_url.startswith("sqlite:///"):
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS notes (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                content TEXT NOT NULL,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)
    else:
        # PostgreSQL syntax
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS notes (
                id SERIAL PRIMARY KEY,
                content TEXT NOT NULL,
                created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
            )
        """)
```

```

        )
    """
)

conn.commit()
conn.close()

```

13.6 The Full Updated app.py with Complete Configuration Externalization

Here is the complete Notes API with all configuration externalized. Two small helpers keep the SQLite/PostgreSQL differences in one place instead of scattering them through every handler: `sql()` lets the queries be written once with `?` placeholders, and `to_iso()` makes timestamps look the same whichever database produced them.

Listing 13.8 app.py with complete configuration externalization

```

"""
Notes API – a simple Flask API for creating and retrieving notes.
All configuration comes from environment variables (or .env file).
"""

from dotenv import load_dotenv
load_dotenv()

from flask import Flask, request, jsonify, g
import os
import sys
import time

# — Configuration —————

def validate_config() -> dict:
    errors = []
    config = {}

    db_url = os.getenv("DATABASE_URL")
    if not db_url:
        errors.append("DATABASE_URL is required")
    else:
        config["DATABASE_URL"] = db_url

    secret_key = os.getenv("SECRET_KEY")

```



```

if not secret_key or len(secret_key) < 24:
    errors.append("SECRET_KEY is required and must be at least 24 characters")
else:
    config["SECRET_KEY"] = secret_key

config["DEBUG"] = os.getenv("FLASK_DEBUG", "false").lower() == "true"
config["PORT"] = int(os.getenv("PORT", "5000"))
config["HOST"] = os.getenv("HOST", "127.0.0.1")

if errors:
    for err in errors:
        print(f"Config error: {err}", file=sys.stderr)
    sys.exit(1)

return config

CONFIG = validate_config()

# — Application —————

app = Flask(__name__)
app.config["SECRET_KEY"] = CONFIG["SECRET_KEY"]
app.config["DEBUG"] = CONFIG["DEBUG"]

# — Database —————

def open_db():
    """Open a database connection for the current request."""
    db_url = CONFIG["DATABASE_URL"]
    if db_url.startswith("sqlite:///"):
        import sqlite3
        conn = sqlite3.connect(db_url[len("sqlite:///"):])
        conn.row_factory = sqlite3.Row
        return conn
    else:
        import psycopg2
        import psycopg2.extras
        conn = psycopg2.connect(db_url)
        # Return rows as dicts
        conn.cursor_factory = psycopg2.extras.RealDictCursor
        return conn

def get_db():
    if "db" not in g:

```

```

        g.db = open_db()
    return g.db

IS_SQLITE = CONFIG["DATABASE_URL"].startswith("sqlite:///")

def sql(query: str) -> str:
    """Queries are written with ? placeholders; psycopg2 expects %s."""
    return query if IS_SQLITE else query.replace("?", "%s")

def to_iso(value) -> str:
    """A timestamp as ISO 8601 in UTC, whichever database produced it."""
    if isinstance(value, str):
        # SQLite: "2024-01-15 14:00:05"
        ↪ (UTC)
        return value.replace(" ", "T") + "+00:00"
    return value.isoformat()
    # PostgreSQL: a timezone-aware
    ↪ datetime

def note_to_dict(row) -> dict:
    note = dict(row)
    note["created_at"] = to_iso(note["created_at"])
    return note

def init_db():
    conn = open_db()
    cur = conn.cursor()
    if IS_SQLITE:
        cur.execute("""
            CREATE TABLE IF NOT EXISTS notes (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                content TEXT NOT NULL,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)
    else:
        cur.execute("""
            CREATE TABLE IF NOT EXISTS notes (
                id SERIAL PRIMARY KEY,
                content TEXT NOT NULL,
                created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
            )
        """)
    conn.commit()
    conn.close()

```

```

init_db() # At startup – also when a production server imports this module

# — Middleware —————

@app.before_request
def start_timer():
    g.start_time = time.time()

@app.before_request
def require_json_for_writes():
    if request.method in ("POST", "PUT", "PATCH") and not request.is_json:
        return jsonify({"error": "Content-Type must be application/json"}), 415

@app.after_request
def add_timing_header(response):
    duration_ms = (time.time() - g.start_time) * 1000
    response.headers["X-Response-Time-Ms"] = f"{duration_ms:.1f}"
    return response

@app.teardown_request
def close_db(exception):
    db = g.pop("db", None)
    if db is not None:
        db.close()

# — Error handlers —————

@app.errorhandler(404)
def not_found(error):
    return jsonify({"error": "Not found"}), 404

@app.errorhandler(405)
def method_not_allowed(error):
    return jsonify({"error": "Method not allowed"}), 405

@app.errorhandler(500)
def internal_error(error):
    return jsonify({"error": "Internal server error"}), 500

# — Routes —————

@app.route("/health", methods=["GET"])
def health_check():
    return jsonify({"status": "ok", "debug": CONFIG["DEBUG"]})

```

```

@app.route("/notes", methods=["GET"])
def list_notes():
    cursor = get_db().cursor()
    cursor.execute("SELECT id, content, created_at FROM notes ORDER BY id DESC")
    notes = [note_to_dict(row) for row in cursor.fetchall()]
    return jsonify({"notes": notes, "count": len(notes)})

@app.route("/notes", methods=["POST"])
def create_note():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return jsonify({"error": "Request body must be a JSON object"}), 400
    content = data.get("content")
    if not isinstance(content, str) or not content.strip():
        return jsonify({"error": "content is required and cannot be empty"}), 400
    content = content.strip()

    db = get_db()
    cursor = db.cursor()
    if IS_SQLITE:
        cursor.execute("INSERT INTO notes (content) VALUES (?)", (content,))
        note_id = cursor.lastrowid
    else:
        # PostgreSQL has no lastrowid; RETURNING hands back the new id
        cursor.execute("INSERT INTO notes (content) VALUES (%s) RETURNING id",
        ↪ (content,))
        note_id = cursor.fetchone()["id"]
    db.commit()
    return jsonify({"id": note_id, "content": content}), 201

@app.route("/notes/<int:note_id>", methods=["GET"])
def get_note(note_id):
    cursor = get_db().cursor()
    cursor.execute(sql("SELECT id, content, created_at FROM notes WHERE id = ?"),
    ↪ (note_id,))
    row = cursor.fetchone()
    if row is None:
        return jsonify({"error": f"Note {note_id} not found"}), 404
    return jsonify(note_to_dict(row))

@app.route("/notes/<int:note_id>", methods=["DELETE"])
def delete_note(note_id):
    db = get_db()

```

```

    cursor = db.cursor()
    cursor.execute(sql("DELETE FROM notes WHERE id = ?"), (note_id,))
    db.commit()
    if cursor.rowcount == 0:
        return jsonify({"error": f"Note {note_id} not found"}), 404
    return jsonify({"message": f"Note {note_id} deleted"})

# — Entry point —————

if __name__ == "__main__":
    app.run(
        host=CONFIG["HOST"],
        port=CONFIG["PORT"],
        debug=CONFIG["DEBUG"]
    )

```

The .env.example file

Commit this file to document required configuration without exposing real values:

```

# .env.example – copy to .env and fill in values
# Do not commit .env to version control

# Required: database connection URL
# For local development with SQLite:
# DATABASE_URL=sqlite:///notes.db
# For local development with Docker PostgreSQL:
# DATABASE_URL=postgresql://notesuser:notespass@localhost:5432/notesdb
DATABASE_URL=

# Required: secret key for Flask session signing (at least 24 characters)
# Generate with: python -c "import secrets; print(secrets.token_hex(32))"
SECRET_KEY=

# Optional: enable debug mode (true/false, default: false)
FLASK_DEBUG=false

# Optional: server port (default: 5000)
PORT=5000

# Optional: server host (127.0.0.1 for local, 0.0.0.0 for all interfaces)
HOST=127.0.0.1

```

Generating a secure secret key

Listing 13.9 Generating a cryptographically secure secret key

```
import secrets

# Run this once to generate a key, copy into .env
print(secrets.token_hex(32))
# Output: a64f3b8c2e7d1a5f9b0c4d6e3f8a2b5c7d1e4f0a3b6c9d2e5f8a1b4c7d0e3f6
```

Managing Configurations Across Environments

Here is how the same application is configured in three environments with different `.env` values (or environment variables set at the system level):

DEVELOPMENT (.env)	STAGING (env vars)	PRODUCTION (env vars)
DATABASE_URL=sqlite:///notes.db SECRET_KEY=dev-only-key-0123456789 FLASK_DEBUG=true	DATABASE_URL=postgresql://... SECRET_KEY=<generated-key> FLASK_DEBUG=false	DATABASE_URL=postgresql://... SECRET_KEY=<generated-key> FLASK_DEBUG=false

The source code (`app.py`) is identical in all three environments. No if-else on environment name. No hardcoded values. The behavior differs because the environment differs. (HOST and PORT only matter when the app runs its built-in development server. In staging and production it runs under Gunicorn inside a container, whose listening address is set where the container is built and run — → [Chapter 18](#) and → [Chapter 23](#).)

Key Takeaways

- **Configuration externalization** means removing all environment-specific values from source code and reading them from environment variables at runtime.
- Use `os.getenv("KEY")` for optional config (returns `None` if missing), `os.getenv("KEY", "default")` for config with defaults, and `os.environ["KEY"]` or a validation function for required config.
- **Validate all configuration at startup** with a clear error message for every problem. The application should either start correctly or fail immediately with a complete list of what is wrong.
- `load_dotenv()` from `python-dotenv` loads a `.env` file into `os.environ`. It must be called before any `os.getenv()` calls. The `.env` file is not committed to version control; `.env.example` is.
- **Switching databases through configuration** (SQLite for development, PostgreSQL for production) works once the code can speak both — a one-time investment kept in a few small helpers. After that, the same code runs everywhere and the environment decides.

- Commit `.env.example` to document required variables. Never commit `.env`. Treat any accidentally committed secret as fully compromised and immediately rotate it.
- A configured dependency is a **contract**, and contracts are harder to change than code because you break everyone on the other side. Every integration — one you call or one you offer — has five dimensions: **shape**, **versioning**, **identity**, **authorization**, and **failure semantics**.

Exercises

1. **Predict.** Start the application with `DATABASE_URL` unset, then with it set to an unreachable PostgreSQL host. What does each attempt print, and when?
2. **Break and fix.** Set `SECRET_KEY` to `changeme`. Does the application start? Which check should stop it, and in which chapter does that check appear?
3. **Extend.** Add a `REQUEST_TIMEOUT_SECONDS` setting: optional, an integer between 1 and 120, default 30, validated at start-up with a clear message for each kind of bad value.
4. **Explain.** Which of the five integration-contract dimensions does `DATABASE_URL` touch, and which does `SECRET_KEY` touch?

Solutions and hints are in the companion repository, in `exercises/chapter-13.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

Dependencies are declared. Configuration is externalized. The next chapter defines the application's runtime contract — a formal statement of what the application requires from its execution environment before it can start.

→ [Chapter 14](#) — Runtime Contract Definition

Runtime Contract Definition

Every piece of software makes demands on its execution environment. It needs a particular version of the language runtime. It needs certain operating system capabilities. It needs certain ports to be available. It needs certain environment variables to be set.

When these demands are undocumented — implicit — they become sources of failure. The application works in the developer’s environment because the developer’s environment happens to satisfy them all. It fails in a new environment because no one knew what those demands were, and no one could verify that the new environment satisfied them.

A **runtime contract** is the explicit, documented statement of what an application requires from its execution environment before it can start and operate correctly. It is not part of the application’s code — it is documentation *about* the code, written for the people (or systems) that will be responsible for creating and managing the execution environment.

Writing a runtime contract is a discipline that forces clarity. It asks: “What exactly does this application need?” The answer, written down, becomes a specification that a server setup (→ [Chapter 21](#)), a Dockerfile (→ [Chapter 18](#)), a deployment checklist (→ [Chapter 23](#)), and a monitoring system (→ [Chapter 35](#)) can all be measured against.

This chapter defines the runtime contract for the Notes API and explains each category in detail.

In this chapter

By the end of this chapter, you will understand:

- What a runtime contract is and why it matters
- How to declare the required Python version and enforce it at startup
- What OS-level capabilities an application might require
- How to document and validate required ports
- How to formally document required environment variables

14.1 Required Python Version

The Notes API is developed and tested on Python 3.11 and supports Python 3.10 and later. Declaring a minimum version is a promise about what you test against — later chapters use the `X | None` type syntax, which needs Python 3.10.

Declaring this is the first element of the runtime contract.

Declaring the version requirement in `pyproject.toml`

If the project uses `pyproject.toml`:

```
# pyproject.toml
[project]
name = "notes-api"
requires-python = ">=3.10"
```

When someone tries to install the project with an older Python:

```
# On Python 3.9:
pip install -e .
# ERROR: Package 'notes-api' requires a different Python:
# 3.9.x not in '>=3.10'
```

`pip` refuses to install with an incompatible Python version.

Declaring the version requirement without `pyproject.toml`

For projects using only `requirements.txt`, add a `.python-version` file — a simple text file containing the target Python version. Tools like `pyenv` and `asdf` read this file and activate the correct Python version automatically:

```
# .python-version
3.11
```

Some CI/CD tools can read this file too (GitHub Actions' `setup-python`, for example, when you point its `python-version-file` option at it).

Enforcing the version at runtime

Add a version check at the top of `app.py`, before anything else:

Listing 14.1 Enforcing the minimum Python version at startup

```

import sys

REQUIRED_PYTHON = (3, 10)

if sys.version_info < REQUIRED_PYTHON:
    print(
        f"Error: This application requires Python "
        f"{REQUIRED_PYTHON[0]}.{REQUIRED_PYTHON[1]} or later.\n"
        f"You are running Python "
        f"{sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}."
        ↪  \n"
        f>Please install the correct Python version and try again.",
        file=sys.stderr,
    )
    sys.exit(1)

```

This check runs before anything else. If the Python version is wrong, the application exits immediately with a clear, actionable error message instead of an obscure failure later.

There is one limit. Python compiles a whole file before running any of it, so if the *same file* uses syntax an old interpreter cannot parse (a match statement, for example), the old interpreter reports a `SyntaxError` before the check ever runs. The check protects you only if it lives in a small entry-point module written in old-compatible syntax, which runs before the rest of the application is imported.

14.2 Required Operating System Features

The Notes API requires several OS-level features. Some are always present on standard Linux installations; others might be absent in minimal or restricted environments.

Network socket support

The application binds to a TCP socket (← [Chapter 3](#)). Socket support is available on all general-purpose operating systems, but some hardened environments or containers with security profiles (like certain seccomp configurations) restrict socket system calls.

This is typically not something to check at startup, but it is worth noting in the runtime contract.

File system write access

The application needs to write to:

- The directory specified in DATABASE_URL (for SQLite) or has access to a PostgreSQL server
- A log directory (after production logging is configured in → Chapter 16)

Verify at startup:

Listing 14.2 Checking filesystem write access at startup

```
import os
import sys

def check_file_access():
    """Verify required filesystem access."""
    db_url = os.getenv("DATABASE_URL", "")

    if db_url.startswith("sqlite:///"):
        db_path = db_url[len("sqlite:///"):]
        db_dir = os.path.dirname(os.path.abspath(db_path)) or "."
        if not os.access(db_dir, os.W_OK):
            print(
                f"Error: Cannot write to database directory: {db_dir}",
                file=sys.stderr
            )
            sys.exit(1)

    log_dir = os.getenv("LOG_DIR", "")
    if log_dir and not os.access(log_dir, os.W_OK):
        print(f"Error: Cannot write to log directory: {log_dir}", file=sys.stderr)
        sys.exit(1)
```

Network connectivity to external services

The application connects to a PostgreSQL database server. That server must be reachable from the application's network. For production deployments on cloud platforms, this means the database and application must be in the same Virtual Private Cloud (VPC), or the network rules must permit the connection.

You can verify database connectivity at startup:

Listing 14.3 Verifying database connectivity at startup

```

import os
import sys

def check_database_connectivity(database_url: str):
    """Verify the database is reachable before accepting requests."""
    if database_url.startswith("sqlite:///"):
        # SQLite is a local file – no network connectivity needed
        return

    try:
        import psycopg2
        conn = psycopg2.connect(database_url, connect_timeout=5)
        conn.close()
        print("Database connectivity: OK", flush=True)
    except Exception as e:
        print(
            f"Error: Cannot connect to database.\n"
            f"DATABASE_URL: {mask_password(database_url)}\n"
            f"Error: {e}\n"
            f"Ensure the database server is running and accessible.",
            file=sys.stderr,
        )
        sys.exit(1)

def mask_password(url: str) -> str:
    """Hide the password in a database URL before printing it."""
    from urllib.parse import urlsplit
    parts = urlsplit(url)
    if parts.password:
        return url.replace(f":{parts.password}@", ":*:*:*@", 1)
    return url

```

Notice `mask_password()`: the URL contains the database password, and an error message ends up in terminal scrollback and log files — exactly where secrets must never go (← [Chapter 9](#)).

Failing fast on database connectivity prevents the application from starting and serving traffic when the database is unreachable — which would cause every request to fail with an opaque error. Better to not start at all with a clear reason than to start and fail on every request.

14.3 Required Ports

The application needs one port available: the port it will bind to (default 5000, or whatever PORT is set to).

There is a secondary concern: if a firewall, container, or security group blocks the port, the application can start but be unreachable. The application itself cannot detect this at startup — it can only verify the port is not already in use.

Checking port availability at startup

Listing 14.4 Checking that the required port is available

```
import errno
import socket
import os
import sys

def check_port_available(host: str, port: int):
    """Check that the port is not already in use."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        try:
            s.bind((host, port))
            # Bind succeeded — port is available
        except OSError as e:
            if e.errno == errno.EADDRINUSE: # 98 on Linux, 48 on macOS
                print(
                    f"Error: Port {port} on {host} is already in use.\n"
                    f"Either another process is using it, or a previous server "
                    f"instance did not shut down cleanly.\n"
                    f"Use: lsof -i :{port} to find the process.",
                    file=sys.stderr,
                )
                sys.exit(1)
            raise
```

This check replaces the bare `OSError: [Errno 98] Address already in use` (errno 48 on macOS) from the socket layer with an explanation of what to do. Two cautions: the port could still be taken in the instant between this check and the real `bind()`, and this check only makes sense when *your own code* is about to bind the port — for the development server started with `python app.py`. A production server such as Gunicorn binds the port itself before it imports your application, so running this check there would find the port “in use” by the very server

that is starting.

Documenting the port requirement

The runtime contract document should state:

Required port: The application requires that the TCP port specified by `PORT` (default: 5000) be available on the host specified by `HOST`. If clients connect from other machines, firewall rules and security groups must allow inbound TCP traffic on this port from them. If the only client is a reverse proxy on the same machine — the setup this book builds — the application should listen on `127.0.0.1` and the port should stay *closed* to the outside.

14.4 Required Environment Variables

All required environment variables have already been established in [Chapter 13](#). The runtime contract consolidates them into a single reference.

The formal environment variable specification

Listing 14.5 Formal environment variable specification as code

```
from dataclasses import dataclass
from typing import Callable
import os
import sys

@dataclass
class EnvVar:
    """Specification for a single environment variable."""
    name: str
    required: bool
    description: str
    example: str
    default: str | None = None
    validator: Callable[[str], bool] | None = None

ENVIRONMENT_SPEC = [
    EnvVar(
        name="DATABASE_URL",
        required=True,
        description="Database connection URL. SQLite or PostgreSQL.",
        example="postgresql://user:pass@localhost:5432/notesdb",
```

```

        validator=lambda v: v.startswith(("sqlite://", "postgresql://")),
    ),
    EnvVar(
        name="SECRET_KEY",
        required=True,
        description="Secret key for Flask session signing. Must be random and at
↪ least 24 characters.",
        example="<generate with: python -c \"import secrets;
↪ print(secrets.token_hex(32))\">",
        validator=lambda v: len(v) >= 24,
    ),
    EnvVar(
        name="FLASK_DEBUG",
        required=False,
        default="false",
        description="Enable Flask debug mode. Must be 'false' in production.",
        example="false",
        validator=lambda v: v.lower() in ("true", "false"),
    ),
    EnvVar(
        name="PORT",
        required=False,
        default="5000",
        description="Port number to bind to. Must be between 1024 and 65535.",
        example="5000",
        validator=lambda v: 1024 <= int(v) <= 65535,
    ),
    EnvVar(
        name="HOST",
        required=False,
        default="127.0.0.1",
        description="IP address to bind to. Use 127.0.0.1 for local, 0.0.0.0 for all
↪ interfaces.",
        example="0.0.0.0",
    ),
]

def load_and_validate_environment() -> dict:
    """
    Load and validate all environment variables per specification.
    Prints a formatted report and exits on failure.
    """
    config = {}
    errors = []

```



```

for spec in ENVIRONMENT_SPEC:
    raw = os.getenv(spec.name, spec.default)

    if spec.required and not raw:
        errors.append(f" x {spec.name}: MISSING (required)")
        errors.append(f"      {spec.description}")
        errors.append(f"      Example: {spec.name}={spec.example}")
        continue

    if raw and spec.validator:
        try:
            valid = spec.validator(raw)
        except (ValueError, TypeError):
            valid = False
        if not valid:
            errors.append(f" x {spec.name}: INVALID value '{raw}'")
            errors.append(f"      {spec.description}")
            continue

    config[spec.name] = raw

if errors:
    print("Environment validation failed:", file=sys.stderr)
    for line in errors:
        print(line, file=sys.stderr)
    sys.exit(1)

return config

```

This specification serves two purposes:

1. **Documentation:** the spec is the authoritative statement of what the application needs
2. **Validation:** the spec drives the startup checks that enforce the contract

The Complete Runtime Contract Document

The runtime contract is also valuable as human-readable documentation. For the Notes API, it looks like this:

NOTES API – RUNTIME CONTRACT

PYTHON VERSION

Required: Python 3.10 or later
 Recommended: Python 3.11 or later
 Check: `python3 --version`

OPERATING SYSTEM

Target: Linux (Ubuntu 22.04 LTS or later, Debian 12 or later)
 Also supported: macOS 13+ (development only)
 Not supported: Python on Windows without WSL

SYSTEM PACKAGES (required before pip install)

None – `psycpg2`-binary bundles the PostgreSQL client library.
 (Building plain `psycpg2` from source would need `libpq-dev`,
`build-essential`, and `python3-dev`.)

PYTHON PACKAGES

Install: `pip install -r requirements.txt`
 (All transitive dependencies are pinned in `requirements.txt`)

REQUIRED PORTS

PORT (default: 5000) must be available on HOST (default: 127.0.0.1)
 With a reverse proxy on the same machine, keep PORT closed to the outside

REQUIRED ENVIRONMENT VARIABLES

Variable	Required	Description
DATABASE_URL	Yes	PostgreSQL or SQLite connection URL
SECRET_KEY	Yes	≥24 char random string for session signing
FLASK_DEBUG	No	'true' or 'false' (default: 'false')
PORT	No	TCP port to bind (default: 5000)
HOST	No	IP to bind (default: 127.0.0.1)

CONNECTIVITY

DATABASE: The server must be able to reach the host in DATABASE_URL on
 PostgreSQL port 5432 (or whatever port the DB uses).
 OUTBOUND: No other outbound connections required by default.

FILESYSTEM

Write access required to:
 – Database directory (if using SQLite)
 – Log directory (if LOG_DIR is set)

STARTUP SEQUENCE

1. Python version check
2. Environment variable validation
3. Port availability check (development server only)
4. Database connectivity check
5. Application created; routes registered
6. Database schema initialization (CREATE TABLE IF NOT EXISTS)
7. Server starts accepting connections

ON FAILURE

All checks fail fast with a descriptive error message and exit code 1.
 No partial startup. No silent degradation.

This document lives in the repository (commonly as `RUNTIME.md`, or as a section of the project's `README.md`). It is the answer to: “What does this application need to run?”

The Consolidated Startup Sequence

Here is a `startup.py` module that runs all contract checks before handing control to Flask:

Listing 14.6 Consolidated startup checks (`startup.py`)

```

"""
Startup checks for the Notes API.
Import and call startup.run() before app.run() to enforce the runtime contract.
"""
import sys
import os
import socket
from urllib.parse import urlsplit

def check_python_version():
    if sys.version_info < (3, 10):
        _fail(
            f"Python 3.10+ required. Running {sys.version_info.major}."
            f"{sys.version_info.minor}."
        )

def check_env_vars():
    required = ["DATABASE_URL", "SECRET_KEY"]
    missing = [v for v in required if not os.getenv(v)]
    if missing:
        _fail(f"Missing required environment variables: {' '.join(missing)}")

    secret = os.getenv("SECRET_KEY", "")
    if len(secret) < 24:
        _fail("SECRET_KEY must be at least 24 characters long.")

def check_port():
    # With FLASK_DEBUG=true, Flask's reloader runs this file a second time in a child
    # process while the parent already holds the port – skip the check there.

```

```

if os.getenv("WERKZEUG_RUN_MAIN") == "true":
    return
host = os.getenv("HOST", "127.0.0.1")
port = int(os.getenv("PORT", "5000"))
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    try:
        s.bind((host, port))
    except OSError:
        _fail(f"Port {port} on {host} is already in use.")

def check_database():
    db_url = os.getenv("DATABASE_URL", "")
    if db_url.startswith("sqlite:///"):
        return # No network connectivity needed

    try:
        import psycopg2
        conn = psycopg2.connect(db_url, connect_timeout=5)
        conn.close()
    except Exception as e:
        _fail(f"Cannot connect to database: {e}\nURL: {_mask(db_url)}")

def _mask(url: str) -> str:
    """Never print a password: hide it before the URL reaches a log."""
    password = urlsplit(url).password
    return url.replace(f":{password}@", ":*:*:*@", 1) if password else url

def run():
    """Run all startup checks. Exits on any failure."""
    print("Running startup checks...", flush=True)
    check_python_version()
    check_env_vars()
    check_port()
    check_database()
    print("All startup checks passed.", flush=True)

def _fail(message: str):
    print(f"\nStartup failed: {message}", file=sys.stderr)
    sys.exit(1)

```

Use it in `app.py`, in the block that starts the development server:

```
# At the bottom of app.py
if __name__ == "__main__":
    import startup
    startup.run()          # checks run before the development server binds the port
    app.run(host=CONFIG["HOST"], port=CONFIG["PORT"], debug=CONFIG["DEBUG"])
```

Keeping the call inside the `__main__` block matters. When a production server imports `app.py` (→ Chapter 18), it has already bound the port, so the port check must not run — Chapter 15 splits these checks so that the ones that still apply run in production too.

Key Takeaways

- A **runtime contract** is the explicit, documented specification of what an application requires from its execution environment: Python version, OS features, ports, and environment variables.
- **Declare the Python version** in `pyproject.toml` (`requires-python`), in `.python-version`, and enforce it at runtime with a startup check.
- **OS requirements** include filesystem write permissions, network connectivity to databases and external services, and potentially system packages (such as `libpq-dev` if you build the PostgreSQL driver from source).
- **Required ports** must be available on the specified host. Verify at startup to prevent the cryptic `Address already in use` error.
- **Required environment variables** should be both documented (for humans) and validated at startup (for automated enforcement). Fail fast with a complete list of problems, not just the first one.
- The runtime contract is the bridge between the application and its infrastructure. Everything in Stages 5–7 (infrastructure, deployment, networking) is the work of satisfying this contract in a real environment.

Exercises

1. **Predict.** Run the application with a Python version older than the one in `requires-python`. What stops it, and where is that check?
2. **Break and fix.** Occupy port 5000 with another program (`python3 -m http.server 5000 --bind 127.0.0.1`) and start `python app.py`. Compare the message with the raw `Address already in use` error you saw in Chapter 3.
3. **Extend.** Write `.python-version` and add a CI step that fails if the Python in CI differs from it.
4. **Explain.** Explain why a runtime contract should be both documented for people and en-

forced by code.

Solutions and hints are in the companion repository, in `exercises/chapter-14.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

Dependencies are declared. Configuration is externalized. The runtime contract is defined. The next step is giving the application a proper structure — organizing its files, defining a clear entry point, and standardizing how it is started.

→ [Chapter 15 — Structuring the Application](#)

Structuring the Application

You have been writing your Notes API in a single file: `app.py`. Every route handler, every database function, every configuration loader, every startup check — all of it lives in one file. For learning, this is fine. For a production system, it creates a problem.

The problem has a precise name — **complexity** — and structure is how we fight it. A single file has no boundaries: no way to separate things that belong apart, no way to know where to look for something, no way to add functionality without touching everything else. When the file grows to 500 lines, finding the database connection function requires reading through middleware. When it grows to 1,000 lines, understanding any one part requires understanding the whole. That growing difficulty *is* complexity, and it is what eventually kills a codebase’s ability to change.

So this chapter does two things at once. On the surface, it restructures the Notes API from a single file into a proper directory layout — the layout used by most production Flask and FastAPI applications (Django prescribes its own, but the principles are the same). Underneath, it uses that restructuring to teach the **craft of writing code that stays simple**: how to recognize complexity, how to design modules that hide it, how to name and shape functions well, and how to refactor toward better structure in safe steps. Structure is the visible result; code craft is the discipline that produces it.

Application structure is the art of dividing a codebase into files and directories so that each part is findable, understandable, and independently modifiable. Good structure does not add features or make the application faster. What it does is make the application *survivable*: a new developer can find the handler for `GET /notes` without reading the configuration system; you, returning after three months, can remember where things are in ten seconds.

In this chapter

By the end of this chapter, you will understand:

- What **complexity** actually is, and the three symptoms — change amplification, cognitive load, and unknown unknowns — by which you recognize it
- Why a flat single-file structure becomes unmanageable, and which **code smells** signal it
- How to organize a Python web application into logical subdirectories with high **cohesion** and low **coupling**
- What a Python **package** is, why `__init__.py` matters, and how packages enforce **information hiding**
- What **deep modules** are, and why the **application factory pattern** is a good one
- How **naming, function design, and command/query separation** turn working code into readable code
- The difference between **strategic and tactical programming**, and why restructuring now is an investment
- How to define a clear entry point and standardize the startup command

15.1 The Problem with a Single File

Everything that has gone wrong with `app.py` over the last fourteen chapters has one name, and it is worth saying out loud before we fix it: **complexity**.

Complexity is not “lots of code,” and it is not “advanced code.” The most useful working definition comes from John Ousterhout: *complexity is anything about the structure of a system that makes it hard to understand or modify*. By that definition a 130-line file can already be complex, and a 100,000-line system can be simple. Complexity is about how hard the thing is to *change safely*, not how big it is. It is the single idea the rest of this chapter — and much of your career — is organized around.

Let us be concrete about what the current `app.py` contains after fourteen chapters of development:

```
# Everything in one file – an outline of app.py after Chapters 13 and 14
from dotenv import load_dotenv
load_dotenv()

from flask import Flask, request, jsonify, g
import os, sys, time

# Configuration loading and validation (Chapter 13)
```



```

def validate_config(): ...
CONFIG = validate_config()

# Flask app creation
app = Flask(__name__)
app.config["SECRET_KEY"] = CONFIG["SECRET_KEY"]

# Database functions and SQLite/PostgreSQL helpers
def open_db(): ...
def get_db(): ...
def sql(query): ...
def to_iso(value): ...
def note_to_dict(row): ...
def init_db(): ...

# Middleware
@app.before_request
def start_timer(): ...

@app.before_request
def require_json_for_writes(): ...

@app.after_request
def add_timing_header(response): ...

@app.teardown_request
def close_db(exception): ...

# Error handlers
@app.errorhandler(404)
def not_found(error): ...
# ... 405 and 500 handlers ...

# Route handlers
@app.route("/health")
def health_check(): ...

@app.route("/notes", methods=["GET"])
def list_notes(): ...

@app.route("/notes", methods=["POST"])
def create_note(): ...

@app.route("/notes/<int:note_id>", methods=["GET"])

```

```
def get_note(note_id): ...

@app.route("/notes/<int:note_id>", methods=["DELETE"])
def delete_note(note_id): ...

# Entry point, with the startup checks (Chapter 14)
if __name__ == "__main__":
    import startup
    startup.run()
    app.run(host=CONFIG["HOST"], port=CONFIG["PORT"], debug=CONFIG["DEBUG"])
```

This is around 130 lines of code right now. As the application grows — authentication, more endpoints, background tasks, scheduled jobs — each new feature adds more lines to this one file, and the categories of code bleed together: database code beside route code beside configuration code beside middleware. In the catalogue of **code smells** — recognizable patterns that signal deeper trouble — this one has a name: a **large module** (informally, a “god file”), a single unit that has quietly absorbed every responsibility in the system. It is the sibling of other smells you will learn to spot: long methods, large classes, **primitive obsession** (passing raw strings and dicts where a named type belongs), **feature envy** (a function more interested in another module’s data than its own), and **shotgun surgery** (one change forcing edits in many places at once).

The specific problems — three symptoms of complexity

Complexity is abstract, so engineers learn to recognize it by three concrete **symptoms**. Once you can name them, you can argue for fixing them. Each problem our single file has is one of these symptoms wearing local clothes.

Symptom 1 — Change amplification: a simple change touches many places. This is the absence of **separation of concerns**. A *concern* is a category of responsibility — configuration loading is one, database access another, HTTP routing another. When concerns are interleaved in one file, a change to one risks breaking another, and editing the database code means scrolling past every route handler to reach it. Shotgun surgery — one logical change scattered across many edits — is change amplification at its worst.

Symptom 2 — High cognitive load: you must hold too much in your head. A new developer cannot answer “where is the database code?” without reading the whole file — there is no **discoverability**. And testing one route in isolation drags in startup checks, environment variables, and a real database, because nothing is *hidden* behind a boundary you could substitute. The amount you must understand to make a safe edit is the truest measure of complexity, and here it is “everything, at once.”

Symptom 3 — Unknown unknowns: you cannot tell what a change will break. When File A imports File B and File B imports File A — a **circular import** — things work or fail depending on which file happens to be imported first; when they fail, Python raises an `ImportError` (“cannot import name ... most likely due to a circular import”). It is the system warning you that the dependencies have no clear direction. The deeper danger is the confident change that breaks something you did not know was connected. A clear structure with a one-way dependency hierarchy eliminates whole classes of these by design.

Why fix it now, and how

We could keep adding to `app.py`. It runs. Every feature is faster to bolt on than to place properly — that is **tactical programming**: optimizing for *this change works today*. The trouble is that each tactical change leaves a little complexity behind, and complexity compounds. The alternative is **strategic programming**: treating clean structure as an investment that keeps every *future* change cheap. Restructuring the Notes API now, before authentication and scheduled jobs arrive, is that investment — and the last cheap moment to make it.

The move itself is a **refactoring**: changing the *structure* of code without changing its *behavior*. Done well, it proceeds in small, safe steps — each one leaving the application runnable — rather than as one big rewrite. The solution is to divide the application into files and directories where each file has one clear responsibility — and hides the rest behind a small interface.

We will also take this moment to make three small, *deliberate* changes to the API’s behavior. In a real project they would be a separate change, announced to clients, because each one alters the contract (← [Chapter 13](#)); we bundle them here and call them out so that nothing changes silently:

1. The collection URL becomes `/notes/`, with a trailing slash — the Flask convention for collections. Flask automatically redirects `/notes` to `/notes/` (a 308 redirect, which keeps the method and body), so old clients keep working as long as they follow redirects (`curl` needs `-L`).
2. **DELETE** returns **204 No Content** with an empty body, instead of **200** with a message — the standard answer for “done, nothing to return.”
3. Notes include **created_at** in ISO 8601 format (`2024-01-15T14:00:05+00:00`) everywhere, including in the response to POST.

Everything else — the endpoints’ names, the list response `{"notes": [...], "count": n}`, the error format — stays exactly as it was. The timing header, the JSON content-type check, and the JSON error handlers from Chapter 13 are left out of this chapter’s listings to keep them short; Chapter 16 gives them their own modules.

15.2 The Standard Python Web Application Directory Structure

Here is the target structure for the Notes API after this chapter:



This is not arbitrary. Each decision follows from a principle:

- **app/** is a **package**: all application code lives under this directory. Separating it from the root keeps the root clean: only entry points and configuration files at the root level.
- **app/config.py**: all configuration loading in one file. When you need to add a new environment variable, there is exactly one place to change.
- **app/database.py**: all database code in one file. When you switch from SQLite to PostgreSQL (or from PostgreSQL to something else), you change one file.
- **app/routes/**: route handlers organized by feature area. As the application grows with dozens of endpoints, this directory grows with it — without touching the database code or configuration code.
- **run.py**: the one entry point for development. Anyone starting the development server runs `python run.py`. No ambiguity. (In production, a WSGI server starts the application instead — → [Chapter 18](#) and [Chapter 24](#).)

This layout is really two design properties made physical. **Cohesion** is how strongly the things inside one unit belong together: `config.py` contains *only* configuration, so it is highly cohesive. **Coupling** is how much one unit must know about another's internals: a route handler calls `get_db()` without knowing whether the database is SQLite or PostgreSQL, so the two are loosely coupled. The target you are aiming for — in this directory tree and in every module you will ever write — is **high cohesion, low coupling**: each part has one clear job, and the parts

touch through the narrowest possible interface. Almost every structural rule in this chapter is a specific application of that one sentence.

15.3 What a Python Package Is

Before restructuring the code, you need to understand a fundamental Python concept: the **package**.

In Python:

- A **module** is a single `.py` file. `config.py` is a module.
- A **package** is a directory of modules. A directory containing a special file called `__init__.py` is a **regular package**.

When Python sees a directory with an `__init__.py` file, it treats that directory as a regular package — a namespace that can contain modules and sub-packages, and that runs `__init__.py` when it is first imported.

The `__init__.py` file

`__init__.py` can be completely empty, or it can contain initialization code. Since Python 3.3, a directory *without* an `__init__.py` can still be imported, as a **namespace package** — a feature meant for splitting one package across several locations. For your own application, always include `__init__.py`: it makes the package explicit, gives you a place for package-level code, and avoids surprises with tools.

```
# A regular package:
my_package/
    __init__.py
    config.py
# You can do: from my_package import config
```

A package's directory name must be a valid Python identifier. `my-package/` (with a hyphen) cannot be imported at all — there is no way to write `import my-package` — so use underscores. (pip does treat `-` and `_` as the same in *distribution* names, such as `python-dotenv`; that is a different thing from the import name, which for that package is `dotenv`.)

How imports work across files

Once `app/` is a package (has `__init__.py`), you can import between its modules:

```
# In app/routes/notes.py:
from app.database import get_db    # Import get_db from app/database.py
from app.config import DATABASE_URL # Import DATABASE_URL from app/config.py
```

Python resolves `app.database` by:

1. Finding the app package (the `app/` directory with `__init__.py`)
2. Finding the database module inside it (`app/database.py`)
3. Looking up `get_db` inside that module

This is the foundation of organized Python code: files can import from each other through package paths.

A package is also a unit of **information hiding** — one of the most important ideas in software design. Each module exposes a few names the rest of the application is meant to use (`get_db`, `create_app`, `DATABASE_URL`) and keeps everything else as private implementation detail. By convention, Python signals “private” with a leading underscore: in `config.py` (coming up) you will see `_require()` and `_bool_env()` — helpers that exist only to build the public values and that no other module should import. The `__init__.py` of a package is where you decide, deliberately, what the package’s public surface is. Good modules expose little and hide much; the secret a module keeps is exactly the complexity its users never have to think about. That is the whole game: every well-placed boundary is a place the reader is *allowed not to look*.

15.4 The Application Factory Pattern

The most important structural decision in a Flask application is whether to create the Flask app object at module level (at the top of a file, when the file is imported) or inside a function (explicitly, when called).

The current approach creates the app at module level:

```
# Current approach — app created when app.py is imported
from flask import Flask
app = Flask(__name__)
```

The **application factory pattern** creates the app inside a function:

```
# Application factory pattern – app created by calling create_app()
from flask import Flask

def create_app():
    app = Flask(__name__)
    # ... configure the app ...
    return app
```

Why the factory pattern is better

Reason 1: Testability. When the app is created at module level, importing the module creates the app — immediately, once, with whatever environment variables happen to be set. With a factory, tests decide *when* an app is created: they can prepare a test configuration (a temporary database, for example) and then call `create_app()` to get a fresh application for each test (→ Chapter 17).

Reason 2: Multiple instances. Rarely needed, but sometimes you want to run two Flask applications in the same process (for example, an API and an admin panel). The factory pattern makes this possible.

Reason 3: Separation of creation from configuration. The factory function is the single, explicit place where the app is assembled: configuration loaded, blueprints registered, middleware attached. There is no question of “is the app configured yet?” — if `create_app()` has been called and returned, the app is ready.

Reason 4: Avoids circular imports. When the app is created at module level, other modules that import from `app.py` might run into the app being partially initialized. The factory pattern defers creation until everything is ready.

`create_app()` as a deep module

The factory is worth pausing on, because it is a clean example of a **deep module** — the single most useful idea for judging whether code is well-designed. A module’s *depth* is the ratio of how much it does to how much you must know to use it. A **deep module** hides a lot of functionality behind a small, simple interface. A **shallow module** exposes nearly as much complexity as it contains, so it barely pays for the cost of existing.

`create_app()` is deep. Its interface is the simplest imaginable — call it with no arguments, receive a finished application:

```
app = create_app() # the entire interface
```

Behind that one line sits everything the application needs, assembled in the right order: startup

checks run, configuration applied, the database teardown handler registered, two Blueprints imported and registered, the schema initialized. The caller knows *none* of it. That asymmetry — trivial interface, substantial hidden work — is what you want from every module you design. When you instead find yourself writing a function whose interface is about as complicated as its body (a thin wrapper that just forwards its arguments, or a “manager” class whose every method you must understand before you can call any of them), you have written a shallow module, and you should ask whether it earns its place or should be inlined away.

The application factory is the standard pattern recommended by Flask’s own documentation and used by virtually every production Flask application — not by fashion, but because it is deep, testable, and explicit.

15.5 Building the Restructured Application

Now build the actual files, one at a time.

Step 1: Create the directory structure

```
# Starting from the notes-api/ root directory
mkdir -p app/routes
touch app/__init__.py
touch app/routes/__init__.py
```

The `mkdir -p` flag creates parent directories as needed. After this, you have:

```
notes-api/
├── app/
│   ├── __init__.py      (empty for now)
│   └── routes/
│       └── __init__.py  (empty for now)
├── app.py               (the old monolith – will be replaced)
├── requirements.txt
├── .env
└── .env.example
```

Warning: Once the `app/` directory exists, `import app` refers to the *package*, not to `app.py` — Python finds a package directory before a module file of the same name. Anything that imports the old file (a test, a server command such as `gunicorn app:app`) now silently gets the new, still-empty package. Rename the old file (for example to `app_old.py`) as the first step, and delete it when the move is complete.

A related naming note: the package is called `app`, and inside it `create_app()` builds a Flask object that is conventionally *also* named `app`. That is common in Flask projects and harmless once you know it — `from app import create_app` imports from the package; `app = create_app()` is the application object. Many teams avoid the double meaning by naming the package after the project (`notes_api/`).

Step 2: `app/config.py` — Configuration

This module loads and validates all environment variables. It is the first thing the application needs, so it is the first module to write.

Listing 15.1 `app/config.py`

```

"""
    Configuration loading and validation for the Notes API.

    All environment variables are read here and here only.
    Other modules import from this module — they never call os.getenv() directly.
"""
import os
from dotenv import load_dotenv

# Load .env file before reading any environment variables.
# load_dotenv() only sets variables that aren't already in the environment,
# so real environment variables (set by the shell or deployment system)
# always take precedence over .env file values.
load_dotenv()

def _require(name: str) -> str:
    """
        Read a required environment variable.

        If the variable is missing, exit immediately with a clear error message
        rather than letting the application start with broken configuration.
    """
    value = os.environ.get(name)
    if not value:
        import sys
        print(
            f"Error: Required environment variable '{name}' is not set.\n"
            f"Add it to your .env file or set it in your shell before starting.",
            file=sys.stderr,
        )

```

```

        sys.exit(1)
    return value

def _bool_env(name: str, default: str = "false") -> bool:
    """
    Read an environment variable and interpret it as a boolean.

    Accepts 'true' or 'false' (case-insensitive).
    Any other value is treated as False.
    """
    return os.getenv(name, default).lower() == "true"

# -----
# Required configuration – the application cannot start without these
# -----

DATABASE_URL: str = _require("DATABASE_URL")
SECRET_KEY: str = _require("SECRET_KEY")

# -----
# Optional configuration – these have safe defaults
# -----

FLASK_DEBUG: bool = _bool_env("FLASK_DEBUG", default="false")
PORT: int = int(os.getenv("PORT", "5000"))
HOST: str = os.getenv("HOST", "127.0.0.1")
LOG_DIR: str = os.getenv("LOG_DIR", "")

# -----
# Derived configuration – computed from other values
# -----

# True if using SQLite, False if using PostgreSQL or another remote database.
# Used by the database module to choose the right connection approach.
IS_SQLITE: bool = DATABASE_URL.startswith("sqlite:///")

```

Key design decisions in this file:

1. `load_dotenv()` is called once, at module import time. Because Python caches module imports (Chapter 2's discussion of `sys.modules`), `load_dotenv()` runs exactly once no matter how many files import from `config.py`.
2. Every environment variable is read in this file and this file only. No other module calls `os.getenv()`. This means that to understand what environment variables the application

uses, you read exactly one file.

- Variables are module-level names (e.g., `DATABASE_URL`, `PORT`). Other modules import them: `from app.config import DATABASE_URL`. This is cleaner than passing `config` as function arguments or storing it in a global dictionary.
- The names do design work.** `_require` reads as a sentence at its call site — `DATABASE_URL = _require("DATABASE_URL")` says *this value is mandatory* with no comment needed, and the leading underscore marks it private (information hiding again). Compare a vague name like `get` or `check`: a good name tells you what you get back and what it costs, so the reader never has to open the function to find out. Naming is not decoration applied after the code works — it is part of the design. A function you struggle to name is usually a function doing too many things, and the naming difficulty is the design telling you to split it.

Step 3: `app/database.py` — Database Access

This module handles all database connections: opening, closing, and initializing.

Listing 15.2 `app/database.py`

```

"""
    Database connection management for the Notes API.

    Uses Flask's application context (the 'g' object) to manage one database
    connection per HTTP request. The connection is opened when first needed
    and closed when the request ends.

    Everything that differs between SQLite and PostgreSQL lives in this file,
    so the route handlers never need to know which database is in use.
"""
import sqlite3
from flask import g
from app.config import DATABASE_URL, IS_SQLITE

def open_db():
    """
        Open a database connection appropriate for the configured DATABASE_URL.

        Both kinds of connection return rows that behave like dictionaries
        (row["id"]), so the code that reads them does not care which it got.
    """
    if IS_SQLITE:
```

```

    # "sqlite:///notes.db" → path "notes.db"
    conn = sqlite3.connect(DATABASE_URL[len("sqlite:///"):])
    conn.row_factory = sqlite3.Row
    return conn

import psycopg2
import psycopg2.extras
# psycopg2 starts a transaction automatically; handlers commit explicitly
return psycopg2.connect(DATABASE_URL,
    ↪ cursor_factory=psycopg2.extras.RealDictCursor)

def get_db():
    """
    Return the database connection for the current request.

    If no connection exists yet (first database call in this request),
    opens one and stores it on Flask's 'g' object for reuse within
    the same request. Subsequent calls return the same connection.
    """
    if "db" not in g:
        g.db = open_db()
    return g.db

def close_db(error=None):
    """
    Close the database connection at the end of a request.

    Flask calls this automatically via teardown_appcontext. Handlers commit
    their own work *before* they return a response, so anything still
    uncommitted here belongs to a request that failed part-way: it is rolled
    back, never committed.
    """
    db = g.pop("db", None) # Remove 'db' from g, return None if not present
    if db is not None:
        db.rollback()      # A no-op after a commit; discards half-finished work
        db.close()

def sql(query: str) -> str:
    """Queries are written with ? placeholders; psycopg2 expects %s."""
    return query if IS_SQLITE else query.replace("?", "%s")

def to_iso(value) -> str:
    """A timestamp as ISO 8601 in UTC, whichever database produced it."""

```

```

if isinstance(value, str):                                # SQLite: "2024-01-15 14:00:05"
    ↪ (UTC)
    return value.replace(" ", "T") + "+00:00"
return value.isoformat()                                # PostgreSQL: a timezone-aware
    ↪ datetime

def row_to_note(row) -> dict:
    """Convert a database row into the note dictionary the API returns."""
    note = dict(row)
    note["created_at"] = to_iso(note["created_at"])
    return note

def init_db(app):
    """
    Create database tables if they do not exist.

    Called once at application startup. Uses CREATE TABLE IF NOT EXISTS
    so it is safe to call on every startup.

    The 'app' parameter is the Flask application instance, needed to
    push an application context so that 'g' is available.
    """
    with app.app_context():
        db = get_db()
        cursor = db.cursor()
        if IS_SQLITE:
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS notes (
                    id            INTEGER PRIMARY KEY AUTOINCREMENT,
                    content       TEXT NOT NULL,
                    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                )
            """)
        else:
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS notes (
                    id            SERIAL PRIMARY KEY,
                    content       TEXT NOT NULL,
                    created_at    TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
                )
            """)
        db.commit()
        print("Database initialized.", flush=True)

```

Key concepts in this file:

- `get_db()` uses the `g` object to store one connection per request. This is the same pattern as [Chapter 4](#) and [Chapter 5](#), but now isolated in its own file.
- `init_db(app)` takes the Flask app as a parameter and uses `app.app_context()`. An **application context** is Flask’s mechanism for making `g` available outside of a running request. We push one temporarily so that `get_db()` can work during startup initialization.
- `close_db()` never commits. A handler that changes data commits *before* it returns its response — so a 201 Created is only ever sent for a note that is really saved. Whatever is left uncommitted at teardown belongs to a request that failed, and is rolled back.
- `sql()`, `to_iso()`, and `row_to_note()` keep the two databases’ differences — placeholder style and timestamp type — inside this one file.

This file is also a small lesson in function design. Look at how the four functions divide the work. `open_db()` is a simple *factory*: give it nothing, get a fresh connection. `get_db()` is a *query that memoizes*: it answers “the connection for this request,” creating one only if needed, and is safe to call repeatedly. `close_db()` is a *command*: it changes state (rolls back, then closes) and returns nothing useful. That split is an instance of **command/query separation** — a principle that a function should either *do* something (a command, with side effects) or *answer* something (a query, side-effect-free), but not both, because functions that quietly do both surprise their callers. Each function here is also small, takes few parameters, and has one job — the practical shape of code that is easy to test and easy to change. When a function instead grows past a screenful, sprouts a fourth or fifth parameter, or needs the word “and” to describe what it does, those are the §15.1 smells reappearing at the function level, and the same fix applies: split it.

Step 4: `app/routes/notes.py` — Notes Route Handlers

Route handlers are the HTTP interface. They validate input, call database functions, and return responses.

Listing 15.3 `app/routes/notes.py`

```
"""
Route handlers for the /notes endpoints.

These functions handle HTTP requests for creating, reading, and deleting
notes. They are registered with Flask via a Blueprint (explained below).
"""
```

```

from flask import Blueprint, request, jsonify
from app.database import get_db, sql, row_to_note

# A Blueprint is Flask's mechanism for grouping related routes.
# Think of it as a mini Flask application that gets "registered"
# onto the main application later.
#
# Arguments:
#   "notes"      - the name of this Blueprint (used internally by Flask)
#   __name__     - tells Flask where this Blueprint is defined (for finding
#                  template files, static files, etc.)
#
# url_prefix="/notes" means every route in this Blueprint is automatically
# prefixed with /notes. So @notes_bp.route("/") becomes /notes/.
notes_bp = Blueprint("notes", __name__, url_prefix="/notes")

@notes_bp.route("/", methods=["GET"])
def list_notes():
    """
    GET /notes/ - Return all notes, newest first.

    Response (HTTP 200):
        {"notes": [{ "id": 1, "content": "First note",
                     "created_at": "2024-01-15T14:00:05+00:00"}, ...],
         "count": 1}
    """
    cursor = get_db().cursor()
    cursor.execute("SELECT id, content, created_at FROM notes ORDER BY id DESC")
    notes = [row_to_note(row) for row in cursor.fetchall()]
    return jsonify({"notes": notes, "count": len(notes)}), 200

@notes_bp.route("/", methods=["POST"])
def create_note():
    """
    POST /notes/ - Create a new note.

    Request body (JSON):
        {"content": "The text of the note"}

    Response (on success, HTTP 201 Created):
        {"id": 42, "content": "The text of the note", "created_at": "..."}

    Response (on error, HTTP 400 Bad Request):
        {"error": "Description of what went wrong"}
    """

```

```

"""
data = request.get_json(silent=True)
# silent=True: None instead of an error if the body is missing or not valid JSON
if not isinstance(data, dict):
    return jsonify({"error": "Request body must be a JSON object"}), 400

content = data.get("content")
if content is None:
    return jsonify({"error": "'content' is required"}), 400
if not isinstance(content, str):
    return jsonify({"error": "'content' must be a string"}), 400

content = content.strip() # Remove leading/trailing whitespace
if not content:
    return jsonify({"error": "'content' cannot be empty"}), 400
if len(content) > 10_000:
    return jsonify({"error": "'content' cannot exceed 10,000 characters"}), 400

db = get_db()
cursor = db.cursor()
# RETURNING (PostgreSQL, and SQLite 3.35+) hands back the new row in one step
cursor.execute(
    sql("INSERT INTO notes (content) VALUES (?) RETURNING id, content,
→   created_at"),
    (content,),
)
note = row_to_note(cursor.fetchone())
db.commit() # Commit before responding: a 201 must mean the note is saved
return jsonify(note), 201

@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id):
    """
    GET /notes/<note_id> – Return a specific note by ID.

    The <int:note_id> in the route pattern tells Flask to extract the
    value from the URL and pass it as an integer to this function.
    If the value in the URL is not a valid integer, Flask returns 404
    automatically before this function is called.

    Response (on not found, HTTP 404):
        {"error": "Note 42 not found"}
    """
    cursor = get_db().cursor()

```



```

    cursor.execute(sql("SELECT id, content, created_at FROM notes WHERE id = ?"),
        ↪ (note_id,))
    row = cursor.fetchone()
    if row is None:
        return jsonify({"error": f"Note {note_id} not found"}), 404
    return jsonify(row_to_note(row)), 200

@notes_bp.route("/<int:note_id>", methods=["DELETE"])
def delete_note(note_id):
    """
    DELETE /notes/<note_id> - Delete a specific note.

    Response (on success, HTTP 204 No Content): empty body
    Response (on not found, HTTP 404): {"error": "Note 42 not found"}
    """
    db = get_db()
    cursor = db.cursor()
    cursor.execute(sql("DELETE FROM notes WHERE id = ?"), (note_id,))
    db.commit()
    if cursor.rowcount == 0:
        return jsonify({"error": f"Note {note_id} not found"}), 404
    return "", 204

```

Notice what is *not* here: no `if IS_SQLITE:` anywhere. Each handler is written once, and `database.py` absorbs the differences between the two databases. That is information hiding at work — and it is what made the handlers short enough to read at a glance.

The Blueprint concept:

A **Blueprint** (imported from `flask`) is Flask's mechanism for grouping related routes. Think of it as a partial Flask application: it holds route definitions but is not itself a running application. When you call `app.register_blueprint(notes_bp)` in the factory, Flask adds all the Blueprint's routes to the main application's route table.

Blueprints solve a specific problem: how to define routes in a file that is separate from the file where the Flask app object is created. Without Blueprints, you would need to import `app` into every routes file, which creates circular imports. With Blueprints, you create a Blueprint object locally, define routes on it, and register it onto the app later.

Step 5: `app/routes/health.py` — Health Check Route

Listing 15.4 `app/routes/health.py`

```

"""
Health check route for the Notes API.

A health check endpoint is a special route that returns a simple success
response. Deployment systems, load balancers, and monitoring tools call
this endpoint to verify that the application is running and responding.

Unlike the /notes endpoints, the health check does not interact with the
database. It only verifies that the Flask application itself is alive.
A separate, more detailed health check could verify database connectivity;
Chapter 35 – Observability builds one.
"""

from flask import Blueprint, jsonify

health_bp = Blueprint("health", __name__)

@health_bp.route("/health", methods=["GET"])
def health():
    """
    GET /health – Return a simple health status.

    This route is intentionally minimal. Its purpose is to answer
    the question "is the process running and accepting HTTP requests?"
    If this endpoint returns 200, the answer is yes.

    Response (HTTP 200 OK):
        {"status": "ok"}
    """
    return jsonify({"status": "ok"}), 200

```

Step 6: `app/startup.py` — Startup Checks

Move the startup checks from [Chapter 14](#)’s `startup.py` into the application package. They now take their values from `app.config` instead of reading the environment themselves — `config.py` stays the one place where environment variables are read — and they are split by *when* they apply: most checks apply wherever the application runs, but the port check applies only to the development server (a production server such as Gunicorn binds the port itself, before it loads the application).

Listing 15.5 app/startup.py

```

"""
Startup validation for the Notes API.

These checks run before the application begins accepting requests.
If any check fails, the process exits immediately with a clear error message
and a non-zero exit code.

This is the enforcement of the runtime contract defined in Chapter 14.
"""
import errno
import os
import socket
import sys
from urllib.parse import urlsplit

from app.config import DATABASE_URL, IS_SQLITE, SECRET_KEY

def check_python_version():
    """Verify that the Python interpreter meets the minimum version requirement."""
    required = (3, 10)
    if sys.version_info < required:
        _fail(
            f"Python {required[0]}.{required[1]}+ is required.\n"
            f"You are running Python {sys.version.split()[0]}."
        )

def check_secret_key():
    """Verify that SECRET_KEY is long enough to be a real secret."""
    # (config.py has already exited if SECRET_KEY or DATABASE_URL is missing)
    if len(SECRET_KEY) < 24:
        _fail(
            f"SECRET_KEY must be at least 24 characters long "
            f"(current length: {len(SECRET_KEY)}).\n"
            f"Generate one with: python -c \"import secrets;"
            f"    print(secrets.token_hex(32))\"
        )

def check_database():
    """Verify that the database is reachable before accepting requests."""
    if IS_SQLITE:
        return # SQLite is a local file – no network connectivity to verify
    try:

```

```

import psycpg2
conn = psycpg2.connect(DATABASE_URL, connect_timeout=5)
conn.close()
print("Database connectivity: OK", flush=True)
except Exception as e:
    _fail(
        f"Cannot connect to the database.\n"
        f"DATABASE_URL: {_mask(DATABASE_URL)}\n"
        f"Error: {e}\n"
        f"Ensure the database server is running and reachable."
    )

def check_port(host: str, port: int):
    """
    Verify that the development server's port is free.

    Only for `python run.py`. Skipped in the child process of Flask's
    reloader (FLASK_DEBUG=true), because the parent already holds the port.
    """
    if os.environ.get("WERKZEUG_RUN_MAIN") == "true":
        return
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        try:
            s.bind((host, port))
        except OSError as e:
            if e.errno != errno.EADDRINUSE:
                raise
            _fail(
                f"Port {port} on {host} is already in use.\n"
                f"Find the process with: lsof -i :{port}"
            )

def run():
    """Run the checks that apply wherever the application starts."""
    print("Running startup checks...", flush=True)
    check_python_version()
    check_secret_key()
    check_database()
    print("All startup checks passed.", flush=True)

def _mask(url: str) -> str:
    """Hide the password in a database URL before it is printed."""
    password = urlsplit(url).password

```

```

    return url.replace(f":{password}@", "****@", 1) if password else url

def _fail(message: str) -> None:
    """Print an error message and exit with code 1."""
    print(f"\nStartup failed:\n {message}\n", file=sys.stderr)
    sys.exit(1)

```

Step 7: app/__init__.py — The Application Factory

This is the central file of the restructured application. It defines `create_app()`, which assembles all the pieces.

Listing 15.6 app/__init__.py

```

"""
Application factory for the Notes API.

Call create_app() to get a configured, ready-to-use Flask application
instance. This is the application factory pattern: instead of creating
the Flask app at module level, we create it inside a function.

Benefits:
- Each call to create_app() returns a fresh instance (useful in tests)
- All assembly happens in one place: checks, configuration, database, blueprints
- No circular imports: blueprints don't need to import the app object
"""
from flask import Flask
from app import startup
from app.config import SECRET_KEY, FLASK_DEBUG
from app.database import close_db, init_db

def create_app() -> Flask:
    """
    Create and configure a Flask application instance.

    This function:
    1. Runs the startup checks (the runtime contract from Chapter 14)
    2. Creates the Flask application object and applies configuration
    3. Registers the teardown handler (to close database connections)
    4. Registers Blueprints (the route handlers)
    5. Initializes the database schema

    Returns the configured application, ready to serve requests.
    """

```

```

"""
startup.run()

app = Flask(__name__)

# -----
# Apply configuration to the Flask application
# -----
# Flask uses app.config as a dictionary for its own settings and for
# settings you want available via current_app.config in route handlers.
app.config["SECRET_KEY"] = SECRET_KEY
app.config["DEBUG"] = FLASK_DEBUG

# -----
# Register the database teardown handler
# -----
# Flask calls teardown_appcontext functions at the end of every request,
# even if an exception occurred, so connections are always closed.
app.teardown_appcontext(close_db)

# -----
# Register Blueprints (route handlers)
# -----
from app.routes.health import health_bp
from app.routes.notes import notes_bp

app.register_blueprint(health_bp)
app.register_blueprint(notes_bp)

# -----
# Initialize the database schema (CREATE TABLE IF NOT EXISTS)
# -----
init_db(app)

return app

```

Why Blueprint imports are inside the function:

Notice that `from app.routes.health import health_bp` appears *inside* `create_app()`, not at the top of the file. Configuration is not the reason — importing `app.config` at the top of this file already loaded and validated it the moment the package was imported. The reason is **import direction**: route modules import from the `app` package (`app.database`), so if the package's `__init__.py` also imported the route modules at its top, the two would import each other while both were half-initialized. Importing the Blueprints

inside the function, when the package is fully loaded, keeps the dependency one-way.

This is the general solution to circular imports in Flask applications. `app/routes/notes.py` imports from `app.database`. `app/__init__.py` imports `app.routes.notes`. If `app/routes/notes.py` also imported the `app` object from `app/__init__.py`, that would be circular. With Blueprints, route files never import the `app` object directly — they only import their Blueprint object.

15.6 The Entry Point: run.py

The entry point is the one file that humans and deployment systems run to start the application.

Listing 15.7 run.py

```

"""
Entry point for the Notes API development server.

    python run.py

In production, a WSGI server calls the application factory directly
(Chapters 18 and 24):

    gunicorn "app:create_app()"

This file is intentionally minimal. Its job is to:
    1. Create the Flask application (create_app() also runs the startup checks)
    2. Check that the development server's port is free
    3. Start the development server

All application logic lives in the app/ package.
"""
from app import create_app
from app.config import FLASK_DEBUG, HOST, PORT
from app.startup import check_port

app = create_app()

if __name__ == "__main__":
    check_port(HOST, PORT)
    print(f"Starting Notes API on http://{HOST}:{PORT}", flush=True)
    app.run(host=HOST, port=PORT, debug=FLASK_DEBUG)

```

Why `run.py` does so little: it no longer loads the `.env` file or reads any environment variable itself — `app/config.py` does both, the first time anything imports the package. It keeps `app = create_app()` at module level so that tools which look for an `app` object in a file (for example `flask --app run shell`) can find one. The `if __name__ == "__main__":` block runs only when you execute `python run.py` directly: that is the one situation in which *this* process will bind the port, so it is the one place to check that the port is free.

15.7 The Updated Project Structure

After creating all these files, the project looks like this:

```

notes-api/
├── app/
│   ├── __init__.py      ← create_app() factory function
│   ├── config.py        ← All environment variable loading
│   ├── database.py      ← Database connection management
│   ├── startup.py       ← Startup checks (runtime contract enforcement)
│   └── routes/
│       ├── __init__.py  ← (empty – marks routes/ as a package)
│       ├── health.py    ← GET /health
│       └── notes.py     ← GET/POST /notes/, GET/DELETE /notes/<id>
├── run.py               ← Entry point
├── requirements.txt     ← Pinned dependencies
├── .env                 ← Local configuration (NOT committed)
├── .env.example         ← Template (committed)
├── .python-version      ← Python version declaration
└── .gitignore           ← Excludes venv/, .env, *.db, __pycache__/

```


What each file is responsible for

File	Single Responsibility
<code>run.py</code>	Start the application
<code>app/__init__.py</code>	Assemble the application (factory)
<code>app/config.py</code>	Load and validate configuration
<code>app/database.py</code>	Manage database connections
<code>app/startup.py</code>	Enforce the runtime contract
<code>app/routes/health.py</code>	Handle GET /health
<code>app/routes/notes.py</code>	Handle all /notes endpoints

Each file has exactly one reason to change. If you change how the database is connected, you change `database.py`. If you add a new configuration variable, you change `config.py`. If you add a new endpoint category (for example, `/users`), you add a new file `app/routes/users.py` without touching any existing file.

15.8 The Startup Sequence in the Restructured Application

When you run `python run.py`, this is the sequence of events:

```
python run.py
```

- 1. Python runs run.py as the `__main__` module
- 2. `from app import create_app`
 Importing anything from the package first runs `app/__init__.py`, which imports:
 `app.startup` → which imports `app.config`:
 `load_dotenv()` reads `.env`;
 `DATABASE_URL` and `SECRET_KEY` are read – or the process exits
 `app.config` (already loaded – taken from `sys.modules`)
 `app.database` (`open_db`, `get_db`, `close_db`, `sql`, `init_db` ...)
 (route modules not yet imported)
- 3. `app = create_app()`
 `startup.run()`:
 `check_python_version()` — passes or exits
 `check_secret_key()` — passes or exits
 `check_database()` — passes or exits
 "All startup checks passed."
 Creates the Flask instance and applies configuration (`SECRET_KEY`, `DEBUG`)
 Registers the teardown handler (`close_db`)
 Imports and registers `health_bp` → loads `app/routes/health.py`
 Imports and registers `notes_bp` → loads `app/routes/notes.py`
 `init_db(app)` → creates tables if they don't exist
 Returns the configured Flask app
- 4. `if __name__ == "__main__":` block runs
 `check_port("127.0.0.1", 5000)` — passes or exits
 "Starting Notes API on `http://127.0.0.1:5000`"
- 5. `app.run(host="127.0.0.1", port=5000, debug=False)`
 Flask's development server starts
 Werkzeug socket binding: listen on `127.0.0.1:5000`
 "Waiting for connections..."

Step 2 holds a subtlety worth remembering: in Python, importing *any* module inside a package – even from `app.startup import ...` – first runs the package's `__init__.py`. That is why configuration is loaded and validated before the startup checks are even defined, and why `config.py` is the right place to fail fast on missing variables.

When a production server starts the application instead (gunicorn "`app:create_app()`"), steps 2 and 3 happen exactly the same way inside each server process; only step 4 and 5 differ – the server has already bound the port, and it takes over from there.

This sequence is explicit and ordered. Each step depends only on what came before it. Compare this to the original `app.py`, where import time and runtime were mixed together and the order of operations was difficult to reason about.

15.9 Verifying the Restructured Application

After creating all the files, verify that everything works:

```
# From the notes-api/ root directory
# Make sure your virtual environment is activated and requirements are installed

# Start the application
python run.py
```

Expected output:

```
Running startup checks...
Database connectivity: OK      ← (only if using PostgreSQL)
All startup checks passed.
Database initialized.
Starting Notes API on http://127.0.0.1:5000
  * Serving Flask app 'app'
  * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use
↳ a production WSGI server instead.
  * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Test each endpoint:

```
# Health check
curl http://127.0.0.1:5000/health
# {"status":"ok"}

# Create a note
curl -X POST http://127.0.0.1:5000/notes/ \
  -H "Content-Type: application/json" \
  -d '{"content": "Testing the restructured application"}'
# {"content":"Testing the restructured
↳ application","created_at":"2024-01-15T14:00:05+00:00","id":1}

# Retrieve all notes
curl http://127.0.0.1:5000/notes/
# {"count":1,"notes":[{"content":"Testing the restructured
↳ application","created_at":"2024-01-15T14:00:05+00:00","id":1}]}
```

```
# The old URL still works through a redirect (-L tells curl to follow it)
curl -L http://127.0.0.1:5000/notes

# Retrieve one note
curl http://127.0.0.1:5000/notes/1
# {"content":"Testing the restructured
↳ application","created_at":"2024-01-15T14:00:05+00:00","id":1}

# Delete a note
curl -X DELETE http://127.0.0.1:5000/notes/1
# (empty body, HTTP 204)
```

If everything works, the restructuring is complete. Apart from the three deliberate changes listed in §15.1 — the `/notes/` URL, 204 on delete, and ISO timestamps — the application behaves as before, with the same data, but is now organized in a maintainable structure.

Common errors during restructuring

A second, empty `notes.db` appears somewhere else

`python notes-api/run.py`, run from the parent directory, *does* start: Python puts the script's own directory (`notes-api/`) on `sys.path`, so the app package is found (← Chapter 2). But `DATABASE_URL=sqlite:///notes.db` is a path relative to the **current working directory**, so SQLite quietly creates a new, empty `notes.db` in the parent directory, and your notes seem to have vanished. Run the application from the project root — or use an absolute path in `DATABASE_URL`.

```
# Surprising – works, but uses ../notes.db:
cd ..
python notes-api/run.py

# Correct – running from project root:
cd notes-api
python run.py
```

ModuleNotFoundError: No module named 'app'

This one appears when a tool imports the package from a directory where `app/` is not on the path — for example, running `pytest` or `python -m app.something` from the wrong directory. Run tools from the project root.

ImportError: cannot import name 'create_app' from 'app'

The `app/__init__.py` file exists but does not define `create_app`. Check that the file was

saved correctly.

sqlite3.OperationalError: unable to open database file

The directory in `DATABASE_URL` does not exist, or the current user cannot write to it. Check the path (it is relative to the directory you start the application from) and its permissions.

15.10 The Standardized Startup Command

Before this chapter, starting the application required knowing to run `python app.py`. After restructuring, it is `python run.py`. This is a convention: the entry point is always called `run.py` (or sometimes `wsgi.py` or `main.py` — projects differ, but they always have a single designated entry point).

Standardize this further by documenting it in two places:

In `.env.example` — add a comment at the top

```
# .env.example — environment variable template
# Run the application with: python run.py
# Development server only — for production, see: gunicorn "app:create_app()"

DATABASE_URL=sqlite:///notes.db
# Required — generate a value with:
# python -c "import secrets; print(secrets.token_hex(32))"
SECRET_KEY=
FLASK_DEBUG=false
PORT=5000
HOST=127.0.0.1
```

In a Makefile (optional but common)

Many Python projects use a Makefile to standardize common commands:

```
# Makefile — common development commands
# Run with: make <target>

.PHONY: run install test clean

# Start the development server
run:
    python run.py

# Install dependencies into the active virtual environment
```

```

install:
    pip install -r requirements.txt

# Run the test suite (Chapter 17 introduces testing)
test:
    pytest

# Remove Python bytecode cache files
clean:
    find . -type d -name __pycache__ -exec rm -rf {} +
    find . -name "*.pyc" -delete

```

With a Makefile:

```

make run      # starts the server
make install  # installs dependencies
make clean    # removes cache files

```

A Makefile is not required, but it reduces the cognitive load for new developers: they read the Makefile, see `make run`, and know how to start the application without reading any other documentation.

Key Takeaways

- A **single-file application** becomes unmanageable as it grows. The solution is a directory structure where each file has one clear responsibility.
- A **Python package** is a directory of modules; with an `__init__.py` file it is a *regular* package, which is what an application should use. Directory names must be valid identifiers (underscores, not hyphens).
- The **application factory pattern** defines `create_app()` as the function that creates and configures the Flask application. Calling this function returns a ready-to-use app. This pattern enables testing, prevents circular imports, and makes the startup sequence explicit.
- **Blueprints** are Flask's mechanism for defining routes in files separate from the Flask app object. Each Blueprint groups related routes (all `/notes` routes in `notes_bp`) and is registered onto the main app in `create_app()`.
- The **entry point** (`run.py`) is the single file you run in development. It is intentionally minimal: create the app (which runs the startup checks), check the port, start the server. In production, a WSGI server calls `create_app()` directly.
- **One responsibility per file** is the organizing principle. `config.py` loads configuration. `database.py` manages connections. `routes/notes.py` handles note endpoints. Changing one area of the application means changing at most one file.

- The **import order matters**: importing anything from a package runs its `__init__.py` first, so configuration is loaded the moment the package is imported. Put checks where they run on every path into the application (`create_app()`), and development-only checks (the port) where only the development server runs them.
- **Complexity is the real enemy**, and you recognize it by three symptoms: change amplification, high cognitive load, and unknown unknowns. Structure exists to reduce all three.
- Aim for **high cohesion and low coupling**: each unit does one job and touches others through the narrowest interface. **Information hiding** — exposing little, hiding much — is how you achieve low coupling.
- Prefer **deep modules** (simple interface, substantial hidden work, like `create_app()`) over shallow ones (interfaces nearly as complex as their bodies).
- **Naming, small single-purpose functions, and command/query separation** are design tools, not cosmetics. A name you cannot find, or a function you cannot describe in one sentence, is the design asking to be split.
- Restructuring is **strategic programming** — paying a small cost now to keep all future changes cheap — carried out as **refactoring**: behavior-preserving changes made in small, safe steps.

Exercises

1. **Predict**. Move a route from the notes blueprint into `create_app()` directly. Does the application still work? What does it cost you in structure?
2. **Break and fix**. Create a circular import: import something from `app.routes.notes` inside `app/database.py`. Read the error, and fix it by moving the import to where it is needed.
3. **Extend**. Add a second blueprint, `stats_bp`, with GET `/stats/` returning the number of notes, registered in `create_app()`.
4. **Explain**. Pick one module in the Notes API and argue whether it is “deep” or “shallow” in the sense of this chapter.

Solutions and hints are in the companion repository, in `exercises/chapter-15.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What’s Next

The application is now structured for maintainability. But there is a critical operational gap: when something goes wrong — an unhandled exception, a database error, a validation failure — where does that information go? Right now it goes to the terminal, which disappears when

the terminal closes, and which no one is reading in production.

The next step is building a proper logging and error handling system: one that persists across process restarts, can be searched and filtered, and surfaces problems to the people responsible for fixing them.

→ [Chapter 16 — Logging and Error Handling](#)

Logging and Error Handling

When a production application fails, someone needs to know. Not eventually, not when a user complains — immediately, with enough information to understand what went wrong and why.

Right now, the Notes API communicates failure through `print()` statements and Python's default exception output: text that appears on the terminal where the server is running. This is adequate when you are sitting at that terminal during development. It is completely inadequate for production.

In production:

- The application runs in the background. There is no terminal to watch.
- The process might restart automatically after a crash. Any `print()` output from before the crash is lost.
- Multiple instances of the application might run simultaneously, each writing output to different places.
- Someone needs to search through hours of output to find the one line that explains why the application failed at 3:00 AM.

Logging is the discipline of writing structured, persistent, searchable records of what an application is doing. A logging system is not just `print()` with timestamps — it is a complete infrastructure for capturing, routing, filtering, storing, and querying application events.

Error handling is the discipline of catching failures before they crash the application, converting them into informative responses, and ensuring that every failure is recorded.

This chapter builds both: a production-grade logging system for the Notes API and a comprehensive error handling layer.

In this chapter

By the end of this chapter, you will understand:

- Why `print()` is insufficient for production logging and what replaces it
- How Python's logging module works: loggers, handlers, formatters, and levels
- How to configure structured logging that writes to both the console and a rotating log file
- How to capture every unhandled exception before it crashes the process
- How to write a Flask error handler that returns consistent JSON error responses
- The error-handling **philosophy** behind that handler: when failure should be a return value versus an exception, *where* an error should be handled, and what to do when it cannot be handled meaningfully
- How to log every HTTP request and response with relevant context
- What structured logging (JSON format) is and why it matters for production systems
- How to debug methodically: reproduce, read the traceback from the bottom up, stop inside the code with `breakpoint()`, and find the commit that broke something with `git bisect`

16.1 What Is Wrong with `print()`

Consider what the current Notes API does when something fails:

```
# The current behavior – no logging, only Python's default traceback
@notes_bp.route("/", methods=["POST"])
def create_note():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return jsonify({"error": "Request body must be a JSON object"}), 400
    # ... if an unexpected exception occurs here, Python prints a traceback
    # to stderr and Flask returns a 500 response
```

If a database connection fails unexpectedly, Python prints something like:

```

Traceback (most recent call last):
  File "/notes-api/app/routes/notes.py", line 71, in create_note
    db = get_db()
  File "/notes-api/app/database.py", line 42, in get_db
    g.db = open_db()
  File "/notes-api/app/database.py", line 29, in open_db
    return psycopg2.connect(DATABASE_URL,
        ↪ cursor_factory=psycopg2.extras.RealDictCursor)
psycopg2.OperationalError: connection to server at "localhost" (127.0.0.1), port 5432
        ↪ failed: Connection refused

```

This output goes to `stderr`. In development, you see it in your terminal. In production:

- If the process is run by `systemd` (Chapter 25), `stderr` is captured by the journal — but only until the journal fills up and older entries are deleted.
- If the process is run in a container (Chapter 18), `stderr` goes to the container runtime’s log buffer — which has a size limit.
- If the process crashes, the output from the moment before the crash might not be flushed to disk.
- If you want to find all database errors from the past week, there is no way to search: the output is unstructured text mixed with all other output.

The specific problems with `print()` are:

1. **No severity levels.** A debug trace and a critical error look identical when printed. There is no way to filter “show me only serious problems.”

2. **No automatic context.** When did this happen? Which endpoint was serving the request? Which user triggered it? What was the request ID? `print()` only writes what you explicitly include.

3. **No routing.** `print()` goes to `stdout` (or `stderr` if you specify). You cannot say “send INFO messages to a log file and ERROR messages to a monitoring service” with `print()`.

4. **No file rotation.** If you redirect print output to a file (`python run.py >> app.log`), the file grows forever. Eventually it fills the disk.

5. **No coordination between threads.** If several threads call `print()` at the same moment, the text of one line and the newline of another can interleave, producing merged or broken lines. Each logging handler holds a lock while it writes a record, so every record comes out whole.

Python’s built-in logging module solves all of these problems.

16.2 Python's Logging Module: The Architecture

The logging module has four main components that work together:

Loggers

A **logger** is the object your code writes to. You get a logger by name:

```
import logging
logger = logging.getLogger("app.routes.notes")
```

Logger names form a hierarchy using dots. "app.routes.notes" is a child of "app.routes", which is a child of "app", which is a child of the **root logger**.

When you write `logger.error("something failed")`, the logger first compares the message's level with its own *effective* level (its own level if it has one, otherwise the level of its nearest ancestor that does). If the message passes, it is handed to the handlers of that logger and then to the handlers of **every** ancestor, all the way up to the root logger, unless a logger on the way has `propagate = False`. The ancestors' own levels are not checked again. This is why attaching a handler to both "app" and the root logger prints every line twice.

Handlers

A **handler** determines where a log message goes. There are many built-in handlers:

- `StreamHandler`: writes to a stream (stdout or stderr)
- `FileHandler`: writes to a file
- `RotatingFileHandler`: writes to a file, rotates when it reaches a size limit
- `TimedRotatingFileHandler`: writes to a file, rotates at a regular time interval
- `SysLogHandler`: writes to the Unix syslog daemon
- `SMTPHandler`: sends an email (useful for critical errors)

You can attach multiple handlers to one logger, sending messages to multiple destinations simultaneously.

Formatters

A **formatter** defines what each log line looks like. A formatter takes the log record — which contains the message, the level, the logger name, the timestamp, the source file, the line number, and more — and converts it to a string (or JSON).

Levels

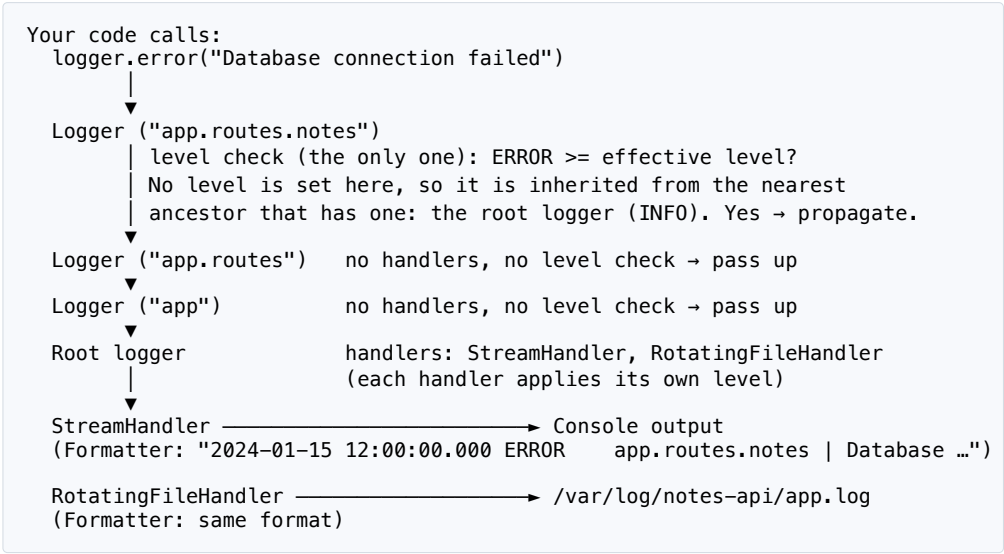
Every log message has a **level**, and every logger and handler can have a minimum level. Messages below the minimum level are silently discarded.

The standard levels, in order from lowest to highest severity:

Level	Integer	When to use
DEBUG	10	Detailed diagnostic information, useful during development
INFO	20	Confirmation that things are working as expected
WARNING	30	Something unexpected happened, but the application continues
ERROR	40	A serious problem occurred; some functionality is broken
CRITICAL	50	A very serious error; the application may not be able to continue

Setting a logger's level to **INFO** means **DEBUG** messages are discarded. Setting it to **WARNING** means **DEBUG** and **INFO** messages are discarded.

How they connect



16.3 Configuring Logging for the Notes API

Logging needs three settings, and Chapter 15 made `app/config.py` the one place where environment variables are read. `LOG_DIR` is already there; add the other two next to it:

```
# app/config.py (additions, in the optional-configuration section)
LOG_LEVEL: str = os.getenv("LOG_LEVEL", "INFO")
LOG_FORMAT: str = os.getenv("LOG_FORMAT", "human").lower() # "human" or "json"
```

Then create a new file `app/logging_config.py` that sets up the logging system for the entire application.

Listing 16.1 `app/logging_config.py`

```
"""
Logging configuration for the Notes API.

Call configure_logging() once at application startup, before any other
code that might produce log output. After calling this function, all
code can use:

    import logging
    logger = logging.getLogger(__name__)
    logger.info("Something happened")

and the output will be routed according to the configuration here.
"""
import json
import logging
import logging.handlers
import os
import sys
from datetime import datetime, timezone

from app.config import LOG_DIR, LOG_FORMAT, LOG_LEVEL

# -----
# Log level configuration
# -----

_LEVELS = {
    "DEBUG": logging.DEBUG,
    "INFO": logging.INFO,
```

```

    "WARNING": logging.WARNING,
    "ERROR": logging.ERROR,
    "CRITICAL": logging.CRITICAL,
}

def _parse_log_level(level_name: str) -> int:
    """
    Convert a log level name (DEBUG, INFO, WARNING, ERROR, CRITICAL)
    to its integer value. Falls back to INFO if the name is unrecognized.
    """
    level = _LEVELS.get(level_name.upper())
    if level is None:
        # Logging is not configured yet, so this one message goes to stderr directly
        print(f"Warning: Unrecognized LOG_LEVEL '{level_name}'. Using INFO.",
            file=sys.stderr)
        return logging.INFO
    return level

# -----
# Formatters
# -----

class HumanReadableFormatter(logging.Formatter):
    """
    A formatter that produces human-readable log lines.

    Output format:
    2024-01-15 14:32:01.234 INFO      app.routes.notes | Note 42 created
    successfully.
    2024-01-15 14:32:01.301 ERROR      app.database      | Connection failed:
    timeout
    """

    def format(self, record: logging.LogRecord) -> str:
        # Format timestamp in ISO 8601 with milliseconds
        # datetime.fromtimestamp converts the record's Unix timestamp
        # to a human-readable datetime. timezone.utc ensures UTC.
        ts = datetime.fromtimestamp(record.created, tz=timezone.utc)
        timestamp = ts.strftime("%Y-%m-%d %H:%M:%S.") + f"{ts.microsecond //
    1000:03d}"

        # Left-pad the level name to 8 characters for alignment
        level = record.levelname.ljust(8)

```

```

    # The logger name (e.g., "app.routes.notes"), padded so the | line up
    name = record.name.ljust(18)

    # The actual log message
    message = record.getMessage()

    # Format the base line
    line = f"{timestamp} {level} {name} | {message}"

    # If there is exception info (a traceback), append it
    if record.exc_info:
        # formatException returns the traceback as a string
        exc_text = self.formatException(record.exc_info)
        line = f"{line}\n{exc_text}"

    return line

class JSONFormatter(logging.Formatter):
    """
    A formatter that produces JSON log lines.

    Each log line is a single JSON object on one line. This format is
    designed for production environments where log aggregation tools
    (Elasticsearch, Datadog, Splunk, CloudWatch) parse structured logs
    for searching and alerting.

    Output format (one line per log record):
    {"timestamp":"2024-01-15T14:32:01.234Z","level":"INFO",
     "logger":"app.routes.notes","message":"Created note id=42.",
     "function":"create_note","line":54}
    """

    # Every attribute a plain LogRecord has. Anything else on a record was
    # added by the caller through extra={...} and is copied into the output.
    # Computing the set (rather than typing it out) keeps it right when a new
    # Python version adds an attribute, as 3.12 did with taskName.
    _STANDARD_ATTRS = set(
        logging.LogRecord("", 0, "", 0, "", (), None).__dict__
    ) | {"message", "asctime"}

    def format(self, record: logging.LogRecord) -> str:
        ts = datetime.fromtimestamp(record.created, tz=timezone.utc)

        log_entry = {

```



```

        "timestamp": ts.strftime("%Y-%m-%dT%H:%M:%S.") + f"{ts.microsecond //
        ↪ 1000:03d}Z",
        "level": record.levelname,
        "logger": record.name,
        "message": record.getMessage(),
        "function": record.funcName,
        "line": record.lineno,
    }

    # Include exception information if present
    if record.exc_info:
        log_entry["exception"] = self.formatException(record.exc_info)

    # Include any extra fields attached to the record
    # These are added via: logger.info("msg", extra={"request_id": "abc"})
    for key, value in record.__dict__.items():
        if key not in self._STANDARD_ATTRS:
            log_entry[key] = value

    # Compact separators: one short line per record, easy to grep.
    # ensure_ascii=False keeps characters such as → readable.
    return json.dumps(log_entry, default=str, separators=(",", ":"),
    ↪ ensure_ascii=False)

# -----
# Main configuration function
# -----

def configure_logging() -> None:
    """
    Configure the logging system for the Notes API.

    Uses three settings from app.config:
        LOG_LEVEL - minimum severity to log (default: INFO)
        LOG_DIR - directory for log files (optional; if empty, no file logging)
        LOG_FORMAT - "human" for readable output, "json" for structured output
                     (default: "human"; set "json" in production)

    Safe to call more than once (each test creates a new app): the root
    logger's handlers are replaced, not added to.

    After calling this function:
        - The root logger is configured with the appropriate level
        - All log output goes to stderr (always)
    """

```

```

    - If LOG_DIR is set, log output also goes to a rotating file
    - Werkzeug's own request log is silenced (we log requests ourselves)
"""
log_level = _parse_log_level(LOG_LEVEL)
log_dir = LOG_DIR
log_format = LOG_FORMAT

# Choose the formatter based on LOG_FORMAT
formatter: logging.Formatter
if log_format == "json":
    formatter = JSONFormatter()
else:
    formatter = HumanReadableFormatter()

# -----
# Console handler – writes to stderr
# -----
# We write to stderr rather than stdout. This convention separates
# application logs (stderr) from application output (stdout), which
# matters when piping output in shell scripts.
console_handler = logging.StreamHandler(sys.stderr)
console_handler.setLevel(log_level)
console_handler.setFormatter(formatter)

# -----
# File handler – writes to a rotating log file (if LOG_DIR is set)
# -----
file_handler = None
if log_dir:
    os.makedirs(log_dir, exist_ok=True) # Create directory if it doesn't exist
    log_file = os.path.join(log_dir, "app.log")

    # RotatingFileHandler automatically manages log file size.
    # When the file reaches maxBytes, it is renamed to app.log.1,
    # the previous app.log.1 becomes app.log.2, etc., up to backupCount.
    # Old files beyond backupCount are deleted.
    #
    # maxBytes = 10 MB = 10 * 1024 * 1024 bytes
    # backupCount = 5 means we keep app.log + app.log.1 through app.log.5
    # Total storage: up to 60 MB of logs.
    file_handler = logging.handlers.RotatingFileHandler(
        filename=log_file,
        maxBytes=10 * 1024 * 1024, # 10 MB
        backupCount=5,

```

```

        encoding="utf-8",
    )
    file_handler.setLevel(log_level)
    file_handler.setFormatter(formatter)

# -----
# Configure the root logger
# -----
# The root logger is the ancestor of all loggers. Configuring it
# affects all loggers in the application and all imported libraries
# that use Python's logging module.
root_logger = logging.getLogger()
root_logger.setLevel(log_level)

# Remove any existing handlers (Python might add a default one)
root_logger.handlers.clear()

root_logger.addHandler(console_handler)
if file_handler:
    root_logger.addHandler(file_handler)

# -----
# Silence noisy third-party loggers
# -----
# Werkzeug (Flask's development server) logs every request at INFO in
# its own format. We raise its level because our middleware logs
# requests. This also hides the server's INFO-level startup lines
# ("Running on http://127.0.0.1:5000", "Press CTRL+C to quit");
# run.py logs its own "Starting Notes API" line instead.
logging.getLogger("werkzeug").setLevel(logging.WARNING)

# Log that logging is configured (this is the first structured log message)
logger = logging.getLogger(__name__)
logger.info(
    "Logging configured. level=%s format=%s log_dir=%s",
    logging.getLevelName(log_level),
    log_format,
    log_dir or "(console only)",
)

```

16.4 Request Logging Middleware

Every HTTP request and response should be logged. This gives you a complete record of every interaction the API had, which is essential for debugging, auditing, and capacity planning.

Chapter 15 left two of Chapter 13's hooks out of its listings: the JSON content-type check and the X-Response-Time-Ms timing header. They belong in the same module as request logging, because all three run around every request. Create `app/middleware.py`:

Listing 16.2 `app/middleware.py`

```

"""
Request middleware for the Notes API.

Hooks that run around every request: the JSON content-type check and the
timing header from Chapter 13, plus request logging, which records every
request when it arrives and every response when it departs.

Log output per request:
→ POST /notes/ (from 127.0.0.1)
← 201 in 4.2ms

And for errors:
→ DELETE /notes/999 (from 127.0.0.1)
← 404 in 1.1ms
"""
import logging
import time
from flask import Flask, g, jsonify, request

logger = logging.getLogger(__name__)

def register_middleware(app: Flask) -> None:
    """
    Register the request hooks on the Flask application.

    Call this from create_app() after creating the Flask instance.
    """

    @app.before_request
    def log_request_start():
        """
        Called by Flask before each request handler runs.

```

```

Records the start time on g so that after_request can compute
the elapsed time. Logs the incoming request.
"""
g.request_start_time = time.perf_counter()

# Log the incoming request.
# We include the remote address (client IP), method, and path.
# In production behind a reverse proxy, request.remote_addr will be
# the proxy's IP. To get the real client IP, you need the
# X-Forwarded-For header (Chapter 28).
logger.info(
    "→ %s %s (from %s)",
    request.method,
    request.path,
    request.remote_addr,
)

@app.before_request
def require_json_for_writes():
    """Reject writes whose body is not declared as JSON (Chapter 13)."""
    if request.url_rule is None:
        return # No route matched: let Flask answer 404 or 405
    if request.method in ("POST", "PUT", "PATCH") and not request.is_json:
        return jsonify({"error": "Content-Type must be application/json"}), 415

@app.after_request
def log_request_end(response):
    """
Called by Flask after each request handler returns a response.

Computes elapsed time, adds the timing header, and logs the response
status code. Must return the response object – if you forget this,
Flask breaks.
"""
    start = g.get("request_start_time", time.perf_counter())
    elapsed_ms = (time.perf_counter() - start) * 1000
    response.headers["X-Response-Time-Ms"] = f"{elapsed_ms:.1f}"

    # Choose the log level based on the response status code:
    # 2xx → INFO (success)
    # 4xx → WARNING (client error – not our fault, but worth noting)
    # 5xx → ERROR (server error – our fault)
    if response.status_code >= 500:
        log_fn = logger.error

```

```

elif response.status_code >= 400:
    log_fn = logger.warning
else:
    log_fn = logger.info

log_fn(
    "← %s in %.1fms",
    response.status_code,
    elapsed_ms,
)

return response # REQUIRED: Flask expects the response back

```

16.5 Error Handling: Catching Failures Before They Reach the Client

Logging tells you what happened. **Error handling** decides what *should* happen when something goes wrong — and that is as much a design discipline as a coding task. Before we register a single Flask handler, it is worth being deliberate about three questions that every piece of code you ever write has to answer: how should a failure be *signaled*, where should it be *handled*, and what do you do when it *cannot* be handled. Get these right and your error handling is a thin, predictable layer; get them wrong and errors leak to clients, hide in silence, or crash the process at 3 a.m. with no trace.

Three ways to signal a failure

When a function cannot do what its name promises, it has exactly three ways to say so, and choosing between them is a real design decision:

1. **Return a value that means “failure.”** The function returns a sentinel — `None`, `-1`, `False`, or a small result object like `{"ok": False, "error": ...}`. The caller must check it. This is explicit: the failure is right there in the return, impossible to miss if you read the signature. Its weaknesses are clutter (every call site grows an `if`) and the danger that a caller forgets to check and sails on with `None`.
2. **Raise an exception.** The function raises, and control jumps — past the caller, past *its* caller — until something catches it (in Python, `except`). This keeps the *happy path* clean: the body reads as if nothing can go wrong, because the error path is elsewhere. Its weakness is the mirror image: the control flow is invisible at the call site. You cannot tell, from `note = get_note(id)`, that it might explode.
3. **Let it crash.** Do nothing — let the exception propagate all the way out and take the pro-

cess down. Inside a request this is usually wrong. But at the *boundaries* of a system it is sometimes the honest choice: a misconfigured `DATABASE_URL` at startup *should* crash the process immediately (Chapter 15’s startup checks do exactly this — **failing fast**), because a server that starts in a broken state is worse than one that refuses to start.

Python’s culture leans on exceptions, and the Notes API follows suit: exceptions for the unexpected, ordinary return values for outcomes that are a normal part of the job. Which brings us to the distinction that decides which tool to reach for.

Expected vs. exceptional: the distinction that drives every choice

The single most useful question in error handling is: **is this failure a normal part of the operation, or a genuine surprise?**

- A `POST /notes/` with empty content is **expected**. Clients send bad input constantly; rejecting it is part of the endpoint’s *job*, not an emergency. So the route handler treats it as ordinary control flow — `return jsonify({"error": ...}), 400`, exactly as you saw in Chapter 15. No exception is raised, because nothing exceptional happened.
- A note that does not exist for `GET /notes/<id>` is also **expected** — `return ..., 404`. A normal outcome, handled with a normal return.
- The database connection dropping in the middle of a query is **exceptional**. There is nothing the route handler can sensibly do about it; repairing the database is not part of the endpoint’s job. So it is allowed to raise and propagate.

Get this distinction right and a clean separation falls out: **route handlers handle the expected errors locally with explicit returns, and let the exceptional ones fly past them.** The handler stays focused on its real work, because it is not littered with defensive `try/except` blocks guarding against things it could not fix anyway.

Where to handle an error: the handling boundary

If a function catches an exception it cannot do anything useful about, it has made things *worse* — it has hidden the problem without solving it. The rule is simple: **handle an error only where you can actually do something about it.** Everywhere else, let it propagate.

For most exceptions in a web application, the place that can “do something” is not the route handler at all — it is a single, well-defined **handling boundary** at the very edge of request processing. That boundary has exactly the context needed for the one decision that *can* be made about an unexpected failure: log it in full, and turn it into a safe HTTP 500 so the client gets a clean response and the process keeps serving everyone else.

That is precisely what `app/errors.py`, below, is — the Notes API’s handling boundary, made concrete. Its `@app.errorhandler(Exception)` catch-all is the one place in the whole application that catches the truly unexpected. *Because it exists*, no other code needs a bare `try/except` to keep the server alive — every route handler and database function can let exceptions propagate, trusting the boundary to catch them. One place to log, one place to convert, one consistent error shape for clients.

Warning: A handling boundary is a safety net, not a license to wrap your own bugs in a smile. It catches *unexpected* failures so one bad request cannot kill the process — it does not excuse code that should have validated its input or checked its preconditions in the first place.

When you cannot handle an error meaningfully

Sooner or later you will write a `try` and stare at the `except`, unsure what to put in it. The answer is almost never this:

```
try:
    do_something()
except Exception:
    pass          # <- often the single most expensive line in a codebase
```

Swallowing an exception silently is how a system acquires **unknown unknowns** (Chapter 15): the failure happened, nobody was told, and the symptom surfaces hours later somewhere unrelated. If you cannot handle an error meaningfully, you have three honest options, in order of preference:

1. **Don’t catch it.** Let it propagate to the handling boundary, which will log and convert it. This is the default, and usually the right one.
2. **Catch, add context, and re-raise.** If you can make the error *more useful* — attach which note ID failed, which external service timed out — do that, then re-raise so the boundary still sees it. Python’s `raise NewError(...) from err` preserves the original cause (exception chaining), so you keep the full story.
3. **Catch and degrade deliberately.** If, and only if, there is a sensible fallback — serve a cached value, skip an optional enrichment — catch the *specific* exception (never bare `Exception`), log that you are degrading, and continue. That is graceful degradation (→ Chapter 37), and it is a *decision*, not an accident.

What unifies all three is that the error is never silently lost. **Catch narrowly, or not at all.** A bare `except Exception: pass` is the code equivalent of unplugging the smoke detector

because the beeping is annoying.

Designing errors out of existence

The best error handling is the error you never have to handle. A surprising amount of error-handling code exists only because the interface *invited* the error, and a small design change would make it impossible.

Consider `DELETE /notes/<id>` for a note that is already gone. You can treat it as an error (404) — or you can decide that “delete” means “ensure this note does not exist,” in which case deleting something already absent is simply *success* (204), and an entire error path disappears. That is **idempotency** (→ [Chapter 37](#)), and also an instance of a deeper principle — *defining errors out of existence*: shape your interfaces so the common case is not an error at all. Every error you design away is error-handling code that nobody has to write, read, or get wrong. (The Notes API keeps the 404: a client deleting a note it never saw probably has a bug worth reporting. Either choice is defensible; what matters is that it is deliberate and documented.)

This is why error handling belongs to *design*, not just coding. A function’s failures are part of its **contract**, as much as its parameters and return type. When you write a function, decide deliberately: what does it promise, how does it signal a broken promise, and have you made the easy cases impossible to break? Document the exceptions a function can raise the same way you document what it returns — the reader is always asking “what can go wrong here?”, and a well-designed error contract answers it for them.

With those principles in hand, the implementation is almost mechanical. Flask gives us **error handlers**: functions that are called when a specific HTTP error or exception is *raised* — by Flask itself, by `abort()`, or by a bug. (A route handler that *returns* an error response, such as `return jsonify(...)`, 404, does not pass through them.) Instead of letting Flask return its default HTML error pages, we register handlers that return JSON — consistent with the rest of the API, and structured so that *expected* errors are reported cleanly while *unexpected* ones are logged in full and reduced to a safe, generic response.

Create `app/errors.py`:

Listing 16.3 `app/errors.py`

```
"""
Error handlers for the Notes API.

Flask calls these functions when an HTTP error or an exception is raised
during request handling. Each handler receives the error and returns a
JSON response.
```

Without these handlers:

- HTTP 404 returns Flask's default HTML error page
- Unhandled exceptions return a generic 500 HTML page
- The client gets HTML when it expects JSON

With these handlers:

- Every error has the same shape: {"error": "description"}
- Unexpected exceptions are logged once, with their full traceback
- The client always gets JSON, whatever went wrong

The request-logging middleware already records each response's status code, so these handlers log only what it cannot see: the traceback of an unexpected exception.

"""

```
import logging
from flask import Flask, jsonify, request
from werkzeug.exceptions import HTTPException
```

```
logger = logging.getLogger(__name__)
```

```
def register_error_handlers(app: Flask) -> None:
```

"""

Register error handlers on the Flask application.

Call this from create_app() after creating the Flask instance.

"""

```
@app.errorhandler(404)
```

```
def not_found(error):
```

"""

HTTP 404 Not Found – no route matches the URL.

Flask raises this automatically, including when a URL converter fails: /notes/abc does not match /notes/<int:note_id>.

"""

```
return jsonify({"error": "Not found"}), 404
```

```
@app.errorhandler(405)
```

```
def method_not_allowed(error):
```

"""

HTTP 405 Method Not Allowed – the route exists, but not for this method.

Example: sending DELETE to /health, which only accepts GET.

HTTP requires a 405 to carry an Allow header listing the methods

```

        that do work; a new response must add it back.
        """
        allow = ", ".join(sorted(error.valid_methods or []))
        return jsonify({"error": "Method not allowed"}), 405, {"Allow": allow}

@app.errorhandler(429)
def too_many_requests(error):
    """
    HTTP 429 Too Many Requests – rate limit exceeded.
    Relevant after rate limiting is added (Chapter 38).
    """
    return jsonify({"error": "Too many requests, please slow down"}), 429

@app.errorhandler(HTTPException)
def handle_http_exception(error):
    """
    Any other HTTP error from Flask, Werkzeug, or abort(): 400 for a
    malformed request, 413 for a body that is too large, and so on.

    error.description is Werkzeug's one-sentence explanation of the error.
    """
    return jsonify({"error": error.description}), error.code

@app.errorhandler(Exception)
def handle_unexpected_exception(error):
    """
    The handling boundary: any exception that is not an HTTP error.

    Flask chooses a handler by the exception's class, so an unexpected
    exception (a dropped database connection, a bug) arrives here. That is
    why this file has no @app.errorhandler(500): for these errors it would
    never run, because this handler matches first.

    We ALWAYS log the full traceback here. Without this, the error
    silently becomes a 500 response and the traceback is lost
    (especially in production where there is no terminal to watch).
    """
    # logger.exception() logs at ERROR level and adds the traceback
    logger.exception("Unhandled exception in %s %s", request.method,
        ↪ request.path)
    return jsonify({
        "error": "An unexpected error occurred",
        # Do NOT include the exception message or traceback in the response.
        # Internal error details can reveal security-sensitive information
    })

```

```
    # (database schemas, file paths, internal implementation details).
  }, 500
```

Critical security note on error responses:

The `handle_unexpected_exception` handler deliberately does not include the exception message in the JSON response. Here is why:

If your database query fails with:

```
psycpg2.ProgrammingError: column "password_hash" of relation "users" does not exist
```

Returning this to the client reveals:

1. You are using PostgreSQL
2. You have a `users` table
3. You have a `password_hash` column (or tried to)

This information helps an attacker understand your database schema. The log entry contains the full exception — developers can find it there. The client response contains only the generic message.

Step back and notice the shape of what you just built. The per-status handlers (404, 405, 429) report the *expected* errors cleanly. The two catch-alls — `@app.errorhandler(HTTPException)` and `@app.errorhandler(Exception)` — are the **handling boundary** from the start of this section, made real: the single place the whole application trusts to catch the unexpected, log it in full, and reduce it to one safe response shape. That boundary is *why* every route handler and database function elsewhere in the Notes API can let exceptions propagate instead of smothering them in defensive `try/except` blocks. The discipline and the implementation are the same idea seen from two sides.

16.6 Integrating Everything into the Application Factory

Update `app/__init__.py` to use the new logging, middleware, and error handling:

Listing 16.4 app/__init__.py (updated)

```

"""
Application factory for the Notes API.
"""

import logging
from flask import Flask
from app import startup
from app.config import SECRET_KEY, FLASK_DEBUG
from app.database import close_db, init_db
from app.errors import register_error_handlers
from app.logging_config import configure_logging
from app.middleware import register_middleware

logger = logging.getLogger(__name__)

def create_app() -> Flask:
    """
    Create and configure a Flask application instance.
    """
    # Configure logging first, so the startup checks and everything
    # after them report through it
    configure_logging()
    startup.run()

    logger.info("Creating Flask application...")
    app = Flask(__name__)

    # Apply configuration
    app.config["SECRET_KEY"] = SECRET_KEY
    app.config["DEBUG"] = FLASK_DEBUG

    # Register database teardown
    app.teardown_appcontext(close_db)

    # Register the request hooks and the error handlers
    register_middleware(app)
    register_error_handlers(app)

    # Register Blueprints
    from app.routes.health import health_bp
    from app.routes.notes import notes_bp
    app.register_blueprint(health_bp)
    app.register_blueprint(notes_bp)

```

```
# Initialize database schema
init_db(app)

logger.info("Flask application created successfully.")
return app
```

The new modules can be imported at the top of the file: unlike the route modules, they do not import anything from the `app` package that could import them back.

With logging in place, the `print()` calls left over from Chapters 14 and 15 should go too. In `app/startup.py`, the status messages become log calls, and `_fail()` logs at CRITICAL before it exits:

```
# app/startup.py (changes)
import logging

logger = logging.getLogger(__name__)

# In check_database(), replace the print() with:
    logger.info("Database connectivity: OK")

def run():
    """Run the checks that apply wherever the application starts."""
    logger.info("Running startup checks...")
    check_python_version()
    check_secret_key()
    check_database()
    logger.info("All startup checks passed.")

def _fail(message: str) -> None:
    """Log the error and exit with code 1."""
    logger.critical("Startup failed: %s", message)
    sys.exit(1)
```

In the same way, `init_db()` in `app/database.py` ends with `logger.info("Database initialized.")` (with `logger = logging.getLogger(__name__)` at the top of that module), and `run.py` announces the server through a logger named "run":

```
# run.py (changes)
import logging
...
if __name__ == "__main__":
    check_port(HOST, PORT)
    logging.getLogger("run").info("Starting Notes API on http://%s:%d", HOST, PORT)
    app.run(host=HOST, port=PORT, debug=FLASK_DEBUG)
```

One message still bypasses logging on purpose: `_require()` in `app/config.py`, which reports a missing required variable. It runs when the package is first imported, before logging exists — and logging’s own settings come from that same configuration — so it keeps writing straight to `stderr`.

16.7 Adding Logging to Route Handlers

With the logging system in place, add logging to the route handlers. Each handler now has a logger that records what it is doing.

Here is the updated `app/routes/notes.py` with logging integrated. The small `_invalid()` helper gives every validation failure the same warning and the same response, so no branch can forget to log:

Listing 16.5 `app/routes/notes.py` (with logging)

```
import logging
from flask import Blueprint, request, jsonify
from app.database import get_db, sql, row_to_note

logger = logging.getLogger(__name__)
# This logger's name will be "app.routes.notes" because __name__ in this
# module is "app.routes.notes". When this logger emits a message, it
# travels up to the root logger and is handled by our configured handlers.

notes_bp = Blueprint("notes", __name__, url_prefix="/notes")

def _invalid(message: str):
    """Log a rejected request and build its 400 response."""
    # Log the reason, never the note's content (it may be private)
    logger.warning("POST /notes/ rejected: %s", message)
    return jsonify({"error": message}), 400

@notes_bp.route("/", methods=["GET"])
```

```

def list_notes():
    cursor = get_db().cursor()
    cursor.execute("SELECT id, content, created_at FROM notes ORDER BY id DESC")
    notes = [row_to_note(row) for row in cursor.fetchall()]
    logger.debug("Returning %d notes.", len(notes))
    return jsonify({"notes": notes, "count": len(notes)}), 200

@notes_bp.route("/", methods=["POST"])
def create_note():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return _invalid("Request body must be a JSON object")

    content = data.get("content")
    if content is None:
        return _invalid("'content' is required")
    if not isinstance(content, str):
        return _invalid("'content' must be a string")

    content = content.strip()
    if not content:
        return _invalid("'content' cannot be empty")
    if len(content) > 10_000:
        return _invalid("'content' cannot exceed 10,000 characters")

    # No try/except here: if the database fails, the exception propagates
    # to the handling boundary in errors.py, which logs it once, in full.
    db = get_db()
    cursor = db.cursor()
    cursor.execute(
        sql("INSERT INTO notes (content) VALUES (?) RETURNING id, content,
    ↪ created_at"),
        (content,),
    )
    note = row_to_note(cursor.fetchone())
    db.commit()
    logger.info("Created note id=%d.", note["id"])
    return jsonify(note), 201

@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id):
    cursor = get_db().cursor()
    cursor.execute(sql("SELECT id, content, created_at FROM notes WHERE id = ?"),
    ↪ (note_id,))

```



```

row = cursor.fetchone()
if row is None:
    logger.info("Note id=%d not found.", note_id)
    return jsonify({"error": f"Note {note_id} not found"}), 404
return jsonify(row_to_note(row)), 200

@notes_bp.route("/<int:note_id>", methods=["DELETE"])
def delete_note(note_id):
    db = get_db()
    cursor = db.cursor()
    cursor.execute(sql("DELETE FROM notes WHERE id = ?"), (note_id,))
    db.commit()
    if cursor.rowcount == 0:
        logger.info("DELETE note id=%d: not found.", note_id)
        return jsonify({"error": f"Note {note_id} not found"}), 404
    logger.info("Deleted note id=%d.", note_id)
    return "", 204

```

Notice what `create_note` does *not* have: a `try/except` around the database work. An earlier draft of this handler caught the exception, logged it, and re-raised it — which logs every database failure twice, once here and once at the handling boundary, and adds no information. The boundary already knows the method and path; the traceback shows the handler.

16.8 What the Log Output Looks Like

With everything configured, here is what the log output looks like for a sequence of requests:

Development (human-readable format)

```

2024-01-15 14:32:00.001 INFO      app.logging_config | Logging configured. level=INFO
↳ format=human log_dir=(console only)
2024-01-15 14:32:00.002 INFO      app.startup         | Running startup checks...
2024-01-15 14:32:00.003 INFO      app.startup         | All startup checks passed.
2024-01-15 14:32:00.004 INFO      app                 | Creating Flask application...
2024-01-15 14:32:00.045 INFO      app.database        | Database initialized.
2024-01-15 14:32:00.046 INFO      app                 | Flask application created
↳ successfully.
2024-01-15 14:32:00.047 INFO      run                 | Starting Notes API on
↳ http://127.0.0.1:5000

2024-01-15 14:32:15.100 INFO      app.middleware      | → POST /notes/ (from 127.0.0.1)

```

```

2024-01-15 14:32:15.103 INFO      app.routes.notes | Created note id=1.
2024-01-15 14:32:15.104 INFO      app.middleware   | ← 201 in 4.1ms

2024-01-15 14:32:20.200 INFO      app.middleware   | → GET /notes/ (from 127.0.0.1)
2024-01-15 14:32:20.202 INFO      app.middleware   | ← 200 in 1.8ms

2024-01-15 14:32:25.300 INFO      app.middleware   | → GET /notes/999 (from
↪ 127.0.0.1)
2024-01-15 14:32:25.301 INFO      app.routes.notes | Note id=999 not found.
2024-01-15 14:32:25.301 WARNING   app.middleware   | ← 404 in 0.9ms

2024-01-15 14:32:30.400 INFO      app.middleware   | → POST /notes/ (from 127.0.0.1)
2024-01-15 14:32:30.401 WARNING   app.routes.notes | POST /notes/ rejected:
↪ 'content' is required
2024-01-15 14:32:30.401 WARNING   app.middleware   | ← 400 in 0.8ms

```

With SQLite there is no “Database connectivity: OK” line: the database check has nothing to connect to.

This output tells the complete story of each request: when it arrived, what happened, what the response was, and how long it took.

Production (JSON format, with LOG_FORMAT=json)

```

{"timestamp":"2024-01-15T14:32:00.001Z","level":"INFO","logger":"app.logging_config",
↪ "message":"Logging configured. level=INFO format=json
↪ log_dir=/var/log/notes-api","function":"configure_logging","line":233}
{"timestamp":"2024-01-15T14:32:15.100Z","level":"INFO","logger":"app.middleware","mes
↪ sage":"→ POST /notes/ (from 10.0.0.5)","function":"log_request_start","line":44}
{"timestamp":"2024-01-15T14:32:15.103Z","level":"INFO","logger":"app.routes.notes","m
↪ essage":"Created note id=1.","function":"create_note","line":54}
{"timestamp":"2024-01-15T14:32:15.104Z","level":"INFO","logger":"app.middleware","mes
↪ sage":"← 201 in 4.1ms","function":"log_request_end","line":83}

```

Each line is a single JSON object. A log aggregation tool (like Elasticsearch/Kibana, Datadog, or AWS CloudWatch) parses these lines and makes them searchable:

- “Show me all ERROR messages in the past hour”
- “Show me all requests to /notes/ that took longer than 100ms”
- “How many 400 responses did we return today?”

JSON logs enable these queries. Human-readable logs do not.

16.9 Logging Best Practices

What to log (and what not to log)

Log these:

- Every request arrival and departure (done by middleware)
- Every resource creation or deletion (Created note id=42, Deleted note id=7)
- Every validation failure with the reason ('content' field missing)
- Every external service call (database, email, third-party API)
- Every configuration decision made at startup
- Every error, with its full traceback

Do not log these:

- Passwords, tokens, secret keys, or any credential
- Full request bodies that might contain sensitive data (log the structure, not the content)
- Personal information (email addresses, names, addresses)
- Credit card numbers, social security numbers, or any PII
- High-frequency data that would fill your log files (e.g., do not log inside a loop that runs 10,000 times per request)

Using the correct log level

```
# DEBUG – visible only when LOG_LEVEL=DEBUG; for detailed tracing
logger.debug("Fetching note id=%d from database.", note_id)
logger.debug("SQL query: %s", query)

# INFO – normal operational events; things that should happen
logger.info("Note id=%d created successfully.", note_id)
logger.info("Server starting on %s:%d", host, port)

# WARNING – something unexpected, but the operation succeeded
logger.warning("POST /notes/ – content exceeds recommended length (%d chars).",
    ↪ length)
logger.warning("Database connection took %.1fs – pool may be exhausted.", elapsed)

# ERROR – an operation failed; intervention may be needed
logger.error("Failed to insert note into database.", exc_info=True)
logger.error("Database connection failed: %s", error)

# CRITICAL – the application cannot continue
logger.critical("Cannot bind to port %d – exiting.", port)
```

String formatting in log calls

```
# Wrong – string concatenation evaluates the format even if the message is filtered
logger.debug("Processing note: " + str(note_data))

# Wrong – f-string evaluates the format even if the message is filtered
logger.debug(f"Processing note: {note_data}")

# Correct – pass the format string and arguments separately
# The logging module only calls str(note_data) if DEBUG messages are enabled
logger.debug("Processing note: %s", note_data)
```

The `%s` format is evaluated lazily. If the logger's level is `INFO`, `DEBUG` messages are filtered before formatting occurs — avoiding unnecessary string operations. With `f`-strings, the formatting always occurs, even when the message is discarded. For high-frequency log calls, this can be a meaningful performance difference.

16.10 Debugging: From a Failure to Its Cause

Logs tell you *that* something went wrong, and roughly where. **Debugging** is the work of getting from there to *why*. Experienced engineers are not faster because they guess better. They are faster because they follow a method, and they stop guessing sooner.

A method, not a mood

1. **Reproduce it.** Make the failure happen on demand: one `curl`, one test, one input. A bug you cannot trigger is a bug you cannot confirm you have fixed. If it happens only in production, find what differs: the data, the configuration, the load, or the database (Chapter 39 is a whole chapter about those differences).
2. **Read the evidence before touching the code.** Read the traceback, the log lines around it, and the exact input. Most bugs announce their cause, if you read to the end.
3. **Form one hypothesis, and test it.** “The note is missing, and nothing checks for that.” Then check exactly that: add one log line, one assertion, or one breakpoint. Change one thing at a time, or you will not know which change mattered.
4. **Narrow it down.** When the evidence does not point anywhere, halve the search space: which half of the code, the input, or the history contains the bug? Repeat.
5. **Fix it, and prove it with a test.** Write the test that fails *before* the fix (Chapter 17, section 17.8), so the bug cannot come back unnoticed.

Reading a traceback

Suppose someone “simplified” `get_note` and removed the check for a missing note. A request for a note that does not exist now gets a 500 instead of a 404, and the catch-all handler logs this:

```

2024-01-15 14:00:05.101 ERROR      app.errors          | Unhandled exception in GET
↳ /notes/99999
Traceback (most recent call last):
  File ".../site-packages/flask/app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
  File ".../site-packages/flask/app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args)
  File "/home/you/notes-api/app/routes/notes.py", line 73, in get_note
    return jsonify(row_to_note(row)), 200
           ^^^^^^^^^^^^^^^^^
  File "/home/you/notes-api/app/database.py", line 84, in row_to_note
    note = dict(row)
TypeError: 'NoneType' object is not iterable

```

Read a Python traceback **from the bottom up**:

- **The last line is what happened:** a `TypeError`, because something tried to iterate over `None`.
- **The frame just above it is where it happened:** `dict(row)` in `row_to_note`, so `row` was `None`.
- **Walk up the frames to find your own code and the caller that passed the bad value:** `get_note` called `row_to_note(row)` with a `row` that `fetchone()` had returned as `None`.
- **Skip the framework's frames** (Flask's `app.py`) unless the evidence leads there. They are almost always innocent. The bug is nearly always in the first frame that belongs to *you*.

The `^^^^` markers (Python 3.11 and later) point at the exact expression within the line. The cause is now plain: nothing handles “no such note” any more. The fix is the missing `if row is None: return ... 404`, and the proof is the test `test_get_nonexistent_note_returns_404`, which had in fact been failing since that commit.

Stopping inside the code: `breakpoint()`

When reading is not enough, stop the program at the interesting moment and look around. Put `breakpoint()` on the line before the suspect one. When execution reaches it, Python opens its debugger, `pdb`, in your terminal:

```

row = cursor.fetchone()
breakpoint() # execution stops here; remove before committing
return jsonify(row_to_note(row)), 200

```

Command	What it does
<code>p row</code>	print a value (<code>pp</code> pretty-prints dictionaries and lists)
<code>n</code>	run the next line, stepping <i>over</i> function calls
<code>s</code>	step <i>into</i> the function called on this line
<code>c</code>	continue until the next breakpoint
<code>w</code>	show where you are: the call stack
<code>u / d</code>	move up or down the stack, to inspect a caller's variables
<code>l</code>	list the source around the current line
<code>q</code>	quit

Debuggers need a terminal attached to the process. So use them where you have one: with the development server (`python run.py`), or in a test. `pytest --pdb` opens the debugger right at the failing line of any failing test, with all of its variables in reach. That is often the fastest route of all. Setting the environment variable `PYTHONBREAKPOINT=0` makes every `breakpoint()` do nothing, which is a safety net for any that slip through. But the real rule is simpler: never commit one. `ruff` flags them (rule `T100`, if you enable it).

Flask's debug mode (`FLASK_DEBUG=true`) shows a similar interactive debugger *in the browser* whenever a request fails. It is convenient on your laptop, and a remote code-execution hole anywhere else, because anyone who can trigger an error can run Python on your server. That is why `FLASK_DEBUG` must be `false` in production, and why the startup checks of [Chapter 38](#) warn when it is not.

Narrowing down in history: `git bisect`

Sometimes the question is not “why does this fail?” but “it worked last week; what changed?” With dozens of commits since the last good version, reading them all is slow. **`git bisect`** does a binary search through history. It checks out the commit halfway between a known-good and a known-bad version, asks whether that one is good or bad, and halves the range again, until one commit is left. And if a test can answer “good or bad”, Git can run the whole search by itself:

```
git bisect start HEAD v1.3.0      # bad: now; good: the last release
git bisect run pytest -x -q tests/test_notes.py
git bisect reset                  # back to where you were
```

```

Bisecting: 4 revisions left to test after this (roughly 2 steps)
running 'pytest' '-x' '-q' 'tests/test_notes.py'
Bisecting: 2 revisions left to test after this (roughly 1 step)
running 'pytest' '-x' '-q' 'tests/test_notes.py'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
running 'pytest' '-x' '-q' 'tests/test_notes.py'
639606a35b076e00419a248aea2788b22f57c337 is the first bad commit
commit 639606a35b076e00419a248aea2788b22f57c337
Author: Dev <dev@example.com>
Date:   Mon Jan 15 14:00:05 2024 +0000

```

```

    Simplify get_note

app/routes/notes.py | 3 ---
bisect found first bad commit

```

Three test runs searched nine commits and found the one that deleted three lines from `get_note`. The search takes about $\log_2(n)$ steps, so a thousand commits need only about ten. This is where small, single-purpose commits pay off (Chapter 17, section 17.9): the commit bisect finds *is* the explanation. A thousand-line commit, by contrast, only tells you the bug is somewhere in those thousand lines.

When it happens only in production

Production is the one place you cannot freely stop, step through or experiment. So the order of work changes:

1. **Logs and metrics first.** Find the failing request in the logs. From Chapter 35 on, its request ID ties together every line it produced. Check whether the error rate changed after a deploy.
2. **Reproduce it locally**, with the same input and, where you can, the same kind of data: a copy of the relevant rows (with personal data removed), and the same database engine.
3. **Look inside the running container only to read, never to fix.** `docker logs notes-api` (Chapter 18) and `docker exec -it notes-api python` (a Python shell inside the running container, with its configuration) are for *looking*. A fix made by hand on a server is lost at the next deploy, and nobody reviews it. The fix goes through a commit, a test, CI and a deploy, like every other change.

And when the failure is not *wrong* but *slow*, the tool is a profiler, which Chapter 36 (section 36.2) introduces.

16.11 The Updated .env.example

Add the new logging-related environment variables to .env.example:

```
# .env.example – environment variable template for the Notes API
# Copy to .env and fill in real values before running.
# Run: python run.py

# Database connection
DATABASE_URL=sqlite:///notes.db

# Required – generate a value with:
# python -c "import secrets; print(secrets.token_hex(32))"
SECRET_KEY=

# Development server settings
FLASK_DEBUG=false
PORT=5000
HOST=127.0.0.1

# Logging settings
# LOG_LEVEL: DEBUG, INFO, WARNING, ERROR, CRITICAL (default: INFO)
LOG_LEVEL=INFO
# LOG_FORMAT: human (readable) or json (structured) (default: human)
LOG_FORMAT=human
# LOG_DIR: directory for log files (optional; if empty, logs go to stderr only)
LOG_DIR=
```

The Updated Project Structure

```

notes-api/
├── app/
│   ├── __init__.py      ← create_app() factory (updated)
│   ├── config.py        ← Configuration loading (LOG_LEVEL, LOG_FORMAT added)
│   ├── database.py      ← Database connection management (logs)
│   ├── errors.py        ← Error handlers (NEW)
│   ├── logging_config.py ← Logging setup (NEW)
│   ├── middleware.py    ← Request hooks and request logging (NEW)
│   ├── startup.py       ← Startup checks (log instead of print)
│   └── routes/
│       ├── __init__.py
│       ├── health.py
│       └── notes.py      ← Updated with logging
├── run.py               ← Logs its startup line
├── requirements.txt
├── .env
├── .env.example         ← Updated with LOG_* variables
├── .python-version
└── .gitignore

```

Key Takeaways

- **print() is not logging.** It has no severity levels, no routing, no file rotation, no structure, and no thread safety. Python's logging module provides all of these.
- **The four components of Python logging** are: loggers (what your code writes to), handlers (where messages go), formatters (what each line looks like), and levels (which messages are important enough to record).
- **Logger hierarchy:** logger names like "app.routes.notes" form a tree. A message that passes the originating logger's level check is handed to the handlers of that logger and of every ancestor up to the root.
- **Two formatter styles:** human-readable (for development, easy to read) and JSON (for production, machine-parseable by log aggregation tools).
- **RotatingFileHandler** prevents log files from filling the disk by automatically rotating them when they reach a size limit.
- **Error handlers** in Flask intercept raised exceptions and HTTP errors, returning JSON responses in one consistent shape. Without them, Flask returns HTML, which API clients cannot parse.
- **Never include internal error details in API responses.** Log the full traceback, but return only a generic message to the client.

- **Expected vs. exceptional is the core distinction.** Bad input and missing resources are a normal part of an endpoint's job — handle them with explicit return values (400, 404). Genuine surprises (a dropped connection) are exceptions — let them propagate.
- **Handle errors only where you can do something about them,** and concentrate the unexpected ones at a single **handling boundary** (the catch-all error handler). Everywhere else, let them propagate — except `Exception: pass` is how systems acquire silent, invisible failures.
- **Catch narrowly, or not at all.** If you must catch, add context and re-raise (`raise ... from err`) or degrade deliberately on a *specific* exception — never swallow.
- **Design errors out of existence.** A function's failures are part of its contract; shaping interfaces so the common case is not an error (e.g. an idempotent delete) removes error-handling code entirely.
- **Log at the right level.** DEBUG for tracing, INFO for normal events, WARNING for unexpected-but-recoverable situations, ERROR for failures, CRITICAL for process-ending problems.
- **Use %s format arguments** (not f-strings) in log calls, so formatting is skipped when the message is filtered by level.
- **Debug with a method:** reproduce the failure on demand, read the evidence (tracebacks bottom-up; your own first frame is usually the culprit), test one hypothesis at a time, halve the search space (`git bisect run` automates it through history), and prove the fix with a test. Use `breakpoint()` and `pytest --pdb` locally, never in committed code; in production, read logs and reproduce locally instead of fixing on the server.

Exercises

1. **Predict.** Set `LOG_LEVEL=WARNING` and create a note. Which log lines disappear? Now set `LOG_FORMAT=json`, and pipe the log output through `jq` with `python run.py 2>&1 >/dev/null | jq -r .message`.
2. **Break and fix.** Make a handler catch every exception and return `{"error": "oops"}` with a 200. What does the monitoring see, and what do clients see? Remove the catch.
3. **Extend.** Add the request's method and path to every log line produced during a request, not only to the middleware's lines.
4. **Explain.** Why does the Notes API handle unexpected exceptions in one place, the handling boundary, instead of in each handler?

Solutions and hints are in the companion repository, in `exercises/chapter-16.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

The application now has structured logging and comprehensive error handling. The next question is: how do you reliably produce the artifact — the built, tested, deployable package — that represents the application at a specific point in time? Building is not just running `pip install`. It is a defined, repeatable process that produces a specific output.

→ [Chapter 17](#) — Build Process

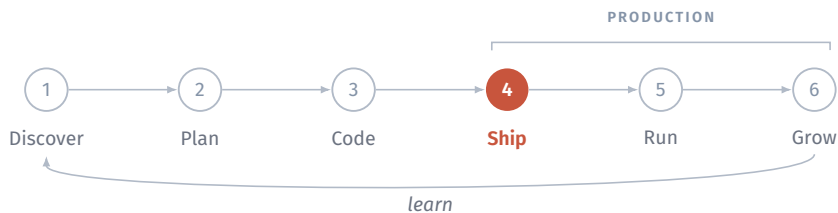
STAGE 4

4

Build and Packaging

How is that portable system packaged?

YOU ARE HERE



The piece: The workshop: automatic checks and the package store

The whole map: *How Software Gets Built*, page 1

Build Process

Imagine you have finished a feature. The code works on your machine. You have tested it manually. Now the question is: how does it get to production?

The naive answer is: copy the files. But “copy the files” glosses over a chain of steps that must happen first: installing dependencies, running tests to verify the code is correct, checking code style, verifying the runtime contract, generating any derived files (compiled assets, documentation, version strings), and packaging the result into a form that the deployment target expects.

A **build process** is the defined, automated sequence of steps that transforms source code into a deployable artifact. “Automated” is the critical word. If the build process requires a human to manually execute ten commands in the correct order, the process will eventually be done incorrectly. Someone will forget a step. Someone will run the steps in the wrong order. Someone will skip the tests because “it’s a small change.”

An automated build process is a script — or a sequence of scripts — that runs all steps in the correct order, fails immediately if any step fails, and produces a known-good artifact at the end or nothing at all. If the tests fail, the build stops. If the dependency installation fails, the build stops. The artifact only exists if every step succeeded.

This chapter builds the automated build process for the Notes API and introduces the testing foundation that makes the build process meaningful.

In this chapter

By the end of this chapter, you will understand:

- What a build artifact is for a Python web application
- What steps belong in a build process and in what order
- How to write automated tests with `pytest` and why tests are a non-negotiable part of the build
- What to test at which level, and how to use parametrized tests, test doubles and factory fixtures to keep a suite fast and trustworthy
- Why tests fail at random, and how to fix them; and how writing the test first fixes bugs for good
- How a change travels through Git: branches, commits, pull requests, code review, and a protected `main`
- How to use `Makefile` targets to define and run the build sequence
- What code quality tools do and how to integrate them
- How the build process connects to continuous integration (CI)

17.1 What a “Build” Means for a Python Web Application

In languages like Java or C++, “building” has a clear meaning: the compiler transforms source code into machine code. The output is a binary file (for Java, `.class` files packaged into a `.jar`; for C++, an executable). You cannot run the source code directly — you must build it first.

Python is different. You can run `python app.py` directly from source: the interpreter compiles each module to bytecode on the fly (Chapter 2), so there is no separate compile step that produces an artifact you ship.

So what does a “build” mean in the Python context?

The Python build artifact

For a Python web application deployed to a server or in a container, the “build artifact” is typically one of:

Option A: The source code itself, with a frozen dependency manifest. The server receives the source files and a `requirements.txt`, installs dependencies, and runs the application. The “artifact” is the source code at a specific commit, pinned with its dependencies.

Option B: A Docker image. The build process runs inside a Docker container, installs everything, and produces a self-contained image. The image is the artifact. (Chapter 18 covers this in depth.)

Option C: A wheel file (.whl). For libraries distributed to other developers, a wheel is the standard Python artifact. For web applications deployed to servers, it is less common but possible.

For the Notes API, the build produces **Option A**: a verified source tree with pinned dependencies, plus a test report confirming that the code works. [Chapter 18](#) wraps this in a Docker image.

The steps in the build

A complete build for a Python web application includes:

1. **Clean**: remove previous build artifacts and cached files
2. **Install dependencies**: install the exact versions from `requirements.txt`
3. **Lint**: check code style and common mistakes
4. **Type check**: verify that function signatures and type annotations are consistent
5. **Test**: run automated tests; fail the build if any test fails
6. **Security scan**: check dependencies for known vulnerabilities
7. **Generate version info**: record the git commit hash or version number
8. **Package** (for Docker builds): produce the container image

Steps 1–6 are covered in this chapter: the `Makefile` runs steps 3–6 as `make build`, and steps 1 and 2 as `make clean` and `make install`. Step 7 happens when the image is tagged with the commit ([Chapter 19](#)), and step 8 is in [Chapter 18](#).

17.2 Setting Up the Test Infrastructure

Tests are the core of the build process. Without tests, you cannot automate confidence in the code. With tests, every build verifies that the application behaves correctly.

Installing pytest

`pytest` is the de facto standard testing framework for Python. Install it in development:

```
pip install pytest
pip freeze > requirements.txt
```

Wait — `requirements.txt` is for production dependencies. You should not install `pytest` in production. (If you already ran those two commands, take `pytest` and its dependencies back out of `requirements.txt`.) The solution is two `requirements` files:

```

notes-api/
├── requirements.txt      ← Production dependencies (what the server needs)
└── requirements-dev.txt ← Development dependencies (what developers need)

```

```

# requirements-dev.txt
# Development and testing dependencies
# Install with: pip install -r requirements-dev.txt
#
# These packages are NOT installed on production servers.

# Everything the application itself needs (Chapter 12's convention)
-r requirements.txt

# Testing
pytest==8.1.1
pytest-cov==5.0.0      # Test coverage measurement (section 17.7)

# Code quality
black==24.3.0          # Code formatter
ruff==0.3.5            # Fast Python linter
mypy==1.9.0            # Static type checker

# Security scanning
pip-audit==2.7.3       # Checks for known vulnerabilities in dependencies

```

```

# Install everything for development (the -r line pulls in requirements.txt):
pip install -r requirements-dev.txt

```

From now on, `requirements.txt` is edited by hand: add the one package you need, with its pinned version. Running `pip freeze > requirements.txt` in this environment would write every development tool into the production list.

Update `.gitignore` to include the pytest cache:

```

# .gitignore additions
.pytest_cache/
.mypy_cache/
.ruff_cache/

```

Flask's test client

Flask has a built-in **test client**, `app.test_client()`: an HTTP client that sends requests straight to your Flask application without starting a real server. This is essential: tests run fast (no network round-trips) and do not require an open port. The `client` fixture in `conftest.py` below hands one to every test. (The `pytest-flask` plugin offers a similar fixture, but with our own fixtures it is not needed.)

The test directory structure

```

notes-api/
├── app/
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_health.py
│   ├── test_notes.py
│   └── test_errors.py
├── run.py
├── requirements.txt
└── requirements-dev.txt

```

- ← Application code
- ← Test code (mirrors app/ structure)
- ← Makes tests/ a Python package
- ← pytest configuration and shared fixtures
- ← Tests for app/routes/health.py
- ← Tests for app/routes/notes.py
- ← Tests for app/errors.py

Each test file is named after the module it tests (`test_notes.py` tests `app/routes/notes.py`), which makes finding tests predictable. Modules such as `config.py` and `startup.py` have no test file yet; section 17.7 shows how coverage reveals gaps like these.

17.3 Writing the Test Infrastructure

tests/conftest.py — Shared test configuration

`conftest.py` is a special file that `pytest` automatically loads before running tests. It defines **fixtures** — reusable objects that tests can request.

Listing 17.1 tests/conftest.py

```

"""
Shared pytest fixtures for the Notes API test suite.

pytest automatically discovers and loads this file before running tests.
Fixtures defined here are available to all test files without importing.

The two critical fixtures here are:

```

```

    app      - a Flask application configured for testing
    client    - a test HTTP client that sends requests to the test app
"""
import os
import shutil
import tempfile

import pytest

# app/config.py reads the environment once, when the app package is first
# imported. So the test settings must be in place BEFORE that import -
# here, at the top of conftest.py, which pytest loads before any test.
# (load_dotenv() never overrides a variable that is already set, so a
# developer's .env file cannot leak into the tests.)
#
# The database is a temporary FILE, not "sqlite:///memory:". The app opens
# a new connection for every request, and every connection to ":memory:"
# gets its own, empty database: the table created at startup would be
# gone by the first request. A file is shared by all connections.
_TEST_DB_DIR = tempfile.mkdtemp(prefix="notes-api-tests-")
os.environ["DATABASE_URL"] = f"sqlite:///({_TEST_DB_DIR})/test.db"
os.environ["SECRET_KEY"] = "test-secret-key-not-for-production-1234"

from app import create_app # noqa: E402 (must come after the settings above)

@pytest.fixture(scope="session")
def app():
    """
    Create a Flask application instance configured for testing.

    'scope="session"' means pytest creates this fixture once per test
    session and reuses it for all tests. Creating a Flask app is
    moderately expensive (imports, database initialization), so
    creating it once is significantly faster than once per test.

    Test configuration differences from production:
    - Uses a temporary SQLite database file, deleted after the run
    - TESTING=True makes Flask re-raise exceptions that no error
      handler catches (ours catches all of them; see test_errors.py)
    - SECRET_KEY is a test-only value
    """
    application = create_app()
    application.config.update({"TESTING": True})

```

```

yield application
# Code after yield runs after all tests using this fixture complete.
shutil.rmtree(_TEST_DB_DIR, ignore_errors=True)

@pytest.fixture(scope="function")
def client(app):
    """
    Create a test client for the Flask application.

    'scope="function"' means pytest creates a fresh client for each
    test function. This ensures test isolation: one test's requests
    do not affect another test's requests.

    The test client behaves like an HTTP client: you call
    client.get("/health") and get a response object back. No server
    needs to be running.
    """
    return app.test_client()

@pytest.fixture(scope="function", autouse=True)
def clean_database(app):
    """
    Clear all database tables before each test.

    'autouse=True' means this fixture runs before every test function
    automatically, without tests needing to request it explicitly.

    This ensures test isolation: one test's database changes do not
    affect another test. Each test starts with a clean slate.
    """
    with app.app_context():
        from app.database import get_db
        db = get_db()
        db.cursor().execute("DELETE FROM notes")
        db.commit()
    yield
    # No cleanup needed after the test – the next test's setup handles it.

```

Understanding **scope**:

Scope	When created	When destroyed
"function"	Before each test function	After each test function
"class"	Before each test class	After each test class
"module"	Before each test module (file)	After each test module
"session"	Once, at the start of the test run	At the end of the test run

Fixtures with broad scope (session, module) are faster but require careful design — a fixture with `scope="session"` is shared across all tests, so any state changes persist. The `clean_database` fixture with `scope="function"` and `autouse=True` ensures each test gets a clean database regardless of what other tests did.

tests/test_health.py — Health check tests

Listing 17.2 tests/test_health.py

```

"""
Tests for the /health endpoint.

Each test function's name starts with 'test_'. pytest discovers test
functions by this naming convention. A test function that returns normally
passes; a test that raises any exception (such as the AssertionError from
a failed assert statement) fails.
"""

class TestHealthEndpoint:
    """
    Group related tests in a class for organization.

    Class names must start with 'Test' for pytest to discover them.
    They must not define __init__: pytest skips test classes that do.
    """

    def test_health_returns_200(self, client):
        """
        GET /health should return HTTP 200 OK.

        The 'client' parameter is the test client fixture defined in
        conftest.py. pytest injects it automatically.
        """
        response = client.get("/health")

```

```

    assert response.status_code == 200

def test_health_returns_json(self, client):
    """GET /health should return a JSON response."""
    response = client.get("/health")
    # response.content_type is the Content-Type header value
    assert "application/json" in response.content_type

def test_health_returns_status_ok(self, client):
    """GET /health should return {"status": "ok"}."""
    response = client.get("/health")
    data = response.get_json() # Parses the JSON response body
    assert data is not None
    assert data["status"] == "ok"

def test_health_post_returns_405(self, client):
    """
    POST /health should return HTTP 405 Method Not Allowed.

    /health only accepts GET. Flask automatically returns 405 for
    methods not registered on a route.
    """
    response = client.post("/health")
    assert response.status_code == 405
    assert response.get_json() == {"error": "Method not allowed"}

```

tests/test_notes.py — Notes endpoint tests

Listing 17.3 tests/test_notes.py

```

"""
Tests for the /notes endpoints.

Tests are organized into classes by endpoint/operation.
Each test is independent – the clean_database fixture in conftest.py
ensures the database is empty before each test.
"""

class TestGetNotes:
    """Tests for GET /notes/"""

    def test_get_notes_empty_returns_200(self, client):
        """GET /notes/ on an empty database returns HTTP 200."""

```

```

        response = client.get("/notes/")
        assert response.status_code == 200

    def test_get_notes_empty_returns_empty_list(self, client):
        """GET /notes/ on an empty database returns an empty list."""
        response = client.get("/notes/")
        data = response.get_json()
        assert data == {"notes": [], "count": 0}

    def test_get_notes_returns_created_notes(self, client):
        """GET /notes/ returns notes that have been created."""
        # Create two notes (json= also sets Content-Type: application/json)
        client.post("/notes/", json={"content": "First note"})
        client.post("/notes/", json={"content": "Second note"})

        response = client.get("/notes/")
        data = response.get_json()

        assert data["count"] == 2
        # Notes are returned newest-first
        assert data["notes"][0]["content"] == "Second note"
        assert data["notes"][1]["content"] == "First note"

    def test_get_notes_response_has_required_fields(self, client):
        """Each note in the GET /notes/ response has id, content, created_at."""
        client.post("/notes/", json={"content": "Test note"})
        response = client.get("/notes/")
        notes = response.get_json()["notes"]

        assert len(notes) >= 1
        note = notes[0]
        assert "id" in note
        assert "content" in note
        assert "created_at" in note

class TestCreateNote:
    """Tests for POST /notes/"""

    def test_create_note_returns_201(self, client):
        """POST /notes/ with valid content returns HTTP 201 Created."""
        response = client.post("/notes/", json={"content": "My new note"})
        assert response.status_code == 201

    def test_create_note_returns_note_data(self, client):

```



```

"""POST /notes/ returns the created note with id and content."""
response = client.post("/notes/", json={"content": "My new note"})
data = response.get_json()

assert data["content"] == "My new note"
assert "id" in data
assert isinstance(data["id"], int)
assert data["id"] > 0
assert "created_at" in data

def test_create_note_without_json_content_type_returns_415(self, client):
    """POST /notes/ with a non-JSON Content-Type returns HTTP 415."""
    # Sending plain text instead of JSON: rejected by the middleware
    response = client.post(
        "/notes/",
        data="not json",
        content_type="text/plain",
    )
    assert response.status_code == 415

def test_create_note_with_malformed_json_returns_400(self, client):
    """POST /notes/ with a body that claims to be JSON but is not returns 400."""
    response = client.post(
        "/notes/",
        data="{not valid json",
        content_type="application/json",
    )
    assert response.status_code == 400
    assert response.get_json() == {"error": "Request body must be a JSON object"}

def test_create_note_without_content_field_returns_400(self, client):
    """POST /notes/ with JSON but no 'content' field returns HTTP 400."""
    response = client.post("/notes/", json={"title": "wrong field name"})
    assert response.status_code == 400
    data = response.get_json()
    assert "error" in data

def test_create_note_with_empty_content_returns_400(self, client):
    """POST /notes/ with empty content returns HTTP 400."""
    response = client.post("/notes/", json={"content": ""})
    assert response.status_code == 400

def test_create_note_with_whitespace_only_content_returns_400(self, client):
    """POST /notes/ with only whitespace content returns HTTP 400."""

```

```

        response = client.post("/notes/", json={"content": "  "})
        assert response.status_code == 400

def test_create_note_with_non_string_content_returns_400(self, client):
    """POST /notes/ with a non-string content value returns HTTP 400."""
    response = client.post("/notes/", json={"content": 42})
    assert response.status_code == 400

def test_create_note_strips_whitespace(self, client):
    """POST /notes/ trims leading/trailing whitespace from content."""
    response = client.post("/notes/", json={"content": "  trimmed note  "})
    assert response.status_code == 201
    data = response.get_json()
    assert data["content"] == "trimmed note"

def test_create_note_persists_to_database(self, client):
    """A created note is retrievable via GET /notes/."""
    # Create
    post_response = client.post("/notes/", json={"content": "Persisted note"})
    note_id = post_response.get_json()["id"]

    # Retrieve
    get_response = client.get(f"/notes/{note_id}")
    assert get_response.status_code == 200
    assert get_response.get_json()["content"] == "Persisted note"

def test_create_multiple_notes_have_unique_ids(self, client):
    """Each created note gets a unique ID."""
    resp1 = client.post("/notes/", json={"content": "Note one"})
    resp2 = client.post("/notes/", json={"content": "Note two"})
    resp3 = client.post("/notes/", json={"content": "Note three"})

    ids = [
        resp1.get_json()["id"],
        resp2.get_json()["id"],
        resp3.get_json()["id"],
    ]
    # All IDs are unique
    assert len(set(ids)) == 3

def test_create_note_content_too_long_returns_400(self, client):
    """POST /notes/ with content over 10,000 chars returns HTTP 400."""
    long_content = "x" * 10_001
    response = client.post("/notes/", json={"content": long_content})

```

```

        assert response.status_code == 400

class TestGetNote:
    """Tests for GET /notes/<note_id>"""

    def test_get_existing_note_returns_200(self, client):
        """GET /notes/<id> for an existing note returns HTTP 200."""
        create_resp = client.post("/notes/", json={"content": "Find me"})
        note_id = create_resp.get_json()["id"]

        response = client.get(f"/notes/{note_id}")
        assert response.status_code == 200

    def test_get_existing_note_returns_correct_content(self, client):
        """GET /notes/<id> returns the note with the correct content."""
        create_resp = client.post("/notes/", json={"content": "Specific content"})
        note_id = create_resp.get_json()["id"]

        response = client.get(f"/notes/{note_id}")
        data = response.get_json()

        assert data["id"] == note_id
        assert data["content"] == "Specific content"

    def test_get_nonexistent_note_returns_404(self, client):
        """GET /notes/<id> for a non-existent note returns HTTP 404."""
        response = client.get("/notes/99999")
        assert response.status_code == 404
        data = response.get_json()
        assert "error" in data

    def test_get_note_with_string_id_returns_404(self, client):
        """
        GET /notes/<non-integer> returns HTTP 404.

        Flask's <int:note_id> converter rejects non-integer values
        and returns 404 before the route handler is called.
        """
        response = client.get("/notes/not-a-number")
        assert response.status_code == 404

class TestDeleteNote:
    """Tests for DELETE /notes/<note_id>"""

```

```

def test_delete_existing_note_returns_204(self, client):
    """DELETE /notes/<id> for an existing note returns HTTP 204 No Content."""
    create_resp = client.post("/notes/", json={"content": "Delete me"})
    note_id = create_resp.get_json()["id"]

    response = client.delete(f"/notes/{note_id}")
    assert response.status_code == 204

def test_delete_existing_note_removes_it(self, client):
    """After DELETE /notes/<id>, the note is no longer retrievable."""
    create_resp = client.post("/notes/", json={"content": "Delete me"})
    note_id = create_resp.get_json()["id"]

    # Delete it
    client.delete(f"/notes/{note_id}")

    # Verify it's gone
    get_response = client.get(f"/notes/{note_id}")
    assert get_response.status_code == 404

def test_delete_nonexistent_note_returns_404(self, client):
    """
    DELETE /notes/<id> for a non-existent note returns HTTP 404.

    This pins down the design choice from Chapter 16 (section 16.5): the
    Notes API reports a missing note instead of treating the delete as
    already done. A test is where such a decision becomes enforceable.
    """
    response = client.delete("/notes/999999")
    assert response.status_code == 404

def test_delete_does_not_affect_other_notes(self, client):
    """Deleting one note does not affect other notes."""
    resp1 = client.post("/notes/", json={"content": "Keep me"})
    resp2 = client.post("/notes/", json={"content": "Delete me"})

    id_to_keep = resp1.get_json()["id"]
    id_to_delete = resp2.get_json()["id"]

    client.delete(f"/notes/{id_to_delete}")

    # The other note is still there
    get_response = client.get(f"/notes/{id_to_keep}")
    assert get_response.status_code == 200

```

```
assert get_response.get_json()["content"] == "Keep me"
```

tests/test_errors.py — Error handler tests

Chapter 16 spent a whole section on the handling boundary; these tests check that it does what it promises. The second one needs an endpoint that fails unexpectedly. pytest's built-in monkeypatch fixture provides it: it temporarily replaces the `get_db` function that the notes routes use with one that raises, and puts the original back after the test.

Listing 17.4 tests/test_errors.py

```
"""Tests for the JSON error handlers in app/errors.py."""

def test_unknown_url_returns_json_404(client):
    """A URL that matches no route gets a JSON 404, not Flask's HTML page."""
    response = client.get("/no-such-page")
    assert response.status_code == 404
    assert response.get_json() == {"error": "Not found"}

def test_unexpected_exception_returns_generic_500(client, monkeypatch):
    """An unexpected exception becomes a generic JSON 500 with no internal
    ↪ details."""
    def broken_get_db():
        raise RuntimeError("database password is hunter2")

    # Replace get_db where the notes routes look it up
    monkeypatch.setattr("app.routes.notes.get_db", broken_get_db)

    response = client.get("/notes/")
    assert response.status_code == 500
    assert response.get_json() == {"error": "An unexpected error occurred"}
    assert b"hunter2" not in response.data # the exception message never leaks
```

Notice that the second test gets a 500 *response* even though TESTING is on. TESTING only re-raises exceptions that no error handler catches, and the catch-all `@app.errorhandler(Exception)` catches all of them.

Running the tests

```
# Run all tests
pytest

# Run with verbose output (shows each test name and result)
pytest -v

# Run a specific test file
pytest tests/test_notes.py

# Run a specific test class
pytest tests/test_notes.py::TestCreateNote

# Run a specific test function
pytest tests/test_notes.py::TestCreateNote::test_create_note_returns_201

# Show print() output during tests (useful for debugging)
pytest -s

# Stop at the first failure
pytest -x
```

Expected output:

```
===== test session starts =====
platform linux -- Python 3.11.8, pytest-8.1.1, pluggy-1.4.0
rootdir: /notes-api
collected 30 items

tests/test_errors.py ..                               [ 6%]
tests/test_health.py ....                             [ 20%]
tests/test_notes.py .....                             [100%]

===== 30 passed in 0.45s =====
```

Thirty tests, under half a second. Fast, comprehensive, automated.

(Once `pyproject.toml` from section 17.6 is in place, its `addopts = "-v"` makes every run verbose, listing each test by name instead of printing dots.)

17.4 Code Quality Tools

Tests verify behavior. Code quality tools verify structure and style. Both are part of the build.

black — Code formatter

black is a Python code formatter. It takes your Python files and rewrites them according to a consistent style: consistent indentation, consistent quote style, consistent line lengths, consistent spacing.

Unlike a linter (which tells you what is wrong), **black** just fixes it:

```
# Check what black would change (without changing anything)
black --check app/ tests/ run.py

# Apply black's formatting
black app/ tests/ run.py
```

Black's philosophy is that style debates are wasteful. By using **black**, everyone on the team writes code that looks the same, regardless of personal preference. Code review focuses on logic, not formatting.

ruff — Linter

A **linter** analyzes code for potential errors, bad practices, and style violations without running the code. **ruff** is an extremely fast Python linter (written in Rust) that catches:

- Unused imports (`import os` but `os` is never used)
- Undefined variables (`print(undefined_variable)`)
- Unreachable code (code after a `return` statement)
- Dangerous patterns (mutable default arguments, bare `except:` clauses)
- Style violations (line too long, comparison to `None` using `==` instead of `is`)

```
# Check code for issues
ruff check app/ tests/ run.py

# Fix automatically-fixable issues
ruff check --fix app/ tests/ run.py
```

Example **ruff** output:

```
app/routes/notes.py:12:1: F401 `os` imported but unused
app/database.py:45:5: E711 comparison to `None` (use `is` or `is not`)
Found 2 errors.
```

mypy — Static type checker

mypy checks Python type annotations. If you annotate a function:

```
def get_note(note_id: int) -> dict:
    ...
```

mypy verifies that everywhere `get_note` is called, the caller passes an `int` and uses the return value as a `dict`. It catches type mismatches without running the code.

```
# Check types
mypy app/ run.py
```

Type checking is particularly valuable for catching bugs that tests might miss — for example, passing a `str` where an `int` is expected.

pip-audit — Security scanner

`pip-audit` checks your installed packages against public databases of known security vulnerabilities: the Python Packaging Advisory Database and OSV.

```
# Check for vulnerabilities in installed packages
pip-audit
```

Example output:

```
Found 1 known vulnerability in 1 package
Name      Version ID                               Fix Versions
-----
werkzeug 3.0.1   GHSA-2g68-c3qc-8985 3.0.3
```

This tells you that Werkzeug 3.0.1 (the version pinned in [Chapter 12](#)) has a known vulnerability — here, a flaw in the debugger — and that 3.0.3 fixes it. Running `pip-audit` as part of every build ensures that vulnerabilities in dependencies are caught as soon as they are discovered.

17.5 The Makefile — Automating the Build

A Makefile defines named sequences of commands. With a Makefile, running the entire build is one command: `make build`.

Listing 17.5 Makefile

```
# Build automation for the Notes API.
#
# Usage:
# make install    Install all dependencies
# make format     Auto-format code with black
# make lint       Check code style and quality
# make typecheck  Run static type checking
# make test       Run the test suite
# make test-coverage Run the tests and report coverage
# make security   Check for dependency vulnerabilities
# make build      Run the complete build (lint + typecheck + test + security)
# make run        Start the development server
# make clean      Remove build artifacts
#
# Recipe lines must start with a TAB character, not spaces. If you copy
# this file from a PDF or a web page, check that the tabs survived:
# "missing separator" means they did not.
#
# Prerequisites: make (on Linux/macOS, built-in; on Windows, install via chocolatey)

# .PHONY tells make that these targets are commands, not files.
# Without .PHONY, if a file named "test" existed, "make test" would
# think the target was already built and skip running it.
.PHONY: install format lint typecheck test test-coverage security build clean run

# Default target (what runs when you just type "make")
.DEFAULT_GOAL := build

# Python interpreter – use python3 explicitly to avoid ambiguity on
# systems where "python" might be Python 2
PYTHON := python3

# Source directories to check
SRC := app/ tests/ run.py

# _____
# Dependency installation
```

```

# -----

install:
    @echo "→ Installing dependencies..."
    ${PYTHON} -m pip install -r requirements-dev.txt
    @echo "✓ Dependencies installed."

# -----
# Code formatting
# -----

format:
    @echo "→ Formatting code with black..."
    black $(SRC)
    @echo "✓ Code formatted."

# -----
# Linting
# -----

lint:
    @echo "→ Checking code style with ruff..."
    ruff check $(SRC)
    @echo "→ Checking formatting with black..."
    black --check $(SRC)
    @echo "✓ Linting passed."

# -----
# Static type checking
# -----

typecheck:
    @echo "→ Running type checks with mypy..."
    mypy app/ run.py --ignore-missing-imports
    @echo "✓ Type checks passed."

# -----
# Testing
# -----

test:
    @echo "→ Running test suite..."
    pytest tests/ -v
    @echo "✓ All tests passed."

```

```

# Run tests with coverage report
test-coverage:
    @echo "> Running tests with coverage..."
    pytest tests/ -v --cov=app --cov-report=term-missing --cov-report=html
    @echo "✓ Coverage report generated in htmlcov/."

# -----
# Security scanning
# -----

security:
    @echo "> Scanning for dependency vulnerabilities..."
    pip-audit
    @echo "✓ No known vulnerabilities found."

# -----
# Complete build
# -----

build: lint typecheck test security
    @echo ""
    @echo "-----"
    @echo "✓ Build complete. All checks passed."
    @echo "-----"

# -----
# Development server
# -----

run:
    $(PYTHON) run.py

# -----
# Cleanup
# -----

clean:
    @echo "> Removing build artifacts..."
    find . -type d -name __pycache__ -exec rm -rf {} + 2>/dev/null || true
    find . -name "*.pyc" -delete 2>/dev/null || true
    find . -type d -name .pytest_cache -exec rm -rf {} + 2>/dev/null || true
    find . -type d -name .mypy_cache -exec rm -rf {} + 2>/dev/null || true
    find . -type d -name .ruff_cache -exec rm -rf {} + 2>/dev/null || true

```

```
find . -type d -name htmlcov -exec rm -rf {} + 2>/dev/null || true
rm -f .coverage 2>/dev/null || true
@echo "✓ Clean."
```

How Makefile targets work:

Each target follows this structure:

```
target-name: dependency1 dependency2
    command-to-run
    another-command
```

- The target name comes before the colon
- Dependencies (other targets that must succeed first) come after the colon
- Commands are indented with a **tab character** (spaces will not work — this is a quirk of Makefile syntax)
- @ before a command suppresses printing the command itself (you see only the output, not the command line)

When you run `make build`:

```
make build
```

Make sees that `build` depends on `lint`, `typecheck`, `test`, and `security`. It runs them in order. If any step fails (returns a non-zero exit code), Make stops immediately. The build fails.

Running the build

Before the first `make build`, run `make format` and `ruff check --fix app/ tests/ run.py` once. The listings in this book are compressed to save space — one blank line between top-level functions instead of PEP 8’s two, and imports not always in the order ruff’s I rules expect — so `black --check` and `ruff check` would reject them exactly as printed. After that one formatting pass:

```
make build
```

Expected output:

```

→ Checking code style with ruff...
All checks passed!
→ Checking formatting with black...
All done! ✨ 🍰 ✨
16 files would be left unchanged.
✓ Linting passed.
→ Running type checks with mypy...
Success: no issues found in 11 source files
✓ Type checks passed.
→ Running test suite...
===== test session starts =====
collected 30 items

tests/test_errors.py::test_unknown_url_returns_json_404 PASSED      [ 3%]
tests/test_errors.py::test_unexpected_exception_returns_generic_500 PASSED [ 6%]
...
tests/test_notes.py::TestDeleteNote::test_delete_does_not_affect_other_notes PASSED [100%]

===== 30 passed in 0.47s =====
✓ All tests passed.
→ Scanning for dependency vulnerabilities...
No known vulnerabilities found
✓ No known vulnerabilities found.



---


✓ Build complete. All checks passed.


---



```

This output confirms that the code passed every automated check. That is not the same as “ready to deploy” — configuration, database migrations, and a smoke test after deployment (Chapter 23) still stand between a green build and a working release — but it is the gate every release must pass first.

17.6 Configuration Files for Code Quality Tools

Create configuration files that standardize the behavior of the quality tools.

pyproject.toml — Unified tool configuration

`pyproject.toml` is the modern Python configuration standard. Many tools (black, ruff, mypy, pytest) read their configuration from this file. If you created one in Chapter 12, keep its `[project]` and `[build-system]` tables and add the `[tool.*]` sections below to it; otherwise create it at the project root:

Listing 17.6 pyproject.toml

```

[project]
name = "notes-api"
version = "0.1.0"
description = "A simple notes REST API"
requires-python = ">=3.10"

# -----
# pytest configuration
# -----

[tool.pytest.ini_options]
# Tell pytest where to find tests
testpaths = ["tests"]
# Verbose output by default (shows each test name)
addopts = "-v"
# Treat warnings as errors (forces you to fix deprecation warnings)
filterwarnings = [
    "error",
    # Ignore specific known warnings from dependencies
    "ignore::DeprecationWarning:werkzeug",
]

# -----
# black configuration
# -----

[tool.black]
# Maximum line length before black wraps the line
line-length = 100
# Minimum Python version to target (affects which syntax black uses)
target-version = ["py310", "py311"]

# -----
# ruff configuration
# -----

[tool.ruff]
line-length = 100
target-version = "py310"

[tool.ruff.lint]
# Rule sets to enable:
#   E, W – pycodestyle (PEP 8 style)
#   F     – pyflakes (common errors: unused imports, undefined names)
#   I     – isort (import ordering)

```

```

# N      - pep8-naming (naming conventions)
# UP     - pyupgrade (modernize Python syntax)
select = ["E", "W", "F", "I", "N", "UP"]

# Rules to ignore:
# E501 - line too long (black handles this)
ignore = ["E501"]

# Exclude directories that don't need linting
exclude = ["venv/", ".venv/", "__pycache__/" ]

# -----
# mypy configuration
# -----
[tool.mypy]
python_version = "3.10"
# Also check the bodies of functions that have no type annotations
# (by default mypy skips them, so "no issues found" would prove little)
check_untyped_defs = true
# Warn when code can never be reached
warn_unreachable = true
# Don't require type stubs for libraries that don't have them
ignore_missing_imports = true
# Check the app package
files = ["app/", "run.py"]

# -----
# coverage configuration (for make test-coverage)
# -----
[tool.coverage.run]
source = ["app"]
omit = ["*/tests/*"]

[tool.coverage.report]
# Fail if coverage drops below 80%
fail_under = 80
show_missing = true

```

17.7 What Test Coverage Means

Test coverage is the percentage of your application code that is executed when your tests run. A coverage tool (like `pytest-cov`) tracks which lines were executed.

```
# Run tests with coverage (pytest-cov is in requirements-dev.txt)
pytest --cov=app --cov-report=term-missing
```

Output:

Name	Stmts	Miss	Cover	Missing
app/__init__.py	26	0	100%	
app/config.py	21	3	86%	28–35
app/database.py	41	5	88%	35–39, 79, 111
app/errors.py	22	2	91%	66, 76
app/logging_config.py	63	17	73%	45–46, 117–140, 174, 193–211, 227
app/middleware.py	27	0	100%	
app/routes/__init__.py	0	0	100%	
app/routes/health.py	5	0	100%	
app/routes/notes.py	57	0	100%	
app/startup.py	48	23	52%	27, 37, 48–55, 70–79, 96–97, 102–103
TOTAL	310	50	84%	

(Line numbers are for the files after `make format`.)

What coverage tells you:

- Lines 28–35 in `config.py` are not tested: the exit path of `_require()`, which runs only when a required variable is missing.
- The missed lines in `database.py` are the PostgreSQL branches — the tests use SQLite.
- `logging_config.py` is at 73%: the tests run with the default human format and no `LOG_DIR`, so the JSON formatter and the log-file handler never run.
- `startup.py` has the lowest coverage at 52%: the failure paths (what happens when a startup check fails), the PostgreSQL connectivity check, and `check_port()`, which only `run.py` calls.

What coverage does not tell you:

Coverage tells you which lines were executed. It does not tell you whether those lines were tested correctly. A test that executes a line but does not assert anything about the result gives 100% coverage for that line but provides no verification.

The goal is not 100% coverage at all costs. The goal is meaningful coverage: every important behavior has a test. The `fail_under = 80` configuration causes the build to fail if total coverage drops below 80%, which is a reasonable minimum.

Testing the PostgreSQL code path

The tests currently use SQLite, which means the PostgreSQL branches in `database.py` (and the database check in `startup.py`) are not covered. The route handlers themselves contain no database-specific code, so they are fully exercised. In a production build pipeline, you would run a second test configuration against a real PostgreSQL database:

```
# In CI, after starting a PostgreSQL service:  
DATABASE_URL=postgresql://user:pass@localhost:5432/test_db pytest
```

For now, noting where PostgreSQL-specific code is untested is sufficient. [Chapter 18](#) (Docker) sets up the infrastructure that makes PostgreSQL testing in CI straightforward.

17.8 A Test Suite You Can Trust

Thirty passing tests are a good start. But a test suite earns trust only when three things are true: it checks the behaviours that matter, it fails when (and only when) something is broken, and it stays fast enough that people run it all the time. This section is about how to get there. It turns the principles from Part III (the test pyramid, “never mock what you don’t own”) into concrete decisions for the Notes API.

What to test, and at which level

Tests come in three sizes, according to how much of the system they run:

Level	What it runs	Speed	In the Notes API
Unit	one function, on its own: no HTTP, no database	microseconds	<code>to_iso()</code> , <code>sql()</code> , <code>_mask()</code> , below
Integration	several real parts together: routing, handler, database	milliseconds	every test-client test in section 17.3
End-to-end	the real, running system, over real HTTP	seconds	the smoke test of the Docker image (Chapter 19); the deploy script's health check (Chapter 23)

The **test pyramid** says you should have many unit tests, fewer integration tests, and very few end-to-end tests. Lower tests are faster, more precise about *what* broke, and cheaper to keep working. Treat the pyramid as a default, not a law. The right shape depends on where your code's logic lives. A service whose logic is mostly “take a request, check it, run a query, return the row” is best tested at the integration level, through the test client, with a real database. That is why most of the Notes API's tests look the way they do. A service with a pricing engine or a scheduling algorithm inside would have many unit tests for that core. In both cases the rule underneath is the same: **test each behaviour at the lowest level that can actually show it working**.

Whatever the level, every endpoint needs tests for the same few kinds of situation:

- **The normal case.** It works, and returns exactly the right shape and status.
- **Bad input.** Missing, wrong type, empty, too long, strange characters. Each one gets a **400** with the right message.
- **Things that do not exist.** A missing note is a **404**, never a **500**.
- **The edges.** An empty list, exactly the maximum length, the first and the last item.
- **The decisions.** Every “we chose X over Y” (for example, **404** for deleting a missing note, section 16.5) deserves a test, because a test is what makes a decision stick.
- **Failure of what it depends on.** The database is down, so the response is a clean **500** with no leaked details (`test_errors.py`).

What *not* to test: the framework itself (Flask's routing works), and the private details of your own code. Test through the public interface (the HTTP API, or a function's arguments and return value), so that you can change *how* something works without rewriting its tests.

Unit tests: small functions on their own

The Notes API has a handful of small functions that need nothing but their arguments. They are ideal unit tests: precise, instant, and independent of everything else.

```
# tests/test_units.py
"""Unit tests: small functions, checked on their own, without HTTP or a database."""

from datetime import datetime, timezone

import pytest

from app import database, startup
from app.database import row_to_note, to_iso

def test_to_iso_turns_sqlite_text_into_utc_iso_8601():
    assert to_iso("2024-01-15 14:00:05") == "2024-01-15T14:00:05+00:00"

def test_to_iso_formats_a_postgresql_datetime():
    value = datetime(2024, 1, 15, 14, 0, 5, tzinfo=timezone.utc)
    assert to_iso(value) == "2024-01-15T14:00:05+00:00"

def test_row_to_note_converts_the_timestamp():
    row = {"id": 1, "content": "Buy milk", "created_at": "2024-01-15 14:00:05"}
    assert row_to_note(row) == {
        "id": 1,
        "content": "Buy milk",
        "created_at": "2024-01-15T14:00:05+00:00",
    }

@pytest.mark.parametrize(
    ("is_sqlite", "expected"),
    [
        (True, "SELECT * FROM notes WHERE id = ?"),
        (False, "SELECT * FROM notes WHERE id = %s"),
    ],
)

def test_sql_uses_the_right_placeholder(monkeypatch, is_sqlite, expected):
    monkeypatch.setattr(database, "IS_SQLITE", is_sqlite)
    assert database.sql("SELECT * FROM notes WHERE id = ?") == expected
```

```

def test_mask_hides_the_password_in_a_database_url():
    url = "postgresql://notes_user:s3cret@db.example.com:25060/notesdb"
    assert startup._mask(url) == (
        "postgresql://notes_user:***@db.example.com:25060/notesdb"
    )

def test_a_short_secret_key_stops_startup(monkeypatch):
    monkeypatch.setattr(startup, "SECRET_KEY", "too-short")
    with pytest.raises(SystemExit) as exit_info:
        startup.check_secret_key()
    assert exit_info.value.code == 1

```

Notice two things. First, `sql()`'s PostgreSQL branch, which the test database never exercises (section 17.7), is now tested directly: `monkeypatch.setattr` swaps the module's `IS_SQLITE` constant for the length of one test. Second, the startup checks' *failure* paths, the least-covered code in the coverage report, can be tested too. `pytest.raises(SystemExit)` asserts that the code tried to exit, and then catches that exit so that the test run itself carries on.

Parametrized tests: one test, many cases

Section 17.3 has five nearly identical tests for bad POST `/notes/` bodies. Each one sends a body and expects a 400. When tests differ only in their data, **parametrize** them: write the test once, and let `pytest` run it once per case.

```

# tests/test_notes.py (replacing the five separate bad-body tests)
import pytest

@pytest.mark.parametrize(
    ("body", "error"),
    [
        ({'title': 'wrong field'}, "'content' is required"),
        ({'content': ''}, "'content' cannot be empty"),
        ({'content': " \t\n "}, "'content' cannot be empty"),
        ({'content': 42}, "'content' must be a string"),
        ({'content': "x" * 10_001}, "'content' cannot exceed 10,000 characters"),
        (["not", "an", "object"], "Request body must be a JSON object"),
    ],
    ids=["missing", "empty", "whitespace", "not-a-string", "too-long", "list"],

```

```

)
def test_create_note_rejects_an_invalid_body(client, body, error):
    response = client.post("/notes/", json=body)
    assert response.status_code == 400
    assert response.get_json() == {"error": error}

```

Each case is reported as a test of its own, named by its ids:

```

tests/test_notes.py::test_create_note_rejects_an_invalid_body[missing] PASSED
tests/test_notes.py::test_create_note_rejects_an_invalid_body[empty] PASSED
tests/test_notes.py::test_create_note_rejects_an_invalid_body[whitespace] PASSED
tests/test_notes.py::test_create_note_rejects_an_invalid_body[not-a-string] PASSED
tests/test_notes.py::test_create_note_rejects_an_invalid_body[too-long] PASSED
tests/test_notes.py::test_create_note_rejects_an_invalid_body[list] PASSED

```

The table of cases is now easy to read, and easy to extend: a new rule is one new line. That is exactly how the next bug fix in this section starts. It also checks the *exact* error message, which the five older tests did not always do.

Test doubles: stubs, fakes, spies and mocks

Sometimes a test needs to replace a real part of the system: to make it fail on purpose, to make it faster, or to watch how it is used. The replacement is called a **test double**, like a stunt double in a film. There are four kinds, and the Notes API has already used most of them:

Kind	What it does	In this book
Stub	returns a fixed answer, or raises a fixed error	<code>broken_get_db()</code> in <code>test_errors.py</code> , which always raises
Fake	a real, working, lighter implementation	the SQLite test database standing in for PostgreSQL; later, a fake email service (Chapter 37)
Spy	lets the real thing run, and records what happened	pytest's <code>caplog</code> fixture, which records every log message
Mock	records how it was called, so the test can check those calls	<code>unittest.mock.Mock</code> , below

A spy is the right tool for a promise the application makes about its *logs*. Chapter 16 promised that a note's content never reaches them, because a note may be private. That promise deserves a test:

```
# tests/test_notes.py (added)
def test_a_rejected_note_is_logged_without_its_content(client, caplog):
    client.post("/notes/", json={"content": "my bank PIN is 4321" + "!" * 10_000})
    assert "POST /notes/ rejected" in caplog.text
    assert "4321" not in caplog.text # a private note never reaches the logs
```

A mock is for when *how something was called* is itself the behaviour. The startup check must give up on an unreachable database after five seconds instead of hanging forever, so the test checks the timeout that `check_database()` passes along:

```
# tests/test_startup.py
import sys
from unittest.mock import Mock

import pytest

from app import startup

def test_an_unreachable_database_stops_startup(monkeypatch):
    fake_psycopg2 = Mock()
    fake_psycopg2.connect.side_effect = Exception("connection refused")
    # check_database() imports psycopg2 when it runs: hand it the fake
```

```

monkeypatch.setitem(sys.modules, "psycpg2", fake_psycpg2)
monkeypatch.setattr(startup, "IS_SQLITE", False)

with pytest.raises(SystemExit):
    startup.check_database()

# The mock recorded how it was called: the check must not wait forever
fake_psycpg2.connect.assert_called_once()
assert fake_psycpg2.connect.call_args.kwargs["connect_timeout"] == 5

```

This test knowingly breaks Part III’s rule, “never mock what you don’t own”, and it is worth seeing why the rule exists. The mock encodes an *assumption* about `psycpg2`: that its function is called `connect` and takes a `connect_timeout` argument. If `psycpg2` changed, this test would go on passing while the real code broke. Here the risk is small, because the test mocks only one well-known call. Still, often there is a way to test without a double at all. Point `DATABASE_URL` at a port nobody listens on (`postgresql://u:p@127.0.0.1:1/notesdb`), and a *real* connection attempt is refused in a few milliseconds. When a real version is that cheap, prefer it.

Where to stop:

- **Replace things at the edges of your system**, like a network service, the clock, or randomness, and not the code in the middle. A test that mocks your own functions checks how the code is *written*, not what it *does*, and breaks at every refactor.
- **Prefer fakes to mocks**. A fake behaves like the real thing, so tests built on it check real behaviour.
- **Know what each double hides**. The SQLite fake cannot show PostgreSQL-only bugs: the search query of [Chapter 33](#), the concurrency of section 33.13, and the bug at the end of this section. Every double buys speed or control with blind spots. Write the blind spots down, and cover them somewhere else. [Chapter 39](#) covers these by running the suite on PostgreSQL.

Factory fixtures: setting up data without repetition

Almost every test in section 17.3 starts by creating a note, and then repeats the same `POST` and the same `.get_json()["id"]`. A **factory fixture** gives tests a function that creates data, with sensible defaults, and checks that the setup itself worked:

```
# tests/conftest.py (added)
@pytest.fixture
def create_note(client):
    """
    A factory: tests call create_note("text") to get a saved note back,
    instead of repeating the POST and its checks in every test.
    """

    def _create(content: str = "A note") -> dict:
        response = client.post("/notes/", json={"content": content})
        assert response.status_code == 201, response.get_json()
        return response.get_json()

    return _create
```

```
# tests/test_notes.py (added)
def test_get_note_returns_what_was_saved(client, create_note):
    note = create_note("Find me")
    response = client.get(f"/notes/{note['id']}")
    assert response.get_json() == note
```

The test now says only what it is *about*. When creating a note one day needs more (an owner in Chapter 38, say), only the factory changes, not fifty tests.

Tests that fail at random

A **flaky test** passes and fails on the same code. It is worse than no test at all, because it teaches people to ignore red builds and press “re-run”. Flakiness always has a cause, and the causes are few:

Cause	Example	Fix
Tests share state	one test leaves a note behind; another counts notes	reset state before every test (<code>clean_database</code>); never depend on test order
Order that is not guaranteed	results with no <code>ORDER BY</code> , or ordered by a value that can tie	order by something unique, such as <code>id</code>
Time	“created today” fails at midnight; timezones differ between laptop and CI	fix the clock in the test, or compare against values captured at the start
Randomness	random test data that sometimes hits an edge case	use a fixed seed, and print it when the test fails
Timing	<code>time.sleep(1)</code> “to be sure it has finished”	wait for the condition itself, with a timeout (as the CI smoke test of Chapter 19 does with <code>curl</code> <code>--retry</code>)
The network	a test that calls a real outside service	use a fake at the edge; test the real thing separately

The Notes API has a quiet example of the second row. `list_notes` orders by `id DESC`, and the test that expects “Second note” before “First note” relies on that. Had it ordered by `created_at DESC` instead, the test would fail now and then. SQLite’s `CURRENT_TIMESTAMP` counts whole seconds, so two notes created in the same test usually get *identical* timestamps, and the database may return tied rows in either order.

The rule for flaky tests is simple: **a flaky test is a bug, and it gets fixed, not re-run**. `pytest --lf` (“last failed”) re-runs just the failures while you investigate. Running the suite in random order (the `pytest-randomly` plugin) flushes out hidden dependencies between tests.

Writing the test first

Everything so far adds tests *after* the code. Sometimes it pays to write the test *first*, especially when fixing a bug. The loop is short and has three steps. Write a test that shows the problem and watch it fail (**red**). Make the smallest change that makes it pass (**green**). Then tidy up with the test as your safety net (**refactor**).

Here is a real bug in the Notes API, as a worked example. JSON can carry a NUL character (“`\u0000`”) inside a string, and SQLite stores it happily. PostgreSQL cannot store NUL in a TEXT column at all: `psycopg2` raises `ValueError: A string literal cannot contain NUL (0x00) characters.`, and the catch-all handler turns that into a `500`. So a client can

crash any write request in production, while every test on the laptop passes.

First the test, which is one new line in the parametrized table:

```
({"content": "a\x00b"}, "'content' contains invalid characters"),
```

(with "nul" added to ids). Run it: **red**. On SQLite the note is simply saved:

```
_____ test_create_note_rejects_an_invalid_body[nul] _____

client = <FlaskClient <Flask 'app'>>, body = {'content': 'a\x00b'}
error = "'content' contains invalid characters"
...
def test_create_note_rejects_an_invalid_body(client, body, error):
    response = client.post("/notes/", json=body)
>    assert response.status_code == 400
E    assert 201 == 400
E        + where 201 = <WrapperTestResponse streamed [201 CREATED]>.status_code
```

Then the smallest fix, in `create_note` right after the length check:

```
# PostgreSQL cannot store a NUL character; SQLite can, so only a test
# written for this rule catches it before production does
if "\x00" in content:
    return _invalid("'content' contains invalid characters")
```

Run it again: **green**, on both databases. The bug is fixed, and the test stays in the suite for good, so it can never quietly return. (When [Chapter 31](#) moves validation into a reusable `Validator`, this rule moves with it, and this test is what proves nothing was lost on the way.)

Test-first is not a religion. Many good engineers write the test right after the code, and some code is easier to explore first and test afterwards. But for bug fixes it is almost always right: a fix without a failing test first is a fix you have not actually seen working.

17.9 Working with Git: Branches, Pull Requests, and Review

Tests and CI check the code. The **Git workflow** decides *how a change travels*, from one engineer's editor to the main line of the code that everyone shares and that gets deployed. Part III recommends **trunk-based development** for small teams. This section is what that looks like, day to day, for the Notes API.

One rule: main is always releasable

Everything in a good Git workflow serves a single rule: **the main branch is always in a state that could be deployed**. From [Chapter 23](#) on, the Notes API is deployed from main automatically, so the rule stops being a nice idea and becomes a requirement. Nobody works on main directly. Every change goes on a short-lived **branch**, is checked by CI and read by a second person in a **pull request**, and only then is merged.

The flow of one change

```
# 1. Start from an up-to-date main
git switch main
git pull

# 2. A branch for this one change, named after what it does
git switch -c fix/reject-nul-in-content

# 3. Work in small steps; commit each one that makes sense on its own
git add app/routes/notes.py tests/test_notes.py
git commit          # opens the editor for the message (below)

# 4. Publish the branch and open a pull request
git push -u origin HEAD
gh pr create --fill # or use the "Compare & pull request" button on GitHub

# 5. CI runs on the pull request (section 17.10); a colleague reviews it.
#   To take new commits from main while you wait:
git pull --rebase origin main
git push --force-with-lease # safe only on YOUR branch, never on main

# 6. Once approved and green, merge, and delete the branch
gh pr merge --squash --delete-branch
```

The branch should live for hours or days, not weeks. The longer a branch lives, the further it drifts from main, and the more painful merging it becomes. A big feature is delivered as a series of small pull requests, each of which is safe to merge on its own. That is the same idea as the expand/contract migrations of [Chapter 33](#).

Commits and their messages

A commit message is written for the person who will run `git log` or `git blame` on this line in two years' time. It needs two things: *what* changed, in a short subject, and *why*, in the body.

Reject NUL characters in note content

PostgreSQL cannot store NUL (0x00) in a TEXT column: psycopg2 raises ValueError, which reached clients as a 500. SQLite accepts it, so the test suite never noticed. Validate it as a 400 instead, with a test.

The conventions: a subject under about 50 characters, in the imperative mood (“Reject...”, not “Rejected...”), a blank line, then a body wrapped at about 72 characters. Each commit should leave the code working, so that any commit can be checked out, tested, or reverted on its own. Some teams also prefix subjects with a type (**fix:** ..., **feat:** ..., known as *Conventional Commits*), so that release notes can be generated from the history.

Pull requests

A **pull request** (GitHub’s name; GitLab calls it a *merge request*) is a request to merge one branch into another. It is also the place where CI results, the review and the discussion about a change are collected. A good pull request:

- **does one thing.** A bug fix and a refactor in the same pull request make both harder to review;
- **is small.** Reviews of more than a few hundred changed lines get noticeably worse, because reviewers skim;
- **explains itself**, in a description like this:

What: Reject note content containing NUL characters (400 instead of 500).

Why: PostgreSQL cannot store NUL in TEXT; the insert failed with a ValueError that surfaced as a 500. Reproduced on PostgreSQL 16.

How to verify: new case "nul" in test_create_note_rejects_an_invalid_body;
POST {"content": "a\u0000b"} now returns 400.

Code review

A review is a second engineer reading the change before it is merged. It catches bugs, but that is not its only value: it spreads knowledge of the code, so that no part of it is understood by only one person. Split the work between machines and people. **Machines** check formatting, lint, types, tests and coverage; that is make build in CI, and a reviewer should never spend attention on them. **People** check what machines cannot:

- **Correctness.** Does it do what the description says? What input breaks it?
- **Tests.** Is the new behaviour tested? Would the tests fail if the fix were undone?
- **Design.** Is it in the right place, and consistent with the rest of the code? Will the next change be easy?
- **Clarity.** Could someone new understand it without the author explaining?
- **Safety.** Secrets, error messages that leak internals, SQL built from strings, a migration that breaks the running version.

Review the code, not the person: “this query can return two rows for one note, can’t it?” rather than “you forgot the join condition”. As an author, answer every comment, either with a change or with a reason, and never take a review personally. A comment on your code is the cheapest bug report you will ever get.

Protecting main

The rules above only hold if the repository enforces them. On GitHub, a **branch protection rule** (Settings → Branches), or a **ruleset** (Settings → Rules), can do exactly that for main:

- require a pull request before merging, with at least one approval;
- require status checks to pass: select the CI jobs from section 17.10, which appear as `build (3.10)`, `build (3.11)` and `build (3.12)` once they have run at least once;
- require the branch to be up to date with `main` before merging;
- block force pushes and deletion;
- do not allow bypassing these settings (without it, administrators, including the owner, can still push past them).

After this, even the repository’s owner cannot push a broken change straight to `main`. That is the point: the protection guards against a hurried Friday afternoon, not against colleagues. (GitHub’s settings pages move around over time; the names of the options are the stable part.)

Merging, undoing, and release tags

GitHub offers three ways to merge a pull request. A **merge commit** keeps every commit of the branch, plus a commit that joins the two histories. A **squash merge** turns the whole pull request into one commit on `main`. A **rebase merge** replays the branch’s commits onto `main` one by one. For small pull requests, squash keeps `main`’s history linear and readable, with one commit per change and its message taken from the pull request. That readability matters when you later hunt for the commit that introduced a bug (`git bisect`, Chapter 16).

To undo a change that is already on `main`, **revert** it:

```
git revert <commit>      # a NEW commit that undoes the old one
```

Never rewrite the history of a shared branch (`git reset`, then `git push --force`), because everyone else's copy of `main` would then disagree with GitHub's. A revert is an ordinary change: it goes through a pull request and CI like any other, and it is deployed like any other.

Finally, mark releases with a **tag**, a permanent name for one commit:

```
git tag -a v1.4.0 -m "Tags on notes; migration runner"
git push origin v1.4.0
```

Version numbers usually follow **semantic versioning**, MAJOR.MINOR.PATCH. Raise PATCH for a fix, MINOR for a feature that keeps existing clients working, and MAJOR for a breaking change (the same distinction as section 34.10's API versioning). The container images in [Chapter 19](#) are tagged with commit hashes for machines. Release tags are for people: changelogs, support tickets, "which version are you on?"

17.10 Continuous Integration: The Build Running Automatically

The build process you have defined in `Makefile` is only valuable if it actually runs. Running `make build` manually before every deployment is better than nothing, but still relies on human discipline.

Continuous Integration (CI) is the practice of running the build process automatically every time code is pushed to the repository. If the build fails, the developer knows immediately — not days later when someone tries to deploy.

GitHub Actions: CI for free

GitHub Actions is a CI platform built into GitHub. It runs your build on GitHub's servers, automatically, on every push.

Create a directory and file:

```
notes-api/
└─ .github/
   └─ workflows/
      └─ build.yml
```

Note: GitHub Actions evolves quickly: action major versions (actions/checkout@v4, actions/setup-python@v5), runner images, and workflow syntax change over time. Check each action's page for its current version.

Listing 17.7 `.github/workflows/build.yml`

```
# GitHub Actions workflow – runs the build on every push and pull request.
#
# GitHub Actions reads YAML files in .github/workflows/ and runs them
# automatically. No configuration in the GitHub UI is needed.

name: Build and Test

# Trigger conditions:
# – on push to any branch
# – on pull requests targeting the main branch
on:
  push:
    branches: ["**"] # "**" matches every branch, including feature/login;
                    # "*" would skip names that contain a slash
  pull_request:
    branches: ["main"]

jobs:
  build:
    # Run on the latest Ubuntu Linux (a fresh, clean virtual machine)
    runs-on: ubuntu-latest

    # Test against multiple Python versions to catch version-specific issues
    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12"]

    steps:
      # Step 1: Get the code
      - name: Check out repository
        uses: actions/checkout@v4
        # This downloads your repository's code to the CI machine

      # Step 2: Set up Python
      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
```

```

python-version: ${{ matrix.python-version }}
# Cache pip downloads to speed up subsequent runs
cache: "pip"
cache-dependency-path: |
  requirements.txt
  requirements-dev.txt

# Step 3: Install dependencies (requirements-dev.txt includes requirements.txt)
- name: Install dependencies
  run: pip install -r requirements-dev.txt

# Step 4: Run the build (lint + typecheck + test + security).
# No DATABASE_URL or SECRET_KEY is needed here: tests/conftest.py
# sets its own test values before the application is imported.
- name: Run build
  run: make build

# Step 5: Measure coverage and keep the report as an artifact
# (viewable in the GitHub Actions UI). This runs the suite a second
# time, which costs well under a second.
- name: Generate coverage report
  run: pytest --cov=app --cov-report=xml

- name: Upload coverage report
  uses: actions/upload-artifact@v4
  with:
    name: coverage-report-py${{ matrix.python-version }}
    path: coverage.xml

```

What this does:

Every time you push code to GitHub:

1. GitHub's servers spin up a fresh Ubuntu virtual machine
2. They install Python (three versions, tested in parallel)
3. They install your dependencies
4. They run `make build`
5. If any step fails, GitHub marks the commit as  and notifies you
6. If all steps pass, GitHub marks the commit as 

This is the fundamental loop of CI: push code → automated build → pass or fail → repeat.

The failing build as a safeguard

When CI fails, it tells you:

- Which specific step failed (lint, typecheck, test, or security)
- The exact error output
- Which Python version failed
- On which commit the failure occurred

A failing build is not a bad thing — it is the system working correctly. The goal is for every build to fail as early as possible in development, not after the code reaches production.

The team rule that makes CI effective: **never merge code that fails the build**. If your CI check is failing, you fix the code before merging. CI only *reports* the result; the rule is *enforced* by the branch protection of section 17.9, which requires these checks to pass before a pull request can be merged. Together they keep `main` in a deployable state.

This is also why the workflow runs on `pull_request` as well as on `push`. The pull request run tests the branch *as it would be once merged into main*, so the result the reviewer sees is the result `main` would get.

17.11 The Complete Updated Project Structure

After this chapter, the project structure is:

```

notes-api/
├── .github/
│   └── workflows/
│       └── build.yml          ← CI configuration
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── database.py
│   ├── errors.py
│   ├── logging_config.py
│   ├── middleware.py
│   ├── startup.py
│   └── routes/
│       ├── __init__.py
│       ├── health.py
│       └── notes.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py           ← pytest fixtures
│   ├── test_errors.py       ← Error handler tests
│   ├── test_health.py       ← Health endpoint tests
│   ├── test_notes.py        ← Notes endpoint tests
│   ├── test_startup.py      ← Startup checks, with a mock (section 17.8)
│   └── test_units.py        ← Unit tests for small functions (section 17.8)
├── Makefile                 ← Build automation
├── pyproject.toml           ← Tool configuration
├── requirements.txt         ← Production dependencies
├── requirements-dev.txt     ← Development dependencies
├── run.py
├── .env
├── .env.example
├── .python-version
└── .gitignore

```

Key Takeaways

- A **build process** is the defined, automated sequence of steps that transforms source code into a verified, deployable artifact. Automation prevents human error.
- For Python web applications, the build artifact is typically the source code itself (plus pinned dependencies) or a Docker image. There is no compiler in the Java sense.
- **Separate production and development dependencies:** `requirements.txt` for the server, `requirements-dev.txt` for developers. Testing tools do not belong in production.
- **pytest** is the standard Python testing framework. Tests are functions named `test_*` in files named `test_*.py`. `pytest` discovers and runs them automatically.

- **Fixtures** in pytest are reusable setup functions. `scope="session"` creates a fixture once per run; `scope="function"` creates it per test. `autouse=True` runs a fixture automatically without being explicitly requested.
- **The test client** (`app.test_client()`) sends HTTP requests to Flask without a running server. Tests are fast and self-contained.
- **Code quality tools**: `black` formats code, `ruff` lints for errors and style, `mypy` checks type annotations, `pip-audit` scans for security vulnerabilities.
- **make build** is the one command that runs the complete build: `lint` → `typecheck` → `test` → `security`. If any step fails, `Make` stops.
- **Continuous integration** (GitHub Actions, etc.) runs the build automatically on every push, providing immediate feedback on whether the build passes.
- **Test coverage** measures which lines are executed by tests. 80% is a reasonable minimum; 100% is not the goal — meaningful tests are.
- **Test each behaviour at the lowest level that can show it**: unit tests for small functions, integration tests through the test client for handlers, a few end-to-end checks of the running system. Every endpoint needs its normal case, bad input, missing things, edges, and its design decisions tested.
- **Parametrize** tests that differ only in data. Use **test doubles** at the edges of the system: stubs to force failures, fakes for speed, spies (`caplog`) to observe, mocks when a call itself is the behaviour. Know what each one hides, and prefer the real thing when it is cheap.
- **A flaky test is a bug**. Its causes are shared state, unordered results, time, randomness, timing and the network, and it gets fixed, not re-run. Fix bugs **test-first**: red, green, refactor.
- **main is always releasable**. Changes travel on short-lived branches, as small, single-purpose pull requests with clear messages, reviewed by a person for what machines cannot check, and merged only when CI passes, which branch protection enforces. Undo with `git revert`; mark releases with tags and semantic versions.

Exercises

1. **Predict**. Delete one line from a handler, a `db.commit()`, and run `make build`. Which step fails, and does the failure message point you to the cause?
2. **Break and fix**. Make one test depend on another by removing `clean_database`'s `autouse=True`. Install `pytest-randomly` (`pip install pytest-randomly`), which runs the tests in a new random order every time, and run the suite a few times; `-p no:randomly` turns the shuffling off again. Explain the flakiness, then restore `autouse=True`.
3. **Extend**. Write a parametrized test for `GET /notes/<id>` covering an existing id, a missing id, and a non-integer id, with the status you expect for each. (Look closely at

<int:note_id> before you decide the third.)

4. **Explain.** When would you reach for a fake instead of a mock? Use an example from this book.

Solutions and hints are in the companion repository, in `exercises/chapter-17.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

The build process produces a verified, tested application. The next step is packaging it in a form that is portable — that can be deployed to any server without “it works on my machine” problems.

Docker is that packaging system. A Docker image captures not just the application code, but the entire execution environment: the specific Python version, all installed packages, the directory structure, the startup command. Building a Docker image is the final step of the build process that makes the application genuinely portable.

→ [Chapter 18](#) — Containerization

Containerization

The Notes API now has pinned dependencies, externalized configuration, a runtime contract, a clean directory structure, structured logging, and a build process with automated tests. But it still runs as a raw Python process, directly on whatever machine it happens to be installed on. The environment — the Python version, the operating system libraries, the directory layout — is assumed but not guaranteed.

This assumption is the root cause of environment drift ([Chapter 7](#)), hidden dependencies ([Chapter 8](#)), and the “it works on my machine” problem ([Chapter 9](#)). Even with careful management, the gap between a developer’s laptop running macOS 14 with Python 3.11.8 and a production server running Debian 12 with Python 3.11.2 can cause subtle failures.

Containerization eliminates this gap by packaging the application together with its complete execution environment into a single, self-contained unit called a **container image**. The image includes:

- The operating system libraries
- The exact Python version
- All installed Python packages at their exact versions
- The application code
- The startup command

When this image runs — on a developer’s laptop, on a CI server, on a production server in Singapore — it runs identically. The execution environment is identical because it travels with the code. The “it works on my machine” problem disappears because “my machine” and “the server” are now running exactly the same image. (One gap remains: CPU architecture. An image built on an Apple Silicon laptop is `arm64`, while most servers are `x86-64`; `docker build --platform linux/amd64` builds for the server’s architecture.)

Docker is the dominant tool for building and running container images. This chapter explains what containers actually are at the operating system level, builds a production-quality Docker image for the Notes API, and establishes the `docker build + docker run` workflow.

In this chapter

By the end of this chapter, you will understand:

- What a Linux container is and how it differs from a virtual machine
- What a Dockerfile is and how Docker uses it to build an image
- How to write a production-quality Dockerfile for the Notes API
- How Docker layers work and why they matter for build speed
- How to run the containerized application with environment variables
- How `docker compose` simplifies multi-container development (application + database)

18.1 What a Container Is

Before writing any Docker files, it is important to understand what a container actually is. Many explanations describe containers with analogies (“like a lightweight VM”), but the accurate explanation is more precise and more useful.

The three Linux kernel mechanisms

A Linux container is an ordinary operating system process with three specific kernel mechanisms applied:

1. Namespaces — isolation

A Linux **namespace** restricts what a process can see. There are different types:

- **PID namespace:** the container process sees its own isolated process ID tree. Inside the container, the first process has PID 1. From outside, it has a different PID.
- **Network namespace:** the container has its own network interfaces, IP addresses, and routing table. Traffic is isolated from the host network and from other containers.
- **Mount namespace:** the container sees its own filesystem. The root (/) inside the container is not the same as the root on the host.
- **UTS namespace:** the container has its own hostname, different from the host’s hostname.

Namespaces restrict visibility. A process inside a namespace cannot see processes outside it. (The host, in the parent namespace, can still see every container’s processes — which is how `ps` on the server shows them.)

2. cgroups — resource limits

Control groups (cgroups) limit how much of a resource a process can consume:

- **CPU:** this container can use at most 0.5 CPU cores

- Memory: this container can use at most 512 MB RAM
- Disk I/O: this container can read/write at most 100 MB/s

Without cgroups, one container could consume all available memory and starve other containers. With cgroups, each container’s consumption is bounded.

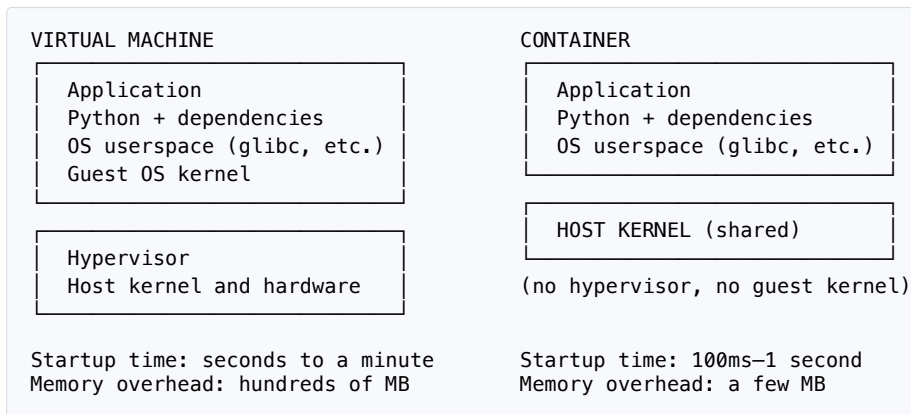
3. A layered filesystem (overlayfs)

The container’s filesystem is built from **layers**, stacked on top of each other. Each layer is read-only. When a process writes a file, the write goes to a writable layer on top of the read-only layers. The read-only layers are shared between all containers running the same image — only the writable layer is unique per container.

Containers vs virtual machines

A **virtual machine (VM)** virtualizes complete hardware: a **hypervisor**, helped by the CPU’s virtualization features, gives each VM its own virtual CPUs, RAM, and disk. A full operating system (kernel + userspace) runs inside the VM. The overhead is noticeable: each VM runs its own kernel and system services, typically costing hundreds of megabytes of RAM before any application starts.

A **container** shares the host’s kernel. No hardware emulation. No separate OS kernel. The container’s “OS” is just userspace files (libraries, binaries) — not a running kernel. The overhead is minimal: a container starts in milliseconds and uses only the RAM that the application itself needs.



Containers are faster, lighter, and more portable than VMs. They are not more secure (sharing the kernel means a kernel vulnerability can escape the isolation), but for most application deployment purposes, containers provide adequate isolation with dramatically lower overhead.

18.2 Docker: The Container Toolchain

Docker is the toolchain that makes containers practical for developers. It provides:

- **Docker daemon:** the background service that manages containers and images
- **Docker CLI:** the docker command you use in your terminal
- **Dockerfile:** a text file that describes how to build an image
- **Docker Hub:** a public registry where images are stored and shared
- **Docker Compose:** a tool for running multiple containers together

Installing Docker

On macOS and Windows, install **Docker Desktop** from docker.com. Docker Desktop includes the daemon, CLI, and Docker Compose.

On Linux, install Docker Engine:

```
# Ubuntu/Debian
sudo apt-get update
sudo apt-get install docker.io docker-compose-v2
sudo usermod -aG docker $USER # Allow running docker without sudo
# Log out and back in for group membership to take effect
```

Verify the installation:

```
docker --version
# Docker version 25.0.3, build 4debf41

docker run hello-world
# Hello from Docker! ...
```

18.3 The Dockerfile

A **Dockerfile** is a text file containing a sequence of instructions that Docker executes, one by one, to build an image. Each instruction that changes files (RUN, COPY, ADD) adds a layer to the image; the others (WORKDIR, ENV, CMD, ...) only record settings.

A minimal Dockerfile (what NOT to use in production)

```
# DO NOT USE – this is a bad Dockerfile for illustration purposes
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "run.py"]
```

This works but has serious problems:

- The `python:3.11` base image is enormous (~1 GB) — it includes compilers and build tools that the application does not need at runtime
- Copying `.` includes `.env`, `venv/`, `.git/`, and other files that should not be in the image
- No non-root user — the application runs as root inside the container, which is a security risk
- No separation between dependency installation and code copying — every code change rebuilds all layers including pip install

The production-quality Dockerfile

Listing 18.1 Dockerfile (production-quality)

```
# =====
# Multi-stage build for the Notes API
#
# A multi-stage build uses multiple FROM instructions. The first stage (builder)
# installs the Python dependencies. The second stage (final) copies only what
# is needed at runtime. Anything the builder needed only to *install* things
# (compilers, headers, pip's temporary files) never reaches the final image.
# =====

# -----
# Stage 1: builder
# Install the Python dependencies into a separate directory.
# -----
FROM python:3.11-slim AS builder

# No system packages are needed here: every dependency in requirements.txt
# ships as a prebuilt wheel (psycpg2-binary bundles its own copy of libpq).
# If a dependency ever has to be compiled from source (plain psycpg2, for
# example), its build tools are installed in THIS stage:
```

```

# RUN apt-get update && apt-get install -y --no-install-recommends \
#     gcc libpq-dev && rm -rf /var/lib/apt/lists/*
# and the final stage then needs only the runtime library (libpq5).

# Set the working directory for subsequent instructions in this stage.
# Nothing in /build is copied to the final image, so it does not appear there.
WORKDIR /build

# Copy ONLY the requirements file first.
# Docker caches each layer. If requirements.txt has not changed,
# Docker reuses the cached pip install layer from the previous build.
# This means code changes (which are copied later) do not trigger
# a full pip reinstall – only dependency changes do.
COPY requirements.txt .

# Install Python dependencies into a known location.
# --no-cache-dir: do not cache downloaded packages (saves space)
# --prefix=/install: install into /install instead of system site-packages
# This makes it easy to copy just the packages to the final image.
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# -----
# Stage 2: final
# The actual container image that will run in production.
# It contains the packages from stage 1, but none of stage 1's leftovers.
# -----
FROM python:3.11-slim AS final

# Create a non-root user for running the application.
# Running as root inside a container is a security risk:
# if the application is compromised, the attacker has root access
# inside the container (and potentially on the host in some configurations).
#
# useradd options:
# --create-home: create a home directory for the user
# --shell /bin/bash: set the login shell
# --uid 1001: set a specific UID (avoids conflicts with host UIDs)
# notes: the username
RUN useradd --create-home --shell /bin/bash --uid 1001 notes

# Copy the installed Python packages from the builder stage.
# COPY --from=builder copies files from the named stage.
# /install contains all the packages installed by pip in stage 1.
# /usr/local is where Python looks for packages by default.

```

```

COPY --from=builder /install /usr/local

# Set the working directory for the application.
# This is where the application code will live inside the container.
WORKDIR /app

# WORKDIR creates /app owned by root. Hand the directory itself to 'notes'
# so the application can create files in it (a SQLite database, say).
# This runs before the code is copied, so it stays cached.
RUN chown notes:notes /app

# Copy the application code, owned by the non-root user.
# Note: this is after copying requirements/installing packages.
# Changing application code only invalidates these layers,
# not the pip install layer above – builds are fast.
#
# --chown sets the owner as the files are copied. (A separate
# "RUN chown -R notes:notes /app" afterwards would re-run on every code
# change and store a second copy of every file in a new layer.)
#
# We use .dockerignore (below) to exclude files that should not be in the image.
COPY --chown=notes:notes app/ ./app/
COPY --chown=notes:notes run.py .

# Switch to the non-root user.
# All subsequent instructions and the running container use this user.
USER notes

# Document which port the application listens on.
# EXPOSE does not actually publish the port – it is documentation.
# The port is published when running the container with -p or in docker-compose.
EXPOSE 5000

# Set default environment variables.
# These are the defaults; they can be overridden at runtime with -e flags.
# Do NOT set SECRET_KEY or DATABASE_URL here – those are secrets and must
# be provided at runtime, not baked into the image.
# (HOST and PORT are not set: they are read only by run.py, and in the
# container Gunicorn's --bind, below, decides the address and port.)
ENV FLASK_DEBUG=false \
    LOG_FORMAT=json \
    LOG_LEVEL=INFO \
    PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1

```

```

# PYTHONUNBUFFERED=1: Python outputs are sent straight to the terminal
# without buffering. Without this, log output might not appear until
# a buffer fills – which means last log entries before a crash might
# never appear.
#
# PYTHONDONTWRITEBYTECODE=1: Python does not write .pyc files inside the
# container. These are unnecessary in a container (the image is immutable)
# and waste space and inode counts.

# Health check: Docker periodically runs this command to determine
# if the container is healthy. If it fails repeatedly, Docker marks
# the container as "unhealthy". Plain Docker only reports that status
# (in docker ps); orchestrators such as Docker Swarm or Kubernetes act
# on it by replacing the container.
#
# The slim image has no curl, so the check uses Python itself:
# urlopen() raises an exception, and python exits non-zero, if the
# request fails, times out, or returns an HTTP error status.
# interval=30s: check every 30 seconds
# timeout=5s: the command must complete within 5 seconds
# retries=3: after 3 consecutive failures, mark as unhealthy
# start-period=10s: give the container 10 seconds to start before checking
HEALTHCHECK --interval=30s --timeout=5s --retries=3 --start-period=10s \
  CMD python -c "import urllib.request;
  ↪ urllib.request.urlopen('http://localhost:5000/health', timeout=3)" || exit 1

# The command that runs when the container starts.
#
# We use Gunicorn instead of Flask's built-in development server.
# Gunicorn is a production WSGI server (Chapter 24) that:
# - Runs several worker processes to serve requests in parallel
#   (Flask's development server is not built for production traffic)
# - Restarts workers that crash or hang
# - Handles graceful shutdown correctly
#
# Gunicorn imports the app package and calls create_app() itself, so run.py
# is not used here. The startup checks still run, because create_app() runs
# them; only the port check (run.py's job) is skipped: Gunicorn binds the port.
#
# gunicorn arguments:
# "app:create_app()"      – import 'app', call 'create_app()' to get the Flask app
# --bind 0.0.0.0:5000      – listen on all interfaces (required in containers)
# --workers 2             – 2 worker processes (handles 2 concurrent requests)

```

```

# --timeout 30          - kill workers that take longer than 30 seconds
# --access-logfile -    - write access logs to stdout (captured by Docker)
# --error-logfile -     - write Gunicorn's own messages to stderr (captured by
↳ Docker)
# --log-level info      - log level for Gunicorn's own messages
CMD ["gunicorn", \
    "app:create_app()", \
    "--bind", "0.0.0.0:5000", \
    "--workers", "2", \
    "--timeout", "30", \
    "--access-logfile", "-", \
    "--error-logfile", "-", \
    "--log-level", "info"]

```

Add gunicorn to `requirements.txt` by hand, pinned like everything else:

```
gunicorn==22.0.0
```

Do not use `pip freeze > requirements.txt` here: your virtual environment now also contains the development tools from [Chapter 17](#), and freezing it would write `pytest`, `black`, `ruff`, and the rest into the production requirements — and into the image.

The `.dockerignore` file

`.dockerignore` tells Docker which files to exclude from the build context (the files sent to the Docker daemon). Without it, every file in the project — including `.git/`, `venv/`, `.env`, test files, and documentation — is included in the build context, slowing down builds and potentially including sensitive files in the image.

```

# .dockerignore
# Files and directories to exclude from the Docker build context.
# Syntax is the same as .gitignore.

# Python artifacts
venv/
__pycache__/
*.pyc
*.pyo
.pytest_cache/
.mypy_cache/
.ruff_cache/

```

```

htmlcov/
.coverage

# Environment files – NEVER include in the image
.env
.env.*
!.env.example

# Version control
.git/
.gitignore

# Development files
requirements-dev.txt
tests/
Makefile

# Documentation
*.md

# Editor and OS files
.DS_Store
*.swp
*.sw0
.idea/
.vscode/

```

18.4 How Docker Layers Work

Understanding Docker layers is important for writing efficient Dockerfiles. Every Dockerfile instruction that produces filesystem changes creates a new **layer**. Layers are:

- **Cached:** Docker caches each layer. If a layer’s instruction has not changed and all preceding layers are also unchanged, Docker reuses the cached layer — it does not re-run the instruction.
- **Immutable:** once created, layers are read-only. They are shared between images and containers.
- **Stacked:** the filesystem a container sees is all layers stacked together (using overlays).

Layer caching in practice

A simplified, single-stage version of Listing 18.1 shows the principle:

```

# Layer 1 – base image (cached after first pull)
FROM python:3.11-slim

# Layer 2 – create the user (cached until the RUN command changes)
RUN useradd --create-home --uid 1001 notes

# Layer 3 – requirements.txt copy (cached until requirements.txt changes)
COPY requirements.txt .

# Layer 4 – pip install (cached until requirements.txt changes)
RUN pip install -r requirements.txt

# Layer 5 – application code (cached until any app/ file changes)
COPY app/ ./app/
COPY run.py .

```

When you change only `app/routes/notes.py`:

- Layers 1–4: **cache hit** (reused, no re-execution)
- Layer 5: **cache miss** (COPY `app/` sees changed files, re-executes)

Build time: fraction of a second (only layer 5 is re-built).

If you had placed `COPY . .` before `pip install -r requirements.txt`:

- Any code change would invalidate the COPY layer
- The pip install layer follows the COPY layer and is also invalidated
- Build time: minutes (pip reinstalls all packages on every code change)

The order of Dockerfile instructions matters enormously for build speed. Always put things that change infrequently (base image, system packages, dependency files) before things that change frequently (application code).

18.5 Building and Running the Image

Building the image

```

# From the notes-api/ root directory
docker build -t notes-api:latest .

# -t notes-api:latest : tag the image with the name "notes-api" and version "latest"
# . : use the current directory as the build context

```

First build output (no cache):

```
[+] Building 21.4s (15/15) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load .dockerignore
=> [builder 1/4] FROM python:3.11-slim
=> [builder 2/4] WORKDIR /build
=> [builder 3/4] COPY requirements.txt .
=> [builder 4/4] RUN pip install --no-cache-dir --prefix=/install -r
↳ requirements.txt
=> [final 1/7] FROM python:3.11-slim
=> [final 2/7] RUN useradd --create-home --shell /bin/bash --uid 1001 notes
=> [final 3/7] COPY --from=builder /install /usr/local
=> [final 4/7] WORKDIR /app
=> [final 5/7] RUN chown notes:notes /app
=> [final 6/7] COPY --chown=notes:notes app/ ./app/
=> [final 7/7] COPY --chown=notes:notes run.py .
=> exporting to image
```

Second build (only code changed):

```
[+] Building 1.3s (15/15) FINISHED
=> CACHED [builder 4/4] RUN pip install --no-cache-dir
=> CACHED [final 3/7] COPY --from=builder /install /usr/local
=> CACHED [final 5/7] RUN chown notes:notes /app
=> [final 6/7] COPY --chown=notes:notes app/ ./app/ ← only this layer re-ran
=> [final 7/7] COPY --chown=notes:notes run.py . ← and this one
```

1.3 seconds instead of 21. Layer caching is working.

Running the container with SQLite

```
docker run \
  --name notes-api \
  --rm \
  -p 5000:5000 \
  -e DATABASE_URL="sqlite:///notes.db" \
  -e SECRET_KEY="your-secret-key-here-at-least-24-chars" \
  -e LOG_FORMAT=human \
  notes-api:latest
```

(`-e LOG_FORMAT=human` overrides the image's JSON default, so the output below is easy to

read.)

Breaking down the `docker run` command:

Flag	Meaning
<code>--name notes-api</code>	Name the container “notes-api” (instead of a random name)
<code>--rm</code>	Remove the container when it stops (cleanup)
<code>-p 5000:5000</code>	Map host port 5000 to container port 5000 (host:container)
<code>-e DATABASE_URL=...</code>	Set environment variable inside the container
<code>-e SECRET_KEY=...</code>	Set another environment variable
<code>-e LOG_FORMAT=human</code>	Override a default set by ENV in the Dockerfile
<code>notes-api:latest</code>	The image to run

After the container starts:

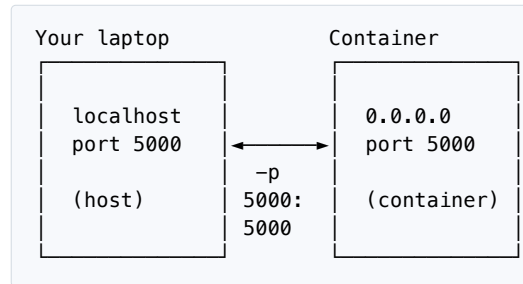
```
[2024-01-15 14:00:00 +0000] [1] [INFO] Starting gunicorn 22.0.0
[2024-01-15 14:00:00 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2024-01-15 14:00:00 +0000] [1] [INFO] Using worker: sync
[2024-01-15 14:00:00 +0000] [7] [INFO] Booting worker with pid: 7
[2024-01-15 14:00:00 +0000] [8] [INFO] Booting worker with pid: 8
2024-01-15 14:00:00.412 INFO      app.logging_config | Logging configured. level=INFO
↪  format=human log_dir=(console only)
2024-01-15 14:00:00.413 INFO      app.startup      | Running startup checks...
2024-01-15 14:00:00.413 INFO      app.startup      | All startup checks passed.
2024-01-15 14:00:00.414 INFO      app              | Creating Flask application...
2024-01-15 14:00:00.431 INFO      app.database    | Database initialized.
2024-01-15 14:00:00.432 INFO      app              | Flask application created
↪  successfully.
... (the same six lines again, from the second worker)
```

Gunicorn’s master process (PID 1 in the container) starts first and binds the port. Each worker then imports the application and calls `create_app()` itself — so the startup checks and `init_db()` run once per worker, and every application line appears twice. (CREATE TABLE IF NOT EXISTS makes the repeat harmless. Gunicorn’s `--preload` option would create the app once, in the master, instead.)

Test it:

```
curl http://localhost:5000/health
# {"status":"ok"}
```

The port mapping explained



The `-p 5000:5000` flag tells Docker to forward traffic arriving at the host's port 5000 to the container's port 5000. Without this flag, the container's port is not accessible from outside the container.

Inside the container, Gunicorn binds to `0.0.0.0:5000` (all interfaces). The `--bind 0.0.0.0:5000` argument in the Dockerfile's CMD is what causes this — not `127.0.0.1` (loopback only, which inside a container means *the container's own* loopback), but `0.0.0.0` (all interfaces, so Docker's port forwarding can reach it).

18.6 Docker Compose: Running Multiple Containers Together

The Notes API depends on a database. In development, it uses SQLite (a local file). In production, it uses PostgreSQL. Running both the application and PostgreSQL in Docker containers — and making them communicate — requires Docker Compose.

Docker Compose is a tool for defining and running multi-container applications using a YAML file.

docker-compose.yml

Listing 18.2 docker-compose.yml

```

# Define the multi-container development environment for the Notes API.
#
# Usage:
#   docker compose up          - start all services
#   docker compose up -d      - start in background (detached mode)
#   docker compose down       - stop and remove containers
#   docker compose logs -f    - follow logs from all services
#   docker compose exec app bash - open a shell inside the app container
#
# (Older Compose files start with a "version:" line. Compose v2 ignores it
# and prints a warning, so it is left out.)

services:

# -----
# The PostgreSQL database service
# -----

db:
  # Use the official PostgreSQL 15 image from Docker Hub
  image: postgres:15-alpine
  # 'alpine' is a minimal Linux distribution - the image is roughly half the size

  # Named volume for persistent storage.
  # Without this, the database contents are lost when the container is
  # removed or recreated (docker compose down, then up).
  # With this, data persists in a Docker-managed volume.
  volumes:
    - postgres_data:/var/lib/postgresql/data

  # Environment variables for the PostgreSQL container.
  # POSTGRES_USER: the database user to create
  # POSTGRES_PASSWORD: the password for that user
  # POSTGRES_DB: the database to create
  environment:
    POSTGRES_USER: notes_user
    POSTGRES_PASSWORD: notes_password
    POSTGRES_DB: notesdb

  # Health check for PostgreSQL.
  # 'pg_isready' is a PostgreSQL utility that checks if the server is
  # accepting connections. The app service will not start until this passes.
  healthcheck:

```

```

    test: ["CMD-SHELL", "pg_isready -U notes_user -d notesdb"]
    interval: 5s
    timeout: 5s
    retries: 5

    # Expose PostgreSQL's port only to other containers in this compose file.
    # Do NOT use ports: here (which would expose it to the host) unless you
    # need to connect with a local psql client.
    expose:
      - "5432"

# -----
# The Notes API application service
# -----

app:
    # Build the image from the Dockerfile in the current directory.
    # 'context: .' means use the current directory as the build context.
    build:
      context: .
      dockerfile: Dockerfile

    # Port mapping: host port 5000 → container port 5000
    ports:
      - "5000:5000"

    # Environment variables for the application.
    # These override the ENV defaults set in the Dockerfile.
    environment:
      # Construct the DATABASE_URL using the 'db' service name as the hostname.
      # Docker Compose creates a shared network where services can reach each
      # other by their service name. 'db' resolves to the PostgreSQL container's
      # IP address inside the Docker network.
      DATABASE_URL: postgresql://notes_user:notes_password@db:5432/notesdb

      # In a real project, SECRET_KEY would come from a secrets manager,
      # not be hardcoded here. For development, this is acceptable.
      SECRET_KEY: dev-secret-key-change-in-production-12345

      FLASK_DEBUG: "false"
      LOG_FORMAT: human
      LOG_LEVEL: INFO

    # Wait for the database to be healthy before starting.
    # 'depends_on' with 'condition: service_healthy' ensures Docker waits

```

```

# until the db healthcheck passes before starting the app.
depends_on:
  db:
    condition: service_healthy

# Restart policy: if the container crashes, restart it.
# 'on-failure' restarts only if the container exits with a non-zero code.
restart: on-failure

# Named volumes: Docker manages these on disk.
# They persist between 'docker compose down' and 'docker compose up'.
# To delete them: docker compose down -v
volumes:
  postgres_data:

```

Running with Docker Compose

```

# Start both services
docker compose up

# Expected output (abridged):
# [+] Running 2/2
# ✓ Container notes-api-db-1 Started
# ✓ Container notes-api-app-1 Started
#
# notes-api-db-1 | ... database system is ready to accept connections
# notes-api-app-1 | [...] [1] [INFO] Starting gunicorn 22.0.0
# notes-api-app-1 | [...] [7] [INFO] Booting worker with pid: 7
# notes-api-app-1 | [...] [8] [INFO] Booting worker with pid: 8
# notes-api-app-1 | ... INFO      app.startup      | Running startup checks...
# notes-api-app-1 | ... INFO      app.startup      | Database connectivity: OK
# notes-api-app-1 | ... INFO      app.startup      | All startup checks passed.
# notes-api-app-1 | ... INFO      app.database      | Database initialized.
# (each application line appears once per worker)

```

Test it:

```
# Create a note
curl -X POST http://localhost:5000/notes/ \
  -H "Content-Type: application/json" \
  -d '{"content": "Running in a container!"}'

# {"content":"Running in a
  ↳ container!","created_at":"2024-01-15T14:00:05.123456+00:00","id":1}

# Retrieve it
curl http://localhost:5000/notes/
# {"count":1,"notes":[{"content":"Running in a
  ↳ container!","created_at":"2024-01-15T14:00:05.123456+00:00","id":1}]}
```

Stop:

```
docker compose down
```

The data persists in the `postgres_data` volume. Running `docker compose up` again brings it back. (Flask's `jsonify` sorts keys alphabetically, which is why `content` comes before `id`; PostgreSQL's timestamps carry microseconds, SQLite's do not.)

18.7 Image Tagging and Versioning

The tag `latest` is convenient but dangerous for deployment: it changes on every build, making it impossible to know which exact version is running on a server.

For production, tag images with a specific version identifier:

```
# Tag with git commit hash – uniquely identifies the exact code version
GIT_HASH=$(git rev-parse --short HEAD)
docker build -t notes-api:sha-${GIT_HASH} -t notes-api:latest .

# Example: notes-api:sha-a1b2c3d
```

Using the git commit hash as the image tag creates an unambiguous link between the running container and the exact commit it was built from. When a production incident occurs, you know the exact code version running. The `sha-` prefix is the convention this book uses everywhere (it is also what the CI tooling in [Chapter 19](#) produces), and it keeps a tag like `sha-1234567` from being mistaken for a version number. Tags are always lowercase. Deploy only these commit tags — never `latest`.

Adding version info to the image

Record build metadata in the image using labels:

```
# Add to Dockerfile, near the top of the final stage
ARG GIT_HASH=unknown
ARG BUILD_DATE=unknown

LABEL org.opencontainers.image.revision="${GIT_HASH}" \
      org.opencontainers.image.created="${BUILD_DATE}" \
      org.opencontainers.image.title="notes-api" \
      org.opencontainers.image.description="Notes REST API"
```

Build with the metadata:

```
GIT_HASH=$(git rev-parse --short HEAD)
BUILD_DATE=$(date -u +"%Y-%m-%dT%H:%M:%SZ")

docker build \
  --build-arg GIT_HASH=${GIT_HASH} \
  --build-arg BUILD_DATE=${BUILD_DATE} \
  -t notes-api:sha-${GIT_HASH} \
  .
```

Inspect the metadata later:

```
docker inspect --format '{{json .Config.Labels}}' notes-api:sha-a1b2c3d
# {"org.opencontainers.image.created":"2024-01-15T14:00:00Z",
#  "org.opencontainers.image.description":"Notes REST API",
#  "org.opencontainers.image.revision":"a1b2c3d",
#  "org.opencontainers.image.title":"notes-api"}
```

18.8 Integrating Docker into the Build Process

Add Docker build targets to the Makefile:

```
# Additions to Makefile – Docker targets

GIT_HASH := $(shell git rev-parse --short HEAD 2>/dev/null || echo "unknown")
BUILD_DATE := $(shell date -u +"%Y-%m-%dT%H:%M:%SZ")
IMAGE_NAME := notes-api
IMAGE_TAG := sha-${GIT_HASH}
```

```

.PHONY: docker-build docker-run docker-compose-up docker-compose-down docker-clean
→ release

# Build the Docker image
docker-build:
    @echo "→ Building Docker image $(IMAGE_NAME):$(IMAGE_TAG)..."
    docker build \
        --build-arg GIT_HASH=$(GIT_HASH) \
        --build-arg BUILD_DATE=$(BUILD_DATE) \
        -t $(IMAGE_NAME):$(IMAGE_TAG) \
        -t $(IMAGE_NAME):latest \
        .
    @echo "✓ Image built: $(IMAGE_NAME):$(IMAGE_TAG)"

# Run the container with SQLite (quick local test)
docker-run:
    docker run --rm \
        -p 5000:5000 \
        -e DATABASE_URL="sqlite:///notes.db" \
        -e SECRET_KEY="docker-test-secret-key-1234" \
        $(IMAGE_NAME):latest

# Start the full stack (app + PostgreSQL)
docker-compose-up:
    docker compose up -d
    @echo "✓ Stack started. API available at http://localhost:5000"

# Stop the full stack
docker-compose-down:
    docker compose down

# Complete release build: run all checks, then build the Docker image
release: build docker-build
    @echo ""
    @echo "===== "
    @echo "✓ Release build complete."
    @echo " Image: $(IMAGE_NAME):$(IMAGE_TAG)"
    @echo "===== "

# Remove all Docker images for this project (every tag, not just latest)
docker-clean:
    docker rmi -f $$$(docker images -q $(IMAGE_NAME)) 2>/dev/null || true
    @echo "✓ Docker images removed."

```


Now:

```
make release
```

Runs the complete pipeline: lint → typecheck → test → security → docker build. The output is a Docker image built from code that passed every check, tagged with the current git commit hash.

18.9 Adding Docker Tests to CI

Update the GitHub Actions workflow to also test the Docker build:

```
# Additions to .github/workflows/build.yml

docker:
  name: Docker Build Test
  runs-on: ubuntu-latest
  needs: build # Only run if the build job passes

steps:
  - name: Check out repository
    uses: actions/checkout@v4

  - name: Build Docker image
    run: |
      docker build \
        --build-arg GIT_HASH=$(git rev-parse --short HEAD) \
        --build-arg BUILD_DATE=$(date -u +"%Y-%m-%dT%H:%M:%SZ") \
        -t notes-api:ci-test \
        .

  - name: Test Docker image starts correctly
    run: |
      # Start the container in the background
      docker run -d \
        --name notes-api-test \
        -p 5000:5000 \
        -e DATABASE_URL="sqlite:///notes.db" \
        -e SECRET_KEY="ci-test-secret-key-not-for-production-1234" \
        notes-api:ci-test

      # Test the health endpoint, retrying while the container starts
```

```
# (a fixed "sleep 5" is flaky on a slow runner)
curl --fail --retry 10 --retry-delay 1 --retry-connrefused \
  http://localhost:5000/health

# Stop and remove the container
docker stop notes-api-test
docker rm notes-api-test
```

18.10 The Complete Updated Project Structure

```
notes-api/
├── .github/
│   └── workflows/
│       └── build.yml          ← CI (updated with Docker job)
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── database.py
│   ├── errors.py
│   ├── logging_config.py
│   ├── middleware.py
│   ├── startup.py
│   └── routes/
│       ├── __init__.py
│       ├── health.py
│       └── notes.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_errors.py
│   ├── test_health.py
│   └── test_notes.py
├── Dockerfile                ← Multi-stage Docker image definition (NEW)
├── .dockerignore              ← Files excluded from Docker build (NEW)
├── docker-compose.yml        ← Multi-container development setup (NEW)
├── Makefile                   ← Build automation (updated with Docker targets)
├── pyproject.toml
├── requirements.txt           ← Updated with gunicorn
├── requirements-dev.txt
├── run.py
├── .env
├── .env.example
├── .python-version
└── .gitignore
```

Key Takeaways

- A **container** is a Linux process with three kernel mechanisms: namespaces (isolation), cgroups (resource limits), and a layered filesystem (shared, immutable layers). Containers share the host kernel — they are not virtual machines.
- **Docker** is the toolchain for building and running containers. A **Dockerfile** describes how to build an image. A **Docker image** is the immutable artifact. A **container** is a running instance of an image.
- **Multi-stage builds** keep the final image small and secure: a builder stage installs (and, if necessary, compiles) dependencies; the final stage copies only what is needed at runtime. Build tools such as a compiler never appear in the production image.
- **Layer caching** makes builds fast. Copy `requirements.txt` and run `pip install` *before* copying application code — then code changes do not trigger a full `pip` reinstall.
- **Never run as root** inside a container. Create a dedicated non-root user, give it the application files with `COPY --chown`, and switch to it with `USER`.
- **`PYTHONUNBUFFERED=1`** ensures log output appears immediately, not when a buffer fills. Critical for seeing the last log line before a crash.
- Use **Gunicorn** (not `app.run()`) as the container's CMD. Gunicorn is a production WSGI server that handles concurrent requests and manages workers.
- **Docker Compose** defines multi-container setups (application + database) in a single YAML file. Services reach each other by service name (`db` resolves to the PostgreSQL container's IP).
- **Tag images with the git commit hash** (e.g., `notes-api:sha-a1b2c3d`) so you always know exactly which code is running in any environment.
- **make release** is the complete pipeline: tests pass → Docker image built → artifact ready to deploy.

Exercises

1. **Predict.** Change one line of `app/routes/notes.py` and rebuild the image. Which layers are rebuilt, and which come from the cache? Now change `requirements.txt` instead.
2. **Break and fix.** Remove `USER notes` from the Dockerfile, build, and run `docker exec notes-api id`. Why is this dangerous? Restore it.
3. **Extend.** Measure the image size with `docker images`, then try `python:3.11-alpine` as the base. Did it get smaller? Did everything still work?
4. **Explain.** What exactly is shared between two containers started from the same image, and what is not?

Solutions and hints are in the companion repository, in `exercises/chapter-18.md`. Try each

exercise before reading its solution: the point is the thinking, not the answer.

What's Next

The Notes API is now packaged in a Docker image that runs the same way everywhere. But the image exists only on the machine that built it. How does it get from your laptop — or from CI — to the server that will run it? Where are images kept, how are they named, and how do you know which image is which a month from now?

→ [Chapter 19](#) — Artifact Storage

Artifact Storage

Chapter 18 produced a Docker image. `make release` runs the full build, all tests pass, and the image exists on your laptop tagged as `notes-api:sha-a1b2c3d`. Now what?

The image needs to reach the production server. The production server is not your laptop. You cannot simply run the image there because it does not exist there. Someone (or something — a deployment script, a CI system) has to get the image from where it was built to where it needs to run.

The mechanism for this is a **container registry**: a server that stores Docker images and makes them available to be pulled. Your laptop pushes the image to the registry; the production server pulls the image from the registry. The registry is the intermediary that bridges the gap between “where images are built” and “where images run.”

Beyond the registry, there is a broader concept: **artifact storage**. An artifact is any output of the build process — a Docker image, a compiled binary, a `.whl` Python package, a tarball of source code, a test report. Artifact storage systems store these outputs, associate them with a specific version (a git commit, a version number, a build ID), and make them retrievable on demand.

Artifact storage solves a problem that becomes critical in any serious deployment operation: reproducibility and rollback. If the production server is running image `sha-a1b2c3d` and something goes wrong, you need to be able to roll back to the previous image `sha-9f8e7d6`. If the images are not stored anywhere, rollback is impossible — you would have to rebuild the previous version (which requires the previous source code and a working build system). With artifact storage, rollback is one command: pull and run the previous image.

In this chapter

By the end of this chapter, you will understand:

- What a container registry is and how it works
- The difference between public registries (Docker Hub) and private registries
- How to push images to and pull images from a registry
- How to tag images systematically so you can find any historical version
- How the registry fits into the complete build-to-deployment pipeline
- What other types of artifacts beyond Docker images need to be stored

19.1 What a Container Registry Is

A **container registry** is a server that stores Docker images and serves them over the Docker registry protocol. At its core, it is a specialized object store with an API designed specifically for Docker images.

The interaction model is simple:

```
PUSH (build machine → registry)
docker push myregistry.io/notes-api:sha-a1b2c3d

PULL (production server → registry)
docker pull myregistry.io/notes-api:sha-a1b2c3d
```

Every Docker image has a **fully qualified name** that specifies where it lives:

```
registry-host / repository-name : tag
|           |           |
|           |           └─ Version identifier
|           └────────── Repository (often org/project)
└────────────────── Registry server (omit for Docker Hub)
```

Examples:

```

docker.io/library/python:3.11-slim
    ← Python official image on Docker Hub
ghcr.io/myorg/notes-api:sha-a1b2c3d
    ← GitHub Container Registry
123456789.dkr.ecr.us-east-1.amazonaws.com/notes-api:sha-a1b2c3d
    ← AWS ECR
us-docker.pkg.dev/my-project/apps/notes-api:sha-a1b2c3d
    ← Google Artifact Registry

```

How image layers are stored

Docker images are not stored as single files. They are stored as individual layers, each identified by its SHA-256 hash (a cryptographic fingerprint of the layer's contents). The registry stores each layer once, regardless of how many images reference it.

When you push `notes-api:sha-a1b2c3d` and then push `notes-api:sha-b2c3d4e` (built from mostly the same layers), Docker only uploads the layers that changed. The `python:3.11-slim` base layers, the user-creation layer, and the installed-packages layer are already in the registry from the previous push — only the new application code layers are uploaded.

This is called **layer deduplication** and it dramatically reduces the bandwidth and storage required for a registry serving many image versions.

19.2 Registry Options

Docker Hub (docker.io)

The default public registry. Every image name without a registry hostname refers to Docker Hub:

```
docker pull python:3.11    # Actually: docker.io/library/python:3.11
```

Docker Hub offers:

- Free public repositories (publicly accessible)
- A limited number of private repositories on the free plan
- Paid plans for more private repositories and higher limits

Docker Hub rate-limits pulls, most strictly for anonymous clients, and the limits have been tightened more than once. A busy CI system that pulls base images anonymously can hit them, so log CI in to Docker Hub even for public images. (Check Docker's current pricing page for

the exact numbers.) Most teams keep their own images in a private registry.

GitHub Container Registry (GHCR)

If your code is on GitHub, GHCR is the natural choice:

- Integrated with GitHub permissions (repo access = registry access)
- Free for public repositories
- Private images count against your account's GitHub Packages storage and transfer quotas
- Images at `ghcr.io/YOUR-USERNAME/IMAGE-NAME`

AWS Elastic Container Registry (ECR)

If your application runs on AWS:

- Each AWS account has its own private registry
- Authentication uses IAM (AWS's identity system) — EC2 instances can pull images without managing credentials
- Pay per storage and data transfer (500 MB/month of storage is free for the first 12 months, on the AWS Free Tier)
- Images at `123456789.dkr.ecr.REGION.amazonaws.com/IMAGE-NAME`

Google Artifact Registry

If your application runs on Google Cloud:

- Similar to AWS ECR — tightly integrated with Google Cloud IAM
- Supports Docker images, Python packages, npm packages, and more (it is a general artifact store, not just for Docker)
- Images at `REGION-docker.pkg.dev/PROJECT-ID/REPOSITORY/IMAGE-NAME`

Running a private registry locally

For development and testing, you can run a registry on your own machine:


```
# Start a local registry on port 5001
docker run -d -p 5001:5000 --name local-registry registry:2

# Push to it
docker tag notes-api:latest localhost:5001/notes-api:latest
docker push localhost:5001/notes-api:latest

# Pull from it
docker pull localhost:5001/notes-api:latest
```

This is useful for testing the push/pull workflow without a cloud account.

19.3 Pushing to GitHub Container Registry

GHCR is the easiest option if your code is already on GitHub. Here is the complete workflow.

Step 1: Authenticate

```
# Log in to GHCR using a GitHub Personal Access Token (PAT)
# Create a PAT at: https://github.com/settings/tokens
# Required scope: write:packages (it includes read:packages).
# Do not grant delete:packages: pushing never needs it.

echo $GITHUB_TOKEN | docker login ghcr.io -u YOUR-GITHUB-USERNAME --password-stdin
# Login Succeeded
```

Step 2: Tag the image with the full GHCR address

```
# Your image must be tagged with the full registry address before pushing.
# Format: ghcr.io/GITHUB-USERNAME/IMAGE-NAME:TAG
# GHCR names must be all lowercase: write your username in lowercase
# even if it has capitals on GitHub, or the push is rejected.

GIT_HASH=$(git rev-parse --short HEAD)

docker tag notes-api:sha-${GIT_HASH} ghcr.io/your-username/notes-api:sha-${GIT_HASH}
docker tag notes-api:sha-${GIT_HASH} ghcr.io/your-username/notes-api:latest
```

Step 3: Push

```
docker push ghcr.io/your-username/notes-api:sha-${GIT_HASH}
docker push ghcr.io/your-username/notes-api:latest
```

Output:

```
The push refers to repository [ghcr.io/your-username/notes-api]
7f3e21c9d0aa: Pushed
5bd08e4f1c62: Pushed
0e9a4d7b3f15: Layer already exists   ← python:3.11-slim base layers
91c6f2e8a4d3: Layer already exists
c47b1a0e5f28: Layer already exists
sha-a1b2c3d: digest: sha256:4f0c9e... size: 2847
```

“Layer already exists” confirms layer deduplication: the unchanged layers from the base image are already in the registry. (The hexadecimal names on the left are layer digests, unrelated to git commit hashes.)

Step 4: Pull (on any machine)

```
# On the production server – no build required, just pull and run
docker pull ghcr.io/your-username/notes-api:sha-a1b2c3d
docker run -d \
  -p 127.0.0.1:5000:5000 \
  -e DATABASE_URL=postgresql://... \
  -e SECRET_KEY=... \
  ghcr.io/your-username/notes-api:sha-a1b2c3d
```

19.4 Integrating Registry Push into CI

Chapter 18 added a Docker build and a smoke test to CI. Now extend the pipeline to push the image to the registry automatically when code is merged to main.

Replace `.github/workflows/build.yml` with the workflow below. It keeps the ideas of Chapters 17 and 18 but reorganizes them: tests run on the production Python version only (add the Chapter 17 matrix back if you support several), and the Docker smoke test moves into the publish job, so the image that is pushed is the image that was just tested.

Listing 19.1 Updated GitHub Actions workflow with registry push

```

name: Build, Test, and Publish

on:
  push:
    branches: ["main"]      # Full pipeline (including push) on main
  pull_request:
    branches: ["main"]      # Tests only (no push) on pull requests

# Required for pushing to GHCR
permissions:
  contents: read
  packages: write

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${github.repository} # e.g., "your-username/notes-api"
  # (GHCR requires lowercase names; metadata-action lowercases this for you)

jobs:

  build-and-test:
    name: Build and Test
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"
          cache-dependency-path: |
            requirements.txt
            requirements-dev.txt

      - name: Install dependencies
        run: pip install -r requirements-dev.txt

      - name: Run build (lint + typecheck + test + security)
        run: make build

```

```

publish:
  name: Build and Push Docker Image
  runs-on: ubuntu-latest
  needs: build-and-test          # Only publish if tests pass
  # Only push on pushes to main (not pull requests)
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'

  steps:
    - name: Check out repository
      uses: actions/checkout@v4

    - name: Extract metadata for Docker
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
        tags: |
          # Tag with the short git SHA (e.g., sha-a1b2c3d)
          type=sha,prefix=sha-
          # Tag with 'latest' on the main branch
          type=raw,value=latest,enable={{is_default_branch}}
          # The generated labels include org.opencontainers.image.revision
          # (the full commit SHA) and .created (the build time); they
          # override the LABEL defaults in the Dockerfile.

    - name: Build the image for testing
      uses: docker/build-push-action@v5
      with:
        context: .
        load: true                # Keep the image in the local Docker, don't push
        tags: notes-api:ci-test
        cache-from: type=gha
        cache-to: type=gha,mode=max

    - name: Smoke-test the image
      run: |
        docker run -d --name smoke -p 5000:5000 \
          -e DATABASE_URL="sqlite:///notes.db" \
          -e SECRET_KEY="ci-test-secret-key-not-for-production-1234" \
          notes-api:ci-test
        curl --fail --retry 10 --retry-delay 1 --retry-connrefused \
          http://localhost:5000/health
        docker rm -f smoke

```

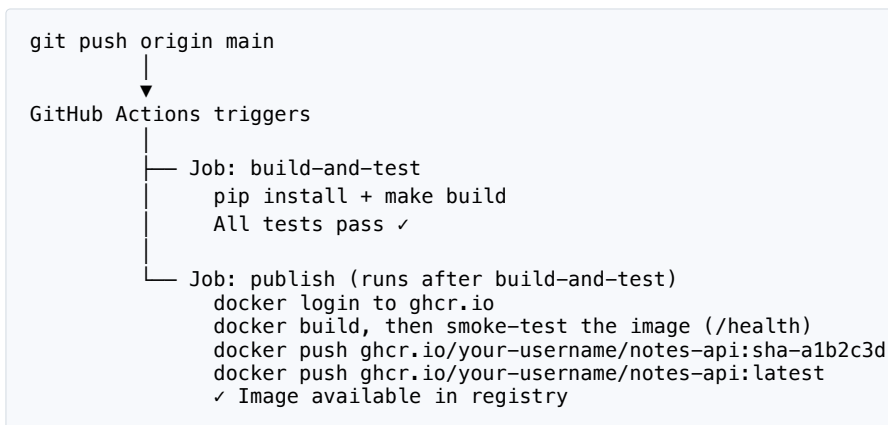
```

- name: Log in to GitHub Container Registry
  uses: docker/login-action@v3
  with:
    registry: ${ env.REGISTRY }}
    username: ${ github.actor }}
    password: ${ secrets.GITHUB_TOKEN }}
    # GITHUB_TOKEN is automatically available in Actions – no setup needed

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: ${ steps.meta.outputs.tags }}
    labels: ${ steps.meta.outputs.labels }}
    # Every layer comes from the cache filled by the test build above,
    # so the pushed image contains exactly what was smoke-tested
    cache-from: type=gha

```

The complete flow when you merge to main:



19.5 The Tagging Strategy

Image tags determine how you identify and deploy specific versions. A consistent tagging strategy is critical.

What NOT to do: latest only

```
# Bad: every build overwrites latest
docker build -t ghcr.io/org/notes-api:latest .
docker push ghcr.io/org/notes-api:latest

# On the server
docker pull ghcr.io/org/notes-api:latest
docker run ghcr.io/org/notes-api:latest
```

Problem: if you run `docker pull notes-api:latest` two weeks apart, you might get different images. You cannot tell which code is running. Rollback means “pull the old latest” — but latest no longer refers to the old version.

What to do: commit hash + latest

```
# Good: each build gets a unique tag, named after its commit
SHORT_HASH=$(git rev-parse --short HEAD) # 7-character abbreviation
IMAGE=ghcr.io/org/notes-api # always include the registry

docker build \
  -t ${IMAGE}:sha-${SHORT_HASH} \
  -t ${IMAGE}:latest \
  .

docker push ${IMAGE}:sha-${SHORT_HASH}
docker push ${IMAGE}:latest
```

(Without the registry prefix, `docker push notes-api:...` would try to push to Docker Hub’s `library/` namespace, which is reserved for official images, and fail.)

Rules:

- **notes-api:sha-a1b2c3d**: refers to the image built from commit `a1b2c3d`. Treat it as immutable: never re-point it. Registries do not enforce this by default — a tag is just a movable name — so turn on tag immutability where the registry offers it (ECR does), or deploy by **digest** (`notes-api@sha256:4f0c9e...`), which can never refer to anything else.
- **notes-api:latest**: mutable. Always refers to the most recently built image from the main branch.

Deploy by specific hash, not latest:

```
# Deploy the specific version – predictable and auditable
docker run ghcr.io/org/notes-api:sha-a1b2c3d

# Rollback: deploy the previous version
docker run ghcr.io/org/notes-api:sha-9f8e7d6
```

Semantic versioning (for released versions)

For projects that have explicit releases, add semantic version tags:

```
# For release v1.2.3:
docker build \
  -t notes-api:1.2.3 \
  -t notes-api:1.2 \
  -t notes-api:1 \
  -t notes-api:latest \
  .
```

- `notes-api:1.2.3` — immutable, this exact patch release
- `notes-api:1.2` — points to the latest patch of 1.2 (mutable as patches are released)
- `notes-api:1` — points to the latest minor release of major version 1 (mutable)
- `notes-api:latest` — the most recent build from `main`, as in the rest of this chapter (projects with formal releases often point it at the newest stable release instead; decide once, and write it down)

This is the strategy used by the official Python image: `python:3.11.8` is pinned to one patch release, `python:3.11` moves with each patch, and `python:3` moves with each minor release.

19.6 Beyond Docker Images: Other Artifacts

Docker images are the primary artifact for containerized applications, but the concept of artifact storage applies broadly.

Python package artifacts (.whl files)

If the Notes API were a library that other Python projects import, the build would produce a `wheel (.whl)` file. (This is illustrative: the Notes API is an application, and its `pyproject.toml` is not set up for packaging.)

```
# Build a wheel
pip install build
python -m build
# Output: dist/notes_api-0.1.0-py3-none-any.whl

# Upload to a package index
pip install twine
twine upload dist/*
```

Public packages go to **PyPI** (pypi.org). Private packages go to private package indexes: **Nexus**, **Artifactory**, **AWS CodeArtifact**, or **Google Artifact Registry** (which handles Python packages as well as Docker images).

Database migrations as artifacts

When the database schema changes, the change is described in a **migration file**:

```
migrations/
├── 0001_initial_schema.sql
└── 0002_add_updated_at.sql
```

Migration files are stored in the git repository (they are source code, not build outputs), but the record of which migrations have been applied to which database is stored in the database itself. Tools like **Alembic** (for projects that use SQLAlchemy) or **Flyway** (for any SQL database) manage this automatically. The Notes API does not have migrations yet: `init_db()` creates the table with `CREATE TABLE IF NOT EXISTS`, which cannot change a table that already exists. Its first real schema change — the `updated_at` column in [Chapter 33](#) — is where that approach runs out, and where Chapter 33 builds a small migration runner of its own.

Build logs and test reports as artifacts

CI systems store build logs and test reports so you can diagnose failures after the fact:


```
# In GitHub Actions – produce the reports, then store them for 30 days
- name: Run tests with reports
  run: pytest --junitxml=test-results.xml --cov=app --cov-report=xml

- name: Upload test report
  uses: actions/upload-artifact@v4
  with:
    name: test-report
    path: |
      test-results.xml
      coverage.xml
    retention-days: 30
```

19.7 Image Lifecycle Management

Images accumulate. Every build produces a new image. Without lifecycle management, the registry fills up with thousands of images, most of which will never be used.

Retention policies

Most registries support **retention policies** — rules that automatically delete old images:

Rule examples:

- Delete any image older than 90 days that is not tagged `latest`
- Keep only the 10 most recent images per repository
- Delete untagged images (leftover from failed builds) after 1 day

GitHub Container Registry has no built-in retention rules; teams run a scheduled workflow that deletes old image versions (for example with the `actions/delete-package-versions` action).

In AWS ECR, a lifecycle policy expresses the rules directly. Rules are checked in priority order, and an image matched by a higher-priority rule is never expired by a lower-priority one — which is how the first rule below protects production images (see the next section):

```
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Keep images deployed to production",
      "selection": {
```

```

        "tagStatus": "tagged",
        "tagPrefixList": ["prod-"],
        "countType": "imageCountMoreThan",
        "countNumber": 1000
    },
    "action": {
        "type": "expire"
    }
},
{
    "rulePriority": 2,
    "description": "Keep last 10 images",
    "selection": {
        "tagStatus": "tagged",
        "tagPrefixList": ["sha-"],
        "countType": "imageCountMoreThan",
        "countNumber": 10
    },
    "action": {
        "type": "expire"
    }
}
]
}

```

Images that should never be deleted

Production-deployed images must never be deleted by retention policies. Before deleting any image, check that it is not currently running anywhere.

One pattern: tag production-deployed images with a `prod-YYYYMMDD` tag that the retention policy excludes:

```

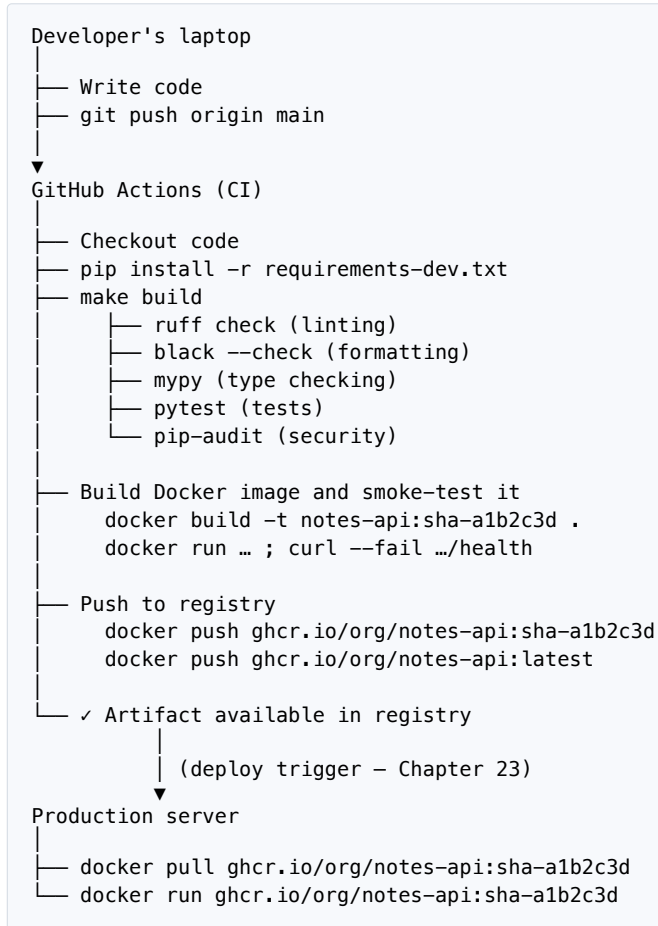
# When deploying to production:
docker tag ghcr.io/org/notes-api:sha-a1b2c3d ghcr.io/org/notes-api:prod-20240115
docker push ghcr.io/org/notes-api:prod-20240115

```

The image now carries both tags. In ECR, the higher-priority `prod-` rule above claims it, so the `sha-` rule can no longer expire it. With GHCR, the cleanup workflow must be told to skip versions that have a `prod-` tag.

19.8 The Complete Artifact Flow

Bringing together everything from Chapter 17 through 19:



The critical property of this flow: **the artifact that runs in production is the same artifact CI built and checked.** The code is tested, the image is built from that same commit and smoke-tested, pushed to the registry, and then pulled and run in production. No rebuilding. No “build it on the production server.” As long as nobody re-points the `sha-a1b2c3d` tag (section 19.5), it always refers to the same image.

19.9 The Updated Makefile

```
# Additions to Makefile – registry operations
# (Replace the release target from Chapter 18 with the one below;
# defining a target's recipe twice makes make warn and use the last one.)

REGISTRY := ghcr.io
# Replace with your GitHub username, in lowercase. (Keep the comment on its
# own line: make would include the spaces before an inline comment in the value.)
GITHUB_USERNAME := your-username

FULL_IMAGE_NAME := $(REGISTRY)/$(GITHUB_USERNAME)/$(IMAGE_NAME)

.PHONY: docker-push

# Push to registry (requires docker login first)
docker-push:
    @echo "→ Pushing $(FULL_IMAGE_NAME):$(IMAGE_TAG)..."
    docker tag $(IMAGE_NAME):$(IMAGE_TAG) $(FULL_IMAGE_NAME):$(IMAGE_TAG)
    docker tag $(IMAGE_NAME):$(IMAGE_TAG) $(FULL_IMAGE_NAME):latest
    docker push $(FULL_IMAGE_NAME):$(IMAGE_TAG)
    docker push $(FULL_IMAGE_NAME):latest
    @echo "✓ Pushed: $(FULL_IMAGE_NAME):$(IMAGE_TAG)"

# Complete release: build, test, docker build, push
release: build docker-build docker-push
    @echo ""
    @echo "_____ "
    @echo "✓ Release complete."
    @echo "  Image: $(FULL_IMAGE_NAME):$(IMAGE_TAG)"
    @echo "  Pull:  docker pull $(FULL_IMAGE_NAME):$(IMAGE_TAG)"
    @echo "_____ "
```

Key Takeaways

- A **container registry** is a server that stores Docker images and makes them retrievable by `docker pull`. It is the intermediary between where images are built (CI) and where they run (production).
- **Layer deduplication** means the registry stores each layer once. Pushing a new image version only uploads the layers that changed — base image layers are already there.
- **Registry options:** Docker Hub (public/default), GitHub Container Registry (integrated with

GitHub), AWS ECR (integrated with AWS IAM), Google Artifact Registry (multi-format). The choice depends on where the application runs.

- **Tag every image with its git commit hash** (e.g., `notes-api:sha-a1b2c3d`). This creates an auditable link between a running container and the exact code it contains — as long as the tag is never moved, which registry tag immutability or deploying by digest can guarantee. `latest` is a convenience alias, not a deployment strategy.
- **Never deploy by pulling `latest`**. Deploy a specific immutable tag. Rollback by deploying the previous immutable tag.
- **The CI-to-registry-to-production flow** ensures that the image CI built and smoke-tested is the exact image running in production: build once, test once, deploy the same artifact.
- **Retention policies** prevent registries from filling with unused images. Protect production-deployed images from deletion.
- **Artifact storage extends beyond Docker images**: Python wheel files, database migrations, test reports, and build logs are also artifacts that need storage and lifecycle management.

Exercises

1. **Predict.** Push the same image twice with the same tag. How much data is uploaded the second time, and why?
2. **Break and fix.** Tag an image `latest`, deploy it, then push a new `latest`. What is now running, and what will a restart run? Why does the book deploy `sha-...` tags only?
3. **Extend.** Write a retention rule for your registry: keep the last 20 images and anything deployed in the last 30 days. How would you enforce “never delete what production runs”?
4. **Explain.** Explain “build once, deploy the same artifact” and what it protects against.

Solutions and hints are in the companion repository, in `exercises/chapter-19.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What’s Next

The artifact (Docker image) is built, tested, and stored in a registry. The infrastructure to run it — a server — still needs to be established. What is a production server? How is it different from a laptop? What decisions must be made when selecting and configuring one?

→ [Chapter 20](#) — Server Fundamentals

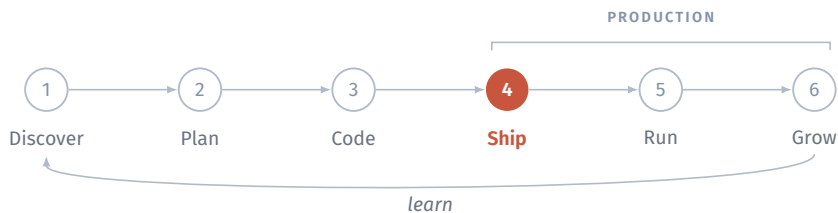
STAGE 5

5

Infrastructure and Environment

What environment will actually run it?

YOU ARE HERE



The piece: The machine

The whole map: *How Software Gets Built*, page 1

Chapters 20–22

Server Fundamentals

The Docker image is built. The registry holds it. Now the image needs somewhere to run. That somewhere is a **server** — a computer that is always on, always connected to the internet, and dedicated to running your application.

This chapter is about understanding what a production server is: what makes it different from a laptop, what the choices are when selecting one, and what properties a server needs to run the Notes API correctly. This understanding is foundational for everything in Stage 5 (infrastructure setup): you cannot configure something you do not understand.

In this chapter

By the end of this chapter, you will understand:

- What a server is at the hardware and operating system level, and how it differs from a developer's laptop
- The difference between physical servers, virtual machines, and cloud instances
- What CPU, RAM, disk, and network mean in the context of choosing a server
- What a Linux distribution is and which one to use
- What “always on” means and why it requires dedicated infrastructure
- How to choose the right server size for the Notes API
- What managed services are and when to use them instead of a raw server

20.1 What a Production Server Is

A **server** is a computer optimized for running services continuously, reliably, and without direct human interaction. The word “server” comes from its function: it serves requests from clients. The hardware and software are chosen with that function in mind.

How a server differs from a laptop

Property	Laptop	Server
Power	Battery-powered, designed to sleep	Always plugged in, never sleeps
Uptime	Typically hours per day	Target: 99.9%+ (less than 9 hours down per year)
CPU	Optimized for burst performance	Optimized for sustained throughput
RAM	8–32 GB typically	1 GB to terabytes, depending on workload
Disk	Fast NVMe SSD, local	Can be local SSD or network-attached storage
Display	Built-in screen	No screen — accessed via SSH
Location	On your desk	In a data center
Cooling	Fan, ambient air	Precision cooling systems
Redundancy	Single power supply	Redundant power supplies, redundant NICs
OS	macOS, Windows, or Linux	Almost always Linux
Management	GUI (click-based)	CLI via SSH

The fundamental difference is **continuous operation**. A laptop is designed to start, be used, and sleep. A server is designed to run indefinitely. When you deploy an application to a server, you are handing it to a machine that will keep running it without any human being in the loop.

No one sits at the server

This is the most important operational difference. When your Flask application crashes on your laptop, you see the error in the terminal and restart it. When your Flask application crashes on a server, no one is there to restart it unless you have configured something to do that automatically.

This is why:

- You need a process supervisor ([Chapter 25](#)) — to restart crashed processes automatically
- You need logging ([Chapter 16](#)) — to know that a crash happened and why
- You need monitoring ([Chapter 35](#)) — to be alerted when something goes wrong
- You need a startup sequence ([Chapter 24](#)) — so the application starts automatically when the server reboots

20.2 Physical Servers vs Virtual Machines vs Cloud Instances

Three categories of servers exist, and understanding the difference matters for how you provision and manage them.

Physical servers (bare metal)

A **physical server** (also called a **bare metal** server) is a real, physical computer in a data center. You have exclusive use of all its hardware: the CPU cores, RAM, and disk belong entirely to your application.

Pros:

- Maximum performance — no virtualization overhead
- Predictable latency — no “noisy neighbor” effect from sharing hardware
- Full control over the hardware configuration

Cons:

- Expensive — you pay for the full machine even when you only use 10% of it
- Slow to provision — ordering a physical server might take days or weeks
- No elasticity — you cannot add more CPUs or RAM without physically replacing hardware
- You are responsible for hardware failures — if a disk fails, you replace it

Physical servers are used for high-performance computing (machine learning training), databases at extreme scale, or applications with strict latency requirements.

For the Notes API: overkill.

Virtual machines

A **virtual machine (VM)** is a software emulation of a physical server. A **hypervisor** (such as VMware, KVM, or Hyper-V) runs on a physical server and creates multiple isolated virtual machines, each appearing to be a complete computer.

```
Physical Server (64 cores, 512 GB RAM)
├── VM 1: 4 cores, 8 GB RAM → your application
├── VM 2: 8 cores, 32 GB RAM → another customer's application
├── VM 3: 2 cores, 4 GB RAM → another customer's application
└── ...
```

The hypervisor manages resource sharing: each VM believes it has exclusive hardware, but the hypervisor arbitrates access to the underlying physical resources.

Pros:

- More affordable than dedicated hardware
- Faster to provision (minutes, not days)
- Can resize (more vCPUs or RAM) with a reboot
- Data center handles hardware failures — if the host machine fails, the provider restarts your VM on healthy hardware (after a short outage; data on local, non-replicated disks can be lost)

Cons:

- Slight performance overhead from virtualization
- “Noisy neighbor” effect — other VMs on the same physical host can affect your performance if they consume disproportionate resources

VMs are the backbone of cloud computing. When you rent a server from AWS, DigitalOcean, or Google Cloud, you are almost always getting a VM.

Cloud instances

A **cloud instance** (also called an **instance** or a **droplet**, depending on the provider) is a VM rented from a cloud provider on demand. The cloud provider handles the physical servers, power, cooling, networking, and hardware maintenance. You provision and manage VMs through an API or web console.

Cloud instances add important capabilities beyond basic VMs:

- **On-demand provisioning:** an instance can be running within seconds of you clicking “create”
- **Elastic scaling:** add more instances when demand increases, remove them when demand decreases
- **Pay-per-use:** pay by the hour (or second) for what you actually use
- **Managed services:** the provider can run your database, load balancer, or cache — you interact with an API, not a server
- **Global availability:** provision servers in dozens of geographic regions

For the Notes API, a cloud instance is the right choice. We will use a **DigitalOcean Droplet** (their term for a cloud VM) as the concrete example, though the concepts apply to any provider.

20.3 Choosing a Cloud Provider

The major cloud providers differ in their feature sets, pricing, complexity, and documentation quality.

Amazon Web Services (AWS)

The largest and most feature-rich provider. AWS has services for almost every conceivable use case. The downside is complexity: AWS has hundreds of services with overlapping functionality, and the learning curve is steep.

Good for: large organizations, teams that need advanced features, applications already using AWS services (RDS, S3, ECS, etc.).

Google Cloud Platform (GCP)

Strong in machine learning and data analytics. Good documentation and developer experience. More straightforward than AWS in some areas.

Good for: ML/AI workloads, applications using BigQuery or Kubernetes (Google created Kubernetes), organizations already using Google Workspace.

Microsoft Azure

Strong in Windows and enterprise integrations (.NET, Active Directory, SQL Server). Large presence in enterprise organizations.

Good for: applications that integrate with Microsoft products, organizations already using Microsoft Entra ID (formerly Azure Active Directory).

DigitalOcean

Simpler, developer-friendly, cheaper than the major cloud providers for basic workloads. Excellent documentation. Fewer advanced services.

Good for: individual developers, small teams, applications that do not need advanced cloud services, learning environments.

For this book, we use DigitalOcean. The concepts (instances, networking, DNS, firewalls) are identical across providers. DigitalOcean's simplicity makes the concepts clearer without the complexity of AWS's hundreds of overlapping services. When you understand the concepts on DigitalOcean, applying them to AWS or GCP is a matter of learning different names for the same ideas.

20.4 Understanding Server Resources

When choosing a server, you are choosing a combination of CPU, RAM, disk, and network. Understanding what each means for a web application guides the choice.

CPU (Central Processing Unit)

The CPU executes instructions. For a web application, CPU matters during:

- Parsing JSON request bodies
- Executing Python code (your route handlers)
- JSON serialization of response bodies
- Database query execution (if the database is on the same server)
- TLS encryption/decryption (HTTPS)

vCPU (virtual CPU) is the unit for cloud instances. One vCPU is typically one hardware thread on a physical CPU core. The Notes API runs as a handful of processes — a Gunicorn master and its workers. More vCPUs allow more concurrent workers.

For the Notes API with 2 Gunicorn workers: **1–2 vCPUs** is sufficient.

RAM (Random Access Memory)

RAM stores the running application's code and data. For a Python web application, RAM usage comes from:

- The Python interpreter itself (~20 MB per process)
- Imported libraries (Flask, psycpg2, etc.)
- In-memory data (cached results, connection pools)
- Each Gunicorn worker is a separate OS process with its own memory

Estimate for the Notes API:

- Base Python + Flask: ~50 MB per Gunicorn worker
- 2 workers: ~100 MB total for the application
- PostgreSQL (only if it runs on the same server): ~100 MB minimum
- Docker itself (dockerd and containerd): ~100 MB
- OS overhead: ~100 MB
- Total: ~300–500 MB

512 MB RAM is the minimum for the Notes API. **1 GB** gives comfortable headroom.

Disk (Storage)

Disk stores:

- The operating system (~5 GB for Ubuntu minimal)
- Docker and container images (~200 MB for the Notes API image; each old version kept for rollback adds only its changed layers)
- Log files (growing over time)
- The SQLite database file (if not using PostgreSQL)
- Temporary files

For the Notes API with PostgreSQL (which stores data on its own server), the application server needs minimal disk:

- OS: ~5 GB
- Docker: ~2 GB
- Logs: 1–5 GB (depends on traffic and retention)

20 GB is comfortable for the application server. (The smallest DigitalOcean droplet comes with 25 GB, which is the figure used below.)

Disk type matters more than size: SSDs (Solid State Drives) are orders of magnitude faster than HDDs (Hard Disk Drives) for random I/O. Database servers especially benefit from SSDs. All major cloud providers offer SSD-backed instances.

Network

Network determines:

- How fast data can flow in and out of the server
- How much data you can transfer per month
- Whether the server has a public IP address

Most cloud instances come with a **1 Gbps** network connection — enough bandwidth for hundreds of concurrent HTTP requests. Network is rarely the bottleneck for web applications (CPU and I/O typically bottleneck first).

Data transfer costs: most providers charge for outbound traffic (data leaving the data center). DigitalOcean includes 1 TB of outbound transfer per month with most instances. The Notes API's responses are small (JSON), so data transfer costs are minimal.

20.5 The Linux Distribution Choice

A **Linux distribution** is a complete operating system built around the Linux kernel. Distributions differ in:

- The package manager (how software is installed)
- The default configuration
- The release cycle (how often security updates are released)
- Support lifetime (how long security updates are provided)

For production servers, the choice is almost always between:

Ubuntu LTS (Long-Term Support)

Ubuntu LTS releases come every two years and receive security updates for 5 years (10 years with paid Extended Security Maintenance). This book uses **Ubuntu 22.04 LTS** (“Jammy”), supported until 2027. Ubuntu 24.04 LTS (“Noble”, April 2024) is the newer release; the commands in this book also work there, but they were written and checked against 22.04.

Ubuntu is the most widely used Linux distribution for cloud servers. Most documentation, tutorials, and cloud provider examples use Ubuntu. The package manager is `apt`.

```
# Update package lists and upgrade installed packages
sudo apt-get update && sudo apt-get upgrade -y

# Install a package
sudo apt-get install -y docker.io
```

Debian

Debian is the upstream distribution from which Ubuntu is derived. Debian supports each release for about 5 years (3 years of regular security support plus 2 years of LTS) and takes a more conservative approach to software versions. Debian’s package manager is also `apt`.

Debian is preferred by some organizations for its stability and security track record.

Amazon Linux (for AWS)

Amazon Linux 2023 is Amazon’s own distribution, optimized for AWS. It includes AWS tools pre-installed and receives security updates for 5 years.

If your application runs exclusively on AWS, Amazon Linux is a reasonable choice. If you want to avoid vendor lock-in, Ubuntu is more portable.

For this book: **Ubuntu 22.04 LTS**. All commands and configurations in Stage 5 use Ubuntu.

20.6 Sizing the Notes API Server

For the Notes API in a learning/small-production context, here is the appropriate server size:

Target: DigitalOcean Droplet (or equivalent on any provider)

CPU: 1 vCPU (sufficient for 2 Gunicorn workers)

RAM: 1 GB (comfortable for application + Docker + OS overhead)

Disk: 25 GB (SSD – OS + Docker images + logs)

Network: 1 Gbps / 1 TB outbound transfer per month

Cost (DigitalOcean as of 2024): ~\$6–\$12/month

Operating System: Ubuntu 22.04 LTS (x86_64)

For comparison, here is how this scales:

Workload	CPU	RAM	Notes
Learning / personal project	1 vCPU	512 MB	Absolute minimum
Small production (< 1000 req/day)	1 vCPU	1 GB	This book’s target
Medium production (< 100K req/day)	2 vCPU	2–4 GB	Add more workers
High traffic (> 100K req/day)	Multiple servers	8+ GB each	Load balancer needed

These are starting points, not exact specifications. Production sizing is always validated with load testing ([Chapter 36](#)).

20.7 Managed Services vs Self-Managed Servers

A recurring choice in infrastructure is: should you manage the server yourself, or use a managed service?

Self-managed server (a “raw” VM)

You provision a VM, install the OS, install dependencies, deploy the application, manage security updates, and handle failures. Full control, full responsibility.

Pros:

- Full control over configuration
- Potentially cheaper at scale

- Works with any technology stack

Cons:

- You are responsible for OS security updates
- You manage backups, monitoring, scaling
- Operational burden grows with the number of servers

Managed database (e.g., DigitalOcean Managed PostgreSQL, AWS RDS)

The cloud provider runs PostgreSQL for you. You get an endpoint to connect to; the provider handles installation, updates, backups, failover, and monitoring.

Pros:

- No server to manage for the database
- Automatic backups (often configurable: daily snapshots, point-in-time recovery)
- Automatic failover (if the primary fails, a replica is promoted)
- Security patches applied automatically

Cons:

- More expensive than running PostgreSQL on the application server
- Less control (cannot install custom PostgreSQL extensions easily)
- Vendor dependency

For the Notes API in production: use a managed PostgreSQL database. The operational complexity of managing a database server correctly (replication, backups, failover) far exceeds the cost of a managed service. Spend your time on the application, not the database infrastructure. Budget for it, though: the smallest managed PostgreSQL plan typically costs two to three times as much as the smallest droplet, so a small production setup is around \$20–25 a month rather than \$6.

So the production topology in this book is: **the application (in Docker) and Nginx on one VM, and PostgreSQL as a managed service on its own server.** SQLite on the VM remains an option for a single-server deployment that can live without the managed database.

Managed container platforms

Managed container platforms (AWS ECS, Google Cloud Run, DigitalOcean App Platform) run Docker containers for you. You provide the image; they handle servers, networking, scaling, and restarts.

Google Cloud Run is notable: it runs a Docker container, scales to zero when there is no traffic (costs nothing), and scales up automatically when requests arrive. The trade-off is the **cold start**: the first request after an idle period waits while a container starts. Cost: you pay only for the CPU and RAM used while handling requests.

For small-to-medium applications, managed container platforms can be significantly cheaper and simpler than managing servers directly. The tradeoff: less control over the execution environment and networking.

For this book: we deploy to a raw VM. Understanding the raw VM gives you the complete picture of what managed platforms abstract away. After you understand how to configure a server, configure a process supervisor, configure Nginx, and configure firewalls manually — then you appreciate what Cloud Run or App Platform does for you.

20.8 What the Production Server Must Provide

Based on the Notes API's runtime contract ([Chapter 14](#)) and the architecture chapters ahead, here is what the production server must provide:

PRODUCTION SERVER REQUIREMENTS FOR THE NOTES API

HARDWARE

CPU: ≥1 vCPU (x86_64 architecture)
 RAM: ≥1 GB
 Disk: ≥20 GB SSD

OPERATING SYSTEM

Ubuntu 22.04 LTS (or Debian 12+)
 Fully updated with security patches

SOFTWARE (to be installed – Chapter 21)

Docker Engine 24+ (to run the container)
 Docker Compose v2 (for local development; optional in production)
 Nginx (as reverse proxy – Chapter 28)
 Certbot (for TLS certificates – Chapter 28)

NETWORK

One static public IPv4 address
 DNS A record pointing domain to the public IP (Chapter 27)
 Inbound TCP allowed on port 80 (HTTP)
 Inbound TCP allowed on port 443 (HTTPS)
 Inbound TCP allowed on port 22 (SSH) – from specific IPs only
 All other inbound ports blocked (firewall – Chapter 22)

PROCESS MANAGEMENT

Docker restart policy (unless-stopped), with Docker itself
 started by systemd at boot (Chapters 24–25)
 Restarts the container on failure and on server reboot

DATABASE

Managed PostgreSQL (separate server, accessed via DATABASE_URL)
 OR SQLite for single-server deployments (simpler, less scalable)

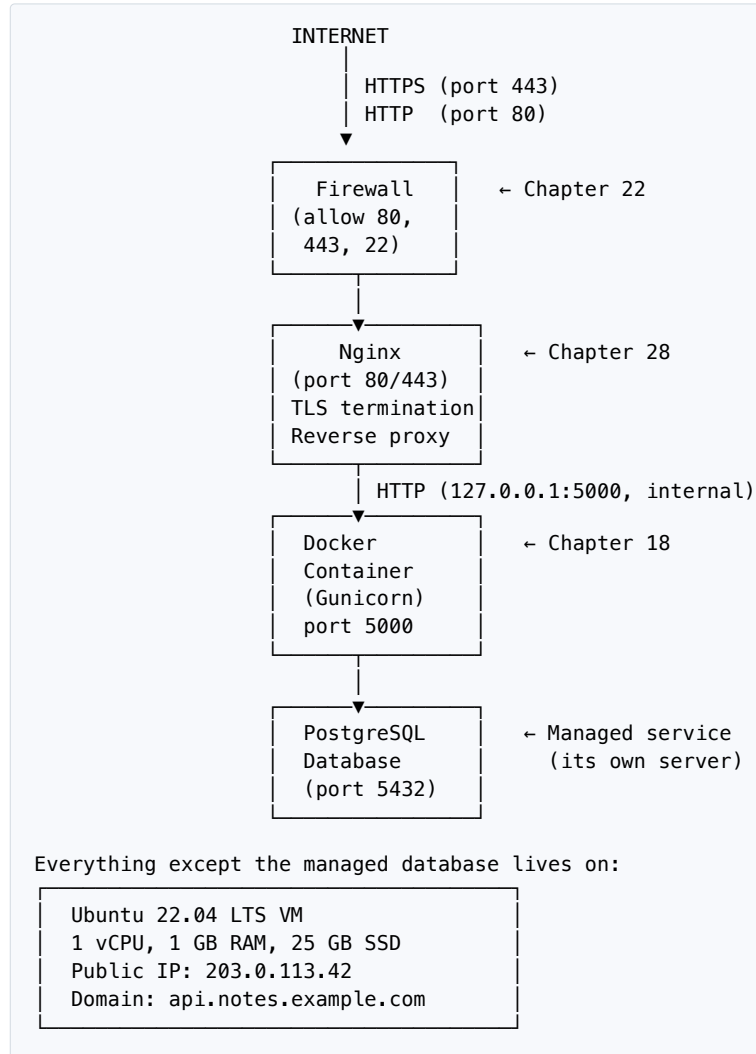
ENVIRONMENT VARIABLES

DATABASE_URL – connection string to the database
 SECRET_KEY – random secret (required by the runtime contract; Flask
 uses it to sign cookies and tokens, which a future feature may need)
 LOG_FORMAT=json, LOG_LEVEL=INFO (defaults baked into the image)
 Set via environment file or systemd unit (Chapter 25)

Each item in this list is addressed in the following chapters.

20.9 The Architecture of the Complete System

Before diving into server setup, it helps to see the complete architecture that the following chapters will build:



Reading from top to bottom:

1. A client sends a request to `api.notes.example.com` (DNS resolves to the server's public IP)
2. The firewall allows traffic on port 443 (HTTPS)
3. Nginx receives the request, terminates TLS, and forwards it to Gunicorn on port 5000
4. Gunicorn passes it to the Flask application
5. Flask queries PostgreSQL (on a separate managed server)

6. The response travels back through the same path

This is the architecture that Stages 5 through 7 build, one layer at a time.

Key Takeaways

- A **server** is a computer optimized for continuous, unattended operation: always on, always connected, managed via SSH rather than a graphical interface.
- The critical operational difference from a laptop: no one is watching. Automated restarts, logging, and monitoring are not optional — they are requirements.
- **Physical servers** offer maximum performance; **virtual machines** offer flexibility and cost efficiency; **cloud instances** (VMs on demand) offer elasticity and managed infrastructure.
- **Cloud providers** (AWS, GCP, DigitalOcean, Azure) differ in features, complexity, and pricing. DigitalOcean is the simplest entry point; AWS is the most feature-rich. The underlying concepts are identical.
- **CPU, RAM, disk, and network** are the four dimensions of server sizing. For the Notes API: 1 vCPU, 1 GB RAM, 25 GB SSD is the target.
- **Ubuntu 22.04 LTS** is the operating system for production Linux servers. “LTS” means 5 years of security updates. The package manager is `apt`.
- **Managed services** (managed databases, managed container platforms) reduce operational burden at the cost of control and (sometimes) money. For the database layer: use managed PostgreSQL. For the application layer: understanding raw VMs first gives the complete picture.
- The production architecture for the Notes API is: Internet → Firewall → Nginx (TLS) → Docker (Gunicorn) → PostgreSQL. Each layer is built in subsequent chapters.

Exercises

1. **Predict.** Estimate the Notes API’s memory use: two Gunicorn workers of about 60 MB each, plus the master. How much headroom does a 1 GB droplet leave, and what will use it?
2. **Break and fix.** In a container limited to 64 MB (`docker run -m 64m`), start the API with eight workers. What happens? Find the evidence in `docker inspect` and `docker events`.
3. **Extend.** Price the architecture of section 20.9 at two providers of your choice, for one year, including backups and the managed database.
4. **Explain.** When is a managed database worth its price, and when is running PostgreSQL yourself reasonable?

Solutions and hints are in the companion repository, in `exercises/chapter-20.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

The server is understood. Now it must be created and configured: installing Docker, setting up users, hardening the OS, and preparing the environment for the application.

→ [Chapter 21 — System Setup](#)

System Setup

You have a fresh Ubuntu 22.04 LTS server — a blank slate. It has an operating system installed, a public IP address, and a root user. That is all. Nothing else is there: no Docker, no application user, no firewall configuration, no security hardening, no monitoring tools.

Setting up a server correctly is a discipline. Done well, it results in a system that is secure, predictable, and easy to maintain. Done poorly, it results in a system where the application runs until something breaks and no one can figure out why — or where a simple vulnerability leaves the server open to attackers.

This chapter walks through the complete initial setup of a production server for the Notes API: from first SSH login to a fully prepared environment ready to receive the application.

In this chapter

By the end of this chapter, you will understand:

- How to connect to a server via SSH and why SSH is the standard remote access method
- How to create a non-root application user and why root should never be used for applications
- How to configure SSH securely (disable root login, use key authentication)
- How to install and configure Docker Engine on Ubuntu
- How to configure automatic security updates
- How to set up the basic directory structure for the application
- How to verify that the server is correctly set up before deploying

21.1 First Contact: SSH

SSH (Secure Shell) is the protocol for remotely managing Linux servers. It provides an encrypted terminal session over the network. Every command you type in the SSH session executes on the remote server.

How SSH works

SSH uses **asymmetric cryptography**. You generate a key pair:

- **Private key** (`~/.ssh/id_ed25519`): stays on your laptop, never shared
- **Public key** (`~/.ssh/id_ed25519.pub`): placed on the server in `~/.ssh/authorized_keys`

When you SSH to the server, the server challenges your SSH client to prove it has the private key corresponding to the public key in `authorized_keys`. This proof happens through a cryptographic protocol — your private key never leaves your machine.

This is more secure than password authentication because:

- Keys are effectively impossible to brute-force (an Ed25519 key offers about 128 bits of security)
- Passwords can be guessed or phished; a private key never leaves your machine, so it is much harder to steal (and the passphrase protects it if the file itself is stolen)

Generating an SSH key pair

```
# Generate an Ed25519 key pair
# Ed25519 is the recommended algorithm — fast, small, secure
ssh-keygen -t ed25519 -C "your-email@example.com"

# When prompted:
#   Enter file in which to save the key: press Enter (uses default:
↳ ~/.ssh/id_ed25519)
#   Enter passphrase: enter a strong passphrase (protects the private key on disk)
```

This creates two files:

```
~/.ssh/id_ed25519      ← Private key (never share this)
~/.ssh/id_ed25519.pub ← Public key (share this with servers)
```

View the public key:

```
cat ~/.ssh/id_ed25519.pub
# ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA... your-email@example.com
```

Adding your key to DigitalOcean

When creating a Droplet on DigitalOcean (or an instance on any cloud provider), you add your public key during setup. DigitalOcean places your public key in the root user's `authorized_keys` file automatically.

Alternatively, after creating the server, you can add the key manually:

```
# Copy public key to the server (if you have password access initially)
ssh-copy-id root@SERVER_IP_ADDRESS
```

The first SSH login

```
ssh root@203.0.113.42
```

(Replace `203.0.113.42` with your server's actual IP address.)

First connection output:

```
The authenticity of host '203.0.113.42 (203.0.113.42)' can't be established.
ED25519 key fingerprint is SHA256:abc123xyz...
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '203.0.113.42' (ED25519) to the list of known hosts.

Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-91-generic x86_64)
...
root@ubuntu-server:~#
```

You are now at a terminal on the remote server. Every command you type runs on the server.

21.2 Initial System Update

The first thing to do on any new server is update the installed packages. The operating system image provided by the cloud provider might be weeks or months old, meaning security patches released since then are not applied.

```

# Update the package list (information about available packages)
apt-get update

# Upgrade all installed packages to their latest versions
apt-get upgrade -y

# Remove packages that are no longer needed
apt-get autoremove -y

# Clean up downloaded package files
apt-get clean

```

Why update immediately: Security vulnerabilities are discovered continuously. A server that has been running for 6 months without updates might have dozens of known, exploitable vulnerabilities. Always update first, before installing anything else or opening any ports.

After upgrading, if the kernel was updated, reboot to run the new kernel:

```

# Check if a reboot is required
cat /var/run/reboot-required 2>/dev/null && echo "Reboot required" || echo "No reboot
↳ needed"

# Reboot if needed
reboot

# Wait 30-60 seconds, then reconnect
ssh root@203.0.113.42

```

21.3 Creating a Non-Root User

Never run the application as root.

Root is the superuser on Linux — it can do anything: delete any file, modify any configuration, install any software, kill any process. Running an application as root means that if the application is compromised, the attacker has root access to the server.

Create a dedicated user for running the application:

```
# Create user 'deploy' with a home directory and bash shell
useradd --create-home --shell /bin/bash deploy

# Set a password. SSH logins will use keys, but sudo asks for this password.
passwd deploy

# Add 'deploy' to the 'sudo' group so they can run administrative commands
usermod -aG sudo deploy
```

(The deploy user also joins the docker group — but that group is created by the Docker installation in section 21.6, so that step comes there.)

Adding SSH access for the deploy user

The root user's SSH key needs to be copied to the deploy user:

```
# Create .ssh directory for deploy user
mkdir -p /home/deploy/.ssh
chmod 700 /home/deploy/.ssh

# Copy root's authorized_keys to deploy user
cp /root/.ssh/authorized_keys /home/deploy/.ssh/authorized_keys
chmod 600 /home/deploy/.ssh/authorized_keys
chown -R deploy:deploy /home/deploy/.ssh
```

Now verify that the deploy user can log in from your laptop:

```
# From your laptop (new terminal):
ssh deploy@203.0.113.42

# The server should not ask for a password. (If you gave your key a
# passphrase, your laptop asks for that; the server never sees it.)
```

21.4 Securing SSH

The default SSH configuration has several insecure settings. Harden it — but not by editing `/etc/ssh/sshd_config` directly. On Ubuntu 22.04, that file first includes every file in `/etc/ssh/sshd_config.d/`, and cloud images ship one there (DigitalOcean's is `50-cloud-init.conf`) that sets `PasswordAuthentication yes`. `sshd` keeps the *first* value it reads for each setting, so a change further down in `sshd_config` is silently ignored. Put the hardening in its own drop-in file, named so that it is read first:

```
nano /etc/ssh/sshd_config.d/00-hardening.conf
```

```
# /etc/ssh/sshd_config.d/00-hardening.conf – key security settings

# Disable password authentication entirely.
# Everyone must use SSH keys.
PasswordAuthentication no

# Disable root login entirely.
# Attackers constantly try to log in as root.
# After this change, you MUST use the 'deploy' user.
PermitRootLogin no

# Allow only the deploy user to SSH in.
# Add more users separated by spaces if needed.
AllowUsers deploy

# Close connections whose client has stopped responding (laptop asleep,
# network gone): probe every 300 seconds, give up after 2 missed replies.
# This detects DEAD connections, not idle users – a connected client
# answers the probes, so an idle session stays open.
ClientAliveInterval 300
ClientAliveCountMax 2

# Limit the number of concurrent unauthenticated connections
# This reduces the impact of brute-force attempts
MaxStartups 3:50:10
```

(You may see Protocol 2 in older guides. Modern OpenSSH supports only protocol 2 and ignores the setting.)

Check the configuration, then restart the SSH service:

```
# Test the configuration for errors first: a typo here can lock you out
sshd -t

# Restart SSH (on Ubuntu the service is called "ssh")
systemctl restart ssh

# Confirm the settings sshd is actually using
sshd -T | grep -Ei '^(passwordauthentication|permitrootlogin|allowusers)'
# passwordauthentication no
# permitrootlogin no
# allowusers deploy
```

CRITICAL: Test the new configuration before closing your current session.

Open a NEW terminal and try to SSH in:

```
# In a NEW terminal on your laptop:
ssh deploy@203.0.113.42
```

If this works, you have configured SSH correctly. If it fails, go back to the old session and fix the configuration before closing it. If you close the old session with a broken SSH configuration, you may lock yourself out of the server permanently (requiring a recovery console through the cloud provider's web interface).

21.5 Configuring the Firewall

Ubuntu uses UFW (Uncomplicated Firewall) as a frontend to the Linux iptables firewall. UFW makes firewall rule management straightforward.

```
# Check current UFW status
ufw status
# Status: inactive (not enabled yet – all traffic allowed)

# Reset to defaults (deny all inbound, allow all outbound)
ufw default deny incoming
ufw default allow outgoing

# Allow SSH (port 22) – DO THIS BEFORE ENABLING THE FIREWALL
# If you forget this, you will lock yourself out immediately
```

```

ufw allow 22/tcp

# Allow HTTP (port 80)
ufw allow 80/tcp

# Allow HTTPS (port 443)
ufw allow 443/tcp

# Review the rules before enabling
ufw show added
# Added user rules (see 'ufw status' for running firewall):
# ufw allow 22/tcp
# ufw allow 80/tcp
# ufw allow 443/tcp

# Enable the firewall
ufw enable
# Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
# Firewall is active and enabled on system startup

ufw status verbose

```

Expected status:

```

Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

```

To	Action	From
---	-----	----
22/tcp	ALLOW IN	Anywhere
80/tcp	ALLOW IN	Anywhere
443/tcp	ALLOW IN	Anywhere

Port 5000 (where the Flask/Gunicorn application runs) is NOT listed, because only Nginx on the same server should reach it.

Warning: Docker bypasses UFW. UFW's rules live in the `INPUT` chain of the kernel's firewall. When Docker publishes a container port (`docker run -p 5000:5000`), it adds its own rules to the `nat` and `FORWARD` chains, and traffic to the container never passes through `INPUT`. The result: `-p 5000:5000` makes the unencrypted application port reachable from the entire internet, even though `ufw status` shows it blocked. Two defenses, and this book uses both:

1. Publish container ports on the loopback address only: `-p 127.0.0.1:5000:5000`. Every `docker run` on the server in the following chapters does this.
2. Add the provider's cloud firewall ([Chapter 22](#)), which filters traffic before it reaches the server at all — Docker cannot bypass it.

Restricting SSH to specific IPs (recommended)

If your team always connects from known IP ranges (office, VPN), further restrict SSH:

```
# First allow SSH from your specific IP (add the new rule before removing
# the old one, so there is never a moment when SSH is blocked)
ufw allow from YOUR.IP.ADDRESS to any port 22 proto tcp

# Example: allow from your home IP
ufw allow from 203.0.113.100 to any port 22 proto tcp

# Then remove the rule that allows SSH from anywhere
ufw delete allow 22/tcp
```

This makes SSH brute-force attacks from random internet addresses impossible — the firewall drops the connection before it even reaches the SSH server. One caveat: most home internet connections get a new IP address from time to time. When yours changes, you are locked out until you update the rule — through the provider's web console, which is why it is worth knowing where that console is before you need it.

21.6 Installing Docker Engine

Install Docker Engine (the open-source version, not Docker Desktop):

Note: Docker's installation steps (the repository URL, key location, and package names) change from time to time. The commands below follow Docker's documentation at the time of writing; if one fails, use the current instructions at docs.docker.com for your Ubuntu release.

```

# Install prerequisite packages
apt-get install -y \
    ca-certificates \
    curl \
    gnupg \
    lsb-release

# Add Docker's official GPG key
# This allows apt to verify that downloaded packages are authentic
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg

# Add the Docker repository to apt's sources list
echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(lsb_release -cs) stable" | \
    tee /etc/apt/sources.list.d/docker.list > /dev/null

# Update package lists (now includes Docker's repository)
apt-get update

# Install Docker Engine, CLI, and container runtime
apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Verify installation
docker --version
# Docker version 25.0.3, build 4debf41

docker compose version
# Docker Compose version v2.24.5

```

Configure Docker to start on boot

```
# Enable Docker to start automatically when the server boots
systemctl enable docker

# Start Docker now
systemctl start docker

# Verify it is running
systemctl status docker
# ● docker.service – Docker Application Container Engine
#    Active: active (running) since ...
```

Let the deploy user run Docker

The installation created the `docker` group. Add `deploy` to it, so that it can run `docker` commands without `sudo`:

```
usermod -aG docker deploy
# Takes effect at deploy's next login
```

Be clear about what this grants: anyone who can talk to the Docker daemon can start a container that mounts the host's root filesystem, so membership of the `docker` group is equivalent to root access. That is acceptable here because `deploy` is the account that administers the server anyway (it is in `sudo` too), while the application itself runs as the unprivileged `notes` user *inside* its container. Docker's **rootless mode** removes this equivalence, at the cost of some extra setup and a few restrictions.

Test Docker works

```
# Test with the hello-world image
docker run hello-world
# Hello from Docker! ...
```

Configure the Docker daemon

Create a Docker daemon configuration file to set production-appropriate defaults:

```
# Create or edit /etc/docker/daemon.json
cat > /etc/docker/daemon.json << 'EOF'
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  },
  "live-restore": true
}
EOF
```

Explanation:

- **log-driver:** `json-file`: Docker stores container logs as JSON files (the default)
- **max-size:** `10m`: each log file is limited to 10 MB
- **max-file:** `3`: keep at most 3 log files per container (30 MB total)
- **live-restore:** `true`: containers keep running if the Docker daemon is restarted (prevents application downtime during Docker updates)

Restart Docker to apply the new configuration:

```
systemctl restart docker
```

21.7 Setting Up Automatic Security Updates

Security vulnerabilities are discovered continuously. Applying security updates manually is risky — you might miss one, or apply it too late. **Unattended upgrades** automatically install security updates.

```
# Install the unattended-upgrades package (already present on Ubuntu Server;
# this makes sure)
apt-get install -y unattended-upgrades apt-listchanges

# Ubuntu's own settings live in 50unattended-upgrades, and by default they
# already install security updates. Leave that file alone and put your
# changes in a file that is read after it (a higher number wins):
cat > /etc/apt/apt.conf.d/52unattended-upgrades-local << 'EOF'
// Email notification when a package is upgraded
// Replace with your email address
```

```
// Unattended-Upgrade::Mail "admin@example.com";

// Automatically remove unused packages
Unattended-Upgrade::Remove-Unused-Dependencies "true";

// Automatically reboot if required by an update (e.g., kernel update).
// Set to "true" only once Docker and the application container come back
// by themselves after a reboot (Chapters 24–25), and pick a quiet time
// with Automatic-Reboot-Time (e.g. "04:00").
Unattended-Upgrade::Automatic-Reboot "false";

// Log output
Unattended-Upgrade::Verbose "false";
EOF

# Configure the schedule (run daily)
cat > /etc/apt/apt.conf.d/20auto-upgrades << 'EOF'
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
APT::Periodic::Download-Upgradeable-Packages "1";
APT::Periodic::AutocleanInterval "7";
EOF

# Test the configuration (dry run)
unattended-upgrade --dry-run --debug
```

With Ubuntu's defaults plus these overrides, security updates from Ubuntu's security repository are applied automatically. Non-security updates (new features, version upgrades) are not applied automatically — those require manual review and testing.

Why `Automatic-Reboot` is `false`: Kernel updates sometimes require a reboot. Automatic reboots during peak traffic cause downtime. Instead, check for required reboots during maintenance windows:

```
# Check if a reboot is pending
[ -f /var/run/reboot-required ] && echo "Reboot required" || echo "No reboot needed"
```

21.8 Creating the Application Directory Structure

Create the directory structure where the application will live:

```

# Create the application directory
mkdir -p /opt/notes-api

# Create subdirectories
mkdir -p /opt/notes-api/config # Environment files
mkdir -p /opt/notes-api/data   # Persistent data (if using SQLite)

# Set ownership to the deploy user
chown -R deploy:deploy /opt/notes-api

# The data directory is written by the application INSIDE the container,
# which runs as the 'notes' user, UID 1001 (Chapter 18). Ownership on a
# bind-mounted directory is by number, so give it to UID 1001.
chown 1001:1001 /opt/notes-api/data

# Set appropriate permissions
# 750 = owner can read/write/execute, group can read/execute, others cannot access
chmod 750 /opt/notes-api
chmod 750 /opt/notes-api/config
chmod 750 /opt/notes-api/data

```

There is no log directory. In a container, the application writes its logs to standard output and standard error; Docker collects them, and the `daemon.json` settings in section 21.6 rotate them. (`docker logs` and, in Chapter 25, `journalctl` read them back.)

The environment file

The application needs environment variables. On a server running Docker, environment variables are passed to the container at runtime. Create a file that holds them:

```

# Create the environment file for the application
# This file is NOT committed to git – it contains secrets
nano /opt/notes-api/config/notes-api.env

```

```
# /opt/notes-api/config/notes-api.env
# Environment variables for the Notes API container
# This file is read by docker run --env-file and by docker compose

DATABASE_URL=postgresql://notes_user:CHANGE_ME@db-host:5432/notesdb
SECRET_KEY=
FLASK_DEBUG=false
LOG_FORMAT=json
LOG_LEVEL=INFO
```

`docker run --env-file` reads each `KEY=value` line literally: no quotes (they would become part of the value), no `export`, and no comments except on lines of their own.

`SECRET_KEY` is deliberately empty: if you forget to fill it in, the application refuses to start instead of running with a placeholder. Generate a real one and paste it after the `=`:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

Set restrictive permissions on the environment file (it contains secrets):

```
chown deploy:deploy /opt/notes-api/config/notes-api.env
chmod 600 /opt/notes-api/config/notes-api.env
# 600 = owner can read/write, no access for group or others
```

21.9 Configuring System Limits

Linux systems have configurable limits on resource usage. Two kinds matter for a web server: how many files and sockets a process may open, and how the kernel queues network connections. Before changing either, it is worth knowing which settings actually reach the application — here, a process inside a Docker container.

File descriptor limits

Every open file (and socket, and pipe) uses a **file descriptor**. A server handling many concurrent HTTP connections might open thousands of sockets simultaneously, and the default limit for a login shell (1024 per process) could be exhausted.

Guides often raise it in `/etc/security/limits.conf`. That file is applied by PAM when a user *logs in*, so it affects SSH sessions — not services started by `systemd`, and not Docker containers. A container inherits its limit from the Docker daemon, whose `systemd` unit already sets a very high one. Check what the container actually gets:

```
# The limit in your login shell
ulimit -n
# 1024

# The limit inside a container
docker run --rm alpine sh -c 'ulimit -n'
# 1048576
```

So there is nothing to raise for the Notes API. (If you ever need a different value for one container, `docker run --ulimit nofile=65536:65536` sets it; "default-ulimits" in `daemon.json` sets it for all.) The kernel-wide maximum, `fs.file-max`, is already far larger on 64-bit Ubuntu than any value you would set by hand.

Network kernel parameters

```
# Put tuning in a drop-in file: re-running this command replaces the file,
# instead of appending duplicate lines to /etc/sysctl.conf
cat > /etc/sysctl.d/99-notes-api.conf << 'EOF'
# Queue of connections waiting to be accept()ed. Ubuntu 22.04's default is
# 4096; a larger value only helps if the listening program asks for a
# larger backlog too (Gunicorn's --backlog, default 2048; Nginx's
# "listen ... backlog=N").
net.core.somaxconn = 8192

# Queue of half-open connections (SYN received, handshake not finished)
net.ipv4.tcp_max_syn_backlog = 8192

# Let OUTGOING connections (to the database, for example) reuse local ports
# still in TIME_WAIT, which lasts about 60 seconds on Linux. It has no
# effect on connections the server accepts.
net.ipv4.tcp_tw_reuse = 1
EOF

# Apply all sysctl configuration files
sysctl --system
```

These settings are safe on a small server, but be honest about what they buy: they matter only at loads far above what a 1 GB droplet will see. They are here so you know where such tuning lives, and what each knob really changes, before you need it.

21.10 Configuring the Server Timezone and Locale

Log timestamps and cron job schedules use the server's timezone. Set it to UTC for production servers.

Why UTC: UTC is the global reference timezone. It never has daylight saving time changes. Log entries from servers in multiple timezones can be compared directly. The alternative — using a local timezone — adds confusion when interpreting logs during daylight saving time transitions.

```
# Set timezone to UTC
timedatectl set-timezone UTC

# Verify
timedatectl
# Local time: Mon 2024-01-15 14:00:00 UTC
# Universal time: Mon 2024-01-15 14:00:00 UTC
# RTC time: Mon 2024-01-15 14:00:00
# Time zone: UTC (UTC, +0000)
```

Configure the locale (language and character encoding settings):

```
# Install locale support
apt-get install -y locales

# Generate UTF-8 locale
locale-gen en_US.UTF-8
update-locale LANG=en_US.UTF-8
# (Set only LANG. LC_ALL overrides every other locale setting and is
# meant for one-off commands, not for the system default.)
```

21.11 Installing Useful Server Tools

A minimal set of tools for diagnosing server issues:

```
apt-get install -y htop curl wget dnsutils lsof jq git vim fail2ban
```

Package	What it is for
htop	Interactive process viewer
curl	HTTP client, for testing endpoints from the server
wget	File downloader
dnsutils	DNS tools: dig, nslookup
lsof	List open files and sockets
jq	JSON processor (for JSON logs and Docker output)
git	Version control
vim	Text editor
fail2ban	Intrusion prevention (bans IPs with too many failed logins)

(A comment after a trailing `\` — `htop \ # viewer` — would break the command: the backslash then escapes a space instead of the newline, and the shell ends the command at the comment. Hence the table. The older `net-tools` package, with `netstat` and `ifconfig`, is deprecated; this book uses `ss` and `ip` instead.)

Configuring fail2ban

fail2ban monitors log files for repeated authentication failures and automatically blocks offending IP addresses:

```
# fail2ban is installed above; configure it for SSH
cat > /etc/fail2ban/jail.local << 'EOF'
[DEFAULT]
# Ban for 1 hour
bantime = 3600
# Consider 3 failed attempts within 10 minutes as an attack
findtime = 600
maxretry = 3

[sshd]
enabled = true
port = ssh
logpath = %(sshd_log)s
backend = %(syslog_backend)s
# On Ubuntu 22.04, sshd's messages reach /var/log/auth.log through rsyslog.
# On a system where they go only to the systemd journal, use instead:
#   backend = systemd
```

EOF

```
systemctl enable fail2ban
systemctl start fail2ban

# Check status
fail2ban-client status sshd
```

After a failed SSH brute-force attempt, fail2ban automatically adds an `iptables` rule blocking the attacker's IP. The firewall bans are visible:

```
fail2ban-client status sshd
# Status for the jail: sshd
# |- Filter
# |   |- Currently failed: 0
# |   |- Total failed: 0
# |   `-- File list: /var/log/auth.log
# `-- Actions
#     |- Currently banned: 0
#     |- Total banned: 0
#     `-- Banned IP list:
```

21.12 Verifying the Setup

Run through a checklist to verify the server is correctly configured before deploying the application.

Listing 21.1 `verify-setup.sh` (server setup verification script)

```
#!/bin/bash
# Run with sudo: ufw and sshd -T need root.
#
# No "set -e" here: a failing check must not stop the script, and bash's
# ((PASS++)) returns a failure status when PASS is 0, which set -e would
# treat as an error after the very first check.

echo "Verifying server setup..."
PASS=0
FAIL=0

check() {
    local desc="$1"
```

```

    local cmd="$2"
    if eval "$cmd" > /dev/null 2>&1; then
        echo "  ✓ $desc"
        PASS=$((PASS + 1))
    else
        echo "  ✗ $desc"
        FAIL=$((FAIL + 1))
    fi
}

# System
check "Ubuntu 22.04" "grep -q 'Ubuntu 22.04' /etc/os-release"
check "No reboot pending" "[ ! -f /var/run/reboot-required ]"

# Users
check "deploy user exists" "id deploy"
check "deploy in docker group" "groups deploy | grep -q docker"

# SSH
# (sshd -T prints the settings sshd really uses, drop-in files included)
check "Root login disabled" "sshd -T | grep -qi '^permitrootlogin no'"
check "Password auth disabled" "sshd -T | grep -qi '^passwordauthentication no'"

# Firewall
check "UFW active" "ufw status | grep -q 'Status: active'"
check "Port 80 open" "ufw status | grep -q '80/tcp.*ALLOW'"
check "Port 443 open" "ufw status | grep -q '443/tcp.*ALLOW'"
check "Port 22 open" "ufw status | grep -q '22/tcp.*ALLOW'"

# Docker
check "Docker installed" "docker --version"
check "Docker running" "systemctl is-active docker"
check "Docker starts on boot" "systemctl is-enabled docker"

# Directory structure
check "/opt/notes-api exists" "[ -d /opt/notes-api ]"
check "deploy owns /opt/notes-api" "[ \$(stat -c '%U' /opt/notes-api) = 'deploy' ]"
check "Config directory exists" "[ -d /opt/notes-api/config ]"
check "Data directory owned by UID 1001" "[ \$(stat -c '%u' /opt/notes-api/data) =
↳ '1001' ]"

# Timezone
check "UTC timezone" "timedatectl | grep -q 'Time zone: UTC'"

```

```
# Security
check "fail2ban running" "systemctl is-active fail2ban"
check "Unattended upgrades configured" "[ -f
↳ /etc/apt/apt.conf.d/50unattended-upgrades ]"

echo ""
echo "Results: $PASS passed, $FAIL failed"
if [ $FAIL -gt 0 ]; then
    echo "Fix the failures before deploying."
    exit 1
else
    echo "Server is ready for deployment."
fi
```

Run this script:

```
# From your laptop: copy the script to the server, then run it with sudo
# (-t gives sudo a terminal to ask for deploy's password)
scp verify-setup.sh deploy@203.0.113.42:
ssh -t deploy@203.0.113.42 'sudo bash verify-setup.sh'
```

Expected output:

```
Verifying server setup...
✓ Ubuntu 22.04
✓ No reboot pending
✓ deploy user exists
✓ deploy in docker group
✓ Root login disabled
✓ Password auth disabled
✓ UFW active
✓ Port 80 open
✓ Port 443 open
✓ Port 22 open
✓ Docker installed
✓ Docker running
✓ Docker starts on boot
✓ /opt/notes-api exists
✓ deploy owns /opt/notes-api
✓ Config directory exists
✓ Data directory owned by UID 1001
✓ UTC timezone
```

- ✓ fail2ban running
- ✓ Unattended upgrades configured

Results: 20 passed, 0 failed
Server is ready for deployment.

21.13 The SSH Config File: Simplifying Connections

Typing `ssh deploy@203.0.113.42` every time is tedious. Create an SSH config file to simplify:

```
# On your laptop: ~/.ssh/config
cat >> ~/.ssh/config << 'EOF'

Host notes-api
    # Your server's IP address, and the user to log in as
    HostName 203.0.113.42
    User deploy
    IdentityFile ~/.ssh/id_ed25519
    # Send a keepalive every 60 seconds; disconnect after 3 missed replies
    ServerAliveInterval 60
    ServerAliveCountMax 3
EOF

chmod 600 ~/.ssh/config
```

(Comments in `~/.ssh/config` must be on lines of their own; `ssh` rejects text after a value.)
After this:

```
ssh notes-api          # Connects to deploy@203.0.113.42
scp notes-api:file .    # Copies file from server to current directory
```

Key Takeaways

- **SSH with key authentication** is the standard way to access a production server. Disable password authentication and root login immediately — in a drop-in file under `/etc/ssh/sshd_config.d/`, checked with `sshd -T`.
- **Create a non-root user** (`deploy`) for all application operations. Applications running as root give attackers root access if compromised.

- **UFW** (Uncomplicated Firewall) makes iptables rule management straightforward. The rule: allow only what is necessary (ports 22, 80, 443). Block everything else by default. **Docker-published ports bypass UFW** — publish them on `127.0.0.1` only, and add the provider's cloud firewall.
- **Docker Engine** (not Docker Desktop) is the server-side container runtime. Enable it to start on boot. Configure log rotation to prevent disk fill.
- **Automatic security updates** (unattended-upgrades) apply security patches without manual intervention. Configure it immediately — vulnerabilities are discovered continuously.
- **System limits** (file descriptors, network parameters): know which settings reach the application before changing them. `limits.conf` affects login sessions, not containers; network tuning belongs in a drop-in file under `/etc/sysctl.d/`.
- **UTC timezone** for all production servers. Log timestamps in UTC are consistent and unambiguous across timezones and daylight saving transitions.
- **fail2ban** automatically blocks IP addresses that repeatedly fail authentication. Essential for any server with SSH exposed to the internet.
- **Verify the setup** with a checklist script before deploying anything. A missed step discovered after deployment is much harder to fix.

Exercises

1. **Predict.** After disabling password authentication, try `ssh -o PubkeyAuthentication=no deploy@server`. What happens?
2. **Break and fix.** Run a test container with `-p 8080:80` on the server, with UFW denying 8080. Can you reach it from outside? Explain why, then fix it the way the book does.
3. **Extend.** Configure `unattended-upgrades` to email you (or write to a log you check) when it installs updates, and to reboot automatically at 04:00 UTC when required.
4. **Explain.** Why is logging in as `root` over SSH disabled, when the `deploy` user can `sudo` anyway?

Solutions and hints are in the companion repository, in `exercises/chapter-21.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

The server is hardened, Docker is installed, and the directory structure is ready. The next step is configuring the server's network: assigning a static IP, understanding how traffic flows from the internet to the server, and configuring the network rules at the cloud provider level (above

and in addition to the server-level UFW firewall).

→ [Chapter 22 — Network Configuration](#)

Network Configuration

The server is prepared: Docker installed, users created, SSH secured, UFW firewall active. But the server is not yet reachable from the internet in a useful way. Traffic from the internet needs a clear path to the application — and most of that path is defined not on the server itself, but at the network layer surrounding it.

This chapter covers network configuration at two levels:

1. **Cloud-level networking:** the virtual network infrastructure managed by the cloud provider (VPCs, reserved IPs, cloud firewalls, load balancers)
2. **Server-level networking:** the network configuration on the server itself (network interfaces, routing, the relationship between Nginx and Gunicorn)

Understanding both levels is necessary because they interact. A cloud firewall blocks traffic before it reaches the server's own firewall. Nginx listens on a different port than Gunicorn. These relationships must be explicit.

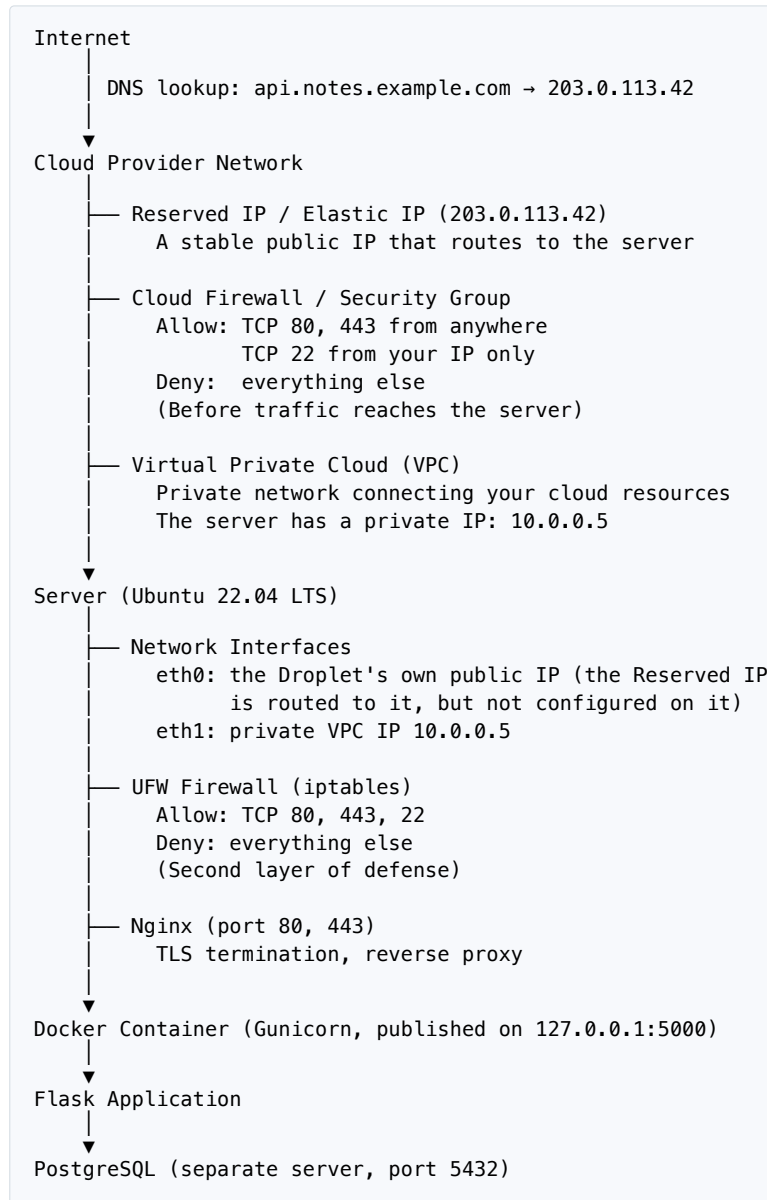
In this chapter

By the end of this chapter, you will understand:

- What a Virtual Private Cloud (VPC) is and why it matters
- What a reserved IP (floating IP, elastic IP) is and why you need one
- How cloud-level firewalls (security groups) differ from server-level firewalls (UFW)
- How to verify that traffic can reach the server
- How the network path from the internet to Gunicorn works
- What NAT is in the context of cloud networking

22.1 The Network Layers

Traffic from the internet to the Notes API passes through several distinct network layers:



Each layer has its own rules. Traffic must pass every layer to reach the application.

22.2 Virtual Private Cloud (VPC)

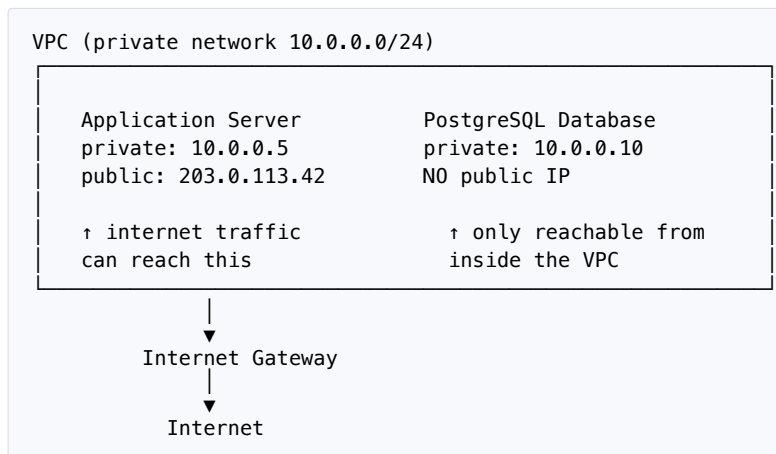
A **Virtual Private Cloud (VPC)** is an isolated private network within the cloud provider's infrastructure. Resources you create (servers, databases, load balancers) are placed within a VPC.

Think of a VPC as a private office building: it has its own private address space, its own internal routing, and controlled connections to the outside world (the internet). Resources inside the VPC communicate with each other using private IP addresses. Only resources explicitly configured to be internet-facing have public IP addresses.

Why VPCs matter for security

Without a VPC, every cloud resource would be on the public internet with a public IP. A PostgreSQL database with a public IP is reachable by anyone who knows the IP and can authenticate. With a VPC:

- The PostgreSQL database lives inside the VPC with only a private IP
- The application server lives inside the VPC, with a public IP
- The application server reaches PostgreSQL using its private IP (never leaving the VPC)
- Attackers cannot directly reach PostgreSQL — there is no route from the internet to the private IP



DigitalOcean creates a default VPC for each region. New Droplets are automatically placed in the VPC. A managed PostgreSQL database joins the VPC too, but it also gets a *public* hostname by default. Use its private (VPC) hostname in `DATABASE_URL`, and restrict the database's **trusted sources** to your Droplet, so that the public endpoint accepts connections from nothing

else.

22.3 The Public IP Address

Every server needs a **public IP address** to be reachable from the internet. This is a globally unique IP that the internet’s routing infrastructure knows how to deliver packets to.

Static vs dynamic IPs

A server’s own public IP is not permanent. On AWS EC2 it changes every time an instance is stopped and started. A DigitalOcean Droplet keeps its IP for as long as the Droplet exists — but a replacement Droplet (after a rebuild, or a move to a bigger machine) gets a new one. Either way, if your DNS record points to 203.0.113.42 and the server’s IP becomes 203.0.113.99, your domain stops working until the DNS record is updated.

A **static IP** (called a **Reserved IP** on DigitalOcean — formerly *Floating IP* — an **Elastic IP** on AWS, and a **static external IP** on Google Cloud) is a public IP that you own independently of any specific server. You can associate it with different servers. If you destroy a server and create a new one, you re-attach the same IP — your DNS record never needs to change.

Creating a Reserved IP on DigitalOcean

1. In the DigitalOcean control panel: Networking → Reserved IPs → Reserve a Reserved IP
2. Assign it to your Droplet

```
# Verify the Reserved IP is reachable
ping 203.0.113.42 # Replace with your actual Reserved IP
```

The Reserved IP does not appear on any of the server’s network interfaces. Look at them on the server:

```
# On the server
ip addr show
# 2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
#      inet 198.51.100.17/20 brd 198.51.111.255 scope global eth0
#      inet 10.17.0.5/16 brd 10.17.255.255 scope global eth0
# 3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
#      inet 10.0.0.5/20 brd 10.0.15.255 scope global eth1
```

The server has:

- 198.51.100.17 on eth0 — the Droplet’s own public IP

- **10.17.0.5** on `eth0` — an **anchor IP**, which DigitalOcean’s network uses to deliver traffic for the Reserved IP (**203.0.113.42**) to this Droplet
- **10.0.0.5** on `eth1` — the private VPC IP (used for internal communication)

Incoming connections to the Reserved IP arrive through the anchor IP; the server’s own outgoing connections still leave from **198.51.100.17**. This book uses the Reserved IP, **203.0.113.42**, as “the server’s IP” from here on.

22.4 Cloud-Level Firewalls

Most cloud providers offer a **cloud-level firewall** that operates before traffic reaches the server. This is different from and complementary to the server-level UFW firewall.

Why two firewalls?

Cloud firewall (DigitalOcean Cloud Firewalls, AWS Security Groups):

- Operated by the cloud provider’s network infrastructure
- Traffic is dropped before it reaches the server’s network interface
- No CPU usage on the server (the server never sees rejected traffic)
- Centrally managed — apply one firewall rule to multiple servers
- Sits outside the server, so it also filters ports that Docker publishes — which bypass UFW (Chapter 21)
- If a rule locks you out, you fix it in the provider’s control panel, not over SSH

Server firewall (UFW/iptables):

- Operates on the server itself
- Provides a second layer of defense if the cloud firewall is misconfigured
- Can implement more granular rules (per-application, per-process)
- Required when the cloud provider does not offer network-level firewalls

Using both provides **defense in depth**: two independent layers of protection. A misconfiguration in one does not immediately expose the server.

Configuring a DigitalOcean Cloud Firewall

In the DigitalOcean control panel: Networking → Firewalls → Create Firewall

Configure inbound rules:

Type	Protocol	Port Range	Sources
SSH	TCP	22	YOUR_IP_ADDRESS/32
HTTP	TCP	80	All IPv4, All IPv6
HTTPS	TCP	443	All IPv4, All IPv6

Leave outbound rules as defaults (allow all outbound).

Assign the firewall to your Droplet.

Restricting SSH to your IP at the cloud level is more effective than restricting it at the server level: the connection attempt never reaches the server. Attackers see their connection dropped by the cloud network, not even establishing a TCP handshake.

22.5 Verifying Network Connectivity

After configuring firewalls, verify that the correct ports are reachable.

From the server (outbound tests)

```
# Test internet connectivity
curl -s https://example.com > /dev/null && echo "Internet: OK"

# Test DNS resolution
dig google.com +short && echo "DNS: OK"

# Test that PostgreSQL is reachable (if using a managed DB).
# Replace with your database's private host; DigitalOcean's managed
# PostgreSQL listens on port 25060
nc -zv -w 5 private-db-host.example.com 25060 && echo "DB: OK"
```

From your laptop (inbound tests)

```
# Test that port 22 is open
nc -zv 203.0.113.42 22
# Connection to 203.0.113.42 22 port [tcp/ssh] succeeded!

# Test that port 80 is open
nc -zv 203.0.113.42 80
# Connection to 203.0.113.42 80 port [tcp/http] succeeded!

# Test that port 443 is open
nc -zv 203.0.113.42 443
# Connection to 203.0.113.42 443 port [tcp/https] succeeded!

# Test that port 5000 is NOT open (should fail)
nc -zv -w 5 203.0.113.42 5000 2>&1
# nc: connect to 203.0.113.42 port 5000 (tcp) timed out: Operation now in progress
```

Port 5000 is used by Gunicorn but must be reachable only from the server itself (via Nginx), not from the internet. Notice *how* the test fails: a firewall that drops packets makes the connection attempt **time out**, while “Connection refused” means the packets reached the server and nothing was listening. Either is a pass here; a *successful* connection means the application port is exposed — check that the container is published on 127.0.0.1 (Chapter 21).

Network diagnostic tools

```
# On the server: show which processes are listening on which ports
# (output simplified; the real Process column reads users:(("sshd",pid=...,fd=3)))
sudo ss -tlnp
# State  Recv-Q  Send-Q  Local Address:Port  Process
# LISTEN  0        128      0.0.0.0:22          sshd
# LISTEN  0        511      0.0.0.0:80          nginx
# LISTEN  0        511      0.0.0.0:443         nginx
# LISTEN  0       4096     127.0.0.1:5000      docker-proxy (gunicorn inside)

# Check routing table
ip route
# default via 198.51.96.1 dev eth0 proto static
# 10.0.0.0/20 dev eth1 proto kernel scope link src 10.0.0.5

# Check network interfaces
ip addr show
```

22.6 Docker Networking Inside the Container

When Docker runs the Notes API container, it creates an internal network for that container. Understanding this network is necessary for configuring Nginx correctly.

Docker bridge network

By default, Docker creates a **bridge network** — a private network inside the host, managed by Docker:

```
# View Docker networks
docker network ls
# NETWORK ID      NAME      DRIVER      SCOPE
# a1b2c3d4e5f6    bridge    bridge      local
# b2c3d4e5f6a7    host      host        local
# c3d4e5f6a7b8    none      null        local
```

When a container runs on the default bridge network:

- The container gets a private IP in the 172.17.0.0/16 range (e.g., 172.17.0.2)
- The host gets 172.17.0.1 (the Docker bridge IP)
- The container can reach the internet via the host's NAT
- The host can reach the container via its container IP

Port mapping creates a rule: traffic arriving at a host port is forwarded to the container's port. On the server, the mapping always names the loopback address, `-p 127.0.0.1:5000:5000`, so only programs on the server itself can use it:

```
Nginx on the host → 127.0.0.1:5000 → Docker bridge NAT → container 172.17.0.2:5000
```

Without the `127.0.0.1:` part, Docker would listen on every interface and add firewall rules that route internet traffic around UFW to the container (Chapter 21).

Nginx reaching Unicorn

When Nginx (running on the host) needs to forward requests to Unicorn (running in the container), it connects to `127.0.0.1:5000` — the host's port, forwarded by Docker's port mapping to the container.


```
# Nginx configuration for proxying to Gunicorn
upstream notes_api {
    server 127.0.0.1:5000;    # Host's port 5000 → Docker → container's port 5000
}
```

Write `127.0.0.1`, not `localhost`: `localhost` can resolve to the IPv6 address `::1` first, where nothing is listening, and Nginx would log a failed connection before falling back.

This is the most straightforward configuration when Nginx and the container run on the same server.

The docker-compose network

When using Docker Compose, all services share a custom network where they reach each other by service name:

```
services:
  app:
    ...
  db:
    ...

# Docker Compose creates a network where:
# - 'app' container can reach 'db' at hostname 'db'
# - 'db' container can reach 'app' at hostname 'app'
```

This is why `DATABASE_URL=postgresql://notes_user:pass@db:5432/notesdb` works in `docker-compose.yml` — `db` resolves to the PostgreSQL container's IP on the compose network.

22.7 Network Monitoring and Diagnostics

These commands are useful for diagnosing network issues on the server:

```
# What is listening on what port?
sudo ss -tlnp          # Modern replacement for netstat -tlnp

# Is the container's port exposed?
docker ps --format "table {{.Names}}\t{{.Ports}}"

# Trace the route a packet takes to a destination
traceroute 8.8.8.8
```

```
# Test connectivity to a specific port
nc -zv 203.0.113.42 443

# Capture network packets (useful for debugging – stop with Ctrl+C)
tcpdump -i eth0 port 80

# Check iptables rules (Docker adds many rules here)
sudo iptables -L -n -v --line-numbers | head -50

# Check Docker's iptables rules. These DNAT rules in the nat table are
# what forward published ports to containers – before UFW's INPUT rules
# ever see the traffic, which is why published ports bypass UFW.
sudo iptables -t nat -L -n -v | grep -A5 DOCKER
```

22.8 Network Security Best Practices

Beyond the firewall configuration, several network-level security practices protect the server:

Fail2ban (configured in Chapter 21)

Automatically bans IPs with repeated authentication failures. This complements firewall rules by dynamically blocking active attack sources.

Private network for database access

The PostgreSQL database should never be accessible from the public internet. Use the database's private (VPC) hostname for the DATABASE_URL, and require TLS:

```
# Wrong – uses the public PostgreSQL endpoint
DATABASE_URL=postgresql://user:pass@db-postgresql-nyc3-12345.db.ondigitalocean.com:25
↳ 060/notesdb

# Correct – uses the private VPC hostname, with TLS required
DATABASE_URL=postgresql://user:pass@private-db-postgresql-nyc3-12345.db.ondigitalocea
↳ n.com:25060/notesdb?sslmode=require
```

Use the hostname, not its IP address: the database's TLS certificate is issued for the name, so connecting by IP breaks certificate verification.

Traffic between the application server and the database stays inside the VPC — it never traverses the public internet, reducing latency and eliminating exposure.

Rate limiting at the network level

DDoS (Distributed Denial of Service) attacks flood the server with traffic. Cloud providers offer DDoS protection at the network level (DigitalOcean includes basic DDoS protection). For application-level rate limiting, see [Chapter 38](#).

Key Takeaways

- Network configuration operates at **two levels**: the cloud provider's network infrastructure (cloud firewalls, reserved IPs, VPCs) and the server's own network stack (UFW, network interfaces, Docker networking).
- A **VPC (Virtual Private Cloud)** is an isolated private network for your cloud resources. Databases should have only private VPC IPs — never public IPs.
- A **Reserved IP** (DigitalOcean; formerly Floating IP — Elastic IP on AWS) is a stable public IP address that persists independently of any specific server. Always use one for production — your DNS records never need to change when a server is replaced.
- **Two firewalls provide defense in depth**: the cloud-level firewall drops traffic before it reaches the server; UFW on the server drops traffic before it reaches its services. A misconfiguration in one does not immediately expose the server — and only the cloud firewall also covers ports that Docker publishes.
- **Docker networking**: containers on the default bridge network get private IPs in 172.17.0.0/16. Port mapping (`-p 127.0.0.1:5000:5000`) forwards traffic from the host's loopback port to the container. Nginx on the host connects to 127.0.0.1:5000 to reach Gunicorn in the container.
- **Only allow necessary ports**: 22 (SSH — from your IP only), 80 (HTTP), 443 (HTTPS). Port 5000 (Gunicorn) must not be accessible from the internet.
- **PostgreSQL must use the private VPC endpoint**, with TLS. Traffic between the application and database should stay inside the VPC — never on the public internet.

Exercises

1. **Predict.** From your laptop, run `nc -zv <server-ip> 5432`. Should it connect? What would it mean if it did?
2. **Break and fix.** Temporarily remove port 443 from the cloud firewall. What does a browser report, and how does it differ from Nginx being stopped? Restore the rule.
3. **Extend.** Draw the network diagram of your deployment, marking every firewall, every IP (public, private, reserved) and every port that is open, and for whom.
4. **Explain.** Why use two firewalls (the cloud's and UFW) when either one could block the same ports?

Solutions and hints are in the companion repository, in `exercises/chapter-22.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

The network layers are configured and verified. Traffic can reach the server on ports 80 and 443. The server is ready to receive the application.

The next step is getting the application code and its Docker image onto the server. This is the code transfer problem: how does the image stored in the registry get pulled and running on the server?

→ [Chapter 23](#) — Code Transfer

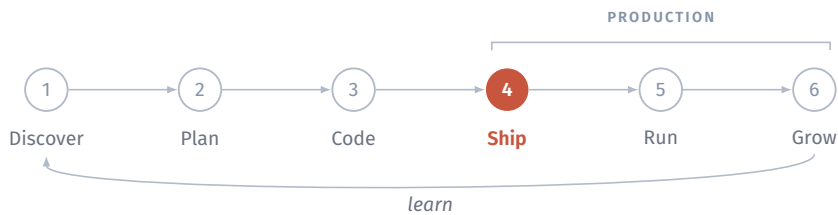
STAGE 6

6

Deployment

How is it deployed and kept alive?

YOU ARE HERE



The piece: The program, placed on the machine and kept running

The whole map: *How Software Gets Built*, page 1

Code Transfer

The application is built. The Docker image is pushed to the registry. The server is configured and waiting. The final preparation step is getting the image from the registry to the server — and establishing the workflow for doing this reliably every time a new version is deployed.

Code transfer is the process of moving a new version of the application from where it was built to where it will run. In a Docker-based deployment, “code transfer” primarily means pulling the Docker image from the registry onto the server. But it also includes any configuration files, environment files, and scripts that the server needs but that are not inside the Docker image.

This chapter covers the mechanics of image transfer, the deployment script that orchestrates the update, and the zero-downtime deployment pattern that replaces old containers with new ones without dropping requests.

In this chapter

By the end of this chapter, you will understand:

- Why you pull images rather than copy code files
- How `docker pull` works and what it transfers
- How to authenticate the server to the registry
- How to write a deployment script that updates the running container
- What zero-downtime deployment is and how to achieve it with Docker
- How to verify a deployment succeeded before declaring it complete
- How to roll back a failed deployment

23.1 Why Pull Images, Not Copy Files

Early in software development history, “deploying” meant copying source files to the server and running them. Some projects still do this with `git pull` on the server or `scp` to copy files.

This approach has problems:

- **Environment differences:** the server might have a different Python version, different system libraries, different pip packages. “Copy and run” brings back all the environment drift problems from [Chapter 7](#).
- **Build on the server:** if you copy source code and install dependencies on the server, you are running `pip install` in production — with internet access, with potential version drift ([Chapter 12](#)), and with build tools that should not be in production.
- **No atomic deployment:** if the server crashes while files are being copied, it runs partially-updated code.

Pulling a Docker image solves all of these:

- **No environment differences:** the image contains the entire environment (Python version, system libraries, packages). It runs the same way on every machine of the same CPU architecture — and CI builds it on x86-64 runners, the architecture of the server.
- **No build on the server:** the image was built in CI. The server only runs `docker pull` and `docker run`.
- **Atomic artifact:** a Docker image is an immutable artifact. Either the pull succeeds (complete image) or it fails (old image unchanged). There is no half-copied state. (The *switchover* from the old container to the new one is a separate step, and it still needs care — sections 23.4 and 23.5.)

23.2 How docker pull Works

`docker pull` downloads a Docker image from a registry to the local machine:

```
docker pull ghcr.io/your-username/notes-api:sha-a1b2c3d
```

What happens:

1. Docker contacts the registry at `ghcr.io`
2. The registry returns the image’s **manifest**: a list of all layers and their SHA-256 hashes
3. For each layer, Docker checks if it already exists locally (by hash)
4. Docker downloads only the layers that are not already present
5. The image is available locally

Layer caching on the server

Just as the Docker build caches layers during build (Chapter 18), Docker caches pulled layers on the server. The `python:3.11-slim` base layers and the user-creation layer are the same across all Notes API image versions. Once pulled once, they are never downloaded again.

For a typical code change:

- Base image layers: already cached — 0 bytes transferred
- `pip install` layer: already cached (if `requirements.txt` unchanged) — 0 bytes transferred
- Application code layer: changed — ~100 KB transferred

Most deployments transfer only kilobytes, not gigabytes.

23.3 Authenticating the Server to the Registry

The registry at `ghcr.io` requires authentication to pull images from private repositories. The server needs credentials to pull the image.

GitHub Personal Access Token approach

Create a GitHub Personal Access Token (classic) with only the `read:packages` scope, just for this server — so it can pull images and nothing else, and you can revoke it without affecting anything else. Then log in once, as the `deploy` user:

```
# On the server, as deploy. read -s keeps the token out of your
# shell history and off the screen.
read -rsp "GitHub token: " GITHUB_TOKEN; echo
echo "$GITHUB_TOKEN" | docker login ghcr.io -u your-username --password-stdin
unset GITHUB_TOKEN
# WARNING! Your password will be stored unencrypted in
↪ /home/deploy/.docker/config.json.
# Login Succeeded
```

The login persists: Docker saves the credentials in `~/.docker/config.json`, and every later `docker pull` by `deploy` uses them, so the deployment script does not need the token at all. Take the warning seriously, though. Without a **credential helper** (a program that keeps secrets in an OS keychain), the token is stored base64-encoded, which is encoding, not encryption. The file is readable only by `deploy` — check with `ls -l ~/.docker/config.json` — and that, plus the token's narrow scope, is the protection. Do not keep a second copy of the token in a script file.

AWS ECR authentication

AWS ECR uses IAM roles for authentication. If the server is an EC2 instance with an appropriate IAM role:

```
# Get authentication token and log in to ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
  123456789.dkr.ecr.us-east-1.amazonaws.com
```

This is more secure than PATs: the EC2 instance's IAM role automatically grants pull access, with no long-lived credentials to manage or rotate. The login token it produces expires after 12 hours, though, so a deployment script must run this command each time (or use the ECR credential helper, which does it automatically).

23.4 The Deployment Script

A deployment script is an executable script that performs all steps of a deployment in the correct order. It should be:

- **Idempotent:** running it twice produces the same result as running it once
- **Safe on failure:** either the new version ends up running and healthy, or the previous version is put back
- **Verified:** the script checks that the new version is actually working before declaring success
- **Logged:** every step writes to the log so you can diagnose failures

Listing 23.1 /opt/notes-api/deploy.sh

```
#!/bin/bash
# Deployment script for the Notes API.
#
# Usage:
#   ./deploy.sh sha-a1b2c3d    Deploy a specific image version
#
# There is deliberately no "latest" default: a deployment names the exact
# image it installs, so it can be audited and rolled back (Chapter 19).
#
# This script:
#   1. Records the running version (for rollback)
#   2. Pulls the new Docker image
```

```

# 3. Stops and removes the old container
# 4. Starts the new container
# 5. Waits for the health check to pass
# 6. If it does not pass, rolls back to the previous version

set -euo pipefail
# set -e: exit immediately if any command fails
# set -u: treat unset variables as errors
# set -o pipefail: if any command in a pipe fails, the whole pipe fails

# -----
# Configuration
# -----

# Replace with your GitHub username, in lowercase
IMAGE_REPO="ghcr.io/your-username/notes-api"
IMAGE_TAG="${1:?Usage: deploy.sh sha-<7-character commit hash>}"

FULL_IMAGE="${IMAGE_REPO}:${IMAGE_TAG}"
CONTAINER_NAME="notes-api"
ENV_FILE="/opt/notes-api/config/notes-api.env"
DEPLOY_LOG_DIR="/opt/notes-api/deploy-logs"
HEALTH_URL="http://127.0.0.1:5000/health"

# -----
# Logging (the directory must exist before the first log line)
# -----

mkdir -p "$DEPLOY_LOG_DIR"
LOG_FILE="${DEPLOY_LOG_DIR}/deploy-$(date +%Y%m%d-%H%M%S).log"

log() {
    local msg="[$(date +%Y-%m-%dT%H:%M:%S)] $*"
    echo "$msg"
    echo "$msg" >> "$LOG_FILE"
}

die() {
    log "ERROR: $*"
    exit 1
}

# Start the container from a given image. One function, so the deployment
# and the rollback cannot drift apart.

```

```

# -p 127.0.0.1:5000:5000  publish on loopback only: Nginx reaches it,
#                        the internet cannot (Docker bypasses UFW)
# --stop-timeout 35      give Gunicorn time to finish in-flight
#                        requests when stopped (Chapter 24)
start_container() {
    docker run -d \
        --name "$CONTAINER_NAME" \
        --restart unless-stopped \
        --stop-timeout 35 \
        -p 127.0.0.1:5000:5000 \
        --env-file "$ENV_FILE" \
        "$1" >> "$LOG_FILE" 2>&1
}

# -----
# Pre-deployment checks
# -----

log "Starting deployment: ${FULL_IMAGE}"

[[ "$IMAGE_TAG" =~ ^sha-[0-9a-f]{7}$ ]] || \
    die "Tag must look like sha-a1b2c3d (lowercase), got: $IMAGE_TAG"
[ -f "$ENV_FILE" ] || die "Environment file not found: $ENV_FILE"

# -----
# Step 1: Record the currently running image (for rollback)
# -----

# Record the image ID (sha256:...), not the name the container was started
# with: a name such as ":latest" may point at a different image by now.
PREVIOUS_IMAGE=""
if docker container inspect "$CONTAINER_NAME" > /dev/null 2>&1; then
    PREVIOUS_IMAGE=$(docker inspect "$CONTAINER_NAME" --format '{{.Image}}')
    PREVIOUS_NAME=$(docker inspect "$CONTAINER_NAME" --format '{{.Config.Image}}')
    log "Current image: ${PREVIOUS_NAME} (${PREVIOUS_IMAGE:7:12})"
else
    log "No existing container found. This is a fresh deployment."
fi

# -----
# Step 2: Pull the new image
# (uses the registry login saved in ~/.docker/config.json – section 23.3)
# -----

```

```

log "Pulling image: ${FULL_IMAGE}"
docker pull "$FULL_IMAGE" >> "$LOG_FILE" 2>&1 || die "Pull failed. Is the tag right,
↳ and is the server logged in to the registry?"
log "Image pull: complete"

# -----
# Step 3: Stop and remove the old container
# -----

# "container inspect" finds the container whether it is running or has
# exited – a crashed container is a common reason to redeploy, and its
# name must be freed before the new container can use it.
if docker container inspect "$CONTAINER_NAME" > /dev/null 2>&1; then
    log "Stopping and removing existing container: $CONTAINER_NAME"
    docker stop "$CONTAINER_NAME" >> "$LOG_FILE" 2>&1
    docker rm "$CONTAINER_NAME" >> "$LOG_FILE" 2>&1
fi

# -----
# Step 4: Start the new container
# -----

log "Starting new container with image: ${FULL_IMAGE}"
start_container "$FULL_IMAGE"
log "Container started: $(docker inspect "$CONTAINER_NAME" --format '{{.Id}}' | cut
↳ -c1-12)"

# -----
# Step 5: Wait for the health check to pass
# -----

log "Waiting for application to become healthy..."

RETRIES=30          # Try 30 times
INTERVAL=2          # Wait 2 seconds between tries (about 60 seconds in all)

for i in $(seq 1 $RETRIES); do
    # The response must be a 2xx (curl -f) AND say "ok"
    if curl -sf --max-time 3 "$HEALTH_URL" 2>/dev/null | grep -q '"status":"ok"';
    then
        log "Health check passed (attempt $i of $RETRIES)."
```

```

if [ "$i" -eq "$RETRIES" ]; then
    log "Health check failed after $((RETRIES * INTERVAL)) seconds."
    log "Container logs (last 50 lines) saved to $LOG_FILE"
    docker logs --tail=50 "$CONTAINER_NAME" >> "$LOG_FILE" 2>&1 || true

    # -----
    # Rollback: start the previous version if there was one
    # -----
    docker rm -f "$CONTAINER_NAME" >> "$LOG_FILE" 2>&1 || true
    if [ -n "$PREVIOUS_IMAGE" ]; then
        log "Rolling back to: ${PREVIOUS_NAME} (${PREVIOUS_IMAGE:7:12})"
        start_container "$PREVIOUS_IMAGE"
        log "Rollback complete. Previous version is running."
    else
        log "No previous version to roll back to."
    fi

    die "Deployment failed. Check logs at: $LOG_FILE"
fi

sleep "$INTERVAL"
done

# -----
# Deployment complete
# -----

log "Deployment complete: ${FULL_IMAGE}"
log "Health: $(curl -sf --max-time 5 "$HEALTH_URL")"
log "Log file: $LOG_FILE"

```

Make the script executable:

```

chmod +x /opt/notes-api/deploy.sh
chown deploy:deploy /opt/notes-api/deploy.sh

```

Running a deployment

```

# From your laptop, trigger a deployment on the server:
ssh deploy@203.0.113.42 '/opt/notes-api/deploy.sh sha-a1b2c3d'

```

Expected output:

```

[2024-01-15T14:00:00] Starting deployment:
                        ghcr.io/your-username/notes-api:sha-a1b2c3d
[2024-01-15T14:00:00] Current image:
                        ghcr.io/your-username/notes-api:sha-9f8e7d6 (3c9d4e1f2a7b)
[2024-01-15T14:00:00] Pulling image:
                        ghcr.io/your-username/notes-api:sha-a1b2c3d
[2024-01-15T14:00:04] Image pull: complete
[2024-01-15T14:00:04] Stopping and removing existing container: notes-api
[2024-01-15T14:00:05] Starting new container with image:
                        ghcr.io/your-username/notes-api:sha-a1b2c3d
[2024-01-15T14:00:06] Container started: d4e5f6a7b8c9
[2024-01-15T14:00:06] Waiting for application to become healthy...
[2024-01-15T14:00:08] Health check passed (attempt 2 of 30).
[2024-01-15T14:00:08] Deployment complete:
                        ghcr.io/your-username/notes-api:sha-a1b2c3d
[2024-01-15T14:00:08] Health: {"status":"ok"}
[2024-01-15T14:00:08] Log file: /opt/notes-api/deploy-logs/deploy-20240115-140000.log

```

(Long lines are wrapped here for the page; the script prints each message on one line.)

23.5 Zero-Downtime Deployment

The deployment script in Section 23.4 stops the old container and starts the new one. There is a brief period — typically 2–10 seconds — when no container is running. Requests arriving during this window fail.

For the Notes API in a learning context, this is acceptable. For a production service with real users, even a 2-second downtime might cause failed requests, timeout errors, and user complaints.

Zero-downtime deployment replaces the old version with the new one without any gap in service.

The blue-green pattern

Blue-green deployment runs two identical environments (called “blue” and “green”):

Before deployment:

Blue container (port 5000) ← receiving traffic
 Green container (port 5001) ← standing by (old version or empty)

During deployment:

Blue container (port 5000) ← still receiving traffic
 Green container (port 5001) ← started with NEW version

After health check passes on green:

Update Nginx to point to green (port 5001)
 Blue container ← now idle (old version)

Cleanup:

Stop blue container
 Next deployment: blue gets the new version

The key: Nginx does the traffic switch, and a reload lets requests already in progress finish on the old container. There is no moment when no container is running.

Blue-green deployment needs Nginx, which is installed and configured in [Chapter 28](#); if you are following along in order, use Listing 23.1 until then and come back to this section afterwards. Two pieces of setup come first. The script runs as `deploy`, but rewriting an Nginx file and reloading Nginx need root, so grant exactly those three commands — and nothing else — through a `sudoers` drop-in:

```
# On the server, as a sudo user. visudo checks the syntax before saving:
# a broken sudoers file can lock you out of sudo entirely.
sudo visudo -f /etc/sudoers.d/notes-api-deploy
```

```
deploy ALL=(root) NOPASSWD: /usr/bin/tee /etc/nginx/conf.d/notes-api-upstream.conf,
↳ /usr/sbin/nginx -t, /usr/bin/systemctl reload nginx
```

And the file `/etc/nginx/conf.d/notes-api-upstream.conf` is the **only** place the `notes_api` upstream is defined. The site configuration in [Chapter 28](#) refers to it by name (`proxy_pass http://notes_api;`) instead of defining its own; two upstream `notes_api` blocks would make `nginx -t` fail with “duplicate upstream”.

Listing 23.2 /opt/notes-api/deploy-bluegreen.sh (zero-downtime, blue-green)

```
#!/bin/bash
set -euo pipefail

IMAGE_REPO="ghcr.io/your-username/notes-api"
IMAGE_TAG="${1:?Usage: deploy-bluegreen.sh sha-<7-character commit hash>}"
FULL_IMAGE="${IMAGE_REPO}:${IMAGE_TAG}"
ENV_FILE="/opt/notes-api/config/notes-api.env"
UPSTREAM_CONF="/etc/nginx/conf.d/notes-api-upstream.conf"

log() { echo "[$(date +%Y-%m-%dT%H:%M:%S)] $*"; }
die() { log "ERROR: $*"; exit 1; }

# Determine which slot is currently active.
# (name=~.$ matches the whole name; a plain name filter matches substrings)
if docker ps -q --filter "name=~notes-api-blue$" | grep -q .; then
    ACTIVE_CONTAINER="notes-api-blue"; INACTIVE="green"; INACTIVE_PORT="5001"
elif docker ps -q --filter "name=~notes-api-green$" | grep -q .; then
    ACTIVE_CONTAINER="notes-api-green"; INACTIVE="blue"; INACTIVE_PORT="5000"
else
    # First blue-green deployment: the container from Listing 23.1
    # ("notes-api") still holds port 5000, so start with green on 5001
    ACTIVE_CONTAINER="notes-api"; INACTIVE="green"; INACTIVE_PORT="5001"
fi

log "Active: $ACTIVE_CONTAINER"
log "Deploying to: notes-api-$INACTIVE (port $INACTIVE_PORT)"

# Pull new image
log "Pulling image: $FULL_IMAGE"
docker pull "$FULL_IMAGE"

# Remove the old inactive container if it exists (running or not)
docker rm -f "notes-api-$INACTIVE" 2>/dev/null || true

# Start the new container on the inactive port (loopback only)
log "Starting notes-api-$INACTIVE on port $INACTIVE_PORT..."
docker run -d \
    --name "notes-api-$INACTIVE" \
    --restart unless-stopped \
    --stop-timeout 35 \
    -p "127.0.0.1:${INACTIVE_PORT}:5000" \
    --env-file "$ENV_FILE" \
```

```

"$FULL_IMAGE"

# Wait for health check on the new container
log "Waiting for health check on port $INACTIVE_PORT..."
for i in $(seq 1 30); do
    if curl -sf --max-time 3 "http://127.0.0.1:${INACTIVE_PORT}/health" > /dev/null;
    ↪ then
        log "Health check passed."
        break
    fi
    if [ "$i" -eq 30 ]; then
        docker rm -f "notes-api-$INACTIVE" || true
        die "Health check failed. $ACTIVE_CONTAINER stays active."
    fi
    sleep 2
done

# Switch Nginx to the new container
log "Switching traffic to notes-api-$INACTIVE..."
printf 'upstream notes_api {\n    server 127.0.0.1:%s;\n}\n' "$INACTIVE_PORT" \
| sudo tee "$UPSTREAM_CONF" > /dev/null

# Test and reload Nginx (graceful: old workers finish their requests)
sudo nginx -t
sudo systemctl reload nginx
log "Traffic switched to notes-api-$INACTIVE (port $INACTIVE_PORT)."
```

*# Let requests already in flight on the old container finish before
stopping it: Nginx's old worker processes may still be proxying them*

```

log "Draining $ACTIVE_CONTAINER for 15 seconds..."
sleep 15

log "Stopping old container: $ACTIVE_CONTAINER"
docker stop "$ACTIVE_CONTAINER" 2>/dev/null || true
docker rm "$ACTIVE_CONTAINER" 2>/dev/null || true

log "Deployment complete. Running: notes-api-$INACTIVE with $FULL_IMAGE"
```

No request finds the door closed: new connections go to the new container from the moment of the reload, and the old container keeps running through the drain period, until the requests Nginx had already sent it are finished. (The 15 seconds is a judgment call: long enough for the Notes API's requests, which take milliseconds. An application with slow requests needs a longer drain.)

23.6 Automating Deployment from CI

The final step: trigger the deployment automatically when code is merged to main (no manual SSH required).

Update the GitHub Actions workflow to include a deployment step:

```
# Additions to .github/workflows/build.yml

deploy:
  name: Deploy to Production
  runs-on: ubuntu-latest
  needs: publish      # Only deploy if the image was published successfully
  if: github.ref == 'refs/heads/main'

  environment:
    name: production
    # GitHub Environments show deployment status in the repository's
    # deployments page, and can require an approval before this job runs
    # (if you add "required reviewers" to the environment's protection rules)

  steps:
    # github.sha is the full 40-character hash; the image tag uses the
    # first 7 characters (Chapter 19)
    - name: Compute the image tag
      id: tag
      run: echo "tag=sha-${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"

    - name: Deploy to production server
      # A third-party action that receives a production SSH key: pin it to
      # a full commit SHA of a release you have reviewed, not a movable tag
      uses: appleboy/ssh-action@<full-commit-sha> # v1.x
      with:
        host: ${ secrets.SERVER_IP }
        username: deploy
        key: ${ secrets.SERVER_SSH_KEY }
        # Verify the server's host key, so CI cannot be tricked into
        # connecting to an impostor (get it with: ssh-keyscan -t ed25519
        # 203.0.113.42 | ssh-keygen -lf -)
        fingerprint: ${ secrets.SERVER_HOST_FINGERPRINT }
        # One command, which the restricted CI key (below) allows
        script: /opt/notes-api/deploy.sh ${ steps.tag.outputs.tag }
```

Setting up GitHub Secrets:

In the GitHub repository settings → Secrets and variables → Actions:

- `SERVER_IP`: the server's IP address (203.0.113.42)
- `SERVER_SSH_KEY`: the contents of a private SSH key that has access to the server (generate a separate key pair just for CI)
- `SERVER_HOST_FINGERPRINT`: the server's SSH host key fingerprint

This job runs `deploy.sh` (Listing 23.1). To deploy with zero downtime instead, call `deploy-bluegreen.sh` (Listing 23.2) — the arguments are the same.

Adding the CI key to the server:

```
# Generate a key pair for CI (on your laptop). -N "" means no passphrase:
# CI cannot type one. The key's protection is where it is stored.
ssh-keygen -t ed25519 -C "ci-deploy" -N "" -f ~/.ssh/notes-api-ci

# Add the private key to GitHub Secrets
cat ~/.ssh/notes-api-ci
# Copy this to GitHub → Settings → Secrets → SERVER_SSH_KEY
```

Then add the public key to `/home/deploy/.ssh/authorized_keys` on the server — not as a plain line, but with restrictions in front of it. `deploy` is in the `sudo` and `docker` groups, so an unrestricted key would give whoever steals it root on the server. With `command=`, the key can run the deployment script and nothing else, whatever command the client asks for:

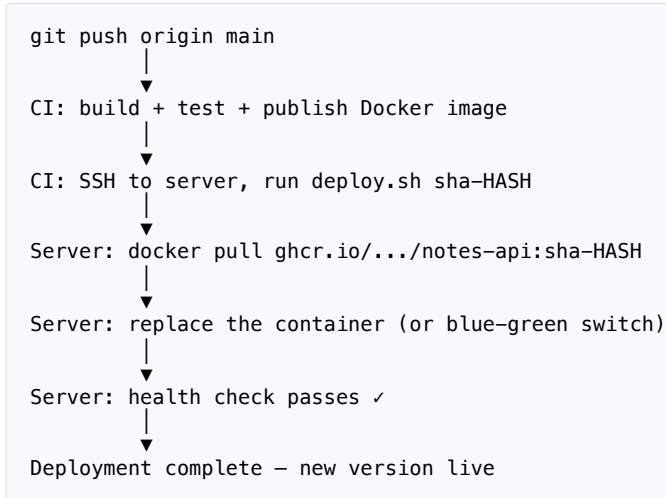
```
command="/opt/notes-api/ci-deploy.sh",no-pty,no-port-forwarding,no-agent-forwarding,
↪ no-X11-forwarding ssh-ed25519 AAAAC3Nza... ci-deploy
```

The small wrapper receives the requested command in `$SSH_ORIGINAL_COMMAND` and allows only a deployment:

```
#!/bin/bash
# /opt/notes-api/ci-deploy.sh - the only thing the CI key may run
set -euo pipefail
read -r -a args <<< "${SSH_ORIGINAL_COMMAND:-}"
[[ "${args[0]:-}" == "/opt/notes-api/deploy.sh" ]] || { echo "Not allowed"; exit 1; }
exec /opt/notes-api/deploy.sh "${args[1]:-}"
```

(`deploy.sh` itself rejects anything that is not a well-formed `sha-` tag. Test the whole path once from your laptop: `ssh -i ~/.ssh/notes-api-ci deploy@203.0.113.42 /opt/notes-api/deploy.sh sha-a1b2c3d`.)

Now the complete pipeline runs automatically:



23.7 Rollback

When a deployment fails (health check times out), the deploy script automatically rolls back (as seen in Listing 23.1). Manual rollback is also possible:

```
# Manual rollback to a previous version
ssh deploy@203.0.113.42 ' /opt/notes-api/deploy.sh sha-9f8e7d6'
```

Because every release image is tagged with the git commit hash and stored in the registry, any previous version can be deployed with one command. This is the core value of artifact storage from Chapter 19. “Any previous version” has a limit, though: the registry’s retention policy (Chapter 19 keeps the last 10 sha- images) defines how far back you can roll. Images tagged prod-... are kept regardless.

Key Takeaways

- **Pull images, do not copy code.** Docker images are immutable, self-contained environments. Pulling eliminates environment drift, and an image arrives whole or not at all.
- **Layer caching** makes `docker pull` efficient: only changed layers are transferred. Code changes typically download kilobytes, not gigabytes.
- **A deployment script** must pull the new image, stop the old container, start the new one, verify health, and roll back on failure. Every step should be logged. On the server,

- containers are always published on 127.0.0.1.
- **The health check endpoint** (GET /health) is the deployment's verification: if it returns {"status": "ok"}, the new process is up and serving HTTP. (It does not prove the database is reachable; [Chapter 35](#) adds a deeper check.)
 - **Zero-downtime deployment** (blue-green) starts the new version alongside the old, waits for it to be healthy, switches Nginx's upstream with a graceful reload, and drains the old container before stopping it. No gap in service.
 - **Automating deployment from CI** removes human error: every merge to main triggers a deployment. The `deploy.sh` script on the server is the deployment unit — CI just calls it, with a key that can do nothing else.
 - **Rollback is trivial** when every version is tagged by commit in the registry. `deploy.sh sha-previous_hash` rolls back to any historical version in seconds.

Exercises

1. **Predict.** Deploy a version whose `/health` returns 500. What does `deploy.sh` do, and what is running afterwards?
2. **Break and fix.** Revoke the server's registry token, then `deploy`. Which step fails, what does the log say, and is the old version still serving?
3. **Extend.** Make the `deploy` script post a one-line message (version, result, duration) to a chat channel through a webhook URL kept in the server's environment file.
4. **Explain.** Why does the book pull images onto the server instead of copying source code and building there?

Solutions and hints are in the companion repository, in `exercises/chapter-23.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

What's Next

The application is now deployed — the container is running, Gunicorn is accepting connections, and the health check passes. But Gunicorn is running on port 5000, not port 80 or 443. There is no TLS. There is no reverse proxy. The application is technically running but not yet reachable in the correct way.

The next step is configuring the process startup: how Gunicorn is configured, how the container is made to start automatically on boot, and what happens when the container crashes.

→ [Chapter 24](#) — Process Startup in Production

Process Startup in Production

Since [Chapter 3](#), the Notes API has run on Flask’s built-in development server: one process, bound to `localhost:5000`, started by `python run.py`. This is fine for development. In production it falls short in three ways:

1. **Not built for production traffic:** the development server (Werkzeug’s) handles each request in a new thread, but it is not tuned for performance, has not been hardened against hostile clients, and its documentation says plainly not to use it in production. With `FLASK_DEBUG=true` it even offers an interactive debugger that can run arbitrary code.
2. **Bound to localhost:** `app.run(host="localhost")` refuses connections from outside the machine. A container needs `0.0.0.0` to accept traffic forwarded from the host.
3. **No process management:** it is a single process. If that process dies — out of memory, a crash in a C extension — the server is gone, and there is nothing to restart it. Production needs worker processes that are restarted independently.

The solution is a **WSGI server** — a production-grade server designed specifically for running Python web applications. This chapter explains what WSGI is, how Gunicorn (the WSGI server used in the Docker image) works, how to configure it correctly, and how to ensure the entire container starts automatically and restarts after crashes.

In this chapter

By the end of this chapter, you will understand:

- What WSGI is and why Python web applications need a WSGI server
- How Gunicorn manages worker processes and handles concurrent requests
- How to configure Gunicorn for production: worker count, timeouts, logging
- What signals (SIGTERM, SIGKILL) are and how they affect graceful shutdown
- How Docker's `--restart` policy manages container restarts
- How systemd can manage Docker containers at the OS level
- The difference between worker processes and threads in a Python web application

24.1 What WSGI Is

WSGI (Web Server Gateway Interface) is a Python standard defined in PEP 3333. It is a specification — an interface definition — that describes how a web server should communicate with a Python web application.

Before WSGI (first defined in 2003 in PEP 333, and updated for Python 3 by PEP 3333), every Python web framework had its own way of interfacing with web servers — CGI scripts, `mod_python`, framework-specific servers such as Zope's — and a framework chose your server for you. WSGI standardized the interface: any WSGI-compatible web server can run any WSGI-compatible web application. Flask and Django were built on it from the start.

The WSGI interface

A WSGI application is any Python callable (function or class) that:

1. Accepts two arguments: `environ` and `start_response`
2. Returns an iterable of byte strings (the response body)


```
# The simplest possible WSGI application
def simple_app(environ, start_response):
    """
    environ: a dictionary containing the HTTP request information
    (request method, path, headers, body, etc.)
    start_response: a callable to send the HTTP status and headers
    """
    status = "200 OK"
    headers = [("Content-Type", "text/plain")]
    start_response(status, headers)
    return [b"Hello, World!"]
```

Flask’s `app` object is a WSGI application. When you call `create_app()`, the returned `app` implements the WSGI interface. Any WSGI server — Gunicorn, uWSGI, Waitress, `mod_wsgi` — can take that `app` and serve it.

WSGI vs ASGI

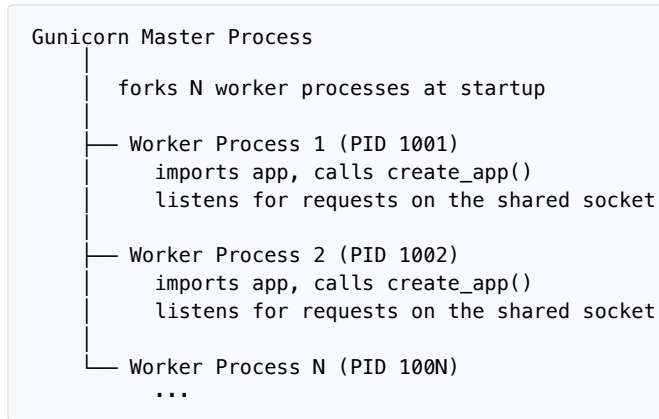
ASGI (Asynchronous Server Gateway Interface) is the modern evolution of WSGI, designed for asynchronous Python frameworks (FastAPI, Starlette, Django with `async views`). ASGI supports WebSockets and other protocols that WSGI cannot handle.

Flask 2.0 and later support `async` route handlers, but Flask itself is a WSGI application. For the Notes API, WSGI and Gunicorn are the correct choice. ASGI and servers like Uvicorn or Hypercorn are relevant if you switch to FastAPI or use Flask’s `async` features extensively.

24.2 How Gunicorn Works

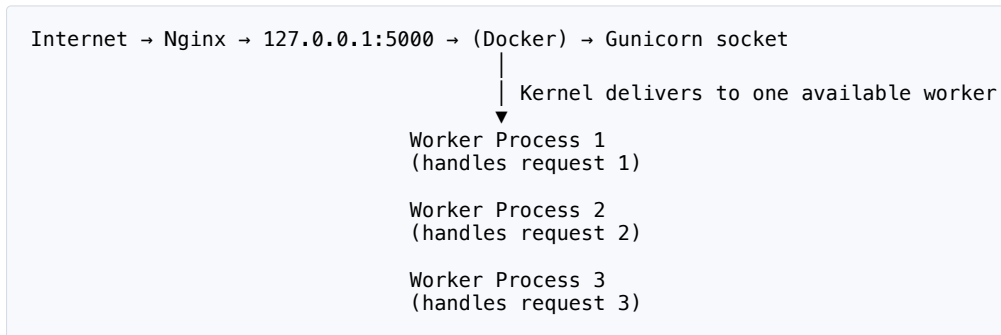
Gunicorn (Green Unicorn) is a production WSGI server for Python. It follows the pre-fork worker model.

The pre-fork worker model



Pre-fork means the master process creates (forks) all worker processes before any requests arrive. Each worker is a complete Python process with its own memory space.

How requests are handled



The kernel's TCP stack accepts connections on the shared socket. When a connection arrives, the kernel delivers it to whichever worker is ready to accept. Multiple workers mean multiple concurrent requests.

Worker lifecycle

```

Master starts
↓
Master forks 2 workers
↓
Worker 1 calls accept() – waits for a connection
Worker 2 calls accept() – waits for a connection
↓
Request 1 arrives → Worker 1 wakes up, handles it, calls accept() again
Request 2 arrives → Worker 2 wakes up, handles it, calls accept() again
↓
Worker 1 is killed (out of memory, a crash in C code, or a timeout) →
Master detects worker death →
Master forks a new Worker 1 →
New Worker 1 calls accept()

```

If a worker dies, the master notices (because the worker process exits) and forks a replacement. This is **fault isolation**: one worker's death does not affect the others. (An ordinary exception in a request handler does not kill a worker at all: Flask's error handlers turn it into a 500 response, and the worker carries on.)

24.3 Configuring Gunicorn

The Dockerfile in [Chapter 18](#) used this CMD:

```

CMD ["gunicorn", \
    "app:create_app()", \
    "--bind", "0.0.0.0:5000", \
    "--workers", "2", \
    "--timeout", "30", \
    "--access-logfile", "-", \
    "--error-logfile", "-", \
    "--log-level", "info"]

```

Let us examine each setting in depth.

app:create_app()

This tells Gunicorn where to find the WSGI application:

- `app` — the Python module (the `app/` package)

- `create_app()` — call this callable to get the WSGI app

Gunicorn imports the `app` module and calls `create_app()`. The returned Flask application is the WSGI application it serves.

Alternatively: `"run:app"` — import `run.py` and use the `app` variable there. The `create_app()` form is preferred because it uses the factory pattern.

--bind 0.0.0.0:5000

Bind to all network interfaces on port 5000. `0.0.0.0` means “accept connections from any network interface.” Inside a Docker container, this is required: the container has its own network interface, and `localhost:5000` would be unreachable from outside the container. Which “outside” can reach it is decided on the host, by how the port is published: on the server, always `-p 127.0.0.1:5000:5000`, so only Nginx can ([Chapter 21](#)).

--workers

The number of worker processes to spawn.

How many workers? The classic formula:

```
workers = (2 × CPU_cores) + 1
```

For a server with 1 CPU core: 3 workers. For a server with 2 CPU cores: 5 workers.

The reasoning: while one worker is waiting for a database response (I/O), another worker can handle a different request. The $2 \times \text{CPU_cores}$ formula accounts for this I/O concurrency.

However, the formula is a starting point, not a law. For the Notes API on a 1-vCPU server with 1 GB RAM, 3 workers would consume ~150 MB (50 MB per worker) — affordable — but this book keeps **2 workers**, the number used since [Chapter 18](#): the application is tiny and the database does the heavy lifting, and a second vCPU is the moment to revisit the number. On a 512 MB server, 2 workers are the safe maximum.

The GIL and worker processes:

Python has a **Global Interpreter Lock (GIL)** that prevents multiple threads in the same process from executing Python bytecode simultaneously. This makes Python’s threading less effective for CPU-bound parallelism.

Workers are **processes**, not threads. Each process has its own GIL. With 2 worker processes, 2 Python functions can run simultaneously (on 2 CPU cores).

This is why Gunicorn’s default is multiple processes rather than threads — processes sidestep the GIL for CPU-bound work. The Notes API, though, is mostly I/O-bound: its requests spend their time waiting for the database, and Python releases the GIL while it waits. For

such applications, threads within each worker (`--threads 4`, which switches to the `gthread` worker class) add concurrency cheaply. (On a 1-vCPU server, nothing runs truly in parallel either way; the gain comes from overlapping the waiting.)

--timeout

Kill a worker if it takes longer than this many seconds to handle a single request. Default: 30 seconds.

If a worker is handling a request and the request takes longer than 30 seconds (perhaps due to a slow database query or an infinite loop), Gunicorn kills the worker (recent versions send `SIGABRT` first, then `SIGKILL`) and forks a new one.

This prevents “stuck” workers from accumulating and exhausting the worker pool.

```
Timeline:
T+0s: Request arrives, Worker 1 starts handling it
T+30s: Gunicorn sends SIGKILL to Worker 1 (timeout)
T+30s: Client receives a 502 Bad Gateway (Nginx sees the connection close)
T+30s: Gunicorn forks a new Worker 1
T+30s: New Worker 1 is ready for the next request
```

For operations that legitimately take longer than 30 seconds (file uploads, complex computations), increase the timeout or, better, use background tasks.

--access-logfile - and --error-logfile -

Write access logs to `stdout` and Gunicorn’s own messages to `stderr` (`-` means “the standard stream”: `stdout` for the access log, `stderr` for the error log). In Docker, both are captured by the container runtime. `docker logs` and log aggregation systems see this output.

Without this: logs go to a file inside the container, which you cannot easily access.

--log-level info

Gunicorn’s own log verbosity. `info` logs:

- Worker starts and stops
- Gunicorn configuration on startup
- Unhandled exceptions in workers

Use `debug` during troubleshooting, `warning` for a quieter production environment.

Additional useful settings

```

--workers 2           # Number of worker processes
--worker-class sync   # Worker type: sync (default), gthread, gevent, eventlet
--threads 1           # Threads per worker (for thread-based concurrency)
--worker-connections 1000 # Max simultaneous clients per worker (async workers)
--max-requests 1000    # Restart workers after handling this many requests
                        # (prevents memory leaks from accumulating)
--max-requests-jitter 50 # Add randomness to max-requests to prevent all
                        # workers restarting simultaneously
--graceful-timeout 30  # After receiving SIGTERM, wait this long for in-flight
                        # requests to complete before killing workers
--preload              # Load the app in the master before forking workers
                        # (saves memory via copy-on-write; use with caution
                        # if the app has state that is not fork-safe)

```

The production Gunicorn configuration file

Instead of long command-line arguments, Gunicorn can read from a configuration file:

Listing 24.1 gunicorn.conf.py

```

"""
Gunicorn configuration for production.

Gunicorn reads this file when started with:
    gunicorn --config gunicorn.conf.py "app:create_app()"
(It also loads ./gunicorn.conf.py automatically; --config makes it explicit.)
"""
import os

# -----
# Server socket
# -----

# Bind to all interfaces inside the container, on port 5000
bind = "0.0.0.0:5000"

# -----
# Worker processes
# -----

# Number of worker processes: 2 for the 1-vCPU, 1 GB server (see above).
# Settable per server without rebuilding the image.

```

```

# (Avoid multiprocessing.cpu_count() here: inside a container it reports
# the HOST's CPUs and ignores any limit set with docker run --cpus.)
workers = int(os.getenv("GUNICORN_WORKERS", "2"))

# Worker class – sync for standard Flask apps
worker_class = "sync"

# Threads per worker (optional; 1 = no threading)
# Use threads > 1 if your application does I/O-bound work and you want
# thread-based concurrency within each worker process.
threads = 1

# _____
# Worker lifecycle
# _____

# Kill workers that take longer than this to handle a request (seconds)
timeout = 30

# After receiving SIGTERM, wait this long for in-flight requests (seconds)
graceful_timeout = 30

# Restart workers after this many requests (prevents memory leak accumulation)
max_requests = 1000

# Add jitter to prevent all workers restarting simultaneously
max_requests_jitter = 100

# _____
# Logging
# _____

# Access log to stdout, Gunicorn's own messages to stderr (Docker captures both)
accesslog = "-"
errorlog = "-"
loglevel = "info"

# Log format: include timestamp, method, path, status, response time
access_log_format = '%(t)s "%(r)s" %(s)s %(b)s %(D)sµs'

# _____
# Process naming
# _____

```

```
# Prefix for process names (visible in ps/htop)
proc_name = "notes-api"
```

Update the end of the Dockerfile (Listing 18.1) to use this file. The COPY goes next to the other application files, and the new CMD replaces the old one:

```
# Copy the gunicorn config into the image (with the application code)
COPY --chown=notes:notes gunicorn.conf.py .

# ... USER, EXPOSE, ENV and HEALTHCHECK exactly as before ...

CMD ["gunicorn", "--config", "gunicorn.conf.py", "app:create_app()"]
```

24.4 Signals and Graceful Shutdown

Signals are a Unix mechanism for inter-process communication. A signal is an integer sent to a process by the kernel, by another process, or by a user pressing Ctrl+C.

When a signal arrives, the process's default signal handler runs (or a custom handler, if the process registered one). Different signals have different default behaviors.

Signals relevant to Gunicorn

Signal	Value	Gunicorn behavior
SIGTERM	15	Graceful shutdown: stop accepting new requests, wait for in-flight requests to complete
SIGKILL	9	Immediate termination — cannot be caught or ignored
SIGINT, SIGQUIT	2, 3	Quick shutdown: exit immediately, without waiting for in-flight requests (Ctrl+C sends SIGINT)
SIGHUP	1	Reload configuration and restart workers gracefully
SIGUSR2	12	Upgrade Gunicorn binary in-place (for zero-downtime updates)

Why graceful shutdown matters

When Docker stops a container, it sends SIGTERM to the main process (Gunicorn's master). With proper handling:

```

SIGTERM arrives at Gunicorn master
↓
Master tells workers: "stop accepting new requests"
↓
Workers finish their current in-flight requests
↓
Workers exit cleanly
↓
Master exits
↓
Container exits

Timeline: 0–30 seconds (graceful_timeout)

```

Without graceful handling:

```

Container receives SIGTERM → Gunicorn exits immediately
↓
Worker processes exit immediately
↓
In-flight HTTP requests are dropped (client sees connection reset)
↓
Work a request had half done stays half done (the database rolls back an
uncommitted transaction, but an email already sent stays sent)

```

If the processes are still running when the stop timeout expires, Docker sends **SIGKILL** — a signal that cannot be caught, and forcibly terminates the process regardless of what it is doing. Docker’s default timeout is only **10 seconds**, which cuts Gunicorn’s 30-second graceful window short unless you change it:

The shutdown sequence in Docker

```

docker stop notes-api
# Docker sends SIGTERM to the container's main process (PID 1)
# If the process has not exited after --time (default 10 seconds),
# Docker sends SIGKILL

# Increase the timeout for graceful shutdown:
docker stop --time 35 notes-api

```

Align the Docker stop timeout with Gunicorn’s `graceful_timeout`:

- Gunicorn `graceful_timeout` = 30 seconds
- Docker stop timeout = 35 seconds (5 extra seconds for overhead)

Rather than remembering `--time 35` on every `docker stop`, set it once when the container is created: `docker run --stop-timeout 35 ...`. Every `docker run` in the deploy scripts of Chapter 23 does this.

24.5 PID 1 and Signal Handling in Containers

In a Docker container, the main process has **PID 1** — the init process. PID 1 is special in two ways. Signals sent to the container (`docker stop` sends **SIGTERM**) go to it. And the kernel protects it: a signal that PID 1 has not installed a handler for is simply **ignored**, instead of triggering the default action (which, for **SIGTERM**, is to terminate).

The PID 1 problem

When Gunicorn starts with the CMD:

```
CMD ["gunicorn", "--config", "gunicorn.conf.py", "app:create_app()"]
```

Gunicorn runs as PID 1. It installs a SIGTERM handler, so when Docker sends SIGTERM, Gunicorn receives it and shuts down gracefully.

Now suppose the container must do something before starting Gunicorn — say, wait for the database — and the CMD becomes a small shell script:

```
CMD ["/bin/sh", "-c", "python wait_for_db.py && gunicorn --config gunicorn.conf.py  
↪ 'app:create_app()'"]
```

The shell runs as PID 1 and starts Gunicorn as its child. When Docker sends SIGTERM:

- SIGTERM goes to the shell (PID 1), which has no handler for it — so the kernel ignores it
- Gunicorn never hears about it and keeps serving
- After the stop timeout, Docker sends SIGKILL to everything
- No graceful shutdown — and every `docker stop` takes the full timeout

(With a single command, such as `sh -c "gunicorn ..."`, many shells quietly replace themselves with that command, and Gunicorn ends up as PID 1 after all. That is why this bug often appears only when a second command is added.)

Three fixes:

```
# 1. Exec form (JSON array) with no shell at all — the default choice:
CMD ["gunicorn", "--config", "gunicorn.conf.py", "app:create_app()"]

# 2. If a shell is needed, make it replace itself with Gunicorn using exec:
CMD ["/bin/sh", "-c", "python wait_for_db.py && exec gunicorn --config  
↪ gunicorn.conf.py 'app:create_app()'"]

# 3. Or run a tiny init process as PID 1 that forwards signals and reaps  
# zombie processes: docker run --init (it runs tini as PID 1)
```

Avoid the **shell form** of CMD (CMD `gunicorn --config gunicorn.conf.py ...`, without brackets): Docker wraps it in `/bin/sh -c`, recreating the problem.

24.6 Docker Restart Policies

The container needs to restart automatically if it crashes. Docker provides **restart policies**:

Policy	Behavior
no	Never restart (default)
on-failure	Restart only if the container exited with a non-zero exit code
unless-stopped	Restart whenever the container stops (and when the Docker daemon starts), unless you stopped it with <code>docker stop</code>
always	The same — except that a container you stopped by hand is started again the next time the Docker daemon starts

```
# Run with restart policy (this is how deploy.sh starts the container):
docker run -d \
  --name notes-api \
  --restart unless-stopped \
  --stop-timeout 35 \
  -p 127.0.0.1:5000:5000 \
  --env-file /opt/notes-api/config/notes-api.env \
  ghcr.io/your-username/notes-api:sha-a1b2c3d
```

With `unless-stopped`:

- If the container crashes: Docker restarts it automatically
- If the server reboots: Docker starts the container when Docker daemon starts
- If you run `docker stop notes-api`: Docker does NOT restart it (until next explicit `docker run`)

This is the policy for a production container that should always be running — and it is **this book’s supervision design**: Docker’s restart policy restarts the container, and `systemd`, which starts Docker itself at boot (`docker.service` is enabled — [Chapter 21](#)), brings the whole stack back after a reboot.

Viewing restart history

```
# See how many times a container has been restarted
docker inspect notes-api --format \
  'Status: {{.State.Status}} RestartCount: {{.RestartCount}} ExitCode:
  ↳ {{.State.ExitCode}} StartedAt: {{.State.StartedAt}}'
```

If RestartCount is climbing, the container is crash-looping. Investigate the logs:

```
docker logs --tail=100 notes-api
```

24.7 systemd Integration (an Alternative)

Docker's `--restart unless-stopped` is sufficient for the Notes API, and it is what this book uses. Some teams prefer to let **systemd** — the Linux service manager — own the container instead. This section shows how, so you can recognize and choose it; it is an **alternative** to the restart policy, never an addition to it.

What systemd offers over Docker restart policies:

- `systemctl status notes-api`: the same management interface as every other service on the server
- `journalctl -u notes-api`: the container's output in the system journal
- Dependencies and ordering between services (e.g., “start after the network is up”)

Why never both: two supervisors for one container fight. `deploy.sh` removes the container to replace it, and systemd immediately starts one of its own; after a reboot, Docker restarts one container and systemd starts another with the same name. Pick one. If you choose systemd, the deploy script changes too: instead of running `docker run`, it writes the new tag into an environment file and runs `systemctl restart notes-api`.

Creating a systemd unit file

The image tag lives in its own small file, so a deployment (or a rollback) edits one line, and a reboot starts exactly the version that was last deployed — never whatever `latest` happens to be:

```
# /opt/notes-api/config/image.env — written by the deploy script
IMAGE=ghcr.io/your-username/notes-api:sha-a1b2c3d
```

```

# Create the systemd service file
cat > /etc/systemd/system/notes-api.service << 'EOF'
[Unit]
Description=Notes API (Docker container)
Documentation=https://github.com/your-username/notes-api
# Start after Docker is running
After=docker.service
Requires=docker.service

[Service]
# The deploy user runs the container (it is in the docker group)
User=deploy
Group=deploy

# Provides $IMAGE: the exact image to run
EnvironmentFile=/opt/notes-api/config/image.env

# Remove any leftover container with the same name before starting
ExecStartPre=/usr/bin/docker rm -f notes-api

# Start the container in the foreground, so systemd can watch it.
# No --restart flag: systemd is the supervisor here.
ExecStart=/usr/bin/docker run \
    --name notes-api \
    --stop-timeout 35 \
    -p 127.0.0.1:5000:5000 \
    --env-file /opt/notes-api/config/notes-api.env \
    ${IMAGE}

# Stop the container gracefully (SIGTERM, then wait up to 35 seconds, then SIGKILL)
ExecStop=/usr/bin/docker stop notes-api

# Restart behavior
Restart=always
RestartSec=5

# Logging
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
EOF

```

(The container is deliberately *not* started with `--rm`: after a crash, `docker inspect` and `docker logs` still have something to show you. `ExecStartPre` removes it before the next start.)

Enable and start the service:

```
# Reload systemd's configuration after creating the unit file
systemctl daemon-reload

# Enable: start automatically on boot
systemctl enable notes-api

# Start now
systemctl start notes-api

# Check status
systemctl status notes-api
```

Expected status output:

```
● notes-api.service - Notes API (Docker container)
   Loaded: loaded (/etc/systemd/system/notes-api.service; enabled; ...)
   Active: active (running) since Mon 2024-01-15 14:00:00 UTC; 2min ago
     Main PID: 12345 (docker)
        Tasks: 7 (limit: 1141)
       Memory: 25.6M
          CPU: 2.345s
      CGroup: /system.slice/notes-api.service
              └─12345 /usr/bin/docker run --name notes-api ...
```

Notice what systemd is actually watching: the `docker run` *client* process (PID 12345). The container's processes — Gunicorn and its workers — are started by Docker's own daemon, under `containerd`, in a separate cgroup. So `Memory: 25.6M` is the Docker client's memory, not the application's; `docker stats notes-api` shows the real figure. systemd learns about a crash because the client exits when the container does.

View logs:

```
journalctl -u notes-api -f          # Follow logs in real time
journalctl -u notes-api --since "1 hour ago" # Logs from the past hour
```

24.8 The Complete Startup Sequence

When the server boots:

1. Linux kernel initializes hardware
2. systemd starts as PID 1
3. systemd starts Docker (docker.service, enabled in Chapter 21)
4. Docker daemon starts containers that have a restart policy: notes-api (`--restart unless-stopped`)
5. Container's PID 1 – the Gunicorn master – starts:
 - reads gunicorn.conf.py, binds 0.0.0.0:5000 inside the container
6. Gunicorn forks 2 worker processes
7. Each worker imports the app package and calls `create_app()`:
 - logging configured
 - startup checks (Python version, SECRET_KEY, database) ✓
 - "All startup checks passed."
 - error handlers, middleware and blueprints registered
 - database schema initialized (CREATE TABLE IF NOT EXISTS)
8. Each worker calls `accept()` on the shared socket
9. Server is ready to handle requests

Two details are easy to get wrong. The startup checks run inside `create_app()`, so they run in every worker (twice, with two workers) – and there is no port check here: that one belongs to `run.py`, which Gunicorn does not use, because Gunicorn binds the port itself. And `create_app()` runs *after* the fork, in each worker; only with `--preload` would the master call it once, before forking.

With the systemd alternative from section 24.7, step 4 becomes: “systemd starts notes-api.service (After=docker.service), which runs `docker run`”.

24.9 Monitoring the Process

After startup, monitor the application's resource usage:

```
```bash
View running containers and their resource usage
docker stats notes-api

View the processes inside the container (run on the host: the slim
image has no "ps" command, so "docker exec notes-api ps" would fail)
docker top notes-api

View the container's PID 1 (Gunicorn master)
docker exec notes-api cat /proc/1/status | head -5
```



```
Interactive shell inside the container (for debugging)
docker exec -it notes-api bash
```

`docker top` lists Gunicorn's processes with their PIDs *on the host*:

```
UID PID PPID C STIME TTY TIME CMD
1001 23101 23080 0 14:00 ? 00:00:00 /usr/local/bin/python
↳ /usr/local/bin/gunicorn --config gunicorn.conf.py app:create_app()
1001 23130 23101 0 14:00 ? 00:00:01 /usr/local/bin/python
↳ /usr/local/bin/gunicorn --config gunicorn.conf.py app:create_app()
1001 23131 23101 0 14:00 ? 00:00:01 /usr/local/bin/python
↳ /usr/local/bin/gunicorn --config gunicorn.conf.py app:create_app()
```

The first line is the Gunicorn master (PID 1 inside the container); the other two, its children, are the workers. All run as UID 1001, the notes user. Each worker is a full Python process.

### Key Takeaways

- **WSGI** is the Python standard interface between web servers and web applications. Flask's app object implements WSGI. Gunicorn is the WSGI server that serves it in production.
- **Gunicorn uses the pre-fork worker model**: the master process forks N worker processes at startup. Each worker is a separate Python process with its own GIL, so on a multi-core server workers run in parallel.
- **Worker count formula**:  $(2 \times \text{CPU\_cores}) + 1$  is the classic starting point. The Notes API uses 2 workers on its 1-vCPU server; adjust based on measured memory and load.
- **Graceful shutdown** (SIGTERM) tells Gunicorn to finish in-flight requests before exiting. Docker's default stop timeout is 10 s: set `--stop-timeout 35` on `docker run` to cover Gunicorn's 30-second `graceful_timeout`.
- **Make Gunicorn PID 1**: use the exec (JSON array) form of CMD, or exec in a shell wrapper, or `docker run --init`. A shell as PID 1 ignores SIGTERM, so shutdown degrades to SIGKILL after the timeout.
- **--restart unless-stopped** ensures Docker restarts the container on crash and on server reboot, until explicitly stopped.
- **systemd** starts Docker at boot. It can also own the container itself (a unit with a pinned image tag) — as an *alternative* to the restart policy, never together with it.
- The **complete startup sequence** on boot is: kernel → systemd → Docker → container → Gunicorn master → workers → `create_app()` and its startup checks → ready.

## Exercises

1. **Predict.** Start a request that takes 20 seconds (add `time.sleep(20)` to a test route), then run `docker stop notes-api`. Does the request finish? What if it slept 40 seconds?
2. **Break and fix.** Change the Dockerfile's CMD to the shell form (CMD `gunicorn ...`). Stop the container and time how long it takes. Explain the result, then restore the exec form.
3. **Extend.** Measure memory per Gunicorn worker (`docker stats` for the container, `docker top notes-api -o pid,rss,args` for each process), then compute how many workers a 2 GB server can run safely.
4. **Explain.** What is the difference between Docker's restart policy and a systemd unit for the container, and why must you not use both?

Solutions and hints are in the companion repository, in `exercises/chapter-24.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

Gunicorn is running inside the container, starts at boot, and restarts after crashes. Keeping it that way is an ongoing job: knowing what is running, reading its logs, investigating crash loops, and running the work that is not a response to a request — scheduled jobs. That is process management, and the next chapter covers the tools: `systemd` and `systemctl`, `journalctl`, and scheduled and background work.

→ [Chapter 25 — Process Management](#)

# Process Management

---

Process management is the discipline of knowing what processes are running, ensuring they stay running, controlling them deliberately, and understanding what they are doing at any moment.

In development, process management is trivial: you run `python run.py`, the process runs in your terminal, and you press Ctrl+C to stop it. If it crashes, you see the error and restart it manually.

In production, none of this is possible. No one is sitting at a terminal. There is no Ctrl+C to press. When the process crashes at 3:00 AM — and it will crash eventually — something needs to:

1. Detect that it crashed
2. Restart it automatically
3. Record what happened so you can investigate later

This chapter examines process management in the production context: `systemd` as the server's service manager, Docker as the container manager, the relationship between the two, and the tools for inspecting and controlling running processes.

**In this chapter**

By the end of this chapter, you will understand:

- What a process supervisor is and why it is required in production
- How systemd manages services: unit files, targets, states, and dependencies
- How to use `systemctl` to start, stop, restart, and check the status of services
- How to use `journalctl` to view and search service logs
- How Docker’s restart policies interact with systemd
- How to manage multiple processes on one server (Nginx, Gunicorn, PostgreSQL)
- How to handle process crashes gracefully
- How to run **scheduled and event-triggered work** (cron, systemd timers, queues) safely — with idempotency, overlap protection, and alerting on missed runs

## 25.1 What a Process Supervisor Is

A **process supervisor** is a program that:

1. Starts a target process when the system boots
2. Monitors the target process while it runs
3. Restarts the target process if it exits unexpectedly
4. Logs the target process’s output

Without a supervisor, a process must be started manually. If it crashes, it stays crashed until someone notices and restarts it. For a web API, this means downtime.

With a supervisor:

- The application starts automatically when the server boots
- If the application crashes, the supervisor restarts it within seconds
- No human intervention required for transient failures

Linux has several process supervisors:

- **systemd**: the init system and service manager on virtually all modern Linux distributions. It starts and tracks every service on the system.
- **supervisord**: a simpler, Python-based supervisor. Easier to configure than systemd, but less deeply integrated with the OS.
- **Docker’s restart policy**: Docker’s built-in supervisor for containers. Sufficient for many use cases.

- **Kubernetes:** a container orchestration platform that includes process management for containerized applications at scale.

For the Notes API: Docker’s restart policy (`--restart unless-stopped`) combined with systemd managing Docker itself provides adequate supervision. That is the setup the deploy scripts of [Chapter 23](#) create.

To teach systemd itself, this chapter uses the **alternative** from [Chapter 24](#), section 24.7 — the `notes-api.service` unit, in which systemd owns the container — as its running example, because one unit file shows every systemd concept in one place. The commands that act on `notes-api.service` apply if you chose that alternative. With the restart-policy setup, the container is not a systemd service; wherever the two differ, the Docker equivalent is shown alongside.

## 25.2 systemd: The Linux Process Manager

**systemd** is the init system on Ubuntu 22.04 (and virtually all modern Linux distributions). It is the first process started by the kernel (PID 1 on the host system) and is responsible for starting and managing all other processes.

### Units and unit files

In systemd terminology, everything it manages is a **unit**. Units are described by **unit files** — configuration files that tell systemd how to start, stop, and manage something.

Types of units:

- **.service:** a long-running service (the Docker container, Nginx, PostgreSQL)
- **.socket:** a network socket (for socket activation)
- **.timer:** a scheduled task (like cron)
- **.target:** a group of units (e.g., `multi-user.target` = “system is ready for multi-user login”)

Unit files live in:

- `/lib/systemd/system/` (the same directory as `/usr/lib/systemd/system/` on Ubuntu): units installed by packages (do not edit these)
- `/etc/systemd/system/`: system-wide units you create (edit these)
- `~/.config/systemd/user/`: user-specific units

## The `notes-api.service` unit file

Chapter 24 (section 24.7) showed this unit file. Let us examine it in detail:

```
/etc/systemd/system/notes-api.service
[Unit]
Description=Notes API (Docker container)
Documentation=https://github.com/your-username/notes-api
After=docker.service
Requires=docker.service
```

**[Unit] section** — general information about the unit:

- **Description:** human-readable description (appears in `systemctl status`)
- **After=docker.service:** start this unit AFTER the Docker service starts
- **Requires=docker.service:** if Docker stops, stop this unit too

```
[Service]
User=deploy
Group=deploy
EnvironmentFile=/opt/notes-api/config/image.env
ExecStartPre=-/usr/bin/docker rm -f notes-api
ExecStart=/usr/bin/docker run \
 --name notes-api \
 --stop-timeout 35 \
 -p 127.0.0.1:5000:5000 \
 --env-file /opt/notes-api/config/notes-api.env \
 ${IMAGE}
ExecStop=/usr/bin/docker stop notes-api
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal
```

**[Service] section** — how to start, stop, and manage the service:

- **EnvironmentFile:** read variables from a file — here `IMAGE`, the exact image tag the last deployment wrote.
- **ExecStartPre:** commands to run before `ExecStart`. A `-` right after the `=` means “ignore errors” — we use it because the container might not exist yet.
- **ExecStart:** the main command. `systemd` runs this and monitors it.
- **ExecStop:** how to gracefully stop the service. `systemd` waits up to `TimeoutStopSec` (de-

fault 90 seconds) for it — comfortably more than the container’s 35-second stop timeout.

- `Restart=always`: restart the service if it exits for any reason.
- `RestartSec=5`: wait 5 seconds between restart attempts.
- `StandardOutput=journal`: send stdout to systemd’s journal.

#### [Install]

`WantedBy=multi-user.target`

**[Install]** section — how the unit fits into the system:

- `WantedBy=multi-user.target`: start this unit when the system reaches multi-user mode (the normal running state, with network and all services).

## 25.3 Managing Services with systemctl

`systemctl` is the command for interacting with `systemd`.

### Starting and stopping

```
Start the service
systemctl start notes-api

Stop the service (sends SIGTERM, waits for graceful shutdown)
systemctl stop notes-api

Restart (stop + start)
systemctl restart notes-api

Reload configuration without stopping – only for units that define
ExecReload= (nginx does; notes-api.service does not, so this fails with
"Job type reload is not applicable")
systemctl reload nginx

Reload if the unit supports it, otherwise restart
systemctl reload-or-restart notes-api

Restart only if the service is already running
systemctl try-restart notes-api
```

## Enabling and disabling

```
Enable: configure to start automatically on boot
systemctl enable notes-api
```

```
Disable: do not start automatically on boot
systemctl disable notes-api
```

```
Enable and start immediately
systemctl enable --now notes-api
```

```
Disable and stop immediately
systemctl disable --now notes-api
```

## Checking status

```
systemctl status notes-api
```

### Output:

```
● notes-api.service – Notes API (Docker container)
 Loaded: loaded (/etc/systemd/system/notes-api.service; enabled; ...)
 Active: active (running) since Mon 2024-01-15 14:00:00 UTC; 2 hours ago
 Process: 12340 ExecStartPre=/usr/bin/docker stop notes-api (code=exited, ...)
 Process: 12341 ExecStartPre=/usr/bin/docker rm notes-api (code=exited, ...)
 Main PID: 12345 (docker)
 Tasks: 7 (limit: 1141)
 Memory: 25.6M
 CPU: 45.2s
 CGroup: /system.slice/notes-api.service
 └─12345 /usr/bin/docker run --name notes-api ...

Jan 15 14:00:00 server docker[12345]: [2024-01-15 14:00:00 +0000] [1] [INFO] Starting gunicorn 22.0.0
Jan 15 14:00:00 server docker[12345]: [2024-01-15 14:00:00 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
Jan 15 14:00:00 server docker[12345]: [2024-01-15 14:00:00 +0000] [7] [INFO] Booting worker with pid: 7
Jan 15 14:00:00 server docker[12345]: [2024-01-15 14:00:00 +0000] [8] [INFO] Booting worker with pid: 8
```

(With the restart-policy setup, `docker ps --filter name=notes-api` and `docker inspect notes-api` answer the same questions: is it running, since when, how often has it restarted.)

Fields in the status output:

- **Loaded:** the unit file is found and valid; `enabled` means it starts on boot
- **Active:** `active (running)` means the service is currently running
- **Main PID:** the PID of the main process (the `docker run` command)
- **Memory:** RAM used by the unit's processes — here only the `docker run` client; the container's own processes live in Docker's cgroup (section 25.7)



- CGroup: the control group (cgroup) that this service belongs to

## Viewing all services

```
List all running services
systemctl list-units --type=service --state=running

List all failed services
systemctl list-units --type=service --state=failed

List all enabled services (will start on boot)
systemctl list-unit-files --type=service --state=enabled
```

## 25.4 Logs with journalctl

systemd captures stdout and stderr from every service it manages and stores it in the **journal** — a binary log store with structured metadata.

```
View all logs for notes-api (paginated, oldest first; -r reverses the
order, -e jumps to the end)
journalctl -u notes-api

Follow in real time (like tail -f)
journalctl -u notes-api -f

Show only recent logs (last 100 lines)
journalctl -u notes-api -n 100

Show logs since a specific time
journalctl -u notes-api --since "2024-01-15 14:00:00"
journalctl -u notes-api --since "1 hour ago"
journalctl -u notes-api --since "today"

Show logs for a time range
journalctl -u notes-api --since "2024-01-15 12:00:00" --until "2024-01-15 14:00:00"

Show only error messages
journalctl -u notes-api -p err

Show in JSON format (useful for piping to jq)
journalctl -u notes-api -o json | jq '.'
```

```
Show only the messages themselves, without journal prefixes
journalctl -u notes-api -o cat

Show logs from the last boot
journalctl -u notes-api -b

Show logs from the previous boot (useful after a crash)
journalctl -u notes-api -b -1
```

With the restart-policy setup, the container's output is not in the journal: read it with `docker logs`, which takes similar options — `docker logs -f notes-api`, `docker logs --since 1h notes-api`, `docker logs --tail 100 notes-api`.

## Searching logs

The application writes JSON log lines (Chapter 16), mixed with Gunicorn's plain-text lines. `jq -R 'fromjson?'` parses each line that is JSON and skips the rest, so you can select on real fields instead of grepping text:

```
Start from the raw log lines – one of:
journalctl -u notes-api -o cat --since "24 hours ago" (systemd unit)
docker logs --since 24h notes-api 2>&1 (restart policy)

Find all server errors (responses with a 5xx status) in the last 24 hours
docker logs --since 24h notes-api 2>&1 | \
 jq -cR 'fromjson? | select(.message | startswith("← 5"))'

Find ERROR-level records (the handling boundary logs every traceback)
docker logs --since 24h notes-api 2>&1 | \
 jq -cR 'fromjson? | select(.level == "ERROR")'

Find database connection failures at startup
docker logs notes-api 2>&1 | grep "Cannot connect to the database"

Count responses by status code in the last hour
docker logs --since 1h notes-api 2>&1 | \
 jq -rR 'fromjson? | .message | select(startswith("← ")) | split(" ")[1]' | \
 sort | uniq -c | sort -rn
412 200
37 201
9 404
2 400
```

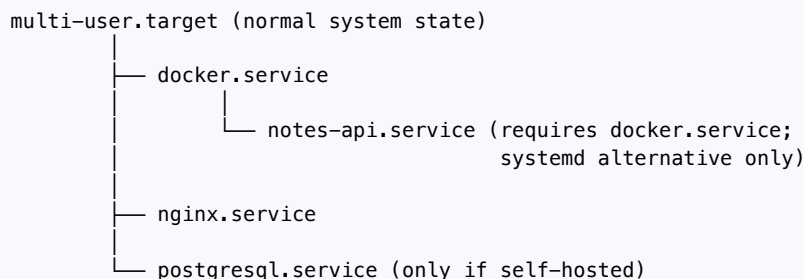
(An ERROR-level record and a 500 response are related but not the same: the handling boundary logs an ERROR *and* returns a 500, while the request-logging middleware logs each 5xx response line itself.)

## 25.5 Managing Multiple Processes on One Server

A production server typically runs several services:

- **Nginx**: receives internet traffic on port 80/443
- **Docker container (Gunicorn)**: the application on port 5000
- **PostgreSQL** (only if self-hosted; this book uses a managed database): the database on port 5432

Each is a separate systemd service with its own unit file. They have dependencies:



### Nginx as a service

Nginx is installed as a package and comes with a pre-configured systemd service:

```

Nginx service management
systemctl status nginx
systemctl start nginx
systemctl stop nginx
systemctl reload nginx # Reload config without dropping connections
systemctl restart nginx # Full restart (drops connections briefly)

```

```
Nginx writes its logs to its own files, not to the journal
tail -f /var/log/nginx/error.log
tail -f /var/log/nginx/access.log

The journal has Nginx's start/stop messages and early startup errors
(e.g. a config error that kept it from starting)
journalctl -u nginx -n 50
```

## Service dependency ordering

systemd respects the dependency declarations in unit files. If `notes-api.service` declares `After=docker.service` `Requires=docker.service`:

- systemd starts Docker before notes-api
- If Docker fails to start, systemd marks notes-api as failed
- If Docker stops while notes-api is running, systemd stops notes-api

This ordering is critical for correct boot behavior.

## 25.6 Process Crash Investigation

When a service repeatedly crashes, `systemctl status` shows the restart count and recent logs:

```
systemctl status notes-api
● notes-api.service - Notes API (Docker container)
Active: activating (auto-restart) (Result: exit-code) since ... 3s ago
...
... notes-api.service: Main process exited, code=exited, status=1/FAILURE
... notes-api.service: Failed with result 'exit-code'.
... notes-api.service: Scheduled restart job, restart counter is at 3.
```

restart counter is at 3 means it has crashed and restarted 3 times. With the restart-policy setup, the equivalent is the container's `RestartCount` and a status of restarting in docker ps:

```
docker inspect notes-api --format '{{.State.Status}} restarts={{.RestartCount}}'
restarting restarts=3
```

## Crash loop detection

If a service crashes immediately after starting, systemd enters a **crash loop**. After too many restarts in a short time, systemd gives up:

```
notes-api.service: Start request repeated too quickly.
notes-api.service: Failed with result 'start-limit-hit'.
```

To investigate:

```
View recent logs (the crash might have log output)
journalctl -u notes-api -n 50 # systemd unit
docker logs --tail=50 notes-api # either setup

View the container's exit code
docker inspect notes-api --format '{{.State.ExitCode}}'
```

(Docker has no equivalent of systemd’s “start limit”: a crash-looping container with `--restart unless-stopped` keeps being restarted, with a growing delay between attempts. `RestartCount` is how you notice.)

Common causes of startup crashes:

- Missing or invalid environment variable (the startup checks from [Chapter 14](#), run by `create_app()`, log a “Startup failed:” line saying which)
- Database unreachable (the same checks log “Cannot connect to the database”)
- Host port already in use (another container or process holding `127.0.0.1:5000` — docker run then fails with “port is already allocated”)
- Docker image not found (check `docker images | grep notes-api`)

## Resetting the failure counter

After fixing the root cause, reset systemd’s failure counter to allow restarts again:

```
systemctl reset-failed notes-api
systemctl start notes-api
```

## 25.7 Process Resource Monitoring

```
View all processes on the server, interactive
htop

View Docker container resource usage (CPU, memory, network, disk)
docker stats notes-api

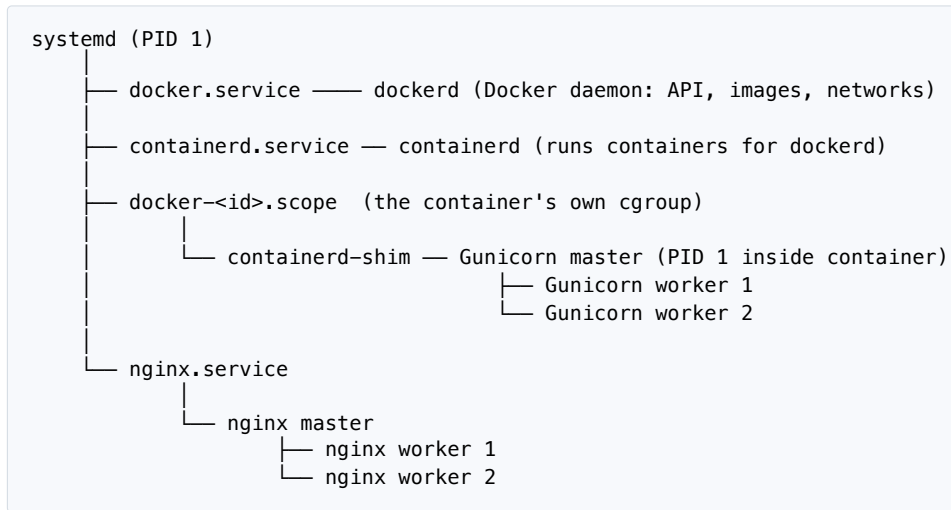
View a specific process's resource usage
Replace PID with the actual PID from docker inspect
cat /proc/PID/status | grep -E "VmRSS|VmPeak|Threads"

The container's cgroup, where the application's memory is really counted
(docker-<full container ID>.scope)
cat /sys/fs/cgroup/system.slice/docker-$(docker inspect -f '{{.Id}}'
↳ notes-api).scope/memory.current
```

Be careful with systemd's own numbers here. With the systemd alternative, `systemctl status notes-api` reports `Memory: 25.6M` — but that is the docker run *client*. The container's processes are not in `notes-api.service`'s cgroup; they are in the Docker scope shown above, and `docker stats` reads the same figures.

## 25.8 The Relationship Between Docker and systemd

Docker itself is managed by systemd. The Docker daemon (`dockerd`) runs as a systemd service (`docker.service`). The relationship:



The container's processes are not children of `dockerd`, and not part of any service you wrote: `dockerd` asks `containerd` to start them, a small `containerd-shim` process becomes their parent, and they get a cgroup of their own. That is why restarting `dockerd` need not stop containers (the `live-restore` setting from [Chapter 21](#)), and why a unit that runs `docker` run supervises only the client (section 25.7).

With the `systemd` alternative, stopping `notes-api.service` runs `ExecStop (docker stop notes-api)`, which sends `SIGTERM` to the container's PID 1 (Gunicorn), which shuts down gracefully.

When the server reboots, `systemd` starts Docker, Docker starts the container (because of `--restart unless-stopped`), and the application is running again without human intervention. (With the `systemd` alternative, `systemd` starts `notes-api.service` after Docker instead.)

## 25.9 Beyond the Always-On Service: Scheduled and Triggered Work

Everything in this chapter so far keeps *one* thing alive: the Notes API, a long-running process that waits for requests. But real systems also need work that is *not* a response to a user request — work that runs on a schedule or in reaction to an event. Delete notes older than a year. Email a weekly digest. Rebuild a search index nightly. Process a payment-confirmation webhook. This is **background work**, and it is the most commonly under-engineered part of a system, because when it silently stops, no one gets an error page — the work just quietly doesn't happen.

## The three kinds of triggered work

- **Time-based (scheduled):** runs on a clock — “every night at 02:00”, “every 15 minutes”. Driven by cron, systemd timers, or a cloud scheduler.
- **Event-based:** runs in reaction to something happening elsewhere — an incoming webhook (an HTTP call from another system; [Chapter 34](#), section 34.10), a message landing on a queue, a file being uploaded.
- **Manual:** a human runs it on demand — a one-off backfill or data migration.

The same job is often reachable more than one way (a nightly cleanup you can also trigger by hand). What they all share is the thing that makes them dangerous: none is driven by an HTTP request, so none appears in your request logs, and none fails in a way a user will report for you.

## cron: the classic scheduler, and its traps

**cron** is the traditional Unix scheduler — a line in a *crontab* maps a time specification to a command:

```
minute hour day-of-month month day-of-week command
0 2 * * * /opt/notes-api/scripts/cleanup.sh
"At 02:00 every day, run the cleanup script."
```

Three traps bite everyone at least once:

- **Timezone.** cron uses the *server*’s local timezone unless told otherwise. Always think and schedule in UTC, or daylight-saving will shift your 02:00 job by an hour twice a year — and “twice a year” is exactly when you have forgotten this. (The server from [Chapter 21](#) is already set to UTC, section 21.10.)
- **A minimal environment.** cron runs jobs with a bare environment: no `.env`, a tiny `$PATH`, and a different working directory. Scripts that run fine in your shell fail under cron because `python` is not found or `DATABASE_URL` is not set. Use absolute paths and load configuration explicitly — the same lesson as the systemd unit file’s fully-specified `ExecStart` earlier in this chapter.
- **Silent failure.** cron’s default reaction to a failing job is to email its output to a local mailbox no one reads — or, on a fresh server with no mail system installed, to discard it (the system log records only “No MTA installed, discarding output”). A broken cron job therefore produces *no signal at all*. You must add your own (see “Visibility” below).



## systemd timers: the modern alternative

On a systemd server — which you already have (§25.2) — **timers** are a more observable replacement for cron. A `.timer` unit triggers a `.service` unit, which means the job becomes a first-class systemd service: its output goes to `journalctl` (debug it exactly like the API — §25.4), it gets proper environment handling, and `systemctl list-timers` shows the last and next run at a glance.

```
/etc/systemd/system/notes-cleanup.timer
[Unit]
Description=Daily cleanup of old notes
[Timer]
OnCalendar=*-*-* 02:00:00 UTC
If the server was off at 02:00, run on the next boot. (systemd allows
comments only on lines of their own, never at the end of a setting.)
Persistent=true
[Install]
WantedBy=timers.target
```

paired with a `notes-cleanup.service` (a `Type=oneshot` unit that runs the script once and exits). `Persistent=true` cures one of cron’s worst traps: a missed run — the server was down at 02:00 — is caught up on the next boot instead of being silently skipped. On managed platforms you will meet the hosted equivalents: cloud schedulers such as AWS EventBridge or GCP Cloud Scheduler that fire a container or function on a cron expression.

## The hard parts every scheduled job must handle

The schedule is the easy bit. These four properties separate a toy cron line from a production job:

- **Idempotency.** A scheduled job *will* run twice eventually — a retry, an overlap, or a manual run on top of the automatic one. It must be safe to run twice. “Delete notes older than a year” is naturally idempotent (the second run finds nothing to do); “email every user a digest” is not, unless you record who has already been emailed. Design every job so a double-run causes no harm. (The same idea governs HTTP methods and webhooks; Chapter 34, section 34.10, returns to it.)
- **Overlap.** What if the 02:00 run is still going when the next one fires, or a slow 15-minute job overruns its window? Two copies mutating the same data at once is a classic corruption source. Prevent it with a **lock** — a database row, a Postgres advisory lock, or systemd’s built-in refusal to start a service instance that is already running.

- **Durability.** What if the job — or the whole server — dies halfway through deleting 10,000 notes? The job must be able to resume or restart without losing or double-processing work. This is why idempotency and small, individually-committed batches matter: each batch either completed or it didn't, and re-running is safe.
- **Visibility.** A missed run is a *silent* failure, and you cannot alert on an error that never fires. So you alert on the **absence of success**: have the job record a heartbeat when it *finishes* — a timestamp, a metric, or a ping to a dead-man's-switch service (Healthchecks.io, Cronitor) — and alert when that heartbeat is overdue. “The nightly cleanup hasn't checked in for 26 hours” is the signal you actually need, and it is the one cron will never give you.

## Event-driven work and queues

Time is not the only trigger. When work should happen *in reaction* to an event, two patterns dominate. A **webhook** is an inbound HTTP call from another system; because senders retry, your handler must be idempotent and fast. A **message queue** (RabbitMQ, Amazon SQS, or Redis-backed task queues like Celery and RQ) decouples work from the request that caused it: instead of doing something slow inside a request — resizing an image, calling a sluggish third party — you drop a message on a queue, and a separate **worker** process, supervised exactly like the API (a container with a restart policy, or a systemd service), consumes it. Queues buy you retries, buffering under load spikes, and the ability to scale workers independently of web servers. This is the doorway to the asynchronous, decoupled architectures that large systems are built from; the idea to carry forward is simply that *not all work belongs inside the request*.

## A checklist for adding a scheduled job

Before you ship any cron line or timer, answer:

1. **What triggers it, and in what timezone?** (UTC.)
2. **Is it idempotent?** If not, make it so *before* scheduling it.
3. **What happens if two runs overlap?** (Lock, or guarantee overlap cannot matter.)
4. **What happens if it crashes mid-run?** (Resume safely; small committed batches.)
5. **How will I know it ran — and how will I know if it *stopped* running?** (Heartbeat plus alert-on-absence.)
6. **Where do its logs go?** (Somewhere you actually read — `journalctl`, not a local mailbox.)

A scheduled job you cannot answer these for is an outage waiting for a quiet night.

## 25.10 A Process Management Cheat Sheet

```

Common process management commands for the Notes API
(this book's setup: Docker restart policy; systemd starts Docker)

Status
docker ps # Running containers
docker stats notes-api # Container resource usage
docker inspect notes-api --format '{{.State.Status}} restarts={{.RestartCount}}'
systemctl status docker # Docker daemon status
systemctl status nginx # Nginx status

Starting and stopping
docker stop notes-api # Graceful stop (stays stopped)
docker start notes-api # Start it again
docker restart notes-api # Stop + start
systemctl reload nginx # Nginx config reload (no downtime)

Logs
docker logs -f notes-api # Follow Notes API logs
docker logs --since 1h notes-api # Logs from the last hour
tail -f /var/log/nginx/error.log # Follow Nginx errors

Deployment
/opt/notes-api/deploy.sh sha-HASH # Deploy (or roll back to) a version
docker pull ghcr.io/.../notes-api:sha-HASH # Manually pull an image

Diagnostics
docker exec -it notes-api bash # Shell inside container
docker inspect notes-api # Full container configuration
docker top notes-api # Gunicorn master and workers
journalctl -u docker -b -1 # Docker's log from the previous boot

With the systemd alternative (section 24.7) instead:
systemctl status notes-api # Service status
systemctl restart notes-api # Stop + start
journalctl -u notes-api -f # Follow Notes API logs
journalctl -u notes-api -b -1 # Logs from previous boot (post-crash)
systemctl reset-failed notes-api # Reset crash counter after fixing issue
systemctl daemon-reload # Reload unit files after editing
systemctl enable notes-api # Ensure it starts on boot
```

### Key Takeaways

- A **process supervisor** ensures a process starts on boot and restarts after crashes. Without supervision, crashes require human intervention.
- **systemd** is the init system on Ubuntu. It manages every service on the server via unit files (`.service` files in `/etc/systemd/system/`).
- **systemctl** controls services: `start`, `stop`, `restart`, `reload`, `enable`, `disable`, `status`.
- **journalctl** queries the systemd journal: `-u SERVICE` for a specific service, `-f` to follow, `--since` for time filtering.
- The **[Service]** section of a unit file defines: `ExecStart` (how to start), `ExecStop` (how to stop), `Restart=always` (restart on any exit), `RestartSec=5` (seconds between restarts).
- **Crash loops** occur when a service crashes immediately after starting. `systemd` eventually stops restarting. Fix the root cause (check logs), then `systemctl reset-failed` and `systemctl start`.
- The **service hierarchy**: `systemd` starts `docker.service` (and `containerd`); containers run under `containerd-shim` in `cgroups` of their own, not inside any service you wrote; inside the container, the Gunicorn master manages its workers. Measure the application with `docker stats`, not with the numbers of a unit that runs `docker run`.
- For **multiple services** (Nginx, Docker container, PostgreSQL), declare `After=` and `Requires=` dependencies. `systemd` starts them in the correct order.
- **Background work** — scheduled (cron, `systemd` timers, cloud schedulers) and event-driven (webhooks, queues) — is not driven by user requests, so it fails *silently*. Prefer **systemd timers** over cron (logs to `journalctl`, `Persistent=true` catches missed runs, visible in `systemctl list-timers`); always schedule in UTC and load config explicitly.
- Every scheduled job must be **idempotent** (safe to run twice), **overlap-protected** (a lock), **durable** (resumable in small committed batches), and **visible** — alert on the *absence of success* (a heartbeat), because a missed run produces no error to alert on.
- When work doesn't belong inside a request, move it to a **queue** with a **worker** process supervised exactly like the API — the entry point to asynchronous architecture.

## Exercises

1. **Predict.** On the server, run `systemctl status docker` and `journalctl -u docker --since "1 hour ago"`. What do you expect to find after a deploy?
2. **Break and fix.** Create a `systemd` timer that runs a script that always fails. How would you notice? Add the “alert on absence of success” heartbeat from section 25.9.
3. **Extend.** Write the `systemd` timer and service that run section 25.9's `/opt/notes-api/scripts/cleanup.sh` daily at 03:00 UTC, with `Persistent=true`. Then check them with `systemctl list-timers` and `journalctl`.

4. **Explain.** Why is “make every scheduled job idempotent” the most important rule for background work?

Solutions and hints are in the companion repository, in `exercises/chapter-25.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

Services are running, supervised, and logged. The next layer to understand is how traffic actually flows through the network — from a packet leaving a client device, through the internet's routing infrastructure, to the server's network interface, through the kernel's network stack, to Nginx, to Gunicorn, to Flask. Understanding this path is essential for diagnosing the inevitable network issues.

→ [Chapter 26 — Network Stack Flow](#)



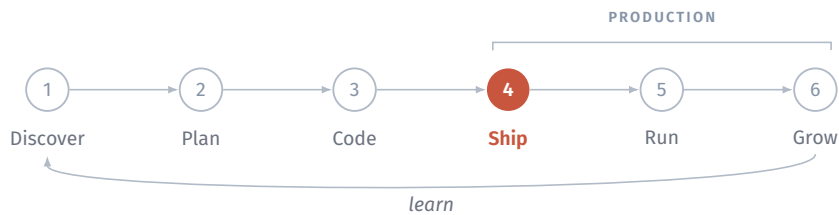
## STAGE 7

# 7

## Making It Reachable

*How does it become reachable from outside?*

### YOU ARE HERE



**The piece:** The path: a name, and the front door

The whole map: *How Software Gets Built*, page 1





# Network Stack Flow

---

When a user sends a request to the Notes API — `curl https://api.notes.example.com/notes/` — a complex sequence of events occurs across multiple layers of software, hardware, and network infrastructure. Most developers have a vague sense that “the request goes to the server somehow,” but the specifics matter enormously for diagnosing problems, understanding latency, and knowing where failures can occur.

This chapter traces the complete path of a packet from a client machine through the internet to the Notes API server, and back. Not hand-wavily, but precisely: which software runs, which kernel calls are made, which buffers are involved, and where in this path problems can emerge.

## In this chapter

By the end of this chapter, you will understand:

- How the Linux kernel’s network stack processes incoming TCP packets
- What each layer of the network stack does (Ethernet, IP, TCP, HTTP)
- How the kernel’s socket buffer system works and why buffer sizes matter
- What the TCP handshake looks like from the server’s perspective
- How packets flow from the NIC through the kernel to Nginx to Unicorn
- Where in this path common network failures occur

## 26.1 The Layered Network Model

Network protocols are organized in **layers**, where each layer provides services to the layer above it and relies on services from the layer below it. The formal reference is the seven-layer **OSI model** (Open Systems Interconnection); the practical one is the **TCP/IP model**. The diagram below is a simplified five-layer version that matches how the Notes API’s traffic actually travels.

APPLICATION LAYER	HTTP/HTTPS
	uses
TRANSPORT LAYER	TCP (reliable, ordered, connection-based)
	uses
INTERNET LAYER	IP (addressing and routing)
	uses
LINK LAYER	Ethernet / Wi-Fi (physical transmission)
	uses
PHYSICAL LAYER	Electrical signals / radio waves

Each layer adds its own **header** to the data, wrapping the payload from the layer above:

```

HTTP data: "GET /notes/ HTTP/1.1\r\nHost: api.notes.example.com\r\n\r\n"
TCP wraps it: [TCP header | HTTP data]
IP wraps it: [IP header | TCP header | HTTP data]
Ethernet wraps it: [Ethernet header | IP header | TCP header | HTTP data]
Physical transmission as electrical signals

```

When this arrives at the server, each layer strips its header and hands the payload up to the layer above.

## 26.2 The Journey of a Single Packet

### On the client side

A `curl` command starts the journey:

```
curl https://api.notes.example.com/notes/
```

#### Step 1: DNS resolution

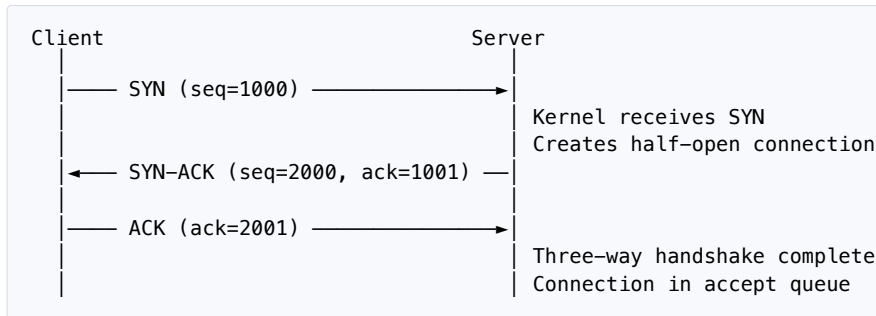
Before any packet is sent, the OS must resolve `api.notes.example.com` to an IP address. The resolver checks:

1. Local cache (if recently resolved, use cached result)
2. `/etc/hosts` file
3. DNS resolver (typically the router at `192.168.1.1`, which queries upstream DNS servers)

Result: `api.notes.example.com` → `203.0.113.42`

#### Step 2: TCP three-way handshake

curl asks the OS to open a TCP connection to 203.0.113.42:443:

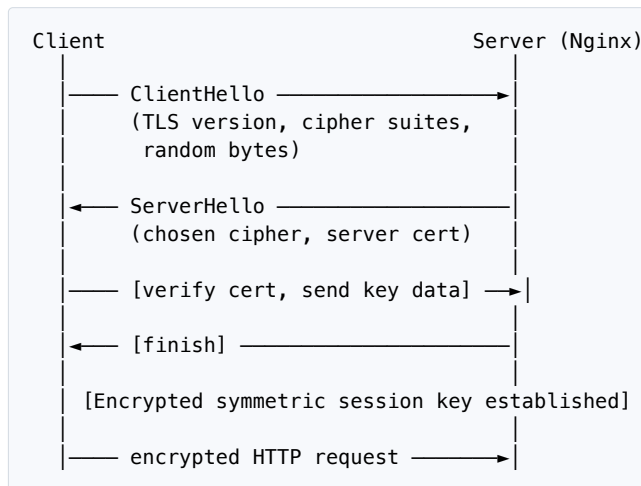


**SYN:** Client says “I want to connect. My sequence number starts at 1000.” **SYN-ACK:** Server says “OK. I’ll use sequence number 2000. Acknowledging your 1000.” **ACK:** Client says “Acknowledged.”

After the three-way handshake, the connection is established but not yet accepted by the application. It sits in the **accept queue** until the program listening on that port calls `accept()` — for the client’s connection to port 443, that is Nginx. (Nginx then opens a second, separate connection to Gunicorn on port 5000, which waits in Gunicorn’s accept queue in exactly the same way.)

### Step 3: TLS handshake

For HTTPS, TLS negotiation happens after TCP:



The TLS handshake takes 1–2 round trips. Modern TLS 1.3 can complete in 1 round trip. The server’s certificate proves the server’s identity (so the client knows it is talking to the real

api.notes.example.com, not an impostor).

After TLS, all data is encrypted. The encryption/decryption happens in Nginx (TLS termination), which then forwards the decrypted HTTP request to Unicorn.

## 26.3 The Server's Network Stack

When a packet arrives at the server:

### PACKET ARRIVAL

1. NIC (Network Interface Card) receives the Ethernet frame
2. NIC writes the frame directly into kernel memory (DMA), into a slot of its receive ring buffer
3. NIC raises an interrupt: "I have data"
4. Kernel interrupt handler (NAPI/softirq) runs:
  - a. Wraps the frame's memory in a socket buffer structure (sk\_buff) – no copy
  - b. Re-enables interrupts (to accept more packets)
5. Network stack processes the frame:
  - a. Ethernet driver: strips Ethernet header, passes IP packet up
  - b. IP layer: checks destination IP (203.0.113.42 = us!), strips IP header
  - c. TCP layer: finds the socket for (src\_ip, src\_port, dst\_ip:443)
  - d. TCP layer: verifies sequence numbers, sends ACK if needed
  - e. TCP layer: copies data to socket's receive buffer
6. Application reads data from socket receive buffer via `recv()/read()`

### The socket receive buffer

Each TCP socket has two buffers:

- **Receive buffer:** data received from the network, waiting to be read by the application
- **Send buffer:** data the application has written, waiting to be sent over the network

```
View socket buffer sizes
sysctl net.ipv4.tcp_rmem # TCP receive buffer: min, default, max (auto-tuned)
sysctl net.core.rmem_default # Default receive buffer size for other sockets
→ (bytes)
sysctl net.core.rmem_max # Maximum receive buffer size
sysctl net.core.wmem_default # Default send buffer size
sysctl net.core.wmem_max # Maximum send buffer size
```

If the receive buffer fills up (because the application is not reading fast enough), the kernel advertises a smaller and smaller **receive window** in its ACKs — until it reaches zero, which tells the sender to stop. The sender pauses and periodically probes until space frees up. This

is **TCP flow control**; it is separate from congestion control, which reacts to packet loss in the network. This is called **receive buffer pressure** and can look like a “slow application” when the real cause is a slow consumer.

## The file descriptor

When the kernel creates a TCP connection, it creates a **socket** — identified by a file descriptor in the application’s process. The file descriptor is an integer (4, 7, 23). Reading from this file descriptor reads from the socket’s receive buffer.

```
Conceptually, what happens when a Gunicorn worker calls accept()
on the connection Nginx opened to port 5000:
import socket

Create the listening socket
server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_sock.bind(('0.0.0.0', 5000))
server_sock.listen(128)

accept() returns a NEW socket for the connected client
client_sock, client_addr = server_sock.accept()
client_sock is a file descriptor – e.g., fd=7
client_addr is the client's (IP, port) – e.g., ('172.17.0.1', 54322)

Read the HTTP request from the client socket
data = client_sock.recv(4096)
```

After `accept()`, Gunicorn has a file descriptor for the client connection. Reading from it reads the HTTP request bytes. Writing to it sends the HTTP response bytes.

## 26.4 TCP States

TCP connections pass through a sequence of states. Understanding these states helps diagnose network issues.

```
View all TCP connections and their states
ss -tn

State Recv-Q Send-Q Local Address:Port Peer Address:Port
ESTABLISHED 0 0 203.0.113.42:443 1.2.3.4:54322
TIME-WAIT 0 0 203.0.113.42:443 1.2.3.5:54323
LISTEN 0 511 0.0.0.0:443 *:*
```

State	Meaning
LISTEN	Waiting for incoming connections
SYN_RECEIVED	SYN received, SYN-ACK sent, waiting for ACK
ESTABLISHED	Connection established, data can flow
FIN_WAIT_1	Our side sent FIN (closing)
FIN_WAIT_2	Received ACK to our FIN, waiting for remote FIN
TIME_WAIT	Both sides closed; waiting to ensure delayed packets are gone
CLOSE_WAIT	Remote side closed; we have not closed yet

**TIME\_WAIT** is the state the side that closed first stays in after both sides close — for 60 seconds on Linux (the standard calls it  $2 \times \text{MSL}$ , twice the Maximum Segment Lifetime). This prevents delayed duplicate packets from being mistaken for packets on a new connection.

A server under heavy traffic might have thousands of **TIME\_WAIT** connections. This is normal. It becomes a problem only if the server exhausts its ephemeral port range — which happens to the side making many *outgoing* connections. On this server that is Nginx, opening a fresh connection to Unicorn for each request; that is exactly the case `net.ipv4.tcp_tw_reuse = 1`, set in [Chapter 21](#), helps with.

## 26.5 The Kernel Receive Path in Detail

Zooming into step 4 of the packet arrival sequence: the kernel uses **NAPI** (New API) to handle high-speed packet reception efficiently.

### Interrupt-driven vs polling

The naive approach: every packet triggers a hardware interrupt → interrupt handler copies the packet. At high packet rates (millions per second), interrupt overhead dominates.

NAPI uses a hybrid approach:

- **First packet:** interrupt fires, handler starts
- **Subsequent packets** (during interrupt handling): poll the NIC ring buffer directly (no interrupt per packet)
- **After a budget of packets:** re-enable interrupts

This dramatically reduces interrupt overhead at high packet rates while maintaining low latency at low packet rates.

## sk\_buff — the socket buffer structure

Every packet in the kernel is represented by an `sk_buff` structure (socket buffer). It contains:

- Pointers to the packet data (not a copy — the kernel avoids copying where possible)
- Metadata: source/destination addresses, protocol, timestamps
- State: whether checksums have been verified, whether this is a clone

As the packet moves up the network stack, each layer's header pointer moves forward (stripping the layer's header without copying the data).

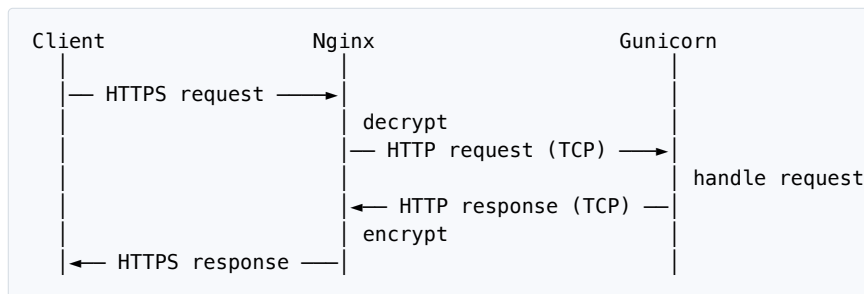
## 26.6 Nginx's Role in the Network Stack

Nginx sits between the network stack and Gunicorn. It:

1. Accepts TLS connections (decrypts HTTPS)
2. Buffers the complete HTTP request
3. Forwards the decrypted request to Gunicorn over a local TCP connection
4. Receives Gunicorn's response
5. Sends the response back to the client (with TLS)

Why buffer the complete request? Gunicorn workers are expensive (full Python processes). If Nginx did not buffer the request and a slow client sent the request one byte at a time, the Gunicorn worker would be occupied waiting for the slow client. With buffering, Nginx holds the connection with the slow client and only talks to Gunicorn when it has the complete request.

### The Nginx-to-Gunicorn connection



The connection between Nginx and Gunicorn is a local TCP connection — same host, no encryption needed. Nginx connects to `127.0.0.1:5000`; Docker forwards this to the container's

port 5000. For connections from the host's own loopback address, the forwarding is done by `docker-proxy`, a small userspace process that listens on `127.0.0.1:5000` and relays the bytes into the container; traffic arriving from other machines is forwarded by iptables DNAT rules instead.

## Nginx buffering settings

```
In nginx configuration:
Buffer size for reading the client request body
client_body_buffer_size 16k;
Maximum allowed size of the client request body (file uploads, etc.)
client_max_body_size 10m;

Buffer for responses from Gunicorn
proxy_buffers 8 16k;
proxy_buffer_size 16k;
```

## 26.7 The Complete Network Path (End to End)

```
CLIENT (curl or browser)
|
| 1. DNS: api.notes.example.com → 203.0.113.42
| 2. TCP SYN to 203.0.113.42:443
|
INTERNET ROUTING
|
| 3. Packet traverses ISP networks, internet exchange points
| BGP routing directs packet toward destination AS
|
SERVER'S CLOUD PROVIDER NETWORK
|
| 4. Cloud firewall: port 443 allowed? → yes
| 5. Reserved IP routes packet to server's NIC
|
SERVER KERNEL (Ubuntu 22.04)
|
| 6. NIC hardware interrupt → NAPI polling
| 7. Ethernet frame → IP packet → TCP segment
| 8. UFW (netfilter INPUT): port 443 allowed? → yes
| 9. TCP: SYN → SYN-ACK → ACK (three-way handshake)
| 10. Connection placed in accept queue
| 11. Socket receive buffer: HTTP request bytes accumulate
|
NGINX (PID ~500, listening on 0.0.0.0:443)
|
```



- 12. `accept()` returns client socket fd
- 13. TLS handshake with client
- 14. Decrypt HTTP request
- 15. Buffer complete request
- 16. Open TCP connection to 127.0.0.1:5000
- 17. Forward HTTP request (unencrypted)

#### DOCKER PORT MAPPING

- 18. `docker-proxy` (listening on 127.0.0.1:5000) relays the connection to 172.17.0.2:5000 (the container's internal IP)

#### CONTAINER NETWORK INTERFACE

- 19. Packet enters container's network namespace
- 20. veth pair: packet crosses from host to container

#### GUNICORN (PID 1 in container, workers on port 5000)

- 21. Kernel: connection ready in accept queue
- 22. Worker calls `accept()` → client socket fd
- 23. Worker reads HTTP bytes from socket receive buffer

#### FLASK APPLICATION

- 24. Werkzeug: parse HTTP request bytes → Request object
- 25. URL routing: `/notes/` + GET → `list_notes` handler
- 26. Before-request hooks run
- 27. Route handler executes
- 28. Database query: open connection to PostgreSQL (managed, over the VPC)
- 29. Execute SQL, receive rows
- 30. Build response JSON
- 31. After-request hooks run
- 32. Return response bytes to Gunicorn

#### GUNICORN (response path)

- 33. Write response bytes to socket
- 34. Worker calls `accept()` again (ready for next request)

#### NGINX (response path)

- 35. Receive response from Gunicorn
- 36. Encrypt response (TLS)
- 37. Send HTTPS response to client

#### CLIENT

- 38. Receive HTTPS response
- 39. Decrypt → HTTP response

```
| 40. Display to user / return to program
```

40 steps for one HTTP request. Most take microseconds. The server's share of a simple GET /notes/ — with the managed database in the same region — is about 5–10 milliseconds; the client adds its network round trips on top. The breakdown:

Phase	Typical time
DNS resolution (cached)	0 ms
TCP handshake	1 RTT = client-server latency
TLS handshake (TLS 1.3)	1 RTT
HTTP request transmission	< 1 ms
Nginx processing	< 1 ms
Gunicorn/Flask processing	1–5 ms
Database query	1–3 ms
Response transmission	< 1 ms
<b>Total (same datacenter)</b>	<b>~5–10 ms</b>
<b>Total (client in different region)</b>	<b>50–150 ms (RTT dominates)</b>

## 26.8 Where Network Failures Occur

Understanding the network path makes it possible to diagnose failures systematically:

```

CLIENT → DNS resolution failed?
└─ Check: dig api.notes.example.com
 nslookup api.notes.example.com

CLIENT → TCP connection refused? (the server answered: nothing listens there)
└─ Check: nc -zv SERVER_IP 443
 Is Nginx running? systemctl status nginx

CLIENT → TCP connection timed out? (packets are dropped: usually a firewall)
└─ Check: sudo ufw status
 The cloud firewall's rules in the provider's control panel

CLIENT → TLS handshake failed?
└─ Check: curl -v https://api.notes.example.com
 openssl s_client -connect api.notes.example.com:443
 Is the certificate expired? certbot certificates

NGINX → Gunicorn connection refused?
└─ Check: Is the container running? docker ps
 Is port 5000 mapped? docker port notes-api
 nc -zv 127.0.0.1 5000

GUNICORN → Application error?
└─ Check: docker logs notes-api
 journalctl -u notes-api

APPLICATION → Database connection failed?
└─ Check: nc -zv DB_HOST DB_PORT (not ping: managed databases
 usually do not answer ICMP)
 DATABASE_URL correct in env file?

```

### Key Takeaways

- The network stack is a series of layers: Physical → Ethernet → IP → TCP → HTTP. Each layer adds/removes headers as packets travel up or down.
- **The TCP three-way handshake** (SYN → SYN-ACK → ACK) must complete before any HTTP data flows. The TLS handshake happens after TCP, before HTTP.
- **NAPI** is the kernel's mechanism for efficient high-speed packet reception: first packet triggers an interrupt, subsequent packets are polled.
- **Socket receive buffers** hold incoming data until the application reads it. If the application is slow, the buffer fills, the advertised receive window shrinks to zero, and TCP flow control pauses the sender.
- **Nginx buffers the complete request** before forwarding to Gunicorn, protecting expensive Python workers from slow clients.
- **The Docker network** forwards host port 5000 to the container's internal IP — through docker-proxy for loopback connections such as Nginx's, and iptables DNAT rules for traffic from other machines. Traffic crosses a virtual Ethernet pair (veth) between the host

and container namespaces.

- **Diagnosing network failures** requires checking each layer in sequence: DNS → TCP connectivity → TLS → Nginx → Container → Application → Database.

## Exercises

1. **Predict.** On the server, run `ss -tan` while you send requests with `curl http://127.0.0.1:5000/health` from a second SSH session. Which connections do you see, and in which states?
2. **Break and fix.** Stop the application container, then call `curl -v http://127.0.0.1:5000/health` on the server. What error do you get, and from which layer? Start the container again.
3. **Extend.** Capture one request on the server with `sudo tcpdump -i lo port 5000 -c 20` while you run `curl http://127.0.0.1:5000/health`, and label the TCP handshake, the request, the response and the close.
4. **Explain.** Follow a failed request down the layers: how would you tell a DNS failure from a TCP failure from a TLS failure, using only `curl -v`?

Solutions and hints are in the companion repository, in `exercises/chapter-26.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

Traffic arrives at the server's network stack. Before it can arrive, it must know where to go — and that knowledge comes from DNS.

→ [Chapter 27](#) — Domain Names and DNS

# Domain Names and DNS

---

Chapter 10 explained why `localhost` is meaningless outside your machine. Chapter 22 showed that the server has a public IP address. But users do not connect to IP addresses — they connect to names like `api.notes.example.com`.

**DNS (Domain Name System)** is the global infrastructure that translates names to IP addresses. It is one of the oldest and most critical pieces of internet infrastructure. Without DNS, every user would need to know the IP address of every service they use — and every time a server’s IP changed, every user would need to update their knowledge.

This chapter covers how DNS works, how to register a domain, how to create DNS records, and how propagation works. It also covers the operational details that trip up developers new to production: TTLs, record types, and the difference between `example.com` and `api.example.com`.

## In this chapter

By the end of this chapter, you will understand:

- How DNS translates domain names to IP addresses (the full resolution chain)
- What DNS record types are (A, AAAA, CNAME, MX, TXT) and what each is used for
- How to register a domain and configure a DNS zone
- What TTL (Time to Live) is and how it affects deployment speed
- How to set up a subdomain for the Notes API
- How to verify DNS propagation
- What DNSSEC is (briefly) and why it exists

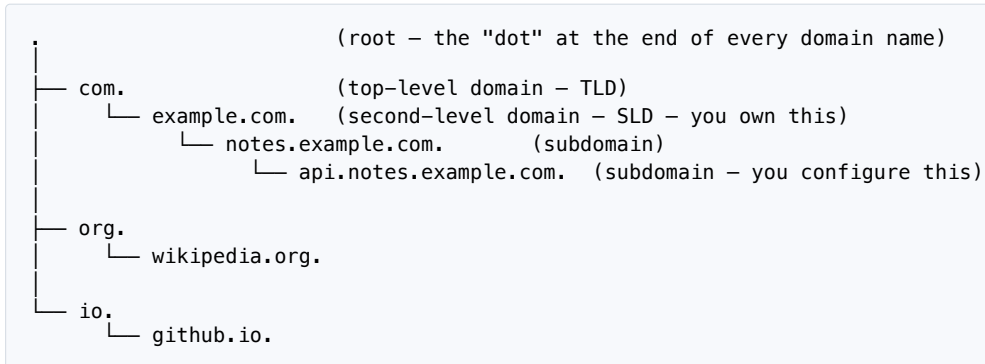
## 27.1 How DNS Works: The Resolution Chain

When your browser navigates to `api.notes.example.com`, before any TCP connection is opened, the operating system must discover the IP address. This discovery involves a hierarchy

of DNS servers.

## The DNS hierarchy

DNS is organized as a tree:



Each level of the hierarchy is managed by different servers:

- **Root servers:** 13 logical root server identities, run by 12 organizations (ICANN and Verisign among them) as well over a thousand anycast instances worldwide, that know which servers handle each TLD
- **TLD servers:** servers that know which servers handle each SLD (e.g., Verisign runs the .com TLD servers)
- **Authoritative nameservers:** servers that know the actual records for your domain (run by your domain registrar or DNS provider like Cloudflare, Route53)
- **Recursive resolvers:** servers that do the work of traversal (typically your ISP or a public resolver like 8.8.8.8)

## The full resolution sequence

```
What happens when resolving api.notes.example.com:

1. Client checks local cache:
 Has api.notes.example.com been resolved recently? No → proceed

2. Client asks its configured resolver (e.g., 8.8.8.8 – Google's DNS):
 "What is api.notes.example.com?"

3. Resolver checks its cache: No
```

```

4. Resolver asks a root server:
 "Who handles .com?"
 Root server: "Ask 192.5.6.30 (Verisign's .com TLD server)"

5. Resolver asks the .com TLD server:
 "Who handles example.com?"
 TLD server: "Ask ns1.dns-provider.example (example.com's authoritative
 nameserver)"

6. Resolver asks example.com's authoritative nameserver:
 "What is api.notes.example.com?"
 Authoritative nameserver: "203.0.113.42 (TTL: 300 seconds)"

7. Resolver returns 203.0.113.42 to the client
 Resolver caches this result for 300 seconds

8. Client connects to 203.0.113.42

```

A completely cold lookup like this takes somewhere between 10 and 100 milliseconds. In practice it is rarely that cold: resolvers keep the root and .com referrals cached for a day or more, so usually only step 6 goes out over the network. Lookups answered from a cache take a millisecond or less.

### The recursive resolver

The **recursive resolver** (also called a **recursive DNS server**) is the server that does the traversal on behalf of the client. Common resolvers:

- 8.8.8.8, 8.8.4.4 — Google Public DNS
- 1.1.1.1, 1.0.0.1 — Cloudflare DNS (privacy-focused)
- Your ISP's resolver (configured automatically via DHCP)
- Your router (which forwards to one of the above)

Most clients use their configured resolver without knowing which root or TLD servers exist.

## 27.2 DNS Record Types

A **DNS zone** is the set of DNS records for a domain. Each record maps a name to a value, with a specific record type.

## A record — IPv4 address

The most common record. Maps a hostname to an IPv4 address.

```
api.notes.example.com. 300 IN A 203.0.113.42
```

Fully qualified domain name (note the trailing dot)  
 TTL in seconds  
 Record class (IN = Internet)  
 Record type (A = IPv4 address)  
 IPv4 address

## AAAA record — IPv6 address

Maps a hostname to an IPv6 address. The modern equivalent of the A record.

```
api.notes.example.com. 300 IN AAAA 2001:db8::1
```

## CNAME record — Canonical name (alias)

Maps a hostname to another hostname. The resolver follows the CNAME to find the ultimate A/AAAA record.

```
www.example.com is an alias for example.com
www.example.com. 300 IN CNAME example.com.
```

**Important CNAME restriction:** A CNAME record cannot coexist with other records for the same name. The domain apex (`example.com.`, without a subdomain) cannot be a CNAME — this is why cloud providers have “ALIAS” or “ANAME” records as workarounds.

## MX record — Mail exchange

Specifies which servers handle email for the domain. Not needed for the Notes API, but ubiquitous.

```
example.com. 3600 IN MX 10 mail1.example.com.
example.com. 3600 IN MX 20 mail2.example.com.
Priority: 10 is tried before 20
```

## TXT record — Text

Arbitrary text associated with a name. Used for:



- Domain ownership verification (e.g., Google Search Console)
- SPF records (email authentication)
- DKIM records (email signing)
- ACME challenges (Let's Encrypt certificate issuance)

```
example.com. 300 IN TXT "v=spf1 include:_spf.google.com ~all"
```

## NS record — Nameserver

Specifies which servers are authoritative for a domain or subdomain.

```
example.com. 86400 IN NS ns1.digitalocean.com.
example.com. 86400 IN NS ns2.digitalocean.com.
example.com. 86400 IN NS ns3.digitalocean.com.
```

## 27.3 Registering a Domain

To create DNS records for `example.com`, you must first own `example.com`. Domains are registered through **domain registrars** — accredited organizations that manage domain registration.

Popular registrars:

- **Namecheap**: affordable, developer-friendly
- **Squarespace Domains** (which took over Google Domains in 2023): clean interface
- **Cloudflare Registrar**: at-cost pricing (no markup), excellent DNS management
- **GoDaddy**: large but has upsell practices

For a long-lived project, **Cloudflare** is a **strong choice** for both registration and DNS management: its DNS is among the fastest globally, it has a clean API for automation, it supports DNSSEC, and it offers free DDoS protection. This chapter's examples use **DigitalOcean's** DNS instead, for one practical reason: it lives in the same account as the server, so there is one control panel and one API token to learn. Everything shown carries over to Cloudflare.

### The registration process

1. Search for availability, on the registrar's site or with `whois notesapi.com` (WHOIS knows registered domains such as `notesapi.com`, not subdomains of them)
2. Purchase a domain (1–3 years, renewable)
3. Provide contact information (WHOIS data)
4. Configure the nameservers to point to your DNS provider

## Delegating to a DNS provider

If you register at Namecheap but want to use Cloudflare's DNS:

1. Add the domain to Cloudflare (free account)
2. Cloudflare assigns your account a pair of nameservers, with names like `ada.ns.cloudflare.com` and `bob.ns.cloudflare.com`
3. In Namecheap's control panel, update the domain's nameservers to Cloudflare's
4. Wait for the change to take effect everywhere — up to a day or two. The registry itself updates within minutes; the wait is for resolvers whose caches still hold the old NS records (`.com` NS records are cached for up to 48 hours).

## 27.4 Creating DNS Records for the Notes API

With the domain registered and DNS delegated to a provider, create the records:

### The zone configuration

```
DNS Zone for example.com
Managed through Cloudflare/DigitalOcean/Route53/etc.

Root domain → server IP (203.0.113.42 is the Reserved IP from
Chapter 22, so the records survive a server replacement)
example.com. 300 IN A 203.0.113.42

www → root domain (alias)
www.example.com. 300 IN CNAME example.com.

API subdomain → server IP
api.notes.example.com. 300 IN A 203.0.113.42
```

### Using DigitalOcean's DNS

DigitalOcean provides free DNS management for domains hosting resources on DigitalOcean:

1. Networking → Domains → Add Domain
2. Enter `example.com`
3. Update your registrar's nameservers to:
  - `ns1.digitalocean.com`
  - `ns2.digitalocean.com`

- ns3.digitalocean.com

#### 4. Create DNS records in the DigitalOcean control panel

### Using the DigitalOcean API to create DNS records

#### Listing 27.1 Creating DNS records via API

```

"""
Create DNS records using DigitalOcean's API.

In practice, you would do this via the web UI or CLI tool.
This script shows the API approach, which enables automation.

Requires the requests library (pip install requests).
Not idempotent: the API creates a new record on every call, so running
the script twice creates duplicate A records. A production version would
first list the existing records and update or skip instead.
"""
import os
import requests

DO_TOKEN = os.environ["DO_API_TOKEN"]
DOMAIN = "example.com"
SERVER_IP = "203.0.113.42"

headers = {
 "Authorization": f"Bearer {DO_TOKEN}",
 "Content-Type": "application/json",
}

def create_record(record_type: str, name: str, data: str, ttl: int = 300):
 """Create a DNS record in the DigitalOcean domain."""
 response = requests.post(
 f"https://api.digitalocean.com/v2/domains/{DOMAIN}/records",
 headers=headers,
 json={
 "type": record_type,
 "name": name,
 "data": data,
 "ttl": ttl,
 },
 timeout=10, # requests waits forever by default
)

```

```

response.raise_for_status()
record = response.json()["domain_record"]
print(f"Created: {record['type']} {record['name']} → {record['data']}")
return record

Root domain A record
create_record("A", "@", SERVER_IP)

API subdomain A record
DigitalOcean uses "@" for the apex and the name for subdomains
"api.notes" → api.notes.example.com
create_record("A", "api.notes", SERVER_IP)

```

## 27.5 TTL: Time to Live

Every DNS record has a TTL (Time to Live) — the number of seconds that resolvers and clients may cache the answer before re-querying.

```

api.notes.example.com. 300 IN A 203.0.113.42
 ↑
 300 seconds = 5 minutes

```

### The TTL tradeoff

#### Low TTL (60–300 seconds):

- DNS changes propagate quickly (within TTL seconds of updating the record)
- More DNS queries (higher load on DNS servers)
- More latency (DNS queries must be made more frequently)

#### High TTL (3600–86400 seconds):

- DNS changes take longer to propagate (old IP cached for up to TTL seconds)
- Fewer DNS queries (cached longer)
- Lower latency for users (fewer DNS lookups)

### TTL strategy for the Notes API

**Normal operation:** TTL = 3600 seconds (1 hour). Clients cache the IP for an hour; DNS servers have low load. (The records above use 300 seconds because the server is new and its setup may still change; raise them to 3600 once things are stable.)

**Before a planned IP change:** reduce TTL to 60–300 seconds at least one old-TTL period in advance (an hour ahead, for a 3600-second TTL). Wait for old records to expire. Then change the IP. Within 60–300 seconds, the new IP is live everywhere.

```
Before changing the server IP:
1. Reduce TTL to 300 seconds (do this 1 hour in advance)
2. Wait 1 hour (for the old 3600s TTL to expire everywhere)
3. Update the A record to the new IP
4. Within 300 seconds, all resolvers have the new IP
5. Increase TTL back to 3600 seconds
```

This is the standard procedure for zero-impact IP migrations.

## 27.6 Verifying DNS Configuration

After creating DNS records, verify they are correct before proceeding with TLS certificate issuance (Chapter 28).

```
Query the DNS record for the Notes API subdomain
dig api.notes.example.com A

Output:
;; QUESTION SECTION:
;api.notes.example.com. IN A
#
;; ANSWER SECTION:
api.notes.example.com. 300 IN A 203.0.113.42
#
;; Query time: 22 msec

Query a specific DNS server (bypass local cache)
dig @8.8.8.8 api.notes.example.com A

Short output (just the IP)
dig +short api.notes.example.com A
203.0.113.42

Query the authoritative nameserver directly
dig +short example.com NS # Find the authoritative nameservers
dig @ns1.digitalocean.com api.notes.example.com A # Query it directly

Check propagation from multiple global locations
```

```
Use a web tool: https://dnschecker.org
Enter: api.notes.example.com, Type: A
```

## Checking TTL countdown

```
The TTL in a cached answer shows how long it may still be cached
dig @8.8.8.8 api.notes.example.com A

First query: api.notes.example.com. 300 IN A 203.0.113.42 (TTL = 300)
Query again 10 seconds later:
Second query: api.notes.example.com. 290 IN A 203.0.113.42 (TTL = 290, counting
↳ down)
```

## 27.7 DNS Propagation

After creating or changing a DNS record, the change does not take effect everywhere simultaneously. This is called **DNS propagation**, but the name misleads: nothing is pushed or spread. The authoritative nameserver has the new record at once; every resolver that cached the *old* answer keeps using it until its TTL expires, and only then asks again.

How long propagation takes:

- If the old TTL was 3600 seconds: up to 3600 seconds (1 hour) for all resolvers to refresh
- If the TTL was 300 seconds: up to 5 minutes
- Your local machine’s cache: may hold the old record until TTL expires (flush it — on macOS: `sudo dscacheutil -flushcache`; `sudo killall -HUP mDNSResponder`; on Ubuntu: `sudo resolvectl flush-caches`)

“DNS propagation is complete” means all the old cached answers have expired. Check at <https://dnschecker.org> for a global view.

### Why “it worked for me but not for others”

Different users may be using different resolvers. User A’s resolver has refreshed; User B’s resolver still has the old cached record. This is normal during propagation. It resolves itself as TTLs expire everywhere.

## 27.8 DNSSEC: DNS Security Extensions

DNSSEC adds cryptographic signatures to DNS records, allowing resolvers to verify that DNS responses are authentic and have not been tampered with.

Without DNSSEC, a malicious actor can attempt **DNS spoofing** — for example **cache poisoning**, injecting false DNS responses into a resolver’s cache — directing users to a malicious server.

With DNSSEC:

- The DNS zone is signed with a private key
- Resolvers verify responses using the corresponding public key
- Tampered responses are detected and rejected

DNSSEC is supported by most major DNS providers (Cloudflare, Route53) — but not by DigitalOcean’s DNS, used in this chapter; to use DNSSEC, host the zone at a provider that supports it. Enabling it is straightforward:

1. Enable DNSSEC in your DNS provider’s control panel
2. Copy the **DS record** they provide
3. Add the DS record at your registrar (links the TLD’s trust anchor to your zone)

For the Notes API in a learning context, DNSSEC is optional but worth understanding.

### Key Takeaways

- DNS translates domain names to IP addresses via a hierarchy: a recursive resolver walks root servers → TLD servers → authoritative nameservers, then returns (and caches) the answer for the client.
- **DNS record types:** A (IPv4), AAAA (IPv6), CNAME (alias), MX (email), TXT (arbitrary text), NS (nameservers).
- **A records** map hostnames to IP addresses. `api.notes.example.com. 300 IN A 203.0.113.42` means “resolve to this IP, cache for 300 seconds.”
- **TTL (Time to Live)** controls how long resolvers cache DNS answers. Low TTL = fast propagation but more queries. High TTL = slow propagation but fewer queries.
- **Before changing a server’s IP:** reduce TTL → wait for old TTL to expire everywhere → change IP → increase TTL again.
- **Verify DNS with `dig api.notes.example.com A`.** Check global propagation at [dnschecker.org](https://dnschecker.org).
- **DNSSEC** prevents DNS spoofing by signing records cryptographically. Enable it at your DNS provider and registrar.

## Exercises

1. **Predict.** Run `dig +trace api.notes.example.com` (with your own domain). Identify the root server, the top-level domain server, and the authoritative server in the output.
2. **Break and fix.** Point a test subdomain at a wrong IP with a TTL of 3600, fix it five minutes later, and check with `dig @1.1.1.1` and `dig @8.8.8.8`. Why do the answers differ, and for how long?
3. **Extend.** Add an AAAA record if your server has an IPv6 address, and check that Nginx listens on IPv6 (`listen [::]:443 ssl;`).
4. **Explain.** Describe the TTL-lowering procedure for moving the API to a new server with no one seeing the old address.

Solutions and hints are in the companion repository, in `exercises/chapter-27.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

DNS is configured. Clients can resolve `api.notes.example.com` to the server's IP. But traffic arriving at port 443 currently goes nowhere — Nginx is not yet installed and configured. The traffic routing layer needs to be built.

→ [Chapter 28](#) — Traffic Routing Layer



# Traffic Routing Layer

---

Traffic from the internet arrives at the server on port 443. It must be decrypted, inspected, and routed to the correct backend service. That routing job belongs to **Nginx**, configured as a **reverse proxy**.

A **reverse proxy** is a server that sits in front of one or more backend services. Clients connect to the reverse proxy; the reverse proxy forwards requests to the appropriate backend. The client does not need to know what is behind the proxy.

Nginx is one of the most widely deployed reverse proxies (and web servers) in the world. In the Notes API architecture, Nginx:

1. Terminates TLS (decrypts HTTPS connections)
2. Redirects HTTP to HTTPS
3. Forwards decrypted requests to Gunicorn
4. Adds security headers to responses
5. Handles static file serving if needed

This chapter configures Nginx for the Notes API and issues a TLS certificate from Let's Encrypt, making the API accessible over HTTPS.

### In this chapter

By the end of this chapter, you will understand:

- What a reverse proxy is and why it is necessary between the internet and Gunicorn
- How Nginx configuration files are structured (server blocks, location blocks)
- What TLS termination is and why it happens in Nginx, not Gunicorn
- How to obtain a free, automatically-renewing TLS certificate from Let's Encrypt using Certbot
- How to configure Nginx to proxy requests to Gunicorn
- What security headers are and why they matter
- How to test and debug Nginx configuration

## 28.1 Why Gunicorn Cannot Face the Internet Directly

In development, `app.run()` handles HTTP directly. In production, Gunicorn handles HTTP. Could Gunicorn face the internet directly — no Nginx in front?

Technically possible but inadvisable for several reasons:

**TLS:** Gunicorn supports TLS (via `--keyfile` and `--certfile` flags), but TLS termination in a reverse proxy is simpler to configure and renew. Nginx handles TLS renewal without restarting Gunicorn.

**Static files:** Nginx serves static files (CSS, JavaScript, images) much faster than Gunicorn. Python is not optimized for serving binary files.

**Connection handling:** Nginx is asynchronous and can hold thousands of slow client connections with minimal resources. Gunicorn workers would be tied up waiting for slow clients to receive responses.

**HTTP/2:** Nginx supports HTTP/2; Gunicorn's default sync workers do not. HTTP/2 allows multiplexing multiple requests over one connection, significantly improving performance for clients with many resources to fetch.

**Security:** Nginx can inspect and filter requests before they reach Python. Rate limiting, IP blocking, header injection prevention — all at the network level, before expensive Python code runs.

**Separation of concerns:** Nginx handles the edge — TLS, slow clients, request limits, headers every response needs — and Gunicorn runs the application. Both speak HTTP; each does the part it is designed for.

## 28.2 Installing Nginx

The commands in this chapter change system configuration: run them as root, or prefix each with `sudo`.

```
On the server (Ubuntu 22.04)
apt-get update
apt-get install -y nginx

Verify installation
nginx -v
nginx version: nginx/1.18.0 (Ubuntu)

Nginx starts automatically after installation
systemctl status nginx
Active: active (running)
```

## 28.3 Nginx Configuration Structure

Nginx configuration files live in `/etc/nginx/`. The structure:

```
/etc/nginx/
├── nginx.conf ← Main configuration file
├── conf.d/ ← Drop-in configuration files (loaded by nginx.conf;
 empty in Ubuntu's package)
├── sites-available/ ← Virtual host configurations (not active)
│ └── default
├── sites-enabled/ ← Symlinks to active configurations
│ └── default → ../sites-available/default
```

`nginx.conf` includes:

```
At the bottom of nginx.conf:
include /etc/nginx/conf.d/*.conf;
include /etc/nginx/sites-enabled/*;
```

Both lines are inside the `http {}` block, so files in `conf.d/` can hold settings shared by every site. We place the Notes API site in `/etc/nginx/sites-available/notes-api` and symlink it to `sites-enabled/`. Two small shared pieces go in `conf.d/`: the upstream (below) and the rate-limiting zone (section 28.6).

## Nginx configuration concepts

**Context:** Nginx config has nested blocks (contexts):

- `http {}`: HTTP server settings
- `server {}`: a virtual host (one server block per domain)
- `location {}`: rules for specific URL paths

**Directive:** a configuration statement ending in `;`

**Block:** a group of directives in `{ }` that applies to a context

## 28.4 Initial Nginx Configuration (HTTP only)

Before configuring TLS, set up an HTTP-only configuration to verify basic connectivity. This is also required for Let’s Encrypt certificate issuance (the ACME HTTP-01 challenge).

First, the **upstream** — the named group of backend servers that the site proxies to. It lives in its own file because [Chapter 23](#)’s blue-green deploy script rewrites exactly this file to switch traffic between containers. It must be defined only here: a second `upstream notes_api` block anywhere else makes `nginx -t` fail with “duplicate upstream”.

```
/etc/nginx/conf.d/notes-api-upstream.conf
upstream notes_api {
 server 127.0.0.1:5000;
}
```

(127.0.0.1, not localhost: localhost may resolve to the IPv6 address `::1` first, where the container is not published.)

### Listing 28.1 /etc/nginx/sites-available/notes-api (HTTP only, initial)

```
HTTP-only configuration for initial setup and ACME challenge.
This will be replaced by the full HTTPS configuration after certificate issuance.

server {
 # Listen on port 80 for IPv4
 listen 80;
 # Listen on port 80 for IPv6
 listen [::]:80;

 # The domain name this server block handles.
 # Nginx compares the incoming HTTP Host header against this value.
```

```

server_name api.notes.example.com;

All requests: proxy to Gunicorn (through the upstream defined above).
No special location is needed for Let's Encrypt's challenge: Certbot's
Nginx plugin adds one temporarily while it proves domain ownership.
location / {
 proxy_pass http://notes_api;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
}
}

```

Enable it:

```

Create the symlink to enable the site
ln -s /etc/nginx/sites-available/notes-api /etc/nginx/sites-enabled/

Remove the default site (optional)
rm /etc/nginx/sites-enabled/default

Test the configuration for syntax errors
nginx -t
nginx: configuration file /etc/nginx/nginx.conf test is successful

Reload Nginx with the new configuration
systemctl reload nginx

```

Test:

```

From your laptop
curl http://api.notes.example.com/health
{"status":"ok"}

```

## 28.5 TLS Certificates with Let's Encrypt

TLS (Transport Layer Security) encrypts the connection between the client and server. Without TLS:

- Anyone between the client and server can read the request and response in plaintext
- Passwords, API keys, and sensitive data are exposed

- Browsers show “Not Secure” warnings

**Let’s Encrypt** is a free, automated, and open Certificate Authority. It issues TLS certificates at no cost. Each certificate is valid for 90 days, and Certbot renews it automatically about 30 days before it expires. (Let’s Encrypt has announced a move to shorter lifetimes over the next few years — automated renewal is what makes that painless.) It changed the web by making HTTPS accessible to everyone.

## How Let’s Encrypt works (ACME protocol)

1. You request a certificate for `api.notes.example.com`
2. Let’s Encrypt responds with a **challenge**: “Prove you control this domain”
3. For the **HTTP-01 challenge**: place a specific file at `http://api.notes.example.com/.well-known/acme-challenge/TOKEN`
4. Let’s Encrypt retrieves the file from the internet — proving the server controls the domain
5. Let’s Encrypt issues the certificate

**Certbot** is the official Let’s Encrypt client. It automates challenge completion and certificate installation.

## Installing Certbot

```
Install certbot and the Nginx plugin
apt-get install -y certbot python3-certbot-nginx
```

(Ubuntu 22.04’s package is an older Certbot release; Certbot’s own documentation recommends installing it as a snap, which stays current. Either works for this chapter.)

## Obtaining the certificate

```
Request a certificate for the API domain
--nginx: use the Nginx plugin (automatically configures Nginx)
--agree-tos: agree to Let's Encrypt's terms of service
--email: your email (for certificate expiry notifications)
certbot --nginx \
 --agree-tos \
 --email admin@example.com \
 --domains api.notes.example.com \
 --non-interactive \
 --redirect # Automatically configure HTTP → HTTPS redirect
```

```
Output (abbreviated):
Requesting a certificate for api.notes.example.com
#
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/api.notes.example.com/fullchain.pem
Key is saved at: /etc/letsencrypt/live/api.notes.example.com/privkey.pem
...
Deploying certificate
Successfully deployed certificate for api.notes.example.com to
↳ /etc/nginx/sites-enabled/notes-api
Congratulations! You have successfully enabled HTTPS on
↳ https://api.notes.example.com
```

After Certbot completes, Nginx automatically reloads with the new configuration. Test HTTPS:

```
curl https://api.notes.example.com/health
{"status":"ok"}
```

## Automatic certificate renewal

Let's Encrypt certificates expire after 90 days. Certbot installs a systemd timer (or cron job) for automatic renewal:

```
Verify the renewal timer
systemctl status certbot.timer
● certbot.timer - Run certbot twice daily
Active: active (waiting)

Test renewal without actually renewing (dry run)
certbot renew --dry-run
Simulating renewal of an existing certificate...
Congratulations, all simulated renewals succeeded.
```

The timer runs twice daily. When a certificate is within 30 days of expiration, Certbot renews it automatically — and, because the certificate was obtained with the `--nginx` plugin, reloads Nginx so it serves the new certificate. (With other methods, such as `--webroot`, add `--deploy-hook "systemctl reload nginx"` to get the same reload.)

## 28.6 The Complete Production Nginx Configuration

After certificate issuance, configure Nginx fully.

**Note:** TLS recommendations (protocol versions, cipher suites, header advice) evolve as attacks and browsers change. The settings below reflect current guidance at the time of writing; before deploying, compare them with Mozilla's SSL Configuration Generator ([ssl-config.mozilla.org](https://ssl-config.mozilla.org)) for your Nginx and OpenSSL versions.

### Listing 28.2 /etc/nginx/sites-available/notes-api (complete production config)

```
The upstream "notes_api" is defined in conf.d/notes-api-upstream.conf
(section 28.4) – not here – because the deploy script rewrites it.
#
(No "keepalive" pool in the upstream: Unicorn's default sync workers
close the connection after every response, so a pool would stay empty.
With the gthread worker class, add "keepalive 16;" to the upstream.)

HTTP server block – redirect all HTTP to HTTPS
server {
 listen 80;
 listen [::]:80;
 server_name api.notes.example.com;

 # (Certbot's Nginx plugin answers the ACME challenge on port 80 itself
 # during renewal, so no challenge location is needed here.)
 location / {
 return 301 https://$host$request_uri;
 }
}

HTTPS server block – the main configuration
server {
 # On Nginx 1.25.1 and later, write "listen 443 ssl;" plus "http2 on;"
 # instead: the http2 parameter of listen is deprecated there.
 listen 443 ssl http2;
 listen [::]:443 ssl http2;
 server_name api.notes.example.com;

 # _____
 # TLS Configuration
 # _____
```



```

Certificate files from Let's Encrypt
ssl_certificate /etc/letsencrypt/live/api.notes.example.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/api.notes.example.com/privkey.pem;

Only allow TLS 1.2 and 1.3 (1.0 and 1.1 have known vulnerabilities)
ssl_protocols TLSv1.2 TLSv1.3;

TLS 1.2 cipher suites: only ECDHE key exchange (forward secrecy) with
AES-GCM or ChaCha20 – OpenSSL shorthand for the core of Mozilla's
"intermediate" profile. TLS 1.3's cipher suites are all strong and
are not affected by this setting.
ssl_ciphers ECDHE+AESGCM:ECDHE+CHACHA20;
ssl_prefer_server_ciphers off;

SSL session cache (improves performance for repeated connections)
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d;
ssl_session_tickets off;

(Older guides also enable OCSP stapling here. Let's Encrypt shut down
its OCSP service in 2025, so for its certificates there is nothing
to staple; Nginx would only log a warning.)

Security Headers

HSTS: tell browsers to always use HTTPS for this domain
max-age=63072000 = 2 years in seconds
includeSubDomains: also applies to subdomains of this name.
Hard to undo: browsers remember it for max-age, so enable it only
once HTTPS works reliably.
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains"
↪ always;

Prevent the browser from MIME-type sniffing
add_header X-Content-Type-Options "nosniff" always;

Prevent clickjacking (embedding in iframes on other sites)
add_header X-Frame-Options "DENY" always;

(X-XSS-Protection is deliberately absent: the browser feature it
controlled is gone, could itself be abused, and never applied to JSON.)

```

```

Referrer Policy: how much referrer info to send with requests
add_header Referrer-Policy "strict-origin-when-cross-origin" always;

Permissions Policy: disable features not needed by the API
add_header Permissions-Policy "geolocation=(), microphone=(), camera=()" always;

Request size limit

Maximum size of the client request body. A note may have 10,000
characters, and in UTF-8 one character can take up to 4 bytes – plus
JSON syntax and escaping. 64k leaves room for every valid note.
client_max_body_size 64k;

Answer the errors Nginx itself produces in the API's JSON format,
instead of Nginx's HTML error pages
error_page 413 = @too_large;
error_page 429 = @rate_limited;

location @too_large {
 default_type application/json;
 return 413 '{"error": "Request body too large"}\n';
}

location @rate_limited {
 default_type application/json;
 return 429 '{"error": "Too many requests, please slow down"}\n';
}

Proxy configuration

location / {
 # Rate limiting: at most 10 requests per second per client IP, with
 # bursts of up to 20 served at once. The zone "api" is defined in
 # conf.d/ratelimit.conf (below). Rejected requests get a 429.
 limit_req zone=api burst=20 nodelay;
 limit_req_status 429;

 # Forward to the upstream (Gunicorn)
 proxy_pass http://notes_api;

```

```

Use HTTP/1.1 for the connection to Gunicorn, and clear the
Connection header (both needed for upstream keep-alive)
proxy_http_version 1.1;
proxy_set_header Connection "";

Pass the original Host header (Flask's request.host uses this)
proxy_set_header Host $host;

Pass the client's real IP address. Without this, Flask sees the
address of the hop in front of it – 172.17.0.1, Docker's bridge
gateway, where docker-proxy relays Nginx's connection.
proxy_set_header X-Real-IP $remote_addr;

Pass the chain of IPs (client → proxy1 → proxy2 → server)
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

Tell the application that the original request was HTTPS
Flask uses this to generate correct redirect URLs
proxy_set_header X-Forwarded-Proto $scheme;

Timeouts
proxy_connect_timeout 5s; # Time to connect to Gunicorn
proxy_send_timeout 60s; # Time to send request to Gunicorn
proxy_read_timeout 60s; # Time to wait for response from Gunicorn

Buffer settings
proxy_buffering on;
proxy_buffer_size 4k;
proxy_buffers 8 4k;
}

Health check endpoint (no rate limiting: limit_req is set only in
"location /", and this more specific location takes precedence)

location /health {
 proxy_pass http://notes_api;
 proxy_http_version 1.1;
 proxy_set_header Connection "";
 proxy_set_header Host $host;
 access_log off; # Don't log health check requests (very frequent)
}

```

```

Logging

access_log /var/log/nginx/notes-api-access.log;
error_log /var/log/nginx/notes-api-error.log warn;
}
```

## Define the rate-limiting zone

The `limit_req` line above refers to a zone named `api`, which must be defined in the `http {}` context. A file in `conf.d/` is inside that context (section 28.3), so it needs no edit to `nginx.conf`:

```
/etc/nginx/conf.d/ratelimit.conf
Define a rate limiting zone:
"api" = zone name
$binary_remote_addr = key (per client IP, binary encoding is more compact)
zone=api:10m = 10 MB of memory to store state (holds ~160,000 IPs)
rate=10r/s = 10 requests per second limit
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
```

## Apply and test

```
Test configuration
nginx -t
nginx: configuration file /etc/nginx/nginx.conf test is successful

Reload (graceful – no dropped connections)
systemctl reload nginx

Test HTTPS from laptop
curl -I https://api.notes.example.com/health
HTTP/2 200
server: nginx
content-type: application/json
strict-transport-security: max-age=63072000; includeSubDomains
x-content-type-options: nosniff
x-frame-options: DENY
referrer-policy: strict-origin-when-cross-origin
permissions-policy: geolocation=(), microphone=(), camera=()
(date, content-length and the timing header omitted)
```

## Check SSL configuration

Grade the TLS configuration with Qualys SSL Labs, in a browser:

```
https://www.ssllabs.com/ssltest/analyze.html?d=api.notes.example.com
```

The test takes a minute or two. A well-configured TLS setup with HSTS, like this one, achieves an A+ grade.

## 28.7 Understanding X-Forwarded-For

When Nginx proxies a request to Gunicorn, Gunicorn sees the previous hop's IP as the client's IP — here 172.17.0.1, the Docker bridge address that docker-proxy connects from — not the real client's IP. The real IP is passed via the X-Forwarded-For header.

Flask needs to be configured to trust this header:

```
In app/__init__.py
from werkzeug.middleware.proxy_fix import ProxyFix

def create_app() -> Flask:
 app = Flask(__name__)
 # ...
 # Tell Flask to trust the headers set by exactly 1 proxy (Nginx)
 # x_for=1: take the client IP from the last entry of X-Forwarded-For
 # x_proto=1: trust X-Forwarded-Proto
 # (No x_host: Nginx does not set X-Forwarded-Host, and trusting a header
 # nobody sets would let clients supply it.)
 app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1)
 return app
```

Without ProxyFix, request.remote\_addr always returns 172.17.0.1. With it, Flask reads the correct client IP from X-Forwarded-For, enabling IP-based logging and rate limiting.

## 28.8 Nginx as a Load Balancer

If the application scales to multiple servers (Chapter 36), Nginx can distribute traffic across them:

```

upstream notes_api {
 # Round-robin by default (each request goes to the next server)
 server 10.0.0.5:5000; # Server 1
 server 10.0.0.6:5000; # Server 2
 server 10.0.0.7:5000; # Server 3

 # Or: least connections (prefer the server with fewest active connections)
 # least_conn;

 # Or: IP hash (sticky sessions – same client always goes to same server)
 # ip_hash;

 keepalive 32;
}

```

With this configuration, Nginx distributes requests across three servers. Open-source Nginx checks backends *passively*: if requests to a server fail (`max_fails` times within `fail_timeout`, by default once in 10 seconds), it stops sending that server traffic for a while, then tries again. (Active health checks — probing a URL on a schedule — are a feature of the commercial NGINX Plus.) And since the Notes API keeps no per-user state in the application, it does not need `ip_hash`'s sticky sessions.

### Key Takeaways

- **Nginx as a reverse proxy** sits between the internet and Gunicorn, handling TLS termination, connection management, and security headers.
- **TLS termination** decrypts HTTPS at Nginx, forwarding plaintext HTTP to Gunicorn. Gunicorn only handles HTTP; TLS complexity is isolated in Nginx.
- **Let's Encrypt + Certbot** provides free, automatically-renewed TLS certificates. The ACME HTTP-01 challenge proves domain ownership by placing a file at `/.well-known/acme-challenge/`.
- **Security headers** (HSTS, X-Content-Type-Options, X-Frame-Options) protect against common attacks. Add them in the Nginx `server{}` block.
- **proxy\_set\_header X-Forwarded-For** (and `X-Real-IP`) pass the client's actual IP to Gunicorn. Flask's ProxyFix middleware reads `X-Forwarded-For` and `X-Forwarded-Proto` to restore the real client IP and scheme.
- **Nginx rate limiting** (`limit_req_zone` + `limit_req`) protects against brute-force attempts and abusive single clients. It does not stop a distributed attack from many IPs — that needs protection upstream of the server (the provider's network, or a CDN).
- **One upstream, one owner**: the `notes_api` upstream lives in `conf.d/notes-api-`

`upstream.conf`, which the deploy script rewrites; the site configuration only refers to it.

- **HTTP redirect:** the HTTP server block should only redirect to HTTPS. All content is served from the HTTPS block.
- **nginx -t** always before `systemctl reload nginx`. A syntax error in the config causes Nginx to fail — test before applying.

## Exercises

1. **Predict.** Send 30 requests in one second from one IP with a small for loop and curl. How many get 429, given `rate=10r/s burst=20 nodelay`?
2. **Break and fix.** Remove `proxy_set_header X-Forwarded-Proto $scheme;` and `ProxyFix`. What does Flask now get wrong about each request? Find one URL it generates that comes out wrong. Restore both.
3. **Extend.** Add a `location = /robots.txt` that returns `User-agent: *\nDisallow: /` directly from Nginx, without reaching the application.
4. **Explain.** Why does TLS end at Nginx rather than at Gunicorn, and what does the traffic between them rely on for protection?

Solutions and hints are in the companion repository, in `exercises/chapter-28.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

The traffic routing layer is configured. HTTPS traffic arrives at Nginx, is decrypted, and forwarded to Gunicorn. Now it is time to trace the complete client-to-server journey — a single request from the browser through every layer to the Flask handler — to synthesize everything covered in the past several chapters.

→ [Chapter 29](#) — Client-to-Server Journey





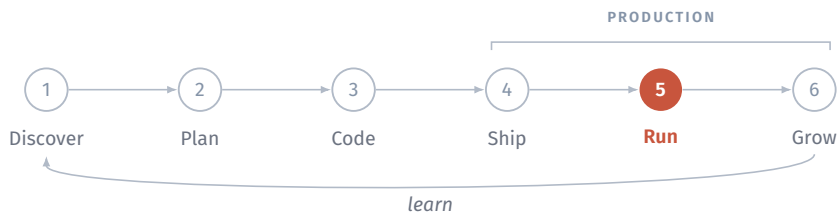
## STAGE 8

# 8

## Request Flow: External to Program

*How does external input arrive at the program?*

YOU ARE HERE



**The piece:** The path, followed by one request from the user to the program

The whole map: *How Software Gets Built*, page 1



# Client-to-Server Journey

---

The previous chapters built each component of the production system in isolation. This chapter synthesizes them by tracing a single request — `POST /notes/` — from the moment a user presses Enter to the moment they see the response, touching every layer that has been built.

This end-to-end trace serves two purposes. First, it confirms that the complete system is coherent — that all the pieces fit together as described. Second, it provides a mental model for diagnosing production problems. When something breaks, you need to know which layer it broke in, and that requires knowing what each layer does.

By the end of this chapter, you will have a complete, detailed understanding of the path a single HTTP request travels through the production Notes API system.

## 29.1 The Request: `POST /notes/`

A user runs:

```
curl -X POST https://api.notes.example.com/notes/ \
-H "Content-Type: application/json" \
-d '{"content": "Buy groceries"}'
```

This single command triggers the following sequence.

## 29.2 Phase 1: DNS Resolution (Client)

**Location:** Client machine (the laptop running curl)

```
curl needs the IP for api.notes.example.com
```

1. Check local DNS cache → not found (or expired)
2. Ask configured resolver (1.1.1.1 – Cloudflare)
3. Resolver checks its cache → found! (TTL: 237 seconds remaining)
4. Resolver returns: api.notes.example.com → 203.0.113.42

```
Time: ~0 ms (cached) or ~20 ms (fresh lookup)
```

### What could fail here:

- DNS record doesn't exist → curl: (6) Could not resolve host
- DNS record points to wrong IP → the connection goes to the wrong server; usually TLS then fails first, because that server's certificate does not match the name
- Resolver unreachable → timeout

## 29.3 Phase 2: TCP Connection (Client → Server)

**Location:** Client and server TCP stacks, internet routing

```
curl opens a TCP connection to 203.0.113.42:443
```

Client (laptop):

1. OS allocates an ephemeral port (e.g., 54832)
2. OS sends SYN packet:
  - src: 192.168.1.5:54832 (client private IP)
  - dst: 203.0.113.42:443

Internet routing:

3. Client's router NATs the packet:
  - src: 89.100.200.50:54832 (client's public IP)
  - dst: 203.0.113.42:443
4. Packet routes through ISP backbone, internet exchange, hosting provider

Server (cloud provider network):

5. Cloud firewall: dst port 443 allowed? → yes
6. Reserved IP (203.0.113.42) routed to the server, via its anchor IP
7. UFW (the server's firewall) checks port 443 → allowed

Server kernel:

7. NIC receives Ethernet frame → NAPI interrupt
8. IP layer: 203.0.113.42 = us → pass up

9. TCP layer: port 443, in LISTEN state
10. TCP layer: SYN received → allocate half-open connection
11. TCP layer: send SYN-ACK
12. ACK received → connection established
13. Connection placed in accept queue

Time: 1 RTT (round-trip time) ≈ 10–100ms depending on client location

#### What could fail here:

- Cloud firewall blocks port 443 → timeout (firewalls drop packets; “Connection refused” means nothing is listening)
- UFW blocks port 443 → timeout
- Server not listening on 443 → Connection refused

## 29.4 Phase 3: TLS Handshake (Client ↔ Nginx)

**Location:** Client TLS stack ↔ Nginx TLS stack

Nginx's `accept()` returns the client socket.  
Nginx starts TLS negotiation.

1. Client sends ClientHello:
  - TLS 1.3 preferred
  - Supported cipher suites
  - Random bytes for key derivation
2. Nginx sends ServerHello:
  - Chosen: TLS 1.3
  - Chosen cipher: TLS\_AES\_256\_GCM\_SHA384
  - Server's certificate (issued by Let's Encrypt for `api.notes.example.com`)
  - Random bytes
3. Client verifies the certificate:
  - Is it for `api.notes.example.com`? → yes
  - Is it signed by a trusted CA? → Let's Encrypt → ISRG Root X1 → trusted
  - Is it expired? → no (valid until 90 days from issuance)
  - Is the server in possession of the private key? → prove it
4. Key exchange (Diffie-Hellman):
  - Client and server exchange key material

- Both derive the same session encryption key
- The private key never leaves the server

5. Handshake complete:

- All subsequent data is encrypted with the session key
- The channel is: authenticated (server identity proven) + encrypted

Time: 1 RTT for TLS 1.3 (2 RTTs for TLS 1.2)

### What could fail here:

- Certificate expired → curl: (60) SSL certificate problem: certificate has expired
- Wrong domain in certificate → same error
- Certificate not trusted → same error
- TLS version mismatch → SSL handshake failure

## 29.5 Phase 4: HTTP Request (Client → Nginx)

Location: Client → Nginx

curl sends the HTTP request over the encrypted TLS channel.

The raw bytes (before encryption):

```
POST /notes/ HTTP/1.1\r\n
Host: api.notes.example.com\r\n
User-Agent: curl/8.1.2\r\n
Accept: */*\r\n
Content-Type: application/json\r\n
Content-Length: 28\r\n
\r\n
{"content": "Buy groceries"}
```

Nginx receives these bytes:

1. TLS layer decrypts the bytes
2. HTTP parser reads request line: POST /notes/ HTTP/1.1
3. HTTP parser reads headers (Host, Content-Type, Content-Length)
4. HTTP parser reads body (28 bytes of JSON)
5. Complete request buffered in Nginx

Time: < 1 ms (local processing)

## 29.6 Phase 5: Nginx → Unicorn Proxy

**Location:** Nginx → TCP → Docker port mapping → Unicorn

Nginx opens a connection to 127.0.0.1:5000 (the notes\_api upstream) and forwards the HTTP request.

Nginx modifies the request:

– Adds headers:

```
X-Real-IP: 89.100.200.50 (client's actual public IP)
X-Forwarded-For: 89.100.200.50 (same for single-proxy setup)
X-Forwarded-Proto: https (original scheme was HTTPS)
Host: api.notes.example.com (original Host header)
```

Nginx sends to 127.0.0.1:5000:

```
POST /notes/ HTTP/1.1\r\n
Host: api.notes.example.com\r\n
X-Real-IP: 89.100.200.50\r\n
X-Forwarded-For: 89.100.200.50\r\n
X-Forwarded-Proto: https\r\n
Content-Type: application/json\r\n
Content-Length: 28\r\n
\r\n
{"content": "Buy groceries"}
```

Docker port mapping (docker-proxy, listening on 127.0.0.1:5000):  
accepts Nginx's connection and opens its own connection from  
172.17.0.1 (the bridge gateway) to 172.17.0.2:5000 (container IP)

Container network namespace:

Packet crosses the veth pair: host namespace → container namespace

Unicorn worker calls `accept()` on the listening socket:

Returns: (client\_socket, ('172.17.0.1', ephemeral\_port))

Time: < 1 ms (local network, no encryption)

**What could fail here:**

- Container not running → Nginx returns 502 Bad Gateway
- Unicorn worker timeout → Nginx returns 504 Gateway Timeout
- All workers busy → requests wait in the accept queue; if none frees up in time, Nginx gives up at `proxy_read_timeout` (504)

## 29.7 Phase 6: Flask Request Processing

**Location:** Inside the Docker container

Gunicorn worker reads HTTP bytes from socket.

Gunicorn parses the HTTP request and builds the WSGI environ;

Flask (with Werkzeug) wraps the environ in a Request object.

1. Request object created:

```
request.method = "POST"
request.path = "/notes/"
request.headers = {
 "Host": "api.notes.example.com",
 "X-Real-IP": "89.100.200.50",
 "X-Forwarded-For": "89.100.200.50",
 "X-Forwarded-Proto": "https",
 "Content-Type": "application/json",
 "Content-Length": "28",
 ...
}
request.data = b'{"content": "Buy groceries"}'
```

2. ProxyFix middleware runs:

```
request.remote_addr = "89.100.200.50" (from X-Forwarded-For)
request.scheme = "https" (from X-Forwarded-Proto)
```

3. Before-request hooks:

```
- log_request_start() runs:
 g.request_start_time = perf_counter()
 logs: "→ POST /notes/ (from 89.100.200.50)"
```

4. URL routing:

```
Werkzeug's Map matches POST + /notes/ → create_note handler
(No URL variable extraction for /notes/)
```

5. create\_note() handler executes:

```
a. data = request.get_json(silent=True)
 → {"content": "Buy groceries"}

b. Validation:
 - data is a JSON object? ✓
 - content present? ✓ "Buy groceries"
 - content is a string? ✓
 - content after strip? "Buy groceries" (non-empty)
```



- content length  $\leq 10,000$ ? ✓ (13 chars)

c. Database interaction (next phase)

Time: < 1 ms (just parsing and validation)

## 29.8 Phase 7: Database Query

**Location:** Container → Managed PostgreSQL

`get_db()` is called inside `create_note()`.

First call in this request? `g` has no "db" key yet.

→ `open_db()` creates a new connection (the app opens one per request):

```
psycpg2.connect("postgresql://notes_user:pass@private-db-....db.ondigitalocean.com:
↳ 25060/notesdb?sslmode=require")
```

TCP connection to the database's private VPC address:

- TLS handshake (managed databases require it: `sslmode=require`)
- PostgreSQL authentication (SCRAM-SHA-256)
- Connection established – several network round trips before any SQL

SQL execution:

```
cursor.execute(
 "INSERT INTO notes (content) VALUES (%s) RETURNING id, content, created_at",
 ("Buy groceries",)
)
```

PostgreSQL processes the INSERT:

1. Begin implicit transaction
2. Acquire row lock on new row
3. Write to WAL (Write-Ahead Log) – durable
4. Insert into heap
5. Return: `id=42, content="Buy groceries",`  
`created_at=datetime(2024, 1, 15, 14, 0, 0, 123456, tzinfo=UTC)`

The handler (running in a Gunicorn worker) receives the row:

```
note = row_to_note(cursor.fetchone())
→ {"id": 42, "content": "Buy groceries",
 "created_at": "2024-01-15T14:00:00.123456+00:00"} (to_iso)
db.commit() – before the response is built: a 201 means "saved"
```

Time: 1–5 ms for the query itself – plus 5–20 ms to open the connection (TCP, TLS and authentication), paid again on every request because the app has no connection pool. Chapter 36 removes that cost.

## 29.9 Phase 8: Response Construction

Location: Flask application

`create_note()` returns the note data.

1. `jsonify(note)` serializes to JSON:

```
b'{"content":"Buy
↳ groceries","created_at":"2024-01-15T14:00:00.123456+00:00","id":42}'
```

2. Flask wraps in a Response object:

```
status_code = 201
headers = {
 "Content-Type": "application/json",
 "Content-Length": "84",
}
```

3. After-request hooks:

```
- log_request_end() runs:
 elapsed = perf_counter() - g.request_start_time = 14.3 ms
 logs: "- 201 in 14.3ms"
```

4. Teardown hooks:

```
- close_db() runs:
 db.rollback() (a no-op: the handler already committed)
 db.close() (close the connection's socket)
```

5. Gunicorn receives the Response object.

Serializes to HTTP/1.1 response bytes:

```
HTTP/1.1 201 Created\r\n
Server: gunicorn\r\n
Date: Mon, 15 Jan 2024 14:00:00 GMT\r\n
Connection: close\r\n
Content-Type: application/json\r\n
Content-Length: 84\r\n
\r\n
{"content":"Buy groceries","created_at":"2024-01-15T14:00:00.123456+00:00","id":42}
```

Gunicorn writes these bytes to the client socket (which leads to Nginx).  
(Gunicorn's sync workers close the connection after each response – hence "Connection: close". Keep-alive to the client is Nginx's job.)

Time: < 1 ms

## 29.10 Phase 9: Response Path (Gunicorn → Client)

**Location:** Gunicorn → Nginx → Internet → Client

Nginx receives response from Gunicorn.

Nginx adds security headers:

```
Strict-Transport-Security: max-age=63072000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
...
```

Nginx encrypts the response using the TLS session key.

Nginx writes encrypted bytes to the client's TLS socket.

Internet routing (reverse of request path):

Bytes travel from server → hosting provider → ISP → client

Client TLS stack decrypts the bytes.

Client HTTP parser receives:

```
HTTP/2 201 Created
content-type: application/json
strict-transport-security: ...
content-length: 84
```

```
{"content":"Buy groceries","created_at":"2024-01-15T14:00:00.123456+00:00","id":42}
```

curl displays the response:

```
{"content":"Buy groceries","created_at":"2024-01-15T14:00:00.123456+00:00","id":42}
```

The client's connection to Nginx remains open (HTTP keep-alive).

Subsequent requests reuse it (no new TCP or TLS handshakes).

Time: 1 RTT

## 29.11 The Complete Timeline

```

T+0ms DNS lookup (cache hit)
T+0ms TCP SYN sent
T+10ms TCP ACK received (10ms RTT assumed)
T+10ms TLS ClientHello sent
T+20ms TLS ServerHello received (another RTT)
T+20ms TLS handshake complete
T+20ms HTTP POST sent
T+20ms Nginx receives and buffers request
T+20ms Nginx forwards to Unicorn
T+20ms Flask receives request
T+21ms Validation complete
T+21ms Database connection opened (TCP + TLS + auth)
T+31ms INSERT executed
T+34ms Database response received (3ms DB round-trip), committed
T+34ms JSON serialized
T+34ms Response sent to Nginx
T+34ms Nginx adds security headers
T+34ms Nginx encrypts and sends to client
T+44ms Client receives response (10ms RTT back)

Total: ~44ms (10ms RTT, ~10ms opening the DB connection, 3ms DB, rest is processing)

For a client in the same datacenter: ~15ms total
For a client on the other side of the world: ~200ms total
(The dominant factor is network latency, not server processing)

```

## 29.12 Log Output for This Request

During this request, the following log entries were written:

Nginx access log (`/var/log/nginx/notes-api-access.log`):

```

89.100.200.50 - - [15/Jan/2024:14:00:00 +0000] "POST /notes/ HTTP/2.0" 201 84 "-"
↪ "curl/8.1.2"

```

Application log (via `journalctl` or `docker logs`):

```
{
 "timestamp": "2024-01-15T14:00:00.020Z",
 "level": "INFO",
 "logger": "app.middleware",
 "message": "→ POST /notes/ (from 89.100.200.50)",
 "function": "log_request_start",
 "line": 44
},
{
 "timestamp": "2024-01-15T14:00:00.034Z",
 "level": "INFO",
 "logger": "app.routes.notes",
 "message": "Created note id=42.",
 "function": "create_note",
 "line": 54
},
{
 "timestamp": "2024-01-15T14:00:00.034Z",
 "level": "INFO",
 "logger": "app.middleware",
 "message": "← 201 in 14.3ms",
 "function": "log_request_end",
 "line": 83
}
```

These logs, combined, tell the complete story:

- Who made the request (IP, user agent)
- What they requested (method, path)
- What happened (note created, id=42)
- How long it took (14.3ms server time, request logged by Nginx)

### Key Takeaways

- A single `POST /notes/` traverses: `DNS` → `TCP` → `TLS` → `Nginx` → `Docker` → `Gunicorn` → `Flask` → `PostgreSQL` → response path back.
- **Network latency dominates** total request time. Server processing is typically 2–10ms; network round-trip can be 10–200ms.
- **Each layer can fail independently.** DNS failure looks different from TCP failure, which looks different from TLS failure, which looks different from a 500 response.
- **The database** is the most expensive server-side cost: 1–5 ms per query over the VPC, plus the connection setup (TCP, TLS, authentication) that this app pays on every request because it opens a new connection each time — the case for connection pooling ([Chapter 36](#)).
- **HTTP keep-alive** means subsequent requests to the same server reuse the TCP and TLS connections, avoiding the handshake overhead.
- **The log entries** across Nginx and the application together provide complete visibility into what happened during a request.

## Exercises

1. **Predict.** Time a request with `curl -w "%{time_namelookup} %{time_connect} %{time_appconnect} %{time_starttransfer} %{time_total}\n" -o /dev/null -s`. Which phase takes longest from your location, and why?
2. **Break and fix.** Send two requests with `curl` in one command (`curl URL URL`), then as two separate commands, and compare their timings. What does connection reuse save?

3. **Extend.** Find the same request in the Nginx access log and the application log, and match them up by time and path. (After Chapter 35, ask what it would take to match them by request ID.)
4. **Explain.** Why does network latency usually dominate a request's total time, and what does that suggest about where to optimize first?

Solutions and hints are in the companion repository, in `exercises/chapter-29.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

The complete request journey is understood. Now examine what happens after the request is received by the kernel but before Flask sees it — the kernel-to-application handoff.

→ [Chapter 30 — Kernel to Application Handoff](#)

# Kernel to Application Handoff

---

Chapter 26 traced how packets travel from the internet to the server's socket receive buffer. Chapter 29 traced the complete request journey. This chapter zooms in on the specific moment of the **kernel-to-application handoff**: how the kernel delivers received bytes to the Gunicorn worker process, and what the worker does with those bytes before Flask sees them.

This handoff is mediated by system calls — the interface between user-space code (Python, Gunicorn, Werkzeug) and the kernel. Understanding system calls, and the overhead they impose, is foundational for understanding and optimizing web application performance.

## In this chapter

By the end of this chapter, you will understand:

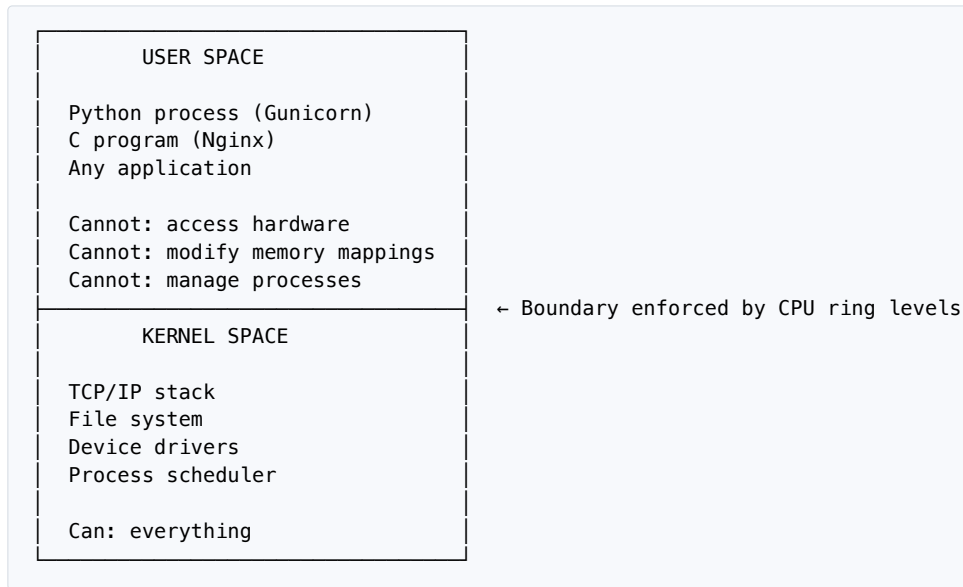
- What a system call is and why there is a boundary between user space and kernel space
- Which system calls mediate the kernel-to-application handoff (`accept`, `recv`, `read`)
- How the kernel notifies the application that data is available (`epoll`, `select`)
- What buffering occurs between the NIC and the application
- How Werkzeug converts raw bytes to the Flask request object
- What happens when the accept queue fills up (connection backlog)

## 30.1 User Space vs Kernel Space

Linux divides memory into two regions:

**Kernel space:** where the operating system kernel runs. Code in kernel space has full access to all hardware (memory, CPU registers, I/O devices). Only trusted kernel code runs here.

**User space:** where application code runs. Python, Gunicorn, Nginx, and your Flask code run here. User-space code cannot directly access hardware — it must ask the kernel.



## System calls

A **system call** is the mechanism for user-space code to request kernel services. Every time Python reads from a file, writes to a socket, creates a process, or grows its memory, it makes a system call. (Most individual allocations do not: Python’s allocator hands out memory from larger blocks it has already obtained from the kernel.)

System calls are expensive compared to regular function calls:

1. User-space code calls a system call (e.g., `read(fd, buf, count)`)
2. CPU switches to kernel mode (a mode switch — roughly 100 ns, more with some CPU-vulnerability mitigations)
3. Kernel executes the requested operation
4. CPU switches back to user mode
5. Return value is in a CPU register

The mode switch is the overhead — not the operation itself. For a web server making thousands of `read()` and `write()` calls per second, this adds up.

## 30.2 The Accept System Call

Chapter 3 showed the server socket lifecycle: `socket()` → `bind()` → `listen()` → `accept()` in a loop.



`accept()` is the system call that hands a new connection from the kernel to the application.

```
// C-level system call signature:
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);

// Returns: a new file descriptor for the connected client socket
```

In Python (via the `socket` module):

```
client_sock, client_addr = server_sock.accept()
client_sock: a new socket object; its file descriptor (e.g., 7)
is client_sock.fileno()
client_addr: a tuple ('172.17.0.1', 54832)
```

## What happens inside `accept()`

1. Gunicorn worker calls `accept(server_fd, ...)`
2. Kernel checks: is there a connection in the accept queue?
  - YES → take it off the queue, give it a file descriptor, return that fd
  - NO → block the worker (put it to sleep) until a connection arrives
3. When a new connection arrives (from the TCP three-way handshake):
  - Kernel wakes up the sleeping worker
  - `accept()` takes the connection's socket off the queue
  - and returns a new file descriptor for it

The worker is now executing with a file descriptor that represents an established TCP connection to the client.

## The accept queue

The **accept queue** (also called the completed connections queue) holds TCP connections that have completed the three-way handshake but have not yet been returned by `accept()`. The maximum size of this queue is the backlog parameter passed to `listen()`.

```
server_sock.listen(128) # Accept queue can hold up to 128 connections
```

If the accept queue fills up (because Gunicorn's workers are all busy and not calling `accept()`), the kernel stops completing new connections. By default Linux silently **drops** the new handshake packets, and clients retry — so an overflowing queue shows up as slow or timed-out con-

nections, not as errors. (Only with `net.ipv4.tcp_abort_on_overflow = 1` does the kernel answer with a reset, “connection refused”.)

This is visible with `ss`, run inside the container’s network namespace (Gunicorn’s listening socket lives there; its default backlog is 2048):

```
On the server: run ss in the container's network namespace
sudo nsenter -t $(docker inspect -f '{{.State.Pid}}' notes-api) -n ss -tln
State Recv-Q Send-Q Local Address:Port
LISTEN 3 2048 0.0.0.0:5000
#
↑ ↑
| | the queue's maximum size (the backlog)
| | connections waiting to be accept()ed right now;
| | if this reaches Send-Q, new connections are being dropped
```

### 30.3 Reading Data: `recv()` and `read()`

After `accept()` returns the client socket, Gunicorn reads the HTTP request bytes using `recv()` or `read()`:

```
Python equivalent of what Gunicorn's HTTP parser does (simplified):
raw_data = b""
while True:
 chunk = client_sock.recv(4096) # Read up to 4096 bytes
 if not chunk:
 break # Connection closed
 raw_data += chunk
 if b"\r\n\r\n" in raw_data:
 break # Found end of HTTP headers
Then: parse Content-Length from the headers, and keep reading until
that many bytes of body have arrived.
```

### Blocking vs non-blocking sockets

By default, sockets are **blocking**: `recv()` blocks the calling thread/process until data is available. While blocked, the process is sleeping — using no CPU.

**Non-blocking** sockets return immediately with an `EAGAIN` error if no data is available. The application must then check back later (via `select/epoll`). Non-blocking I/O allows a single process to manage many connections simultaneously — the basis of async frameworks.

Gunicorn with sync workers (the default) uses blocking sockets. Each worker handles one connection at a time. While waiting for data, the worker is blocked and unavailable for other

requests. This is why multiple workers are needed for concurrent requests — and why Nginx’s request buffering ([Chapter 26](#), section 26.6) matters: Nginx waits for a slow client, so a worker never has to.

## epoll: efficient I/O notification

The kernel provides a mechanism for efficiently monitoring many file descriptors for I/O readiness: **epoll** (Linux) or **kqueue** (macOS/BSD).

```
Conceptual Python illustration of epoll usage (Linux only)
import select

server_sock.setblocking(False) # never let accept() block the loop

Create an epoll object
ep = select.epoll()

Register the server socket for incoming connections
ep.register(server_sock.fileno(), select.EPOLLIN)

Keep every client socket object alive, keyed by its fd. (If only the fd
number were kept, Python would garbage-collect the socket object and
close the connection behind the loop's back.)
clients = {}

Event loop
while True:
 events = ep.poll(timeout=1.0) # Wait for up to 1 second
 for fd, event in events:
 if fd == server_sock.fileno():
 # New connection available
 client_sock, addr = server_sock.accept()
 client_sock.setblocking(False)
 clients[client_sock.fileno()] = client_sock
 ep.register(client_sock.fileno(), select.EPOLLIN)
 elif event & select.EPOLLIN:
 # Data available on a client socket
 data = clients[fd].recv(4096)
 if not data: # client closed the connection
 ep.unregister(fd)
 clients.pop(fd).close()
 # ... otherwise handle data ...
```

Nginx uses `epoll` internally to handle thousands of concurrent connections with a small num-

ber of worker processes. Gunicorn’s `gevent` or `eventlet` workers also use `epoll` for async request handling.

## 30.4 From Bytes to Request Object

Gunicorn reads raw bytes from the socket. These bytes are a raw HTTP/1.1 message:

```
POST /notes/ HTTP/1.1\r\n
Host: api.notes.example.com\r\n
Content-Type: application/json\r\n
Content-Length: 28\r\n
\r\n
{"content": "Buy groceries"}
```

In production, **Gunicorn** parses these bytes — its HTTP parser is Python code, and the first Python code that runs for each request. It turns the request into the structure the WSGI standard defines: the `environ` dictionary. (Werkzeug, Flask’s underlying HTTP library, has its own parser too, but it is only used by the development server, `python run.py`.)

### The `environ` dictionary

```
What Gunicorn builds from the request (simplified):
from io import BytesIO

environ = {
 # Server information (the address the request arrived at, inside
 # the container)
 "SERVER_NAME": "0.0.0.0",
 "SERVER_PORT": "5000",
 "SERVER_PROTOCOL": "HTTP/1.1",

 # Request information
 "REQUEST_METHOD": "POST",
 "PATH_INFO": "/notes/",
 "QUERY_STRING": "",

 # Headers (HTTP_ prefix, uppercase, hyphens → underscores) – except
 # Content-Type and Content-Length, which PEP 3333 gives without the prefix
 "HTTP_HOST": "api.notes.example.com",
 "CONTENT_TYPE": "application/json",
 "CONTENT_LENGTH": "28",
```

```

 # Proxy headers
 "HTTP_X_REAL_IP": "89.100.200.50",
 "HTTP_X_FORWARDED_FOR": "89.100.200.50",
 "HTTP_X_FORWARDED_PROTO": "https",

 # Body
 "wsgi.input": BytesIO(b'{"content": "Buy groceries"}'),

 # WSGI specifics
 "wsgi.url_scheme": "http", # Inside container, Nginx handles HTTPS
}

```

## Flask's Request object

Flask (using Werkzeug's classes) wraps the `environ` dict in a `Request` object that provides a clean Python interface:

```

from flask import Request

request = Request(environ)

Accessing request data:
request.method # "POST"
request.path # "/notes/"
request.headers # Headers dict-like object
request.headers.get("Content-Type") # "application/json"
request.get_json() # {"content": "Buy groceries"} (parsed JSON)
request.remote_addr # "89.100.200.50" (after ProxyFix)

```

## The WSGI call

Flask's app object is callable. Gunicorn calls it with the `environ` dict:

```

What Gunicorn does (simplified):
def handle_request(env, client_socket):
 response_started = False
 status_code = None
 headers = None

 def start_response(status, response_headers, exc_info=None):
 nonlocal response_started, status_code, headers
 response_started = True
 status_code = status
 headers = response_headers

 # Call the Flask app – this runs all of Flask's request handling
 response_body_iterable = flask_app(env, start_response)

 # Send the response back to the client
 write_http_response(client_socket, status_code, headers, response_body_iterable)

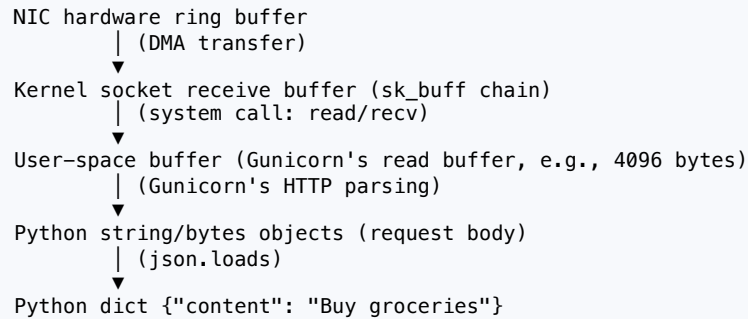
```

Flask's `__call__` method:

1. Creates the Request object from `environ`
2. Pushes a request context (makes `request`, `g` available)
3. Runs before-request hooks
4. Routes the request to the correct handler
5. Runs after-request hooks (which may still change the response)
6. Calls `start_response` with the status and headers
7. Returns the response body as an iterable
8. Runs teardown hooks when the request context is popped

## 30.5 The Buffer Chain

Data travels through multiple buffers between the NIC and the application:



At each stage, there is potential for copying. The kernel tries to minimize copies (using zero-copy techniques in some paths), but Python code always works with its own copies of data.

## Buffer sizing

```

Gunicorn reads in chunks:
CHUNK_SIZE = 4096 # bytes per read() call

If the HTTP request body is larger than 4096 bytes,
multiple read() calls are needed.

For a 100-byte JSON body: 1 read() call
For a 10KB JSON body: 3 read() calls
For a 1MB upload: 256 read() calls

```

This is one more reason for `client_max_body_size` in Nginx. The main one is protection — an oversized body is rejected before it can consume memory or disk, or reach the application at all — but it also bounds the work of reading it.

## 30.6 System Call Tracing

You can observe the actual system calls a running process makes using `strace`:

```
Find a Gunicorn worker's PID *on the host*. (Inside the container the
PIDs are different – and the slim image has no ps command anyway.)
docker top notes-api
UID PID PPID ... CMD
1001 23101 23080 gunicorn ... ← master
1001 23130 23101 gunicorn ... ← worker

Trace network system calls on that worker (install with: apt-get install strace)
sudo strace -p 23130 -e trace=network 2>&1 | head -30
```

While making a request to the API, you would see something like this (abbreviated; with two workers, the request may go to the other one):

```
accept4(5, {sa_family=AF_INET, sin_port=htons(54832),
→ sin_addr=inet_addr("172.17.0.1")}, [16], SOCK_CLOEXEC) = 7
recvfrom(7, "POST /notes/ HTTP/1.1\r\nHost: api"... , 8192, 0, NULL, NULL) = 255
sendto(7, "HTTP/1.1 201 Created\r\nServer: gu"... , 176, 0, NULL, 0) = 176
sendto(7, "{\"content\":\"Buy groceries\",\"cre\""... , 84, 0, NULL, 0) = 84
close(7) = 0
```

Each line is a system call:

- `accept4(5, ...)` → accepts connection on server socket `fd=5`, returns client socket `fd=7`
- `recvfrom(7, ...)` → reads the request from the client socket: 227 bytes of headers and the 28-byte body arrived together, 255 bytes in one call
- `sendto(7, ...)` → sends the response: first the 176 bytes of status line and headers, then the 84-byte JSON body
- `close(7)` → closes the client connection

### Key Takeaways

- **User space and kernel space** are separate memory regions. System calls are the controlled bridge between them. Every `read()`, `write()`, `accept()` is a system call with a mode-switch overhead (~100ns).
- **`accept()`** retrieves a connection from the kernel's accept queue and returns a new file descriptor for that connection. If the queue is empty, the calling process blocks.
- **The accept queue** holds fully-connected TCP sessions waiting to be `accept()`ed. If it fills (workers too busy), Linux silently drops new connection attempts — clients see slowness and timeouts.
- **Blocking sockets** (Gunicorn's default) block the worker process while waiting for data.



Multiple workers provide concurrent request handling.

- **epoll** (used by Nginx and async Gunicorn workers) efficiently monitors many file descriptors for readiness, enabling one process to handle many concurrent connections.
- **Gunicorn** converts raw HTTP bytes into an `environ` dict, which Flask wraps in a `Request` object. Gunicorn's parsing is the first Python code that executes on each request.
- **Buffers exist at every layer**: NIC ring buffer, kernel socket buffer, Gunicorn read buffer, Python objects. Each layer adds potential for copying and potential for bottlenecks.

## Exercises

1. **Predict.** Run `strace -f -e trace=accept4,recvfrom,sendto -p <gunicorn worker pid>` and send one request. Which system calls do you see, and in what order?
2. **Break and fix.** Set Gunicorn's backlog to 1 (`--backlog 1`) and send 50 concurrent requests with `xargs -P 50`. What happens to the extra connections? Restore the default.
3. **Extend.** Write a Python program that uses `select.select` to serve two clients at once from one thread, and compare it with a version that blocks on `recv`.
4. **Explain.** What is a system call, and why does the number of system calls per request matter for performance?

Solutions and hints are in the companion repository, in `exercises/chapter-30.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

The handoff from kernel to application is understood. The application now has a `Request` object. The next chapter examines how Flask processes the input: validates it, sanitizes it, and prepares it for the business logic.

→ [Chapter 31](#) — Input Handling

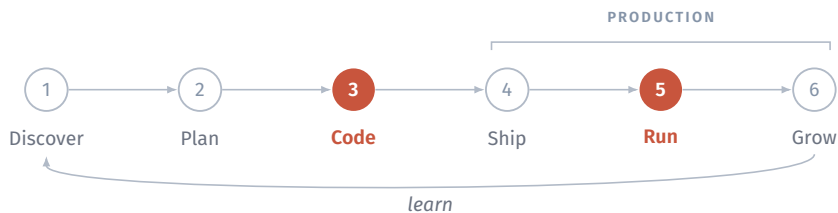


# 9

## Inside the Running Program

*What does the program do with that input?*

YOU ARE HERE



**The piece:** The program and its data, under real traffic

The whole map: *How Software Gets Built*, page 1



# Input Handling

---

Every piece of data that enters an application from the outside world is potentially malicious, malformed, or unexpected. A well-designed application treats all external input as untrusted until explicitly validated.

**Input handling** is the discipline of receiving, parsing, validating, and sanitizing data that arrives via HTTP requests before it touches any business logic or database. Done well, it catches errors early (returning clear error messages to the client), prevents security vulnerabilities (injection attacks, resource exhaustion), and makes the rest of the code simpler (because downstream code can trust the data it receives).

The Notes API already has basic input handling in the route handlers (checking that content exists, is a string, is not empty, is not too long). This chapter examines input handling in depth: what categories of validation exist, how to structure validation code, how to test it, and what happens when validation fails.

## In this chapter

By the end of this chapter, you will understand:

- Why all external input must be treated as untrusted
- The four categories of input validation: presence, type, format, and business rules
- How to structure validation to return all errors at once (not just the first)
- How to safely parse JSON, URL parameters, query strings, and headers
- What SQL injection is and why parameterized queries prevent it
- What cross-site scripting (XSS) is and why it does not directly apply to a JSON API
- How to write comprehensive validation tests

## 31.1 Why Input Is Untrusted

Every HTTP request comes from somewhere outside your application. The client might be:

- A well-behaved user using your frontend
- A developer testing with curl
- A bug in a client application that sends wrong data
- An attacker probing for vulnerabilities

Your application cannot know which. It must handle all of them correctly.

## The threat model

```
Your application receives:
data = request.get_json(silent=True)

What could 'data' be?
None # No body, or invalid JSON
[1, 2, 3] # Valid JSON – but not an object
"hello content" # Valid JSON – a bare string
{} # Empty object
{"content": ""} # Empty content
{"content": None} # Wrong type
{"content": 42} # Wrong type
{"content": "A" * 100_000} # Extremely long (DoS attempt)
{"content": "'; DROP TABLE notes; --"} # SQL injection attempt
{"content": "<script>alert(1)</script>"} # XSS attempt
{"content": "\x00\x01\x02"} # Binary data / null bytes
{"content": "\ud800"} # A lone surrogate: valid JSON, but not
 # encodable as UTF-8 (no driver can send it)
{"extra_key": "value"} # Unexpected fields
```

An application that does not validate input will fail or be exploited by one of these inputs.

## 31.2 The Four Categories of Validation

### Category 1: Presence

Is the required data there at all?

```

Check: is the JSON body present – and is it an object?
data = request.get_json(silent=True)
if not isinstance(data, dict):
 return jsonify({"error": "Request body must be a JSON object"}), 400

Check: is the required field present?
if "content" not in data:
 return jsonify({"error": "'content' is required"}), 400

```

The `isinstance(data, dict)` check matters more than it looks. `null`, `[1, 2]`, and `"hello content"` are all valid JSON. Without the check, `"content" not in data` would do a *substring* test on the string `"hello content"` — and pass — and the next line, `data.get(...)`, would crash with an `AttributeError`, which the handling boundary turns into a 500.

## Category 2: Type

Is the data the expected Python type?

```

content = data.get("content")

Is content a string?
if not isinstance(content, str):
 return jsonify({"error": "'content' must be a string"}), 400

```

## Category 3: Format

Is the data in the expected format (within the string type)?

```

content = content.strip()

Is it non-empty after stripping whitespace?
if not content:
 return jsonify({"error": "'content' cannot be empty"}), 400

Is it within the length limit?
if len(content) > 10_000:
 return jsonify({"error": "'content' cannot exceed 10,000 characters"}), 400

Is it encodable as UTF-8? Python strings are Unicode, but JSON can carry
lone surrogates ("\ud800"), which cannot be encoded as UTF-8 for any database.
try:
 content.encode("utf-8")

```

```

except UnicodeEncodeError:
 return jsonify({"error": "'content' contains invalid characters"}), 400

Does it contain control characters (other than newline, tab, return)?
import unicodedata
if any(unicodedata.category(c) == "Cc" and c not in "\n\t\r"
 for c in content):
 return jsonify({"error": "'content' contains invalid control characters"}), 400

```

### Category 4: Business rules

Does the data satisfy application-specific rules that cannot be expressed as simple type or format checks?

```

Business rule: a note with identical content to a recent note might
indicate an accidental double-submit.
(This is a hypothetical example – not in the current Notes API.
PostgreSQL syntax.)

def check_duplicate(content: str, db) -> bool:
 """Check if an identical note was created in the last 5 seconds."""
 cursor = db.cursor()
 cursor.execute("""
 SELECT COUNT(*) FROM notes
 WHERE content = %s
 AND created_at > NOW() - INTERVAL '5 seconds'
 """, (content,))
 count = cursor.fetchone()[0]
 cursor.close()
 return count > 0

```

## 31.3 A Reusable Validation Framework

As the API grows, repeating the same validation logic in every handler becomes error-prone. A validation class collects errors and reports them all at once:



**Listing 31.1** app/validation.py

```

"""
Input validation utilities for the Notes API.

Usage:
 v = Validator(request.get_json(silent=True))
 content = v.require_string("content", min_length=1, max_length=10_000)
 if v.has_errors():
 return v.error_response()

Error responses keep the API's one error shape (Chapter 16):
 one problem: {"error": "'content' is required"}
 several problems: {"error": "Invalid request",
 "details": ["'content' is required", "..."]}
"""
from typing import Any, Optional

from flask import jsonify

class Validator:
 """
 Accumulates validation errors, returning all at once.

 This is better than raising on the first error – the client can see
 all problems and fix them in one request instead of submitting,
 failing, fixing one thing, submitting, failing, etc.
 """

 def __init__(self, data: Any):
 self.data = data
 self.errors: list[str] = []
 self._body_checked = False

 def has_errors(self) -> bool:
 """Return True if any validation errors have been accumulated."""
 return len(self.errors) > 0

 def error_response(self):
 """The 400 response for the accumulated errors, in the API's error shape."""
 if len(self.errors) == 1:
 return jsonify({"error": self.errors[0]}), 400
 return jsonify({"error": "Invalid request", "details": self.errors}), 400

```

```

def _add_error(self, message: str) -> None:
 self.errors.append(message)

def require_dict(self) -> Optional[dict]:
 """
 Verify that the input data is a dictionary (JSON object).
 Returns the dict if valid, None if not. Reports the problem only
 once, however many field checks call it.
 """
 if not isinstance(self.data, dict):
 if not self._body_checked:
 self._add_error("Request body must be a JSON object")
 self._body_checked = True
 return None
 self._body_checked = True
 return self.data

def require_string(
 self,
 field: str,
 *,
 min_length: int = 1,
 max_length: int = 10_000,
 strip: bool = True,
) -> Optional[str]:
 """
 Extract and validate a required string field.

 Arguments:
 field: The field name to look up in self.data
 min_length: Minimum acceptable length (after stripping, if strip=True)
 max_length: Maximum acceptable length
 strip: If True, strip leading/trailing whitespace before validation

 Returns the validated string, or None if validation failed.
 """
 # A field check on a body that is not an object is an error too –
 # never a silent "no value"
 if self.require_dict() is None:
 return None

 if field not in self.data:
 self._add_error(f"'{field}' is required")
 return None

```

```

value = self.data[field]

if not isinstance(value, str):
 self._add_error(f"'{field}' must be a string")
 return None

if strip:
 value = value.strip()

if len(value) < min_length:
 if min_length == 1:
 self._add_error(f"'{field}' cannot be empty")
 else:
 self._add_error(
 f"'{field}' must be at least {min_length:,} characters long"
)
 return None

if len(value) > max_length:
 self._add_error(f"'{field}' cannot exceed {max_length:,} characters")
 return None

Characters that cannot be stored: NUL, which PostgreSQL rejects
(Chapter 17, section 17.8), and lone surrogates such as "\ud800",
which JSON can carry but UTF-8 cannot encode, for any database
if "\x00" in value:
 self._add_error(f"'{field}' contains invalid characters")
 return None

try:
 value.encode("utf-8")
except UnicodeEncodeError:
 self._add_error(f"'{field}' contains invalid characters")
 return None

return value

def require_int(
 self,
 field: str,
 *,
 min_value: Optional[int] = None,
 max_value: Optional[int] = None,
) -> Optional[int]:

```

```

"""
Extract and validate a required integer field.
"""
if self.require_dict() is None:
 return None

if field not in self.data:
 self._add_error(f"'{field}' is required")
 return None

value = self.data[field]

bool is a subclass of int in Python: reject True/False explicitly
if not isinstance(value, int) or isinstance(value, bool):
 self._add_error(f"'{field}' must be an integer")
 return None

if min_value is not None and value < min_value:
 self._add_error(f"'{field}' must be at least {min_value}")
 return None

if max_value is not None and value > max_value:
 self._add_error(f"'{field}' must be at most {max_value}")
 return None

return value

def optional_string(
 self,
 field: str,
 *,
 max_length: int = 1000,
 default: Optional[str] = None,
) -> Optional[str]:
 """
 Extract and validate an optional string field.
 Returns the value if present and valid, default if not present,
 or None if present but invalid (with error added).
 """
 if self.require_dict() is None:
 return default

 if field not in self.data:
 return default

```

```
return self.require_string(field, min_length=0, max_length=max_length)
```

The Validator is less strict than the format checks shown earlier. It rejects only the characters that cannot be stored at all, and accepts other control characters such as `\x01`. A note is free text, and rejecting everything unusual turns away real input. Where a field must never contain control characters (a tag, an email address), its own pattern rules them out.

## Using the Validator in route handlers

### Listing 31.2 Updated `create_note` with Validator

```
from app.validation import Validator

@notes_bp.route("/", methods=["POST"])
def create_note():
 v = Validator(request.get_json(silent=True))

 # Validate the body (it must be a JSON object) and the content field
 content = v.require_string("content", min_length=1, max_length=10_000)
 if v.has_errors():
 logger.warning("POST /notes/ rejected: %s", "; ".join(v.errors))
 return v.error_response()

 # content is now a clean, validated string – proceed with database
 db = get_db()
 # ...
```

The error messages are the same ones the handler produced before, so clients see no change:

```
Sending the wrong data type:
curl -X POST http://localhost:5000/notes/ -H "Content-Type: application/json" \
 -d '{"content": 42}'
{"error":"'content' must be a string"}

Sending an empty object:
curl -X POST http://localhost:5000/notes/ -H "Content-Type: application/json" \
 -d '{}'
```

With a single field, a request has at most one problem. The accumulation pays off as soon as an endpoint validates several fields: a hypothetical endpoint that also required an integer

priority would answer {} with every problem at once:

```
{
 "error": "Invalid request",
 "details": ["'content' is required", "'priority' is required"]}

```

## 31.4 URL Parameters and Query Strings

### URL path parameters

Flask's type converters handle basic URL parameter parsing:

```
@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id):
 # note_id is already an int – Flask's <int:> converter
 # returned 404 if the URL contained a non-integer.
 # No additional type check needed.
 pass

```

But the converter only verifies that the value is an integer. Application-level validation (e.g., `note_id` must be positive) is still the application's responsibility:

```
@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id):
 if note_id <= 0:
 return jsonify({"error": "Note ID must be a positive integer"}), 400
 # ...

```

(Flask's `int` converter never matches a negative number — `/notes/-1` is already a 404 — so the only value this check can catch is 0.)

### Query string parameters

Query string parameters come from the URL after `?`:

```
GET /notes/?limit=10&offset=20&sort=created_at

```

Flask provides these via `request.args`:

```
@notes_bp.route("/", methods=["GET"])
def list_notes():

```

```

request.args is an ImmutableMultiDict (like a dict but allows multiple values)

Parse 'limit' with a default of 50
try:
 limit = int(request.args.get("limit", "50"))
except ValueError:
 return jsonify({"error": "'limit' must be an integer"}), 400

if not (1 <= limit <= 100):
 return jsonify({"error": "'limit' must be between 1 and 100"}), 400

Parse 'offset' with a default of 0
try:
 offset = int(request.args.get("offset", "0"))
except ValueError:
 return jsonify({"error": "'offset' must be an integer"}), 400

if offset < 0:
 return jsonify({"error": "'offset' cannot be negative"}), 400

Sort field (whitelist approach – only allow known field names)
sort_field = request.args.get("sort", "id")
ALLOWED_SORT_FIELDS = {"id", "created_at"}
if sort_field not in ALLOWED_SORT_FIELDS:
 return jsonify({
 "error": f"'sort' must be one of: {'',
 ↪ '.join(sorted(ALLOWED_SORT_FIELDS))}"
 }), 400

Now safe to use limit, offset, sort_field in a query
db = get_db()
cursor = db.cursor()

Using a whitelist for the sort field prevents SQL injection
(we cannot use a parameterized placeholder for field names)
query = f"""
 SELECT id, content, created_at
 FROM notes
 ORDER BY {sort_field} DESC
 LIMIT ? OFFSET ?
"""
cursor.execute(sql(query), (limit, offset))
notes = [row_to_note(row) for row in cursor.fetchall()]
...

```

(An `int()` of user input can fail, which is why each one sits in a `try`. Flask also offers `request.args.get("limit", 50, type=int)`, but that silently falls back to the default on bad input instead of telling the client — for an API, the explicit 400 is kinder.)

## 31.5 SQL Injection and Parameterized Queries

**SQL injection** is one of the most common and dangerous web vulnerabilities. It occurs when user-supplied input becomes part of the SQL *text* of a query, where the database parses it as code.

### The attack

Suppose the code builds a SQL query by string concatenation:

```
DANGEROUS — DO NOT DO THIS
note_id = request.args.get("id")
query = f"SELECT * FROM notes WHERE id = {note_id}"
cursor.execute(query)
```

An attacker sends: GET /notes/?id=1 OR 1=1

The query becomes:

```
SELECT * FROM notes WHERE id = 1 OR 1=1
```

This returns ALL notes, not just note 1. Worse, the attacker can read data from *other* tables:

Attacker sends: GET /notes/?id=0 UNION SELECT id, email, password\_hash FROM users

```
SELECT * FROM notes WHERE id = 0 UNION SELECT id, email, password_hash FROM users
```

The response now lists user accounts instead of notes. With a driver that runs several statements in one `execute()` call — `psycopg2` does; Python's `sqlite3` refuses — the attacker can even append a second statement: `1; DROP TABLE notes; --`.

### The prevention: parameterized queries

Always use parameterized queries (also called prepared statements). The database driver separates the SQL structure from the data:



```
SAFE – the database driver handles escaping
note_id = int(request.args.get("id", 0))
cursor.execute(
 "SELECT * FROM notes WHERE id = %s",
 (note_id,) # The value is passed separately from the SQL
)
```

With parameterized queries:

- The value travels separately from the SQL text. (`sqlite3` really does prepare the statement first and bind the value; `psycopg2` inserts the value into the query itself on the client side, but always as a correctly quoted literal.)
- Either way, the value is never parsed as SQL
- `'; DROP TABLE notes; --` is treated as a literal string value, not SQL

The Notes API already uses parameterized queries (written with `?`; `sql()` converts them to `psycopg2`'s `%s`). This is non-negotiable — never build SQL strings by concatenation with user input. The one thing parameters cannot carry is an *identifier*, such as a column name in `ORDER BY`; for those, use an allow-list, as the sort field in section 31.4 does.

## 31.6 Input Handling for Headers

Request headers can also contain malicious content. Be careful when using header values:

```
Reading a custom header
api_key = request.headers.get("X-API-Key", "")

Never use a header value directly in SQL
This would be SQL injection:
cursor.execute(f"SELECT user FROM api_keys WHERE key = '{api_key}'")

Correct (a hypothetical api_keys table; PostgreSQL placeholder):
cursor.execute("SELECT user FROM api_keys WHERE key = %s", (api_key,))
```

Headers used in logging need care too:

```
A User-Agent header containing log injection attempts:
User-Agent: normal browser\nERROR: database compromised

Sanitize for logging:
user_agent = request.headers.get("User-Agent", "")[:200] # Limit length
user_agent = user_agent.replace("\n", " ").replace("\r", " ") # Remove newlines
logger.info("Request from: %s", user_agent)
```

The JSON log format from Chapter 16 is already safe here — `json.dumps` escapes a newline as `\n`, so it cannot start a fake log line. The human-readable format is the vulnerable one.

## 31.7 Testing Input Validation

Input validation must be thoroughly tested. Chapter 17 wrote tests for the happy path (valid input). Tests must also cover all invalid input cases.

### Listing 31.3 Additional validation tests for `tests/test_notes.py`

```
class TestCreateNoteValidation:
 """Comprehensive validation tests for POST /notes/"""

 # Presence checks
 def test_no_body_returns_415(self, client):
 # No Content-Type, no body: rejected by the middleware (Chapter 16)
 response = client.post("/notes/")
 assert response.status_code == 415

 def test_empty_body_returns_400(self, client):
 response = client.post(
 "/notes/", data="", content_type="application/json"
)
 assert response.status_code == 400

 def test_plain_text_body_returns_415(self, client):
 response = client.post("/notes/", data="hello", content_type="text/plain")
 assert response.status_code == 415

 def test_array_body_returns_400(self, client):
 """JSON arrays are not accepted – must be an object."""
 response = client.post("/notes/", json=["content"])
 assert response.status_code == 400
```

```

Type checks
def test_content_as_int_returns_400(self, client):
 response = client.post("/notes/", json={"content": 42})
 assert response.status_code == 400
 assert response.get_json() == {"error": "'content' must be a string"}

def test_content_as_bool_returns_400(self, client):
 response = client.post("/notes/", json={"content": True})
 assert response.status_code == 400

def test_content_as_null_returns_400(self, client):
 response = client.post("/notes/", json={"content": None})
 assert response.status_code == 400

def test_content_as_list_returns_400(self, client):
 response = client.post("/notes/", json={"content": ["a", "b"]})
 assert response.status_code == 400

Format checks
def test_empty_string_content_returns_400(self, client):
 response = client.post("/notes/", json={"content": ""})
 assert response.status_code == 400

def test_whitespace_only_content_returns_400(self, client):
 for whitespace in [" ", "\t", "\n", " \n "]:
 response = client.post("/notes/", json={"content": whitespace})
 assert response.status_code == 400, \
 f"Expected 400 for content={repr(whitespace)}"

def test_exactly_10000_chars_is_accepted(self, client):
 response = client.post("/notes/", json={"content": "x" * 10_000})
 assert response.status_code == 201

def test_10001_chars_returns_400(self, client):
 response = client.post("/notes/", json={"content": "x" * 10_001})
 assert response.status_code == 400

def test_characters_that_cannot_be_stored_return_400(self, client):
 for bad in ["a\x00b", "a\ud800b"]: # NUL; a lone surrogate
 response = client.post("/notes/", json={"content": bad})
 assert response.status_code == 400, f"Expected 400 for {bad!r}"
 assert response.get_json() == {
 "error": "'content' contains invalid characters"

```

```

 }

 # Valid content
 def test_unicode_content_is_accepted(self, client):
 """Unicode characters (emoji, non-ASCII) are valid."""
 response = client.post("/notes/", json={"content": "Hello 你好 🍌"})
 assert response.status_code == 201
 data = response.get_json()
 assert data["content"] == "Hello 你好 🍌"

 def test_multiline_content_is_accepted(self, client):
 """Content can span multiple lines."""
 response = client.post("/notes/", json={"content": "Line 1\nLine 2\nLine 3"})
 assert response.status_code == 201

 def test_special_characters_in_content(self, client):
 """Special characters that might confuse SQL or HTML are handled safely."""
 content = "It's a 'test' with \"quotes\" & <special> chars"
 response = client.post("/notes/", json={"content": content})
 assert response.status_code == 201
 data = response.get_json()
 assert data["content"] == content.strip()

 # Error response format
 def test_error_response_is_json(self, client):
 """Error responses must be JSON."""
 response = client.post("/notes/", json={})
 assert response.status_code == 400
 data = response.get_json()
 assert data is not None
 # The API's one error shape (Chapter 16)
 assert "error" in data

```

### Key Takeaways

- **All external input is untrusted.** Validate every field: presence, type, format, and business rules. Do not assume the client sends what you expect.
- **The four categories of validation:** presence (is it there?), type (is it the right Python type?), format (does it match the expected pattern?), business rules (does it satisfy application-specific constraints?).
- **Return all errors at once,** not just the first. The `Validator` class accumulates errors and reports them together — in the API's one error shape, `{"error": ...}` plus a `details` list

when there are several.

- **Parameterized queries** are the only safe way to put user-supplied *values* into SQL. Never build SQL strings with string concatenation or f-strings containing user data; for identifiers (such as a sort column), use an allow-list.
- **Whitelist, don't blacklist.** For sort fields, allowed values, and other finite sets, accept only known-good values. Reject anything not on the list.
- **Sanitize inputs used in logging.** Remove or escape newlines to prevent log injection attacks.
- **Test all invalid input cases**, not just valid input. A good validation test suite covers every error path.

## Exercises

1. **Predict.** Send `{"content": "x", "content": "y"}` (a duplicated key) to `POST /notes/`. Which value is stored, and is that a validation problem?
2. **Break and fix.** Write a query with an f-string, `f"SELECT * FROM notes WHERE id = {note_id}"`, in a test-only route, and exploit it with `curl -G --data-urlencode 'id=0 OR 1=1' http://localhost:5000/...`. Then fix it with a parameter.
3. **Extend.** Replace section 31.4's sort parameter with one that accepts only newest or oldest, validated against an allow-list, with a `400` for anything else.
4. **Explain.** Why does the `Validator` return all errors at once, and when would returning only the first be acceptable?

Solutions and hints are in the companion repository, in `exercises/chapter-31.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

Input has been received, parsed, and validated. The next step is routing: how Flask matches the incoming URL and HTTP method to the correct handler function.

→ [Chapter 32 — Routing](#)



# Routing

Chapter 31 validated the incoming request. Before validation can even begin, however, Flask must determine *which handler function* the request belongs to. That determination is **routing**: the process of matching a URL and HTTP method to a Python function.

Flask's routing system is provided by **Werkzeug**, the library that Flask is built on top of. Werkzeug's routing is sophisticated — it handles URL patterns with variables, HTTP method constraints, custom type converters, and detailed error responses — but it is also fully transparent once you know where to look.

## In this chapter

By the end of this chapter, you will understand:

- How `@app.route()` registers a route at import time
- What the Werkzeug Map and Rule objects are and what they contain
- How URL matching works: the algorithm, the priority rules, the edge cases
- What URL variable converters are and how to define custom ones
- How 404 (Not Found) and 405 (Method Not Allowed) responses are generated
- How Blueprint routes integrate into the application URL map
- How to inspect the registered routes in a running application

## 32.1 Registration: What `@app.route()` Does

When you write:

```
@app.route("/notes/", methods=["GET"])
def list_notes():
 ...
```

this is Python decorator syntax. It is exactly equivalent to:

```
def list_notes():
 ...

app.route("/notes/", methods=["GET"])(list_notes)
```

Which is exactly equivalent to:

```
def list_notes():
 ...

app.add_url_rule("/notes/", endpoint="list_notes", view_func=list_notes,
↳ methods=["GET"])
```

`add_url_rule` is the actual method that does the work. The decorator is syntactic sugar.

### When does registration happen?

Route registration happens at **import time** — when the module containing the `@app.route()` decorator is imported by Python. It does *not* happen when a request arrives.

Timeline (production: gunicorn "app:create\_app()"; in development, python run.py does the same through run.py):

```
Gunicorn worker imports the app package (app/__init__.py)
↓
Gunicorn calls create_app()
↓
create_app() imports and registers the notes Blueprint
↓
Python imports app/routes/notes.py
↓
@notes_bp.route("/") decorator executes → Rule added to Blueprint
@notes_bp.route("/<int:note_id>") decorator executes → Rule added
↓
create_app() calls app.register_blueprint(notes_bp)
↓
Blueprint's rules are copied into app.url_map
↓
URL map is complete. Ready to match requests.
```

---



First request arrives → the URL map is matched → handler found quickly

The URL map is built once at startup and then used for every subsequent request. This is intentional — routing lookup must be fast, and building the map only once makes it fast.

## What `add_url_rule` does internally

```
Flask's add_url_rule (simplified to show the concepts)
def add_url_rule(self, rule, endpoint=None, view_func=None, methods=None, **options):
 # 1. Determine the endpoint name
 if endpoint is None:
 endpoint = view_func.__name__ # "list_notes"

 # 2. Determine which HTTP methods this rule handles
 if methods is None:
 methods = getattr(view_func, "methods", None) or ("GET",)
 methods = {m.upper() for m in methods} # Normalize to uppercase

 # 3. Add HEAD wherever GET is allowed (Werkzeug's Rule does this:
 # a HEAD request is a GET without the body)
 if "GET" in methods:
 methods.add("HEAD")

 # 4. Add OPTIONS (Flask answers it automatically, unless the view
 # sets provide_automatic_options = False)
 methods.add("OPTIONS")

 # 5. Create a Werkzeug Rule object
 rule_obj = self.url_rule_class(
 rule, # The URL pattern: "/notes/"
 methods=methods, # {"GET", "HEAD", "OPTIONS"}
 endpoint=endpoint,
)

 # 6. Add the Rule to the URL Map
 self.url_map.add(rule_obj)

 # 7. Register the view function for this endpoint
 self.view_functions[endpoint] = view_func
```

After registration, Flask has:

1. A `url_map` containing Rule objects (URL patterns)

2. A `view_functions` dict mapping endpoint names to callable functions

When a request arrives, Flask uses `url_map` to find the endpoint name, then uses `view_functions` to find the callable.

## 32.2 The URL Map and Rule Objects

### Werkzeug's Map

The Map is a collection of Rule objects. You can inspect it directly:

```
Run this in a Flask shell: flask shell
from app import create_app
app = create_app()

Print all registered rules
for rule in app.url_map.iter_rules():
 print(f"{rule.endpoint:30s} {' '.join(sorted(rule.methods)):30s} {rule.rule}")
```

For the Notes API as it stands after [Chapter 16](#), this produces (with the column widths narrowed to fit the page):

<code>static</code>	<code>GET,HEAD,OPTIONS</code>	<code>/static/&lt;path:filename&gt;</code>
<code>health.health</code>	<code>GET,HEAD,OPTIONS</code>	<code>/health</code>
<code>notes.list_notes</code>	<code>GET,HEAD,OPTIONS</code>	<code>/notes/</code>
<code>notes.create_note</code>	<code>OPTIONS,POST</code>	<code>/notes/</code>
<code>notes.get_note</code>	<code>GET,HEAD,OPTIONS</code>	<code>/notes/&lt;int:note_id&gt;</code>
<code>notes.delete_note</code>	<code>DELETE,OPTIONS</code>	<code>/notes/&lt;int:note_id&gt;</code>

Several things are apparent:

- Each route has an **endpoint** name (e.g., `notes.list_notes` — the Blueprint name prefixed)
- Methods are automatically extended with `OPTIONS`, and with `HEAD` wherever `GET` is allowed
- Flask registers a `static` route automatically for serving static files
- Multiple rules can share the same URL pattern but differ by HTTP method

### Werkzeug's Rule

A Rule is the unit of URL matching. Each Rule contains:

```
rule.rule # The URL pattern string: "/notes/<int:note_id>"
rule.methods # The set of HTTP methods: {"GET", "HEAD", "OPTIONS"}
rule.endpoint # The endpoint name: "notes.get_note"
```

Conceptually, each Rule is a pattern that a path either matches or not, and the `<int:note_id>` part is a **converter** — it specifies both the pattern for that segment (the regular expression `\d+`) and the Python type to convert the matched string to (`int`):

```
"/notes/<int:note_id>"
 ↓ conceptually
^/notes/(?P<note_id>\d+)$
```

(Older Werkzeug versions compiled each rule to one regular expression like this and tried the rules in turn. Since Werkzeug 2.2, the Map compiles all rules together into a **state machine** that walks the path one segment at a time — `notes`, then `42` — so matching stays fast however many routes there are. The outcome is the same; the trace below shows the conceptual version.)

## 32.3 URL Matching: How a Request Finds Its Handler

When a request arrives, Flask calls:

```
adapter = app.url_map.bind_to_environ(environ)
endpoint, values = adapter.match()
```

Let's trace this for GET `/notes/42`:

```
Step 1: bind_to_environ
 Creates a MapAdapter bound to the current request's host and scheme.
 The adapter knows: host="api.notes.example.com", scheme="https"

Step 2: adapter.match() (conceptually)
 Checking Rule: "/notes/"
 "/notes/42" has a second segment "42" → no match

 Checking Rule: "/notes/<int:note_id>" (segment pattern: \d+)
 "/notes/42" → MATCH
 Captured: {"note_id": "42"}

Step 3: Method check
 The matched Rule handles: {"GET", "HEAD", "OPTIONS"}
```

```
The request method is: "GET"
→ GET is in the allowed set → proceed
```

Step 4: Type conversion

```
"42" is passed to the int converter → 42 (Python int)
```

Step 5: Return

```
endpoint = "notes.get_note"
values = {"note_id": 42}
```

Flask then looks up `view_functions["notes.get_note"]` and calls it with `note_id=42`.

## What happens when there is no match

**Path not found** — no Rule matches the URL path at all:

```
GET /nonexistent/

adapter.match() raises: werkzeug.exceptions.NotFound
Flask catches this and calls: app.handle_http_exception(NotFound())
Which calls the registered 404 error handler
Result: HTTP 404 response
```

**Method not allowed** — a Rule matches the path, but not for this HTTP method:

```
DELETE /notes/ (there is a GET rule for /notes/ but not DELETE)

adapter.match() raises:
werkzeug.exceptions.MethodNotAllowed(valid_methods=["GET", "HEAD", "OPTIONS",
↪ "POST"])
(both rules for /notes/ count: list_notes and create_note)
Flask calls the 405 error handler, which returns HTTP 405 with the header
Allow: GET, HEAD, OPTIONS, POST
```

The distinction between 404 and 405 matters:

- **404:** The URL doesn't exist. The client should not retry with a different method.
- **405:** The URL exists but the method is wrong. The Allow header tells the client what methods work.

This distinction is why methods belong in the route declaration (`methods=[...]`) rather than in `if request.method == ...` branches inside a view: only declared methods let Flask an-

swer 405 with a correct Allow header.

### 32.4 URL Variable Converters

Converters are the `<type:name>` syntax in URL patterns. They perform two jobs:

1. **Pattern:** control what part of the URL matches (via regex)
2. **Conversion:** transform the matched string into a Python value

#### Built-in converters

Converter	Pattern	Python type	Example
string	<code>[^/]+</code>	str	<code>&lt;string:name&gt;</code> matches foo
int	<code>[0-9]+</code>	int	<code>&lt;int:id&gt;</code> matches 42
float	<code>[0-9]+\.[0-9]+</code>	float	<code>&lt;float:ratio&gt;</code> matches 3.14
path	<code>.+</code>	str	<code>&lt;path:filename&gt;</code> matches dir/file.txt
uuid	UUID pattern	uuid.UUID	<code>&lt;uuid:id&gt;</code> matches 550e8400-...

The default converter (used when you write just `<name>`) is string.

```
These are equivalent:
@app.route("/users/<name>")
@app.route("/users/<string:name>")

int converter: only matches digits, converts to int
@app.route("/notes/<int:note_id>")
/notes/42 → note_id=42 (int)
/notes/abc → 404 Not Found (no match)

path converter: matches slashes
@app.route("/files/<path:filename>")
/files/images/photo.jpg → filename="images/photo.jpg"
```

#### Why `<int:note_id>` is better than `<note_id>`

Consider what happens with each approach when someone requests `/notes/abc`:

```

With string converter (default):
@app.route("/notes/<note_id>")
def get_note(note_id):
 # note_id = "abc" (a string)
 # You must validate and convert it yourself
 try:
 note_id = int(note_id)
 except ValueError:
 return {"error": "note_id must be an integer"}, 400
 ...

With int converter:
@app.route("/notes/<int:note_id>")
def get_note(note_id):
 # note_id = 42 (already an int, guaranteed)
 # /notes/abc never reaches here - 404 before the function is called
 ...

```

The `int` converter means `/notes/abc` returns 404 automatically — “no such route” — which is correct. There is no note at `/notes/abc`; that URL doesn’t exist.

## Defining a custom converter

If the built-in converters don’t cover your case, you can write your own:

```

app/converters.py
from werkzeug.routing import BaseConverter

class SlugConverter(BaseConverter):
 """
 Matches URL-safe slugs: lowercase letters, digits, hyphens.
 Example: /articles/my-first-post
 """
 # This regex is what the converter matches in the URL
 # (a single character, or several that do not start or end with "-")
 regex = r"[a-z0-9](?:[a-z0-9-]*[a-z0-9])?"

 def to_python(self, value: str) -> str:
 """Called when matching: convert the URL string to a Python value."""
 return value # Already the right type

 def to_url(self, value: str) -> str:
 """Called when building URLs: convert a Python value to a URL string."""

```

```
return value # Build URLs from slugs exactly as they are
```

Register the converter in `create_app()`:

```
app/__init__.py
from app.converters import SlugConverter

def create_app():
 app = Flask(__name__)
 # Register the converter BEFORE any blueprint that uses <slug:...>:
 # the rules are built when the blueprint is registered, and an
 # unknown converter name is an error at that moment
 app.url_map.converters["slug"] = SlugConverter
 # ... rest of create_app (register_blueprint calls come after)
 return app
```

Use it in routes:

```
@bp.route("/articles/<slug:article_slug>")
def get_article(article_slug: str):
 # /articles/my-first-post → article_slug="my-first-post"
 # /articles/CAPS → 404 (doesn't match regex)
 ...
```

## 32.5 Routing Priority and Ambiguity

When multiple Rules could match the same URL, Werkzeug uses **specificity ordering** to pick the winner.

### Specificity rules

More specific rules take priority over less specific ones:

```

Rule 1: Specific static segment
@app.route("/notes/new")

Rule 2: Variable segment that accepts any text
@app.route("/notes/<name>") # <name> = <string:name>

GET /notes/new:
Rule 1 matches: endpoint="new_note_form"
Rule 2 ALSO matches (name="new") – both rules fit
Rule 1 wins because static segments are more specific than variable segments

GET /notes/groceries:
Rule 1 doesn't match ("groceries" ≠ "new")
Rule 2 matches: name="groceries"

```

Static path segments always beat variable segments. `/notes/new` is more specific than `/notes/<int:note_id>`.

## Trailing slash handling

Flask and Werkzeug have a specific rule about trailing slashes that catches many developers off guard.

**Rule with trailing slash:** `@app.route("/notes/")`

- GET `/notes/` → matches (200 OK)
- GET `/notes` → redirect to `/notes/` (308 Permanent Redirect; older Werkzeug versions used 301)

**Rule without trailing slash:** `@app.route("/notes")`

- GET `/notes` → matches (200 OK)
- GET `/notes/` → 404 Not Found

The behavior with a trailing slash mimics how web servers handle directory URLs. The 308 status matters for an API: unlike 301, it tells the client to repeat the *same* method with the *same* body, so a POST `/notes` is re-sent as POST `/notes/`. But the client must follow redirects at all — curl does only with `-L`, and some HTTP libraries do not re-send request bodies — so for API clients a redirect is a trap, not a convenience. The Notes API uses a trailing slash for its collection endpoint (`/notes/`) and none for item endpoints (`/notes/42`), the convention Flask's documentation suggests, and documents the exact URLs so clients never depend on the redirect. (Many JSON-API style guides choose the opposite, no trailing slashes anywhere;



either works if it is consistent.)

```
Collection endpoints: trailing slash (so /notes redirects to /notes/)
@notes_bp.route("/", methods=["GET"]) # GET /notes/
@notes_bp.route("/", methods=["POST"]) # POST /notes/

Item endpoints: no trailing slash
@notes_bp.route("/<int:note_id>", methods=["GET"]) # GET /notes/42
@notes_bp.route("/<int:note_id>", methods=["PUT"]) # PUT /notes/42 (Chapter 33)
@notes_bp.route("/<int:note_id>", methods=["DELETE"]) # DELETE /notes/42
```

## 32.6 Blueprints and the URL Map

A **Blueprint** is a collection of routes that can be registered on an app. Blueprints have their own route registry, separate from the app's `url_map`. When you call `app.register_blueprint(bp)`, the Blueprint's rules are copied into the app's `url_map`.

### The `url_prefix`

```
In app/routes/notes.py:
notes_bp = Blueprint("notes", __name__, url_prefix="/notes")

In app/__init__.py:
app.register_blueprint(notes_bp)

@notes_bp.route("/") # The Blueprint-relative path
def list_notes():
 ...

@notes_bp.route("/<int:note_id>") # Blueprint-relative
def get_note(note_id):
 ...
```

After registration, the actual URL patterns in the app's `url_map` are:

- `/notes + / = /notes/`
- `/notes + /<int:note_id> = /notes/<int:note_id>`

The `url_prefix` acts as a namespace for all routes in the Blueprint. (It can also be passed to `register_blueprint(notes_bp, url_prefix="/notes")` instead, which overrides the Blueprint's own; the Notes API sets it on the Blueprint, next to its routes.)

## Blueprint endpoint names

Blueprint route endpoint names are automatically prefixed with the Blueprint name:

```
Blueprint name: "notes"
Route function: list_notes

Resulting endpoint name: "notes.list_notes"
```

This matters when building URLs with `url_for()`:

```
from flask import url_for

Inside a request context:
url_for("notes.list_notes") # "/notes/"
url_for("notes.get_note", note_id=42) # "/notes/42"

Without the Blueprint prefix, this raises BuildError:
url_for("list_notes") # ERROR: no endpoint named "list_notes"
```

The Blueprint prefix prevents endpoint name collisions between Blueprints. If two Blueprints both have a `list` function, their endpoints are `users.list` and `notes.list` — no conflict.

## 32.7 URL Building with `url_for()`

`url_for()` is the reverse of URL matching: given an endpoint name (and any required variables), it produces the URL string.

```

from flask import url_for

Basic usage
url_for("notes.list_notes") # "/notes/"
url_for("notes.get_note", note_id=42) # "/notes/42"
url_for("notes.delete_note", note_id=7) # "/notes/7"

Extra keyword arguments become query string parameters
url_for("notes.list_notes", page=2, limit=10) # "/notes/?page=2&limit=10"

_external=True builds an absolute URL
url_for("notes.get_note", note_id=42, _external=True)
"http://localhost:5000/notes/42" (in dev)
"https://api.notes.example.com/notes/42" (in production – the host comes from
the request's Host header, and the scheme from X-Forwarded-Proto via ProxyFix)

```

## Why use `url_for()` instead of hardcoding URLs?

```

Fragile: hardcoded URL
return redirect("/notes/" + str(note_id))

Correct: use url_for
return redirect(url_for("notes.get_note", note_id=note_id))

```

If you change the URL pattern in `@notes_bp.route("/<int:note_id>")` to something else, every hardcoded URL breaks. `url_for()` recomputes the URL from the current rule, so it always stays in sync.

## `url_for()` requires a request context

`url_for()` normally requires an active request context (or an application context with `SERVER_NAME` configured — a setting best left unset behind a proxy, because it also makes Flask reject requests for any other host name). In test code that is not itself handling a request, push one:

```

with app.test_request_context():
 url = url_for("notes.get_note", note_id=42)

```

(During a request made with the test client, the request context is pushed automatically — but only for the code that runs inside that request.)

## 32.8 Inspecting Routes

Several tools let you see all registered routes at runtime.

### Flask CLI command

```
flask --app run routes
```

*# Output:*

Endpoint	Methods	Rule
health.health	GET	/health
notes.create_note	POST	/notes/
notes.delete_note	DELETE	/notes/<int:note_id>
notes.get_note	GET	/notes/<int:note_id>
notes.list_notes	GET	/notes/
static	GET	/static/<path:filename>

(HEAD and OPTIONS are hidden unless you add `--all-methods`.)

### Programmatic inspection

```
In a Flask shell or test:
from app import create_app
app = create_app()

Sorted by URL pattern for readability
rules = sorted(app.url_map.iter_rules(), key=lambda r: r.rule)
for rule in rules:
 methods = ",".join(sorted(m for m in rule.methods if m not in ("HEAD",
↪ "OPTIONS"))))
 print(f" {methods:10s} {rule.rule:30s} {rule.endpoint}")
```

### Building a route documentation endpoint

Some APIs expose their own route list for client developers:

```

In a Blueprint for administrative/meta endpoints:
from flask import current_app

@admin_bp.route("/routes")
def list_routes():
 """Return all registered routes as JSON. Development only."""
 if not current_app.debug:
 # 404, not 403: in production, do not even confirm the endpoint exists
 return {"error": "Not found"}, 404

 routes = []
 for rule in sorted(current_app.url_map.iter_rules(), key=lambda r: r.rule):
 routes.append({
 "endpoint": rule.endpoint,
 "methods": sorted(m for m in rule.methods if m not in ("HEAD",
↪ "OPTIONS")),
 "rule": rule.rule,
 })
 return {"routes": routes}

```

## 32.9 How Flask Calls the Handler

Once routing identifies the endpoint and values, Flask calls the handler function. The full call sequence inside Flask is:

```

Simplified from Flask's full_dispatch_request method:
def full_dispatch_request(self):
 try:
 # 1. Run before_request hooks
 rv = self.preprocess_request()
 if rv is None:
 # 2. Dispatch to the handler
 rv = self.dispatch_request()
 except Exception as e:
 # 3. Handle exceptions: the matching error handler produces rv
 rv = self.handle_user_exception(e)
 # 4. Make a response from the return value (and run after_request hooks)
 return self.finalize_request(rv)

def dispatch_request(self):
 # Get the routing result from the request context
 rule = request.url_rule # The matched Rule object

```

```

view_args = request.view_args # The extracted URL variables

Look up the handler function
return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args)

```

`view_args` contains the extracted URL variables. For GET `/notes/42`, `view_args = {"note_id": 42}`, and Flask calls `get_note(note_id=42)`.

The `ensure_sync()` call is Flask's support for async view functions (Python `async def`). For regular synchronous functions, it's a no-op passthrough.

## 32.10 Routing in the Notes API

Let's examine the complete routing structure of the Notes API — as it will look once Chapter 33 adds the PUT route for updating a note:

```

app/routes/notes.py

from flask import Blueprint

notes_bp = Blueprint("notes", __name__, url_prefix="/notes")

@notes_bp.route("/", methods=["GET"])
def list_notes():
 """List all notes, newest first."""
 ...

@notes_bp.route("/", methods=["POST"])
def create_note():
 """Create a new note. Body: {"content": "..."}"""
 ...

@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id: int):
 """Get a specific note by ID."""
 ...

@notes_bp.route("/<int:note_id>", methods=["PUT"])
def update_note(note_id: int):
 """Update a note's content. Body: {"content": "..."}"""
 ...

```

```
@notes_bp.route("/<int:note_id>", methods=["DELETE"])
def delete_note(note_id: int):
 """Delete a note."""
 ...
```

```
app/__init__.py
def create_app():
 app = Flask(__name__)
 # ...
 from app.routes.notes import notes_bp
 app.register_blueprint(notes_bp)
 return app
```

The resulting URL map:

Method	URL	Handler
GET	/notes/	notes.list_notes
POST	/notes/	notes.create_note
GET	/notes/<int:note_id>	notes.get_note
PUT	/notes/<int:note_id>	notes.update_note
DELETE	/notes/<int:note_id>	notes.delete_note

This design is **resource-oriented**: URLs identify resources (/notes/, /notes/42), and HTTP methods identify actions (GET = read, POST = create, PUT = replace, DELETE = remove). A note has only one field a client can set, so PUT (replace the note) and PATCH (change some fields) would do the same thing here; with more fields, the difference matters. This is RESTful API design, and it maps cleanly onto Flask's routing primitives.

### Key Takeaways

- **@app.route()** is a decorator that calls `add_url_rule()`, which creates a Werkzeug Rule and adds it to the app's `url_map`. Registration happens at import time.
- The **url\_map** is a collection of Rule objects. Each Rule contains the URL pattern, the allowed HTTP methods, and the endpoint name; Werkzeug compiles them all into one state machine for matching.
- **URL matching** finds the most specific rule that fits the path. If no Rule matches the path, 404 is raised. If a Rule matches the path but not the method, 405 is raised.
- **Converters** (<int:note\_id>) both constrain what the URL segment must look like and convert the matched string to the appropriate Python type. /notes/abc returns 404 when the route uses <int:note\_id>.

- **Trailing slash rules:** routes with a trailing slash redirect requests without one (308); routes without a trailing slash return 404 for requests with one. API clients should use the exact URLs.
- **Blueprints** collect routes separately and are merged into the app's `url_map` at `register_blueprint()` time. Endpoint names are prefixed with the Blueprint name.
- **`url_for()`** builds URLs from endpoint names. Always use `url_for()` instead of hardcoding paths so that URL changes propagate automatically.

## Exercises

1. **Predict.** Request `/notes` (no trailing slash) with `curl -i`. What status and header come back, and why is that a problem for a `POST`?
2. **Break and fix.** Add `@notes_bp.route("/<note_id>")` (no converter) next to the `int` route. Which one handles `/notes/5`? Remove the ambiguity.
3. **Extend.** Print the application's URL map (`flask --app 'app:create_app()' routes`) and compare it with the endpoints listed in section 32.10. Is there a rule you did not expect?
4. **Explain.** Why should code build URLs with `url_for()` instead of writing paths as strings?

Solutions and hints are in the companion repository, in `exercises/chapter-32.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

The routing layer has identified the handler function and passed in the extracted URL variables. The handler now executes its logic: fetching data from the database, applying business rules, computing results. That execution is the subject of the next chapter.

→ Chapter 33 — Logic Execution



# Logic Execution

---

The routing layer (Chapter 32) identified the handler function and called it with the validated URL variables. The input handling layer (Chapter 31) validated and cleaned the request data. Now the handler runs its **business logic** — the code that actually does what the API is supposed to do.

For the Notes API, business logic is simple: read from and write to a PostgreSQL database. Real-world applications have more complex logic — but the structural patterns are the same. This chapter traces how logic executes inside a Flask handler, from the function call to the database and back, and explains the tools and techniques that make that execution reliable, observable, and maintainable.

## In this chapter

By the end of this chapter, you will understand:

- What “business logic” means and where it lives in the code structure
- How the Flask request context makes `request`, `g`, and `current_app` available
- How database connections are managed per-request with `g`
- The complete lifecycle of a database query: connection, cursor, execute, fetch, commit
- How transactions protect against partial writes
- How a migration runner keeps every database’s schema in step with the code, and where migrations fit in a deployment
- How to turn a schema design into tables whose constraints protect the data
- How to query across tables with joins and aggregates, and how to avoid the N+1 problem
- What happens when two requests change the same data at once, and the tools that keep it correct
- When raw SQL, a query builder, or an ORM is the right choice
- How to structure handler functions for readability and testability
- How errors during logic execution are handled and logged

### 33.1 What Is Business Logic?

**Business logic** is the code that implements the rules of your application's domain.

For the Notes API:

- Creating a note means inserting a row into the notes table
- Getting a note means selecting a row by its id
- Updating a note means replacing its content
- Deleting a note means removing the row

These rules are specific to the Notes domain. They are distinct from:

- **Infrastructure logic** (how to connect to PostgreSQL, how to run a query)
- **HTTP logic** (how to parse JSON, how to set response headers)
- **Validation logic** (whether the input data is acceptable)

Keeping these concerns separate makes code easier to test, modify, and reason about.

#### Where does business logic live?

There are two common patterns:

**Pattern 1: Logic in the handler (small apps)**

```
@notes_bp.route("/", methods=["POST"])
def create_note():
 data = request.get_json(silent=True)
 # ... validation ...
 db = get_db()
 cursor = db.cursor()
 cursor.execute(
 sql("INSERT INTO notes (content) VALUES (?) RETURNING id, content,
 ↪ created_at"),
 (content,),
)
 note = row_to_note(cursor.fetchone())
 db.commit()
 return jsonify(note), 201
```

**Pattern 2: Logic in a service layer (larger apps)**

```

app/services/notes.py
def create_note(db, content: str) -> dict:
 cursor = db.cursor()
 cursor.execute(...)
 note = row_to_note(cursor.fetchone())
 db.commit()
 return note

app/routes/notes.py
@notes_bp.route("/", methods=["POST"])
def create_note():
 data = request.get_json(silent=True)
 # ... validation ...
 db = get_db()
 note = note_service.create_note(db, content)
 return jsonify(note), 201

```

For the Notes API at its current scale, Pattern 1 is appropriate. Pattern 2 becomes valuable when the same business logic needs to be called from multiple handlers (e.g., a CLI command and an HTTP endpoint both creating notes), or when the logic grows complex enough that it needs its own tests independent of HTTP.

This chapter uses Pattern 1 because it is direct and easier to trace.

## 33.2 The Flask Request Context

When Flask calls your handler function, a **request context** has already been pushed. The request context is the mechanism that makes Flask's global-looking objects — `request`, `g`, `current_app` — work correctly even when multiple threads or processes are handling different requests simultaneously.

### What the contexts contain

Flask actually keeps two contexts, pushed together for every request:

```

Application Context (pushed for each request, and by app.app_context())
├─ current_app → The Flask application handling the request
└─ g → A namespace for storing data during this context

Request Context (one per active request)
├─ request → The Request object (parsed HTTP request)
└─ session → The session object (signed cookie)

These are "context locals". Separate Gunicorn worker PROCESSES are isolated
anyway; the context machinery matters when several THREADS (or greenlets)
in one worker handle requests at the same time – each sees its own request
and its own g.

```

Because `g` belongs to the application context, and Flask pushes a fresh one for every request, `g` is in practice per-request storage.

## How Flask implements context locals

Flask uses Python's `contextvars` module (Python 3.7+) to store context-local data. Each coroutine or thread has its own context variable storage, so `flask.request` in Thread A is a different object from `flask.request` in Thread B – even though they look like the same global variable.

```

What you write (looks like a global):
from flask import request, g

@notes_bp.route("/<int:note_id>")
def get_note(note_id):
 # request.method is the METHOD of THIS request
 # g.request_start_time was set by a before_request hook for THIS request
 ...

What Flask does under the hood (very simplified):
import contextvars
from werkzeug.local import LocalProxy

_cv_request = contextvars.ContextVar("flask.request_ctx")

"request" is a proxy OBJECT: every attribute access (request.method)
looks up the request context that is current right now, in this thread
request = LocalProxy(_cv_request, "request")

Each time a new request arrives, Flask does:
ctx = RequestContext(app, environ)

```

```
ctx.push() # Sets _cv_request (and the app context) for this request
```

You don't need to manage context variables yourself. Flask handles this. The important thing to understand is that `request` and `g` are **request-scoped**: they exist for the duration of a single request and are discarded afterward.

## The g object

`g` is a per-request namespace for storing data that needs to be shared between the parts of request processing (before-request hooks, the handler, after-request hooks) without passing it explicitly through function arguments.

Common uses:

```
before_request hook: store the start time
@app.before_request
def log_request_start():
 g.request_start_time = time.perf_counter()

after_request hook: compute and log the duration
@app.after_request
def log_request_end(response):
 elapsed_ms = (time.perf_counter() - g.request_start_time) * 1000
 logger.info("← %s in %.1fms", response.status_code, elapsed_ms)
 return response

Database connection management
def get_db():
 if "db" not in g:
 g.db = open_db_connection()
 return g.db

@app.teardown_appcontext
def close_db(error):
 db = g.pop("db", None)
 if db is not None:
 db.rollback()
 db.close()
```

`g` is not a cache that persists between requests. It is wiped clean at the end of each request.

### 33.3 Database Connection Management

Chapter 4 introduced `get_db()` — the function that opens (or reuses) a database connection for the current request — and Chapter 15 moved it, with everything else database-specific, into `app/database.py`. Let's examine it in full.

#### The connection pool question

Ideally, you would use a **connection pool** — a set of pre-opened database connections that are checked out for a request and returned afterward, avoiding the overhead of opening a new connection for every request.

The `psycopg2.pool` module (part of `psycopg2`) or SQLAlchemy's connection pool provide this. For the Notes API, we use a simpler pattern: **one connection per request**, opened on first use and closed when the request ends.

The overhead of opening a PostgreSQL connection is real: a TCP handshake, a TLS handshake (managed databases require TLS), and authentication — typically 10–30 ms over a VPC. And every open connection counts against the database's connection limit, which on the smallest managed plans is only a few dozen. For a small API, one connection per request is acceptable; as traffic grows, a pool (or a pooler such as PgBouncer, Chapter 36) becomes essential.

#### The `get_db()` pattern

This is the connection-handling half of Listing 15.2, `app/database.py`, unchanged:

```
app/database.py (from Chapter 15)
import sqlite3
from flask import g
from app.config import DATABASE_URL, IS_SQLITE

def open_db():
 """
 Open a database connection appropriate for the configured DATABASE_URL.

 Both kinds of connection return rows that behave like dictionaries
 (row["id"]), so the code that reads them does not care which it got.
 """
 if IS_SQLITE:
 conn = sqlite3.connect(DATABASE_URL[len("sqlite:///"):])
 conn.row_factory = sqlite3.Row
 return conn
```

```

import psycopg2
import psycopg2.extras
psycopg2 starts a transaction automatically; handlers commit explicitly
return psycopg2.connect(DATABASE_URL,
 ↪ cursor_factory=psycopg2.extras.RealDictCursor)

def get_db():
 """Return the per-request database connection, opening it if needed."""
 if "db" not in g:
 g.db = open_db()
 return g.db

def close_db(error=None):
 """
 Close the database connection at the end of a request.

 Handlers commit their own work *before* they return a response, so
 anything still uncommitted here belongs to a request that failed
 part-way: it is rolled back, never committed.
 """

 db = g.pop("db", None)
 if db is not None:
 db.rollback() # A no-op after a commit; discards half-finished work
 db.close()

```

It is worth seeing why `close_db()` does not do what many tutorials do — “commit if error is None, otherwise roll back”:

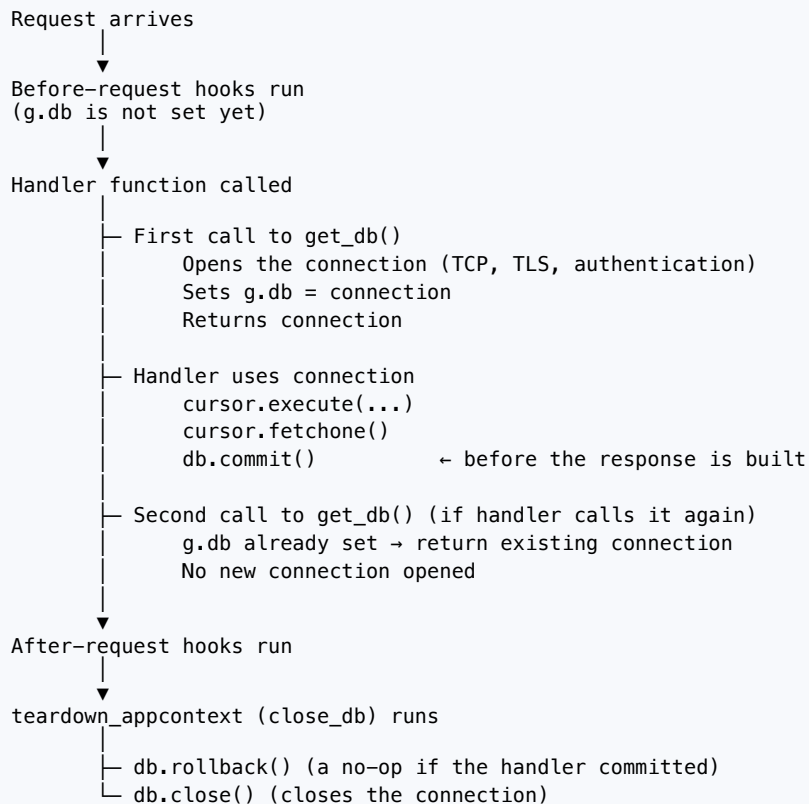
1. Teardown runs *after* the response has been built. A commit that fails there cannot change the 201 Created the client is about to receive for a note that was never saved.
2. Flask passes an error to teardown only for exceptions that no error handler caught. The Notes API’s catch-all handler ([Chapter 16](#)) catches every exception, so error is always None — and a “commit if no error” teardown would commit the half-finished work of a request that just failed with a 500.

So the rule is simple: **a handler that changes data commits before it responds; teardown only ever rolls back.**

```
app/__init__.py
from app.database import close_db

def create_app():
 app = Flask(__name__)
 # ...
 app.teardown_appcontext(close_db)
 return app
```

## Connection lifecycle diagram



## 33.4 Executing a Database Query

With a database connection in hand, the handler executes queries. PostgreSQL queries go through a **cursor** — an object that manages query execution and result fetching.



## The cursor

```
db = get_db()
cursor = db.cursor()

On PostgreSQL, cursor is a psycopg2.extras.RealDictCursor (open_db passes
cursor_factory=RealDictCursor); on SQLite, rows are sqlite3.Row objects.
Either way, row["id"] works.
```

## Parameterized queries

Every database query that includes variable data **must** use parameterized queries. Chapter 31 explained why: SQL injection. The mechanism:

```
WRONG – vulnerable to SQL injection:
cursor.execute(f"SELECT * FROM notes WHERE id = {note_id}")

CORRECT – parameterized query:
cursor.execute(sql("SELECT * FROM notes WHERE id = ?"), (note_id,))
```

The `?` is a placeholder; `sql()` (Chapter 15) turns it into `psycopg2`'s `%s` when the database is PostgreSQL. The value never becomes part of the SQL text as code: `sqlite3` binds it to a prepared statement, and `psycopg2` inserts it into the query on the client side as a correctly quoted literal. Either way, it cannot be interpreted as SQL syntax.

Note the comma after `note_id` in `(note_id,)` – this creates a **tuple** with one element. Without the comma, `(note_id)` is just `note_id` in parentheses (not a tuple), and the driver fails with a confusing error.

```
Common mistake:
cursor.execute(sql("SELECT * FROM notes WHERE id = ?"), (note_id))
psycopg2: TypeError: 'int' object does not support indexing
sqlite3: ValueError: parameters are of unsupported type

Correct:
cursor.execute(sql("SELECT * FROM notes WHERE id = ?"), (note_id,)) # tuple with one
↪ element
cursor.execute(sql("SELECT * FROM notes WHERE id = ?"), [note_id]) # list also
↪ works
```

## Fetching results

After `execute()`, fetch the results:

```
Fetch one row (or None if no rows matched)
row = cursor.fetchone()
On PostgreSQL, a dict (or None):
{"id": 42, "content": "Buy groceries", "created_at": datetime(...)}

Fetch all rows
rows = cursor.fetchall()
List of dicts (empty list if no rows matched)

Fetch N rows at a time (for large result sets)
rows = cursor.fetchmany(size=100)
```

## INSERT with RETURNING

PostgreSQL (and SQLite since version 3.35) supports a `RETURNING` clause that returns data from the inserted row. This is useful because it gives you the auto-generated `id` and `created_at` values without a separate `SELECT`:

```
cursor.execute(
 sql("""
 INSERT INTO notes (content)
 VALUES (?)
 RETURNING id, content, created_at
 """),
 (content,),
)
row = cursor.fetchone()
On PostgreSQL:
row = {"id": 42, "content": "Buy groceries",
"created_at": datetime(2024, 1, 15, 14, 0, 0, 123456, tzinfo=timezone.utc)}
```

Without `RETURNING`, you would need two queries: the `INSERT` followed by `SELECT lastval()` (and a `SELECT` for the other columns). `RETURNING` does it in one round trip.

## 33.5 Transactions

A **transaction** is a group of database operations that are treated as a single atomic unit: either all operations succeed (and are committed), or all are rolled back (and have no effect).

## Why transactions matter

Consider an operation that needs to do two things atomically:

```
Hypothetical: move a note to an archive table
cursor.execute("INSERT INTO archived_notes SELECT * FROM notes WHERE id = %s",
↳ (note_id,))
cursor.execute("DELETE FROM notes WHERE id = %s", (note_id,))
db.commit()
```

Both statements run inside one transaction, which the `commit()` ends. If the process crashes after the `INSERT` but before the `commit`, the database rolls the whole transaction back: the note is not left in both tables. That is the guarantee transactions give — and `psycopg2`'s automatic transactions (below) give it here without any extra code.

What a `try/except` adds is not atomicity but tidiness when an *exception* happens in the middle:

```
try:
 cursor.execute("INSERT INTO archived_notes SELECT * FROM notes WHERE id = %s",
↳ (note_id,))
 cursor.execute("DELETE FROM notes WHERE id = %s", (note_id,))
 db.commit() # Both operations committed atomically
except Exception:
 db.rollback() # End the failed transaction now, so the connection is usable again
 raise
```

(In the Notes API, `close_db()` already rolls back at the end of every request, so handlers do not need this.)

## psycopg2's default transaction mode

`psycopg2` begins a transaction automatically when you execute the first query after `connect()` (or after the previous `commit()` or `rollback()`). You explicitly end the transaction with `commit()` or `rollback()`.

```

connect()
 ↓
 ▼ (transaction implicitly started by first execute)
execute("INSERT ...")
execute("UPDATE ...")
 |
 ├── commit() → Both changes written to database permanently
 └── rollback() → Neither change written; database unchanged

```

For simple APIs, each request is typically one logical operation (one INSERT, or one SELECT), so there is one transaction per request.

### Autocommit mode

For queries that must not be in a transaction (like CREATE DATABASE or VACUUM), you can set:

```
conn.autocommit = True
```

In autocommit mode, each `execute()` is immediately committed. There is no multi-statement transaction. This is generally not what you want for web application queries.

### Commit strategy for the Notes API

The Notes API commits explicitly in each handler that changes data, *before* it returns the response; `close_db()` only ever rolls back:

```

In the handler (explicit commit, before responding):
db.commit()

In close_db (anything left uncommitted belongs to a failed request):
db.rollback()
db.close()

```

A handler that forgets to commit therefore loses its change — loudly, in the first test that reads the data back — instead of having it committed silently after the response. Section 33.3 explains why the “commit in teardown” alternative is unsafe.

## 33.6 The Complete Handler: `create_note`

Let’s trace the complete execution of `POST /notes/` through the handler — Chapter 16’s handler, with the `Validator` from Chapter 31 in place of the individual checks:

```

app/routes/notes.py
import logging
from flask import Blueprint, jsonify, request
from app.database import get_db, sql, row_to_note
from app.validation import Validator

logger = logging.getLogger(__name__)
notes_bp = Blueprint("notes", __name__, url_prefix="/notes")

@notes_bp.route("/", methods=["POST"])
def create_note():
 """
 Create a new note.

 Request body: {"content": "note text"}
 Response: 201 Created with the created note as JSON
 """
 # — Phase 1: Parse the request body —————
 data = request.get_json(silent=True)
 # silent=True means parse errors return None instead of raising an exception

 # — Phase 2: Validate input —————
 validator = Validator(data)
 content = validator.require_string(
 "content",
 min_length=1,
 max_length=10_000,
 strip=True,
)

 if validator.has_errors():
 logger.warning("POST /notes/ rejected: %s", "; ".join(validator.errors))
 return validator.error_response() # 400, {"error": ...}

 # — Phase 3: Business logic (database interaction) —————
 db = get_db()
 cursor = db.cursor()

 cursor.execute(
 sql(
 """
 INSERT INTO notes (content)
 VALUES (?)
 RETURNING id, content, created_at
 """,
),
)

```

```

 (content,),
)

 note = row_to_note(cursor.fetchone())
 db.commit() # before responding: a 201 must mean the note is saved

 logger.info("Created note id=%d.", note["id"])

 # — Phase 4: Build and return the response —————
 return jsonify(note), 201

```

## Phase-by-phase trace

### Phase 1: `request.get_json(silent=True)`

- Flask reads `Content-Type: application/json` from headers
- Calls `json.loads()` on the request body bytes
- Returns a Python dict: `{"content": "Buy groceries"}`
- If the body is not valid JSON, returns `None` (due to `silent=True`)

### Phase 2: `Validator(data).require_string(...)`

- Checks that `data` is a JSON object (not `None`, a list, or a bare string)
- Checks that `data["content"]` exists and is a string
- Strips leading/trailing whitespace
- Checks length is within bounds
- If any check fails, adds a message to `validator.errors` and returns `None`
- If there are errors, the handler returns a 400 response immediately

### Phase 3: Database interaction

- `get_db()` returns the per-request connection (opens it if this is the first call)
- `cursor.execute(INSERT ...)` sends the parameterized query to PostgreSQL
- PostgreSQL inserts the row and returns the new row's data
- `cursor.fetchone()` retrieves that returned row, and `row_to_note()` turns it into the note dictionary
- `db.commit()` commits the transaction, making the insert permanent
- The logger records the successful creation with the new note's ID

### Phase 4: Response construction

- On PostgreSQL, the row's `created_at` is a timezone-aware datetime — which `jsonify` would turn into an HTTP-style date ("Mon, 15 Jan 2024 14:00:00 GMT"), not ISO 8601
- So `row_to_note()` (through `to_iso()`, Chapter 15) converts it to "2024-01-15T14:00:00.123456+00:00" — the same ISO 8601 form the SQLite branch produces
- `jsonify(note)` serializes the dict to JSON and sets Content-Type: application/json
- HTTP 201 Created status indicates resource creation

### 33.7 Reading a Note: get\_note

```
@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id: int):
 """
 Retrieve a note by its ID.

 URL parameter: note_id (int, guaranteed by <int:> converter)
 Response: 200 OK with the note, or 404 if not found
 """
 cursor = get_db().cursor()

 cursor.execute(
 sql("SELECT id, content, created_at FROM notes WHERE id = ?",
 (note_id,)),
)

 row = cursor.fetchone()

 if row is None:
 # Note not found — this is a well-defined 404
 logger.info("Note id=%d not found.", note_id)
 return jsonify({"error": f"Note {note_id} not found"}), 404

 return jsonify(row_to_note(row)), 200
```

#### Key points:

- The URL converter guarantees `note_id` is a valid integer. No need to validate it.
- `fetchone()` returns `None` if no row matches the `WHERE id = ?` clause.
- A missing note is a 404, not a 500. The handler explicitly checks for `None` and returns the appropriate status code.
- No `db.commit()` is needed after a `SELECT` (reads don't modify data).

## 33.8 Listing Notes with Filtering

```
@notes_bp.route("/", methods=["GET"])
def list_notes():
 """
 List notes, newest first, optionally filtered by a search term.

 Query parameter: ?search=term
 Response: 200 OK with {"notes": [...], "count": n}
 """
 # Optional search parameter
 search = request.args.get("search", "").strip()

 cursor = get_db().cursor()

 if search:
 # In a LIKE pattern, % and _ are wildcards. Escape them (and the
 # escape character itself) so a search for "50%" means "50%".
 escaped = (search.replace("\\", "\\\\"))
 .replace("%", "\\%")
 .replace("_", "_")
 # Case-insensitive search using PostgreSQL's ILIKE
 cursor.execute(
 """
 SELECT id, content, created_at
 FROM notes
 WHERE content ILIKE %s ESCAPE '\\'
 ORDER BY id DESC
 """,
 (f"%{escaped}%",),
)
 else:
 cursor.execute("SELECT id, content, created_at FROM notes ORDER BY id DESC")

 notes = [row_to_note(row) for row in cursor.fetchall()]
 return jsonify({"notes": notes, "count": len(notes)}), 200
```

### Key points:

The ILIKE pattern `%{escaped}%` uses `%` as the SQL wildcard (match any sequence of characters). It is passed as a parameter *value* — not written into the SQL string. Two kinds of `%` meet here, and it helps to keep them apart:

- `%s` in the SQL text is psycopg2's parameter placeholder.



- The % characters inside the parameter *value* are LIKE wildcards. psycopg2 passes the value through as data; PostgreSQL sees exactly the pattern %groceries%.
- A literal % in the SQL *text* of a parameterized query must be written %, because psycopg2 reads % there as the start of a placeholder.

Two more things to know about this query:

- **It is PostgreSQL-only.** ILIKE does not exist in SQLite, and this branch is written with %s directly rather than through sql(). The tests from [Chapter 17](#) run on SQLite and never call ?search=, so they cannot notice. That gap — code that only the production database ever runs — is exactly the subject of [Chapter 39](#).
- **It reads the whole table.** A pattern that starts with % cannot use an ordinary B-tree index, so PostgreSQL scans every row. That is fine for thousands of notes; for millions, a trigram index (the pg\_trgm extension) or full-text search is the fix ([Chapter 36](#)).

## 33.9 Updating and Deleting Notes

Updating a note needs something the notes table does not have yet: a record of *when* it was last changed. The table was created by init\_db() with CREATE TABLE IF NOT EXISTS — which does nothing at all for a table that already exists. So the new column needs a **migration**: a one-time change to every existing database, stored with the code ([Chapter 19](#) introduced the idea):

```
-- migrations/0002_add_updated_at.sql
-- Run once against each existing database (development and production).
-- NULL until the note is first updated.
ALTER TABLE notes ADD COLUMN updated_at TIMESTAMPTZ; -- PostgreSQL
-- ALTER TABLE notes ADD COLUMN updated_at TIMESTAMP; -- SQLite
```

```
Production: run it with psql against the managed database
psql "$DATABASE_URL" -f migrations/0002_add_updated_at.sql
```

Add the column to the CREATE TABLE statements in init\_db() too (updated\_at TIMESTAMPTZ for PostgreSQL, updated\_at TIMESTAMP for SQLite), so that new databases — the test database, for one — get it from the start. And teach row\_to\_note() in app/database.py to convert the new timestamp as well:

```
def row_to_note(row) -> dict:
 """Convert a database row into the note dictionary the API returns."""
 note = dict(row)
 for field in ("created_at", "updated_at"):
 if note.get(field) is not None: # updated_at is NULL until an update
 note[field] = to_iso(note[field])
 return note
```

The order matters when you deploy: run the migration *before* the new code. The old code simply ignores the extra column; the new code fails without it. A nullable new column is an **additive** change (Chapter 6): safe for the code already running, which is exactly what makes this order work. (Running migrations by hand does not scale past the first one; section 33.10 turns it into a system.)

Now the handler:

```
@notes_bp.route("/<int:note_id>", methods=["PUT"])
def update_note(note_id: int):
 """Replace a note's content."""
 validator = Validator(request.get_json(silent=True))
 content = validator.require_string("content", min_length=1, max_length=10_000,
 ↪ strip=True)

 if validator.has_errors():
 return validator.error_response()

 db = get_db()
 cursor = db.cursor()

 cursor.execute(
 sql("""
 UPDATE notes
 SET content = ?, updated_at = CURRENT_TIMESTAMP
 WHERE id = ?
 RETURNING id, content, created_at, updated_at
 """),
 (content, note_id),
)

 row = cursor.fetchone()

 if row is None:
 # No row was updated – the note doesn't exist
```

```

 return jsonify({"error": f"Note {note_id} not found"}), 404

 note = row_to_note(row)
 db.commit()
 logger.info("Updated note id=%d.", note_id)

 return jsonify(note), 200

```

**Key point for UPDATE:** After executing an UPDATE, `fetchone()` returns `None` if no rows matched the WHERE clause. This distinguishes between “note updated successfully” and “note not found” — without requiring a separate SELECT.

```

@notes_bp.route("/<int:note_id>", methods=["DELETE"])
def delete_note(note_id: int):
 """Delete a note."""
 db = get_db()
 cursor = db.cursor()
 cursor.execute(sql("DELETE FROM notes WHERE id = ?"), (note_id,))
 db.commit()

 # rowcount: how many rows the DELETE removed (0 or 1 here)
 if cursor.rowcount == 0:
 logger.info("DELETE note id=%d: not found.", note_id)
 return jsonify({"error": f"Note {note_id} not found"}), 404

 logger.info("Deleted note id=%d.", note_id)

 # 204 No Content: success, but nothing to return
 return "", 204

```

**Key point for DELETE:** HTTP 204 No Content is the conventional response for successful deletes. A 204 response must not include a body, hence the empty string `""`. (Whether deleting a note that does not exist should be a 404, as here, or a 204 — “it is gone either way” — is the design choice discussed in Chapter 16, section 16.5.)

## 33.10 Migrations as a System

Section 33.9 made the Notes API’s first schema change by hand: write a SQL file, run it with `psql`, and add the same column to `init_db()`. That worked once. But it exposed three problems, and each one gets worse with every change that follows:

1. **Nobody knows which changes a database has had.** Was 0002 run on staging? On production? On your colleague's laptop? The only record is somebody's memory.
2. **The schema is written down twice.** `init_db()`'s `CREATE TABLE` builds new databases (the test database, a new developer's), while the migration files change old ones. Every change has to be made in both places, and sooner or later the two drift apart. When they do, the database your tests run against is no longer the one production runs.
3. **A person is part of the deploy.** A release that needs someone to remember a `psql` command will, one day, go out without it.

Almost every production team solves this the same way, with three parts:

- **Every schema change is a numbered file in the repository.** The migrations are the *only* definition of the schema. A brand-new database is built by running all of them, from the first.
- **Each database records which migrations it has had,** in a table of its own.
- **A migration runner applies the missing ones,** in order, as a step of every deployment.

Tools that do exactly this exist for every stack: Alembic (for SQLAlchemy), Flyway and Liquibase (any SQL database), the migration systems built into Django and Rails, `golang-migrate`. The Notes API writes its own runner instead, because at a little over 200 lines, nearly a third of them comments, docstrings and blank lines, it is short enough to understand completely. Once you have seen how one works, every one of those tools will feel familiar.

## The migrations directory

```
migrations/
├── postgresql/
│ ├── 0001_create_notes.sql
│ ├── 0002_add_updated_at.sql
│ └── 0003_add_tags.sql ← section 33.11
└── sqlite/
 ├── 0001_create_notes.sql
 ├── 0002_add_updated_at.sql
 └── 0003_add_tags.sql
```

There are two folders because the two databases speak slightly different SQL: `SERIAL` against `INTEGER PRIMARY KEY AUTOINCREMENT`, `TIMESTAMPZ` against `TIMESTAMP`. Until [Chapter 39](#) moves everything to PostgreSQL, every migration is written twice. That duplication is part of the price of running two databases, and Chapter 39 counts it.

0001 is the table that `init_db()` used to create, now written down as the first migration. 0002 is section 33.9's file, split in two:

```
-- migrations/postgresql/0001_create_notes.sql
-- The notes table as init_db() created it (Chapters 13–16).
CREATE TABLE notes (
 id SERIAL PRIMARY KEY,
 content TEXT NOT NULL,
 created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
-- migrations/postgresql/0002_add_updated_at.sql
-- NULL until the note is first updated (section 33.9).
ALTER TABLE notes ADD COLUMN updated_at TIMESTAMPTZ;
```

The rules that make migrations trustworthy are few, and every team that ignores one of them eventually learns why it exists:

- **Numbered, applied in order, never renumbered.** The number is the migration's identity. Zero-padding (0001, not 1) makes plain alphabetical sorting put them in the right order.
- **Never edit a migration that has run anywhere except your own laptop.** It has already happened to those databases. Editing the file changes what *new* databases get, and old and new drift apart. A mistake in a migration is fixed by the *next* migration.
- **Every migration must be safe for the code that is already running.** It runs *before* the new code starts, while the old version is still serving, and the old version may be rolled back to later. Additive changes are safe. A breaking change, such as a rename, a drop, or a new NOT NULL, goes through expand/contract (Chapter 6), spread over several releases.
- **One migration, one transaction.** PostgreSQL can change a schema inside a transaction, so a migration that fails half-way leaves nothing behind. (A few statements, such as CREATE INDEX CONCURRENTLY, refuse to run inside a transaction. This small runner does not support them; the full-size tools do.)

## The runner

```
app/migrate.py
"""
The migration runner: brings a database's schema up to date.

 python -m app.migrate apply every pending migration
```

```
python -m app.migrate --status show which migrations are applied
python -m app.migrate --baseline 0002 record 0001-0002 as applied (not run)
```

*Migrations are SQL files in migrations/sqlite/ or migrations/postgresql/, named NNNN\_description.sql, and run in the order of their numbers. Each one runs in its own transaction together with the row that records it in schema\_migrations, so a migration is either applied and recorded, or neither.*

```
"""

import argparse
import logging
import sqlite3
import sys
from pathlib import Path

from app.config import IS_SQLITE
from app.database import open_db, sql

Named explicitly: run as "python -m app.migrate", __name__ is "__main__"
logger = logging.getLogger("app.migrate")

MIGRATIONS_DIR = (
 Path(__file__).resolve().parent.parent
 / "migrations"
 / ("sqlite" if IS_SQLITE else "postgresql")
)

Any constant will do: every runner asks PostgreSQL for the same lock number
_LOCK_ID = 331_000

def available() -> list[tuple[str, Path]]:
 """Every migration file as (version, path), in order: [("0001", ...), ...]."""
 return [
 (path.name[:4], path)
 for path in sorted(MIGRATIONS_DIR.glob("[0-9][0-9][0-9][0-9]*.sql"))
]

def _lock(conn) -> None:
 """
 Make sure only one runner changes the schema at a time. PostgreSQL: a
 session-level advisory lock, held until the connection closes; any other
```

```

runner waits here. SQLite needs nothing more: each migration's
BEGIN IMMEDIATE (in _begin) takes the database's single write lock.
"""

if not IS_SQLITE:
 conn.cursor().execute("SELECT pg_advisory_lock(%s)", (_LOCK_ID,))
 conn.commit() # a session-level lock outlives the transaction

def _prepare(conn) -> None:
 """Create the table that records applied migrations (only under the lock)."""
 cursor = conn.cursor()
 cursor.execute("""
 CREATE TABLE IF NOT EXISTS schema_migrations (
 version TEXT PRIMARY KEY,
 applied_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
)
 """)
 conn.commit()

def _has_record_table(conn) -> bool:
 cursor = conn.cursor()
 if IS_SQLITE:
 cursor.execute(
 "SELECT name FROM sqlite_master "
 "WHERE type = 'table' AND name = 'schema_migrations'"
)
 return cursor.fetchone() is not None
 cursor.execute("SELECT to_regclass('schema_migrations') AS name")
 return cursor.fetchone()["name"] is not None

def _applied(conn) -> set[str]:
 cursor = conn.cursor()
 cursor.execute("SELECT version FROM schema_migrations")
 return {row["version"] for row in cursor.fetchall()}

def _statements(text: str) -> list[str]:
 """
 Split a migration file into statements. PostgreSQL accepts the whole
 file in one execute(); sqlite3 accepts one statement at a time.
 """
 if not IS_SQLITE:

```

```

 return [text]
 statements, current = [], ""
 for line in text.splitlines(keepends=True):
 current += line
 if sqlite3.complete_statement(current):
 statements.append(current)
 current = ""
 leftover = [
 ln
 for ln in current.splitlines()
 if ln.strip() and not ln.strip().startswith("--")
]
 if leftover:
 raise ValueError(
 f"Incomplete SQL statement at the end of the file: {leftover[0]!r}"
)
 return statements

def _begin(conn) -> None:
 """
 Start one migration's transaction. SQLite: BEGIN IMMEDIATE takes the
 write lock at once. (psycpg2 starts a transaction by itself.)
 """
 if IS_SQLITE:
 conn.cursor().execute("BEGIN IMMEDIATE")

def pending() -> list[str]:
 """
 The versions of the migrations not yet applied to the database. It only
 reads, so every worker can safely call it at startup.
 """
 conn = open_db()
 try:
 done = _applied(conn) if _has_record_table(conn) else set()
 return [version for version, _ in available() if version not in done]
 finally:
 conn.close()

def migrate() -> list[str]:
 """Apply every pending migration, in order. Returns the versions applied."""
 conn = open_db()

```



```

try:
 _lock(conn)
 _prepare(conn)
 newly_applied = []
 for version, path in available():
 _begin(conn)
 # Checked inside the lock: another runner may have applied it
 if version in _applied(conn):
 conn.rollback()
 continue
 logger.info("Applying migration %s", path.name)
 try:
 cursor = conn.cursor()
 for statement in _statements(path.read_text()):
 cursor.execute(statement)
 cursor.execute(
 sql("INSERT INTO schema_migrations (version) VALUES (?)",
 (version,)),
)
 conn.commit()
 except Exception:
 conn.rollback()
 logger.error(
 "Migration %s failed; nothing from it was applied.", path.name
)
 raise
 newly_applied.append(version)
 return newly_applied
finally:
 conn.close() # also releases the PostgreSQL lock

def baseline(up_to: str) -> list[str]:
 """
 Record every migration up to and including `up_to` as applied, without
 running it – for a database whose schema was built before the runner
 existed (by init_db() and by hand).
 """
 versions = [version for version, _ in available() if version <= up_to]
 if up_to not in versions:
 raise ValueError(f"No migration numbered {up_to}")
 conn = open_db()
 try:
 _lock(conn)

```

```

 _prepare(conn)
 _begin(conn)
 done = _applied(conn)
 cursor = conn.cursor()
 recorded = [v for v in versions if v not in done]
 for version in recorded:
 cursor.execute(
 sql("INSERT INTO schema_migrations (version) VALUES (?)"
 (version,)),
)
 conn.commit()
 return recorded
 finally:
 conn.close()

def main(argv: list[str] | None = None) -> int:
 from app.logging_config import configure_logging

 parser = argparse.ArgumentParser(
 prog="python -m app.migrate", description=__doc__.split("\n\n")[0]
)
 group = parser.add_mutually_exclusive_group()
 group.add_argument(
 "--status", action="store_true", help="show which migrations are applied"
)
 group.add_argument(
 "--baseline",
 metavar="VERSION",
 help="record migrations up to VERSION as applied",
)
 args = parser.parse_args(argv)
 configure_logging()

 if args.status:
 waiting = set(pending())
 for version, path in available():
 print(f'{{pending }} if version in waiting else {{applied }} {{path.name}}')
 return 0
 if args.baseline:
 recorded = baseline(args.baseline)
 print(f'Recorded as applied: {{', '.join(recorded) or 'nothing new'}}")
 return 0
 applied_now = migrate()

```

```

print(
 f"Applied: {' ', ' '.join(applied_now)}"
 if applied_now
 else "Database schema is up to date."
)
return 0

if __name__ == "__main__":
 sys.exit(main())

```

The runner does five things:

- **schema\_migrations is the database’s own record.** It has one row per applied migration. `pending()` compares that record with the files on disk; everything on disk and not in the table is still to do.
- **A migration and its record commit together.** `migrate()` runs a file’s statements *and* inserts its version into `schema_migrations` in the same transaction, then commits once. A crash can therefore never leave a migration “applied but not recorded”, which would make the next run try it a second time. Either both happened, or neither did.
- **Only one runner changes the schema at a time.** Two deploys started at once, or two servers deploying together, must not both apply 0003. On PostgreSQL, `_lock()` takes an **advisory lock**: a lock on a number that the application chooses, held until the connection closes. Every runner asks for the same number, so they queue up. On SQLite, each migration’s `BEGIN IMMEDIATE` takes the database’s single write lock. Inside the lock, the runner checks `schema_migrations` *again*, because another runner may have applied the migration while this one waited.
- **Everything that writes happens under the lock, including creating `schema_migrations` itself.** That line exists because of a real bug, found by starting six runners at once against an empty PostgreSQL database. `CREATE TABLE IF NOT EXISTS` sounds safe to run concurrently, but it is not: two sessions can both see “not there” and both try to create it, and one fails with a `UniqueViolation`. For the same reason `pending()` never creates anything. Every Gunicorn worker calls it at startup, several at the same moment, so it only reads. When the bookkeeping table does not exist yet, it simply reports every migration as pending.
- **SQLite needs one statement at a time.** `psycopg2` happily runs a whole file in one `execute()`, but Python’s `sqlite3` refuses more than one statement per call. `_statements()` splits the file, and uses `sqlite3.complete_statement()` to find where

each statement ends, so a `;` inside a quoted string does not fool it.

Using it:

```
$ python -m app.migrate --status
pending 0001_create_notes.sql
pending 0002_add_updated_at.sql
pending 0003_add_tags.sql

$ python -m app.migrate
2024-01-15 14:00:05.474 INFO app.migrate | Applying migration
↳ 0001_create_notes.sql
2024-01-15 14:00:05.476 INFO app.migrate | Applying migration
↳ 0002_add_updated_at.sql
2024-01-15 14:00:05.476 INFO app.migrate | Applying migration
↳ 0003_add_tags.sql
Applied: 0001, 0002, 0003

$ python -m app.migrate
Database schema is up to date.
```

Running it twice is safe. The second run finds nothing pending and changes nothing, so a deploy can run it every time without thinking.

### Baselining a database that already exists

The empty database above is the easy case. Your development database and the production database already *have* a notes table with an `updated_at` column: `init_db()` built the first, and section 33.9's `psql` command added the second. They do not have a `schema_migrations` table yet, so the runner believes nothing has been applied. Run `migrate` against one of them, and `0001` fails immediately:

```
psycopg2.errors.DuplicateTable: relation "notes" already exists
```

That is harmless: the transaction rolls back, and the database is exactly as it was. The runner simply needs to be told the truth, once:

```
$ python -m app.migrate --baseline 0002
Recorded as applied: 0001, 0002

$ python -m app.migrate
2024-01-15 14:02:11.120 INFO app.migrate | Applying migration
↳ 0003_add_tags.sql
Applied: 0003
```

--baseline records migrations as applied *without running them*. You use it once on each database that existed before the runner (your laptop, staging and production) and never again. Every database created from now on is built by the migrations from 0001. Flyway calls this exact operation “baseline”; Alembic calls it `stamp`.

### `init_db()` retires, and startup checks the schema

With the schema defined by the migrations, `init_db()` and its `CREATE TABLE` statements are deleted from `app/database.py`, and the `init_db(app)` call is deleted from `create_app()`. There is now one source of truth.

Something must take its place at startup, though. Code that starts against a database that is missing a migration would run normally until the first request touches the missing table, and then fail on every such request. The startup checks from [Chapter 14](#) gain one more:

```
app/startup.py (added)
def check_schema():
 """
 Verify that every migration has been applied. The schema belongs to the
 migrations (python -m app.migrate); code that starts on a database that
 is behind it would fail on the first request that needs what is missing.
 """
 from app.migrate import pending

 waiting = pending()
 if waiting:
 _fail(
 f"The database schema is out of date: migration(s) "
 f"{', '.join(waiting)} not applied.\nRun: python -m app.migrate"
)
 logger.info("Database schema: up to date")
```

`run()` calls it straight after `check_database()`. A process that refuses to start fails the deploy’s health check, and the deploy script from [Chapter 23](#) rolls back on its own. That is fail-

fast doing its job.

Why not simply run the migrations at startup, and skip the separate step? It is tempting, and some teams do it. But every Gunicorn worker on every server would start by racing to change the schema. A slow migration would hold up startup until health checks give up. A failing one would put the whole service into a restart loop. A migration is a deliberate, once-per-release step with its own success or failure, not a side effect of starting a process. So it gets its own step in the deploy.

## Migrations in the deployment

The migration files have to travel inside the image, next to the code that expects them. The Dockerfile from [Chapter 18](#) (with [Chapter 24](#)’s `gunicorn.conf.py`) gains one line:

```
COPY --chown=notes:notes app/ ./app/
New: the schema travels with the code that expects it
COPY --chown=notes:notes migrations/ ./migrations/
COPY --chown=notes:notes run.py .
COPY --chown=notes:notes gunicorn.conf.py .
```

The CI smoke test from [Chapter 19](#) starts the new image against a brand-new SQLite file inside the container. That database has no tables, so the application now (correctly) refuses to start, and the smoke test would fail. The smoke test builds the schema first, in the same container, and then starts Gunicorn exactly as the image’s CMD does:

```
- name: Smoke-test the image
 run: |
 # A new, empty database: build the schema, then start as the image would
 docker run -d --name smoke -p 5000:5000 \
 -e DATABASE_URL="sqlite:///notes.db" \
 -e SECRET_KEY="ci-test-secret-key-not-for-production-1234" \
 notes-api:ci-test \
 sh -c "python -m app.migrate && \
 exec gunicorn --config gunicorn.conf.py 'app:create_app()'"
 curl --fail --retry 10 --retry-delay 1 --retry-connrefused \
 http://localhost:5000/health
 docker rm -f smoke
```

As a bonus, every build now proves that the migrations run from `0001` on an empty database, inside the real image. (The `docker-run` target of [Chapter 18](#)’s Makefile needs the same `sh -c "python -m app.migrate && exec ..."` command.)

The deploy script from [Chapter 23](#) gains a step between pulling the new image (Step 2) and

stopping the old container (Step 3):

```

Step 2b: Apply database migrations: from the NEW image, while the OLD
version is still serving

log "Applying database migrations"
docker run --rm --env-file "$ENV_FILE" "$FULL_IMAGE" python -m app.migrate \
 >> "$LOG_FILE" 2>&1 \
 || die "Migrations failed. The previous version is still running, unchanged."
log "Migrations: done"
```

This is a throwaway container (`--rm`). It runs from the *new* image, so it has the new migration files, and it uses the same environment file, so it reaches the same database. If a migration fails, nothing else has happened yet and the old container is still serving. `deploy-bluegreen.sh` gets the same step, before the new colour starts.

Notice what a rollback does *not* do: it does not undo the schema. When the deploy script rolls back to the previous image, that image keeps running on the newer schema. That is why the rule “safe for the code that is already running” matters so much. It is also why most teams do not write “down” migrations at all. Reversing a data change is rarely safe, so they *fix forward* with a new migration instead.

Two smaller places change too. The tests build their database with the same runner, so the schema they test is, file for file, the one production has (section 33.16). And on your laptop, after pulling code that brings a new migration, you run `python -m app.migrate`. If you forget, the startup check tells you.

## 33.11 Growing the Schema: Tags

Chapter 6 designed a grown-up version of the Notes API’s schema on paper, with tags and a join table between notes and tags. Now it is time to build it, and to make the database itself enforce the design.

The feature: a note can carry short labels such as *work*, *urgent* or *to-do*. One note can have many tags, and one tag can label many notes. That is a many-to-many relationship, so the connection needs a table of its own. This is migration **0003**:

```

-- migrations/postgresql/0003_add_tags.sql
-- Tags: short labels such as "work" or "urgent". A note has many tags,
-- and a tag belongs to many notes, so the link is a table of its own.
CREATE TABLE tags (
 id SERIAL PRIMARY KEY,
 name TEXT NOT NULL UNIQUE
 CHECK (length(name) BETWEEN 1 AND 30 AND name = lower(name))
);

CREATE TABLE note_tags (
 note_id INTEGER NOT NULL REFERENCES notes(id) ON DELETE CASCADE,
 tag_id INTEGER NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
 PRIMARY KEY (note_id, tag_id)
);

-- The primary key (note_id, tag_id) already serves "the tags of note 42".
-- "The notes with tag 7" needs an index that starts with tag_id.
CREATE INDEX idx_note_tags_tag_id ON note_tags (tag_id);

```

(The SQLite version differs only in its first column: `id INTEGER PRIMARY KEY AUTOINCREMENT`.)

### Constraints: rules the database enforces

Every line of those two tables is a rule that the database will now enforce against *every* piece of code that ever writes to it:



Constraint	The rule	What it prevents
NOT NULL	the value must be present	a tag with no name; a link to no note
UNIQUE on tags.name	one row per tag name	two different work tags, which would split one label in two
CHECK (...)	1–30 characters, lower case	Work and work becoming different tags; empty or huge names
PRIMARY KEY (note_id, tag_id)	each pair at most once	a note carrying the same tag twice
REFERENCES ... (a foreign key)	the id must exist in the other table	a link to a note that does not exist

The input validation of [Chapter 31](#) checks the same tag rules, so why say them twice? Because the two do different jobs. **Validation is for the client:** it rejects bad input early, with a helpful 400 and a message. **Constraints are for the data:** they hold even for code that forgets to validate, a one-off script, a `psql` session at 2 a.m., or a second service written next year by another team. The application is one visitor to the data. The constraints protect the data from all of them.

## What happens on delete

A foreign key also has to answer a question: what happens to the links when the thing they point at is deleted? `ON DELETE` gives the answer:

<code>ON DELETE</code>	When the note (or tag) is deleted...	Use it when
CASCADE	its links are deleted with it	the child means nothing without the parent, as here
RESTRICT / NO ACTION (the default)	the delete is refused while links exist	the child must never be lost silently, like a user's notes
SET NULL	the link column becomes NULL	the child can live on without the parent

This is a business decision written in SQL. Deleting a note should remove its tags, so `note_tags` uses `CASCADE`. Deleting a *user* who still has notes probably should not quietly delete years of their writing, so a foreign key from notes to users would use `RESTRICT`, or the application would soft-delete instead ([Chapter 6](#)).

**Warning:** SQLite does *not* enforce foreign keys unless each connection switches them on. Without that one line, every REFERENCES clause above is silently ignored, including in your tests. `open_db()` in `app/database.py` gains it:

```
if IS_SQLITE:
 conn = sqlite3.connect(DATABASE_URL[len("sqlite:///") :])
 conn.row_factory = sqlite3.Row
 # SQLite ignores FOREIGN KEY constraints unless each connection
 # switches them on (PostgreSQL always enforces them)
 conn.execute("PRAGMA foreign_keys = ON")
 return conn
```

## Indexes for the new tables

PostgreSQL automatically creates an index for every primary key and every UNIQUE constraint, but **not for foreign keys**. The primary key of `note_tags` is (`note_id`, `tag_id`). An index on two columns works like a phone book sorted by surname and then first name: it finds everyone called Smith quickly, but it does not help you find everyone called John. So it serves “the tags of note 42” and not “the notes with tag 7”, which is why the migration adds a second index that starts with `tag_id`. The rule of thumb: **index every foreign key column you will search by**. Chapter 36 (section 36.2) shows how to read a query plan and prove that an index is used.

Add constraints when you create a table. Adding them later is much harder. SQLite cannot add a constraint to an existing table at all; the table has to be rebuilt and its data copied. PostgreSQL can add one with `ALTER TABLE ... ADD CONSTRAINT`, but it checks every existing row while holding a lock. On a large table, that means adding the constraint `NOT VALID` first and validating it in a second step. Rules written in one day one cost nothing.

## The tags API

Three endpoints expose the feature:

Request	Meaning	Response
GET /notes/42/tags	the tags of note 42	{"tags": ["urgent", "work"]}
PUT /notes/42/tags with {"tags": ["work", "urgent"]}	replace note 42's tags	{"tags": ["urgent", "work"]}
GET /tags/work/notes	the notes tagged work, newest first	{"notes": [...], "count": 2}

PUT replaces the *whole set* rather than adding or removing one tag at a time. That makes it idempotent (section 34.10): sending the same request twice leaves the same result. It is also simpler for clients, which just send the tags they want. {"tags": []} removes them all.

The Validator from Chapter 31 gains a method for lists of short strings. It cleans each item, removes duplicates, and checks every item against a pattern:

```
app/validation.py (added to the Validator class; needs "import re" at the top)
def require_string_list(
 self,
 field: str,
 *,
 max_items: int = 10,
 max_length: int = 30,
 allowed: tuple[re.Pattern, str] | None = None,
) -> list[str] | None:
 """
 Extract and validate a required list of strings.

 Each item is stripped and lower-cased, and duplicates are removed.
 `allowed` is an optional (pattern, description) pair that every item
 must match in full. Returns the items sorted, or None if validation
 failed. An empty list is valid: it means "none".
 """
 if self.require_dict() is None:
 return None

 if field not in self.data:
 self._add_error(f"'{field}' is required")
 return None

 value = self.data[field]
 if not isinstance(value, list) or not all(
```

```

 isinstance(item, str) for item in value
):
 self._add_error(f"'{field}' must be a list of strings")
 return None

 items = sorted({item.strip().lower() for item in value})

 if len(items) > max_items:
 self._add_error(f"'{field}' cannot have more than {max_items} items")
 return None
 if any(not 1 <= len(item) <= max_length for item in items):
 self._add_error(
 f"Each of '{field}' must be 1 to {max_length} characters long"
)
 return None
 if allowed and any(not allowed[0].fullmatch(item) for item in items):
 self._add_error(f"Each of '{field}' may contain only {allowed[1]}")
 return None

 return items

```

The handlers themselves are the subject of the next two sections, because each one teaches something different. Reading tags teaches **joins**. Writing them teaches **transactions under concurrency**.

## 33.12 Querying Across Tables

Once data is split across tables, which is exactly what normalization asks for (Chapter 6), almost every useful question needs several tables at once. “Which notes are tagged work?” touches notes, note\_tags and tags. The tool that puts them back together is the **join**.

### JOIN: combining rows that match

```

app/routes/tags.py
import logging

from flask import Blueprint, jsonify

from app.database import get_db, row_to_note, sql

logger = logging.getLogger(__name__)

```

```

tags_bp = Blueprint("tags", __name__, url_prefix="/tags")

@tags_bp.route("/<name>/notes", methods=["GET"])
def notes_with_tag(name: str):
 """The notes that carry one tag, newest first: {"notes": [...], "count": n}."""
 cursor = get_db().cursor()
 cursor.execute(
 sql("""
 SELECT n.id, n.content, n.created_at
 FROM notes n
 JOIN note_tags nt ON nt.note_id = n.id
 JOIN tags t ON t.id = nt.tag_id
 WHERE t.name = ?
 ORDER BY n.id DESC
 """),
 (name.lower(),),
)
 notes = [row_to_note(row) for row in cursor.fetchall()]
 # An unknown tag is not an error: it simply has no notes
 return jsonify({"notes": notes, "count": len(notes)}), 200

```

(create\_app() registers the new blueprint next to the other two: app.register\_blueprint(tags\_bp).)

The letters *n*, *nt* and *t* are **aliases**, short names for the tables within this query. A JOIN ... ON produces one combined row for every pair of rows that satisfy the ON condition. It helps to see it with real rows:

notes		note_tags		tags	
id	content	note_id	tag_id	id	name
1	Buy milk	1	2	1	work
2	Ship the report	2	1	2	home
3	Call the bank	2	2		

```

notes JOIN note_tags ON nt.note_id = n.id JOIN tags ON t.id = nt.tag_id:
n.id n.content t.name
1 Buy milk home
2 Ship the report work
2 Ship the report home

```

(note 3 has no links, so no rows)

... WHERE t.name = 'work' → one row: note 2

Two things in that picture are worth remembering. A note appears once *per matching tag*, so joins can multiply rows. And a note with no tags does not appear at all, because a plain JOIN (an **inner join**) keeps only rows that found a partner.

### LEFT JOIN: keeping rows without a partner

Sometimes the rows without a partner are exactly the ones you need. “The tags of note 42” must tell apart two situations: *note 42 has no tags* and *there is no note 42*. That is the job of a **LEFT JOIN**, which keeps every row from the table on the left and fills the missing side with **NULL**:

```

app/routes/notes.py (added)
@notes_bp.route("/<int:note_id>/tags", methods=["GET"])
def get_note_tags(note_id: int):
 """The tags of one note: {"tags": ["urgent", "work"]}."""
 cursor = get_db().cursor()
 # LEFT JOIN keeps the note's row even when it has no tags (name is then
 # NULL), so "no such note" (no rows) and "no tags" (one NULL row) differ
 cursor.execute(
 sql("""
 SELECT t.name
 FROM notes n
 LEFT JOIN note_tags nt ON nt.note_id = n.id
 LEFT JOIN tags t ON t.id = nt.tag_id
 WHERE n.id = ?
 ORDER BY t.name
 """),

```

```

 (note_id,),
)
 rows = cursor.fetchall()
 if not rows:
 return jsonify({"error": f"Note {note_id} not found"}), 404
 return (
 jsonify({"tags": [row["name"] for row in rows if row["name"] is not None]}),
 200,
)

```

For note 3 in the example, the result is one row whose name is NULL, which becomes {"tags": []}. For a note that does not exist, the result is no rows at all, which becomes a 404. One query answers both questions.

## Aggregates: counting and grouping

Some questions are about groups rather than rows: “how many notes use each tag?” GROUP BY collapses rows that share a value into one row per group, and **aggregate functions** such as COUNT, SUM, MIN, MAX and AVG compute a value over each group. Queries like these are what you run in psql to understand your data, or behind an admin report:

```

-- How many notes use each tag, including tags no note uses any more
SELECT t.name, COUNT(nt.note_id) AS notes
FROM tags t
LEFT JOIN note_tags nt ON nt.tag_id = t.id
GROUP BY t.name
ORDER BY notes DESC, t.name;

-- Only the popular ones: HAVING filters groups, as WHERE filters rows
SELECT t.name, COUNT(*) AS notes
FROM tags t
JOIN note_tags nt ON nt.tag_id = t.id
GROUP BY t.name
HAVING COUNT(*) >= 10;

-- Notes with no tags at all: NOT EXISTS asks "is there no matching row?"
SELECT n.id, n.content
FROM notes n
WHERE NOT EXISTS (SELECT 1 FROM note_tags nt WHERE nt.note_id = n.id);

```

The first query shows a detail that trips everyone up once. COUNT(\*) counts rows, but COUNT(nt.note\_id) counts only the rows where nt.note\_id is not NULL. A tag that no note

uses still gets one row from the `LEFT JOIN`, with `NULL` on the `note_tags` side. So `COUNT(*)` would report 1 for it, while `COUNT(nt.note_id)` correctly reports 0.

It also helps to know the order in which the database *logically* evaluates a query, because it is not the order in which the query is written:

FROM / JOIN	→	WHERE	→	GROUP BY	→	HAVING	→	SELECT	→	ORDER BY	→	LIMIT
(build rows)		(filter)		(group rows)		(filter groups)		(compute columns)		(sort)		(cut)

This explains rules that otherwise look arbitrary. `WHERE` cannot use `COUNT(*)`, because the groups do not exist yet when `WHERE` runs; that is what `HAVING` is for. And `ORDER BY` *can* use the alias `notes` from the `SELECT` list, because it runs afterwards.

## The N+1 problem

Suppose the list endpoint returned every note *with* its tags. The natural first version asks for the notes, then asks for each note's tags inside a loop:

```
cursor.execute("SELECT id, content, created_at FROM notes ORDER BY id DESC")
notes = [row_to_note(row) for row in cursor.fetchall()]
for note in notes: # one more query for EVERY note
 cursor.execute(
 sql("SELECT t.name FROM tags t JOIN note_tags nt ON nt.tag_id = t.id "
 "WHERE nt.note_id = ?"),
 (note["id"],),
)
 note["tags"] = [row["name"] for row in cursor.fetchall()]
```

For  $N$  notes, that is  $1 + N$  queries: the **N+1 problem**. On a laptop, each query takes a fraction of a millisecond and nobody notices. In production, each one is a network round trip to the database, and a page of 100 notes becomes 101 round trips. The fix is to ask for all the tags of all the notes *at once*, and sort them out in Python:

```
def attach_tags(cursor, notes: list[dict]) -> None:
 """Add a "tags" list to every note, with one query for all of them."""
 if not notes:
 return
 placeholders = ", ".join("? " for _ in notes)
 cursor.execute(
 sql(f"""
```



```

 SELECT nt.note_id, t.name
 FROM note_tags nt
 JOIN tags t ON t.id = nt.tag_id
 WHERE nt.note_id IN ({placeholders})
 ORDER BY t.name
 """),
 [note["id"] for note in notes],
)
tags_by_note: dict[int, list[str]] = {note["id"]: [] for note in notes}
for row in cursor.fetchall():
 tags_by_note[row["note_id"]].append(row["name"])
for note in notes:
 note["tags"] = tags_by_note[note["id"]]

```

That is two queries, however many notes there are. Notice how the `IN (...)` list is built. Only the *placeholders* (`?, ?, ?`) go into the SQL text. The ids themselves still travel as parameters, so this is not the string-building that [Chapter 31](#) forbids. [Chapter 39](#) shows a single-query PostgreSQL version, and section 36.2 measures what N+1 costs.

### 33.13 Transactions in Depth: When Requests Collide

Section 33.5 introduced transactions for one request working alone. Real servers are never alone. Several Gunicorn workers handle requests *at the same time*, and two of them can easily touch the same row: a user with two browser tabs, a phone app retrying after a timeout, two colleagues editing one note. This section is about what happens then.

#### Several statements, one unit

Here is the handler that sets a note's tags. It needs three steps:

```

app/routes/notes.py (added; needs "import re" at the top, and for_update
imported from app.database)
A tag: lowercase letters and digits, with single hyphens between them ("to-do")
TAG_NAME = re.compile(r"[a-z0-9]+(?:-[a-z0-9]+)*")

@notes_bp.route("/<int:note_id>/tags", methods=["PUT"])
def set_note_tags(note_id: int):
 """Replace a note's tags: PUT {"tags": ["work", "urgent"]}; [] removes all."""
 validator = Validator(request.get_json(silent=True))
 names = validator.require_string_list(

```

```

 "tags",
 max_items=10,
 max_length=30,
 allowed=(TAG_NAME, "lowercase letters, digits and single hyphens"),
)
 if names is None: # validation failed
 return validator.error_response()

 db = get_db()
 cursor = db.cursor()

 # 1. Lock the note: a second PUT for the same note waits here until this
 # transaction ends (section 33.13)
 cursor.execute(for_update(sql("SELECT id FROM notes WHERE id = ?")), (note_id,))
 if cursor.fetchone() is None:
 return jsonify({"error": f"Note {note_id} not found"}), 404

 # 2. Make sure every tag exists. ON CONFLICT DO NOTHING skips names that
 # are already there, even one that another request inserted a moment ago
 cursor.executemany(
 sql("INSERT INTO tags (name) VALUES (?) ON CONFLICT (name) DO NOTHING"),
 [(name,) for name in names],
)

 # 3. Replace the note's links: remove all, then add the new set
 cursor.execute(sql("DELETE FROM note_tags WHERE note_id = ?"), (note_id,))
 if names:
 # One placeholder per name. Only "?"s are built into the SQL text;
 # the names themselves still travel as parameters.
 placeholders = ", ".join("?" for _ in names)
 cursor.execute(
 sql(f"""
 INSERT INTO note_tags (note_id, tag_id)
 SELECT ?, id FROM tags WHERE name IN ({placeholders})
 """),
 (note_id, *names),
)

 db.commit() # all three steps, or none of them
 logger.info("Set %d tag(s) on note id=%d.", len(names), note_id)
 return jsonify({"tags": names}), 200

```

(INSERT ... SELECT inserts the rows a query returns, which here is one link per requested tag. executemany() runs one statement once for each set of parameters.)

All three steps run in one transaction and commit once, at the end. If step 3 fails, for example because the connection drops, `close_db()` rolls back everything, including the tags that step 2 created. The note keeps its old tags; it never ends up with half of the new ones. This is the atomicity of section 33.5 doing real work.

### The race: two requests, one note

Now take away step 1, the lock, and send two requests for the same note at the same moment. Request A sets ["work"] and request B sets ["home"]. On PostgreSQL, one possible timing looks like this:

Request A: PUT {"tags": ["work"]}      DELETE links of note 42 (none yet)          INSERT link (42, work) (not committed)   COMMIT	Request B: PUT {"tags": ["home"]}      DELETE links of note 42              A's new link is not committed,              so B cannot see it: nothing to delete          INSERT link (42, home)   COMMIT
------------------------------------------------------------------------------------------------------------------------------------------------------------	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

Result: note 42 has BOTH "work" and "home". Neither request asked for that.

Each transaction behaved correctly on its own, yet the combination is wrong. This is a **race condition**. Tests never catch it, because tests send one request at a time. It shows up only in production, only occasionally, and it looks like a bug that cannot happen.

### Isolation levels

How much one transaction sees of another is its **isolation level**. The SQL standard defines four, and PostgreSQL implements three of them:

Level	What a transaction sees	Cost
Read Committed (PostgreSQL's default)	each <i>statement</i> sees what was committed before that statement began	cheapest; races like the one above are possible
Repeatable Read	the whole <i>transaction</i> sees one snapshot, taken at its start	when two transactions update the same row, one fails and must be retried
Serializable	the result is as if the transactions ran one after another	the strongest guarantee; conflicting transactions fail and must be retried

Stronger isolation catches more races, but it does so by *aborting* transactions, which the application must then retry. Most applications keep the default, and handle the few places where concurrency matters with the tools below.

SQLite takes a different approach altogether. A transaction that writes locks the *whole database*, so writers always take turns, and the race above cannot happen on SQLite. That sounds like good news, but it has an unwelcome consequence: **tests that run on SQLite cannot find this kind of bug**. It is one more reason, alongside the search query of section 33.8, why Chapter 39 runs the tests on PostgreSQL.

### Four tools for concurrent writes

**1. Do it in one statement.** A single SQL statement is atomic on its own. `UPDATE counters SET value = value + 1 WHERE id = ?` can never lose an increment. Reading the value into Python, adding one, and writing it back can: two requests read 5, and both write 6.

**2. Lock first (pessimistic).** `SELECT ... FOR UPDATE` locks the rows it returns until the transaction ends. Any other transaction that asks for the same lock waits until then. That is step 1 of `set_note_tags`: the second PUT for note 42 waits at step 1 until the first commits, and then does its whole job on top of the first one's result. The last request wins, cleanly. SQLite has no `FOR UPDATE`, and does not need it, so `app/database.py` keeps the difference out of the handlers, just as `sql()` does:

```
app/database.py (added)
def for_update(query: str) -> str:
 """
 Lock the rows a SELECT returns until the transaction ends (PostgreSQL).
 SQLite has no row locks: a transaction that writes locks the whole
 database, which already makes writers take turns.
 """
 return query if IS_SQLITE else query + " FOR UPDATE"
```

Locks make other requests wait, so hold them briefly. **Never do slow work inside a transaction that holds locks**, such as calling another service over the network (Chapter 37) or anything that waits on a person.

3. **Check at write time (optimistic).** `update_note` (section 33.9) has a quieter version of the race, called a **lost update**. Two people open the same note, both edit it, and both press Save. The second PUT silently replaces the first person’s work. Locking cannot help here, because the two edits happen minutes apart, in two separate requests. The fix is to record *which version* each edit was based on, and to refuse a write based on an old one:

```
-- A "version" column that every update increments
UPDATE notes
SET content = ?, version = version + 1
WHERE id = ? AND version = ? -- the version the client started from
-- 0 rows updated → someone else saved first → 409 Conflict: reload and retry
```

HTTP has standard headers for exactly this (an ETag on the response, and If-Match on the next PUT). Adding it to the Notes API would change the API contract, which section 34.10 discusses, so it stays a sketch here. But recognize the pattern: whenever people edit shared data, lost updates happen unless something like this prevents them.

4. **Let a constraint decide.** Step 2 of `set_note_tags` guards against a race of its own. Two requests both want the tag urgent, which does not exist yet. The obvious code is “SELECT it; if it is missing, INSERT it”, and both requests can run the SELECT before either runs the INSERT. Then both insert, the UNIQUE constraint rejects the second, and that request fails with a 500. The fix is not to check first at all. Let the UNIQUE constraint be the single judge, and tell the database what to do when it objects: `INSERT ... ON CONFLICT (name) DO NOTHING`. Whichever request comes second quietly uses the row the first one created.

## Deadlocks

Locks can also cause a failure of their own. Suppose transaction A locks note 1 and then asks for note 2, while transaction B has locked note 2 and asks for note 1. Each now waits for the other, forever. That is a **deadlock**. PostgreSQL detects it after about a second, aborts one of the two with `ERROR: deadlock detected`, and lets the other finish.

The prevention is dull but effective. When a transaction needs several locks, take them in a consistent order, for example by ascending id, and keep transactions short. When a deadlock or a serialization failure does happen anyway, the right response is to **retry the whole transaction**. Chapter 37 covers retrying safely.

### 33.14 Raw SQL, Query Builders, and ORMs

The Notes API writes its SQL by hand. Many Python teams do not. Most use **SQLAlchemy**, or the ORM built into Django. It is worth knowing what those tools do, and what this chapter's hand-written SQL teaches you about using them well.

There are three levels to choose from:

- **Raw SQL** through the database driver, as in this book. You write the exact SQL that runs.
- **A query builder**, such as SQLAlchemy Core. You build queries from Python objects, and the library writes the SQL for your database's dialect.
- **An ORM** (object-relational mapper), such as SQLAlchemy's ORM or Django's. You define classes, the library maps them to tables, and you work with objects instead of rows.

This is `notes_with_tag` written with SQLAlchemy's ORM:

```
from sqlalchemy import Column, ForeignKey, String, Table, Text, select
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
 pass

note_tags = Table(
 "note_tags",
 Base.metadata,
 Column("note_id", ForeignKey("notes.id", ondelete="CASCADE"), primary_key=True),
 Column("tag_id", ForeignKey("tags.id", ondelete="CASCADE"), primary_key=True),
)
```

```

class Tag(Base):
 __tablename__ = "tags"
 id: Mapped[int] = mapped_column(primary_key=True)
 name: Mapped[str] = mapped_column(String(30), unique=True)

class Note(Base):
 __tablename__ = "notes"
 id: Mapped[int] = mapped_column(primary_key=True)
 content: Mapped[str] = mapped_column(Text)
 tags: Mapped[list[Tag]] = relationship(secondary=note_tags)

The notes tagged "work", newest first (session: an open SQLAlchemy Session)
query = (
 select(Note)
 .join(Note.tags)
 .where(Tag.name == "work")
 .order_by(Note.id.desc())
)
notes = session.scalars(query).all()

```

The trade-offs:

	Raw SQL	ORM
What runs	exactly the SQL you wrote	SQL the library generates from your code
Repetitive code	more: turning rows into dictionaries, building <code>IN</code> lists	much less
Two databases	your job ( <code>sql()</code> , two migration folders)	mostly handled for you
Performance traps	visible in the code	easy to miss: reading <code>note.tags</code> in a loop quietly runs the N+1 queries of section 33.12
Migrations	written by hand (section 33.10)	<i>generated</i> from model changes (Alembic's <code>autogenerate</code> ), then reviewed by a person

Neither choice is wrong. An ORM is the normal choice in a large codebase with many tables. Raw SQL is common in small services, and in code where performance matters and every query should be visible. What matters is that **an ORM does not replace knowing SQL; it depends on it**. The engineers who use ORMs well are the ones who could write the SQL themselves. They look at what the ORM generates (SQLAlchemy prints every statement when the engine is created with `echo=True`), they spot the N+1 loop, and they drop down to plain SQL for the one report query that needs it.

The same goes for migrations. Alembic has the same three parts as this chapter's runner. A `versions/` directory holds the migration files, an `alembic_version` table records what has been applied, and `alembic upgrade head` applies what is missing. Even the baseline has an equivalent: `alembic stamp`. Once you have seen one runner, you have seen them all.

### 33.15 Error Handling During Logic Execution

What happens when the database is unavailable or the query fails?



## Database errors

```
db = get_db()
If PostgreSQL is down: psycopg2.OperationalError, raised here – when
get_db() opens the connection – not at execute()

cursor = db.cursor()
cursor.execute(sql("SELECT * FROM notes WHERE id = ?"), (note_id,))
If the notes table doesn't exist: raises psycopg2.errors.UndefinedTable
If the query violates a constraint: raises psycopg2.IntegrityError
```

These are exceptions. If the handler does not catch them, they propagate up through Flask's call stack.

## Flask's exception handling

An exception that escapes a handler is caught by Flask, which:

1. Looks for an error handler registered for the exception's class
2. Uses it to build the response — for the Notes API, the catch-all from Chapter 16, which logs the traceback and returns a 500

This is the correct behavior. A database error is a server-side fault — the client correctly learns that something went wrong on the server side (500), without learning the internal details (no database schema or error messages in the response body).

The application's handling boundary (Chapter 16) catches these:

```
@app.errorhandler(Exception)
def handle_unexpected_exception(error):
 logger.exception("Unhandled exception in %s %s", request.method, request.path)
 return jsonify({"error": "An unexpected error occurred"}), 500
```

The `logger.exception()` call logs the full traceback, which is visible in `docker logs notes-api` or via the centralized logging system. The response to the client says only “An unexpected error occurred” — no internal details.

## Distinguishing error types

Some errors should be 400 (invalid input from the client), some should be 404 (resource not found), and some should be 5xx (server fault). The handler controls this:

```

def get_note(note_id: int):
 try:
 db = get_db()
 cursor = db.cursor()
 cursor.execute(sql("SELECT ... WHERE id = ?"), (note_id,))
 row = cursor.fetchone()
 except psycopg2.OperationalError as e:
 # Database unreachable – a server-side problem, and probably a
 # temporary one: 503 Service Unavailable rather than a plain 500
 logger.error("Database error fetching note %d: %s", note_id, e)
 return jsonify({"error": "Service temporarily unavailable"}), 503

 if row is None:
 # Note doesn't exist – this is a 404
 return jsonify({"error": f"Note {note_id} not found"}), 404

 # Success – this is a 200
 return jsonify({...}), 200

```

The explicit `try/except` lets the handler control what error message and status code the client sees. Without it, the catch-all handler turns the `psycopg2.OperationalError` into a 500.

For the Notes API, the catch-all handler is acceptable because the error detail comes through in the application logs, and a database being down is an unusual event. In a larger system, you might distinguish between transient errors (503 Service Unavailable, with a `Retry-After` header) and permanent server errors (500).

## Constraint violations

The constraints of section 33.11 add a new kind of database error: `IntegrityError` (in `psycopg2`, subclasses such as `psycopg2.errors.UniqueViolation` and `ForeignKeyViolation`). It is tempting to catch it and return a 400. Resist that. With validation in front of the database, a constraint violation during a request means one of two things. Either there is a race the code did not handle, whose fix is locking or `ON CONFLICT` (section 33.13), or the validator and the constraint disagree about a rule, which is a bug. Either way the fault is on the server's side, so the catch-all handler's 500, with the full traceback in the logs, is the honest response. Catch an `IntegrityError` only where it is an expected outcome that the code has a plan for.

## 33.16 Structuring Logic for Testability

The handler structure shown above — parse, validate, query, return — is easy to test with Flask’s test client:

```
tests/test_notes.py

def test_create_note_success(client):
 response = client.post(
 "/notes/",
 json={"content": "Buy groceries"},
)
 assert response.status_code == 201
 data = response.get_json()
 assert data["content"] == "Buy groceries"
 assert "id" in data
 assert "created_at" in data

def test_create_note_missing_content(client):
 response = client.post("/notes/", json={})
 assert response.status_code == 400
 assert response.get_json() == {"error": "'content' is required"}

def test_update_note(client):
 note_id = client.post("/notes/", json={"content": "Old"}).get_json()["id"]
 response = client.put(f"/notes/{note_id}", json={"content": "New"})
 assert response.status_code == 200
 data = response.get_json()
 assert data["content"] == "New"
 assert data["updated_at"] is not None

def test_get_note_not_found(client):
 response = client.get("/notes/99999")
 assert response.status_code == 404

def test_delete_note(client):
 # First create a note
 create_response = client.post("/notes/", json={"content": "To delete"})
 note_id = create_response.get_json()["id"]

 # Then delete it
 delete_response = client.delete(f"/notes/{note_id}")
 assert delete_response.status_code == 204
```

```
Then verify it's gone
get_response = client.get(f"/notes/{note_id}")
assert get_response.status_code == 404
```

These tests exercise the complete logic path (including the database) through Flask's test client. The test database is the temporary SQLite file configured in `tests/conftest.py` (Chapter 17), emptied before each test.

With the migration runner, `conftest.py` changes in two places. The test database is now built by the same migration files that build production, instead of by `init_db()`. And cleaning up between tests has one more table to empty:

```
tests/conftest.py (changed lines)
from app import create_app # noqa: E402 (must come after the settings above)
from app.migrate import migrate # noqa: E402

@pytest.fixture(scope="session")
def app():
 migrate() # a fresh database: build the schema exactly as production does
 application = create_app()
 ...

@pytest.fixture(scope="function", autouse=True)
def clean_database(app):
 with app.app_context():
 from app.database import get_db

 db = get_db()
 cursor = db.cursor()
 # Deleting the notes also deletes their note_tags rows (ON DELETE CASCADE)
 cursor.execute("DELETE FROM notes")
 cursor.execute("DELETE FROM tags")
 db.commit()

 yield
```

The new features get tests of their own. Some check behaviour through the API, and some reach past it to check that the database's own rules hold:

```

tests/test_tags.py (excerpt)
def make_note(client, content="A note"):
 return client.post("/notes/", json={"content": content}).get_json()["id"]

def test_put_replaces_the_whole_set(client):
 note_id = make_note(client)
 client.put(f"/notes/{note_id}/tags", json={"tags": ["a", "b"]})
 client.put(f"/notes/{note_id}/tags", json={"tags": ["b", "c"]})
 assert client.get(f"/notes/{note_id}/tags").get_json() == {"tags": ["b", "c"]}

def test_deleting_a_note_removes_its_links(client, app):
 note_id = make_note(client)
 client.put(f"/notes/{note_id}/tags", json={"tags": ["work"]})
 client.delete(f"/notes/{note_id}")

 with app.app_context():
 from app.database import get_db

 cursor = get_db().cursor()
 cursor.execute("SELECT COUNT(*) AS n FROM note_tags")
 assert cursor.fetchone()["n"] == 0 # ON DELETE CASCADE did its job

def test_constraints_back_up_the_validation(app):
 """Even code that skips the Validator cannot store a bad tag."""
 import pytest

 with app.app_context():
 from app.database import get_db

 db = get_db()
 cursor = db.cursor()
 # Every DB-API connection carries its driver's exception classes,
 # so db.IntegrityError works on SQLite and PostgreSQL alike
 with pytest.raises(db.IntegrityError): # the CHECK: lower case only
 cursor.execute("INSERT INTO tags (name) VALUES ('Work')")
 db.rollback() # PostgreSQL refuses further statements after an error
 with pytest.raises(db.IntegrityError): # the FOREIGN KEY
 cursor.execute(
 "INSERT INTO note_tags (note_id, tag_id) VALUES (99999, 1)"
)

```

```
tests/test_migrations.py
from app.migrate import available, migrate, pending

def test_every_migration_is_applied():
 assert pending() == []

def test_running_again_changes_nothing():
 assert migrate() == []

def test_versions_are_unique_and_in_order():
 versions = [version for version, _ in available()]
 assert versions == sorted(set(versions))
 assert versions[0] == "0001"
```

Try removing the `PRAGMA foreign_keys = ON` line from `open_db()`. SQLite then ignores `ON DELETE CASCADE`, the link outlives its note, and `test_deleting_a_note_removes_its_links` fails, along with the foreign-key half of `test_constraints_back_up_the_validation`. Those tests exist to catch exactly that mistake.

Notice what these tests cannot cover. The first gap is the PostgreSQL-only search branch of `list_notes`. The second is subtler: the race of section 33.13. On SQLite, writers take turns for the whole database, so the race cannot happen there, and `for_update()` adds nothing. A test suite that runs only on SQLite can never tell you whether the PostgreSQL lock is right. Chapter 39 closes both gaps by running the tests against PostgreSQL.

### Key Takeaways

- **Business logic** is the code that implements the application's domain rules. For the Notes API, it is the database queries that read and write notes.
- **Flask's contexts** (the application context and the request context) use Python's `contextvars` to make `request`, `g`, and `current_app` available as apparent globals that are actually per-request state.
- **`g`** is per-request storage. `get_db()` stores the database connection in `g.db` so it is opened once and reused within a request. Handlers commit before they respond; `teardown_appcontext` only rolls back and closes.
- **Parameterized queries** (`?` placeholders, converted by `sql()`) are mandatory. Interpolating values into SQL strings creates SQL injection vulnerabilities.

- **RETURNING** lets INSERT and UPDATE return the affected row's data in a single round trip, avoiding a follow-up SELECT.
- **Schema changes need migrations.** CREATE TABLE IF NOT EXISTS never changes an existing table; a new column arrives through an ALTER TABLE, run before the code that uses it.
- **Migrations are a system, not a habit.** Numbered files are the only definition of the schema. A schema\_migrations table records what each database has had, and a runner applies the rest, one transaction per migration, as its own step in every deploy, before the new code starts. Never edit an applied migration; fix forward. An existing database is baselined once.
- **Every migration must be safe for the code already running.** The old version keeps serving during the deploy and can be rolled back to, and a rollback does not undo the schema.
- **Constraints protect the data; validation serves the client.** Use NOT NULL, UNIQUE, CHECK, and foreign keys with a deliberate ON DELETE. Index the foreign-key columns you search by. SQLite enforces foreign keys only with PRAGMA foreign\_keys = ON.
- **Joins put normalized data back together.** An inner JOIN keeps only rows with a partner, and a LEFT JOIN keeps them all. GROUP BY with aggregates answers questions about groups, and HAVING filters them. Avoid N+1 queries by fetching related rows for all items at once.
- **Concurrent requests race.** Tools that keep data correct: do it in one statement; lock first (SELECT ... FOR UPDATE); check a version at write time (optimistic concurrency, against lost updates); let a UNIQUE constraint decide (ON CONFLICT). Keep transactions short, take locks in a consistent order, and retry deadlocks and serialization failures.
- **An ORM does not replace SQL; it depends on it.** Know what SQL your tool generates, and when to write it yourself.
- **fetchone() returning None** after an UPDATE or DELETE means no row matched the WHERE clause — the resource wasn't found (404), not a server error (500).
- **HTTP 204 No Content** is the conventional response for successful DELETE operations.
- **Unhandled exceptions** in handlers propagate to the catch-all error handler, which logs the traceback and returns a safe 500 response. Explicit try/except blocks let you control the error message and status code.

## Exercises

1. **Predict.** In two psql sessions, run BEGIN; UPDATE notes SET content = 'A' WHERE id = 1; in one, then the same with 'B' in the other. What does the second session do? What happens when the first commits?
2. **Break and fix.** Write the next migration (0004 at this point) so that it adds a NOT NULL column without a default to notes. Run it against a database with rows. Read the error,

then write it the expand/contract way.

3. **Extend.** Add a GET `/tags/` report: every tag, with the number of notes that carry it. Use a LEFT JOIN and GROUP BY, and test that a tag no longer on any note shows 0. (After Chapter 38 adds accounts, ask what this report would reveal about other users.)
4. **Explain.** Explain why the runner records a migration in the same transaction as the migration itself.

Solutions and hints are in the companion repository, in `exercises/chapter-33.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

The handler has executed its logic and computed a result. Now that result must be converted into an HTTP response and sent back to the client. The process of constructing responses — status codes, headers, body serialization, content negotiation — is the subject of the next chapter.

→ [Chapter 34 — Response Handling](#)



# Response Handling

---

The business logic has run. The handler has a result: a new note, a list of notes, a validation error, a “not found” condition. Now that result must become an HTTP response — a byte stream with a status code, headers, and a body — that travels back through Gunicorn, Nginx, and the network to the client.

This chapter covers how Flask constructs HTTP responses from the return values of handler functions, what the HTTP status codes mean and when to use each one, how headers control client behavior, and how JSON serialization works.

## In this chapter

By the end of this chapter, you will understand:

- How Flask converts handler return values into `Response` objects
- What HTTP status codes mean and the correct codes for common situations
- Which response headers matter and what they communicate
- How JSON serialization works in Flask and what `jsonify()` does
- How to handle the datetime serialization problem
- How error responses differ from success responses (and how they should be structured)
- How Nginx adds headers to Flask’s responses before the client sees them
- How to treat the API as a **contract**: idempotency, versioning, pagination, error design, webhooks, and a written OpenAPI specification that CI keeps honest with a validator and contract tests

## 34.1 From Handler Return Value to HTTP Response

Flask handler functions can return values in several forms. Flask normalizes all of them into a `Response` object.

### Form 1: A tuple (body, status\_code)

The most common form used in the Notes API:

```
return jsonify(note), 201
```

Flask sees a tuple where the first element is a body (or Response) and the second is a status code. If the first element is already a Response object (which `jsonify()` produces), Flask uses it as-is but overrides the status code.

### Form 2: A tuple (body, status\_code, headers)

```
return (
 jsonify({"error": "Rate limit exceeded"}),
 429,
 {"Retry-After": "60"},
)
```

The third element adds or overrides headers in the response.

### Form 3: Just a body

```
return jsonify({"status": "ok"})
Status defaults to 200
```

### Form 4: A dict (or a list)

```
return {"status": "ok"}
Flask converts a returned dict or list to JSON, exactly as jsonify() would
```

### Form 5: A Response object directly

```
from flask import Response
import json

response = Response(
 response=json.dumps({"id": 42}),
 status=201,
 mimetype="application/json",
)
return response
```

This is verbose and rarely necessary — `jsonify()` creates the `Response` object for you.

## Form 6: An empty response

```
return "", 204
Body is empty, status is 204 No Content
```

## Flask's `make_response()` function

If you need to modify a response object before returning it (for example, to set a cookie), use `make_response()`:

```
from flask import make_response, jsonify

response = make_response(jsonify({"id": note_id}), 201)
response.headers["X-Custom-Header"] = "value"
response.set_cookie("session_id", value="abc123", httponly=True, secure=True,
 ↪ samesite="Lax")
return response
```

## What Flask does with the return value

Flask's `finalize_request()` method handles the conversion:

```
Flask internals (simplified):
def finalize_request(self, rv, from_error_handler=False):
 response = self.make_response(rv)
 # ...apply after_request hooks...
 return response

def make_response(self, rv):
 # None → TypeError: "The view function did not return a valid response"
 # (every view must return something, even if it is "", 204)
 # Handle (body, status) tuples
 # Handle (body, status, headers) tuples
 # Handle Response objects directly
 # Wrap plain strings/bytes in a Response
 # ... (many cases)
 return response # Always a Response object
```

After `make_response()`, the `Response` object is passed through `after_request` hooks, then handed to Gunicorn's WSGI layer, which serializes it to HTTP/1.1 bytes and sends them to

Nginx.

## 34.2 HTTP Status Codes

HTTP status codes are three-digit numbers that tell the client what happened with its request. They are divided into five classes:

Class	Range	Meaning
1xx	100–199	Informational — request received, continuing
2xx	200–299	Success — request received, understood, accepted
3xx	300–399	Redirection — further action needed
4xx	400–499	Client error — request has a problem
5xx	500–599	Server error — server failed to fulfill a valid request

### Status codes used by the Notes API

**200 OK** The standard success response for GET requests and PUT/PATCH requests that return the updated resource.

```
return jsonify(note), 200
```

**201 Created** The resource was successfully created. Used for POST requests that create a new resource.

```
return jsonify(note), 201
```

The Location header conventionally points to the new resource's URL. It is optional, and the Notes API's create\_note returns the whole note instead (its id is enough to build the URL); adding it is one line:

```
return jsonify(note), 201, {"Location": url_for("notes.get_note",
↳ note_id=note["id"])}
```

**204 No Content** Success, but no response body. Used for DELETE operations where there is nothing meaningful to return.

```
return "", 204
```

**301 Moved Permanently / 308 Permanent Redirect** The resource has moved to a new URL permanently. Clients should update their bookmarks. Used for HTTP → HTTPS redirects and URL canonicalization. The difference matters for APIs: after a 301, many clients repeat a POST as a GET (dropping the body); a 308 requires them to repeat the same method and body. Werkzeug’s trailing-slash redirect is a 308 (Chapter 32).

**302 Found / 307 Temporary Redirect** The resource is temporarily at a different URL. Clients should continue using the original URL for future requests. Used for login redirects. 307 is the method-preserving version of 302.

**400 Bad Request** The request is malformed or invalid — the body is not valid JSON, or its data fails validation. The Notes API uses 400 for every validation failure.

```
return jsonify({"error": "Request body must be a JSON object"}), 400
```

**415 Unsupported Media Type** The body is in a format the server does not accept — for the Notes API, anything not declared as `application/json` (the middleware check from Chapter 16).

**401 Unauthorized** (actually means “unauthenticated”) The client is not authenticated. It must provide credentials. Used when an API requires authentication and the client has not sent a token.

```
return jsonify({"error": "Authentication required"}), 401
```

**403 Forbidden** The client is authenticated but not authorized for this action. The server understood the request but refuses it.

```
return jsonify({"error": "You do not have permission to delete this note"}), 403
```

**404 Not Found** The requested resource does not exist.

```
return jsonify({"error": f"Note {note_id} not found"}), 404
```

**405 Method Not Allowed** The URL exists but does not support the HTTP method used. Flask generates this automatically, with an `Allow` header listing the methods that work.

**409 Conflict** The request conflicts with the current state of the resource — for example, trying to create a resource that already exists (unique constraint violation).

```
return jsonify({"error": "A note with this content already exists"}), 409
```

**422 Unprocessable Content** (called “Unprocessable Entity” before RFC 9110) The request is well-formed (valid JSON, correct Content-Type) but its data fails validation. Some APIs use 422 for that case and keep 400 for malformed requests; others, like the Notes API, use 400 for both. Either is defensible — what matters is choosing one and using it everywhere.

**429 Too Many Requests** The client has exceeded the rate limit. Include a `Retry-After` header.

```
return jsonify({"error": "Rate limit exceeded"}), 429, {"Retry-After": "60"}
```

**500 Internal Server Error** An unexpected error occurred on the server. This is the catch-all for server bugs. The client should not retry without waiting.

```
return jsonify({"error": "An unexpected error occurred"}), 500
```

**503 Service Unavailable** The server is temporarily unable to handle requests (overloaded, database down, maintenance). Include a `Retry-After` header if possible.

```
return jsonify({"error": "Service temporarily unavailable"}), 503
```

## Choosing the right status code

The most common mistakes:

- Using 200 for errors (always use a 4xx or 5xx for errors)
- Mixing 400 and 422 for the same kind of validation failure (pick one)
- Using 500 for resource-not-found (use 404)
- Using 404 for authentication failures (use 401)
- Using 401 for authorization failures (use 403)

The distinction between 401 and 403 matters for security. A 401 says “tell me who you are.” A 403 says “I know who you are, and you can’t do this.” But a 403 also confirms that the resource *exists*. When its existence is itself private — another user’s note, a private repository — return **404** instead, as if it did not exist (GitHub does exactly this). Use 403 when existence is no secret and the client simply lacks permission.

## 34.3 Response Headers

HTTP response headers communicate metadata about the response to the client.

### Headers Flask sets automatically

Flask’s `jsonify()` function produces a `Response` that automatically includes:

```
Content-Type: application/json
Content-Length: <length of the response body in bytes>
```

The `Content-Type` tells the client how to interpret the body. `application/json` means “this is JSON-encoded data.” Without this header, a client might not know how to parse the body.

Flask itself does not add a `Server` header — the server running it does. The development server adds `Server: Werkzeug/3.0.1 Python/3.11.8`; Gunicorn adds `Server: gunicorn`. In production, Nginx replaces it with its own `Server: nginx` (and `server_tokens off`; in Nginx hides its version number).

### Headers Nginx adds

Chapter 28 configured Nginx to add security headers to every response:

```
Strict-Transport-Security: max-age=63072000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
```

These headers instruct browsers on security policy. Flask doesn’t need to set them — Nginx adds them uniformly to all responses.

### Headers you should set in Flask

**Cache-Control:** Tell clients and proxies how long to cache responses.

```
For resources that change frequently (like a note list):
response.headers["Cache-Control"] = "no-store"

For data that may be cached by the client itself, but never by shared
caches (proxies, CDNs) – anything that belongs to one user:
response.headers["Cache-Control"] = "private, max-age=60"

"public, max-age=3600" is only for data that is the same for everyone
and never changes, such as a versioned static file
```

**Location:** After a 201 Created, point to the new resource's URL.

```
response.headers["Location"] = url_for("notes.get_note", note_id=note_id,
↳ _external=True)
```

**Retry-After:** For 429 and 503, tell the client when to retry.

```
response.headers["Retry-After"] = "60" # Seconds to wait
```

**ETag and Last-Modified:** For cache validation (conditional requests). Not implemented in the basic Notes API, but worth knowing:

```
Client's first request:
Server response: ETag: "abc123"

Client's second request:
Client sends: If-None-Match: "abc123"
Server checks: has the note changed?
No → return 304 Not Modified (empty body, client uses cached copy)
Yes → return 200 OK with new ETag
```

## Content-Type for different response types

Most Notes API responses return JSON. But the Content-Type must match the actual body:

```
JSON response
return jsonify(data), 200
Content-Type: application/json

Plain text response
from flask import Response
```



```

return Response("pong", mimetype="text/plain"), 200
Content-Type: text/plain

HTML response
return render_template("index.html"), 200
Content-Type: text/html; charset=utf-8

CSV export – use the csv module: note content may contain commas,
quotes, and newlines, which hand-built CSV would get wrong
import csv
import io
from flask import Response

buffer = io.StringIO()
writer = csv.writer(buffer)
writer.writerow(["id", "content", "created_at"])
for note in notes:
 writer.writerow([note["id"], note["content"], note["created_at"]])
(Spreadsheet programs treat cells starting with =, +, - or @ as formulas;
prefix such values with ' if the file will be opened in one.)
return Response(
 buffer.getvalue(),
 mimetype="text/csv",
 headers={"Content-Disposition": "attachment; filename=notes.csv"},
), 200

```

## 34.4 JSON Serialization

`jsonify()` serializes through Flask's **JSON provider** (`DefaultJSONProvider`), which builds on Python's `json` module. The `json` module handles:

- Dicts → JSON objects `{}`
- Lists → JSON arrays `[]`
- Strings → JSON strings `""`
- Integers and floats → JSON numbers
- True/False → `true/false`
- None → `null`

Flask's provider adds a few types plain `json.dumps()` rejects: `datetime` and `date` objects (as HTTP-date strings), `Decimal` and `UUID` (as strings), and `dataclasses` (as objects). Anything else — your own classes, for example — raises `TypeError`.

## The datetime problem

PostgreSQL returns datetime objects for timestamp columns:

```
row["created_at"] # datetime(2024, 1, 15, 14, 0, 0, tzinfo=timezone.utc)
```

If you pass this directly to `jsonify()`, it does not fail — but the format is not the one an API wants:

```
return jsonify({"created_at": row["created_at"]}), 200
{"created_at": "Mon, 15 Jan 2024 14:00:00 GMT"}
(An HTTP-date string: no fractional seconds, awkward for clients to parse.
Plain json.dumps() would raise TypeError instead.)
```

### Solution 1: Convert to ISO 8601 string in the handler

```
return jsonify({
 "created_at": row["created_at"].isoformat(),
}), 200
"created_at": "2024-01-15T14:00:00.123456+00:00"
```

The Notes API does this in one place: `row_to_note()` in `app/database.py`, through `to_iso()` (Chapter 15).

ISO 8601 (YYYY-MM-DDTHH:MM:SS+HH:MM) is the standard format for representing timestamps in JSON APIs. The `+00:00` suffix indicates UTC. Most programming languages and HTTP clients can parse this format.

### Solution 2: Custom JSON provider

For a consistent approach across all responses, define a custom JSON provider. Flask 2.3+ replaced the old `JSONEncoder` API with a `JSONProvider` system that is more composable:

```
app/json_provider.py
from datetime import date, datetime
from flask.json.provider import DefaultJSONProvider

class APIJSONProvider(DefaultJSONProvider):
 """
 JSON provider that writes dates and times as ISO 8601.
 Registered once at app creation; applies to every jsonify() call automatically.
 """
 def default(self, obj):
 if isinstance(obj, (datetime, date)):
 return obj.isoformat() # "2024-01-15T14:00:00.123456+00:00"
 # Everything else as before: Decimal and UUID become strings,
 # unsupported types raise TypeError
 return super().default(obj)
```

Register it with the Flask app during creation:

```
app/__init__.py
from flask import Flask
from app.json_provider import APIJSONProvider

def create_app():
 app = Flask(__name__)
 app.json = APIJSONProvider(app) # replaces the default provider
 # ...
```

Now `datetime` objects serialize as ISO 8601 everywhere:

```
return jsonify({"created_at": row["created_at"]}), 200
{"created_at": "2024-01-15T14:00:00.123456+00:00"}
```

**Historical note (Flask ≤ 2.2):** Older Flask versions used `app.json_encoder`. If you maintain a legacy codebase you may encounter this pattern — it was deprecated in Flask 2.2 and removed in 2.3. On Flask 3.x the assignment below still runs, but nothing reads the attribute, so the encoder is **silently ignored**:

```
LEGACY – do not use in Flask 2.3+
import json
from datetime import datetime

class APIJSONEncoder(json.JSONEncoder):
 def default(self, obj):
 if isinstance(obj, datetime):
 return obj.isoformat()
 return super().default(obj)

app.json_encoder = APIJSONEncoder # ignored on Flask 2.3+
```

## Timezone awareness

PostgreSQL timestamps can be:

- **With timezone** (TIMESTAMPTZ): stored as UTC, returned in the session’s TimeZone setting — UTC on managed databases by default; set it explicitly if in doubt — 2024-01-15T14:00:00+00:00
- **Without timezone** (TIMESTAMP): stored as-is, no UTC conversion — 2024-01-15T14:00:00

Always use TIMESTAMPTZ for API-facing timestamps. Clients in different timezones need to know the reference timezone.

```
-- The notes table after migrations 0001 and 0002 (Chapter 33)
CREATE TABLE notes (
 id SERIAL PRIMARY KEY,
 content TEXT NOT NULL,
 created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMPTZ
);
```

## 34.5 Error Response Structure

Error responses need a consistent structure so that API clients can handle errors programmatically.

## Inconsistent error responses (bad)

```
404 response: {"error": "Note not found"}
422 response: {"errors": ["content is required"]}
400 response: "Bad request"
500 response: {"message": "Something went wrong"}
```

These four responses use different keys (`error`, `errors`, no key, `message`) and different value types (string, list of strings, plain string). A client cannot write consistent error handling code for these. It is an easy trap to fall into: an application grows one handler at a time, and each new piece of code invents a shape. Early drafts of this book's own Notes API did exactly this before Chapter 16 settled on one.

## Consistent error responses (good)

Choose one structure and use it everywhere:

```
{
 "error": {
 "code": "NOT_FOUND",
 "message": "Note not found",
 "details": []
 }
}
```

Or a simpler flat structure:

```
{
 "error": "Note not found"
}
```

Or the flat structure, extended with an optional list for requests that fail in several ways at once:

```
Single error: {"error": "Note 42 not found"}
Several validation errors: {"error": "Invalid request",
 "details": ["'content' is required",
 "'priority' is required"]}
```

The Notes API uses this last structure: every error response has an `"error"` string, so a client can always show *something*, and `"details"` appears only when there is more to say. Whatever

you choose, be consistent.

## Defining error helpers

```
app/responses.py

from flask import jsonify

def error_response(message: str, status_code: int, details: list[str] | None = None):
 """Return an error response in the API's one error shape."""
 body = {"error": message}
 if details:
 body["details"] = details
 return jsonify(body), status_code

def not_found_response(resource: str, resource_id: int):
 """Return a standard 404 response, e.g. {"error": "Note 42 not found"}."""
 return error_response(f"{resource} {resource_id} not found", 404)
```

(The Validator from Chapter 31 builds its 400 responses in the same shape.) Using the helper:

```
from app.responses import not_found_response

@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id: int):
 cursor = get_db().cursor()
 cursor.execute(sql("SELECT id, content, created_at FROM notes WHERE id = ?"),
 ↪ (note_id,))
 row = cursor.fetchone()
 if row is None:
 return not_found_response("Note", note_id)
 return jsonify(row_to_note(row)), 200
```

## 34.6 Content Negotiation

**Content negotiation** is the mechanism where the client tells the server what content formats it can accept, and the server chooses the best match.

The client sends an Accept header:

```
Accept: application/json, text/html;q=0.9, */*;q=0.8
```

This says: “I prefer application/json (quality=1.0, the default). I’ll also accept text/html (quality=0.9). I’ll accept anything else (quality=0.8).”

Flask provides `request.accept_mimetypes` to read this:

```
from flask import render_template, request

@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id: int):
 cursor = get_db().cursor()
 cursor.execute(sql("SELECT id, content, created_at FROM notes WHERE id = ?"),
 ↪ (note_id,))
 row = cursor.fetchone()
 if row is None:
 return not_found_response("Note", note_id)

 note = row_to_note(row)

 # Content negotiation
 best = request.accept_mimetypes.best_match(["application/json", "text/html"])
 if best == "text/html":
 return render_template("note.html", note=note), 200
 else:
 return jsonify(note), 200
```

For a pure API (no HTML), content negotiation is simple: always return JSON, and document that in your API specification. The Notes API is a pure JSON API, so `jsonify()` is always appropriate.

## 34.7 After-Request Hooks

After the handler returns, Flask runs `after_request` hooks before sending the response. These hooks can inspect and modify the response:

```
app/middleware.py – inside register_middleware(app), next to Chapter 16's hooks

@app.after_request
def add_common_headers(response):
 """Add headers that should appear on every response."""
 # Disable caching for API responses by default
 if "Cache-Control" not in response.headers:
 response.headers["Cache-Control"] = "no-store"
 return response
```

The other `after_request` hook in the Notes API is Chapter 16's `log_request_end`, which adds the `X-Response-Time-Ms` header and logs the status and duration. (Register each hook once: a second copy of `log_request_end` elsewhere would log every request twice.)

**Important:** `after_request` hooks must return the response object. If they don't, Flask raises an error. The hook can return a modified version of the response, or the original, but it must return *something*.

`after_request` hooks run in reverse registration order (last registered runs first).

## Teardown vs after\_request

`teardown_appcontext` (which calls `close_db`) runs **after** `after_request` hooks, and it always runs. `after_request` hooks run for every response Flask produces — including the ones built by error handlers. They are skipped only when an exception escapes *all* error handlers; in the Notes API, with its catch-all handler, that does not happen.

```
Request arrives
↓
before_request hooks
↓
Handler executes → returns response
↓
after_request hooks ← modifies response
↓
Response sent to client
↓
teardown_appcontext ← closes DB, releases resources
```

If the handler raises an unhandled exception:



```

Request arrives
↓
before_request hooks
↓
Handler executes → raises Exception
↓
Flask catches exception → calls error handler → returns error response
↓
after_request hooks (run with the error response)
↓
Response sent to client
↓
teardown_appcontext (error is None – an error handler dealt with the
 exception; it is set only for exceptions no handler
 caught. This is why close_db() must never decide
 between commit and rollback based on it.)

```

## 34.8 The Complete Response Journey

Let's trace the complete response for a successful POST /notes/ request:

1. Handler returns:
 

```

return jsonify(note), 201
where note = {"id": 42, "content": "Buy groceries",
 "created_at": "2024-01-15T14:00:00.123456+00:00"}

```
2. Flask's `make_response()` wraps this into a `Response` object:
 

```

status: 201 Created
headers: Content-Type: application/json
 Content-Length: 84
body: b'{"content":"Buy
↪ groceries","created_at":"2024-01-15T14:00:00.123456+00:00","id":42}\n'
(jsonify sorts the keys, and ends the body with a newline: 83 + 1 bytes)

```
3. `after_request` hooks run:
 

```

add_common_headers() adds:
 Cache-Control: no-store
log_request_end() (Chapter 16) adds X-Response-Time-Ms and logs:
 "← 201 in 14.3ms"

```
4. Flask returns the `Response` object to Unicorn's WSGI wrapper.

5. Unicorn serializes the Response to HTTP/1.1 bytes:  
 HTTP/1.1 201 Created\r\n
 Server: gunicorn\r\n
 Date: Mon, 15 Jan 2024 14:00:00 GMT\r\n
 Connection: close\r\n
 Content-Type: application/json\r\n
 Content-Length: 84\r\n
 X-Response-Time-Ms: 14.3\r\n
 Cache-Control: no-store\r\n
 \r\n
 {"content":"Buy  
 ↪ groceries","created\_at":"2024-01-15T14:00:00.123456+00:00","id":42}
6. Unicorn writes these bytes to the socket connected to Nginx.
7. Nginx receives the bytes.  
 Nginx adds security headers:  
 Strict-Transport-Security: max-age=63072000; includeSubDomains  
 X-Content-Type-Options: nosniff  
 X-Frame-Options: DENY  
 Nginx removes: Server: gunicorn (Nginx hides upstream server identity)  
 Nginx adds: Server: nginx
8. Nginx encrypts the response using the TLS session key.
9. Nginx sends the encrypted bytes to the client's TCP socket.
10. Client decrypts and receives (security headers abbreviated):  
 HTTP/2 201 Created  
 server: nginx  
 date: Mon, 15 Jan 2024 14:00:00 GMT  
 content-type: application/json  
 content-length: 84  
 x-response-time-ms: 14.3  
 cache-control: no-store  
 strict-transport-security: max-age=63072000; includeSubDomains  
 x-content-type-options: nosniff  
 x-frame-options: DENY  
 {"content":"Buy  
 ↪ groceries","created\_at":"2024-01-15T14:00:00.123456+00:00","id":42}

The response the client sees has contributions from three layers:

- **Flask:** Content-Type, Content-Length, Cache-Control and X-Response-Time-Header (via `after_request`), the JSON body
- **Gunicorn:** Server: gunicorn (before Nginx removes it), Date, Connection (HTTP/2 has no Connection header, so Nginx drops it)
- **Nginx:** Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options, Server: nginx

## 34.9 Pagination for List Endpoints

The `list_notes` handler returns all notes at once. For large datasets, this is impractical — a database with 100,000 notes should not return all 100,000 in one response.

**Pagination** breaks large result sets into pages. The two common approaches: **offset-based** (“page 3, 20 per page”) and **cursor-based** (“the 20 notes after note 1234”).

Both keep the list response’s envelope from [Chapter 15](#) — `{"notes": [...], "count": n}` — and only add fields to it, so existing clients can still *parse* the response. They do not get the same *data*, though: a client that expected every note now receives only the first page. That is acceptable here only because the Notes API has no outside clients yet (section 34.10 comes back to this). Both listings also leave out the `?search=` filter of section 33.8, to keep them short; it combines with either style as one more `WHERE` condition.

### Offset-based pagination

```
@notes_bp.route("/", methods=["GET"])
def list_notes():
 # Read pagination parameters from the query string.
 # (type=int falls back to the default on bad input, such as page=abc;
 # to answer 400 instead, validate as in Chapter 31, section 31.4.)
 page = request.args.get("page", 1, type=int)
 per_page = request.args.get("per_page", 20, type=int)

 # Validate and clamp
 page = max(1, page)
 per_page = max(1, min(100, per_page)) # Between 1 and 100

 offset = (page - 1) * per_page

 cursor = get_db().cursor()

 # Total count (for the client to know how many pages there are)
```

```

 cursor.execute("SELECT COUNT(*) AS total FROM notes")
 total = cursor.fetchone()["total"]

 # Paginated results
 cursor.execute(
 sql("SELECT id, content, created_at FROM notes ORDER BY id DESC LIMIT ?
 ↪ OFFSET ?"),
 (per_page, offset),
)
 notes = [row_to_note(row) for row in cursor.fetchall()]

 total_pages = max(1, (total + per_page - 1) // per_page) # Ceiling division

 return jsonify({
 "notes": notes,
 "count": len(notes),
 "pagination": {
 "page": page,
 "per_page": per_page,
 "total": total,
 "total_pages": total_pages,
 "has_next": page < total_pages,
 "has_prev": page > 1,
 },
 }), 200

```

The Link header (RFC 8288, which replaced RFC 5988) is a standard way to communicate pagination links:

```

response = make_response(jsonify({...}), 200)

links = []
if page > 1:
 links.append(f'<{url_for("notes.list_notes", page=page-1, per_page=per_page,
↪ _external=True)}>; rel="prev"')
if page < total_pages:
 links.append(f'<{url_for("notes.list_notes", page=page+1, per_page=per_page,
↪ _external=True)}>; rel="next"')
links.append(f'<{url_for("notes.list_notes", page=1, per_page=per_page,
↪ _external=True)}>; rel="first"')
links.append(f'<{url_for("notes.list_notes", page=total_pages, per_page=per_page,
↪ _external=True)}>; rel="last"')

response.headers["Link"] = ", ".join(links)
return response

```

Offset pagination is simple and lets a client jump to any page, but it has two weaknesses. The database must still read and discard every skipped row, so page 5,000 is slow. And the pages shift under the client: if a note is created while someone reads page 1, the last note of page 1 appears again at the top of page 2.

## Cursor-based pagination

A **cursor** is a pointer to the last item the client has seen. The client asks for “the next 20 after this one”, and the query starts exactly there:

```

@notes_bp.route("/", methods=["GET"])
def list_notes():
 limit = max(1, min(100, request.args.get("limit", 20, type=int)))
 after = request.args.get("after", type=int) # id of the last note seen, or None

 cursor = get_db().cursor()
 if after is None:
 cursor.execute(
 sql("SELECT id, content, created_at FROM notes ORDER BY id DESC LIMIT
↪ ?"),
 (limit + 1,)
)
 else:
 cursor.execute(
 sql("SELECT id, content, created_at FROM notes WHERE id < ? ")

```

```

 "ORDER BY id DESC LIMIT ?"),
 (after, limit + 1),
)
 rows = cursor.fetchall()

 # One extra row was requested only to learn whether another page exists
 has_more = len(rows) > limit
 notes = [row_to_note(row) for row in rows[:limit]]

 return jsonify({
 "notes": notes,
 "count": len(notes),
 "next_cursor": notes[-1]["id"] if has_more else None,
 }), 200

```

GET /notes/?limit=20	→ 20 notes, "next_cursor": 1234
GET /notes/?limit=20&after=1234	→ the next 20, "next_cursor": 1214
...	→ the last page has "next_cursor": null

WHERE id < ? uses the primary key's index to go straight to the starting point, so every page costs the same however deep the client goes, and new notes cannot shift the pages. What cursor pagination gives up is jumping: there is no “go to page 37”. For an API whose clients scroll or sync — like the Notes API — that is usually the right trade. (Real APIs often encode the cursor as an opaque string, so they can change what it contains without breaking clients.)

## 34.10 The API as a Contract

Everything in this chapter — status codes, headers, the JSON shapes of success and error — adds up to something bigger than any one response: the **contract** the Notes API offers its clients. Chapter 13 introduced integration contracts and their five dimensions (shape, versioning, identity, authorization, failure semantics) from the consumer's side. Here is the provider's side, now that the API's own responses exist. Such a contract is harder to change than almost any code, because changing it breaks everyone on the other side — and you cannot deploy the other side.

### Shape: choosing and reading the protocol

REST — resources plus HTTP verbs, carrying JSON — is what the Notes API speaks, and the right default: simple, cacheable, and universally understood. The alternatives earn their

place only with a concrete reason: **GraphQL** lets the client choose exactly which fields it wants (handy for diverse front-ends, at the cost of server complexity); **gRPC** is binary, fast, and strongly typed (excellent service-to-service, poor for browsers); **events/queues** are asynchronous and decoupled in time. Reach for the simplest shape that fits.

Within REST, design URLs that age well: name **resources (nouns)**, **not actions** — `/notes/41`, never `/getNote?id=41`. Use plural collections and hierarchy for relationships (`/users/7/notes`). The Notes API already follows this. URLs are part of the published contract; once clients depend on them, they are very hard to change.

### Methods and idempotency: which calls are safe to retry

HTTP methods each carry a promise. **GET** is *safe* — it changes nothing and can be repeated freely. **PUT** and **DELETE** are **idempotent**: performing them twice has the same effect as once (deleting note 41 twice still just leaves note 41 gone — the Notes API’s second call returns **404** instead of **204**, but the *state* is the same, and that is what idempotency is about). **POST** is *not* idempotent — two **POST** `/notes/` create two notes.

This is not pedantry; it decides what is safe to retry. When a request times out, you frequently *do not know* whether it succeeded, so the natural reaction is to retry — and retrying a non-idempotent **POST** can create a duplicate order or double-charge a customer. The fix is an **idempotency key**: the client sends a unique key with the request, the server records it, and on a retry with the same key it returns the original result instead of acting again. This is exactly how payment APIs make “create a charge” safe to retry, and it is the pattern to reach for whenever a create operation must tolerate retries.

### Versioning: changing a contract without breaking callers

A contract is a promise to people you cannot redeploy. Once others depend on it, you cannot simply change it — you must distinguish **non-breaking** changes (additive: a new optional field, a new endpoint — old clients harmlessly ignore them) from **breaking** changes (removing or renaming a field, changing a type, adding a required input). Breaking changes require a **new version**.

The common strategies are **URI versioning** (`/v1/notes` → `/v2/notes`) — explicit, obvious, and most widely used — and **header / content-negotiation versioning** — cleaner URLs, more plumbing. Whichever you choose, **deprecate gracefully**: run the old and new versions in parallel, announce a timeline, and give clients time to migrate. This is the same expand/contract discipline you met for database schemas in [Chapter 6](#), now applied to an API.

## Pagination: never return an unbounded list

A GET `/notes/` that returns *every* note is a latent outage: fine at ten notes, fatal at ten million. Collection endpoints must paginate, and the pagination style is itself part of the contract.

Section 34.9 showed both styles — offset/limit, simple but drifting under inserts and slow at depth; cursor/keyset, stable and fast at any depth. The contract point is *when* to decide: before anyone depends on the endpoint. Adding pagination to an endpoint that clients already call as “returns everything” is itself a breaking change: old clients still parse the response, but silently receive only the first page. The Notes API could make that change freely only because nobody depended on it yet. It still added its pagination fields to the existing `{"notes": [...], "count": n}` envelope, instead of changing its shape, so that nothing that *parses* the response broke.

## Failure semantics and error design

Errors are part of the contract, and they must be *consistent*: one shape for every error (for the Notes API, `{"error": "..."}` , plus a `details` list when a request fails validation for several reasons — section 34.5), correct status codes (4xx means the caller’s fault, 5xx means the server’s fault), and never leaking internal details. And as a *consumer* of someone else’s contract, assume every external call can be slow or fail: set explicit timeouts, retry only idempotent calls (with backoff), and have a fallback for when the dependency is simply down. (→ Chapter 37 — Reliability covers retries, exponential backoff, and circuit breakers.)

## Webhooks: when the other system calls you

Sometimes the integration runs in reverse: an external service notifies *you* of an event — a payment cleared, a build finished — by calling a URL you expose. That is a **webhook**, and receiving one has its own contract obligations:

- **Verify authenticity.** The sender signs the payload with a shared secret; you recompute and check the signature. Without this, anyone who learns your webhook URL can forge events.
- **Expect retries, so be idempotent.** Senders re-deliver on any failure or timeout, which means your handler *will* receive the same event more than once and must process it twice safely (the same idempotency idea as above).
- **Respond fast.** Acknowledge receipt immediately and do the real work asynchronously, or the sender will time out and retry a job that is actually still running.



## Write the contract down: the API as a first-class artifact

A contract that lives only in code and tribal memory will be misunderstood by the people on the other side. Document it explicitly. For REST, an **OpenAPI** (formerly Swagger) specification describes every endpoint, parameter, and response shape in a single machine-readable file that can generate documentation, client libraries, and tests — and serves as the one source of truth both sides agree to. The discipline is to design and write the contract *before* exposing it publicly, because the moment clients integrate against it, the contract becomes the one thing you can no longer change freely.

## The Notes API's contract, written down

Here is the Notes API's contract as an OpenAPI 3.1 file at the root of the repository. It covers the notes endpoints, with the cursor pagination of section 34.9. The complete file also describes the tag endpoints of Chapter 33; they are left out here to save space.

```
openapi.yaml: the Notes API's contract, in one machine-readable file
openapi: 3.1.0
info:
 title: Notes API
 version: 1.4.0
 description: Store and retrieve short text notes.

paths:
 /notes/:
 get:
 summary: List notes, newest first
 parameters:
 - name: limit
 in: query
 schema: {type: integer, minimum: 1, maximum: 100, default: 20}
 - {name: after, in: query, description: id of the last note already seen,
 schema: {type: integer}}
 responses:
 "200":
 description: One page of notes
 content:
 application/json: {schema: {$ref: "#/components/schemas/NoteList"}}
 post:
 summary: Create a note
 requestBody:
 required: true
```

```

 content:
 application/json: {schema: {$ref: "#/components/schemas/NoteInput"}}
 responses:
 "201":
 description: The created note
 content: {application/json: {schema: {$ref: "#/components/schemas/Note"}}}
 "400": {$ref: "#/components/responses/BadRequest"}
 "415": {$ref: "#/components/responses/BadRequest"}

/notes/{note_id}:
 parameters:
 - {name: note_id, in: path, required: true, schema: {type: integer}}
 get:
 summary: Get one note
 responses:
 "200":
 description: The note
 content: {application/json: {schema: {$ref: "#/components/schemas/Note"}}}
 "404": {$ref: "#/components/responses/NotFound"}
 put:
 summary: Replace a note's content
 requestBody:
 required: true
 content:
 application/json: {schema: {$ref: "#/components/schemas/NoteInput"}}
 responses:
 "200":
 description: The updated note
 content: {application/json: {schema: {$ref: "#/components/schemas/Note"}}}
 "400": {$ref: "#/components/responses/BadRequest"}
 "404": {$ref: "#/components/responses/NotFound"}
 delete:
 summary: Delete a note
 responses:
 "204": {description: Deleted; no body}
 "404": {$ref: "#/components/responses/NotFound"}

components:
 schemas:
 Note:
 type: object
 required: [id, content, created_at]
 properties:
 id: {type: integer}

```

```

 content: {type: string, minLength: 1, maxLength: 10000}
 created_at: {type: string, format: date-time}
 updated_at: {type: [string, "null"], format: date-time}
NoteInput:
 type: object
 required: [content]
 properties:
 content: {type: string, minLength: 1, maxLength: 10000}
NoteList:
 type: object
 required: [notes, count]
 properties:
 notes: {type: array, items: {$ref: "#/components/schemas/Note"}}
 count: {type: integer, minimum: 0}
 next_cursor:
 type: [integer, "null"]
 description: pass as ?after= for the next page; null on the last page
Error:
 type: object
 required: [error]
 properties:
 error: {type: string}
 details: {type: array, items: {type: string}}
responses:
 BadRequest:
 description: The request was invalid
 content: {application/json: {schema: {$ref: "#/components/schemas/Error"}}}
 NotFound:
 description: No such note
 content: {application/json: {schema: {$ref: "#/components/schemas/Error"}}}

```

The structure is always the same. `paths` lists each URL and method with its parameters, request body and possible responses. `components` defines each shape once, so that responses can refer to it with `$ref`. Notice how much of the chapter is in this one file: the status codes of section 34.2, the error shape of section 34.5, the envelope with its optional `next_cursor`. Tools read it too. Swagger UI or Redoc turn it into browsable documentation, and generators turn it into client libraries in a dozen languages.

A written contract is only worth something while it is *true*. Two checks keep it that way, and both run in CI.

1. The file itself must be valid OpenAPI. `openapi-spec-validator` (add it to `requirements-dev.txt`, with PyYAML for the test below) checks the file against the OpenAPI specification:

```
$ openapi-spec-validator openapi.yaml
openapi.yaml: OK
```

A response written without its required description, say, fails with 'description' is a required property and the path to the mistake. In the workflow of [Chapter 17](#), it is one more step, before the tests:

```
- name: Check the API contract
 run: openapi-spec-validator openapi.yaml
```

2. The API must actually behave as the file says. A contract test sends real requests through the test client and validates each response body against the schema the file promises. The `jjsonschema` library, which `openapi-spec-validator` already installs, does the checking:

```
tests/test_contract.py
"""The API's real responses must match what openapi.yaml promises."""

from pathlib import Path

import yaml
from jjsonschema import Draft202012Validator

SPEC = yaml.safe_load((Path(__file__).parent.parent / "openapi.yaml").read_text())

def assert_matches(schema_name: str, body) -> None:
 """Validate a response body against one of the spec's schemas."""
 # The "$ref" resolves inside this wrapper, which carries the spec's components
 schema = {"$ref": f"#/components/schemas/{schema_name}", **SPEC}
 Draft202012Validator(schema).validate(body)

def test_a_created_note_matches_the_contract(client):
 response = client.post("/notes/", json={"content": "Buy milk"})
 assert_matches("Note", response.get_json())

def test_the_list_matches_the_contract(client, create_note):
 create_note("First")
 assert_matches("NoteList", client.get("/notes/").get_json())
```

```
def test_errors_match_the_contract(client):
 assert_matches("Error", client.get("/notes/99999").get_json())
 assert_matches("Error", client.post("/notes/", json={}).get_json())
```

Now suppose someone renames `created_at` to `created` in `row_to_note()`, a “harmless” tidy-up. Every existing test that does not look at that field still passes, but the contract test fails:

```
E jsonschema.exceptions.ValidationError: 'created_at' is a required property
...
E On instance:
E {'content': 'Buy milk', 'created': '2024-01-15T14:00:05+00:00', 'id': 1}
```

That is a breaking change caught *before* any client saw it. From here on, every change to the API’s shape goes into `openapi.yaml` in the same pull request. That is how [Chapter 37](#)’s email endpoint and [Chapter 38](#)’s login endpoints and bearer-token security scheme join the contract. Reviewers see the contract change next to the code change, and CI proves the two agree.

### Key Takeaways

- **Handler return values** are normalized by Flask into Response objects. The most common forms are `(jsonify(data), status_code)` and `"", 204`.
- **HTTP status codes** are semantic. Use 201 for creation, 204 for deletion, 404 for not found, 400 (or 422 — pick one) for validation failure, 500 for server errors. Never use 200 for errors.
- **jsonify()** serializes a Python dict to JSON, sets `Content-Type: application/json`, and returns a Response object. It converts `datetime` objects — but to HTTP-date strings, not ISO 8601.
- **ISO 8601 strings** are the standard JSON representation for timestamps. Convert `datetime` objects with `.isoformat()` (the Notes API’s `row_to_note()`) or a custom JSON provider.
- **Consistent error structure** is essential for API clients. Choose one error response format and use it everywhere.
- **after\_request hooks** run after the handler and can modify the response (add headers, log timing). They must return the response object.
- **teardown\_appcontext** runs after `after_request` hooks, even if an exception occurred. This is where the Notes API rolls back anything uncommitted and closes the database connection — handlers have already committed their work.
- **Nginx adds security headers** (HSTS, X-Content-Type-Options, etc.) to all Flask responses. Flask doesn’t need to set them.

- **An API is a contract** with clients you cannot redeploy. **Idempotency** decides what is safe to **retry** (GET/PUT/DELETE are idempotent; POST is not — an **idempotency key** makes retried creates safe). **Version deliberately**: additive changes are non-breaking; removals and type changes need a new version (`/v1` → `/v2`) with graceful deprecation. **Webhooks** (the other system calling you) must verify signatures, be idempotent, and respond fast. Write the contract down (**OpenAPI**) *before* exposing it, and keep it true: validate the file in CI, and let **contract tests** check real responses against its schemas, so a renamed field fails the build instead of a client.
- **Pagination** is necessary for list endpoints with potentially large datasets. Offset pagination (LIMIT/OFFSET) allows jumping to any page; cursor pagination (WHERE `id < last_seen`) stays fast at any depth and does not shift when data changes. Add the pagination fields to the existing response envelope.

## Exercises

1. **Predict.** What does `curl -i -X DELETE .../notes/1` return the second time you run it, and is DELETE still idempotent?
2. **Break and fix.** Make `list_notes` return a bare JSON array instead of `{"notes": [...], "count": n}`. Which contract test fails, and what would it have cost to change this after release?
3. **Extend.** Add an ETag to GET `/notes/<id>` and return 304 when If-None-Match matches. Test it with `curl -H 'If-None-Match: "..."'`.
4. **Explain.** When is 422 better than 400, and why does the Notes API use 400 for everything that fails validation?

Solutions and hints are in the companion repository, in `exercises/chapter-34.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

The request has been received, validated, processed, and responded to. The complete request-response cycle is now understood in detail. The next chapter addresses a different dimension of production systems: how you know what is happening *while* the system is running — the infrastructure of observation, measurement, and alerting.

→ Chapter 35 — Observability

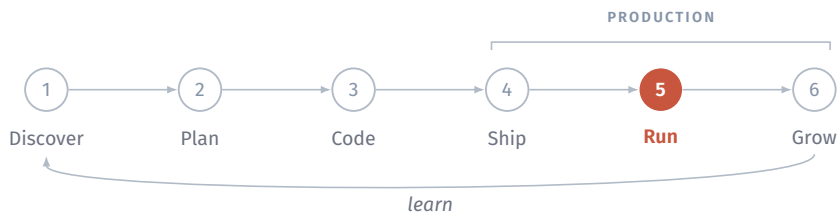
## STAGE 10

# 10

## Production Reality

*What changes when the system must survive reality?*

### YOU ARE HERE



**The piece:** The whole running system

The whole map: *How Software Gets Built*, page 1

Chapters 35–39





# Observability

---

A production system that you cannot observe is a system you cannot maintain. When something breaks at 3 AM — a spike in error rates, a slow database, a process consuming all available memory — you need to know quickly, understand what is happening, and fix it.

**Observability** is the property of a system that allows you to understand its internal state from its external outputs. An observable system emits enough data that you can answer questions like:

- Is the system healthy right now?
- How many requests per second are we handling?
- What was the error rate in the last hour?
- Which requests are slowest, and why?
- When did the problem start?

The three pillars of observability are **logs**, **metrics**, and **traces**. This chapter covers each one in the context of the Notes API, along with health checks and alerting.

## In this chapter

By the end of this chapter, you will understand:

- What structured logging is and why it is superior to plain text logs for production
- What metrics are, how to collect them, and what to measure
- What distributed tracing is (conceptually) and when it matters
- How health check endpoints work and what they should check
- How to set up alerting so problems wake you up before users notice
- How to query and analyze logs in a running production system

## 35.1 The Three Pillars of Observability

## Logs

**Logs** are discrete events that happened in the system, recorded as text or structured data.

```
{ "timestamp": "2024-01-15T14:00:00.024Z", "level": "INFO", "message": "Created note
 ↳ id=42.", "request_id": "a1b2c3" }
{ "timestamp": "2024-01-15T14:00:01.001Z", "level": "ERROR", "message": "Database
 ↳ connection failed", "error": "FATAL: role \"notes_app\" does not exist" }
```

Logs are good for: understanding what happened in a specific request, debugging a specific error, answering “what exactly happened at time X?”

## Metrics

**Metrics** are numerical measurements aggregated over time.

```
http_requests_total{method="POST",path="/notes/",status="201"} 1492
http_request_duration_seconds{quantile="0.5"} 0.004
http_request_duration_seconds{quantile="0.99"} 0.028
```

(Each line is one time series: a metric name, its labels, and the current value. Section 35.4 shows what a real application exposes.)

Metrics are good for: trending (“is error rate going up?”), capacity planning (“how many requests per second can we handle?”), alerting (“alert me if p99 latency exceeds 500ms”).

## Traces

**Traces** follow a single request through all the services it touches. In a microservices architecture, a single user request might touch 10 different services. A trace shows the timeline: which service did what, in what order, and how long each step took.

For the Notes API (a single-service application), tracing is less critical. A single request touches: Flask → PostgreSQL. The important timing information can come from logs (request duration) and database slow query logs.

Traces become essential when you have multiple services and need to find which one is causing latency.

## 35.2 Structured Logging in Depth

Chapter 16 introduced structured logging — JSON-formatted log messages. Let’s go deeper on *what* to log and *how* to make logs useful.

## What every request log should contain

A request log entry should answer:

- **Who:** the client IP, the authenticated user (if applicable), an API key (if applicable)
- **What:** the HTTP method, the URL path, the request ID (a unique identifier for this request)
- **When:** the timestamp (with milliseconds and timezone)
- **How long:** the duration in milliseconds
- **Outcome:** the HTTP status code

Chapter 16's middleware already logs every request's arrival and departure. Two changes turn those lines into this record. Each request gets a **request ID**, and the log calls pass the details as extra fields, which the JSON formatter from Chapter 16 copies into the output:

```
app/middleware.py – changes to Chapter 16's register_middleware()
import uuid

@app.before_request
def log_request_start():
 # A unique ID for this request: every log line it produces carries
 # it, so all the lines of one request can be found together
 g.request_id = str(uuid.uuid4())
 g.request_start_time = time.perf_counter()
 logger.info(
 "→ %s %s (from %s)",
 request.method,
 request.path,
 request.remote_addr,
 extra={
 "method": request.method,
 "path": request.path,
 "remote_addr": request.remote_addr,
 },
)

@app.after_request
def log_request_end(response):
 start = g.get("request_start_time", time.perf_counter())
 elapsed_ms = (time.perf_counter() - start) * 1000
 response.headers["X-Response-Time-Ms"] = f"{elapsed_ms:.1f}"
 # Give the ID to the client too: a user reporting a problem can
 # quote it, and you can find that exact request in the logs
 response.headers["X-Request-ID"] = g.get("request_id", "")
```

```

Log level by status code, as in Chapter 16
if response.status_code >= 500:
 log_fn = logger.error
elif response.status_code >= 400:
 log_fn = logger.warning
else:
 log_fn = logger.info
log_fn(
 "%s in %.1fms",
 response.status_code,
 elapsed_ms,
 extra={
 "method": request.method,
 "path": request.path,
 "status_code": response.status_code,
 "duration_ms": round(elapsed_ms, 2),
 "content_length": response.content_length,
 },
)
return response

```

The messages stay the same — the log searches in [Chapter 25](#) still work — but each line now also carries separate, typed fields. (The human-readable formatter prints only the message; the extra fields appear in JSON output.)

### The request\_id: correlated logs

The `request_id` is a UUID generated at the start of each request. When every log message for that request includes the same `request_id`, you can filter log output to see everything that happened during one specific request:

```

In a log aggregation system, filter by request_id:
request_id = "a1b2c3d4-e5f6-7890-abcd-ef1234567890"

```

Output (abbreviated: the function and line fields are left out):

```
{
 "timestamp": "2024-01-15T14:00:00.020Z",
 "level": "INFO",
 "logger": "app.middleware",
 "message": "→ POST /notes/ (from 89.100.200.50)",
 "request_id": "a1b2c3d4...",
 "method": "POST",
 "path": "/notes/",
 "remote_addr": "89.100.200.50"
}
{"timestamp": "2024-01-15T14:00:00.034Z",
 "level": "INFO",
 "logger": "app.routes.notes",
 "message": "Created note id=42.",
 "request_id": "a1b2c3d4..."}
{"timestamp": "2024-01-15T14:00:00.034Z",
 "level": "INFO",
 "logger": "app.middleware",
 "message": "← 201 in 14.3ms",
 "request_id": "a1b2c3d4...",
 "method": "POST",
 "path": "/notes/",
 "status_code": 201,
 "duration_ms": 14.3,
 "content_length": 84}
```

Without the `request_id`, logs from concurrent requests are interleaved and impossible to untangle.

## Including `request_id` in all log messages

The challenge: `g.request_id` is set in a `before_request` hook, but it needs to appear in log messages from any module (`app.database`, `app.routes.notes`, etc.). Passing the `request_id` explicitly through every function call is impractical.

Solution: use Python's logging filter mechanism to inject the `request_id` into every log record. Add the filter to Chapter 16's `app/logging_config.py`, and attach it to its handlers:

```
app/logging_config.py (additions)
from flask import g, has_request_context

class RequestIDFilter(logging.Filter):
 """
 A logging filter that injects the current request ID into log records.

 Attached to the log handlers: every record that passes through them
 gets a 'request_id' attribute, which the JSON formatter from
 Chapter 16 copies into the output like any other extra field.
 """
 def filter(self, record):
 # has_request_context() returns True when inside a Flask request
 if has_request_context():
 record.request_id = g.get("request_id")
 else:
 record.request_id = None # startup messages, for example
 return True # Always emit the log record

In configure_logging(), after creating the handlers:
```

```

console_handler.addFilter(RequestIDFilter())
if file_handler:
 file_handler.addFilter(RequestIDFilter())

```

Now every `logger.info("Created note id=%d.", note_id)` call automatically includes the `request_id` in the JSON output — from any module, without passing anything around. Nothing else in `logging_config.py` changes: the JSON formatter already copies every non-standard attribute of a record, and its timestamp is already ISO 8601 in UTC.

## What NOT to log

Logs are often stored long-term and may be accessed by many people. Never log:

- Passwords or password hashes
- API keys or tokens
- Credit card numbers
- Social security numbers or government IDs
- Full request bodies (they may contain sensitive data)
- Database connection strings (they contain passwords)

For the Notes API:

```

WRONG:
logger.info("DB connection: %s", DATABASE_URL)
This logs the database password!

CORRECT: log only the parts that are safe to show
parsed = urlparse(DATABASE_URL)
logger.info("Connecting to database at %s/%s", parsed.hostname,
 ↪ parsed.path.lstrip("/"))
(The startup checks from Chapter 15 mask the password the same way: _mask())

```

## 35.3 Health Checks

A **health check endpoint** is a URL that external systems (load balancers, orchestrators, monitoring systems) can query to determine if the application is functioning.

Docker's `HEALTHCHECK` directive, load balancers (including the commercial NGINX Plus — open-source Nginx only checks passively), Kubernetes probes, and uptime monitors all call a health check endpoint and interpret the response.

## What a health check should do

A good health check verifies:

1. **The application is running** (the endpoint responds at all)
2. **Critical dependencies are reachable** (the database is accessible)
3. **The application can serve requests** (it can actually do work, not just listen)

```
app/routes/health.py (replaces the Chapter 15 version)
import logging
import time
from flask import Blueprint, jsonify
from app.database import get_db

logger = logging.getLogger(__name__)
health_bp = Blueprint("health", __name__)

@health_bp.route("/health")
def health_check():
 """
 Deep health check: is the application able to do its job?

 Returns 200 {"status": "ok", ...} if healthy, 503 {"status": "unhealthy", ...}
 if not. Called by:
 - the deploy scripts, to verify a new container (Chapter 23)
 - uptime monitors (section 35.6)
 """
 checks = {}
 overall_healthy = True

 # Database connectivity
 db_start = time.perf_counter()
 try:
 cursor = get_db().cursor()
 cursor.execute("SELECT 1")
 cursor.fetchone()
 checks["database"] = {
 "status": "ok",
 "latency_ms": round((time.perf_counter() - db_start) * 1000, 2),
 }
 except Exception:
 # Log the details; tell the caller only that the check failed.
 # (An exception message from the driver can include the database
 # host, port and user name – nothing a public endpoint should show.)
```

```

 logger.exception("Health check: database unhealthy")
 checks["database"] = {"status": "unhealthy"}
 overall_healthy = False

 status_code = 200 if overall_healthy else 503
 return jsonify({
 "status": "ok" if overall_healthy else "unhealthy",
 "checks": checks,
 }), status_code

@health_bp.route("/ping")
def ping():
 """Shallow health check: is the process up and answering HTTP?"""
 return jsonify({"status": "ok"}), 200

```

The top-level status is still "ok" when all is well, so every existing consumer — the `grep` `"status:ok"` in the deploy scripts, the smoke test in CI — keeps working; and a failing check now also changes the HTTP status to 503, which `curl -f` treats as a failure. (Checking the status code is the sturdier habit: it does not depend on the exact JSON.)

## Shallow vs deep health checks

The check above is a **deep health check** — it verifies external dependencies (the database). This is the most useful kind, but it has a cost: every health check makes a database query.

A **shallow health check** (also called a liveness check) — the `/ping` route above — just verifies the process is running.

Some systems use both:

- **Liveness check** (`/ping`): fast, no dependencies. If this fails, the process is completely unresponsive — restart it.
- **Readiness check** (`/health`): verifies dependencies. If this fails, stop routing traffic here, but don't restart the process (the process is alive; the dependency is down).

This distinction decides which URL each consumer should call. The Dockerfile's `HEALTHCHECK` (Chapter 18) is a liveness check: point it at `/ping` from now on. Otherwise a short database outage marks *every* container unhealthy at once, and an orchestrator that restarts unhealthy containers would restart them all — which cannot fix a database. The deploy scripts keep calling `/health`: a new version that cannot reach its database should not go live.



## Protecting the health endpoint

The health check endpoint must be reachable by the monitoring infrastructure, but not necessarily by the public. Options:

- Rate-limit it in Nginx (it should only be called a few times per minute, not thousands)
- Restrict it to known IP addresses in Nginx
- Or simply leave it public

The Notes API leaves `/health` public: it reveals only `ok` or `unhealthy` per check, and an external uptime monitor (section 35.6) must be able to reach it. If you restrict it, remember to allow your monitor's addresses too:

```
Allow health checks only from the server itself, the VPC, and the monitor
location /health {
 allow 127.0.0.1;
 allow 10.0.0.0/8; # VPC range
 allow 198.51.100.0/24; # your uptime monitor's published addresses
 deny all;
 proxy_pass http://notes_api;
}
```

## 35.4 Metrics Collection

Logs tell you *what* happened. Metrics tell you *how much* and *how fast*.

The most common metrics to collect for a web API:

Metric	Type	What it tells you
<code>http_requests_total</code>	Counter	Total requests over time; rate = requests per second. A <code>status</code> label breaks it down by status code
<code>http_request_duration_seconds</code>	Histogram	Latency distribution (p50, p95, p99)
<code>database_query_duration_seconds</code>	Histogram	Database latency distribution
<code>active_requests</code>	Gauge	Currently in-flight requests
<code>memory_usage_bytes</code>	Gauge	Memory used by the process

## Prometheus: the de facto standard

**Prometheus** is the most widely-used open-source metrics collection system. It works by **scraping** — periodically fetching a `/metrics` endpoint from your application and storing the values in a time-series database.

The `prometheus-flask-exporter` library instruments Flask automatically. One detail decides how to set it up: the Notes API runs under **Gunicorn with several worker processes**, and each worker would count only the requests *it* handled. A scrape of `/metrics` would reach one random worker and report only that worker’s numbers, so graphs would jump around and totals would be wrong. The library’s **multiprocess mode** fixes this: every worker writes its counters to files in a shared directory, and a small HTTP server in the Gunicorn master adds them up.

```
pip install prometheus-flask-exporter # and add it, pinned, to requirements.txt
```

```
app/__init__.py
from prometheus_flask_exporter.multiprocess import GunicornPrometheusMetrics

from app.config import PROMETHEUS_MULTIPROC_DIR

def create_app():
 app = Flask(__name__)
 # ...

 # Multiprocess metrics need their shared directory, which the Docker image
 # provides (below). Elsewhere (the tests, python run.py) the library would
```

```

refuse to start, so the application runs without metrics instead.
if PROMETHEUS_MULTIPROC_DIR:
 # Automatically tracks, for every request:
 # - flask_http_request_duration_seconds (histogram: method, path, status)
 # - flask_http_request_total (counter: method, status)
 metrics = GunicornPrometheusMetrics(app)

 # An "info" metric carrying labels about the running application
 metrics.info("app_info", "Application info", version="1.0.0")

return app

```

PROMETHEUS\_MULTIPROC\_DIR is read in app/config.py, like every other setting: PROMETHEUS\_MULTIPROC\_DIR: str = os.getenv("PROMETHEUS\_MULTIPROC\_DIR", ""). Without that condition, GunicornPrometheusMetrics(app) raises ValueError: one of env PROMETHEUS\_MULTIPROC\_DIR or env prometheus\_multiproc\_dir must be set and be a directory, and every test fails at create\_app().

```

gunicorn.conf.py (additions)
import os

from prometheus_flask_exporter.multiprocess import GunicornPrometheusMetrics

Like create_app(), only with the metrics directory set (the Docker image).
Without it, both calls raise ValueError and Gunicorn would not start.
METRICS = bool(os.getenv("PROMETHEUS_MULTIPROC_DIR"))

def when_ready(server):
 # Serve the combined metrics of all workers on port 9200
 if METRICS:
 GunicornPrometheusMetrics.start_http_server_when_ready(9200)

def child_exit(server, worker):
 # Tell the metrics library that a worker has gone
 if METRICS:
 GunicornPrometheusMetrics.mark_process_dead_on_child_exit(worker.pid)

```

Multiprocess mode needs one more setting: an empty, writable directory for the workers' files, named by an environment variable. In the Dockerfile:

```
ENV PROMETHEUS_MULTIPROC_DIR=/tmp/prometheus
RUN mkdir -p /tmp/prometheus && chown notes:notes /tmp/prometheus
```

The metrics are now served on port 9200, separately from the API — so they never go through Nginx to the public. On the server, publish that port on the loopback address too, by adding `-p 127.0.0.1:9200:9200` to the docker run commands in the deploy scripts.

```
curl http://127.0.0.1:9200/metrics

Output (Prometheus text format, abbreviated):
HELP flask_http_request_duration_seconds Flask HTTP request duration in seconds
TYPE flask_http_request_duration_seconds histogram
flask_http_request_duration_seconds_bucket{le="0.005",method="GET",path="/notes/",sta
↪ tus="200"} 145.0
flask_http_request_duration_seconds_bucket{le="0.01",method="GET",path="/notes/",stat
↪ us="200"} 198.0
...
flask_http_request_duration_seconds_count{method="GET",path="/notes/",status="200"}
↪ 203.0
flask_http_request_duration_seconds_sum{method="GET",path="/notes/",status="200"}
↪ 0.892
HELP flask_http_request_total Total number of HTTP requests
TYPE flask_http_request_total counter
flask_http_request_total{method="GET",status="200"} 203.0
flask_http_request_total{method="POST",status="201"} 1492.0
```

## Custom metrics

Track application-specific quantities (in multiprocess mode, counters and histograms are combined across workers automatically):

```
from prometheus_client import Counter, Histogram

notes_created = Counter(
 "notes_created_total",
 "Total number of notes created",
)

note_content_length = Histogram(
 "note_content_length_bytes",
 "Length of note content at creation time",
 buckets=[10, 50, 100, 500, 1000, 5000, 10000],
```

```

)

@notes_bp.route("/", methods=["POST"])
def create_note():
 # ... validation ...
 # ... database insert ...
 notes_created.inc() # Increment the counter
 note_content_length.observe(len(content)) # Record the content length
 return jsonify(note), 201

```

## Setting up Prometheus and Grafana

In a complete production setup:

```

docker-compose.monitoring.yml (monitoring stack, on the same server)
services:
 prometheus:
 image: prom/prometheus:v2.53.0 # pinned, never :latest (Chapter 19)
 # Host networking: Prometheus can reach 127.0.0.1:9200, and its own
 # port stays behind UFW – nothing is published through Docker, so
 # nothing bypasses the firewall (Chapter 21)
 network_mode: host
 command:
 - "--config.file=/etc/prometheus/prometheus.yml"
 - "--web.listen-address=127.0.0.1:9090" # no authentication: loopback only
 volumes:
 - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
 - prometheus-data:/prometheus

 grafana:
 image: grafana/grafana:11.1.0
 network_mode: host
 environment:
 GF_SERVER_HTTP_ADDR: "127.0.0.1"
 env_file:
 - /opt/notes-api/config/grafana.env # GF_SECURITY_ADMIN_PASSWORD=... (mode 600)
 volumes:
 - grafana-data:/var/lib/grafana

volumes:
 prometheus-data:
 grafana-data:

```

```
prometheus.yml
global:
 scrape_interval: 15s # Scrape every 15 seconds

scrape_configs:
 - job_name: "notes-api"
 static_configs:
 - targets: ["127.0.0.1:9200"] # The metrics server in the Unicorn master
```

Both web interfaces listen on the server's loopback address only. To use them from your laptop, forward the ports over SSH:

```
ssh -L 3000:127.0.0.1:3000 -L 9090:127.0.0.1:9090 notes-api
then open http://localhost:3000 (Grafana) in your browser
```

Grafana connects to Prometheus and lets you build dashboards: requests per second, error rates, p99 latency, database query times, all plotted over time.

## The four golden signals

Google's Site Reliability Engineering book identifies four "golden signals" that should be monitored for any user-facing service:

1. **Latency:** How long requests take. Track p50 (median), p95, p99. A rising p99 means the slowest 1% of requests are getting slower.
2. **Traffic:** How many requests per second. Sudden increases might indicate an attack; sudden decreases might indicate a problem with clients.
3. **Errors:** The rate of requests that fail (5xx status codes, or 4xx for client errors that shouldn't be occurring). A rising error rate is usually the first sign of a problem.
4. **Saturation:** How full the system is. CPU usage, memory usage, the number of open database connections compared with the database's limit. Saturation predicts future problems before they become failures.

## 35.5 Log Aggregation

When your application runs in Docker on a single server, logs go to stdout/stderr and are captured by Docker's log driver. You can view them with `docker logs notes-api`.

For production, you need logs to be:

- **Persistent:** available even after the container restarts
- **Searchable:** queryable by timestamp, level, field values
- **Aggregated:** from all servers in one place (when you scale to multiple servers)

## The ELK Stack

The most common open-source log aggregation system:

- **Elasticsearch:** stores and indexes logs
- **Logstash** (or **Filebeat**): ships logs from servers to Elasticsearch
- **Kibana:** web UI for searching and visualizing logs

```
Application → stdout → Docker log driver → Filebeat → Elasticsearch → Kibana
```

## Cloud logging services

For a small production setup, cloud logging services are simpler than running your own ELK stack:

- **DigitalOcean Monitoring:** free server metrics (CPU, memory, disk) and alert policies for Droplets — but no search over application logs
- **AWS CloudWatch:** managed log storage and search
- **Datadog:** all-in-one monitoring (logs + metrics + traces), paid
- **Better Stack, Papertrail,** and similar hosted log services: ship logs to them, search them in a web UI

(Plans, free tiers, and even product names in this market change often — check the current offer before you choose.)

For the Notes API on one Droplet, DigitalOcean’s metrics plus `docker logs` and `jq` are enough to start with.

## Querying logs in production

With Docker on a single server:

```
View all logs for the notes-api container
docker logs notes-api

Follow logs in real time
docker logs -f notes-api
```

```

Show only the last 100 lines
docker logs --tail 100 notes-api

Filter logs since a specific time
docker logs --since "2024-01-15T14:00:00" notes-api

Pipe to jq for JSON parsing. -R reads raw lines, and fromjson? parses
the ones that are JSON and skips the rest (Gunicorn's own plain-text lines)
docker logs notes-api 2>&1 | jq -cR 'fromjson? | select(.level == "ERROR")'

Find all errors in the last hour
docker logs --since "1h" notes-api 2>&1 | jq -cR 'fromjson? | select(.level ==
↳ "ERROR")'

Find all requests that took more than 100ms
docker logs notes-api 2>&1 | jq -cR 'fromjson? | select(.duration_ms > 100)'

Find all 5xx responses
docker logs notes-api 2>&1 | jq -cR 'fromjson? | select(.status_code >= 500)'

Everything one request logged
docker logs notes-api 2>&1 | jq -cR 'fromjson? | select(.request_id == "a1b2c3d4-...")'

```

## 35.6 Alerting

Observability is only valuable if problems surface before users notice. **Alerting** is the process of automatically detecting problematic conditions and notifying the on-call engineer.

### What to alert on

**Alert on symptoms, not causes.** A symptom is something the user experiences: slow responses, errors, downtime. A cause is something internal: high CPU, slow database queries, disk filling up.

Alert on:

- **Error rate** exceeds 1% of requests (5xx responses)
- **p99 latency** exceeds 500ms (the slowest 1% of requests are too slow)
- **Health check** fails for more than 60 seconds (the service is down)
- **Disk space** above 85% used (risk of running out)

Consider monitoring (but not paging for) causes:



- High CPU usage
- High memory usage
- Slow database queries (warn in logs, but only page if they cause user-visible symptoms)

## Implementing alerts with UptimeRobot (free tier)

**UptimeRobot** monitors URLs and alerts you if they return errors or respond slowly. At the time of writing, its free plan checks up to 50 URLs every 5 minutes (check the current terms: they have changed, including for commercial use).

Setup:

1. Create an account at [uptimerobot.com](https://uptimerobot.com)
2. Add a monitor for `https://api.notes.example.com/health`
3. Set alert condition: if response is not 200 (the deep check answers 503 when the database is unreachable), or response time > 10 seconds
4. Add notification channels: email, SMS, Slack webhook, PagerDuty

This gives you basic uptime monitoring without any code changes — just the `/health` endpoint.

## Implementing alerts with Prometheus Alertmanager

For metric-based alerting:

```
prometheus_rules.yml
groups:
 - name: notes-api
 rules:
 # Alert if error rate exceeds 1%
 - alert: HighErrorRate
 expr: |
 (
 sum(rate(flask_http_request_total{status=~"5.."}[5m]))
 /
 sum(rate(flask_http_request_total[5m]))
) > 0.01
 for: 2m # Must be true for 2 minutes before firing
 labels:
 severity: page
 annotations:
 summary: "High error rate: {{ $value | humanizePercentage }}"
```

```

 description: "More than 1% of requests are failing over the last 5
 ↪ minutes."

 # Alert if p99 latency exceeds 500ms
 - alert: HighLatency
 expr: |
 histogram_quantile(
 0.99,
 sum(rate(flask_http_request_duration_seconds_bucket[5m])) by (le)
) > 0.5
 for: 5m
 labels:
 severity: page
 annotations:
 summary: "High p99 latency: {{ $value | humanizeDuration }}"

 # Alert if Prometheus cannot scrape the application at all
 # (this is a scrape failure, not a /health check: the uptime
 # monitor above watches /health)
 - alert: TargetDown
 expr: up{job="notes-api"} == 0
 for: 1m
 labels:
 severity: page
 annotations:
 summary: "Notes API metrics endpoint is unreachable"

```

## Alert fatigue

Too many alerts → engineers start ignoring them (alert fatigue). Rules for good alerts:

- **Actionable:** every alert should have a clear action to take
- **Urgent:** only page for things that need immediate attention
- **Accurate:** low false positive rate
- **Novel:** don't alert on the same issue multiple times

For a small notes API, start with just two alerts: the service is down, and the error rate is high. Add more alerts as you learn what problems actually occur in practice.

## 35.7 The Nginx Access Log

Nginx's access log provides another view of incoming traffic, independent of the application's own logs. [Chapter 28](#) used Nginx's default format, `combined`, which records no timings. Define a format that adds two — a `log_format` belongs in the `http {}` context, so a file in `conf.d/` is the place:

```
/etc/nginx/conf.d/log-format.conf
log_format timed '$remote_addr - $remote_user [$time_local] '
 '$request' $status $body_bytes_sent '
 '$http_referer' '$http_user_agent' '
 '$request_time $upstream_response_time';
```

Then use it in the site configuration, in place of the plain `access_log` line:

```
access_log /var/log/nginx/notes-api-access.log timed;
```

This produces lines like:

```
89.100.200.50 - - [15/Jan/2024:14:00:00 +0000] "POST /notes/ HTTP/2.0" 201 84 "-"
↪ "curl/8.1.2" 0.016 0.015
```

Fields:

- **89.100.200.50**: client IP
- **[15/Jan/2024:14:00:00 +0000]**: timestamp
- **POST /notes/ HTTP/2.0**: method, path, protocol
- **201**: status code
- **84**: response body size in bytes
- **-**: referrer (empty)
- **curl/8.1.2**: User-Agent
- **0.016**: request time in seconds — the *whole* request, from the first byte read from the client to the last byte sent back
- **0.015**: upstream response time — how long Unicorn took to answer. A big gap between the two points at the network or a slow client, not the application

The Nginx access log captures traffic that the application log misses — like requests rejected by Nginx itself (rate limits, wrong host headers) — and provides a ground truth for traffic volume independent of the application.

## Analyzing Nginx logs

Nginx runs on the host (Chapter 28), so these run directly on the server. The log file covers everything since its last rotation — usually one day:

```
Count requests by status code (in the current log file)
sudo awk '{print $9}' /var/log/nginx/notes-api-access.log | sort | uniq -c | sort -rn
Output:
1492 201
203 200
7 404
2 429

Find the slowest requests (request time is the second-to-last field)
sudo awk '{print $(NF-1), $7}' /var/log/nginx/notes-api-access.log | sort -rn | head
↵ -20

Count requests per minute (for rate analysis)
sudo awk '{print $4}' /var/log/nginx/notes-api-access.log | cut -c1-18 | sort | uniq
↵ -C
```

## 35.8 The Observability Stack for the Notes API

Here is the complete observability setup for the Notes API at production scale:

```

Monitoring layer:
├─ UptimeRobot (free)
│ └─ Pings /health every 5 minutes → email alert if down
├─ Application logs (structured JSON → stdout → Docker)
│ └─ Viewed with: docker logs notes-api | jq ...
│ └─ Rotated by size: 3 files × 10 MB (daemon.json, Chapter 21)
├─ Nginx access logs
│ └─ Location: /var/log/nginx/notes-api-access.log
│ └─ Stored 14 days by logrotate
└─ Server metrics (DigitalOcean Monitoring, free)
 └─ CPU usage (alert at 80%)
 └─ Memory usage (alert at 90%)
 └─ Disk usage (alert at 85%)

Next level (when traffic grows):
├─ Prometheus + Grafana (metrics)
│ └─ flask_http_request_total by method and status
│ └─ flask_http_request_duration_seconds histograms
│ └─ Alertmanager for metric-based alerts
└─ Log aggregation (a hosted service, Datadog, or ELK)
 └─ Centralized search across log fields
 └─ Alert on log patterns ("ERROR" rate exceeds threshold)

```

### Key Takeaways

- **Observability** is the ability to understand system state from external outputs. The three pillars are logs (events), metrics (numbers over time), and traces (request flows across services).
- **Structured logs** (JSON format) are machine-readable. They enable filtering by field values, extracting specific fields, and aggregating across many log lines.
- **Request IDs** (UUIDs set at the start of each request) appear in all log messages for that request. They are essential for correlating log lines across concurrent requests.
- **Health check endpoints:** `/health` (deep) verifies that the application and its dependencies (database) are working — for deploy scripts and uptime monitors; `/ping` (shallow) only that the process answers — for Docker's HEALTHCHECK. Neither reveals internal error details.
- **The four golden signals** to monitor: latency, traffic, errors, saturation.
- **Prometheus** scrapes a `/metrics` endpoint to collect metrics. `prometheus-flask-exporter` instruments Flask automatically — under a multi-worker Gunicorn, in its multiprocess mode, served on a port that is not public.

- **Alert on symptoms** (slow responses, high error rate, service down), not causes (high CPU). Too many alerts cause alert fatigue.
- **Nginx access logs** provide an independent record of all incoming traffic, including requests that the application itself never processes.

## Exercises

1. **Predict.** Stop the database and call `/ping` and `/health`. What does each return, and which would you give to a load balancer?
2. **Break and fix.** Remove the request-ID middleware and try to find all the log lines of one failing request among 1,000 concurrent ones. Restore it and try again.
3. **Extend.** Add a Prometheus alert rule for “95th-percentile latency above 500 ms for 5 minutes” and test it by adding a delay to a route.
4. **Explain.** Why alert on symptoms (errors, latency) rather than causes (CPU, memory)?

Solutions and hints are in the companion repository, in `exercises/chapter-35.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What’s Next

The system is observable. Now consider what happens when the system encounters more traffic than anticipated, or when you need to deploy to multiple servers. The next chapter covers scaling and load handling.

→ [Chapter 36 — Scaling and Load Handling](#)

# Scaling and Load Handling

---

The Notes API runs on a single server with a fixed number of Gunicorn workers. This works for low to moderate traffic. But what happens when traffic grows — when the number of requests per second exceeds what a single server can handle? Or when traffic spikes temporarily during a product launch?

This chapter covers the concepts and techniques for handling load: understanding capacity, identifying bottlenecks, and the strategies for scaling when you hit limits. The goal is not to prematurely optimize — premature optimization wastes engineering time on problems that may never materialize — but to understand the levers available and recognize when to use them.

## In this chapter

By the end of this chapter, you will understand:

- How to determine the current capacity of the Notes API
- Where bottlenecks typically appear in web applications, and how a profiler (`cProfile`, `py-spy`) finds the slow code
- How to scale vertically (bigger server) and horizontally (more servers)
- What a load balancer is and how it distributes requests
- How connection pooling helps database throughput
- What caching is and when it helps
- How rate limiting protects the system from abuse
- How async Python (`async/await`) achieves concurrency, and why the Notes API does not need it

## 36.1 Understanding Capacity

**Capacity** is the maximum load the system can handle while still meeting performance requirements (e.g., p99 latency < 500ms).

Before optimizing, measure. You need to know:

1. How much traffic are you handling now?
2. How fast is traffic growing?
3. What is the current capacity limit?

## Measuring current throughput

From the Nginx access log (Nginx runs on the host, [Chapter 28](#)):

```
Requests per minute, for the last five minutes in the log
sudo tail -n 10000 /var/log/nginx/notes-api-access.log | \
 awk '{print $4}' | cut -c1-18 | sort | uniq -c | tail -5
583 [15/Jan/2024:14:01
611 [15/Jan/2024:14:02
...
600 requests in a minute = 10 requests per second
```

(Count within a time window, as here: the log file as a whole covers everything since its last rotation, usually a day.)

From Prometheus (if configured, [Chapter 35](#)):

```
Requests per second averaged over 5 minutes:
sum(rate(flask_http_request_total[5m]))
```

## Load testing: finding the limit

Load testing sends artificial traffic to find the breaking point. The tools:

- **locust**: Python-based, web UI, write scenarios in Python
- **k6**: JavaScript, runs in CI pipelines, excellent reporting
- **wrk**: lightweight C tool for raw throughput testing
- **ab** (Apache Bench): simple, built into many systems

```
locustfile.py – a simple load test for the Notes API
import random
from locust import HttpUser, task, between

class NotesAPIUser(HttpUser):
 # Simulated user waits between 1 and 3 seconds between requests
 wait_time = between(1, 3)
```



```

@task(3) # Weight 3: list_notes runs 3x more often than create_note
def list_notes(self):
 self.client.get("/notes/")

@task(1) # Weight 1
def create_note(self):
 # json= also sets the Content-Type header
 response = self.client.post("/notes/", json={"content": "Load test note"})
 if response.status_code == 201:
 self.note_ids.append(response.json()["id"])

@task(2) # Weight 2
def get_note(self):
 # Read notes this user created – not the same note over and over,
 # which would measure one cached row, not a realistic workload
 if self.note_ids:
 self.client.get(f"/notes/{random.choice(self.note_ids)}",
 ↪ name="/notes/[id]")

def on_start(self):
 self.note_ids = []

```

```

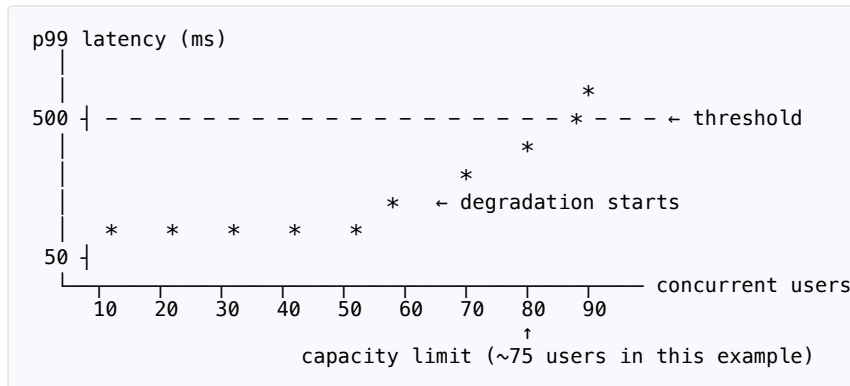
Run the load test – against a staging copy of the system, not production
locust -f locustfile.py --host https://staging.notes.example.com \
 --users 100 --spawn-rate 10 --run-time 60s --headless

```

This run ramps up to 100 simulated users and holds them for a minute. To find the limit, repeat it with more users each time (or use Locust’s web UI to raise the count while watching the charts) until response times start degrading: the point where p99 latency exceeds your threshold is your capacity limit. Remember that the test comes from one IP address — Nginx’s per-IP rate limit (Chapter 28) will throttle it long before the application is stressed, so raise the limit on the staging server, or you will measure Nginx’s 429s instead of the application.

## What the results tell you

A typical capacity curve:



If your Notes API handles 75 concurrent users before latency crosses the threshold, and your current peak is 15 concurrent users, you have ~5x headroom — you don't need to scale yet.

## 36.2 Identifying Bottlenecks

The bottleneck is the part of the system that limits throughput. Scaling other parts won't help until the bottleneck is addressed.

### Finding the bottleneck

**CPU-bound:** the application is spending most of its time doing computation.

```
During a load test (the slim image has no top or ps, so look from the host):
docker stats notes-api
CONTAINER ID NAME CPU % MEM USAGE / LIMIT ...
3f2a1b... notes-api 98.7% 142MiB / 957MiB
CPU near 100% per core (100% on 1 vCPU, 200% on 2) → CPU-bound
```

For Python web applications, CPU-bound behavior is uncommon because most time is spent waiting for I/O (network, database). If you see it, look for: JSON parsing of very large payloads, complex string operations, or heavy computation in request handlers.

**I/O-bound (database):** the application is waiting for the database. The application does not log how long each query takes, but PostgreSQL can. On a managed database, enable the `pg_stat_statements` extension, then ask for the slowest queries:

```

-- Once per database (managed PostgreSQL providers support this extension)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- The queries that take the most time in total
SELECT query, calls, round(mean_exec_time::numeric, 2) AS mean_ms
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

```

(Alternatively, the `log_min_duration_statement` setting makes PostgreSQL log every query slower than a threshold.)

If database query times are high (> 50ms for simple queries), look for:

- Missing indexes
- Slow queries (use `EXPLAIN ANALYZE` in PostgreSQL)
- Too many database queries per request (N+1 problem)
- Database server under-resourced (needs more RAM for caching)

**I/O-bound (network):** the application is waiting for network responses to external services.

**Worker starvation:** all Gunicorn workers are busy and new requests queue up.

```

Check Gunicorn's accept queue. It lives in the container's network
namespace - ss on the host would show docker-proxy's socket instead.
sudo nsenter -t $(docker inspect -f '{{.State.Pid}}' notes-api) -n ss -tln
State Recv-Q Send-Q Local Address:Port
LISTEN 12 2048 0.0.0.0:5000
A Recv-Q that stays above zero means requests are waiting for a free
worker: increase the worker count or scale horizontally

```

**Memory exhausted:** the server is using swap (virtual memory on disk, which is slow).

```

free -h
If "used" under "Swap" is significant, you're swapping → scale up server RAM

```

## Profiling: where does the time go inside the code?

The checks above tell you *which resource* is the bottleneck. When the answer is “the application itself is slow”, a **profiler** tells you *which code*. It records every function call, how often it happened, and how long it took, so you stop guessing.

Python ships with one, `cProfile`. It is simplest to run it around the code in question, in a script or a test, on your own machine. Here it measures the two ways of listing notes with

their tags from section 33.12: the N+1 loop, and the batched version. The setup is 200 notes, and the whole list is fetched 10 times, against a local PostgreSQL:

```
import cProfile
import pstats

with cProfile.Profile() as profiler:
 for _ in range(10):
 list_with_tags_n_plus_1(cursor)

pstats.Stats(profiler).sort_stats("cumulative").print_stats(4)
```

67142 function calls in 0.511 seconds

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
10	0.002	0.000	0.511	0.051	prof.py:12(list_with_tags_n_plus_1)
2010	0.001	0.000	0.492	0.000	psycpg2/extras.py:233(execute)
2010	0.490	0.000	0.491	0.000	{function RealDictCursor.execute}
2010	0.001	0.000	0.013	0.000	psycpg2/extras.py:98(fetchall)

Read it by the columns that matter. **cumtime** is the time spent in a function including everything it called, and **ncalls** is how many times it ran. Almost all of the 0.51 seconds went into `execute`, called 2,010 times: 201 queries per listing. The same measurement for the batched version shows 20 calls to `execute` and 0.042 seconds in total, twelve times faster, with the database on the same machine. Across a real network, where each round trip costs a millisecond or more, the gap grows much wider. The profile turned “listing is slow” into “listing makes 201 queries”, and the fix follows directly.

`cProfile` slows the code it measures and must be started around it. For a process that is *already running*, such as a Gunicorn worker that is slow in production right now, use a **sampling profiler**. `py-spy` reads the stack of a running Python process many times a second from the outside, without restarting it and with little overhead:

```
pip install py-spy

A live, top-like view of where a process spends its time
sudo py-spy top --pid <PID>

Record 30 seconds and write a flame graph (open the SVG in a browser)
sudo py-spy record --pid <PID> --duration 30 -o profile.svg
```

For a process inside a container, find its host PID with `docker inspect -f '{{.State.Pid}}'` notes-api and run `py-spy` on the host. It needs root, or a container started with `--cap-add SYS_PTRACE`, because reading another process's memory is exactly what the kernel normally forbids.

A **flame graph** draws the sampled stacks as nested bars: the wider a bar, the more time was spent there. You look for the widest bars that belong to your code. And whatever the tool, measure *before* optimizing. The slow part is rarely where intuition says it is.

## Database indexes: the first optimization

The most common bottleneck for a simple CRUD API is missing database indexes. Without an index, PostgreSQL must scan every row to find matching rows.

```
-- Without an index on created_at: PostgreSQL reads every row
EXPLAIN ANALYZE
SELECT * FROM notes WHERE created_at > now() - interval '1 day';
-- Seq Scan on notes (cost=0.00..2137.00 rows=412 ...) (actual time=0.015..15.234
↳ rows=398 ...)

CREATE INDEX idx_notes_created_at ON notes (created_at);

-- With the index:
EXPLAIN ANALYZE
SELECT * FROM notes WHERE created_at > now() - interval '1 day';
-- Index Scan using idx_notes_created_at on notes (...) (actual time=0.021..0.310
↳ rows=398 ...)
```

The index makes this query about 50x faster on a table of 100,000 notes, and the gap grows with the table. (Lookups by id are fast already: a PRIMARY KEY always comes with an index.)

For the Notes API's search (`?search=`, [Chapter 33](#)), the index must match the query content `ILIKE '%term%'` starts with a wildcard, which an ordinary B-tree index cannot serve — not even one on `LOWER(content)`. A **trigram** index can:

```
-- Trigram index: serves LIKE/ILIKE with wildcards anywhere
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_notes_content_trgm ON notes USING GIN (content gin_trgm_ops);

-- Full-text search index: serves only queries written with to_tsvector(...)
-- @@ to_tsquery(...), i.e. word search rather than substring search
CREATE INDEX idx_notes_content_fts ON notes USING GIN (to_tsvector('english',
↳ content));
```

### 36.3 Vertical Scaling

**Vertical scaling** (scaling up) means moving to a larger server with more CPU, RAM, or faster storage.

As a rough order of magnitude — measure your own — the Notes API on a 1 vCPU, 1 GB RAM server (DigitalOcean Droplet: \$6/month) can handle:

- 25–75 concurrent users
- 50–150 requests per second for simple GET requests, fewer while every request opens its own TLS connection to the database (section 36.5 removes that cost)
- 20–60 requests per second for POST requests with database writes

Upgrading to a 2 vCPU, 4 GB RAM server (\$24/month):

- ~2x concurrent users (more Gunicorn workers due to more CPUs)
- ~2x requests per second
- Slower per-request latency improvement (the bottleneck shifts to the database, not the application)

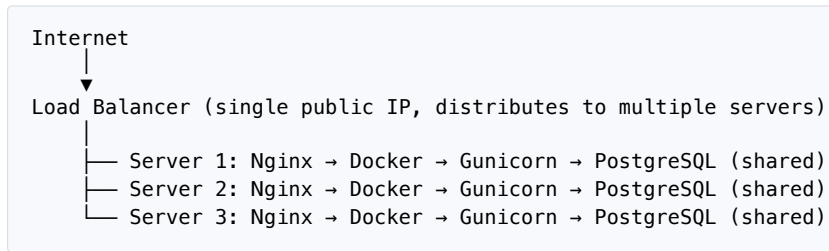
**Gunicorn worker count:** Gunicorn’s documentation suggests  $2 \times \text{cores} + 1$  as a starting point — a number that already assumes workers spend much of their time waiting for I/O. (For truly CPU-bound work, about one worker per core is enough.) The upper limit is not CPUs but memory and database connections: each worker is a full Python process (~50 MB), and each may hold a database connection. The `gunicorn.conf.py` from [Chapter 24](#) reads the count from `GUNICORN_WORKERS`, so a bigger server gets more workers without rebuilding the image:

```
On a 2 vCPU, 4 GB server: add to the docker run in deploy.sh
-e GUNICORN_WORKERS=5
```

Vertical scaling is simple — no architecture change required. Its limit is the largest server available (or affordable).

### 36.4 Horizontal Scaling

**Horizontal scaling** (scaling out) means adding more servers and distributing load across them with a load balancer.



All application servers share the same PostgreSQL database (typically a managed service). The database is a shared state; the application servers are stateless replicas.

## Prerequisites for horizontal scaling

The application must be **stateless** — no local state that persists between requests.

Stateless ✓:

- All state is in the database (PostgreSQL)
- No local file storage
- No in-memory session state
- No local caches that must be consistent across servers

Stateful ✗ (requires changes before horizontal scaling):

- Flask sessions stored in server memory (instead of database or encrypted cookie)
- File uploads written to local disk (instead of object storage like S3)
- Rate limiting counted in server memory (instead of Redis) — including Nginx's own `limit_req`: with three servers, each enforces its limit separately, so a client effectively gets three times the limit

The Notes API is naturally stateless — all application state is in the database. It can be horizontally scaled without code changes. (Operational data — logs, metrics, rate-limit counters — stays per server, and needs aggregating: [Chapter 35](#).)

## Load balancing strategies

**Round-robin**: requests distributed sequentially across servers. Simple, works when all requests have similar duration.

```
Request 1 → Server 1
Request 2 → Server 2
Request 3 → Server 3
Request 4 → Server 1
...
```

**Least connections:** new requests go to the server with the fewest active connections. Better when request durations vary.

**IP hash:** requests from the same client IP always go to the same server. Useful if you have server-local caches (but avoid this if possible — prefer statelessness).

**Weighted round-robin:** different servers handle different fractions of traffic. Useful if servers have different capacities.

## Load balancing with DigitalOcean

DigitalOcean's Load Balancer service sits in front of your Droplets:

1. Create a DigitalOcean Load Balancer (priced per node — about \$12/month per node at the time of writing)
2. Add your Droplets as targets
3. Configure health check: GET /health must return 200
4. The Load Balancer's IP becomes the public IP for the domain
5. Update the DNS A record to point to the Load Balancer's IP

```
api.notes.example.com → Load Balancer IP
Load Balancer → healthy Droplets only (if a Droplet's /health fails, it's removed
↳ from rotation)
```

## Nginx as a load balancer

For smaller setups, Nginx can act as a load balancer:

```
upstream notes_api {
 # Least connections – better for variable-duration requests
 least_conn;

 # Passive health checks (built into open-source Nginx): after 3 failed
 # requests within 30 seconds, stop using a server for 30 seconds.
 # (Active checks that probe /health on a schedule need NGINX Plus.)
 server 10.0.0.1:5000 max_fails=3 fail_timeout=30s; # Server 1
```



```

server 10.0.0.2:5000 max_fails=3 fail_timeout=30s; # Server 2
server 10.0.0.3:5000 max_fails=3 fail_timeout=30s; # Server 3
}

server {
 listen 443 ssl;
 # ssl_certificate and the other TLS settings as in Chapter 28
 location / {
 proxy_pass http://notes_api;
 }
}

```

Note what this changes on the application servers. Until now, each container was published on `127.0.0.1` only, because Nginx ran on the same machine. Here Nginx reaches the containers over the VPC, so each server publishes on its *private* address instead — `-p 10.0.0.1:5000:5000` — and a cloud firewall rule allows port 5000 only from the load balancer’s private address.

## 36.5 Connection Pooling

Each Gunicorn worker opens a database connection when a request needs one and closes it when the request ends. At 100 requests per second, that is about 100 connection setups (and 100 teardowns) per second, however many workers share the load.

Opening a PostgreSQL connection costs a TCP handshake, a TLS handshake (managed databases require TLS), and authentication — typically 10–30 ms over a VPC. At high request rates, this overhead dominates the cost of the queries themselves.

**Connection pooling** pre-opens connections and keeps them open, checking them out for a request and returning them afterward.

### PgBouncer: external connection pooler

**PgBouncer** is a lightweight connection pool that sits between the application and PostgreSQL:

```

Gunicorn workers → PgBouncer → PostgreSQL
(on 5 app servers) (manages) (20 connections)

```

Picture the horizontally scaled setup: 5 servers × 5 workers = 25 processes, each wanting its own connection — more than the smallest managed PostgreSQL plans allow (around 25 connections). With PgBouncer, they share a pool of 20.

PostgreSQL’s maximum connections (the `max_connections` setting) is a resource limit. PgBouncer allows many more application-side connections than the database can handle natively.

With a **managed** database — this book’s setup — you do not install anything: DigitalOcean’s managed PostgreSQL has PgBouncer built in, as **connection pools** you create in the control panel. Each pool gets its own connection string (a different port, and the pool’s name as the database name); point `DATABASE_URL` at it.

On a database server you run yourself, PgBouncer runs next to PostgreSQL:

```
; /etc/pgbouncer/pgbouncer.ini (apt-get install pgbouncer)
[databases]
notesdb = host=127.0.0.1 port=5432 dbname=notesdb

[pgbouncer]
listen_addr = 10.0.0.10 ; the database server's private address
listen_port = 6432 ; PgBouncer's usual port — PostgreSQL has 5432
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt ; users and password hashes
pool_mode = transaction ; a server connection per transaction
max_client_conn = 1000
default_pool_size = 20
```

The application’s connection string changes only in host and port:

<code>postgres://user:pass@db-host:5432/notesdb</code>	→ directly to PostgreSQL
<code>postgres://user:pass@db-host:6432/notesdb</code>	→ through PgBouncer

Transaction pooling has one catch: consecutive transactions from the same client may run on *different* server connections, so anything tied to a session — SET commands, session-level advisory locks, server-side prepared statements — does not carry over. The Notes API uses none of these.

## psycopg2’s built-in pool

For simpler setups, psycopg2 has a built-in connection pool. Two details make or break it. The pool must be created *in each worker process* — a connection opened before Gunicorn forks (with `--preload`, for example) would be shared by several processes at once, corrupting its protocol stream. And its connections must use the same `RealDictCursor` as before, or every `row["id"]` in the handlers breaks:

```

app/database.py – the PostgreSQL branch, with a pool
import os
import psycopg2.extras
import psycopg2.pool

_pool = None
_pool_pid = None

def _get_pool():
 """One pool per process, created on first use – never before the fork."""
 global _pool, _pool_pid
 if _pool is None or _pool_pid != os.getpid():
 _pool = psycopg2.pool.SimpleConnectionPool(
 minconn=1,
 maxconn=4, # workers × maxconn must stay below the database's limit
 dsn=DATABASE_URL,
 cursor_factory=psycopg2.extras.RealDictCursor,
)
 _pool_pid = os.getpid()
 return _pool

def get_db():
 if "db" not in g:
 g.db = _get_pool().getconn()
 return g.db

def close_db(error=None):
 db = g.pop("db", None)
 if db is not None:
 db.rollback() # handlers already committed; discard the rest
 _get_pool().putconn(db) # return to the pool instead of closing

```

SimpleConnectionPool is enough here: a sync worker handles one request at a time. (ThreadedConnectionPool is for workers that run several threads.) Size it deliberately: 5 workers × 4 connections = 20, which must fit under the database's connection limit together with everything else that connects to it. And know one new failure mode a pool brings: a connection that sat idle in the pool may have died — the database restarted, a network device dropped it — and the next request to borrow it fails. [Chapter 37](#) returns to this.

## 36.6 Caching

**Caching** stores computed results and returns the cached version for subsequent identical requests, avoiding redundant computation or database queries.

### What to cache

Good candidates for caching:

- Expensive database queries whose results change infrequently
- API responses for resources that are read much more often than written
- Computed values (markdown rendering, image resizing)

Poor candidates:

- Write operations (POST, PUT, DELETE)
- User-specific data (unless the cache is per-user)
- Data that must always be fresh (account balances, real-time status)

For the Notes API: `list_notes` and `get_note` are read-heavy. If most clients are reading the same notes repeatedly, caching could reduce database load significantly.

### Response caching with HTTP headers

The simplest caching is client-side caching via HTTP headers:

```
@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id: int):
 cursor = get_db().cursor()
 cursor.execute(sql("SELECT id, content, created_at FROM notes WHERE id = ?"),
 ↪ (note_id,))
 row = cursor.fetchone()
 if row is None:
 return jsonify({"error": f"Note {note_id} not found"}), 404

 response = jsonify(row_to_note(row))

 # Let the client itself (never a shared cache) reuse this for 60 seconds
 response.headers["Cache-Control"] = "private, max-age=60"

 # ETag: a fingerprint of the body (Werkzeug hashes it for us)
 response.add_etag()
```

```
If the request's If-None-Match matches the ETag, this turns the
response into a bodiless 304 Not Modified – keeping the ETag and
Cache-Control headers, as a 304 should
return response.make_conditional(request)
```

With `max-age=60`, the client reuses the response for 60 seconds without asking the server at all. After that, it asks with `If-None-Match`, and an unchanged note costs a small 304 instead of the full body. Note what this saves: *bandwidth* and client time. The server still ran the query to compute the ETag; reducing database load is the job of a server-side cache, next.

## Server-side caching with Redis

For shared caching across multiple application servers, use **Redis** (a fast in-memory key-value store):

```
pip install redis # and pin it in requirements.txt
```

Redis is a new service to run, secure, and monitor — on the server as a container (published on 127.0.0.1 only), or as a managed service. None of the deployment chapters provisioned it; it arrives only when measurements show the database needs relief.

```
app/cache.py
import json
import logging
from typing import Optional

import redis

from app.config import REDIS_URL # a new setting in config.py, like all others

logger = logging.getLogger(__name__)

Create the Redis client lazily (one client per process)
_redis: Optional[redis.Redis] = None

def get_cache() -> redis.Redis:
 global _redis
 if _redis is None:
 _redis = redis.from_url(REDIS_URL, socket_timeout=0.5)
 return _redis
```

*# A cache must "fail open": if Redis is down, the API gets slower (every  
# read goes to the database), but it must not start answering 500.  
# So every operation below swallows Redis errors – narrowly, and logged.*

```
def cache_get(key: str) -> Optional[dict]:
 """Retrieve a cached value. Returns None if not found, expired, or
 ↪ unavailable."""
 try:
 data = get_cache().get(key)
 except redis.RedisError:
 logger.warning("Cache unavailable (get %s)", key)
 return None
 return None if data is None else json.loads(data)

def cache_set(key: str, value: dict, ttl_seconds: int = 60) -> None:
 """Store a value in the cache with an expiration time."""
 try:
 get_cache().setex(key, ttl_seconds, json.dumps(value))
 except redis.RedisError:
 logger.warning("Cache unavailable (set %s)", key)

def cache_delete(key: str) -> None:
 """Remove a value from the cache."""
 try:
 get_cache().delete(key)
 except redis.RedisError:
 logger.warning("Cache unavailable (delete %s)", key)
```

Using the cache in the handler:

```
@notes_bp.route("/<int:note_id>", methods=["GET"])
def get_note(note_id: int):
 cache_key = f"note:{note_id}"

 # Try the cache first
 cached = cache_get(cache_key)
 if cached is not None:
 return jsonify(cached), 200 # Cache hit – no database query!

 # Cache miss – query the database
 cursor = get_db().cursor()
 cursor.execute(sql("SELECT id, content, created_at FROM notes WHERE id = ?"),
 ↪ (note_id,))
```

```

row = cursor.fetchone()

if row is None:
 return jsonify({"error": f"Note {note_id} not found"}), 404

note = row_to_note(row) # plain strings and numbers: JSON-serializable

Store in cache for 60 seconds
cache_set(cache_key, note, ttl_seconds=60)

return jsonify(note), 200

@notes_bp.route("/<int:note_id>", methods=["PUT"])
def update_note(note_id: int):
 # ... update logic, ending with db.commit() (Chapter 33) ...

 # Invalidate the cache after update
 cache_delete(f"note:{note_id}")

 return jsonify(note), 200

@notes_bp.route("/<int:note_id>", methods=["DELETE"])
def delete_note(note_id: int):
 # ... delete logic, ending with db.commit() ...

 # Invalidate the cache after delete
 cache_delete(f"note:{note_id}")

 return "", 204

```

**Cache invalidation** — removing stale entries when data changes — is the hard part of caching. The rule: whenever data changes (PUT, DELETE), immediately invalidate the corresponding cache entries. Even that leaves a small race: a GET that read the old row just before the update committed can write it into the cache just *after* the update deleted the key, and the stale copy then lives until its TTL expires. The 60-second TTL is what bounds such mistakes — which is why every cache entry should have one.

The famous quip by Phil Karlton: “There are only two hard things in Computer Science: cache invalidation and naming things.”

## 36.7 Rate Limiting

Rate limiting prevents any single client from sending too many requests, protecting the server from intentional or accidental abuse.

Chapter 28 configured Nginx-level rate limiting:

```
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s; # conf.d/ratelimit.conf
limit_req zone=api burst=20 nodelay; # in location /
```

This limits each IP address to 10 requests per second, with a burst of 20.

### Application-level rate limiting with Redis

For more sophisticated rate limiting (per API key, per user, with different limits for different endpoints):

```
app/rate_limit.py
import time
import uuid
from functools import wraps

from flask import jsonify, request

from app.cache import get_cache

def check_rate_limit(key: str, max_requests: int, window_seconds: int) -> bool:
 """
 Check if the rate limit has been exceeded.

 Uses a sliding window algorithm with Redis: a sorted set holds one
 member per request, scored by its time. Returns True if the request
 is allowed, False if rate limited.
 """
 r = get_cache()
 now = time.time()
 window_start = now - window_seconds

 pipe = r.pipeline()
 pipe.zremrangebyscore(key, 0, window_start) # Remove old requests
 # A unique member per request: two requests in the same instant (from
 # different workers, say) must count twice, not once
 pipe.zadd(key, {f"{now}-{uuid.uuid4().hex}": now}) # Record this request
```



```

 pipe.zcard(key) # Count requests in
 ↪ window
 pipe.expire(key, window_seconds + 1) # Auto-expire the key
 _, _, count, _ = pipe.execute()

 # Rejected requests are recorded too, so a client that keeps hammering
 # stays limited until it slows down
 return count <= max_requests

def rate_limit(max_requests: int = 60, window_seconds: int = 60):
 """
 Decorator to apply a per-client-IP rate limit to a view function.

 Usage:
 @notes_bp.route("/", methods=["POST"])
 @rate_limit(max_requests=10, window_seconds=60) # 10 per minute
 def create_note():
 ...

 request.remote_addr is the real client IP only with ProxyFix configured
 (Chapter 28). Without it, every client appears as the proxy's address
 and the whole world shares one bucket.
 """
 def decorator(f):
 @wraps(f)
 def decorated_function(*args, **kwargs):
 key = f"rate:{request.remote_addr}:{f.__name__}"
 if not check_rate_limit(key, max_requests, window_seconds):
 return (
 jsonify({"error": "Rate limit exceeded"}),
 429,
 {"Retry-After": str(window_seconds)},
)
 return f(*args, **kwargs)

 return decorated_function
 return decorator

```

```

Usage:
@notes_bp.route("/", methods=["POST"])
@rate_limit(max_requests=10, window_seconds=60) # Max 10 POSTs per minute per IP
def create_note():
 ...

```

(A limit per *user* or per API key follows the same pattern, with the user's ID in the key — once the API has users.)

## 36.8 Gunicorn Worker Types for Higher Concurrency

Chapter 24 used Gunicorn's default **sync** workers (one request per worker at a time). For high-concurrency I/O-bound applications, **async workers** can handle more concurrent requests with the same number of workers.

### Sync workers (default)

```
Worker 1: handles Request A → waits for database (idle) → receives result → sends
↳ response
Worker 2: handles Request B → waits for database (idle) → ...
Worker 3: handles Request C → waits for database (idle) → ...
```

While waiting for the database, the worker is idle — it can't handle other requests. More workers = more concurrency, but also more memory.

### Gevent workers (async)

```
Worker 1: starts Request A → starts Request B → starts Request C
 (while A waits for DB, B and C start)
 → A receives DB result → sends response
 → B receives DB result → sends response
 → C receives DB result → sends response
```

A single gevent worker can handle hundreds of concurrent I/O-waiting requests.

```
pip install "gunicorn[gevent]" psycogreen # quoted: zsh treats [] as a pattern
```

```
gunicorn.conf.py
worker_class = "gevent"
workers = 2 # Fewer workers needed (each can handle many concurrent
↳ requests)
worker_connections = 100 # Max concurrent requests per worker

def post_fork(server, worker):
 # psycopg2 is a C extension: gevent's monkey-patching cannot make its
 # socket I/O cooperative. psycopg2 installs a "wait callback" that
 # does – without it, every database call blocks the whole worker.
 from psycopg2.gevent import patch_psycopg2
 patch_psycopg2()
```

**Warning:** gevent uses **cooperative multitasking** — code must explicitly yield control (via I/O operations) for other greenlets to run. Code that blocks without yielding (CPU-intensive code, C libraries that gevent cannot patch — psycopg2 without psycopg2-gevent is the classic example) blocks all concurrent requests in that worker. And each concurrent request that opens a database connection holds one: 2 workers × 100 connections would overwhelm a small database, so gevent workers need a connection pool (or PgBouncer) in front of it.

For the Notes API, a simpler step comes first: **thread workers**, `worker_class = "gthread"` with `threads = 4`. psycopg2 releases Python's GIL while it waits for the database, so threads overlap that waiting without any patching. Reach for gevent when you need hundreds of concurrent requests per worker. For CPU-intensive applications, use process-based concurrency (more sync workers).

### async and await: the other road to concurrency

gevent makes ordinary code concurrent *behind its back*, by patching the libraries it calls. Python's own **asyncio** does it *in the open*. Functions declared `async def` are **coroutines**, and `await` marks each point where one may pause, typically while it waits for the network, so that an **event loop** can run another coroutine in the meantime:

```

import asyncio
import time

async def fetch(name: str, seconds: float) -> str:
 await asyncio.sleep(seconds) # stands in for a network call
 return name

async def main() -> None:
 started = time.perf_counter()
 results = await asyncio.gather(
 fetch("profile", 1.0), fetch("notes", 1.0), fetch("settings", 1.0)
)
 print(results, f"in {time.perf_counter() - started:.2f} s")

asyncio.run(main())

```

```
['profile', 'notes', 'settings'] in 1.00 s
```

Three one-second waits finished in one second, because all three waited *at the same time*, on one thread. Now replace `await asyncio.sleep(seconds)` with an ordinary blocking call, `time.sleep(1.0)`:

```
['profile', 'notes', 'settings'] in 3.03 s
```

It takes three seconds. The blocking call never gave the event loop a chance to switch, so the “concurrent” code ran one piece after another. That is the rule that shapes everything about async Python: **concurrency works only if every slow operation in the chain is itself async**. One blocking call, such as a `psycopg2` query, a `requests.post()` or a slow computation, stalls *every* request that the loop is serving.

That is why async is a whole-stack decision, not a keyword you add:

Layer	Synchronous (the Notes API)	Asynchronous
Web framework	Flask	FastAPI, Quart, Starlette
Server interface	WSGI (Gunicorn)	ASGI (Uvicorn, or Gunicorn with Uvicorn workers)
Database driver	psycopg2	psycopg 3 (async mode), asyncpg
HTTP client	requests	httpx (async), aiohttp

The payoff is large for services that spend almost all their time waiting on many slow network calls at once: a chat server holding thousands of open connections, an API gateway, or a service that calls five other services per request. It is small, or even negative, for a service like the Notes API. There, each request makes one or two fast database queries, and throughput is limited by the database, not by threads waiting. Async also does nothing for CPU-bound work, which still needs processes.

So the Notes API stays synchronous, and it has everything it needs without async. Thread workers overlap database waits. More workers and servers add capacity (section 36.4). And the one genuinely slow call the API will gain, to an email service (Chapter 37), will move out of the request entirely, into a background worker (Chapter 41). When you do meet an async codebase, the vocabulary is the same: an event loop, coroutines that `await`, and the one `sin`, a blocking call inside an `async def`.

## 36.9 Capacity Planning Summary

A practical approach to scaling the Notes API:

Traffic level	Approach (rough guide – your measurements decide)
< 20 req/sec	Single server, 2–3 sync workers. No scaling needed. \$6/month DigitalOcean Droplet.
20–100 req/sec	Single server: optimize queries, add database indexes, connection pooling (the managed database's pools), tune worker count, gthread workers. Check for N+1 queries. \$12–24/month Droplet.
100–500 req/sec	Bigger server or 2–3 servers behind a load balancer. Redis for caching frequently-read notes. A larger database plan.
500–5000 req/sec	Horizontal scaling: several application servers + load balancer. PostgreSQL read replica for read-heavy endpoints. Redis for rate limiting and caching.
Beyond	Careful profiling, a CDN, more replicas. A single well-tuned PostgreSQL serves many thousands of simple queries per second; sharding is a last resort, far beyond this – and by then this is a real business with real engineering resources.

The vast majority of web applications never reach the need for horizontal scaling. Optimize first (indexes, queries, caching), then scale vertically, then horizontally.

### Key Takeaways

- **Measure before optimizing.** Load test to find the actual bottleneck; don't guess.
- **The bottleneck** is usually the database (missing indexes, slow queries) or worker count (all workers busy). Fix the bottleneck; scaling other parts won't help.
- **Vertical scaling** (bigger server, more workers) is simple and handles most cases. Try this first.
- **Horizontal scaling** (multiple servers + load balancer) requires a stateless application. The Notes API is stateless (all state in the database) and horizontally scalable without code changes.
- **Connection pooling** (PgBouncer – built into managed databases – or a per-process psycopg2 pool) reduces the overhead of database connection setup and keeps the number of connections under the database's limit.
- **Caching** reduces database queries for frequently-read, infrequently-changing data. Cache invalidation (removing stale entries on write) is the hard part.
- **Rate limiting** in Nginx (10 req/s per IP, enabled in Chapter 28) protects the server from abuse. Application-level rate limiting with Redis enables per-user, per-endpoint limits, shared across servers.
- **Profile before optimizing code:** cProfile around the code in question (read cumtime and

ncalls), py-spy against a running process, flame graphs to see where the time goes.

- **async/await** gives many concurrent I/O waits on one thread, but only if *every* slow call in the chain is async (framework, server, database driver, HTTP client). One blocking call stalls the whole event loop. It suits services that wait on many network calls at once, not a database-bound API like the Notes API.
- **Thread and gevent workers** let one Gunicorn worker handle several I/O-waiting requests at once. Threads work with psychopg2 as it is; gevent needs psycopg2, and both need the database connection count kept in check. Neither helps CPU-bound work.

## Exercises

1. **Predict.** Before running it, guess the requests per second your Notes API sustains on its current server. Run the Locust test from section 36.1 and compare.
2. **Break and fix.** Drop the index on `note_tags(tag_id)`, fill the table with 100,000 links, and run the query behind `GET /tags/work/notes` under `EXPLAIN ANALYZE` in `psql`. Recreate the index and run it again.
3. **Extend.** Profile `GET /notes/` with `cProfile` over 100 calls and report the three functions with the highest cumulative time.
4. **Explain.** Why must the application be stateless before you can scale it horizontally? Give one example of state that would break.

Solutions and hints are in the companion repository, in `exercises/chapter-36.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

Scaling handles the load problem. But what about reliability — ensuring the system stays up even when individual components fail? The next chapter covers failure modes and reliability patterns.

→ [Chapter 37 — Reliability](#)





# Reliability

---

A system that works perfectly when everything is fine is not a production system — it is a demo. Production systems run for months and years, through hardware failures, network outages, bad deployments, and unexpected traffic patterns. **Reliability** is the property of a system that continues to work correctly in the face of these adversities.

This chapter covers the failure modes that affect web applications, the patterns and practices that make systems more reliable, and the operational habits that minimize the impact when failures do occur (because they always do).

## In this chapter

By the end of this chapter, you will understand:

- What availability means quantitatively (the “nines”)
- The failure modes specific to each layer of the Notes API stack
- How to design for graceful degradation rather than total failure
- What database backups are, why they matter, and how to verify them
- How to manage data across its lifetime: classification, **retention policy**, archival, safe cleanup, and **PII-aware deletion** (the right to be forgotten)
- How to call another service safely: timeouts, what a timeout does and does not mean, safe retries with idempotency keys, and a clear error for each kind of failure
- How to implement retries with exponential backoff and circuit breakers
- What a runbook is and why you should write one
- How to conduct a postmortem after an outage

## 37.1 Availability and the “Nines”

**Availability** is the percentage of time a system is operational and accessible. It is commonly expressed as “nines”:

Availability	Annual Downtime	Weekly Downtime
90% (one 9)	36.5 days	16.8 hours
99% (two 9s)	3.65 days	1.68 hours
99.9% (three 9s)	8.76 hours	10.1 minutes
99.99% (four 9s)	52.6 minutes	1.01 minutes
99.999% (five 9s)	5.26 minutes	6.05 seconds

For a notes API used by individuals, 99.9% availability (8.76 hours of downtime per year) is often acceptable. Achieving four 9s requires significant infrastructure investment — redundant servers, automatic failover, rigorous on-call practices.

The Notes API, running on a single DigitalOcean Droplet, realistically achieves somewhere between 99% and 99.9%, depending on:

- How often the server reboots (kernel updates, hardware failures)
- How quickly failures are detected and recovered
- How often deployments cause brief downtime

## 37.2 Failure Modes by Layer

Every layer of the stack can fail. Understanding the failure modes helps you design recovery procedures.

### The server (Droplet) itself

**Hardware failure:** DigitalOcean’s physical machines fail occasionally. Virtualization means your Droplet may be live-migrated or restarted on another machine.

- **Recovery:** when the Droplet boots again, systemd starts Docker and Docker restarts the container (its restart policy)
- **Time to recover:** typically a minute or two after the Droplet is back — if the Droplet’s disk survived. A Droplet lost for good is rebuilt from the setup chapters (21–23), which is why they are scripted and why the data lives in a managed database

**Kernel crash / out-of-memory (OOM):** if the server runs out of memory, the Linux OOM killer terminates processes to free memory.

- Which process? The one with the highest “OOM score” — often the largest memory user.

- Recovery: if a Gunicorn *worker* is killed, the Gunicorn master starts a replacement; if the whole container dies, Docker’s restart policy restarts it
- Prevention: monitor memory usage; set `--memory` limits on Docker containers to prevent one container from consuming all RAM

**Disk full:** if the disk fills up, application writes fail, logs stop recording, and the database may crash.

- Cause: unbounded log growth, large database, uploaded files
- Prevention: rotation of container logs (`daemon.json`, [Chapter 21](#)) and of Nginx logs (`logrotate`), `client_max_body_size` in Nginx ([Chapter 28](#)), disk usage alerts

## Nginx

**Certificate expiry:** TLS certificates expire. If Certbot fails to renew, HTTPS stops working.

- Prevention: Certbot’s systemd timer (`certbot.timer`) tries renewal twice daily. Monitor the certificate’s expiry date anyway.
- Detection: UptimeRobot will alert if HTTPS fails. You can also check with:

```
Check certificate expiry
echo | openssl s_client -connect api.notes.example.com:443 2>/dev/null | \
 openssl x509 -noout -dates
notBefore=Dec 16 00:00:00 2023 GMT
notAfter=Mar 15 00:00:00 2024 GMT
```

**Nginx misconfiguration:** a bad `nginx.conf` prevents Nginx from restarting.

- Prevention: always `nginx -t` (test config) before `systemctl reload nginx`
- Detection: the reload fails; Nginx continues serving with the old config (which is the correct behavior — Nginx only applies a new config if it’s valid)

## The Docker container

**Container crash loop:** if the application crashes repeatedly, Docker restarts it but keeps seeing the same failure.

- Detection: `docker ps` shows the container status as “Restarting”; `docker logs notes-api` shows the error
- Recovery: fix the root cause — usually a new image or a corrected environment file — then redeploy (`deploy.sh`, which also rolls back if the new version fails). Restarting the same

broken container only repeats the loop

**Container runs but application is unhealthy:** the container is “Up” but the application inside isn’t serving requests.

- Detection: health checks fail; `/health` returns non-200
- Recovery: `docker restart notes-api` (or investigate the log)

**The Docker daemon itself crashes:** rare, but possible.

- Prevention: `live-restore: true` in `daemon.json` (Chapter 21) keeps containers running if `dockerd` crashes
- Recovery: `systemctl restart docker`

## Gunicorn

**All workers are busy:** if all Gunicorn workers are handling requests, new requests wait in the accept queue. If the queue fills, Linux silently drops new connection attempts, and clients see timeouts.

- Detection: high response latency; Nginx 504 errors (its `proxy_read_timeout` expires while requests wait); a growing `Recv-Q` in the container’s accept queue (Chapter 36)
- Recovery: add more workers; identify the slow endpoint causing queuing. (Lowering the worker timeout does not add capacity — it sheds load by turning slow requests into errors, which can be the right emergency measure)
- Prevention: set appropriate timeout in `gunicorn.conf.py`; don’t do slow operations in handlers

**Worker timeout:** Gunicorn kills workers that take longer than timeout seconds to respond.

- Effect: the request fails: Gunicorn closes the connection, so Nginx returns 502 (or 504, if Nginx’s own `proxy_read_timeout` is shorter and expires first)
- Detection: `[CRITICAL] WORKER TIMEOUT` in Gunicorn logs
- Cause: slow database query, external API call, infinite loop
- Recovery: fix the slow operation; consider setting a shorter timeout with a 503 response

## The database

**Connection refused:** PostgreSQL is down or unreachable.

- Effect: all requests that need the database fail with 500

- Detection: health check fails; application log shows `OperationalError: could not connect`
- Recovery: with a managed database, check the provider’s status page and control panel (you cannot restart the server yourself); check network connectivity and trusted sources between app and database. On a self-hosted server: check and restart PostgreSQL

**Database is up but queries are slow:** perhaps a missing index was added to a large table; perhaps a long-running query is locking rows.

- Detection: p99 latency spikes; database slow query log shows long-running queries
- Investigation:

```
-- Find long-running queries (PostgreSQL)
SELECT pid, now() - pg_stat_activity.query_start AS duration, query, state
FROM pg_stat_activity
WHERE state != 'idle' AND now() - pg_stat_activity.query_start > interval '5
↳ seconds';

-- Cancel the running query (gentle: the session survives)
SELECT pg_cancel_backend(pid);

-- If that does not work: end the whole session (its open transaction rolls back)
SELECT pg_terminate_backend(pid);
```

**Database disk full:** PostgreSQL writes WAL files and data; if disk fills, writes fail.

- Prevention: monitor disk usage on the database server; set up alerts
- Recovery: increase the disk size (on a managed database, a plan or storage change in the control panel). Then find what grew — bloated tables, forgotten data — and deal with it deliberately, not by dropping things under pressure

## 37.3 Graceful Degradation

**Graceful degradation** means returning a partial or degraded response rather than failing completely when a dependency is unavailable.

For the Notes API, the database is the essential dependency. (If you added the Redis cache and rate limiter from [Chapter 36](#), Redis is a second one — which is why that cache code “fails open”, falling back to the database when Redis is down.) When the database is down:

```
Without graceful degradation:
@notes_bp.route("/", methods=["GET"])
def list_notes():
 db = get_db() # Raises OperationalError if DB is down
 cursor = db.cursor()
 cursor.execute("SELECT ...") # Fails
 # Flask catches the exception and returns 500
```

```
With graceful degradation:
@notes_bp.route("/", methods=["GET"])
def list_notes():
 try:
 db = get_db()
 cursor = db.cursor()
 cursor.execute("SELECT id, content, created_at FROM notes ORDER BY id DESC")
 rows = cursor.fetchall()
 except psycopg2.OperationalError as e:
 logger.error("Database unavailable for list_notes: %s", e)
 return jsonify({"error": "Service temporarily unavailable"}), 503,
 ↪ {"Retry-After": "10"}

 # ... return notes ...
```

The 503 response (instead of 500) tells the client the problem is temporary; the `Retry-After` header says when to try again.

## What can be degraded vs what cannot

Not all functionality can gracefully degrade. For the Notes API:

**Can degrade** (return a cached response):

- GET `/notes/` → return a cached note list, if there is one — otherwise 503. Never a fake *empty* list: a client that syncs could take “no notes” literally and delete its local copies
- GET `/health` → return 503 with specific subsystem status

**Cannot degrade** (must fail explicitly):

- POST `/notes/` → creating a note without the database is impossible; return 503
- DELETE `/notes/42` → cannot delete without the database; return 503

## 37.4 Database Backups

The database is the single source of truth for all application data. If the database is lost and there is no backup, all data is gone permanently.

**Backup first, deploy second** is the rule. Before any major deployment or infrastructure change, take a backup.

### PostgreSQL backup with pg\_dump

```
Create a full backup of the database. The connection string is the app's
own DATABASE_URL (with sslmode=require); --format=custom is compressed
and allows selective restores.
pg_dump "$DATABASE_URL" \
 --format=custom \
 --file=notesdb_backup_$(date +%Y%m%d_%H%M%S).dump

Verify the backup is readable
pg_restore --list notesdb_backup_20240115_140000.dump | head -20
```

Two practical traps. `pg_dump` must be at least as new as the server: Ubuntu 22.04's `postgresql-client` package is version 14, while managed databases usually run 15 or 16, and an older `pg_dump` refuses to run (“server version mismatch”) — install a matching client from the PostgreSQL project’s apt repository. And keep the password out of the command line and shell history: it is inside `DATABASE_URL`, read from the environment file, or in a `~/.pgpass` file with mode 600.

### Managed database backups (DigitalOcean)

DigitalOcean Managed Databases provide:

- **Automatic daily backups:** retained for 7 days
- **Point-in-time recovery (PITR):** restore to any moment in the retention window
- **No additional cost** for basic backup retention

This is the primary reason to use a managed database for production: backups are handled automatically.

Still, periodically create manual backups before significant changes:

```
#!/bin/bash
backup.sh – run manually before major deployments
```

```

set -euo pipefail

DATABASE_URL comes from the same environment file the container uses
set -a; source /opt/notes-api/config/notes-api.env; set +a

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="notesdb_${TIMESTAMP}.dump"
BACKUP_DIR="/opt/notes-api/backups"
BUCKET="s3://notes-api-backups" # a Spaces (or S3) bucket, private

mkdir -p "$BACKUP_DIR"

pg_dump "$DATABASE_URL" --format=custom --file="$BACKUP_DIR/$BACKUP_FILE"

Copy it off the server: a backup on the same Droplet dies with the Droplet
s3cmd put "$BACKUP_DIR/$BACKUP_FILE" "$BUCKET/"

Keep only the last 7 local copies (the bucket's own lifecycle rule
expires old objects there)
ls -lt "$BACKUP_DIR"/notesdb_*.dump | tail -n +8 | xargs -r rm --

echo "Backup created: $BACKUP_FILE ($(du -h "$BACKUP_DIR/$BACKUP_FILE" | cut -f1))"

```

## The crucial rule: test your restores

A backup that has never been tested is not a backup — it is a hope. Regularly verify that backups can actually be restored:

```

Test restore procedure (do this on a separate test environment, never production)

1. Create a test database
createdb notesdb_test

2. Restore the backup
pg_restore \
 --host=localhost \
 --username=notes_user \
 --dbname=notesdb_test \
 --no-owner \
 notesdb_backup_20240115_140000.dump

3. Verify the data looks correct
psql --host=localhost --username=notes_user --dbname=notesdb_test \

```



```

--command="SELECT COUNT(*) FROM notes;"

4. Check for a specific note to verify data integrity
psql --host=localhost --username=notes_user --dbname=notesdb_test \
--command="SELECT id, content, created_at FROM notes LIMIT 5;"

5. Drop the test database
dropdb notesdb_test

echo "Restore test successful!"

```

Schedule monthly restore tests. Document the procedure so anyone on the team can perform it.

## From backups to data management: the whole data lifecycle

Backups answer one question — *how do we not lose the data we are keeping?* But a system that only ever *accumulates* data has two slow-motion problems: it grows more expensive and slower over time, and it becomes a mounting legal and security liability. Every record you hold is a record that can be subpoenaed, breached, or demanded for deletion. “Store everything forever” is not a policy; it is the absence of one. Managing data across its lifetime — deciding what to keep, for how long, where, and how to delete the rest — is its own discipline, and it is increasingly governed by law (GDPR, CCPA, HIPAA).

### Classify the data first

You cannot write one rule for all data, because not all data is equal. Before deciding anything, classify it: **PII** (personally identifiable information — names, emails, and the *content* of a user’s notes), **sensitive** (hashed passwords, payment details, health data), **operational** (logs, metrics), and **public**. The class determines how long you may keep a thing, how it must be protected, and what you must do when a user asks you to erase it. For the Notes API, a note’s content is user-generated PII; its `id` and `created_at` are operational metadata. They sit in the same row, but they are not the same kind of data — and the difference matters as soon as you design deletion or analytics: metadata can often be kept, anonymized, long after the content must go.

### Retention policy: how long should data live?

A **retention policy** assigns each class of data a lifespan, by criteria you choose — age, account status, legal requirement. Soft-deleted notes purged after 30 days; request logs kept 90 days then deleted; database backups kept 30 days; audit logs kept seven years. The policy is simply

a table — *data category* × *how long* × *why* — but writing it down matters, because “how long do we keep X?” is a question auditors, lawyers, and engineers all eventually ask. Two forces pull in opposite directions: keep data *longer* for analytics, debugging, and compliance; keep it *shorter* for cost and to shrink the blast radius of a breach. The retention policy is where you resolve that tension on purpose instead of by neglect.

## Storage tiers and archival

Not all retained data needs to be instantly queryable. **Hot** storage (your PostgreSQL) is fast and expensive — for data in active use. **Warm** and **cold** storage (cheaper databases; object storage like S3 or Glacier) is slow and cheap — for data you must keep but rarely touch. **Archival** is moving old-but-required data from hot to cold without losing it: a year-old note nobody reads should not sit in your primary database competing for cache and slowing every query against the active set. Archive it to object storage and drop it from the hot path. The skill is matching each category’s access pattern to the cheapest tier that still meets it.

## Cleanup jobs, done safely

Retention is *enforced* by **cleanup jobs** — exactly the scheduled work from [Chapter 25](#) — that delete or archive data past its window. Deletion is dangerous (one wrong WHERE clause erases data you needed), so cleanup jobs follow safety rituals:

- **Dry-run first.** Log precisely what *would* be deleted, review it, and only then delete for real. A cleanup job with no dry-run mode is a loaded gun.
- **Delete in small, committed batches**, so a crash leaves the job partly done and safely resumable rather than rolling back one enormous transaction (or worse).
- **Soft-delete before hard-delete** where reversibility matters: mark the rows, wait out a grace period, then purge.
- **Write an audit log** of what was deleted, when, and why.

## Deletion that actually deletes

“Delete” is not one operation. **Soft delete** (the `deleted_at` pattern discussed in [Chapter 6](#); the Notes API itself uses hard deletes) hides a row, but the data is still physically present and recoverable. **Hard delete** (`DELETE FROM`) removes it from the table — yet copies persist in **backups** for the length of the backup retention window. That last point is the trap most teams miss: a user’s “deleted” data lives on in last night’s backup for another 30 days, and your privacy commitments must account for it. **Cryptographic deletion** (encrypt the data, then destroy the key) is how you render data unrecoverable when physically erasing every

copy is impractical — without the key, the ciphertext is junk. Know which of the three your obligations actually require.

## PII and the right to be forgotten

Privacy law (GDPR Article 17, CCPA) gives users the right to demand deletion of their personal data, and you must be able to honor it — which means you must know *everywhere a user's PII lives*: the primary table, search indexes, caches, logs (did you log note content? you should not have — [Chapter 16](#)), analytics pipelines, and backups. Two habits make this tractable. **Data minimization**: do not collect or copy PII you do not need — the cheapest PII to protect is the PII you never stored. **Pseudonymization**: replace identifying fields with tokens so most of your systems operate on data that is not directly identifying. Designing for deletion from day one is dramatically easier than retrofitting it onto a system that has scattered PII into a dozen places.

## Audit logs and legal holds: the exceptions

Two categories cut against normal retention. **Audit logs** — the record of who did what — frequently *must* be retained for security and compliance, and specifically must *not* be deletable on user request; their integrity is the entire point, so they are append-only by design. And a **legal hold** can override your retention policy outright: when data is relevant to litigation or investigation, you are legally required to *stop* deleting it, even data that would normally have aged out. A retention system therefore needs an off switch — the ability to suspend deletion for specified data on demand.

## A data-management matrix for the Notes API

Bring it together as the artifact every system should have but few do — a table of *data category* × *retention* × *deletion behavior*. The version below is written for the Notes API as it would be once it has **user accounts** and an audit log (today it has neither), because that is when the questions become real:

Data	Class	Retention	On account deletion	In backups
Note content	PII	While account active	Hard-deleted	Gone when the last backup holding it expires
Note metadata (id, timestamps)	Operational	While account active	Anonymized, kept for stats	As backups
Request logs	Operational (no PII)	Container logs: rotated by size; Nginx logs: 14 days	n/a (no PII logged)	n/a
Audit log	Compliance	7 years	Retained (cannot be deleted)	7 years
Backups	Mixed	7 days (managed-database backups) + 30 days off-site	(governed per category above)	—

The exact numbers are yours to choose; *having the table* is the point. When someone asks “what happens to a user’s notes when they delete their account, and how long until they are truly gone?”, you can answer in a sentence — which, increasingly, you are legally required to be able to do.

## 37.5 Calling Another Service

So far the Notes API has depended on one other system, its database. Real applications depend on many: a payment provider, an email service, a maps API, another team’s service. Each one is reached over the network, and each one can be slow, down, overloaded, or simply wrong, at any moment. How an application calls other services decides whether *their* bad day becomes *yours*.

This section adds the Notes API’s first outside dependency: an **email service**. A new endpoint emails a note to an address: `POST /notes/42/email` with `{"to": "ana@example.com"}`. The feature is small, but getting it right covers everything that matters about calling another service. The remaining sections of this chapter, retries and circuit breakers, then have a real dependency to protect.

## The email service, locally and in production

In production, the email service is a company whose job is delivering email: Amazon SES, Postmark, SendGrid, Mailgun and others. They all work the same way. You send an HTTP request with the message, and they deliver it. On your laptop you want the same shape of API without sending real email to real people. **Mailpit** is a small email catcher, run in Docker, that accepts messages exactly like a provider would, delivers none of them, and shows every message it caught in a web page:

```
docker run -d --name mailpit -p 127.0.0.1:8025:8025 axllent/mailpit
Send API: http://localhost:8025/api/v1/send
Web inbox: http://localhost:8025
```

The application reads the service's address from configuration, like everything else (Chapter 13):

```
app/config.py (added)
The email service (Chapter 37). Locally: Mailpit's send API, which
catches every message instead of delivering it.
EMAIL_API_URL: str = os.getenv("EMAIL_API_URL", "http://localhost:8025/api/v1/send")
EMAIL_API_KEY: str = os.getenv("EMAIL_API_KEY", "") # Mailpit needs none
EMAIL_FROM: str = os.getenv("EMAIL_FROM", "notes@example.com")
```

In production, EMAIL\_API\_URL and EMAIL\_API\_KEY come from the provider. The key is a secret, and lives in the server's .env file with the others (Chapter 38, section 38.5). Every provider's JSON looks slightly different, which is the first design point below.

## One module owns the conversation

The most important decision is where the calling code lives. The answer: in **one module**, which is the only place in the application that knows the provider exists:

```
app/email.py
"""
The one module that talks to the email service.

The rest of the application knows only send_email() and the exceptions
below. Switching to another provider means rewriting this file, nothing else.
"""

import logging
```

```

import random
import time
import uuid
from typing import Any

import requests

from app.config import EMAIL_API_KEY, EMAIL_API_URL, EMAIL_FROM

logger = logging.getLogger(__name__)

CONNECT_TIMEOUT = 2.0 # seconds to open the connection
READ_TIMEOUT = 5.0 # seconds to wait for the answer, once the request is sent
MAX_ATTEMPTS = 2
MAX_WAIT = 2.0 # never sleep longer than this between attempts

class EmailError(Exception):
 """Sending failed. The subclass says what the failure means."""

class EmailUnavailableError(EmailError):
 """Not sent: the service was unreachable or overloaded. Try again later."""

class EmailOutcomeUnknownError(EmailError):
 """The request went out but no answer came back: it may have been sent."""

class EmailRejectedError(EmailError):
 """The service refused the request itself: a bug on our side. Do not retry."""

def send_email(to: str, subject: str, text: str) -> str:
 """Send one email and return the service's message id."""
 payload: dict[str, Any] = {
 "From": {"Email": EMAIL_FROM},
 "To": [{"Email": to}],
 "Subject": subject,
 "Text": text,
 }
 # The same key on every attempt: a service that supports idempotency
 # keys sends the email once, however many times we ask
 headers = {"Idempotency-Key": str(uuid.uuid4())}

```

```

if EMAIL_API_KEY:
 headers["Authorization"] = f"Bearer {EMAIL_API_KEY}"

for attempt in range(1, MAX_ATTEMPTS + 1):
 started = time.perf_counter()
 wait = random.uniform(0.25, 0.75) * attempt # backoff with jitter
 try:
 response = requests.post(
 EMAIL_API_URL,
 json=payload,
 headers=headers,
 timeout=(CONNECT_TIMEOUT, READ_TIMEOUT),
)
 except requests.ReadTimeout:
 # The request reached the service; only the answer is missing.
 # Retrying could send the email twice, so we do not.
 _log(attempt, started, "no answer")
 raise EmailOutcomeUnknownError(
 f"no answer within {READ_TIMEOUT} s"
) from None
 except requests.ConnectionError as error: # includes ConnectTimeout
 _log(attempt, started, f"connection failed ({type(error).__name__})")
 failure = EmailUnavailableError("could not reach the email service")
 else:
 _log(attempt, started, f"HTTP {response.status_code}")
 if response.ok:
 return response.json()["ID"]
 if response.status_code != 429 and response.status_code < 500:
 raise EmailRejectedError(
 f"HTTP {response.status_code}: {response.text[:200]}"
)
 # 429 Too Many Requests, or a 5xx: the service's problem, maybe
 # temporary. Wait as long as it asks (Retry-After), within reason.
 retry_after = response.headers.get("Retry-After", "")
 if retry_after.isdigit():
 wait = float(retry_after)
 failure = EmailUnavailableError(f"HTTP {response.status_code}")
 if attempt < MAX_ATTEMPTS and wait <= MAX_WAIT:
 time.sleep(wait)
 else:
 break
raise failure

```

```
def _log(attempt: int, started: float, outcome: str) -> None:
 # Never the recipient or the text: both are the user's private data
 elapsed_ms = (time.perf_counter() - started) * 1000
 logger.info("Email API attempt %d: %s in %.0f ms", attempt, outcome, elapsed_ms)
```

The rest of the application sees one function, `send_email()`, and three exceptions. Nothing else knows about URLs, JSON shapes or HTTP status codes. Switching providers later means rewriting this one file. It also gives the tests a clean seam: they replace *our* `send_email()`, not the `requests` library (Part III’s “never mock what you don’t own”).

Add the HTTP library to `requirements.txt`, pinned as always: `requests==2.31.0`.

### Rule 1: every call has a timeout

`requests` waits **forever** by default. A provider that accepts the connection and never answers would hold that Gunicorn worker until the process is restarted. A few such requests, and every worker is stuck waiting, and the whole API is down because another company’s API is slow. So *every* network call gets a timeout, and here it gets two:

- **The connect timeout** (2 seconds) limits how long it may take to *open* the connection. If it expires, the request never left: nothing happened on the other side.
- **The read timeout** (5 seconds) limits how long to wait for the *answer* once the request has been sent. If it expires, the request did arrive. Whether the email was sent is unknown.

That difference matters so much that it gets a rule of its own.

### Rule 2: a timeout does not mean it failed

When a call to another service times out while waiting for the answer, there are three possibilities. The service never received it, or it received it and failed, or it received it, **sent the email**, and the answer got lost or delayed. From the caller’s side these look identical. A system that treats “timeout” as “failed” and simply tries again will, sooner or later, send the same email twice, charge the same card twice, or create the same order twice.

That is why the module has *three* failure types, each meaning something different: `EmailUnavailableError`, `EmailOutcomeUnknownError` and `EmailRejectedError`.



Failure	What happened	Safe to retry?	The API's answer
Unavailable	never got through (connection failed), or the service said “not now” (429, 5xx)	yes	503 Service Unavailable, Retry-After: 60
Outcome unknown	sent, but no answer in time	not without an idempotency key	504 Gateway Timeout: “the email may have been sent”
Rejected	the service refused our request (400, 401, 422...)	no: the request itself is wrong	502 Bad Gateway, and an error in the log

The status codes are the standard ones for a server that depends on another server. 502 and 504 say “the problem is behind me”, and 503 says “come back later”. The client learns what actually happened, including, honestly, that nobody knows.

### Rule 3: retry only what is safe, and only briefly

The module retries a failed attempt once, and only when the failure is `EmailUnavailableError`, with a short, randomized pause (the jitter of section 37.6). For a 429 Too Many Requests or 503, the service may say how long to wait in a `Retry-After` header. The module honours it, but only up to two seconds, because it is waiting inside someone’s HTTP request. A service that asks for a minute gets a 503 passed back to our own client, with our own `Retry-After`.

It never retries `EmailOutcomeUnknownError`, because a second attempt could deliver a second email. The way to make even that retry safe is an **idempotency key** (the idea of section 34.10). Each email gets a unique key, and every attempt sends the *same* key in an `Idempotency-Key` header. A provider that supports idempotency keys (many payment and some email APIs do; Mailpit ignores the header) recognizes the repeat and does the work only once. With such a provider, the read timeout could be retried as well.

(Strictly, a connection that breaks *after* the request was sent is as uncertain as a read timeout, but `requests` reports it as a `ConnectionError`, exactly like a connection that never opened, so the module cannot tell the two apart. The retry is one attempt, sent with the same idempotency key. That is an acceptable risk for a note sent by email, and would not be acceptable for a payment.)

## Rule 4: log every call, but not its contents

Each attempt writes one log line: the attempt number, the outcome, and how long it took.

```
INFO app.email | Email API attempt 1: HTTP 503 in 12 ms
INFO app.email | Email API attempt 2: HTTP 200 in 9 ms
```

When a user reports that “the email never arrived”, these lines, joined to the request by its request ID (Chapter 35), tell you whether it was ever sent. The same numbers make good metrics: the calls, their failures, and their latency, per outside service. The lines deliberately leave out the recipient and the text. Those are the user’s data, and logs are read by far more people than the database is.

## The endpoint

```
app/validation.py (added)
Deliberately loose: the only real test of an address is sending it mail
EMAIL = re.compile(r"^[^\s]+@[^\s]+\.[^\s]+")
```

```
(a method of the Validator class)
def require_email(self, field: str) -> str | None:
 """A required email address, lower-cased."""
 value = self.require_string(field, max_length=254)
 if value is None:
 return None
 if not EMAIL.fullmatch(value):
 self._add_error(f"'{field}' is not a valid email address")
 return None
 return value.lower()
```

```
app/routes/notes.py (added)
@notes_bp.route("/<int:note_id>/email", methods=["POST"])
def email_note(note_id: int):
 """Email a note: POST {"to": "someone@example.com"} -> 202 Accepted."""
 validator = Validator(request.get_json(silent=True))
 to = validator.require_email("to")
 if to is None:
 return validator.error_response()

 db = get_db()
```

```

cursor = db.cursor()
cursor.execute(sql("SELECT content FROM notes WHERE id = ?"), (note_id,))
row = cursor.fetchone()
if row is None:
 return jsonify({"error": f"Note {note_id} not found"}), 404
End the (read-only) transaction BEFORE the slow network call: never
hold a transaction open while waiting on another service (33.13)
db.rollback()

try:
 message_id = send_email(
 to=to, subject=f"Note {note_id}", text=row["content"]
)
except EmailUnavailableError:
 return (
 jsonify({"error": "The email service is unavailable; try again later"}),
 503,
 {"Retry-After": "60"},
)
except EmailOutcomeUnknownError:
 message = (
 "The email service did not answer in time; the email may have been sent"
)
 return jsonify({"error": message}), 504
except EmailRejectedError:
 logger.exception(
 "The email service rejected a request for note id=%d.", note_id
)
 return jsonify({"error": "The email could not be sent"}), 502

logger.info("Emailed note id=%d (message %s).", note_id, message_id)
202 Accepted: the service has taken the email; delivery happens later
return jsonify({"status": "accepted"}), 202

```

Four details deserve a note:

- **The transaction ends before the network call.** Reading the note opened a transaction (psycopg2 always does). `db.rollback()` ends it *before* waiting on the email service, following the rule from section 33.13: never hold a transaction open while waiting on something slow. A connection “idle in transaction” for seconds holds back the database’s clean-up work, and with locks involved it blocks other requests.
- **202 Accepted, not 200 OK.** The email service has *accepted* the message; delivery happens later, and can still fail (a full mailbox, a typo in the address). 202 says exactly that. It also

means the contract does not change when [Chapter 41](#) moves the sending into a background job: the answer will still be “accepted”.

- **The contract grows.** The new endpoint, its 202 and its three error statuses go into `openapi.yaml` in the same change (section 34.10), and the contract test checks them.
- **The request waits for the provider.** In the worst case, two attempts with their timeouts keep one Gunicorn worker busy for more than ten seconds. That is acceptable for a rarely used feature, and it is exactly the problem that Chapter 41 solves.

**Warning:** an endpoint that sends email to *any address* is a gift to spammers. They send one request per victim, and your domain’s reputation (and the provider’s patience) pays for it. Before this goes live, it needs accounts ([Chapter 38](#)), which ensure only logged-in users can send, and a per-user limit on how many emails an hour. Many products go further and send only to the user’s own, verified address.

## Testing it without sending email

Two kinds of test cover the feature. The endpoint’s tests replace `send_email()` with a **fake**: a small class that records what would have been sent, or raises the failure the test asks for.

```
tests/test_email_note.py (excerpt)
class FakeEmail:
 """Stands in for app.email.send_email: records messages, or fails on request."""

 def __init__(self):
 self.sent = []
 self.fail_with = None

 def __call__(self, to, subject, text):
 if self.fail_with:
 raise self.fail_with
 self.sent.append({"to": to, "subject": subject, "text": text})
 return "fake-message-id"

@pytest.fixture
def fake_email(monkeypatch):
 fake = FakeEmail()
 # Replace OUR function where the route looks it up, not the requests library
 monkeypatch.setattr("app.routes.notes.send_email", fake)
 return fake
```

```
def test_emailing_a_note_sends_its_content(client, create_note, fake_email):
 note = create_note("Buy milk")
 response = client.post(
 f"/notes/{note['id']}/email", json={"to": "Ana@example.com"}
)
 assert response.status_code == 202
 assert fake_email.sent == [
 {
 "to": "ana@example.com",
 "subject": f"Note {note['id']}",
 "text": "Buy milk",
 }
]
```

A parametrized test (section 17.8) then checks that each of the three failures produces its own status code.

The email module itself is tested against a **real HTTP server**: a tiny one from Python's standard library, started in a background thread for each test and scripted to misbehave. It answers 503 and then 200; it refuses with 422; or it answers only after the read timeout. There is no mock anywhere: real sockets, real timeouts, real retries.

```
tests/test_email_client.py (excerpt)
def test_a_temporary_failure_is_retried_with_the_same_key(email_service):
 email_service.script = [(503, {"Retry-After": "0"}, 0), (200, {}, 0)]
 assert email.send_email("ana@example.com", "Hi", "Hello") == "msg-1"
 keys = [r["headers"]["Idempotency-Key"] for r in email_service.requests]
 assert len(keys) == 2 and keys[0] == keys[1]

def test_a_slow_answer_means_the_outcome_is_unknown(email_service):
 email_service.script = [(200, {}, 1.0)] # answers after the read timeout
 with pytest.raises(email.EmailOutcomeUnknownError):
 email.send_email("ana@example.com", "Hi", "Hello")
 assert len(email_service.requests) == 1 # sent once, never retried
```

(The `email_service` fixture is about thirty lines: an `http.server.ThreadingHTTPServer` on a free port, whose handler records each request and replays the scripted answers. It also shortens `READ_TIMEOUT` to 0.3 seconds with `monkeypatch`, so that the one-second answer above counts as too late, and the test does not have to wait five. Its clean-up stops the server *and* closes its socket. Forgetting the second step leaves a socket open, and with `filterwarnings =`

`["error"]` in `pyproject.toml` (Chapter 17), the resulting `ResourceWarning` fails a *different* test, later in the run. That is a genuinely confusing failure, and a good example of the “shared state” cause of flaky tests in section 17.8.)

Finally, an end-to-end check by hand, against Mailpit:

```
$ curl -s -X POST http://localhost:5000/notes/1/email \
 -H "Content-Type: application/json" -d '{"to": "ana@example.com"}'
{"status":"accepted"}
The message appears in Mailpit's inbox at http://localhost:8025

$ docker stop mailpit
$ curl -s -X POST http://localhost:5000/notes/1/email \
 -H "Content-Type: application/json" -d '{"to": "ana@example.com"}'
{"error":"The email service is unavailable; try again later"}
```

The second call comes back in under a second, as a clean 503 with `Retry-After`. It is not a 500, and it is not a hang. When another service fails, this is what your service should look like.

## 37.6 Retry Logic and Exponential Backoff

Transient failures — temporary network blips, brief database overload — often resolve themselves if the operation is retried after a short wait. **Retry logic** handles these transient failures automatically.

### Naive retry (wrong)

```
for attempt in range(3):
 try:
 result = call_external_service()
 break
 except Exception:
 continue
```

Problems:

1. **Retry storm:** if the external service is down, 1000 clients all retry at the same time, creating 3000 requests that overwhelm the recovering service.
2. **No backoff:** retries happen immediately, not giving the service time to recover.
3. **No distinction:** retries transient errors (correct) and permanent errors (wrong — no point retrying a 404).

## Exponential backoff with jitter (correct)

```
import time
import random
import logging
from typing import TypeVar, Callable, Optional

logger = logging.getLogger(__name__)
T = TypeVar("T")

def retry_with_backoff(
 func: Callable[[], T],
 max_attempts: int = 3,
 base_delay: float = 0.5, # Initial delay in seconds
 max_delay: float = 10.0, # Maximum delay in seconds
 backoff_factor: float = 2.0,
 retryable_exceptions: tuple = (ConnectionError, TimeoutError),
) -> T:
 """
 Call func with exponential backoff on failure.

 Delay schedule (base_delay=0.5, backoff_factor=2):
 - Attempt 1 fails: wait 0.5 * (0.5 to 1.5) seconds (jitter)
 - Attempt 2 fails: wait 1.0 * (0.5 to 1.5) seconds
 - Attempt 3 fails: raise the exception

 The jitter (random multiplier) prevents all retrying clients
 from hitting the service at the same instant.

 Only the exceptions in retryable_exceptions are retried – pass the
 transient ones for your case. Anything else (a bug, a 404, bad
 input) is raised at once: retrying cannot fix it.
 """
 delay = base_delay
 last_exception = None

 for attempt in range(1, max_attempts + 1):
 try:
 return func()
 except retryable_exceptions as e:
 last_exception = e
 if attempt == max_attempts:
 break
 # Exponential backoff with ±50% jitter
```

```

 # (jitter spreads retries across a time window instead of synchronizing
 ↳ them)
 jittered_delay = random.uniform(delay * 0.5, delay * 1.5)
 actual_delay = min(jittered_delay, max_delay)
 logger.warning(
 "Attempt %d/%d failed: %s. Retrying in %.2f seconds.",
 attempt, max_attempts, e, actual_delay,
)
 time.sleep(actual_delay)
 delay = min(delay * backoff_factor, max_delay)

 raise last_exception

```

Using it for database operations:

```

import psycopg2
from flask import g

def get_note_with_retry(note_id: int) -> Optional[dict]:
 def attempt():
 try:
 cursor = get_db().cursor()
 cursor.execute(sql("SELECT id, content, created_at FROM notes WHERE id =
↳ ?"), (note_id,))
 return cursor.fetchone()
 except psycopg2.OperationalError:
 # get_db() keeps the connection in g.db. After a connection error
 # it is dead: throw it away, so the next attempt opens a new one
 # instead of failing on the same broken connection
 db = g.pop("db", None)
 if db is not None:
 db.close()
 raise

 return retry_with_backoff(
 attempt,
 max_attempts=3,
 base_delay=0.1,
 retryable_exceptions=(psycopg2.OperationalError,), # Only retry connection
↳ errors
 # Don't retry ProgrammingError (bad SQL) – that's a code bug
)

```



One caution: a retry inside a request handler holds a Gunicorn worker while it sleeps. Keep the delays short (here, a few hundred milliseconds in total). With two or three sync workers, long retries during an outage would tie up every worker and turn a database blip into a full outage of the API.

## The tenacity library

The tenacity library provides a more complete retry implementation:

```
pip install tenacity
```

```
import logging

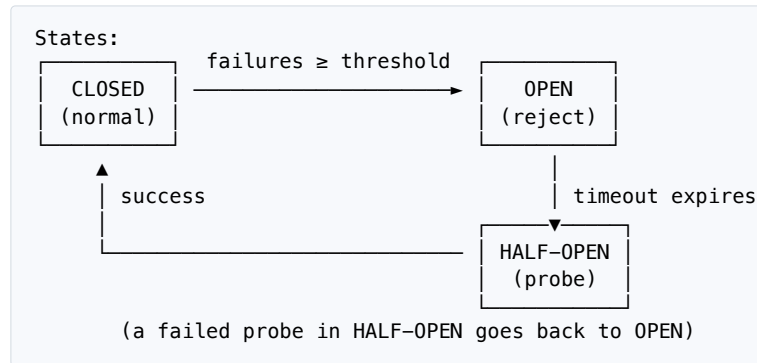
import psycopg2
from tenacity import (
 retry,
 stop_after_attempt,
 wait_random_exponential,
 retry_if_exception_type,
 before_sleep_log,
)

logger = logging.getLogger(__name__)

@retry(
 stop=stop_after_attempt(3),
 wait=wait_random_exponential(multiplier=0.5, max=10),
 retry=retry_if_exception_type(psycopg2.OperationalError),
 before_sleep=before_sleep_log(logger, logging.WARNING),
)
def fetch_all(query, params=()):
 # Opens (or reuses) the connection inside the retried function, so
 # the same connection-discarding rule as above applies
 cursor = get_db().cursor()
 cursor.execute(query, params)
 return cursor.fetchall()
```

## 37.7 Circuit Breaker Pattern

The **circuit breaker** prevents an application from repeatedly calling a failing service, giving the service time to recover and preventing the failure from cascading.



- **CLOSED (normal)**: requests pass through. If failures exceed a threshold, open the circuit.
- **OPEN (failing)**: immediately reject requests (return error without calling the service). After a timeout, try again.
- **HALF-OPEN (recovering)**: let a trial request through. If it succeeds, close the circuit. If it fails, open again.

```

Simplified circuit breaker
import time
from enum import Enum
from typing import Optional

class CircuitOpenError(Exception):
 """Raised instead of calling the dependency while the circuit is open."""

class CircuitState(Enum):
 CLOSED = "closed"
 OPEN = "open"
 HALF_OPEN = "half_open"

class CircuitBreaker:
 def __init__(self, failure_threshold: int = 5, recovery_timeout: float = 30.0,
 failure_exceptions: tuple = (ConnectionError,)):
 self.failure_threshold = failure_threshold
 # Only these count as the dependency failing. A bad-input error or
 # a "not found" is the caller's problem, and must not open the
 # circuit for everyone.
 self.failure_exceptions = failure_exceptions
 self.recovery_timeout = recovery_timeout
 self.state = CircuitState.CLOSED
 self.failure_count = 0

```

```

self.last_failure_time: Optional[float] = None

def call(self, func):
 if self.state == CircuitState.OPEN:
 if time.time() - self.last_failure_time > self.recovery_timeout:
 self.state = CircuitState.HALF_OPEN
 else:
 raise CircuitOpenError("Circuit is OPEN – request rejected")

 try:
 result = func()
 except self.failure_exceptions:
 self._on_failure()
 raise
 self._on_success()
 return result

def _on_success(self):
 self.failure_count = 0
 self.state = CircuitState.CLOSED

def _on_failure(self):
 self.failure_count += 1
 self.last_failure_time = time.time()
 if self.failure_count >= self.failure_threshold:
 self.state = CircuitState.OPEN

Usage: count only connection-level database errors as failures
db_circuit = CircuitBreaker(
 failure_threshold=5,
 recovery_timeout=30.0,
 failure_exceptions=(psycopg2.OperationalError,),
)

def fetch_note(note_id):
 def query():
 cursor = get_db().cursor()
 cursor.execute(sql("SELECT id, content, created_at FROM notes WHERE id = ?"),
 ↪ (note_id,))
 return cursor.fetchone()

 return db_circuit.call(query)

And turn an open circuit into a fast, honest 503 – not a 500:

```

```
@app.errorhandler(CircuitOpenError)
def circuit_open(error):
 return jsonify({"error": "Service temporarily unavailable"}), 503, {"Retry-After":
 ↳ "30"}
```

Two limits of this simple version. Its state lives in one process: with three Gunicorn workers there are three independent breakers, each learning about an outage on its own. And in HALF-OPEN it does not limit the trial to one request — several concurrent requests (threads, greenlets) may all get through. For a real outside dependency, such as the email service of section 37.5, a maintained library is the better choice.

For production use, consider the pybreaker library:

```
pip install pybreaker
```

## 37.8 Graceful Shutdown

When Gunicorn receives SIGTERM (from `docker stop` or `systemd`), it should:

1. Stop accepting new requests
2. Allow in-progress requests to complete
3. Close database connections cleanly
4. Exit

Chapter 24 configured this via:

- `graceful_timeout = 30` in `gunicorn.conf.py` (30 seconds to finish in-progress requests)
- JSON array `CMD` in the Dockerfile (so Gunicorn is PID 1 and receives SIGTERM directly)

What happens during a `docker stop notes-api`:

```

T+0s docker stop sends SIGTERM to PID 1 (Gunicorn master)
T+0s Gunicorn master receives SIGTERM
T+0s Gunicorn stops calling accept() – no new requests accepted
T+0s Nginx starts receiving connection errors → 502 briefly
 (this is why a single-container deploy has a short gap, and why
 blue-green deployment, Chapter 23, drains before stopping)
T+0s In-progress requests continue to completion
T+3s Last in-progress request finishes
T+3s Gunicorn workers exit gracefully
T+3s Gunicorn master exits

If requests don't finish by T+30s:
T+30s Gunicorn sends SIGKILL to remaining workers
T+30s Workers terminate immediately (requests aborted)

```

For the graceful shutdown to protect in-progress requests:

- `docker stop` waits 10 seconds by default before sending SIGKILL
- This must be longer than `graceful_timeout`

The deploy scripts (Chapter 23) therefore create the container with a longer stop timeout, which every later `docker stop` then uses:

```

35 seconds = graceful_timeout + 5 seconds margin
docker run --stop-timeout 35 ...

(or, for a single stop: docker stop --time 35 notes-api)

```

Or in `docker-compose.yml`:

```

services:
 notes-api:
 stop_grace_period: 35s

```

## 37.9 Runbooks

A **runbook** (or **playbook**) is a step-by-step guide for responding to a specific operational situation. When it is 3 AM and the service is down, you do not want to be thinking clearly from scratch — you want to follow a checklist.

## Example runbook: service is down

### # Runbook: Notes API is Down

#### ## Symptoms

- UptimeRobot alert: "notes-api is DOWN"
- /health returns 5xx or times out
- Users report they cannot access the API

#### ## Diagnostic steps (in order)

1. SSH to the server:
 

```
ssh deploy@203.0.113.42
```
2. Is the container running?
 

```
docker ps | grep notes-api
```

  - If NOT running: go to step 5 (restart procedure)
  - If running: go to step 3
3. Is the application healthy? (curl runs on the host: the image has none)
 

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:5000/health
```

  - If 200: the application is fine; the problem is in front of it – Nginx (step 7), TLS certificate, DNS, or the cloud firewall
  - If 503: a dependency is down (the body says which) – go to step 6
  - If 5xx or no answer: the application is unhealthy – go to step 4
4. View recent application logs:
 

```
docker logs --tail 50 notes-api
```

  - Look for ERROR or CRITICAL messages
  - Common causes: database connection error, OOM killed, misconfiguration
5. Restart the container (or, if it is missing, redeploy the last good version: /opt/notes-api/deploy.sh sha-<last good tag>):
 

```
docker restart notes-api
```

Wait 10 seconds, then repeat step 3

  - If 200: service restored. Monitor for recurrence.
  - If still failing: go to step 6
6. Check database connectivity from inside the container:
 

```
docker exec notes-api python -c 'import os, pycpg2;
→ pycpg2.connect(os.environ["DATABASE_URL"]); print("DB OK")'
```

  - If it fails: the database is the problem. Check the managed database's status in the DigitalOcean control panel.

```

7. Check Nginx:
 systemctl status nginx
 sudo nginx -t
 → If Nginx is stopped: sudo systemctl start nginx
 → If config error: fix the config, then sudo systemctl reload nginx

```

### ## Escalation

If service is not restored within 15 minutes: contact [senior engineer name/contact].

Write runbooks for each class of failure: service down, slow responses, database issues, certificate expiry, disk full.

## 37.10 Postmortems

A **postmortem** (also called incident review) is a written analysis of what went wrong, why, and how to prevent it in the future. It is conducted after an incident is resolved.

A good postmortem is blameless. The goal is to improve the system, not to assign fault to individuals. Any reasonably skilled engineer, in the same circumstances, would have made the same choices.

### Postmortem template

```

Postmortem: [Incident Title]

Date: 2024-01-15
Duration: 47 minutes (14:03 – 14:50 UTC)
Severity: P1 (service completely unavailable)
Author: [Name]

```

### ## Summary

The Notes API was unavailable for 47 minutes due to a certificate expiry. The certificate expired at 14:03 UTC. Every automatic renewal attempt since December 15 had failed, because DNS still pointed to the old server after a migration, and no alert existed for renewal failures or certificate expiry.

### ## Timeline

Time	Event

```

|-----|-----|
| 14:00 | Certificate expires in 3 minutes (not detected) |
| 14:03 | Certificate expires. Clients reject Nginx's expired certificate; every
↳ HTTPS request fails. |
| 14:15 | First user reports "cannot access notes" via support email |
| 14:22 | On-call engineer receives forwarded support email |
| 14:29 | Engineer identifies certificate expiry from browser error |
| 14:31 | certbot renew runs manually, succeeds |
| 14:50 | Certificate deployed, Nginx reloaded, service restored |

```

## ## Root Cause

The server was migrated on December 15, but the DNS record for `api.notes.example.com` was left pointing to the old server's IP. Certbot tried to renew twice a day for the whole final month of the certificate's life; every ACME HTTP-01 challenge went to the old server and failed. Each failure was logged to `/var/log/letsencrypt/letsencrypt.log`, but no alert was configured for Certbot failures.

## ## Contributing Factors

1. No monitoring of certificate expiry date
2. No alerting on Certbot renewal failures
3. Certbot runs from a systemd timer; its failures are not surfaced to the team
4. No runbook for certificate issues

## ## Impact

- 47 minutes of complete service unavailability
- All HTTPS requests failed
- Estimated N requests failed ( $N = \text{requests per minute} \times 47$ )

## ## Action Items

```

| Action | Owner | Due Date |
|-----|-----|-----|
| Add a certificate-expiry monitor (alert 14 days before expiry; a paid feature of
↳ some uptime services) | Alice | 2024-01-22 |
| Configure alerting on Certbot log errors | Bob | 2024-01-22 |
| Write certificate renewal runbook | Alice | 2024-01-29 |
| Test certificate renewal in staging | Bob | 2024-02-01 |

```

## ## What Went Well



- Once the cause was identified, the fix was straightforward
- The service-down runbook's diagnostic steps pointed quickly at the TLS layer

## ## What Did Not Go Well

- 12-minute gap between certificate expiry and first user report
- 7-minute gap between user report and engineer awareness
- No established runbook for certificate issues

## ## Lessons Learned

Monitoring certificate expiry is a basic reliability requirement. The absence of expiry monitoring was a gap that should have been caught during initial production setup.

Postmortems improve systems. Every incident, properly analyzed, results in improvements that prevent recurrence or reduce impact. Teams that write postmortems after every significant incident continuously improve their systems.

## Key Takeaways

- **Availability** is the percentage of time the system is operational. 99.9% means 8.76 hours of downtime per year. Achieving more requires redundancy and fast detection/recovery.
- **Every layer can fail:** hardware, Nginx, Docker, Gunicorn, and the database each have specific failure modes with specific recoveries.
- **Graceful degradation** returns a degraded response (503 with Retry-After) rather than a confusing 500 when a dependency is unavailable.
- **Database backups** are mandatory. Test restores regularly — an untested backup is an untested assumption.
- **Calling another service:** keep the conversation in one module; give every call a connect *and* a read timeout; remember that a read timeout means “unknown”, not “failed”; retry only what is safe (never a possibly-completed action without an **idempotency key**); map each kind of failure to an honest status (503, 504, 502); log every call without its private contents; and never hold a transaction open while you wait.
- **Exponential backoff with jitter** retries transient failures while spreading retry load across time to prevent retry storms.
- **Circuit breakers** stop retrying a failing service, giving it time to recover while returning immediate errors to clients.
- **Graceful shutdown** allows in-progress requests to complete before the process exits. Requires PID 1 to receive SIGTERM directly (JSON array CMD) and a container stop timeout (`--stop-timeout 35`, or `stop_grace_period` in Compose) longer than

`graceful_timeout`.

- **Runbooks** are step-by-step guides for common failure scenarios. Write them when calm so you can follow them when stressed.
- **Postmortems** analyze what went wrong and produce action items to prevent recurrence. They are blameless — the goal is to improve the system, not assign fault.
- A backup answers “don’t lose what we keep”; **data management** answers what to keep, for how long, and how to delete the rest. “Store everything forever” is a liability, not a policy.
- **Classify data** (PII / sensitive / operational / public), set a written **retention policy** per class, and use storage **tiers** (hot/warm/cold) with **archival** to control cost and query speed.
- Enforce retention with **cleanup jobs** that dry-run, batch, soft-delete-before-hard-delete, and audit. Remember deleted data lingers in **backups**; honor the **right to be forgotten** via minimization, pseudonymization, and knowing everywhere PII lives — while **audit logs** and **legal holds** are deliberate exceptions to deletion.

## Exercises

1. **Predict.** With the email service stopped, how long does `POST /notes/1/email` take to answer in the synchronous version of section 37.5, and what status does it return?
2. **Break and fix.** Remove the `timeout=` argument from the `requests.post` call, and point `EMAIL_API_URL` at a server that accepts connections but never answers (`nc -l 9999`). What happens to the worker? Restore the timeout.
3. **Extend.** Wrap `send_email` in the circuit breaker of section 37.7, so that after five failures in a row it answers `503` immediately for 30 seconds.
4. **Explain.** Why is a read timeout “unknown”, not “failed”, and what makes a retry after one safe?

Solutions and hints are in the companion repository, in `exercises/chapter-37.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What’s Next

The system is reliable. Now consider security — who can access it, how to protect it from malicious actors, and what defenses need to be in place.

→ Chapter 38 — Security

# Security

---

Security is not a feature you add at the end — it is a property that must be designed in from the beginning. The Notes API has already incorporated several security measures: HTTPS (Chapter 28), SQL injection prevention (Chapter 31), rate limiting (Chapter 28 and Chapter 36). This chapter gathers those measures into a coherent security model, fills in the remaining gaps, and explains the reasoning behind each defense.

Security thinking follows a useful mental model: **defense in depth**. No single defense is perfect. Attackers are creative and persistent. The goal is to stack multiple layers of defense so that compromising one layer does not compromise the entire system.

## In this chapter

By the end of this chapter, you will understand:

- The threat model for the Notes API (who might attack, with what goals)
- How each security measure addresses specific threats
- How to add real user accounts: password hashing with bcrypt, login tokens that can expire and be revoked, and protection that is on by default
- How to enforce authorization in every query, why the answer is 404 rather than 403, and how to test that one user can never reach another's data
- How secrets management prevents credential leaks
- What OWASP's Top 10 vulnerabilities are and how the Notes API addresses each
- How to audit your application's security posture

## 38.1 Threat Modeling

**Threat modeling** is the process of identifying who might attack your system, what they want, and how they might achieve it. It helps prioritize security work.

For the Notes API:

**Who might attack?**

- Opportunistic bots that scan for known vulnerabilities (SQL injection, default passwords, unpatched CVEs)
- Curious users who probe the API's behavior
- Competitors trying to read users' data
- Disgruntled users who try to delete others' data

### What do they want?

- Read notes that don't belong to them (confidentiality breach)
- Modify or delete others' notes (integrity breach)
- Make the service unavailable (denial of service)
- Gain access to the underlying server (full compromise)

### How might they achieve it?

- Exploit vulnerabilities in the application code (SQL injection, command injection)
- Exploit vulnerabilities in dependencies (outdated libraries)
- Steal credentials from environment variables or logs
- Brute-force weak authentication
- Send malformed requests that crash the application
- Overwhelm the server with traffic (DDoS)

## The current threat surface

The Notes API, as deployed, has:

- No authentication (any client can read and write any note). The shared API-key check sketched in [Chapter 5](#) was never carried into the restructured application.
- Public internet access on port 443 (and 80, which only redirects)
- Nothing else reachable from the internet — *provided* the container port is published on 127.0.0.1 only and the cloud firewall is in place. Docker-published ports bypass UFW ([Chapter 21](#)); a `-p 5000:5000` anywhere would put Unicorn, unencrypted, on the public internet. This is the most dangerous mistake a setup like this one invites, which is why [section 38.9's](#) checklist tests for it from the outside.

Until now, the lack of authentication was deliberate: it kept the earlier chapters focused on one thing at a time. A real notes API cannot go without it, so [sections 38.3 and 38.4](#) add accounts, login, and per-user ownership of notes, and the rest of the chapter covers the security measures beyond them.

## 38.2 Transport Security: HTTPS Everywhere

Chapter 28 configured HTTPS with TLS 1.2+ and strong cipher suites. The key security properties:

**Encryption:** all data in transit is encrypted. An attacker observing network traffic cannot read note content or API responses.

**Authentication:** the TLS certificate proves that the server is the genuine `api.notes.example.com` (not an attacker intercepting traffic). Clients verify the certificate against trusted Certificate Authorities.

**HTTP Strict Transport Security (HSTS):** the `Strict-Transport-Security` header tells browsers to only ever connect via HTTPS. A first-time visitor who types `http://api.notes.example.com` still makes one plain-HTTP request, which Nginx answers with a redirect to HTTPS. From then on, HSTS takes over: the browser will not even attempt an HTTP connection to the domain. (That first request is the remaining gap; the **HSTS preload list** built into browsers closes it for domains that register.)

```
From Chapter 28 Nginx configuration:
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains" always;
```

`max-age=63072000` is 2 years. Once a browser has seen this header, it enforces HTTPS for 2 years even if the server later removes the header.

**What HTTPS does NOT protect against:**

- Attackers who compromise the server itself (HTTPS is between client and server, not within the server)
- Mistakes in the application code (SQL injection, XSS, etc.)
- Attackers with valid certificates for a domain they've stolen (DNSSEC and Certificate Transparency help here)

## 38.3 Authentication: Who Is Asking?

**Authentication** answers “who are you?” **Authorization** (section 38.4) answers “what are you allowed to do?” The Notes API has had neither, so any client could read, change or delete any note. This section gives it real user accounts: people register with an email and a password, log in to receive a **login token**, and send that token with every request.

## The design, and the alternatives it rejects

There are several established ways to authenticate an API. The choice is worth making deliberately:

Approach	How it works	Good for	The catch
<b>Opaque login tokens</b> (this chapter)	a long random string; the server stores its hash and looks it up on every request	first-party apps and APIs	one database lookup per request
<b>JWT (JSON Web Token)</b>	a signed token that carries the user's id and an expiry inside it; the server only checks the signature	many services checking tokens without a shared database	cannot be revoked before it expires, unless you add the very lookup it was meant to avoid
<b>API keys</b>	an opaque token that does not expire, made for a program rather than a person	scripts and integrations	must be rotated by hand, and a leaked one works until someone notices
<b>“Log in with Google” (OAuth 2.0 / OpenID Connect)</b>	another company confirms who the user is, and your service trusts its answer	consumer apps; companies with a central login	a protocol with many moving parts; always use a well-tested library

The Notes API uses opaque login tokens. They are the simplest design that is fully correct. Logging out *really* ends a session (delete its row, and the token is dead at once), and a stolen database reveals only hashes of tokens, which are useless to an attacker. JWTs are often recommended for APIs, but their real advantage, checking a token without asking a database, matters only once many services must check tokens independently ([Chapter 40](#)). They pay for it with the one thing a simple service needs most: the ability to cancel a token. An API key, if the Notes API ever needs one for scripts, is built exactly like a login token without the expiry. And “Log in with Google” can be added later, *on top of* accounts like these, as a second way of proving who someone is.

The design needs two new tables and an owner for every note:

```
-- migrations/postgresql/0004_add_users_and_sessions.sql
-- (its SQLite twin differs only in the column types, as in Chapter 33)
```

```

-- Accounts, the login sessions that belong to them, and an owner for
-- every note (Chapter 38).
CREATE TABLE users (
 id SERIAL PRIMARY KEY,
 email TEXT NOT NULL UNIQUE CHECK (email = lower(email)),
 password_hash TEXT NOT NULL,
 created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- A login token is stored only as its SHA-256 hash; expires_at is a Unix
-- timestamp (seconds), which compares the same way in every database.
CREATE TABLE sessions (
 id SERIAL PRIMARY KEY,
 user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
 token_hash TEXT NOT NULL UNIQUE,
 expires_at BIGINT NOT NULL,
 created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Expand step: nullable, so the notes that already exist stay valid.
-- RESTRICT: an account that still owns notes cannot simply be deleted.
ALTER TABLE notes
 ADD COLUMN user_id INTEGER REFERENCES users(id) ON DELETE RESTRICT;
CREATE INDEX idx_notes_user_id ON notes (user_id);

```

Three details are worth a second look:

- **users.email** is **UNIQUE** and must be lower case. Ana@Example.com and ana@example.com are one person, so the application lower-cases emails, and the CHECK constraint makes sure no code path forgets.
- **sessions.expires\_at** is a plain number, seconds since 1970 (a *Unix timestamp*). It compares correctly in SQLite and PostgreSQL alike, with no time-zone handling in either.
- **notes.user\_id** is added as nullable. Notes created before accounts existed have no owner yet. This is the *expand* step of expand/contract (Chapter 6). Once every existing note has been assigned to an account (with an UPDATE run by an operator), a later migration can make the column NOT NULL: the *contract* step. Until then, notes without an owner are simply invisible, because no user's queries match them. ON DELETE RESTRICT means an account that still owns notes cannot be deleted by accident.

## Passwords are never stored, only their hashes

A password must never be stored in a form that can be turned back into the password. Not in plain text, obviously. And not encrypted either, because whoever steals the database can often steal the key as well. The server stores a **hash**: the output of a one-way function. At login it hashes the password it was given and compares the two hashes.

But the hash function matters enormously. General-purpose hashes like SHA-256 are designed to be *fast*: an ordinary graphics card computes billions per second. An attacker with a stolen table of SHA-256 password hashes can try every common password against every account in minutes. Password hashing needs the opposite: a function that is **deliberately slow**, and **salted** (mixed with a random value unique to each user), so that identical passwords produce different hashes and nothing can be precomputed. **bcrypt** is the long-standing standard. (Argon2 is the newer choice, and scrypt another; all three are fine.)

```
app/auth.py
"""
Authentication for the Notes API: passwords, login tokens, and the check
that a protected request carries a valid token.
"""

import hashlib
import secrets
import time
from datetime import datetime, timezone
from functools import wraps

import bcrypt
from flask import Blueprint, g, jsonify, request

from app.config import BCrypt_ROUNDS
from app.database import get_db, sql

TOKEN_LIFETIME_SECONDS = 30 * 24 * 60 * 60 # 30 days

— Passwords —

def hash_password(password: str) -> str:
 """A bcrypt hash; the random salt is stored inside the hash itself."""
 return bcrypt.hashpw(password.encode(), bcrypt.gensalt(BCRYPT_ROUNDS)).decode()
```



```

Checked when the email is unknown, so that a login attempt takes as long
for an unknown email as for a wrong password
_DUMMY_HASH = hash_password(secrets.token_urlsafe(16))

def check_credentials(cursor, email: str, password: str) -> dict | None:
 """The user with this email and password, or None. Never says which failed."""
 cursor.execute(
 sql("SELECT id, email, password_hash FROM users WHERE email = ?"), (email,)
)
 user = cursor.fetchone()
 password_bytes = password.encode()
 if len(password_bytes) > 72:
 # Never a valid password (register rejects it), and newer bcrypt
 # versions raise an error for it: check the dummy hash instead
 user, password_bytes = None, password_bytes[:72]
 stored = user["password_hash"] if user else _DUMMY_HASH
 password_ok = bcrypt.checkpw(password_bytes, stored.encode())
 return (
 {"id": user["id"], "email": user["email"]} if user and password_ok else None
)

— Login tokens —————

def _token_hash(token: str) -> str:
 return hashlib.sha256(token.encode()).hexdigest()

def create_session(cursor, user_id: int) -> dict:
 """Store a new login session; return the token, which the client sees once."""
 token = "ns_" + secrets.token_urlsafe(32) # 256 bits of randomness
 expires_at = int(time.time()) + TOKEN_LIFETIME_SECONDS
 cursor.execute(
 sql(
 "INSERT INTO sessions (user_id, token_hash, expires_at) "
 "VALUES (?, ?, ?)"
),
 (user_id, _token_hash(token), expires_at),
)
 expires = datetime.fromtimestamp(expires_at, timezone.utc).isoformat()
 return {"token": token, "expires_at": expires}

```

```

def end_session(cursor, token: str) -> None:
 cursor.execute(
 sql("DELETE FROM sessions WHERE token_hash = ?"), (_token_hash(token),)
)

def _user_for_token(token: str) -> dict | None:
 """The user a token belongs to, or None if it is unknown or expired."""
 cursor = get_db().cursor()
 cursor.execute(
 sql("""
 SELECT u.id, u.email
 FROM sessions s
 JOIN users u ON u.id = s.user_id
 WHERE s.token_hash = ? AND s.expires_at > ?
 """),
 (_token_hash(token), int(time.time())),
)
 row = cursor.fetchone()
 return dict(row) if row else None

def bearer_token() -> str | None:
 """The token from "Authorization: Bearer <token>", or None."""
 scheme, _, token = request.headers.get("Authorization", "").partition(" ")
 return token.strip() or None if scheme.lower() == "bearer" else None

```

# — The check —

---

```

def authenticate_request():
 """
 Set g.user from the request's token, or return a 401 response.
 (A before_request hook that returns a response ends the request there.)
 """
 token = bearer_token()
 if token is None:
 message = "Authentication required"
 else:
 g.user = _user_for_token(token)
 if g.user is not None:

```

```

 g.token = token # logout needs to know which session to end
 return None
 message = "Invalid or expired token"
 # A 401 must say which authentication scheme to use (RFC 9110)
 return jsonify({"error": message}), 401, {"WWW-Authenticate": "Bearer"}

def protect(blueprint: Blueprint) -> None:
 """Require login for EVERY route in the blueprint, even ones added later."""
 blueprint.before_request(authenticate_request)

def login_required(view):
 """Require login for a single route."""

 @wraps(view)
 def wrapped(*args, **kwargs):
 return authenticate_request() or view(*args, **kwargs)

 return wrapped

```

`bcrypt.gensalt()` takes the **cost factor**: each step up doubles the work. At cost 12, one check takes about a fifth of a second on a modern CPU. A user logging in never notices, but an attacker's billions of guesses per second shrink to a handful. The cost comes from a setting in `app/config.py`:

```

app/config.py (added)
bcrypt's cost factor: each +1 doubles the time a password check takes.
12 (a fifth of a second or so) in production; the tests use 4.
BCRYPT_ROUNDS: int = int(os.getenv("BCRYPT_ROUNDS", "12"))

```

(The tests set `BCRYPT_ROUNDS=4`, because a test suite that registers dozens of users at a fifth of a second each would crawl. Nothing in production changes.) Add the library to `requirements.txt` as usual, pinned: `bcrypt==4.1.2`.

Two more lines in that module answer questions you might otherwise not think to ask:

- **Why check a dummy hash for an unknown email?** Without it, a login for an email that does not exist returns instantly, while a wrong password takes a fifth of a second. By timing the response, anyone could find out which emails have accounts: one wrong password, and the response time tells them. Checking against `_DUMMY_HASH` makes both cases take the same time. (Measured on this machine: 0.1665 s for a wrong password, 0.1667 s for an

unknown email.) This is the same family of problem as the timing attack below.

- **Why is a token stored only as a SHA-256 hash, when passwords need bcrypt?** A password is chosen by a person, so it is guessable, and a slow hash is what protects it. A token is 256 random bits, so it cannot be guessed at any speed, and a fast hash is enough. What matters is that the database holds only hashes, so a copy of it cannot be used to log in as anyone.

## Timing attacks

The dummy hash guards against one kind of **timing attack**: learning a secret from *how long* the server takes to answer. The classic form targets string comparison. `==` on two strings stops at the first character that differs, so comparing `"abcd"` with `"axyz"` stops after the second character, and a match of several characters takes measurably longer than none:

```
Guess "aaa..." → fast (mismatch at position 0)
Guess "naa..." → slower (prefix 'n' matches)
Guess "nka..." → slower still (prefix 'nk' matches)
```

With enough measurements, an attacker can recover a secret one character at a time. This has been demonstrated against real systems. The defence, whenever code compares a secret directly, is `hmac.compare_digest(a, b)`, which takes the same time wherever the strings differ. The Notes API never compares secrets with `==`. `bcrypt.checkpw` compares in constant time internally. And tokens are found by looking up the *hash* of the token, which is not the secret itself: learning which hashes are “close” to a stored hash tells an attacker nothing about any token.

## The endpoints

```
app/routes/auth.py
import logging

from flask import Blueprint, g, jsonify, request

from app.auth import (
 check_credentials,
 create_session,
 end_session,
 hash_password,
 login_required,
)
```

```

from app.database import get_db, sql
from app.validation import Validator

logger = logging.getLogger(__name__)

auth_bp = Blueprint("auth", __name__, url_prefix="/auth")

def _credentials validator: Validator -> tuple[str | None, str | None]:
 """The email (lower-cased) and password from the request body."""
 email = validator.require_email("email")
 # Never strip a password: its spaces are part of it
 password = validator.require_string("password", strip=False, max_length=1000)
 return email, password

@auth_bp.route("/register", methods=["POST"])
def register():
 """Create an account: {"email", "password"} -> 201 {"id", "email"}."""
 validator = Validator(request.get_json(silent=True))
 email, password = _credentials(validator)
 if password is not None:
 if len(password) < 12:
 validator.errors.append(
 "'password' must be at least 12 characters long"
)
 elif len(password.encode()) > 72: # bcrypt reads only the first 72 bytes
 validator.errors.append("'password' cannot be longer than 72 bytes")
 if email is None or password is None or validator.has_errors():
 return validator.error_response()

 db = get_db()
 cursor = db.cursor()
 cursor.execute(
 sql("""
 INSERT INTO users (email, password_hash) VALUES (?, ?)
 ON CONFLICT (email) DO NOTHING
 RETURNING id, email
 """),
 (email, hash_password(password)),
)
 user = cursor.fetchone()
 if user is None:
 return jsonify({"error": "An account with this email already exists"}), 409

```

```

db.commit()
logger.info("Registered user id=%d.", user["id"]) # the id; never the email
return jsonify(dict(user)), 201

@auth_bp.route("/login", methods=["POST"])
def login():
 """Exchange email and password for a token: 200 {"token", "expires_at"}."""
 validator = Validator(request.get_json(silent=True))
 email, password = _credentials(validator)
 if email is None or password is None or validator.has_errors():
 return validator.error_response()

 db = get_db()
 cursor = db.cursor()
 user = check_credentials(cursor, email, password)
 if user is None:
 logger.warning("Failed login attempt.")
 # 401 needs WWW-Authenticate (RFC 9110); "Bearer" names the scheme
 # that the token from a successful login is used with
 return (
 jsonify({"error": "Invalid email or password"}),
 401,
 {"WWW-Authenticate": "Bearer"},
)

 session = create_session(cursor, user["id"])
 db.commit()
 logger.info("User id=%d logged in.", user["id"])
 return jsonify(session), 200

@auth_bp.route("/session", methods=["DELETE"])
@login_required
def logout():
 """Log out: delete this session, so its token stops working at once."""
 db = get_db()
 end_session(db.cursor(), g.token)
 db.commit()
 logger.info("User id=%d logged out.", g.user["id"])
 return "", 204

```

(create\_app()) registers auth\_bp like the other blueprints. The email check is the Validator's require\_email(), added in section 37.5.) In use:

```
$ curl -s -X POST https://api.notes.example.com/auth/register \
 -H "Content-Type: application/json" \
 -d '{"email": "ana@example.com", "password": "correct horse battery"}'
{"email":"ana@example.com","id":1}

$ curl -s -X POST https://api.notes.example.com/auth/login \
 -H "Content-Type: application/json" \
 -d '{"email": "ana@example.com", "password": "correct horse battery"}'
{"expires_at":"2024-02-14T14:00:05+00:00","token":"ns_Qe3k..."}

$ curl -s https://api.notes.example.com/notes/ -H "Authorization: Bearer ns_Qe3k..."
{"count":0,"notes":[]}

$ curl -s -X DELETE https://api.notes.example.com/auth/session \
 -H "Authorization: Bearer ns_Qe3k..."
```

Each decision in these handlers protects against something specific:

- **One error for both failures.** “Invalid email or password” never reveals which of the two was wrong, and the dummy hash makes the timing match too. Registration *does* reveal that an email is taken (409 Conflict). Almost every site accepts that trade-off, because “this email already has an account” is useful to real users. The protections against someone abusing it are the rate limits below.
- **Passwords: at least 12 characters, no composition rules.** Length is what makes a password hard to guess. Rules like “one digit and one symbol” mostly produce Password1!. Current guidance (NIST SP 800-63B) says exactly this, and adds that sites should check new passwords against lists of known-leaked ones. The upper limit of 72 *bytes* comes from bcrypt itself, which reads only the first 72 bytes of a password. Older versions of the library silently ignore the rest, and newer ones raise an error. Rejecting longer passwords up front makes the behaviour clear.
- **Logout is DELETE /auth/session.** Logging out destroys the session, so DELETE is the honest HTTP method. It also needs no body, and DELETE is not among the methods that the “writes must be JSON” check of [Chapter 16](#) covers (POST, PUT and PATCH).
- **The contract says it too.** `openapi.yaml` (section 34.10) gains the three endpoints and a security scheme, `bearerAuth: {type: http, scheme: bearer}`, applied to every protected path, so that client developers see exactly which calls need a token.
- **Nothing secret is logged.** The handlers log user *ids*, never emails, passwords or tokens. The request log ([Chapter 16](#)) records only the method and path, never the `Authorization` header.

## Protecting routes by default

`protect()` attaches the token check as a `before_request` hook to a whole blueprint:

```
app/routes/notes.py (and the same in app/routes/tags.py)
from app.auth import protect

notes_bp = Blueprint("notes", __name__, url_prefix="/notes")
protect(notes_bp) # every /notes/ route requires login
```

The alternative, a decorator on each route, works, and `login_required` provides one for single routes such as `logout`. But it makes security something each new route has to *remember*. With `protect()`, a route added to the notes blueprint next year is protected before anyone thinks about it. Secure by default beats secure by discipline. The public routes, `/health`, `/ping`, `/auth/register` and `/auth/login`, live in blueprints that are simply not protected.

## Limiting login attempts

A login endpoint is an open invitation to guess passwords. `bcrypt` makes each guess expensive for someone who has stolen the database. Against someone guessing through the API itself, the defence is a limit on how often one client may try. The general `api` zone from [Chapter 28](#) allows 10 requests a second, which is far too generous for logins. Give the login endpoint a zone of its own:

```
/etc/nginx/conf.d/ratelimit.conf (added): 5 login attempts a minute per IP
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;
```

```
In the server block, next to "location /": the exact-match location wins
for this one path, so logins get the strict limit instead of the general one
location = /auth/login {
 limit_req zone=login burst=5 nodelay;
 limit_req_status 429;
 proxy_pass http://notes_api;
 proxy_http_version 1.1;
 proxy_set_header Connection "";
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
}
```



With `burst=5 nodelay`, the first attempt and five more are let through at once, and the seventh in quick succession gets a 429. After that, one attempt is allowed every twelve seconds. (Tested against a real Nginx with this exact configuration: 401 401 401 401 401 401 429 429, where the 401s are wrong passwords reaching the application.)

A per-IP limit does not stop an attacker who spreads guesses over thousands of addresses, each trying one common password against many accounts (“password spraying”). Larger services therefore also count failures *per account*, and alert on unusual volumes of failed logins in total (Chapter 35). Both belong on the list of things to add as the service grows.

## Password reset, in outline

Every real service needs a way back in for someone who has forgotten their password. The Notes API does not build one, because a reset email should be sent in the background, with the machinery that Chapter 41 builds. But the design is standard, and every step of it defends against something:

1. The user asks for a reset by entering their email. The response is always the same (“if an account exists, we have sent an email”), so the form cannot be used to discover accounts.
2. The server creates a **single-use reset token**. Like a login token it is random, stored only as a hash, and short-lived (an hour at most). The server emails a link containing it.
3. The link lets the user choose a new password. Using the token deletes it, and so does every other open reset token for that account.
4. After a reset, **all of the user’s sessions are deleted**. If an attacker was logged in, they are now logged out.

## Tests

Authentication brings a practical problem for the test suite: every existing test calls `/notes/`, which now requires a token. The fixtures solve it once, in `conftest.py`. The `client` fixture that every test already uses becomes a client that is *logged in* as a test user, so none of the existing tests has to change:

```
tests/conftest.py (added and changed)
os.environ["BCRYPT_ROUNDS"] = "4" # next to the other test settings

@pytest.fixture
def anon_client(app):
 """A test client that is not logged in."""
```

```

 return app.test_client()

@pytest.fixture
def login_as(app):
 """
 A factory: login_as("alice@example.com") registers that user, logs in,
 and returns a test client that sends the user's token with every request.
 """

 def _login(email: str, password: str = "correct horse battery") -> FlaskClient:
 client = app.test_client()
 client.post("/auth/register", json={"email": email, "password": password})
 response = client.post(
 "/auth/login", json={"email": email, "password": password}
)
 assert response.status_code == 200, response.get_json()
 token = response.get_json()["token"]
 client.environ_base["HTTP_AUTHORIZATION"] = f"Bearer {token}"
 return client

 return _login

@pytest.fixture
def client(login_as):
 """
 The client most tests use: logged in as a test user, so tests written
 before authentication existed keep working unchanged.
 """
 return login_as("tester@example.com")

```

(from flask.testing import FlaskClient joins the imports at the top, and clean\_database now also runs DELETE FROM users, whose sessions go with them through ON DELETE CASCADE.) environ\_base holds values that the test client adds to every request it sends. This is the factory-fixture idea from [Chapter 17](#), applied to logged-in users.

A few of the new tests in tests/test\_auth.py:

```

def test_a_wrong_password_and_an_unknown_email_look_the_same(anon_client):
 register(anon_client)
 wrong_password = anon_client.post(
 "/auth/login",

```

```

 json={"email": "ana@example.com", "password": "wrong password!"},
)
 unknown_email = anon_client.post(
 "/auth/login", json={"email": "nobody@example.com", "password": PASSWORD}
)
 assert wrong_password.status_code == unknown_email.status_code == 401
 assert wrong_password.get_json() == unknown_email.get_json()

def test_an_expired_token_is_rejected(client, app):
 with app.app_context():
 from app.database import get_db

 db = get_db()
 db.cursor().execute("UPDATE sessions SET expires_at = 0") # expire them all
 db.commit()
 assert client.get("/notes/").status_code == 401

def test_logout_ends_the_session(client):
 assert client.get("/notes/").status_code == 200
 assert client.delete("/auth/session").status_code == 204
 assert client.get("/notes/").status_code == 401

```

## 38.4 Authorization: What May They Do?

Knowing *who* is asking is only half the job. The other half is making sure every request touches only what that person is allowed to touch. The Notes API's rule is simple: **a note belongs to the user who created it, and nobody else can see it, change it, or learn that it exists.**

### The check lives in the query

The most reliable place to enforce ownership is the database query itself. Every query on notes carries `AND user_id = ?`, with the id of the logged-in user from `g.user`:

```
app/routes/notes.py: create stores the owner...
cursor.execute(
 sql(
 "INSERT INTO notes (content, user_id) VALUES (?, ?) "
 "RETURNING id, content, created_at"
),
 (content, g.user["id"]),
)

...and every read, update and delete asks for "this note, if it is mine"
cursor.execute(
 sql("SELECT id, content, created_at FROM notes WHERE id = ? AND user_id = ?"),
 (note_id, g.user["id"]),
)
```

The same condition goes into the list query (`WHERE user_id = ?`, together with section 34.9's pagination), `update_note`, `delete_note`, both tag handlers, `GET /tags/<name>/notes`, and the email endpoint of section 37.5, which must never email someone else's note. That makes nine queries, and none of them may be missed. Finding one that was missed is the most common serious security bug in web applications: it heads the OWASP Top 10 (A01, section 38.6).

A query written this way cannot leak, whatever the rest of the handler does. The alternative, loading the note and then checking `if note["user_id"] != g.user["id"]` in Python, works too. But it spreads the rule across code paths, and each new handler is another chance to forget it.

## Every shortcut must respect the boundary

**Warning:** the ownership check in the query protects only code that runs the query. Anything that answers *without* querying bypasses it, and the classic culprit is a cache. The Redis cache of [Chapter 36](#) (section 36.6) stores a note under the key `note:{note_id}` and serves a cache hit before any SQL runs. Once notes have owners, that is a data leak: Alice reads note 42, it is cached, and Bob's request for note 42 is answered from the cache. The fix is to make the owner part of every cache key, `f"note:{g.user['id']}:{note_id}"`, and to use the same key when an update or delete removes the entry. The HTTP caching of that section is already safe: `Cache-Control: private` tells shared caches (proxies, CDNs) never to store the response.

The same question applies to anything that stores or reuses responses: search indexes, exports, background jobs, logs. For each one, ask whose data it holds, and who can reach it.

## 404, not 403

When Bob asks for Alice’s note 42, the query finds nothing, because nothing matches both `id = 42` and `user_id = <Bob>`. The handler answers exactly as it would for a note that does not exist: 404 Not Found. A 403 Forbidden would be technically accurate, but it would confirm that note 42 exists and belongs to someone. That is a small leak, but a leak all the same. Use 403 when the caller may know the thing exists but may not do this particular action to it, for example a read-only member of a shared notebook trying to edit it.

## Roles, and sharing, when you need them

Ownership is the simplest authorization model, and it is enough for the Notes API as it stands. Two extensions come up in almost every real product:

- **Roles.** A role column on users (`'member'`, `'admin'`) and a check such as `if g.user["role"] != "admin": return 404` for administrative endpoints. Keep role checks in one place (a `require_role("admin")` helper, like `protect()`), never scattered through the handlers.
- **Sharing.** Letting Alice give Bob access to one note turns “the note’s owner” into “the people with access”, stored in a join table (`note_shares(note_id, user_id, permission)`). The queries become joins, as in [Chapter 33](#). The project at the end of Part IV builds exactly this.

## Tests that prove the boundary

Authorization is the kind of rule that one forgotten `AND user_id = ?` silently breaks, so it deserves tests that try every door:

```
tests/test_authorization.py
"""Every note belongs to one user, and nobody else can see or change it."""

import pytest

@pytest.fixture
def alice(login_as):
 return login_as("alice@example.com")

@pytest.fixture
def bob(login_as):
```

```

 return login_as("bob@example.com")

@pytest.fixture
def alices_note(alice):
 response = alice.post("/notes/", json={"content": "Alice's private note"})
 return response.get_json()

@pytest.mark.parametrize(
 ("method", "path", "body"),
 [
 ("get", "/notes/{id}", None),
 ("put", "/notes/{id}", {"content": "Bob was here"}),
 ("delete", "/notes/{id}", None),
 ("get", "/notes/{id}/tags", None),
 ("put", "/notes/{id}/tags", {"tags": ["hacked"]}),
],
)
def test_another_users_note_does_not_exist_for_bob(
 bob, alices_note, method, path, body
):
 url = path.format(id=alices_note["id"])
 response = getattr(bob, method)(url, json=body)
 # 404, not 403: Bob must not even learn that the note exists
 assert response.status_code == 404

def test_bobs_attempts_changed_nothing(alice, bob, alices_note):
 bob.put(f"/notes/{alices_note['id']}", json={"content": "Bob was here"})
 bob.delete(f"/notes/{alices_note['id']}")
 response = alice.get(f"/notes/{alices_note['id']}")
 assert response.status_code == 200
 assert response.get_json()["content"] == "Alice's private note"

```

With these in place, the full suite passes on SQLite and on PostgreSQL. Try deleting `AND user_id = ?` from any one query: at least one of these tests fails.

## 38.5 Secrets Management

A **secret** is a credential that must be kept confidential: database passwords, API keys, TLS private keys. If a secret is exposed, an attacker can use it to gain unauthorized access.

## Where secrets must NOT be stored

**Source code (git repositories):** git history is permanent. Even if you remove a secret from a file, it remains in the git history forever. Anyone who clones the repository can find it.

```
NEVER DO THIS:
DATABASE_URL = "postgresql://notes_user:SuperSecret123@db.example.com/notesdb"
SECRET_KEY = "hardcoded-flask-secret-key-please-dont-do-this"
```

**Dockerfile:** Docker image layers are often pushed to public registries and inspected by others.

```
NEVER DO THIS:
ENV DATABASE_URL=postgresql://notes_user:SuperSecret123@db/notesdb
```

**Log files:** logs may be stored long-term, shipped to third parties, or accidentally made public.

```
NEVER DO THIS:
logger.info("Connecting to database: %s", os.environ["DATABASE_URL"])
This logs the password!
```

## Where secrets SHOULD be stored

**Environment variables:** passed to the container at startup, not visible in the image.

```
The deploy.sh script (Chapter 23) passes secrets as environment variables:
docker run -d \
 --name notes-api \
 --env-file /opt/notes-api/config/notes-api.env \
 ...
 ghcr.io/your-username/notes-api:sha-a1b2c3d
```

```
/opt/notes-api/config/notes-api.env
This file has permissions 600 (owner read/write only)
DATABASE_URL=postgresql://notes_user:<password>@private-db-...db.ondigitalocean.com:2
↪ 5060/notesdb?sslmode=require
SECRET_KEY=<the output of: python3 -c "import secrets; print(secrets.token_hex(32))">
```

Environment variables keep secrets out of the image and the repository, but they are not a vault. Anyone who can run `docker inspect notes-api` — every member of the docker group — can read them, as can anything that reads the process's `/proc/<pid>/environ`. Treat access to the server's Docker as access to the secrets.

**Secrets management services:** for larger teams:

- **HashiCorp Vault:** open-source, self-hosted, full-featured
- **AWS Secrets Manager:** managed, integrates with IAM
- **GCP Secret Manager:** managed, integrates with GCP IAM
- **Doppler:** developer-focused secrets management

These services:

- Store secrets encrypted at rest
- Provide audit logs of who accessed which secret
- Support automatic secret rotation
- Allow secrets to be updated without redeploying the application

## Checking for accidentally committed secrets

Before pushing code to a repository:

```
Install detect-secrets (Yelp's scanner) and the pre-commit framework
pip install detect-secrets pre-commit

Record the current state: known, accepted findings go in the baseline
detect-secrets scan > .secrets.baseline
```

Then let the pre-commit framework run it before every commit, on the files being committed:

```
.pre-commit-config.yaml
repos:
- repo: https://github.com/Yelp/detect-secrets
 rev: v1.5.0
 hooks:
 - id: detect-secrets
 args: ["--baseline", ".secrets.baseline"]
```

```
pre-commit install # installs the hook into .git/hooks/pre-commit
```

**GitHub secret scanning:** GitHub automatically scans repositories for known secret patterns (AWS keys, GitHub tokens, Stripe API keys) and alerts you if they are committed. This is a safety net, not a primary defense.



## The .gitignore for secrets

```
.gitignore – created in Chapter 6; these are the secret-specific lines

Environment files (may contain secrets)
.env
.env.*
*.env
...except the example, which holds no real secrets. (Comments must be on
lines of their own: text after a pattern becomes part of the pattern.)
!.env.example

Local configuration
config/local.py
notes-api.env

TLS private keys
*.pem
*.key
```

## 38.6 OWASP Top 10

The Open Web Application Security Project (OWASP) publishes a list of the 10 most critical web application security risks. Every web developer should be familiar with them. This section follows the **2021 edition**; OWASP revises the list every few years (a 2025 edition reshuffles it, adding software supply-chain failures and the mishandling of exceptional conditions as their own categories), but the risks below remain the core.

### A01: Broken Access Control

**What it is:** users can access resources or perform actions they should not be able to.

**Example:** user A can delete user B's notes by calling `DELETE /notes/42` even though note 42 belongs to user B.

**Notes API mitigation** (section 38.4): every query filters by `user_id`:

```
cursor.execute(
 sql("DELETE FROM notes WHERE id = ? AND user_id = ?"),
 (note_id, g.current_user["id"]),
)
If note_id doesn't belong to the current user, nothing is deleted
(cursor.rowcount == 0) and the handler answers 404
```

## A02: Cryptographic Failures

**What it is:** data transmitted or stored without adequate encryption.

**Notes API mitigation:**

- HTTPS (TLS 1.2+) for all data in transit — including to the database (`sslmode=require`)
- Database reached over the private VPC network, with its public endpoint restricted to trusted sources
- API keys stored as SHA-256 hashes, not plaintext
- Environment variables for secrets (not hardcoded)

## A03: Injection

**What it is:** user-supplied data is interpreted as code. SQL injection is the most common form.

**Notes API mitigation:** [Chapter 31](#) — parameterized queries for all database operations: values are never interpolated into SQL strings. (The one piece of SQL built from input, the sort column, comes from an allow-list.)

```
VULNERABLE:
cursor.execute(f"SELECT * FROM notes WHERE id = {note_id}")

SAFE:
cursor.execute(sql("SELECT * FROM notes WHERE id = ?"), (note_id,))
```

## A04: Insecure Design

**What it is:** fundamental design flaws that no amount of coding fixes. An insecure design cannot be fixed by just adding security controls around it.

**Notes API design considerations:**

- Authentication is a design-level requirement, not an afterthought
- User data is isolated by `user_id` at the database query level
- Rate limiting is designed in, not bolted on

## A05: Security Misconfiguration

**What it is:** default configurations left unchanged, unnecessary features enabled, error messages disclosing system internals.

**Notes API mitigations:**

- Debug mode disabled in production (`FLASK_DEBUG=false`)

- No internal error details in error responses (just “An unexpected error occurred”)
- Server headers reduced: Nginx replaces Gunicorn’s Server header with its own; add `server_tokens off`; to the Nginx configuration so it does not advertise its version either
- Security headers added (HSTS, X-Content-Type-Options, X-Frame-Options)
- Minimal Docker image (no debug tools, no compilers)
- Non-root user in Docker container

Check for security misconfiguration:

```
app/startup.py – an extra startup check for production
from app.config import DATABASE_URL, FLASK_DEBUG, SECRET_KEY

KNOWN_PLACEHOLDERS = ("CHANGE", "change", "<", "your-", "example")

def check_security_configuration():
 issues = []
 if FLASK_DEBUG:
 issues.append("FLASK_DEBUG is on – must be off in production")
 if any(marker in SECRET_KEY for marker in KNOWN_PLACEHOLDERS):
 issues.append("SECRET_KEY looks like a placeholder – generate a random key")
 if DATABASE_URL.startswith("sqlite:///"):
 issues.append("Using SQLite in production – use PostgreSQL")
 for issue in issues:
 logger.warning("Security configuration issue: %s", issue)
```

(A placeholder that someone forgot to replace is the realistic risk: it passes the length check from Chapter 14 while being published in the repository’s own `.env.example`.)

## A06: Vulnerable and Outdated Components

**What it is:** using libraries with known security vulnerabilities.

**Notes API mitigation:** Chapter 17 introduced `pip-audit`:

```
pip-audit

Output (example):
Name Version ID Fix Versions

werkzeug 3.0.1 GHSA-2g68-c3qc-8985 3.0.3
```

Run `pip-audit` in CI. Update dependencies regularly. Subscribe to security advisories for major dependencies (Flask, Werkzeug, pycopg2).

## A07: Identification and Authentication Failures

**What it is:** weak authentication implementation — weak passwords, missing rate limiting on authentication, insecure session management.

**Notes API mitigations** (section 38.3):

- Passwords hashed with bcrypt at cost 12 (never SHA-256, which is far too fast for passwords), with a minimum length of 12 and no composition rules
- Login tokens with 256 bits of randomness, stored only as hashes, expiring after 30 days, and revoked at logout
- One error message, and equal timing, for an unknown email and a wrong password
- A strict rate limit on /auth/login (5 attempts a minute per IP), on top of the general limit

## A08: Software and Data Integrity Failures

**What it is:** not verifying the integrity of software updates, critical data, or CI/CD pipelines.

**What the Notes API does today:**

- Docker images built only in CI and tagged by commit (traceable to the exact code; tags are kept unchanged by convention, or made immutable by the registry — [Chapter 19](#))
- The deploy action pinned to a full commit SHA; the CI deploy key restricted to one command ([Chapter 23](#))

**What to adopt next:**

- `pip install --require-hashes` (verify package checksums)
- The base image pinned by digest (`python:3.11-slim@sha256:...`), so rebuilding the same commit gives the same image
- OIDC federation between CI and the cloud provider, replacing long-lived secrets such as the deploy key

```
Generate requirements with hashes:
pip-compile --generate-hashes requirements.in

requirements.txt with hashes (auto-generated):
flask==3.0.0 \
 --hash=sha256:abc123... \
 --hash=sha256:def456...
```

## A09: Security Logging and Monitoring Failures

**What it is:** not logging security-relevant events, or not monitoring for attacks.

**Notes API mitigations** (Chapter 35):

- All requests logged with IP, method, path, status code
- Failed authentication attempts logged (if you implement auth)
- Error rates monitored with alerting
- Nginx access logs capturing all traffic

Log these security events explicitly:

```
In the auth middleware:
if user is None:
 logger.warning(
 "Authentication failed",
 extra={
 "remote_addr": request.remote_addr,
 "user_agent": request.headers.get("User-Agent"),
 "path": request.path,
 },
)
 return jsonify({"error": "Invalid API key"}), 401
```

## A10: Server-Side Request Forgery (SSRF)

**What it is:** an attacker tricks the server into making HTTP requests to internal resources on their behalf.

**Example:** the API accepts a URL parameter and fetches content from it. An attacker passes `http://169.254.169.254/latest/meta-data/` (AWS metadata endpoint) to read internal cloud metadata.

**Notes API mitigation:** the Notes API doesn't accept URLs as input or make outbound HTTP requests. If you add features that do (e.g., fetching URL previews), validate and restrict the allowed URLs:

A blocklist of hostnames is not enough: `http://127.1/`, `http://[::1]/`, `http://2130706433/` (127.0.0.1 as a decimal number), or a public DNS name that *resolves* to an internal address all get past it. Check the address the name actually resolves to, and refuse to follow redirects:

```

import ipaddress
import socket
from urllib.parse import urlparse

ALLOWED_SCHEMES = {"http", "https"}

def validate_url(url: str) -> bool:
 """Allow only http(s) URLs whose host resolves to public addresses."""
 parsed = urlparse(url)
 if parsed.scheme not in ALLOWED_SCHEMES or not parsed.hostname:
 return False
 try:
 infos = socket.getaddrinfo(parsed.hostname, parsed.port or 443)
 except socket.gaierror:
 return False
 for info in infos:
 addr = ipaddress.ip_address(info[4][0])
 # Loopback, private (10/8, 172.16/12, 192.168/16, ...), link-local
 # (169.254/16 – where cloud metadata services live, DigitalOcean's
 # included), and other reserved ranges are all refused
 if not addr.is_global:
 return False
 return True

When fetching: requests.get(url, allow_redirects=False, timeout=5) – a
redirect to an internal address would otherwise undo the check. And
connect to the address you checked: DNS can answer differently the
second time ("DNS rebinding").

```

Where possible, go further: allow only a fixed list of domains the feature actually needs.

## 38.7 Security Headers Deep Dive

Chapter 28 configured security headers in Nginx. Let's understand what each one does.

### Strict-Transport-Security (HSTS)

```
Strict-Transport-Security: max-age=63072000; includeSubDomains
```

Tells browsers: for the next 2 years, only connect to this domain via HTTPS. Even if the user types `http://`, the browser silently upgrades to HTTPS. Prevents protocol downgrade attacks.

`includeSubDomains` applies the policy to all subdomains. Be careful — this means ALL

subdomains must have valid HTTPS.

## X-Content-Type-Options

```
X-Content-Type-Options: nosniff
```

Prevents browsers from “sniffing” the content type and treating a response as something other than what the Content-Type header says. Prevents attacks where malicious data is uploaded and served as JavaScript.

## X-Frame-Options

```
X-Frame-Options: DENY
```

Prevents the page from being embedded in an `<iframe>`. Protects against **clickjacking** — an attack where an attacker overlays an invisible iframe over a legitimate button, tricking users into clicking the attacker’s content.

For an API, DENY is always correct.

## X-XSS-Protection

This header controlled the browser’s built-in XSS filter, which modern browsers have removed (it could itself introduce vulnerabilities). Chapter 28’s configuration leaves it out; if you meet it in an older configuration, the only sensible value is `0`, which switches the filter off. Use Content-Security-Policy instead.

## Content-Security-Policy (CSP)

CSP is the most powerful security header, and the most complex. It specifies which sources of content the browser may load. For a pure API (no browser UI), CSP is less relevant — add it if you ever serve HTML.

```
Content-Security-Policy: default-src 'self'; script-src 'none'; object-src 'none'
```

For an API response (JSON, no HTML), a minimal policy:

```
Content-Security-Policy: default-src 'none'
```

## 38.8 Input Sanitization vs Validation

Chapter 31 covered validation: rejecting input that doesn't meet requirements. **Sanitization** is different: transforming input to remove dangerous content before using it.

For the Notes API:

**Validation** (Chapter 31):

- Content must be a non-empty string
- Content must be 10,000 characters or fewer

**Sanitization:**

- Strip leading/trailing whitespace (the `strip=True` parameter in the Validator)
- Optionally limit to specific Unicode character ranges

(Stripping HTML tags from stored content is tempting, but it destroys legitimate text — a note about `<div>` layout, say. The better rule is to store what the user wrote and escape it at the moment it is rendered as HTML, as below.)

For an API that only returns JSON (not HTML), HTML injection in the content field is not a direct security risk — the JSON is not rendered as HTML. If you later add a web frontend that renders note content as HTML, you must escape it:

```
In a Jinja2 template (Flask's default templating engine):
<p>{{ note.content }}</p>
Jinja2 auto-escapes: & → & < → < > → > etc.
Safe against XSS

DANGEROUS — bypasses escaping:
<p>{{ note.content | safe }}</p>
Only use |safe for content you know is HTML-safe (e.g., content you generated)
```

## 38.9 Security Audit Checklist

Use this checklist periodically to audit the security of a Flask application:

### Secrets and Configuration

- ☐ No secrets in source code or Dockerfile
- ☐ `.env` files in `.gitignore`
- ☐ `SECRET_KEY` is randomly generated (`secrets.token_hex(32)`: 64 characters), not a placeholder



- ☐ `DEBUG = False` in production
- ☐ `pip-audit` runs in CI with no high-severity findings
- ☐ Dependencies updated in the last 90 days

## Transport Security

- ☐ HTTPS enforced (HTTP redirects to HTTPS)
- ☐ TLS 1.2+ only (TLS 1.0 and 1.1 disabled)
- ☐ Valid, non-expired certificate
- ☐ HSTS header with long `max-age`
- ☐ Certificate monitored for upcoming expiry

## Application Security

- ☐ Every route except `/health`, `/ping`, `/auth/register` and `/auth/login` returns 401 without a valid token
- ☐ Every query on user data includes the owner (`AND user_id = ?`), and a test tries each endpoint as another user
- ☐ Passwords stored with `bcrypt` (or `Argon2`), never with a fast hash; tokens stored only as hashes
- ☐ All database queries use parameterized statements
- ☐ User input validated before use
- ☐ Error responses don't include internal details (stack traces, SQL)
- ☐ Rate limiting on all public endpoints

## Infrastructure Security

- ☐ Non-root user in Docker container
- ☐ Cloud firewall allows only ports 22, 80, 443
- ☐ UFW configured as a second firewall layer
- ☐ Container ports published on `127.0.0.1` only: `sudo ss -tlnp` shows 5000 (and 9200, 9090, 3000 if you run the Chapter 35 monitoring) bound to `127.0.0.1`
- ☐ Tested from outside: `nc -zv -w 5 <server-ip> 5000` from your laptop times out or is refused
- ☐ SSH key authentication only (password SSH disabled)
- ☐ Unattended upgrades enabled for security patches
- ☐ PostgreSQL not accessible from public internet

## Logging and Monitoring

- ☐ Failed authentication attempts logged
- ☐ Security headers present in responses
- ☐ Access logs retaining 14 days minimum
- ☐ Alerting on high error rates

### Key Takeaways

- **Defense in depth:** stack multiple layers of security so that compromising one layer does not compromise the entire system — and verify the layers from outside: Docker-published ports bypass UFW.
- **Threat modeling:** identify who might attack, what they want, and how they might do it. Prioritize defenses based on the most realistic threats.
- **Authentication** (who are you?) and **authorization** (what can you do?) are distinct, and a real API needs both. The Notes API uses **opaque login tokens**: random, stored only as hashes, expiring, and revoked at logout. JWTs trade away revocation for checks that need no database, which is a trade worth making only across many services.
- **Hash passwords with a deliberately slow, salted function** (bcrypt, Argon2), never a fast one like SHA-256. Require length (12+ characters), not composition rules. Give one error, and equal timing, for an unknown email and a wrong password, and rate-limit the login endpoint separately.
- **Protect by default.** Attach the login check to whole blueprints, so that a new route cannot forget it; make only the few public routes public on purpose.
- **Enforce ownership in every query** (AND `user_id = ?`), answer 404 rather than 403 for another user's data, and test every endpoint as a second user. A missed check here is the most common serious web vulnerability (OWASP A01).
- **Constant-time comparison** (`hmac.compare_digest`) is required whenever code compares a secret directly, to prevent timing attacks.
- **Never store secrets in source code.** Use environment variables, with a secrets manager for larger deployments.
- **OWASP Top 10** is the essential reading list for web application security. The deployed Notes API covers several of them — parameterized queries (A03), HTTPS (A02), security headers and hardened configuration (A05), pip-audit (A06), structured logging (A09) — and this chapter adds access control (A01) and authentication (A07).
- **Security headers** (HSTS, X-Content-Type-Options, X-Frame-Options) are added by Nginx and tell browsers how to handle responses safely.
- **The security audit checklist** is a practical tool for periodically verifying the security posture of a deployed application.

## Exercises

1. **Predict.** Log in twice from two terminals, then log out in one. Does the other token still work? Why?
2. **Break and fix.** Change `check_credentials` to return early, skipping `bcrypt`, when the email is unknown. Measure both kinds of failed login with `curl -w %{time_total}`. What can an attacker learn? Restore it.
3. **Extend.** Add `GET /auth/sessions`, listing the current user's active sessions (created time, expiry, never the token), and `DELETE /auth/sessions/<id>`, to log out a session elsewhere.
4. **Explain.** Why does the Notes API use opaque tokens rather than JWTs, and what would make you choose JWTs?

Solutions and hints are in the companion repository, in `exercises/chapter-38.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

The final chapter before the appendices addresses a persistent confusion for new developers: why does the production system behave differently from the local development environment, and how to bridge that gap.

→ [Chapter 39](#) — Local vs Production



# Local vs Production

Every developer has experienced the frustration: “It works on my machine.” The code runs perfectly locally, passes all the tests, and then behaves differently — or fails entirely — in production. This is not a coincidence or bad luck. The local development environment and the production environment are fundamentally different systems, and those differences cause real problems.

This chapter catalogs the specific ways in which local and production environments differ, explains why each difference exists, and provides concrete strategies for reducing the gap between them. The goal is not to make local and production identical (which is impossible), but to make their differences explicit and manageable.

## In this chapter

By the end of this chapter, you will understand:

- The specific differences between local and production that cause “works on my machine” failures
- Why running the database locally vs. PostgreSQL remotely causes different behavior
- How environment variables work differently across environments
- What Docker does and does not guarantee about environment consistency
- How to use staging environments to catch production-specific bugs before deployment
- Practical techniques: environment parity, feature flags, and local production simulation
- How to record significant decisions as **Architecture Decision Records (ADRs)** so the *why* behind the system survives staff turnover

## 39.1 The Differences That Matter

The Notes API runs in two environments:

**Local development:**

- Developer’s laptop (macOS or Linux, 16-32 GB RAM, fast SSD)
- Python installed directly or in a virtualenv
- SQLite database (a local file; the tests use a temporary one)
- The Flask development server (`python run.py`)
- Debug mode enabled, auto-reload enabled
- Secrets in `.env` file or shell environment
- Network requests go through localhost

#### Production:

- DigitalOcean Droplet (1 vCPU, 1 GB RAM, 25 GB SSD)
- Python in a Docker container
- PostgreSQL on a managed database service
- Gunicorn WSGI server (multiple workers, no auto-reload)
- Debug mode disabled
- Secrets in `/opt/notes-api/config/notes-api.env`
- Network requests traverse the internet, Nginx, Docker networking

These differences are not failures of engineering — they reflect different purposes. Development must be fast to iterate on; production must be stable and secure. But unmanaged, the differences cause bugs that are hard to diagnose because they only appear in production.

### Every difference above was a decision — record the ones that matter

Many of these differences are revisited here with everything the book has built since Stage 2 first catalogued them (environment drift, [Chapter 7](#); configuration coupling, [Chapter 9](#); the reliability and security gaps, [Chapter 11](#)).

Read that list again and notice what it really is: a list of *decisions*. SQLite locally but PostgreSQL in production. Gunicorn instead of the development server. Docker. Nginx in front. Each was chosen for reasons that felt obvious at the time — and six months from now those reasons will be gone, while the decision remains. Someone will look at it and ask “why is this like this?” The code answers *what* the system does; it almost never answers *why* it is the way it is. That *why* is the most valuable and most perishable knowledge in any project, and capturing it is its own small discipline.

**Decisions are the most perishable knowledge in a project.** Without a record, a team does one of two things with a forgotten decision, both bad: it re-litigates the question from scratch every time it resurfaces, or — worse — someone “cleans up” the odd-looking thing and removes the very safeguard the decision put there. The cure is to write the decision down at the moment

it is made.

## What an Architecture Decision Record is

An **Architecture Decision Record (ADR)** is a short document that captures one significant decision: the *context* that forced it, the *decision* made, the *alternatives* considered, and the *consequences* accepted. It is deliberately not a design document (a proposal for how to build something) and not an RFC (a request for comment *before* deciding). It is a *record*, written when the decision is made and then left immutable. One decision, one file.

Not every decision earns one. A decision deserves an ADR when it is **hard or expensive to reverse**, when it **affects many parts of the system**, or when someone will reasonably ask “why?” later. Adopting the application factory pattern, choosing PostgreSQL, putting Nginx in front of the app — yes. Naming a variable or ordering two functions — no.

## The Nygard template

The widely used lightweight format, from Michael Nygard, has just five parts:

- **Title** — numbered and sequential, stating the decision in one line: **ADR-0007: Use PostgreSQL in production instead of SQLite.**
- **Status** — Proposed → Accepted → Deprecated → Superseded by ADR-XXXX. The decision’s place in its lifecycle.
- **Context** — the forces that made the decision necessary, written so it still makes sense in two years (assume the reader does *not* remember today’s situation). This is the most important section.
- **Decision** — the choice itself, stated in active voice: “We will...”.
- **Consequences** — what becomes easier *and* what becomes harder. The honest downsides you are knowingly accepting, plus the **alternatives considered and why they lost** — often the single most valuable lines in the whole record, because they stop a future engineer from re-proposing a path you already rejected for good reason.

## A worked ADR for a decision this book actually made

Here is the SQLite-versus-PostgreSQL decision the book has been living with since Chapter 13, written up as an ADR:

```
ADR-0007: PostgreSQL in production, SQLite in local development
```

```
Status: Superseded by ADR-0012 (was: Accepted, 2024-03-10)
```

## ## Context

The Notes API uses SQLite, a file-based database with no server process (Chapter 6) – ideal for local development: zero setup, no network. But production needs concurrent writes from multiple Gunicorn workers, network access from a separate app server, managed backups, and features SQLite lacks (concurrent writers, time-zone-aware timestamps, ILIKE). SQLite's single-writer lock would serialize our workers under load.

## ## Decision

We will run PostgreSQL in production and keep SQLite for local development, selecting between them at runtime via the DATABASE\_URL environment variable (Chapter 13). Application code stays database-agnostic.

## ## Consequences

- + Production gets real concurrency, managed backups, and richer SQL.
- + Local development stays zero-setup and fast.
- We now maintain two code paths and must test against PostgreSQL, not just SQLite, or we get "works on my machine" bugs (this chapter, §39.2).
- Developers must run PostgreSQL locally to reproduce production exactly.

## ## Alternatives considered

- PostgreSQL everywhere, including local: rejected – heavier local setup and slower onboarding for little benefit on a laptop.
- SQLite everywhere, including production: rejected – cannot handle concurrent writes from multiple workers; no managed backups.

That one short file answers, permanently, the exact question §39.2 is about to raise – and it answers it for a colleague who joins two years from now and was never in the room. Notice its status line: §39.2 is about to change the decision, and the record that changes it looks like this:

### # ADR-0012: PostgreSQL everywhere, including local development and tests

Status: Accepted (2024-04-02). Supersedes ADR-0007.

## ## Context

ADR-0007 predicted it: code that only PostgreSQL runs (the ILIKE search



added to GET /notes/, Chapter 33) was never exercised by the test suite, which ran on SQLite. Meanwhile Docker made a local PostgreSQL a one-line command, removing ADR-0007's main reason for SQLite.

### ## Decision

Tests, CI, and local development run against PostgreSQL, the same major version as production, in a container. SQLite is no longer supported.

### ## Consequences

- + The code that runs in production is the code that is tested.
- + One database dialect: the IS\_SQLITE branches can be deleted.
- Running the tests needs Docker (or a local PostgreSQL).

## Status, lifecycle, and project archaeology

You never edit an accepted ADR's decision. When the thinking changes, you write a *new* ADR that supersedes it and mark the old one Superseded by ADR-0021. The trail of superseded records becomes the project's intellectual history — you can read not just where the design landed but *how the thinking evolved to get there*. This is why, when you join an unfamiliar project, reading its ADRs in order is the fastest way to understand why the system looks the way it does. It is archaeology with the reasons still attached.

## Where ADRs live, and how they fail

ADRs live **in the repository**, beside the code (commonly docs/adr/0001-\*.md), numbered sequentially, so they version alongside what they describe and travel with the project. The common failure modes are all variations on neglect: written too *late* (after everyone forgot the context), too *vague* (no alternatives, no honest consequences), too *grand* (a ten-page essay nobody finds time to write or read), or simply *ignored*. The cure for every one of them is the same — small, immediate, honest, one decision per file.

## 39.2 The Database Difference

The most common source of “works locally but fails in production” is the **database**.

### SQLite vs PostgreSQL

Chapter 17 configured the test suite to use SQLite (a temporary file). Local development uses SQLite too, for convenience. PostgreSQL is used in production.

SQLite and PostgreSQL are both SQL databases, but they are not identical:

Feature	SQLite	PostgreSQL
Case sensitivity	LIKE is case-insensitive (for ASCII letters only)	LIKE is case-sensitive; use ILIKE for insensitive
Date functions	Limited	Full DATE_TRUNC, INTERVAL, timezone support
JSON operations	Limited	Full jsonb operators
Concurrent writes	Single writer at a time	Full MVCC concurrent writes
RETURNING clause	Supported since 3.35 (2021)	Full support
SERIAL type	Equivalent: INTEGER PRIMARY KEY	Auto-increment via sequence
Full-text search	Limited	GIN indexes, tsvector
Custom types	Not supported	Extensible

If your code uses any PostgreSQL-specific feature, it will fail against SQLite.

**Example failure** — the Notes API's own search ([Chapter 33](#)):

```
This works on PostgreSQL (ILIKE is case-insensitive):
cursor.execute("SELECT * FROM notes WHERE content ILIKE %s", (f"%{search}%",))

On SQLite, the same query cannot even be parsed (and sqlite3 would
first reject the %s placeholder, which it writes as ?):
sqlite3.OperationalError: near "ILIKE": syntax error
```

So the test suite, running on SQLite, simply never calls `?search=` — it could not. The search code is tested for the first time by users in production. Any bug in it, from a typo to a wrong escape, is a production-only bug.

### The fix: test against the same database

The most reliable solution is to use PostgreSQL everywhere:

```
tests/conftest.py (PostgreSQL version)
import os
from urllib.parse import urlsplit
```

```

import pytest

As in Chapter 17: the settings must be in place BEFORE the app package
is imported, because app/config.py reads the environment at import time.
os.environ["DATABASE_URL"] = os.environ.get(
 "TEST_DATABASE_URL",
 "postgresql://notes_user:test_password@localhost:5432/notes_test",
)
os.environ["SECRET_KEY"] = "test-secret-key-not-for-production-1234"
os.environ["BCRYPT_ROUNDS"] = "4" # fast password hashing, as in Chapter 38

The fixtures below delete every note. Refuse to run against anything
that is not obviously a test database.
assert urlsplit(os.environ["DATABASE_URL"]).path.endswith("_test"), \
 "Refusing to run the tests against a database whose name does not end in _test"

from app import create_app # noqa: E402 (must come after the settings above)
from app.migrate import migrate # noqa: E402

@pytest.fixture(scope="session")
def app():
 migrate() # builds the schema on a new database; a no-op on one already up to
 ↪ date
 application = create_app()
 application.config.update({"TESTING": True})
 yield application

@pytest.fixture(scope="function", autouse=True)
def clean_database(app):
 """Empty the data tables (and reset their id sequences) before each test."""
 with app.app_context():
 from app.database import get_db
 db = get_db()
 # CASCADE: the tables that refer to these (note_tags, sessions) are
 # emptied too
 db.cursor().execute("TRUNCATE notes, tags, users RESTART IDENTITY CASCADE")
 db.commit()
 yield

The client, anon_client, login_as and create_note fixtures are unchanged
from Chapters 17 and 38.

```

Run PostgreSQL locally for testing using Docker:

```

Start a local PostgreSQL for testing – the same major version as the
production database (check it in the provider's control panel)
docker run -d \
 --name postgres-test \
 -e POSTGRES_USER=notes_user \
 -e POSTGRES_PASSWORD=test_password \
 -e POSTGRES_DB=notes_test \
 -p 127.0.0.1:5432:5432 \
 postgres:16-alpine

Run tests with the PostgreSQL database
TEST_DATABASE_URL=postgresql://notes_user:test_password@localhost/notes_test pytest

```

In CI, GitHub Actions starts the same database as a **service container** next to the job. Add to the test job of `.github/workflows/build.yml` (Chapter 19):

```

services:
 postgres:
 image: postgres:16-alpine
 env:
 POSTGRES_USER: notes_user
 POSTGRES_PASSWORD: test_password
 POSTGRES_DB: notes_test
 ports:
 - "5432:5432"
 # Wait until PostgreSQL accepts connections before running steps
 options: >-
 --health-cmd pg_isready
 --health-interval 5s
 --health-timeout 5s
 --health-retries 10

```

(The `conftest`'s default `TEST_DATABASE_URL` already points at `localhost:5432/notes_test`, so the test step needs no extra settings.) And now that nothing runs on SQLite any more, the `IS_SQLITE` branches in `app/database.py` can go, and `sql()` shrinks to one job, turning `?` into PostgreSQL's `%s`: the code has one dialect, and the search query can use `sql()`'s `?` like every other query. The `migrations/sqlite/` folder goes too, and with it the chore of writing every migration twice. And the concurrency of Chapter 33's section 33.13, which SQLite made impossible to test, can now be tested at last.

Or in `docker-compose.yml` for local development:

```

docker-compose.yml (for local development, not production)
services:
 db:
 image: postgres:16-alpine # same major version as production
 environment:
 POSTGRES_USER: notes_user
 POSTGRES_PASSWORD: dev_password
 POSTGRES_DB: notesdb
 ports:
 - "127.0.0.1:5432:5432" # for psql on the laptop; loopback only
 volumes:
 - postgres-data:/var/lib/postgresql/data
 # Required by "condition: service_healthy" below
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U notes_user -d notesdb"]
 interval: 5s
 timeout: 5s
 retries: 5

 api:
 build: .
 depends_on:
 db:
 condition: service_healthy
 environment:
 DATABASE_URL: postgresql://notes_user:dev_password@db/notesdb
 SECRET_KEY: dev-secret-key-not-for-production-1234
 FLASK_DEBUG: "true"
 LOG_FORMAT: human
 ports:
 - "127.0.0.1:5000:5000"
 volumes:
 - ./app # Mount source code for live reloading in development
 # The development server with auto-reload, instead of the image's Gunicorn
 # (after bringing the schema up to date; the app refuses to start otherwise)
 command: ["sh", "-c", "python -m app.migrate && exec flask --app
↪ 'app:create_app()' run --host=0.0.0.0 --debug"]

volumes:
 postgres-data:

```

With this setup, local development uses the same PostgreSQL version as production. The only difference is that local uses a volume (persistent across container restarts) and production uses a managed service. (One Linux-specific wrinkle: the container runs as UID 1001, and the bind-

mounted source directory belongs to your own user; if Python cannot write `__pycache__` there, it simply runs without it. macOS's Docker Desktop maps the ownership for you.)

### 39.3 The Environment Variable Difference

Local development typically uses a `.env` file or environment variables set in the terminal. Production uses environment variables from `/opt/notes-api/config/notes-api.env`.

#### Problems with different environment variable values

**Unset variables that are required:** if a required environment variable is not set locally, the startup check (Chapter 14) catches it. But what if the variable is set to a valid value locally but an *incorrect* value in production?

```
Local .env:
DATABASE_URL=sqlite:///./dev.db

Production:
DATABASE_URL=postgresql://user:pass@private-db-...db.ondigitalocean.com:25060/notesdb?
→ sslmode=require
```

The application runs locally with SQLite and fails with PostgreSQL-specific queries in production.

**Different secret values:** `SECRET_KEY` *should* differ between environments — a production secret must never sit on a laptop. Flask uses it to sign session cookies, so a cookie issued in one environment is not accepted in another; for this JSON API, which issues none, that changes nothing.

#### The `.env.example` file

Document all required environment variables in a `.env.example` file that is committed to the repository:

```
.env.example
Copy this file to .env and fill in the values.
Never commit .env to the repository.

Required: Database connection string (the local PostgreSQL from §39.2)
DATABASE_URL=postgresql://notes_user:dev_password@localhost:5432/notesdb
```

```
Required – generate a value with:
python -c "import secrets; print(secrets.token_hex(32))"
SECRET_KEY=

Development server settings (python run.py)
FLASK_DEBUG=false
PORT=5000
HOST=127.0.0.1

Logging (Chapter 16)
LOG_LEVEL=INFO
LOG_FORMAT=human
LOG_DIR=
```

This is the same file as in [Chapter 16](#), with PostgreSQL in place of SQLite. SECRET\_KEY stays empty on purpose: a copied .env without a real key fails at startup instead of running with a placeholder.

Any developer joining the project:

1. Copies .env.example to .env
2. Fills in real values
3. Has the correct environment

Any missing or misconfigured variable is explicitly documented.

## Validating environment variables at startup

[Chapter 14](#) introduced startup validation, and it is the single most effective tool for catching environment variable problems. The Notes API already has all of it — there is nothing new to write here, only to know where each check lives:

- app/config.py (Chapter 15): `_require()` stops the process if DATABASE\_URL or SECRET\_KEY is missing.
- app/startup.py (Chapters 14–16): the Python version, SECRET\_KEY’s length, and database connectivity, checked by `create_app()` before the first request.
- The security check from [Chapter 38](#): warns if SECRET\_KEY looks like a placeholder, if debug mode is on, or if production points at SQLite.

A misconfigured deployment therefore fails loudly, in the logs, before it serves anything:

```
... CRITICAL app.startup | Startup failed: Cannot connect to the database.
DATABASE_URL: postgresql://notes_user:***@private-db-....db.ondigitalocean.com:25060/
↳ notesdb?sslmode=require
Error: connection to server ... failed: timeout expired
Ensure the database server is running and reachable.
```

This catches environment problems during deployment — before the first user request — rather than during a user’s request at an unpredictable time.

## 39.4 The Network Difference

In local development, all services run on `localhost`. In production, services are on different hosts connected over a network:

```
Local:
 Flask → localhost:5432 (PostgreSQL)
 Flask → localhost:6379 (Redis, if you added it in Chapter 36)
 Client → localhost:5000 (Flask)

Production:
 Flask → private-db-....ondigitalocean.com:25060 (managed PostgreSQL, over the VPC,
 ↳ TLS)
 Flask → 10.0.0.11:6379 (Redis on its own server, if you added it)
 Client → 203.0.113.42 (Nginx, which proxies to Docker)
```

### Network-related bugs

**Latency:** local database queries run in <1ms (loopback). Production database queries run in 2-10ms (VPC network). Code that makes 100 database queries per request (N+1 problem) might be acceptable locally but unbearably slow in production.

With the tags from [Chapter 33](#) — the `tags` table and the `note_tags` join table:



```
N+1 query problem – runs fine locally, very slow in production
def list_notes_with_tags():
 cursor.execute("SELECT id, content FROM notes")
 notes = [dict(row) for row in cursor.fetchall()]
 for note in notes:
 # This executes N additional queries – one per note!
 cursor.execute(
 """SELECT t.name FROM tags t
 JOIN note_tags nt ON nt.tag_id = t.id
 WHERE nt.note_id = %s""",
 (note["id"],),
)
 note["tags"] = [row["name"] for row in cursor.fetchall()]
 return notes
```

Fix: fetch the related rows for all notes at once. Section 33.12 did it with a second query that works on both databases; on PostgreSQL alone, one query can do everything:

```
def list_notes_with_tags():
 cursor.execute("""
 SELECT n.id, n.content,
 -- FILTER drops the NULL a note without tags would contribute;
 -- COALESCE turns "no tags" into an empty array, not NULL
 COALESCE(ARRAY_AGG(t.name) FILTER (WHERE t.name IS NOT NULL), '{}') AS
 ↪ tags
 FROM notes n
 LEFT JOIN note_tags nt ON nt.note_id = n.id
 LEFT JOIN tags t ON t.id = nt.tag_id
 GROUP BY n.id, n.content
 """)
 return cursor.fetchall()
```

**DNS resolution:** locally, localhost always resolves instantly. In production, `api.notes.example.com` is looked up via DNS, which takes 0-100ms for the first lookup. If the application makes outbound HTTP requests to its own API (an anti-pattern, but it happens), the latency difference is significant.

**Connection timeouts:** local connections never time out (they're on the same machine). Production connections across a VPN or the internet can time out. Always set explicit timeout values:

```
Without timeout: hangs indefinitely if remote service is unresponsive
requests.get("https://external-api.example.com/data")

With timeout: raises requests.exceptions.Timeout after 5 seconds
requests.get("https://external-api.example.com/data", timeout=5)
```

**Firewall rules:** local development has no firewall. Production has a cloud firewall (Chapter 22). If a required port is blocked by the firewall, the connection silently hangs in production but works locally.

## 39.5 The Performance Difference

The developer's laptop has more RAM, faster CPU, and faster storage than the production Droplet. Code that runs fast locally may be too slow in production.

### Memory

The developer's laptop: 16-32 GB RAM. The production Droplet: 1 GB RAM.

If the application loads a large file into memory, or builds a large Python object, it may work locally (plenty of RAM) and crash the production server with an OOM kill.

```
Dangerous: loads entire file into memory
with open("large_file.csv", "r") as f:
 data = f.read() # Might be 500MB – fine on laptop, kills 1GB Droplet

Better: process in chunks
with open("large_file.csv", "r") as f:
 for chunk in iter(lambda: f.read(8192), ""):
 process(chunk)
```

### CPU

Python's Global Interpreter Lock (GIL) means one Python process runs Python code on only one CPU core at a time (threads still overlap their *waiting*, for the database for example). The laptop might have 8 cores; the production Droplet has 1. With several Gunicorn workers — separate processes — the laptop can run requests truly in parallel, while the Droplet only interleaves them. A load test on the laptop can therefore promise several times the throughput the server will deliver.

## Disk I/O

The laptop's NVMe SSD reads at several gigabytes per second. The Droplet's disk is also an SSD, but a virtualized one, shared with other customers and throttled by the provider — often an order of magnitude slower. If the application writes or reads large files, this difference matters.

## 39.6 The Concurrency Difference

Local development typically sees one request at a time — from one developer — even though Flask's development server could handle several at once in threads. Production runs multiple Gunicorn workers handling truly concurrent requests from many clients.

### Race conditions

A **race condition** is a bug that occurs only when two operations happen simultaneously. They don't appear in local testing (single process, sequential requests) but appear under production load (concurrent requests).

**Example** (for a hypothetical version of the Notes API whose notes have a `title` that must be unique): two concurrent requests both try to create a note with the same title.

```
Dangerous: check-then-set is a race condition
cursor.execute("SELECT id FROM notes WHERE title = %s", (title,))
if cursor.fetchone() is not None:
 return jsonify({"error": "Note with this title already exists"}), 409

If two concurrent requests both reach here, both see no existing note,
and both proceed to insert – creating a duplicate.
cursor.execute("INSERT INTO notes (title, ...) VALUES (%s, ...)", (title, ...))
```

Fix: use a database **UNIQUE** constraint, which enforces uniqueness atomically:

```
ALTER TABLE notes ADD CONSTRAINT notes_title_unique UNIQUE (title);
```

```

try:
 cursor.execute("INSERT INTO notes (title, ...) VALUES (%s, ...)", (title, ...))
 db.commit()
except psycopg2.errors.UniqueViolation:
 # The failed statement aborted the transaction: end it, so the
 # connection is usable again
 db.rollback()
 return jsonify({"error": "Note with this title already exists"}), 409

```

The database handles the race condition correctly; the application doesn't need to.

## Global state

Variables at module level persist across requests in the same Gunicorn worker. This is intentional for things like logging configuration. But accidental global state causes bugs:

```

Dangerous: global list that accumulates across requests
_pending_emails = []

@app.route("/subscribe", methods=["POST"])
def subscribe():
 email = request.json["email"]
 _pending_emails.append(email) # BUG: never cleared; grows forever
 return "", 204

```

In local development (sequential single-process testing), this might not appear as a problem. In production, `_pending_emails` grows without bound across all requests in the worker, consuming memory.

Fix: use the database or a message queue for any state that persists across requests.

## 39.7 What Docker Guarantees (and What It Doesn't)

Docker containers were introduced in Chapter 18 specifically to reduce the “works on my machine” problem. A Docker image captures:

- The Python version
- Installed Python packages (with pinned versions)
- The operating system user and directory structure
- Environment variable defaults (not secrets)

Docker does **not** guarantee:

- The database schema (migrations must be applied separately)
- The database content (data differs between environments)
- The runtime configuration (environment variables with secrets)
- The underlying host hardware (different CPU, different I/O speed)
- Network topology (localhost vs. VPC networking)

Docker dramatically reduces environment drift — but it does not eliminate it. The remaining differences (database, network, configuration, scale) must be managed explicitly.

## 39.8 The Staging Environment

A **staging environment** is a production-like environment where code is deployed and tested before going to production. It sits between local development and production:

```
Local → CI (automated tests) → Staging → Production
```

Staging characteristics:

- Same Docker image as production (identical code and dependencies)
- Same database type (PostgreSQL) but with different data (test data, not real user data)
- Same Nginx configuration
- Same supervision (Docker restart policy, deploy scripts)
- Different secrets (staging uses staging secrets, not production secrets)
- Public access (so developers can manually test)
- May run on a smaller server (cost optimization)

Staging catches the bugs that don't appear locally or in CI:

- PostgreSQL-specific behavior that SQLite doesn't catch
- Configuration that works locally but fails in the production Docker environment
- Network timeouts and latency that don't appear on localhost
- Performance issues that only appear under realistic load

### Promoting through environments

A deployment pipeline:

```
git push → CI runs tests → Docker image built → image tagged
```

Manual trigger:

1. Deploy image to Staging
2. Run smoke tests against Staging
3. Manually verify functionality on Staging
4. Deploy same image to Production

The **same image** is promoted through environments. This is critical: rebuilding from source for each environment risks introducing differences (dependency versions, build-time environment). The exact same bytes that were tested in staging reach production — guaranteed if you deploy by digest (`notes-api@sha256:...`), and by convention if you deploy by an unchanging `sha-` tag (Chapter 19).

The server-side `deploy.sh` from Chapter 23 needs no change for this: staging is simply another server with its own copy of the script and its own environment file. A small script on your laptop (or a CI job) chooses the target:

```
#!/bin/bash
deploy-to.sh — run from a laptop or CI: deploy-to.sh staging sha-a1b2c3d
set -euo pipefail
ENVIRONMENT="${1:?Usage: deploy-to.sh staging|production sha-<commit>}"
IMAGE_TAG="${2:?Usage: deploy-to.sh staging|production sha-<commit>}"

case "$ENVIRONMENT" in
 staging) SSH_HOST="staging.notes.example.com" ;;
 production) SSH_HOST="api.notes.example.com" ;;
 *) echo "Unknown environment: $ENVIRONMENT"; exit 1 ;;
esac

Same image tag, same script, different server
ssh "deploy@${SSH_HOST}" "/opt/notes-api/deploy.sh ${IMAGE_TAG}"
```

## 39.9 Simulating Production Locally

When you need to debug a production-specific issue locally, simulate the production environment:

## Run with Gunicorn instead of flask run

```
Instead of:
python run.py

Run as it runs in production:
gunicorn --config gunicorn.conf.py "app:create_app()"
```

Differences:

- Multiple workers (production-like concurrency)
- No auto-reload (production behavior)
- Timeout enforcement

## Run with PostgreSQL

```
Start a local PostgreSQL container:
docker run -d \
 --name postgres-local \
 -e POSTGRES_USER=notes_user \
 -e POSTGRES_PASSWORD=dev_password \
 -e POSTGRES_DB=notesdb \
 -p 127.0.0.1:5432:5432 \
 postgres:16-alpine

Build the schema: every migration, from 0001 (Chapter 33)
DATABASE_URL=postgresql://notes_user:dev_password@localhost/notesdb \
 python -m app.migrate

Run the application against it:
DATABASE_URL=postgresql://notes_user:dev_password@localhost/notesdb \
 gunicorn --config gunicorn.conf.py "app:create_app()"
```

## Run the full Docker stack

```
Build the production image:
docker build -t notes-api:local .

Run the full stack (Nginx + Gunicorn + PostgreSQL):
docker compose -f docker-compose.prod-local.yml up
```

```
docker-compose.prod-local.yml
services:
 db:
 image: postgres:16-alpine
 environment:
 POSTGRES_USER: notes_user
 POSTGRES_PASSWORD: dev_password_not_for_real_production
 POSTGRES_DB: notesdb
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U notes_user -d notesdb"]
 interval: 5s
 timeout: 5s
 retries: 5

 api:
 image: notes-api:local
 depends_on:
 db:
 condition: service_healthy
 environment:
 DATABASE_URL:
 ↪ postgresql://notes_user:dev_password_not_for_real_production@db/notesdb
 SECRET_KEY: local-test-secret-key-also-not-for-production
 # Bring the schema up to date first: the app refuses to start otherwise
 command: ["sh", "-c", "python -m app.migrate && exec gunicorn --config
 ↪ gunicorn.conf.py 'app:create_app()'"]

 nginx:
 image: nginx:1.24-alpine
 ports:
 - "127.0.0.1:8080:80"
 volumes:
 - ./nginx/local.conf:/etc/nginx/conf.d/default.conf:ro
 depends_on:
 - api
```

The production site configuration from Chapter 28 cannot be reused as it is: it listens on 443 with Let's Encrypt certificates that do not exist here, and its upstream is 127.0.0.1:5000 — which, *inside the Nginx container*, is the Nginx container itself. A small local configuration proxies to the `api` service by its Compose name instead:



```
nginx/local.conf
server {
 listen 80;
 client_max_body_size 64k;
 location / {
 proxy_pass http://api:5000;
 proxy_set_header Host $host;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
 }
}
```

Now `http://localhost:8080/notes/` behaves like production (Nginx proxy → Docker → Gunicorn → PostgreSQL), without HTTPS (which requires a real domain and certificate).

## 39.10 Environment Parity Principles

**12-Factor App** (12factor.net) is a methodology for building software-as-a-service applications. Its principle on development/production parity:

“Keep development, staging, and production as similar as possible.”

This means:

1. **The time gap:** deploy often. Code deployed hours after writing is more likely to be in sync with the developer’s mental model than code deployed weeks later.
2. **The personnel gap:** developers who wrote the code should be involved in deploying it. Separation between “Dev” and “Ops” creates communication gaps that amplify environment differences.
3. **The tools gap:** use the same services in all environments. Don’t use SQLite locally and PostgreSQL in production. Don’t use the Python logging module locally and a different logging system in production.

### The environment variable matrix

Document all environment variables and their values across environments:

Variable	Local	CI	Staging	Production
DATABASE_URL	local container (...@localhost/ notesdb)	CI service (...@localhost/ notes_test)	staging database (private host)	production database (private host)
SECRET_KEY	(any long value)	(test value)	(staging secret)	(production secret)
LOG_LEVEL	DEBUG	WARNING	INFO	INFO
LOG_FORMAT	human	human	json	json
FLASK_DEBUG	true	false	false	false
REDIS_URL (if used)	local container	CI service	staging Redis	production Redis

(Older Flask guides also set `FLASK_ENV`; it was removed in Flask 2.3 — `FLASK_DEBUG` replaced it. And note the Redis row: if the cache from Chapter 36 is in the code, every environment needs a Redis, or the cache must be optional. “Not set locally” is exactly the kind of difference that hides bugs.)

Differences between columns are potential bugs. The fewer differences, the fewer production-specific bugs.

### 39.11 Delivering and Tracking the System Over Time

The pipeline in §39.8 promotes one image to production. But a real system is not deployed once — it is *delivered continuously*, over years, as a stream of changes. That stream has to be tracked, or no one can answer the questions every team eventually faces: **What is actually running in production right now? What changed since last week? Why is this broken — was it broken before? What are we doing next?** The engineering is finished; this is the project hygiene that keeps a shipped system legible to the humans who operate and depend on it. It is less glamorous than sockets and schemas, but every team that ships well has practices for it.

#### Deploy vs. release: not the same event

A **deploy** is a *technical* event — new code reaches production. A **release** is a *product* event — a change becomes visible to users. They often happen together, but separating them is a superpower. A **feature flag** (a simple configuration switch) lets you *deploy* code “dark” — present in production but switched off — and *release* it later by flipping the flag, with no redeploy, and roll it back instantly if it misbehaves. Decoupling deploy from release turns a risky big-bang

launch into a calm, reversible toggle.

## Versioning: naming what you ship

Every release needs a name you can point at, and the standard is **semantic versioning**: MAJOR.MINOR.PATCH. Bump **PATCH** (1.4.2 → 1.4.3) for backward-compatible bug fixes; **MINOR** (1.4.3 → 1.5.0) for backward-compatible new features; **MAJOR** (1.5.0 → 2.0.0) for a breaking change. The version is a promise to everyone who depends on you: *upgrading within the same MAJOR will not break you*. It is the same breaking-versus-non-breaking distinction you applied to API contracts (Chapter 34) and database schemas (Chapter 6), now applied to the release as a whole. Internally you also tag the built artifact by git hash (Chapter 19), so every deploy traces to exact source — the semantic version is the *human* name for a release, the git hash is its *machine* name.

## The changelog: what changed, for humans

A **changelog** is a human-readable, chronological record of notable changes, newest first. The widely used *Keep a Changelog* convention groups each version’s entries under fixed headings — Added, Changed, Deprecated, Removed, Fixed, Security:

```
[1.2.0] - 2024-04-01

Added
- PUT /notes/<id> updates a note; notes now have an updated_at timestamp.
- Case-insensitive search on GET /notes/ (?search=).

Fixed
- A request body that was valid JSON but not an object (e.g. a list)
 caused a 500; it now returns 400.

Security
- Nginx now answers oversized request bodies with a JSON 413.
```

A changelog is written for *people* — users and teammates — which is what distinguishes it from the git log, written by and for tooling. Distinguish **release notes** (curated, stakeholder-facing — “what’s new for you”) from the **developer changelog** (complete, technical). Disciplined commit messages (the **Conventional Commits** convention) let much of the changelog be generated automatically.

## Knowing what is live: current state

The most basic operational question — *what version is in production right now?* — is astonishingly often unanswerable on teams that do not deliberately track it. Keep one source of truth: the deployed version (a release tag or git hash, surfaced by the `/health` endpoint or a status page), the date of the last change, and any known limitations or active workarounds. “Everything is latest” — the anti-pattern Chapter 19 warned about — is exactly how a team ends up unable to say what is actually running. If a colleague cannot answer “what’s in production?” without SSHing into a server, that is a gap worth closing.

## Known issues and technical debt

Every real system carries **technical debt** — shortcuts taken to ship, code that works but is hard to change. Debt is not inherently bad; *untracked* debt is. The useful lens is the **technical-debt quadrant**: debt is either **deliberate or inadvertent**, and either **prudent or reckless**. (The quadrant is Martin Fowler’s.) “We shipped without pagination, knowing we would add it before scale” is deliberate-and-prudent, and entirely fine. “We don’t know why this query is slow” is inadvertent-and-reckless, and dangerous. Keep a **debt register** — each item with its impact, the current workaround, and the proper fix — so debt is paid down strategically against feature work instead of compounding in silence.

## The roadmap: what comes next, and why

A **roadmap** records what is planned, prioritized (High / Medium / Low), with the *rationale* for each priority and a rough time horizon (this quarter, this year, eventually). Its real job is alignment: it lets the team and stakeholders argue about priorities *once*, in one place, instead of re-arguing them in every meeting. A roadmap is a living document — revised whenever reality intervenes, which is a feature, not a failure.

## Delivery phases: shipping in stable milestones

Large work is best broken into **delivery phases** — discrete milestones, each with a clear goal and a definition of “done,” shipped and stabilized before the next begins. Phases create natural checkpoints and guard against the two classic failure modes: drifting forever, or cramming everything into a rushed finale. A *phase* is a stable milestone; a *task* is one work item inside it. The Notes API of this book was itself delivered in phases — local, portable, containerized, deployed, observable, hardened — each a working system before the next layer was added. That was not incidental; it is how durable systems get built. (Part III’s engineering practices and the planning guide in Part II treat roadmaps and phases in more depth.)

You now have the whole single-server arc — from a command that starts a process to a

system that is observed, scaled, hardened, *and tracked* across its entire life. The code was never the hard part. Keeping a running system understandable — to its users, and to the humans who will maintain it long after you — is the real work of engineering.

### Key Takeaways

- “**Works on my machine**” is caused by specific, identifiable differences between local and production environments: database type, network topology, environment variables, concurrency, memory, and disk I/O.
- **Use PostgreSQL everywhere**, including local development and CI tests. Differences between SQLite and PostgreSQL cause bugs that only appear in production.
- **Startup validation** (Chapter 14) catches environment variable problems during deployment, not during user requests.
- **Network latency** (2-10ms to the database) in production vs. near-zero locally reveals N+1 query problems and missing indexes. Load-test in an environment that matches production network topology.
- **Concurrency bugs** (race conditions, unintended global state) don’t appear in sequential local testing but do appear under production load with multiple Unicorn workers. Use database constraints for atomicity.
- **Docker** guarantees the Python version, installed packages, and OS environment — but not the database schema, configuration, network, or hardware.
- **Staging environments** catch production-specific bugs by running the same Docker image against a PostgreSQL database with the same Nginx configuration as production.
- **Environment parity** — keeping all environments as similar as possible — is the most effective long-term strategy for reducing production-specific bugs.
- Record significant, hard-to-reverse decisions as **Architecture Decision Records** (Nygard template: context, decision, alternatives, consequences) — one immutable file per decision, numbered, living in the repo. They capture the *why*, which is the most valuable and most perishable knowledge in a project; reading them in order is the fastest way to understand an inherited system.
- A system is **delivered continuously, not deployed once**. Separate **deploy** (technical) from **release** (user-visible) using feature flags; name releases with **semantic versioning**; keep a human-readable **changelog**; and always know what version is live.
- Track **technical debt** deliberately (the deliberate/inadvertent × prudent/reckless quadrant), maintain a prioritized **roadmap**, and ship in **delivery phases** — discrete, stabilized milestones, the way the Notes API itself was built.

## Exercises

1. **Predict.** Before switching the tests to PostgreSQL, run the suite of Chapter 38 twice: as it is, on SQLite, and then with `DATABASE_URL` pointing at a PostgreSQL database. Which parts of the application behave differently, and why?
2. **Break and fix.** Write a query using `ILIKE` and run the tests on SQLite. Read the error, then explain why tests on SQLite could never catch this in the first place.
3. **Extend.** Set up a staging environment: a second droplet or Docker Compose stack, the same image, a separate database, and a subdomain. Deploy the same tag to staging and then to production.
4. **Explain.** Name three things Docker guarantees between laptop and production, and three it does not.

Solutions and hints are in the companion repository, in `exercises/chapter-39.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

The Notes API is complete as a single service on a single server. The final chapters look beyond that boundary: what changes when one service becomes several, talking to each other over a network — starting with the problems every distributed system inherits.

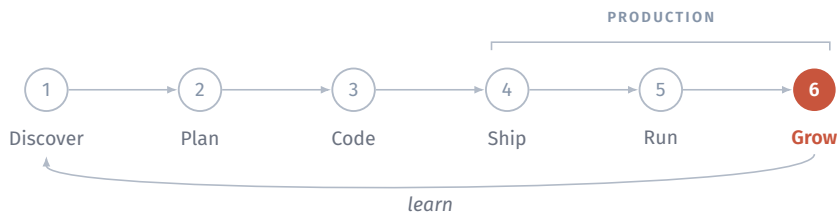
→ [Chapter 40](#) — Distributed Systems

# 11

## Beyond a Single Service

*How does the model stretch to distributed, asynchronous, data-heavy, and orchestrated systems?*

YOU ARE HERE



**The piece:** Many programs, on many machines

The whole map: *How Software Gets Built*, page 1





# Distributed Systems

---

*Stage 11 — Beyond a Single Service.* The previous thirty-nine chapters followed one service, on one machine, as far as it could go. The next four widen the lens to the paradigms you will meet in larger systems — distributed topologies, asynchronous flows, data platforms, and orchestrated fleets. They are deliberately *mental-model first*: lighter on code, heavier on the thinking, because what you most need when you land in unfamiliar territory is a map, not a tutorial. Each builds on the substrate you already have — processes, sockets, the request lifecycle, supervision, observability — so none of it is mysterious, only new.

This chapter is about what changes when one service becomes *many*.

The Notes API has run as a single Unicorn server behind one Nginx, talking to one database. That works until it doesn't: until one machine cannot serve the traffic, or until one machine failing means the whole service is down. The answer to both is the same — *more than one machine* — and the moment you have more than one, you are building a **distributed system**, and a new class of problems arrives that simply does not exist on a single box.

## In this chapter

By the end of this chapter, you will understand:

- Why systems distribute at all — scale and availability — and what it costs you
- The fundamental shift: the network is unreliable, and partial failure becomes normal
- **Replication and partitioning (sharding)** — the two ways to spread data across machines
- **Consistency models** and the CAP trade-off — strong vs eventual consistency
- Why distributed transactions are hard, and the **saga** pattern that replaces them
- Why **idempotency** becomes essential, not optional, at scale
- How coordination works — consensus, quorums, leader election — at a conceptual level

## 40.1 Why Distribute — and What It Costs

There are two main reasons to run more than one machine, and it is worth being precise about which you are buying:

- **Scale.** One machine has a ceiling — CPU, memory, connections. When demand exceeds it, you add machines and spread the load. This is **horizontal scaling** (Chapter 36), now taken to its conclusion: not two app servers, but fifty, and a database too big for one host.
- **Availability.** One machine is a single point of failure. If it dies, the service dies. Run the work on several machines and the service survives any one of them failing.

(Others come later, and more rarely: serving users on several continents with low latency, or keeping data inside a particular country's borders.)

Neither is free. The price of distribution is a new and permanent source of complexity: **the machines must talk over a network, and the network is not reliable**. Everything hard about distributed systems flows from that one fact. A wise default follows directly: *do not distribute until a single machine genuinely cannot do the job*. A single beefy server with a managed database carries most applications further than people expect, and it spares you every problem in this chapter. Distribution is a tool you reach for under proven pressure, not by default.

## 40.2 The Network Changes Everything

On a single machine, a function call either returns or it doesn't, and if your process is alive, so are the functions it calls. Across a network, none of that holds. The classic **fallacies of distributed computing** (L Peter Deutsch and colleagues at Sun Microsystems listed eight of them) are the comforting assumptions that quietly break: that the network is reliable, latency is zero, bandwidth is infinite, the topology never changes. It is none of those.

The deepest consequence is **partial failure**. On one machine, things mostly fail all-or-nothing — the process is up or down. In a distributed system, *some* parts work while others fail, and worst of all, **you often cannot tell which**. When the Notes API calls another service and the call times out, three things are indistinguishable from where you stand: the request never arrived; it arrived, succeeded, and the *reply* was lost; or it is still running. You sent a write and you genuinely do not know if it happened.

This single uncertainty — *did my request succeed?* — is the seed of most distributed-systems machinery. Retries, idempotency, timeouts, and consistency models all exist to cope with not knowing. Hold onto it; it explains everything that follows.

## 40.3 Spreading Data: Replication and Partitioning

Stateless app servers are easy to multiply — run fifty copies of the Notes API container behind a load balancer, and because each handles one request and holds nothing between requests, they need no coordination (this is exactly why [Chapter 36](#) checked that the app is stateless before scaling it out). **State is the hard part.** There are two fundamental moves for spreading data across machines, and real systems use both:

- **Replication** — keep *copies* of the same data on several machines. This buys availability (lose one replica, the others still serve) and read scale (spread reads across copies). Its cost is keeping the copies in agreement: when a note is written to one replica, the others are briefly stale. Replication is usually *leader-follower* — writes go to one leader, which streams changes to followers (this is what a managed database’s read replicas and automatic failover — part of the managed service [Chapter 20](#) chose — are doing under the hood).
- **Partitioning (sharding)** — *split* the data so each machine holds a different slice. Notes for users A–M live on shard 1, N–Z on shard 2. This buys write scale and lets the dataset exceed any single machine. Its cost is that queries spanning shards become hard, and choosing the **partition key** is a fateful decision — a bad one creates “hot” shards that concentrate load and defeat the point.

The mental model: replication is *the same data in many places* (for safety and read scale); partitioning is *different data in different places* (for size and write scale).

## 40.4 Consistency and the CAP Trade-off

Once data is replicated, a hard question appears: when you read, do you always get the *latest* write? The answer is a spectrum called the **consistency model**, and its two anchors are:

- **Strong consistency** — every read returns the most recent write, as if there were a single copy. Intuitive, but it requires replicas to coordinate on every write, which costs latency and stalls when they cannot reach each other.
- **Eventual consistency** — replicas converge *given time*, but a read just after a write may return stale data. Cheaper and more available, at the price of temporary disagreement your application must tolerate.

The reason you cannot simply always pick “strong” is captured by the **CAP theorem**: when a network **P**artition occurs (some machines cannot reach others — and partitions *will* occur), you must choose between **C**onsistency and **A**vailability. Refuse to serve possibly-stale data

and you sacrifice availability (the system stops); serve it and you sacrifice consistency. You cannot have both *during a partition*.

CAP is not abstract — it is a product decision. For the Notes API, a note appearing a few seconds late on one replica is mostly harmless: choose availability and eventual consistency. Mostly — a user who creates a note and immediately lists their notes expects to see it, so reads right after a write should go to the leader (**read-your-own-writes** consistency). For a bank balance or a “username already taken” check, stale data is dangerous: choose consistency, and accept that the feature may pause during a partition. The engineering skill is matching each piece of data to the weakest consistency it can tolerate, because weaker consistency is cheaper and more available.

## 40.5 Transactions Across Services: the Saga

Chapter 33 relied on database **transactions**: several writes that all commit or all roll back, atomically. That guarantee lives inside *one* database. The moment a business operation spans *two* services — say, a Notes service and a separate Billing service — there is no shared transaction to wrap them, and you cannot hold a lock across the network while waiting on another team’s system.

The standard replacement is the **saga**: break the operation into a sequence of local steps, each in its own service with its own local transaction, and for every step define a **compensating action** that undoes it. If step 3 fails, you run the compensations for steps 2 and 1 to walk the system back to consistency. There is no instant in which everything is atomically consistent; instead the system is *eventually* consistent, having either completed all steps or compensated for the ones it did. Sagas trade the simplicity of a transaction for the only thing available across services: explicit, reversible steps.

## 40.6 Idempotency Becomes Mandatory

Recall §40.2: when a call times out, you do not know if it succeeded, so you retry — and retries are the lifeblood of a reliable distributed system. But a retry of a non-idempotent operation does the work *twice*: two charges, two notes, two emails. Therefore, in a distributed system, **idempotency stops being a nice-to-have and becomes a requirement**. Every operation that can be retried must be safe to apply more than once.

You met the tools already, in the single-service world, and now you see why they mattered so much: idempotent HTTP methods and **idempotency keys** (Chapter 34), idempotent web-hook and job handlers (Chapters 25 and 34). At scale these are not polish — they are what

makes “retry on failure” safe instead of catastrophic. A useful instinct: whenever you add a network call, ask “what happens if this runs twice?” and design so the answer is “nothing bad.”

## 40.7 Coordination: Consensus, Quorums, Leaders

Sometimes distributed machines must *agree* on a single fact: which replica is the leader, whether a lock is held, what the next sequence number is. Agreement across unreliable machines is the deepest problem in the field, and it is solved by **consensus** algorithms (Raft and Paxos are the famous ones). You will rarely implement one, but you must recognize when you are leaning on one, because they are doing heavy lifting on your behalf.

Two ideas are enough to carry:

- **Quorum** — a decision is made by a *majority* of machines, not all of them. Requiring a majority both to write and to read guarantees the two sets overlap, so every read reaches at least one machine that has the latest committed write (and, by comparing versions, picks it) — agreement without needing every machine present, which is what lets the system tolerate some being down.
- **Leader election** — the machines use consensus to pick one leader to make decisions; if it dies, they elect another. This is how a replicated database promotes a new primary on failover. The danger to know by name is **split-brain** — two machines both believing they are leader during a partition — which quorums exist to prevent.

When you find yourself wanting a distributed lock or a single source of truth across machines, reach for a system built on consensus (ZooKeeper, etcd, Consul) rather than inventing your own. Hand-rolled coordination is where careers go to learn humility.

### Key Takeaways

- Distribute mainly for two reasons — **scale** and **availability** — and not before a single machine truly cannot cope. Distribution’s price is permanent complexity born from one fact: **the network is unreliable**.
- **Partial failure** is the defining problem: parts fail while others work, and a timeout leaves you unable to tell whether your write succeeded. Most distributed machinery exists to cope with that uncertainty.
- Spread data two ways: **replication** (same data, many copies — for availability and read scale) and **partitioning/sharding** (different data, different machines — for size and write scale). Keep app servers **stateless** so they multiply freely.
- **CAP**: during a network partition you choose **consistency** or **availability**, not both. Match each piece of data to the weakest consistency it can tolerate.

- Across services, there are no shared transactions — use **sagas** (local steps + compensating actions) and accept **eventual consistency**.
- Retries are essential, so **idempotency is mandatory**: every retryable operation must be safe to run twice.
- Agreement across machines is **consensus**; lean on **quorums** and **leader election** via battle-tested tools, and never hand-roll a distributed lock.

## Exercises

1. **Predict.** Two replicas of a database accept writes, and the network between them breaks. What can each side do if you choose consistency? If you choose availability?
2. **Break and fix.** Simulate a lost response: make an endpoint commit a transfer, then return 500. Call it with a client that retries. What happens, and how does an idempotency key fix it?
3. **Extend.** Design, on paper, a saga for “create a user and a billing account in another service”, with a compensating action for each step.
4. **Explain.** Why is “never hand-roll a distributed lock” such common advice?

Solutions and hints are in the companion repository, in `exercises/chapter-40.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What’s Next

Distribution forces work apart in *space*. The next chapter decouples it in *time*: instead of one service calling another and waiting, work is handed off as events and messages to be processed when convenient — the asynchronous, event-driven architectures that large systems are built from.

→ [Chapter 41](#) — Event-Driven and Asynchronous Architecture

# Event-Driven and Asynchronous Architecture

---

The Notes API, like most of this book, is **synchronous and request-driven**: a client calls, the server does the work *right then* while the client waits, and a response comes back. That model is simple and correct, and it is the right default. But it has two limits. Slow work makes the caller wait (resize an image, call a sluggish third party, and the request hangs). And it couples systems tightly in time — the caller and the callee must both be up, at the same moment, for anything to happen.

**Event-driven architecture** loosens both. Instead of calling another service and waiting, a service announces that something happened — an **event** — and moves on. Other services react to that event later, on their own schedule. Where [Chapter 40](#) decoupled work in *space* (across machines), this chapter decouples it in *time*.

## In this chapter

By the end of this chapter, you will understand:

- The difference between synchronous request/response and asynchronous, event-driven flow
- What a **message queue** is, and the producer / broker / consumer model
- Why teams reach for async: decoupling, buffering, resilience, and independent scaling
- **Delivery semantics** (at-least-once, the exactly-once myth) and why idempotency returns
- **Queues** vs **publish/subscribe**, and what an “event” really is
- How to build a job queue in PostgreSQL (SKIP LOCKED, leases, retries with backoff, scheduled jobs) and run a worker next to the API, with its own metrics
- **Event sourcing** and **CQRS** at a conceptual level, and what log-based **streaming** adds
- **Backpressure**, and the real costs: eventual consistency and harder debugging

## 41.1 Two Ways for Systems to Talk

Hold two shapes in mind.

**Synchronous (request/response):** Service A calls Service B and *blocks* until B replies. A depends on B being available, fast, and correct, right now. If B is slow, A is slow; if B is down, A’s request fails. This is every call the Notes API has made so far: each query to its database waits for the answer, and so, since [Chapter 37](#), does its call to the email service.

**Asynchronous (message/event):** Service A writes a **message** to an intermediary and immediately continues. Later — milliseconds or minutes — Service B picks the message up and processes it. A does not wait for B, does not need B to be up, and does not even need to know B exists. The intermediary holds the message until someone is ready.

The shift sounds small and changes everything. The synchronous question is “who do I call?” The asynchronous question is “what happened, and who might care?” That inversion — from *commands aimed at a known recipient* to *events broadcast to unknown listeners* — is the heart of event-driven design.

## 41.2 The Message Queue

The intermediary, in its simplest form, is a **message queue**. Three roles:

- A **producer** puts a message on the queue (the Notes API, having accepted a new note, enqueues “index this note for search”).
- A **broker** — the queue system itself (RabbitMQ, Amazon SQS, Redis-backed Celery or RQ) — stores the message durably until it is handled.
- A **consumer** (a **worker** process, supervised by systemd exactly like the API — [Chapter 25](#)) takes the message off and does the work.

You met this exact pattern in §25.9, as the answer to “what about work that doesn’t belong inside a request?” Here is why it is worth the extra moving part:

- **Decoupling.** Producer and consumer never talk directly; either can be deployed, restarted, or scaled without the other knowing.
- **Buffering / load-leveling.** A traffic spike fills the queue instead of overwhelming the workers. The queue absorbs the burst; workers drain it at their own steady pace. The system bends instead of breaking.
- **Resilience.** If the workers are down, messages wait in the queue rather than being lost. Work resumes when they return.
- **Independent scaling.** Slow work that piles up? Add more workers. The web tier and the



worker tier scale separately, each to its own bottleneck.

The cost is that the result is no longer immediate. The note becomes searchable *soon*, not *now* — which is fine for a search index and unacceptable for “is this password correct?” Async is for work the caller does not need the answer to *right now*.

### 41.3 Delivery Semantics, and Why Idempotency Returns

Because a broker and its consumers are themselves a small distributed system, the uncertainty of Chapter 40 reappears: after a worker processes a message, it must tell the broker “done” so the message is removed — and that acknowledgement can be lost. So brokers offer **delivery guarantees**, and you must know which you have:

- **At-most-once** — each message delivered zero or one time. Simple, but messages can be silently dropped. Rarely acceptable.
- **At-least-once** — each message delivered one *or more* times. Nothing is lost, but a message can be **redelivered** (the work happened, the ack was lost, so the broker hands it out again). This is the common, practical default.
- **Exactly-once** — the holy grail, and largely a myth in the strict sense. What real systems do is combine at-least-once delivery with **idempotent consumers**, achieving *effectively-once* processing.

So idempotency arrives once more (after Chapters 25, 34, and 40), and the lesson is now unmistakable: a **message consumer will eventually process the same message twice, so it must be safe to do so**. Record which messages you have handled, or design the work so re-applying it changes nothing. Build the consumer any other way and a lost acknowledgement becomes a duplicate charge.

A related subtlety is **ordering**: most queues do *not* guarantee messages are processed in the order produced (workers run in parallel; redelivery reshuffles). If order matters — “created” must precede “updated” — you need an ordering key or a system that preserves order per key, and you should not assume it for free.

### 41.4 Queues vs Pub/Sub, and What an “Event” Is

Two distribution shapes sit on top of brokers:

- **Queue (point-to-point)**: each message is handled by *exactly one* consumer from a pool. Use it to *distribute work* — “index this note” should happen once, by whichever worker is

free.

- **Publish/subscribe (pub/sub):** each message is delivered to *every* interested subscriber. Use it to *announce facts* — “a new note was created” might interest a search-indexer, an analytics pipeline, and a notifications service, each reacting independently.

Pub/sub is where event-driven architecture comes into its own, and it turns on the meaning of an **event**. A *command* says “do this” to a known recipient (“index note 42”). An **event** states “this happened” as a fact, addressed to no one in particular (“NoteCreated”, “UserSignedUp”). The producer of an event does not know or care who consumes it. That is what makes the architecture extensible: to add a feature that reacts to new notes, you subscribe a new service to NoteCreated and *change nothing* in the Notes API. New behavior is added by adding listeners, not by editing the source of the event.

## 41.5 Building It: A Job Queue for the Notes API

Everything so far in this chapter has been ideas. This section builds the smallest real version of them. Chapter 37 gave the Notes API an endpoint that emails a note, and it calls the email service *inside the request*. The client waits while the provider thinks, and a slow provider can hold a Unicorn worker for more than ten seconds. That is the textbook case for background work: the client does not need the email *sent*, only *accepted*.

The plan:

- The endpoint stops sending. It records a **job**, “email note 42 to this address”, and answers 202 Accepted at once. The answer is the same 202 it already gave, so no client notices the change.
- A separate **worker** process, running the same code, takes jobs and does them. It retries failures with growing delays, gives up after a limit, and never loses or duplicates work when it crashes.
- The queue is a **table in PostgreSQL**.

### Why a table, and not a message broker?

Section 41.2 listed RabbitMQ, Amazon SQS and Redis-based queues. Any of them would work. The Notes API uses the database it already has, for three reasons:

- **No new system to run.** A broker is one more service to deploy, secure, monitor and back up. The database is already all of those.
- **The job and the change that causes it commit together.** Inserting the job row happens in the *same transaction* as the rest of the request’s work. If the transaction commits, the

job exists; if it rolls back, the job never existed. With a separate broker, you would write to two systems, and a crash between the two leaves them disagreeing: a user created with no welcome email, or an email about a user who was never created. (This is called the **transactional outbox** pattern, and teams that *do* use a broker often keep it for exactly this reason: they write jobs to an “outbox” table, and a small process forwards them to the broker.)

- **PostgreSQL has the one feature a queue needs:** `SELECT ... FOR UPDATE SKIP LOCKED`, which lets many workers take jobs at the same time without ever taking the same one.

A database is not the right queue for everything. At thousands of jobs a second, or when many independent consumers need the same events (the publish/subscribe of section 41.4), a dedicated broker wins. But a database table serves a very large range of real applications well, and it teaches every mechanism a broker has.

## The jobs table

This migration exists only for PostgreSQL. Since [Chapter 39](#), PostgreSQL is the only database the Notes API uses:

```
-- migrations/postgresql/0005_add_jobs.sql
-- Background jobs (Chapter 41): work the API accepts now and does later.
CREATE TABLE jobs (
 id BIGSERIAL PRIMARY KEY,
 kind TEXT NOT NULL, -- what to do, e.g. 'email_note'
 payload JSONB NOT NULL, -- its arguments
 status TEXT NOT NULL DEFAULT 'pending'
 CHECK (status IN ('pending', 'running', 'done', 'failed')),
 attempts INTEGER NOT NULL DEFAULT 0,
 max_attempts INTEGER NOT NULL DEFAULT 5,
 run_at TIMESTAMPTZ NOT NULL DEFAULT now(), -- not before this time
 locked_until TIMESTAMPTZ, -- a running job's lease (see the worker)
 last_error TEXT,
 created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
 finished_at TIMESTAMPTZ
);

-- The worker's question, "what is ready to run?", touches only unfinished
-- jobs: a partial index keeps it small however many finished jobs pile up
CREATE INDEX idx_jobs_ready ON jobs (run_at) WHERE status IN ('pending', 'running');
```

Each column answers a question the worker will ask:

Column	The question it answers
kind, payload	What should be done, and with what arguments?
status	Is it waiting, being worked on, finished, or given up on?
attempts, max_attempts	How many times has it been tried, and how many tries does it get?
run_at	When may it run? A retry, or a job scheduled for later, sets it in the future.
locked_until	If a worker took it, until when is that worker's claim valid?
last_error	Why did the last attempt fail? This is the first thing to read when a job fails.

## Queueing a job

```
app/jobs.py
"""
The job queue: a table in PostgreSQL that the API writes to and the worker
reads from (Chapter 41).
"""

import json
import random
from datetime import datetime

from app.database import sql

LEASE_SECONDS = 300 # a job must finish within this, or it counts as abandoned

class PermanentError(Exception):
 """Raised by a task when trying again cannot help: the job fails at once."""

def enqueue(
 cursor, kind: str, payload: dict, run_at: datetime | None = None
) -> int:
 """
 Add a job, inside the caller's transaction: it becomes visible to the
 worker only when that transaction commits, together with everything else.
 """
```

```

cursor.execute(
 sql("""
 INSERT INTO jobs (kind, payload, run_at)
 VALUES (?, CAST(? AS JSONB), COALESCE(?, now()))
 RETURNING id
 """),
 (kind, json.dumps(payload), run_at),
)
return cursor.fetchone()["id"]

def claim(conn) -> dict | None:
 """
 Take the next job that is ready, or None. SKIP LOCKED lets many workers
 claim at once without waiting for, or taking, each other's jobs.
 """
 cursor = conn.cursor()
 cursor.execute(
 sql("""
 UPDATE jobs
 SET status = 'running',
 attempts = attempts + 1,
 locked_until = now() + make_interval(secs => ?)
 WHERE id = (
 SELECT id FROM jobs
 WHERE (status = 'pending' AND run_at <= now())
 OR (status = 'running' AND locked_until < now()) -- abandoned
 ORDER BY run_at
 LIMIT 1
 FOR UPDATE SKIP LOCKED
)
 RETURNING id, kind, payload, attempts, max_attempts
 """),
 (LEASE_SECONDS,),
)
 job = cursor.fetchone()
 conn.commit() # the claim is its own short transaction
 return dict(job) if job else None

def complete(conn, job_id: int) -> None:
 _finish(conn, job_id, "done", None)

```

```

def fail(conn, job_id: int, error: str) -> None:
 _finish(conn, job_id, "failed", error)

def retry_later(conn, job: dict, error: str) -> None:
 """Put the job back, to run again after a growing delay (with jitter)."""
 delay = min(30 * 2 ** (job["attempts"] - 1), 3600) * random.uniform(0.8, 1.2)
 cursor = conn.cursor()
 cursor.execute(
 sql("""
 UPDATE jobs
 SET status = 'pending', locked_until = NULL, last_error = ?,
 run_at = now() + make_interval(secs => ?)
 WHERE id = ?
 """),
 (error[:1000], delay, job["id"]),
)
 conn.commit()

def _finish(conn, job_id: int, status: str, error: str | None) -> None:
 cursor = conn.cursor()
 cursor.execute(
 sql("""
 UPDATE jobs
 SET status = ?, locked_until = NULL, last_error = ?, finished_at = now()
 WHERE id = ?
 """),
 (status, error[:1000] if error else None, job_id),
)
 conn.commit()

def stats(conn) -> dict:
 """How many jobs are waiting, and for how long the oldest has waited."""
 cursor = conn.cursor()
 cursor.execute("""
 SELECT
 count(*) FILTER (WHERE status = 'pending' AND run_at <= now()) AS ready,
 count(*) FILTER (WHERE status = 'running') AS running,
 count(*) FILTER (WHERE status = 'failed') AS failed,
 coalesce(extract(epoch FROM now()) - min(run_at)
 FILTER (WHERE status = 'pending' AND run_at <= now()), 0) AS oldest
 FROM jobs
 """)

```

```

 """
 row = dict(cursor.fetchone())
 conn.commit()
 return row

```

The endpoint now queues instead of sending:

```

app/routes/notes.py (the email endpoint, now queueing; add "import uuid")
@notes_bp.route("/<int:note_id>/email", methods=["POST"])
def email_note(note_id: int):
 """Email a note: POST {"to": "someone@example.com"} → 202 Accepted."""
 validator = Validator(request.get_json(silent=True))
 to = validator.require_email("to")
 if to is None:
 return validator.error_response()

 db = get_db()
 cursor = db.cursor()
 cursor.execute(
 sql("SELECT id FROM notes WHERE id = ? AND user_id = ?"),
 (note_id, g.user["id"]),
)
 if cursor.fetchone() is None:
 return jsonify({"error": f"Note {note_id} not found"}), 404

 # Not sent here: queued, and sent by the worker (Chapter 41)
 job_id = enqueue(
 cursor,
 "email_note",
 {
 "note_id": note_id,
 "user_id": g.user["id"],
 "to": to,
 "idempotency_key": str(uuid.uuid4()),
 },
)
 db.commit()
 logger.info("Queued job id=%d: email note id=%d.", job_id, note_id)
 return jsonify({"status": "accepted"}), 202

```

Two decisions are hidden in the payload:

- It stores the note's id, not its content. The worker reads the note when it runs, so it sends

the note as it is *then*, or nothing if the note was deleted in the meantime. The other choice, storing a snapshot of the content, is right when the job must describe the moment it was created, such as “email the receipt for this order”. Choose deliberately.

- The **idempotency key** is chosen now, once. Every attempt to send this email, including one made by a *different* worker after a crash, uses the same key, so an email service that supports keys sends it exactly once (section 37.5).

For that, `send_email()` from section 37.5 takes the key as a new, optional parameter, instead of always making a fresh one:

```
app/email.py (changed): the caller may choose the key
def send_email(
 to: str, subject: str, text: str, idempotency_key: str | None = None
) -> str:
 ...
 headers = {"Idempotency-Key": idempotency_key or str(uuid.uuid4())}
```

The endpoint’s contract changes too. It no longer talks to the email service, so it can no longer answer 503, 504 or 502. Those three responses leave `openapi.yaml`, which now promises 202, 400 and 404. Chapter 37’s endpoint tests, which replaced `send_email()` with a fake and checked those statuses, become queue tests: after a 202, exactly one `email_note` job is waiting, with the right payload. The failure cases move to the worker’s tests, below.

## Taking a job: FOR UPDATE SKIP LOCKED

The heart of the queue is `claim()`. Its inner `SELECT` finds the oldest job that is ready. `FOR UPDATE` locks that row, and **SKIP LOCKED** makes other workers, running the same query at the same moment, *skip* rows that are locked instead of waiting for them. Three workers polling together therefore take three different jobs, never the same one twice, and never queue up behind each other. The outer `UPDATE` marks the job as running, counts the attempt, and records a *lease* (`locked_until`, five minutes from now), all in one short transaction.

The lease is how the queue survives a crash. Suppose a worker takes a job and then dies, killed by the out-of-memory killer, or its server lost. The job is stuck at running, and nobody is working on it. The lease makes that state temporary. Once `locked_until` has passed, `claim()` treats the job as abandoned and hands it out again. The cost is one rule: **a job must finish well within its lease**, or a second worker will start it while the first is still busy.

Together, this gives the queue the guarantee of section 41.3: **at-least-once**. A job is never lost, but it may run twice. For example, a worker sends the email and crashes before marking the job done, so after the lease runs out the job runs again. That is why tasks must be



idempotent. This one is, thanks to the idempotency key.

## The worker

```
app/tasks.py
"""
What each kind of job does. A task receives a database connection and the
job's payload; it raises PermanentError when trying again cannot help.
"""

import logging

from app.database import sql
from app.email import EmailRejectedError, send_email
from app.jobs import PermanentError

logger = logging.getLogger(__name__)

def email_note(conn, payload: dict) -> None:
 cursor = conn.cursor()
 cursor.execute(
 sql("SELECT content FROM notes WHERE id = ? AND user_id = ?"),
 (payload["note_id"], payload["user_id"]),
)
 row = cursor.fetchone()
 conn.rollback() # end the read transaction before the slow call (33.13)
 if row is None:
 logger.info("Note id=%d is gone; nothing to send.", payload["note_id"])
 return
 try:
 send_email(
 to=payload["to"],
 subject=f"Note {payload['note_id']}",
 text=row["content"],
 # Chosen once, when the job was created: every attempt, even one
 # after a worker crash, asks the service for the same email
 idempotency_key=payload["idempotency_key"],
)
 except EmailRejectedError as error:
 raise PermanentError(str(error)) from error

HANDLERS = {"email_note": email_note}
```

```

app/worker.py
"""
The worker: a second process, running the same code as the API, that
takes jobs from the queue and runs them.

 python -m app.worker
"""

import logging
import signal
import sys
import time

from prometheus_client import Counter, Gauge, start_http_server

from app import jobs, startup
from app.database import open_db
from app.logging_config import configure_logging
from app.tasks import HANDLERS

Named explicitly: run as "python -m app.worker", __name__ is "__main__"
logger = logging.getLogger("app.worker")

POLL_SECONDS = 1.0 # how long to wait when the queue is empty
STATS_SECONDS = 15.0 # how often to refresh the queue metrics

JOBS_DONE = Counter("worker_jobs_total", "Jobs processed", ["kind", "outcome"])
QUEUE_READY = Gauge("worker_queue_ready_jobs", "Jobs ready to run, waiting")
QUEUE_OLDEST = Gauge("worker_queue_oldest_seconds", "Wait of the oldest ready job")
QUEUE_FAILED = Gauge("worker_queue_failed_jobs", "Jobs that failed for good")

class Worker:
 def __init__(self):
 self.stopping = False

 def stop(self, signum, frame):
 """SIGTERM: finish the current job, then exit (Chapter 37)."""
 logger.info(
 "Received %s; stopping after the current job.",
 signal.Signals(signum).name,
)
 self.stopping = True

```

```

def run(self) -> None:
 conn = open_db()
 last_stats = 0.0
 try:
 while not self.stopping:
 if time.monotonic() - last_stats > STATS_SECONDS:
 self.refresh_stats(conn)
 last_stats = time.monotonic()
 job = jobs.claim(conn)
 if job is None:
 time.sleep(POLL_SECONDS)
 continue
 self.process(conn, job)
 finally:
 conn.close()
 logger.info("Worker stopped.")

def process(self, conn, job: dict) -> None:
 kind, job_id = job["kind"], job["id"]
 started = time.perf_counter()
 try:
 HANDLERS[kind](conn, job["payload"])
 except jobs.PermanentError as error:
 conn.rollback()
 jobs.fail(conn, job_id, str(error))
 outcome = "failed"
 except Exception as error: # anything else may be temporary
 conn.rollback()
 if job["attempts"] >= job["max_attempts"]:
 jobs.fail(conn, job_id, repr(error))
 outcome = "failed"
 else:
 jobs.retry_later(conn, job, repr(error))
 outcome = "retry"
 else:
 jobs.complete(conn, job_id)
 outcome = "done"
 JOBS_DONE.labels(kind, outcome).inc()
 elapsed_ms = (time.perf_counter() - started) * 1000
 log = logger.warning if outcome != "done" else logger.info
 log(
 "Job id=%d (%s, attempt %d): %s in %.0f ms",
 job_id,
 kind,

```

```

 job["attempts"],
 outcome,
 elapsed_ms,
)

 def refresh_stats(self, conn) -> None:
 numbers = jobs.stats(conn)
 QUEUE_READY.set(numbers["ready"])
 QUEUE_OLDEST.set(float(numbers["oldest"]))
 QUEUE_FAILED.set(numbers["failed"])

 def main() -> int:
 configure_logging()
 startup.run() # the same checks as the API: config, database, schema
 worker = Worker()
 signal.signal(signal.SIGTERM, worker.stop)
 signal.signal(signal.SIGINT, worker.stop) # Ctrl+C, when run by hand
 start_http_server(9201) # metrics for Prometheus, like the API's 9200
 logger.info("Worker started.")
 worker.run()
 return 0

if __name__ == "__main__":
 sys.exit(main())

```

The worker is a loop: claim a job, run it, record the outcome, and repeat. When the queue is empty, it sleeps for a second before asking again. That polling costs one tiny query a second, and it means a new job waits up to a second. (PostgreSQL's LISTEN/NOTIFY can wake workers instantly; it is a refinement to add when that second matters.)

Every attempt ends in exactly one of three outcomes:

What happened	Outcome	Next
the task finished	done	nothing
the task raised <code>PermanentError</code> (the email service rejected the request)	failed	a person looks at <code>last_error</code>
anything else went wrong (service down, timeout, a bug)	retry	runs again after 30 s, 60 s, 120 s..., with jitter, until <code>max_attempts</code> (5), then failed

Treating *unknown* errors as temporary is deliberate. A bug that raises `KeyError` will fail five times over about eight minutes and then land in `failed`, with its traceback text in `last_error`. That costs a little, and it never loses a job to a failure that would have passed on its own.

Two details make the worker a good citizen:

- **Graceful shutdown.** `docker stop` sends `SIGTERM` (Chapter 37, section 37.8). The worker’s handler only sets a flag, so the job in progress finishes and is recorded, and the loop then exits. Docker’s `--stop-timeout 35` gives it half a minute to do so. The email task takes seconds at most, well within that.
- **The same startup checks as the API.** `startup.run()` refuses to start with a bad configuration, an unreachable database, or a schema that is behind (section 33.10). A worker running old code against a new schema, or new code against an old one, fails loudly at startup instead of mysteriously later.

## Scheduled jobs

`run_at` makes scheduling almost free. A job enqueued with `run_at` set to tomorrow at nine simply is not ready until then:

```
tomorrow_at_nine = datetime(2024, 1, 16, 9, 0, tzinfo=timezone.utc)
enqueue(cursor, "email_note", payload, run_at=tomorrow_at_nine)
```

This is how you build reminders (“email me this note on Friday”), delayed follow-ups, and retries. Recurring work, like a nightly clean-up, still needs something to create the job every night. That is a cron job or systemd timer (Chapter 25, section 25.9) whose only task is to enqueue it, so that the work itself gets the queue’s retries, visibility and single-runner guarantee.

## Seeing the queue

A queue is where work goes to be forgotten. [Chapter 25](#) warned that background work fails silently, and a queue without monitoring is the purest example. Every fifteen seconds the worker refreshes three **gauges**, and it counts every job it processes. It serves them on port 9201, just as the API serves its metrics on 9200 ([Chapter 35](#)):

```
worker_jobs_total{kind="email_note",outcome="done"} 1.0
worker_queue_ready_jobs 0.0
worker_queue_oldest_seconds 0.0
worker_queue_failed_jobs 0.0
```

Add the worker to Prometheus's `scrape_configs` as a second job (`targets: ["127.0.0.1:9201"]`). Then alert on the two numbers that mean trouble:

- **worker\_queue\_oldest\_seconds > 300**: work is waiting too long. Either the workers are down, or they cannot keep up. This is the backpressure signal of [section 41.7](#), as one number.
- **worker\_queue\_failed\_jobs rising**: something is failing for good, and each failure is a user whose email never came. A person should look at `last_error`, fix the cause, and requeue the failed jobs if they are still wanted (`UPDATE jobs SET status = 'pending', attempts = 0, run_at = now() WHERE status = 'failed' AND ...`).

Finished jobs pile up. Delete old done jobs on a schedule (say, after a week). That is the retention policy of [Chapter 37](#), [section 37.4](#), applied to your own bookkeeping.

## Running the worker

The worker runs from **the same image** as the API; only the command differs. In production it is a second container next to the API's. Add it to the deploy script from [Chapter 23](#), *after* the new API container has passed its health check (a release that fails the check rolls back before it touches the worker):

```

Step 6: Replace the worker. It takes no traffic, so no blue-green: stop the
old one (SIGTERM: it finishes its current job), start the new one.

if docker container inspect notes-worker > /dev/null 2>&1; then
 docker stop notes-worker >> "$LOG_FILE" 2>&1
 docker rm notes-worker >> "$LOG_FILE" 2>&1
fi
docker run -d \
 --name notes-worker \
 --restart unless-stopped \
 --stop-timeout 35 \
 --no-healthcheck \
 -p 127.0.0.1:9201:9201 \
 --env-file "$ENV_FILE" \
 "$FULL_IMAGE" python -m app.worker >> "$LOG_FILE" 2>&1
log "Worker started."
```

`--no-healthcheck` matters. The image's `HEALTHCHECK` (Chapter 18) asks `http://localhost:5000/health`, and the worker serves no web requests, so Docker would mark it unhealthy forever. The worker's health is its metrics: is `worker_queue_oldest_seconds` staying low? The environment file gains the email settings from section 37.5 (`EMAIL_API_URL`, `EMAIL_API_KEY`, `EMAIL_FROM`), which only the worker now uses.

Locally, `docker-compose.yml` from Chapter 39 gains the worker and Mailpit:

```
worker:
 build: .
 depends_on:
 db:
 condition: service_healthy
 environment:
 DATABASE_URL: postgresql://notes_user:dev_password@db/notesdb
 SECRET_KEY: dev-secret-key-not-for-production-1234
 EMAIL_API_URL: http://mailpit:8025/api/v1/send
 LOG_FORMAT: human
 volumes:
 - ../app
 # Migrating is safe here too: the runner's lock (section 33.10) makes
 # the API and the worker take turns
 command: ["sh", "-c", "python -m app.migrate && exec python -m app.worker"]
 healthcheck:
```

```

 disable: true

mailpit:
 image: axllent/mailpit
 ports:
 - "127.0.0.1:8025:8025" # the web inbox: http://localhost:8025

```

Scaling is now a separate decision for each half. When the queue’s oldest job keeps growing, run a second worker container (notes-worker-2, same command). SKIP LOCKED has made that safe from the start. That is the “independent scaling” of section 41.2, for real.

## Tests

Every test now starts with an empty queue too. In tests/conftest.py, the table list becomes TRUNCATE notes, tags, users, jobs RESTART IDENTITY CASCADE; without jobs, jobs left by one test would be claimed by the next.

The worker’s tests run it one job at a time, with send\_email replaced by a fake that can be told to fail. Here is the most important one, which proves that a retried job reuses its key:

```

tests/test_worker.py (excerpt)
def test_a_temporary_failure_is_retried_later_with_the_same_key(
 conn, queued_email, fake_email
):
 fake_email.failures = [email.EmailOutcomeUnknownError("no answer")]
 run_next_job(conn)
 row = job_row(conn)
 assert row["status"] == "pending" and row["attempts"] == 1 and row["later"]

 conn.cursor().execute("UPDATE jobs SET run_at = now()") # skip the wait
 conn.commit()
 run_next_job(conn)
 assert job_row(conn)["status"] == "done"
 keys = [attempt["key"] for attempt in fake_email.attempts]
 assert len(keys) == 2 and keys[0] == keys[1] # the service can deduplicate

def test_a_crashed_workers_job_is_taken_again(conn, queued_email, fake_email):
 abandoned = jobs.claim(conn) # a worker claims it... and dies
 assert jobs.claim(conn) is None # while its lease runs, nobody else takes it
 conn.cursor().execute(
 "UPDATE jobs SET locked_until = now() - interval '1 second'"
)

```



```
conn.commit() # the lease expires
again = jobs.claim(conn)
assert again["id"] == abandoned["id"] and again["attempts"] == 2
```

Other tests in the file cover a rejected email (failed at once), the attempt limit, a note deleted before its job ran (nothing sent, job done), and two workers claiming at the same moment (different jobs). Run end to end against PostgreSQL and Mailpit, the change is easy to see: the API now answers 202 in a few milliseconds instead of waiting on the provider, and the email lands in Mailpit’s inbox about a second later, when the worker’s next poll picks it up.

## 41.6 Event Sourcing, CQRS, and Streaming

Three ideas you will hear, worth recognizing even from a distance:

- **Event sourcing** — instead of storing only the current state (the notes table as it is *now*), store the full, append-only log of events that produced it (`NoteCreated`, `NoteEdited`, `NoteDeleted`). Current state becomes a *replay* of the log. This gives a perfect audit trail and the ability to reconstruct any past state, at the cost of more complexity and no simple “just read the row.”
- **CQRS** (Command Query Responsibility Segregation) — use *different models* for writing and for reading. Writes go through one path (often event-sourced); reads come from separate, denormalized views optimized for queries and kept up to date by consuming the events. Powerful when read and write patterns differ sharply; overkill for most CRUD.
- **Streaming** — log-based brokers like **Apache Kafka** blur the line between “message queue” and “database.” Events are kept in an ordered, replayable, durable log that many consumers read independently at their own positions. This underpins both event-driven microservices and the data pipelines of the next chapter, because the same event stream can feed an operational consumer *and* an analytics one.

You do not need these for a Notes API. You need to recognize them when you join a system built on them, and to know that each is a heavier trade made for a specific payoff — audit, read/write asymmetry, or replayable scale.

## 41.7 Backpressure and the Real Costs

A queue that fills faster than workers drain it is a warning, not a relief: latency climbs, memory or disk fills, and eventually something breaks. **Backpressure** is the system’s way of signaling “slow down” — bounding queue size, rejecting or shedding load, or scaling consumers — so a

producer cannot indefinitely outrun its consumers. An async system without a backpressure plan has simply moved its breaking point out of sight, not removed it.

And be honest about what async costs, because it is not free:

- **Eventual consistency** (Chapter 40, again): the world is not immediately up to date after an event, and the UI and the user must tolerate the lag.
- **Harder debugging.** A synchronous request is one stack trace. An event-driven flow is a chain of hops across producers, brokers, and consumers, with no single trace tying them together — unless you invest in **distributed tracing** and correlation IDs (the request-ID discipline of Chapter 35, extended across services). Observability stops being optional the moment work goes async.

Reach for asynchronous, event-driven design when decoupling, buffering, or independent scaling genuinely pay for that complexity — and keep the synchronous path for everything that needs an answer now.

#### Key Takeaways

- **Synchronous** = call and wait (simple, immediate, tightly coupled in time). **Asynchronous/event-driven** = announce and move on (decoupled, buffered, eventually consistent). Async is for work the caller does not need answered *right now*.
- A **message queue** has a **producer**, a durable **broker**, and **consumer** workers. It buys decoupling, load-leveling, resilience, and independent scaling — at the price of immediacy.
- Real brokers deliver **at-least-once**, so messages get **redelivered**: consumers must be **idempotent** (idempotency, once more) and must not assume ordering.
- **Queues** distribute work to one consumer; **pub/sub** broadcasts **events** (“this happened”) to many. Events let you add features by adding subscribers, changing nothing upstream.
- A **job table in PostgreSQL** is a real queue: enqueue in the same transaction as the change that causes it (the outbox idea), claim with `FOR UPDATE SKIP LOCKED`, protect against crashed workers with a **lease**, retry with growing delays up to a limit, and fail permanently only for errors that retrying cannot fix. Run the worker from the same image, stop it with `SIGTERM`, and watch its queue (the age of the oldest waiting job, and failed jobs).
- **Event sourcing** (store the event log, not just state), **CQRS** (separate read/write models), and **streaming** (Kafka’s replayable log) are heavier patterns for audit, read/write asymmetry, and replayable scale.
- Plan for **backpressure**, and pay for **distributed tracing** — async turns one stack trace into a cross-service chain.

## Exercises

1. **Predict.** Queue 1,000 email jobs with one worker running. Watch `worker_queue_oldest_seconds`. Now start a second worker. How does the curve change?
2. **Break and fix.** Remove `SKIP LOCKED` from `claim()`, queue 2,000 jobs that do almost nothing, and time how long four workers take to empty the queue, with and without it. Is any job taken twice? Why is the difference smaller than you might expect, and when would it grow?
3. **Extend.** The worker already counts retries: `worker_jobs_total{outcome="retry"}`. Write the PromQL for a Grafana panel (or the Prometheus graph page) showing retries per minute by kind, and an alert rule that fires when there are more than ten a minute for ten minutes.
4. **Explain.** Why does the book enqueue the job in the same transaction as the request's other changes?

Solutions and hints are in the companion repository, in `exercises/chapter-41.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

Events and streams carry not just *commands to act on* but *data to learn from*. The next chapter follows that data off the request path entirely, into the analytics plane — the warehouses, lakes, and pipelines a system grows alongside its transactional core.

→ [Chapter 42](#) — The Data and Analytics Plane



# The Data and Analytics Plane

Everything the Notes API does with data so far is **operational**: store a note, fetch a note, update a note — small, fast reads and writes that keep the application running. That is the **transactional plane**, and PostgreSQL is built for it.

Then a different kind of question arrives, and it does not come from a user. It comes from the business: *How many notes are created per day, and is that growing? What fraction of users who sign up are still active after a month? Which features correlate with retention?* These are **analytical** questions — they sweep across millions of rows, span months of history, and aggregate rather than fetch. Answer them by running big queries against your production database and you will slow or crash the very system serving your users. So real systems grow a second, separate **data plane** for analytics. This chapter is the map of it.

## In this chapter

By the end of this chapter, you will understand:

- The difference between **OLTP** (operational) and **OLAP** (analytical) workloads, and why one database cannot serve both well
- Why analytics needs **columnar** storage and a separate system
- **Data warehouses**, **data lakes**, and the **lakehouse** — what each is for
- **ETL** vs **ELT**, and **batch** vs **streaming** pipelines
- The shape of a modern data platform: ingest → store → transform → serve
- Analytical data modeling — **facts and dimensions** (the star schema)
- Why **data quality**, **lineage**, and **governance** become first-class concerns

## 42.1 Two Workloads That Pull in Opposite Directions

The split has a name on each side:

- **OLTP — Online Transaction Processing**. Many small, concurrent operations, each touch-

ing a few rows: insert a note, read a note by id. It wants low latency per operation, high concurrency, and strong consistency. It is optimized to find and change *individual records* fast. This is the Notes API’s PostgreSQL.

- **OLAP — Online Analytical Processing.** Few, large queries, each scanning vast numbers of rows to aggregate: “average notes per user per week for the last year.” It wants high *throughput* over huge scans, and it does not care about millisecond latency or row-level writes.

These wants conflict at the storage level. Operational databases store data **row by row**, because to serve “give me note 42” you want every column of that one row together. Analytical systems store data **column by column**, because to compute “average length of all notes” you want to read just the content column across millions of rows — and skip every other column entirely. **Columnar storage** also compresses far better (a column holds values of one kind) and lets the engine scan only the columns a query touches. The same data, laid out for the opposite access pattern.

It is hard to serve both access patterns well from one layout, which is the technical reason the planes separate. (Some “HTAP” databases keep both a row store and a column store; they are the exception.) The operational reason reinforces it: a runaway analytical query must never be able to starve the database your users depend on. Keep analytics off the production database — the usual first step is a **read replica**, a copy that analysts can query without touching the primary, and the next is a separate warehouse.

## 42.2 Where Analytical Data Lives

Three storage ideas, often combined:

- **Data warehouse** — a columnar analytical database (Snowflake, BigQuery, Amazon Redshift) holding *structured, cleaned* data modeled for querying. This is where analysts and dashboards run SQL. It is the polished, query-ready store.
- **Data lake** — cheap object storage (S3 and friends) holding *raw* data in its original form: structured, semi-structured, or unstructured (logs, JSON, events, images), kept because storage is cheap and you may want it later. It is *schema-on-read* — you impose structure when you query, not when you store.
- **Lakehouse** — the modern convergence: warehouse-style querying and management (tables, transactions, schemas) directly over lake storage, so you get the lake’s cheap scale and the warehouse’s query discipline without copying data between two systems.

The mental model: the **lake** is the cheap, raw, keep-everything tier; the **warehouse** is the cu-

rated, query-ready tier; the **lakehouse** tries to be both. Each is the analytical-plane counterpart to ideas you already met — the storage *tiers* of [Chapter 37](#) (hot/warm/cold), now organized around *use* rather than age.

## 42.3 Getting Data There: Pipelines

Operational data starts in the OLTP database (and in event streams, and in third-party APIs). Getting it into the analytical plane is the job of **data pipelines**, and two distinctions describe most of them.

**ETL vs ELT** — the order of two steps:

- **ETL** (Extract, *Transform*, Load) — pull the data out, clean and reshape it, *then* load the finished result into the warehouse. The classic approach when storage and compute were expensive.
- **ELT** (Extract, Load, *Transform*) — pull the data out, load it raw into the cheap warehouse/lake *first*, then transform it *in place* using the warehouse’s own power. The modern default, because cloud warehouses make in-place transformation cheap and flexible, and you keep the raw data for re-processing.

**Batch vs streaming** — the cadence:

- **Batch** — process data in scheduled chunks: every night, recompute yesterday’s metrics. Simple, efficient, and fine when “yesterday’s numbers this morning” is good enough. This is the cleanup-job and scheduled-work discipline of [Chapter 25](#), scaled up to data.
- **Streaming** — process events continuously as they arrive (often off a Kafka log from [Chapter 41](#)), so dashboards update within seconds. Necessary for real-time needs (fraud detection, live operations), heavier to build and operate.

A common, healthy architecture: stream raw events into the lake continuously, then run batch transformations to build curated warehouse tables. Most businesses need fresh-enough, not instant; reach for streaming only where latency genuinely earns its cost.

## 42.4 Modeling Data for Analysis

The schema-design discipline of [Chapter 6](#) still applies, but the goals invert. Operational schemas **normalize** to keep writes correct and anomaly-free. Analytical schemas deliberately **denormalize** to make reads simple and fast, because there are few writes and the priority is straightforward aggregation.

The dominant pattern is the **star schema**, built from two kinds of table:

- **Fact tables** — the measurable events, one row per occurrence: a `note_created` fact with a timestamp, a user reference, and measures like note length. Facts are numerous and append-only.
- **Dimension tables** — the descriptive context you slice facts *by*: a `users` dimension, a `dates` dimension, a `features` dimension. Dimensions are fewer and richer.

A query like “notes created per week by country” then becomes: scan the fact table, join to the date and user dimensions, group, aggregate. The shape — a central fact table surrounded by dimensions — is what gives the star schema its name, and it maps cleanly onto how people actually ask analytical questions: *some measure, sliced by some attributes, over some time range*.

## 42.5 Trust: Quality, Lineage, and Governance

An analytical plane is only as valuable as it is *trusted*, and at scale trust must be engineered, not assumed:

- **Data quality** — automated checks that catch nulls where there should be none, duplicates, impossible values, sudden volume drops. A dashboard quietly fed bad data is worse than no dashboard, because people *act* on it.
- **Lineage** — knowing where each number came from: which source tables and transformations produced this dashboard figure. When a metric looks wrong, lineage is how you trace it back to the cause.
- **Governance and privacy** — the data plane copies operational data, including PII, into more systems and more hands, which multiplies the obligations from [Chapter 37](#). Retention, access control, the right to be forgotten, and “what are we even allowed to compute on this data” all extend into the warehouse and lake. A user’s deletion request must reach the analytical plane too, or you have not honored it.

The throughline: the analytical plane is a *second* system with its own storage, pipelines, modeling, and governance — built so the business can learn from its data without ever endangering the operational system that produces it.

### Key Takeaways

- **OLTP** (small, fast, row-oriented, operational) and **OLAP** (large scans, columnar, analytical) are opposite workloads. One database rarely serves both well — **keep analytics off your production database** (a read replica first, a warehouse later).



- Analytical data lives in a **warehouse** (curated, query-ready, columnar), a **data lake** (cheap raw object storage, schema-on-read), or a **lakehouse** (both at once).
- **Pipelines** move data across: **ELT** (load raw first, transform in the warehouse) is the modern default; choose **batch** by default and **streaming** only where real-time latency truly pays.
- Model analytics with **star schemas** — append-only **fact** tables of events, surrounded by descriptive **dimension** tables — and denormalize for fast reads (the inverse of operational normalization).
- At scale, **data quality, lineage, and governance** are first-class: an untrusted or non-compliant data plane is a liability, and **PII obligations follow the data** into the warehouse.

## Exercises

1. **Predict.** Run a report query that scans all notes (counts per user per month) on a *staging* database while a load test runs against it. What happens to the API's latency?
2. **Break and fix.** Load one day of notes into a local DuckDB file (or a separate PostgreSQL schema) as a star schema: a fact table with one row per note created, and dimensions for user and date.
3. **Extend.** Write an ELT job, as a queue job run nightly, that copies yesterday's new notes into the fact table of exercise 2, and that is idempotent if it runs twice.
4. **Explain.** Why do analytical and operational workloads want different storage layouts?

Solutions and hints are in the companion repository, in `exercises/chapter-42.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What's Next

The transactional plane, the async plane, and the analytical plane all have to *run somewhere*. The final capstone returns to infrastructure — but at fleet scale, where the single VM of Stage 5 becomes a cluster managed by an orchestrator, and the operational ideas you already know are automated.

→ [Chapter 43](#) — Cloud-Native and Orchestration



# Cloud-Native and Orchestration

---

Earlier in this book you packaged the Notes API into a container, put it on a single Linux server, and had Docker’s restart policy — with systemd starting Docker — keep it alive. That is a complete, real production setup, and for one service it is often the right one. But the distributed, async, multi-plane systems of the last three chapters do not run on one box — they run as dozens or hundreds of containers across a fleet of machines. And the moment you have a fleet, a new question dominates: *who places all these containers, keeps the right number running, replaces the dead ones, routes traffic to them, and rolls out new versions — without a human watching?*

The answer is an **orchestrator**, and the surrounding philosophy is called **cloud-native**. The reassuring news, and the reason this chapter can be short, is that orchestration does not introduce new fundamentals. It *automates the operational work you already understand* — supervision, health checks, restarts, load balancing, scaling — across many machines. Kubernetes is, in one line, **systemd for a fleet**.

## In this chapter

By the end of this chapter, you will understand:

- What “cloud-native” means, and why containers at scale demand an orchestrator
- The core ideas of **Kubernetes**: desired state, the control loop, pods, deployments, services
- How orchestration **automates the Part-5/6 operational work** you already know
- **Infrastructure as Code** — managing infrastructure declaratively
- **Serverless / FaaS** and **managed services** — the opposite move, owning less
- The central trade-off: orchestration’s power versus its complexity, and when *not* to use it

### 43.1 From One Server to a Fleet

Stack up what changes when one container on one VM becomes fifty containers across ten machines. By hand, you would have to: decide which machine each container runs on (**scheduling**); notice when one crashes and start a replacement (**self-healing**); track which machines are healthy and have spare capacity; add or remove containers as load changes (**autoscaling**); discover where each container currently is so others can reach it (**service discovery**); spread traffic across them (**load balancing**); and roll out a new version without downtime and roll back if it misbehaves (**deployment**).

On one server, `systemd` and `Nginx` handled the small version of this. At fleet scale, doing it manually is impossible — there are too many moving parts changing too fast. An **orchestrator** does all of it, continuously and automatically. That is the entire reason the category exists.

“**Cloud-native**” is the design style built around this reality: applications packaged as containers, composed of small services, run on elastic infrastructure, and operated through automation rather than hand-tended servers. Treating servers as disposable, interchangeable capacity — **cattle, not pets** — is its defining attitude.

### 43.2 Kubernetes: Desired State and the Control Loop

**Kubernetes** (k8s) is the dominant orchestrator, and beneath its sprawl sits one elegant idea worth internalizing even if you never run it yourself: **declarative desired state reconciled by a control loop**.

You do not tell Kubernetes *how* to do things step by step. You declare *what you want* — “run 5 replicas of the Notes API, version 1.4.0, each with this much memory, reachable on this port” — and hand it that description. A **control loop** then runs forever, comparing the **desired state** to the **actual state** of the cluster and taking action to close any gap. A container died and only 4 are running? It starts a 5th. A whole machine failed? It reschedules that machine’s containers onto healthy ones. This is exactly the restart-policy instinct of [Chapters 24–25](#) (`--restart unless-stopped`, or `systemd`’s `Restart=always`) — “keep the desired thing running” — generalized from one process on one host to any number of containers across a cluster, and made self-healing by default.

A few core objects give the vocabulary:

- **Pod** — the smallest unit Kubernetes runs: one (or a few tightly-coupled) containers sharing an address. For us, a pod is one Notes API container.
- **Deployment** — a declaration of “keep N identical pods of this version running,” and the engine of **rolling updates**: change the version and Kubernetes replaces pods gradually,

watching health, ready to roll back. This is the blue-green deployment of [Chapter 23](#) grown up — rolling updates, and canaries that send a small share of traffic to the new version first — automated.

- **Service** — a stable address and load balancer in front of a changing set of pods. Pods come and go with their own IPs; the Service gives clients one durable name and spreads traffic across whatever pods are currently healthy. This is service discovery plus the reverse-proxy load balancing of [Chapter 28](#), built in.
- **Liveness and readiness probes** — Kubernetes periodically calls a health endpoint on each pod (restarting unhealthy ones, withholding traffic from not-yet-ready ones). These are *literally* the checks from [Chapter 35](#), now wired into the platform’s control loop: the shallow `/ping` as the liveness probe (restart only a process that does not answer), the deep `/health` as the readiness probe (stop sending traffic while the database is unreachable — restarting would not fix that).

Notice that every one of these is a concept you already met on the single server. Kubernetes did not invent supervision, health checks, rolling deploys, or load balancing — it **automated and generalized** them. That is why the substrate this book taught is the real prerequisite: orchestration is far less mysterious once you know what it is orchestrating.

### 43.3 Infrastructure as Code

A fleet needs more than running containers — it needs the machines, networks, load balancers, databases, and DNS underneath them. Clicking these together in a cloud console by hand is unrepeatable and undocumented: no one can say exactly what exists or recreate it after a disaster.

**Infrastructure as Code (IaC)** applies the disciplines of this whole book to infrastructure itself. With a tool like **Terraform**, you *declare* the infrastructure you want in version-controlled files — “a cluster of this size, a managed PostgreSQL of this tier, this network, these firewall rules” — and the tool makes reality match the declaration, the same desired-state pattern as Kubernetes, one level down. The payoff is enormous: infrastructure becomes reviewable in pull requests, reproducible across environments (the environment-parity goal of [Chapter 39](#), finally achievable), versioned with a changelog, and rebuildable from scratch. Servers stop being hand-tended pets and become a declaration you can apply anywhere.

### 43.4 The Other Direction: Serverless and Managed Services

Orchestration is about running more, at scale. The opposite move — increasingly popular — is to *own less*.

- **Managed services** push operational responsibility to the cloud provider. You met this in [Chapter 20](#) with managed PostgreSQL: the provider runs, patches, backs up, and fails over the database, and you just use it. The same exists for queues, caches, search, and Kubernetes itself (managed control planes). Every managed service is operational work you no longer do.
- **Serverless / Functions-as-a-Service** (AWS Lambda, Cloud Functions) takes it furthest: you deploy a *function*, and the platform runs it on demand, scales it from zero to thousands and back automatically, and bills only for actual execution. There is no server, container, or process for you to supervise — the platform owns all of it. The trade-offs are real (cold-start latency, execution limits, vendor lock-in, and a programming model bent around statelessness), but for spiky, event-driven, or glue work it removes nearly all the operational burden of the last twenty chapters.

These are not contradictions of orchestration but points on a spectrum of **how much of the stack you operate yourself** — from a hand-run VM, to an orchestrated cluster you manage, to managed services, to serverless where you manage almost nothing. Mature systems mix them deliberately, owning what is differentiating and renting what is not.

### 43.5 The Trade-off: Power vs Complexity

Kubernetes and its ecosystem are extraordinarily powerful, and extraordinarily complex. A real cluster drags in networking overlays, ingress controllers, secret management, persistent-volume plumbing, autoscalers, observability stacks, and a **service mesh** (a layer like Istio that handles service-to-service traffic, retries, and security between pods). Running it well is a full specialty.

So the most important judgment in this chapter is *when not to reach for it*. A single service with modest traffic does not need Kubernetes — the container-on-a-VM-with-systemd of Stage 6 is simpler, cheaper, and easier to reason about, and it is the right answer for most projects most of the time. The pull toward orchestration is real only when you genuinely have a fleet of services and the operational toil of running them by hand exceeds the (substantial) cost of the platform. Adopt cloud-native infrastructure when the scale demands it — not because it is fashionable, and not before the simpler thing has actually stopped working. The same instinct closes [Chapter 40](#) and this one: distribute, and orchestrate, only under proven pressure.

### Key Takeaways

- A **fleet** of containers needs an **orchestrator** to schedule, self-heal, scale, discover, load-balance, and deploy them automatically — work that is impossible to do by hand at scale.
- **Kubernetes** runs on one idea: **declarative desired state reconciled by a control loop** — you say what you want, it continuously makes reality match. Pods, Deployments, Services, and health **probes** are the single-server concepts of Stages 5–6 (supervision, rolling deploys, load balancing, /health) **automated across a cluster**.
- **Infrastructure as Code** (Terraform) declares infrastructure in version-controlled files — reviewable, reproducible, and rebuildable — bringing this book’s disciplines to the servers themselves.
- The opposite move is to **own less**: **managed services** and **serverless/FaaS** hand operational responsibility to the provider, trading control for freedom from toil.
- Orchestration’s power comes with heavy complexity. **Don’t reach for Kubernetes until a fleet genuinely demands it** — a container on a VM with systemd is the right answer for most projects.

## Exercises

1. **Predict.** In a local Kubernetes cluster (kind or minikube), run a Deployment with three replicas of the Notes API image, then delete one pod. What happens, and what did you not have to write?
2. **Break and fix.** Give the Deployment a readiness probe on a path that returns 404. What happens to traffic and to the rollout? Point it at /health instead.
3. **Extend.** Write the Terraform (or your provider’s CLI script) that creates the droplet, the reserved IP, and the firewall of Chapter 22, and destroy and recreate them from it.
4. **Explain.** Map each Kubernetes concept you met (Deployment, Service, probes, rolling update) to the part of this book that does the same job by hand.

Solutions and hints are in the companion repository, in `exercises/chapter-43.md`. Try each exercise before reading its solution: the point is the thinking, not the answer.

## What’s Next

That completes the journey — from a single command starting a process, all the way to orchestrated fleets, asynchronous flows, and data platforms. What remains is to put it to work: the next chapter is a project that takes the Notes API to a small real product, through every

stage of the book. After it come reference material to keep close as you build: setup steps, a glossary, a command cheat-sheet, a procedure for finding your feet in an unfamiliar system, and a one-sitting refresher of the whole book.

→ [Chapter 44](#) — Notes for Many Users: A Project for Part IV



# Notes for Many Users

*A Project for Part IV*

---

Reading about a system and building one are different skills. Reading gives you the map: you know what a migration runner is for, why a worker needs a lease, why a query must say `AND user_id = ?`. Building gives you what the map cannot: the fifteen small decisions nobody wrote down, the error that makes no sense until it suddenly does, and the confidence that comes from having shipped something yourself.

This chapter is a project. It turns the Notes API into a small real product, **Notes for Many Users**, in five milestones. Each one takes a trip around the whole loop from the opening chapter: decide, plan, build, ship, run, learn. Every milestone uses techniques from the book, and none needs a technique the book has not taught. What is new is that *you* make the decisions, and that the pieces have to work together.

By the end of this project, you will have:

- Designed, built, tested and shipped five features into a live system, each through a pull request and CI
- Extended a database schema safely with migrations, and an authorization model beyond “owner only”
- Put background work, scheduled work and an outside service behind a queue, with retries you can trust
- Kept a written API contract true, with a test suite you would stake a release on
- Run the result in production: deployed, monitored, alerted on, load-tested, deliberately broken, and written up

## 44.1 How to Work Through This Project

**Start from where the book ends.** The starting point is the Notes API as Chapter 43 leaves it: running on PostgreSQL, with migrations, tags, accounts, cursor pagination, the email endpoint,

the background worker, CI, and a deployment to a real server. If you have built along with the book, that is your code. If not, the companion repository's `start` tag has it.

**Go around the loop for every milestone.** Each milestone below gives you requirements and a “done when” list. It does not give you a design. That part is yours, and it is the point:

1. **Discover and plan.** Write a short problem statement and a design note (Parts I and II). A page is plenty: what changes in the schema, which endpoints, who may call them, what could go wrong. Record any decision someone might later question as an ADR ([Chapter 39](#)).
2. **Code.** Work on a branch ([Chapter 17](#), section 17.9), in small commits. Write the tests alongside the code, and for anything that is a rule (“a viewer cannot edit”), write the test first.
3. **Ship.** Add a migration for every schema change ([Chapter 33](#)). Update `openapi.yaml` for every API change (section 34.10). Open a pull request, let CI pass, merge, and let the pipeline deploy.
4. **Run and learn.** Watch the dashboards after the deploy. Note anything that surprised you. It belongs in the next milestone’s plan.

**Use the reference solution the right way.** The companion repository contains a complete, tested solution, tagged at the end of each milestone (`project-m1` to `project-m5`). Resist opening it until you have finished a milestone yourself. Then compare (section 44.8 explains how). There is always more than one good solution. The reference is *a* good one, with its reasons written down, not *the* answer.

**Estimate honestly.** Working evenings and weekends, most people need one to two weeks per milestone. Milestone 5 takes the longest, because production always does.

## 44.2 Milestone 1: Accounts and Ownership

If you built [Chapter 38](#) as you read it, this milestone is mostly *verification*: proving that what you built really holds. If you did not, build it now. Everything else rests on it.

### Requirements

- People register with an email address and a password, log in to receive a token, and log out.
- Every note belongs to the user who created it. Nobody else can read, change, delete, tag or email it.
- Passwords are stored only as bcrypt hashes; tokens only as SHA-256 hashes. Tokens expire after 30 days.
- Login attempts are rate-limited per IP address in Nginx.

**Done when**

- ☐ For *every* endpoint that takes a note id, a test logs in as a second user and gets 404.
- ☐ Removing any single AND `user_id = ?` from the code makes at least one test fail. Try it for each one; this is **mutation testing** done by hand, and it is how you know the tests guard the rule.
- ☐ A wrong password and an unknown email produce the same response, in the same time.
- ☐ `curl` without a token gets 401 with `WWW-Authenticate: Bearer`, from the live server.
- ☐ Seven quick login attempts from one address: the first six reach the application (and get 401 for a wrong password), and the seventh gets 429 from Nginx.

**44.3 Milestone 2: Sharing, Search, and the Trash**

Three features that users ask for as soon as a product has more than one user.

**Requirements: sharing**

- The owner of a note can share it with another registered user, either read-only or with permission to edit, and can change or remove that access.
- A user sees the notes shared with them in a list of their own, apart from their own notes.
- Access is decided like this:

Action	Owner	Editor (shared, can edit)	Viewer (shared, read-only)	Anyone else
read the note and its tags	yes	yes	yes	404
change its content	yes	yes	403	404
delete, share, tag, email, set reminders	yes	404	404	404

(Why 403 for a viewer who tries to edit, when everyone else gets 404? Because the viewer already knows the note exists: it was shared with them. Hiding its existence protects nothing, and a clear “read-only” message helps. Section 38.4 drew exactly this line.)

**Requirements: search**

- GET `/notes/?search=words` finds the user’s own notes containing those words, as *words*: `drafts` finds “draft”, while `budg` finds nothing. The results use the same cursor pagination as the plain list.

- Search stays fast with a million notes.

### Requirements: the trash

- Deleting a note moves it to a trash, where it stays restorable for 30 days. Deleted notes vanish from every other endpoint.
- Notes in the trash for more than 30 days are deleted for good, by a background job that runs daily.

### Hints

- *One place decides access.* Write a single function that answers “what is this user’s role for this note?” (owner, editor, viewer, or none), and have every handler use it. Scattered checks are how authorization bugs are born.
- *Search.* A **generated column** keeps itself up to date: a `tsvector` column computed from `to_tsvector('english', content)`, declared `GENERATED ALWAYS AS (...) STORED`. Give it a GIN index, and query it with `websearch_to_tsquery`, which understands what people type into search boxes. Then *look* at the plans with `EXPLAIN ANALYZE` (section 36.2), and expect a surprise. With `ORDER BY id DESC LIMIT 21` and a common word, PostgreSQL often ignores the GIN index and walks the primary key or the per-user index instead, because finding 21 matches that way is cheaper. It uses the GIN index for rare words. That is the planner working correctly, and it only shows up with realistic data.
- *The trash* is the “soft delete” of [Chapter 6](#). Remember that *every* query must now exclude deleted notes, including the ones in the worker’s tasks. The purge job is a queue job ([Chapter 41](#)), queued daily by a systemd timer ([Chapter 25](#)).
- *Migrations.* All of this is additive. The new table and the new columns are safe for the running code, so the migration can run before the deploy as usual.

### Done when

- ☐ The access table above is a parametrized test, with every cell tested.
- ☐ With 100,000 generated notes spread over 1,000 users (one of them owning 20,000), searches for a common and a rare word, as a light and as a heavy user, all stay under 20 ms. For each of the four, you can say which index PostgreSQL chose, and why. (In the reference setup: about 0.2 ms for a light user, 2 ms for a rare word, and 7 to 9 ms for a common word for the heavy user.)
- ☐ A note deleted, then restored, is exactly as it was: its tags, its shares and its content.
- ☐ A test proves that the worker does not email a note that is in the trash.
- ☐ `openapi.yaml` describes every new endpoint, and the contract tests check them.

## 44.4 Milestone 3: Reminders

### Requirements

- The owner of a note can ask to be emailed about it at a chosen time: `PUT /notes/{id}/reminder` with a date and time. It must include a time zone, lie in the future, and be within a year.
- Setting a new time replaces the old one. The reminder can be cancelled.
- The email goes to the owner's own address, at the chosen time (give or take the worker's polling interval), exactly once.

### Hints

- A reminder is a queue job with `run_at` set to the chosen time (section 41.5). The hard part is *changing your mind*: the old job is already in the queue. You can delete it, or you can let it run and have it check whether it is still wanted (store the reminder time on the note, and let the task compare). The second design needs no bookkeeping, and it survives races, such as a cancellation arriving while the job is running. The reference solution uses it.
- Choose the idempotency key when the reminder is *set*, not when it is sent. A worker that crashes after sending, and a job that is retried after a timeout, must not send a second email.
- Time zones: store and compare in UTC (`TIMESTAMPZ`), and reject a time without a time zone. `"2030-01-01T09:00"`, with no zone, means something different on every server.

### Done when

- ☐ Tests cover: a reminder is not ready before its time; at its time the owner is emailed; a cancelled or replaced reminder sends nothing; bad times are rejected.
- ☐ Against Mailpit, a reminder set two minutes ahead arrives within about a minute of its time.
- ☐ Killing the worker (`docker kill`, not `docker stop`) while it sends a reminder leads to exactly one email once the lease has expired. That depends on an email service that honours idempotency keys. Mailpit does not, so check the *requests* instead: both carry the same key.

## 44.5 Milestone 4: A Suite and a Contract You Can Trust

By now the test suite has grown chapter by chapter. This milestone makes it something you would stake a release on.

## Requirements

- Every endpoint has tests for its normal case, bad input, missing or foreign notes, and its design decisions (section 17.8).
- `openapi.yaml` describes the whole API, including the login requirement (the `bearerAuth` security scheme), and passes `openapi-spec-validator`.
- Contract tests check the response of every endpoint against its schema.
- The suite runs in CI against PostgreSQL (Chapter 39), in under a minute, with no flaky tests.

## Done when

- ☐ Coverage is above 90% for `app/`, and every uncovered line is one you have looked at and decided about.
- ☐ Ten runs in a row in random order (`pytest -p randomly`, from the `pytest-randomly` plugin) all pass.
- ☐ Renaming any response field makes a contract test fail.
- ☐ A new team member could learn the API from the rendered `openapi.yaml` alone. Test this with a friend if you can.

## 44.6 Milestone 5: Production

Everything so far could pass on a laptop. This milestone is about the system in the world.

### Requirements

- **Deploy both processes.** A merge to `main` deploys both the API and the worker, runs migrations first, and rolls the API back automatically if its health check fails (Chapters 23 and 41).
- **Measure.** Prometheus collects the API's metrics (Chapter 35) and the worker's (section 41.5).
- **Alert.** Alerts fire for: an error rate above 1% for five minutes; p95 latency above 500 ms; the oldest queued job waiting more than five minutes; any job that failed permanently; the certificate expiring within 14 days.
- **Write a runbook** (Chapter 37) for each alert.
- **Load-test** with Locust (section 36.1), against a staging copy: a realistic mix of listing, reading, creating, searching and sharing, by logged-in users.
- **Hold a game day.** Break the system on purpose, in staging, while watching the dashboards: stop the worker, then start it again; stop the database for a minute; point the worker at an

email address that does not exist; fill the queue with 10,000 jobs.

- **Write a postmortem** for the most interesting failure, using the template of section 37.10.

### Done when

- ☐ A one-line change goes from pull request to production with no manual step except the review.
- ☐ Every alert has fired at least once during the game day, and its runbook led to the fix.
- ☐ The load test states its result as a number with conditions: “sustains  $N$  requests a second at p95 under  $M$  ms on one 2 GB droplet”. It also names the bottleneck, found with the method of section 36.2.
- ☐ The postmortem has a timeline, a root cause, and action items with owners, and at least one of those action items has been done.

## 44.7 Stretch Goals

For when the five milestones are done and you want more. Each one exercises a different part of the map:

- **Limit emails per user** (say, 20 an hour), in the application, with a counter per user and time window. This closes the spam hole that section 37.5 warned about.
- **Password reset**, exactly as section 38.3 outlines it: single-use, hashed, short-lived tokens, sent by the worker, and all sessions revoked afterwards.
- **API keys** for scripts, created and revoked by a logged-in user and shown only once. They are stored and checked like login tokens, but never expire.
- **An export**: POST /exports queues a job that writes all of a user’s notes to a JSON file and emails a link. It raises the question of where large files belong (object storage, not the database).
- **An admin role**, with an endpoint that shows the queue and lets a person re-queue failed jobs, all behind `require_role("admin")`.

## 44.8 Comparing Your Solution with the Reference

When a milestone is done, check out its tag in the companion repository and compare. Do not compare line by line. Compare by *decisions*, with these questions:

- **Where does each rule live?** In the reference, access is decided in one function, `_access()`, that returns a role; ownership of tags, shares, email and reminders is `role == "owner"`.

If your rules are spread across handlers, count the places a future change would have to touch.

- **What does each failure say?** For every error your API can return, is the status code the most honest one available? Check 403 against 404 for viewers, 409 for a duplicate email, and 202 for queued work.
- **What happens twice?** List every operation that could run twice: a retried request, a redelivered job, a double-clicked button. For each one, what makes the second run harmless?
- **What happens halfway?** For every handler that makes more than one change, what does the database look like if the process dies between them?
- **What would you see at 3 a.m.?** For each feature, which metric or log line would tell you it had broken, before a user did?

The reference solution also records a few choices that are easy to miss:

- **The reminder checks itself.** The task compares the note's `remind_at` with the time in its own payload, and does nothing if they differ. Cancelling or rescheduling therefore needs no access to the queue at all.
- **Search is a generated column.** PostgreSQL keeps search up to date on every insert and update, so no application code can forget to.
- **Restoring from the trash takes an empty JSON body, {}.** The middleware from [Chapter 16](#) requires every POST to declare a JSON body (`Content-Type: application/json`), and a request that declares JSON should carry some. The reference keeps that rule simple and asks clients for {}, rather than adding exceptions to it. (Logout avoided the question by being a DELETE.) Both are defensible; this one is written in `openapi.yaml`, so no client has to guess.
- **A deleted note keeps its tags and shares until it is purged.** That is what makes restoring exact.

### Key Takeaways

- A project turns the map into skill: the decisions the book could only describe, you now make, and see the consequences of.
- Go around the whole loop for every feature: a short plan, a branch, tests (first, for rules), a migration, a contract update, CI, a deploy, and a look at the dashboards.
- Put each rule in one place, and prove it with tests that fail when it is broken. Mutation testing by hand, “does removing this line fail a test?”, is a cheap way to know.
- Design for twice and for halfway: every retry, redelivery and crash is a question your code



must already have answered.

- Production is a milestone of its own. Deploys, alerts, runbooks, load tests and game days are what turn working code into a working system.

## What's Next

The project is the last step of Part IV. The closing chapter steps back from the code, and replays the whole journey of this book as a team would take it. After that come the reference appendices.



# The Same Process in a Real Team

*What you did alone, as a team does it — and the same map laid over any kind of software*

---

You have now been around the loop from [How Software Gets Built](#) once, from start to finish, entirely on your own. Every job was yours. You decided what to build and wrote the plan. You wrote the code and its tests, packaged it, rented a machine, deployed to it and gave it a name. Then you watched it, secured it, scaled it, and saw how it would grow into many services. That was deliberate: doing every job once is the only way to see how the jobs connect.

Real software is rarely built by one person. This chapter replays the same journey with a team. It shows who does each step, what they hand to each other, and where things go wrong between them. Then it lays the same map over other kinds of software, to show what stays the same when the technology changes. You will not meet many new techniques here. You will see the techniques you already know from a new angle: as the way a group of people builds one thing together.

## The Team

Imagine a small company that has grown the Notes API into a product. People use it through a website and a phone app, and both talk to the API. The people who build it:

Role	How many	What they own
Product manager	1	what gets built, and why; what comes first
Designer	1	what users see and do, on the website and the app
Tech lead	1	the design of the whole system; also writes backend code
Backend engineers	2	the Notes API, its database and its background workers
Frontend engineer	1	the website
Mobile engineer	1	the phone app
Platform engineer	shared	the road to production: automatic checks, packaging, servers, deployment
Site reliability engineer	shared	reliability; leading the response when something breaks
Security engineer	shared	reviews of anything that touches access, data or secrets
Engineering manager	1	the people, their priorities, and how the team works

Eight people, five of whom write code for this product, plus three specialists shared with the rest of the company. It is a very ordinary team. Every job you did alone in this book belongs to at least one of them.

## The First Version, as a Team

Here is the first trip around the loop, as this team would make it.

**Discover.** The product manager spends two weeks with people who take notes for their work. The aim is to learn what they do today, what annoys them, and what they would pay for. The designer sketches screens on paper and tries them with five of those people. The tech lead joins a few of the conversations to spot, early, the ideas that would be very expensive to build. The result is a one-page **problem statement**. It says who has the problem, what “solved” means in measurable terms, and what is out of scope for the first version (no sharing, no attachments). The whole team reads it and questions it before anyone plans anything. This is **Part I**, done by several people instead of one.

**Plan.** The tech lead writes the **design document** (**Part II**). It names the pieces: the API, its database, the website and the app. It describes the data and how the service will be run. Most importantly, it defines the **API contract** early: the exact requests the API accepts and the

answers it gives. Three groups will build against that contract at the same time — backend, web and mobile — so it has to be settled first. [Chapter 34](#) (section 34.10) explains why a contract is so hard to change once others depend on it. Big decisions go into **decision records** ([Chapter 39](#)), such as which database to use and why.

Before any code is written, others review the design. The platform engineer asks whether the service can be run on the company’s usual setup. The security engineer asks what could go wrong: who might attack it, and what they would want ([Chapter 38](#)). Finally, the work is cut into small pieces, often called **tickets**. Each takes a few days and has a clear definition of “done”.

**Code.** Each backend engineer takes a ticket. They work on a separate copy of the code (a **branch**) and open a **pull request**, a proposed change that others can see. The automatic checks run the tests and code-quality tools on it ([Chapter 17](#)). Another engineer reads the change and approves it or asks for changes. Only then is it merged into the main code. The shared conventions and review habits of [Part III](#) are what let several people change one codebase without stepping on each other.

Meanwhile, the frontend and mobile engineers build their screens against the agreed contract. Until the real API exists, they use a stand-in that returns realistic answers. Because the contract was settled first, nobody waits for anybody.

**Ship.** In a company this size, nobody builds the road to production from scratch for each service. The platform engineer built it once, and every team uses it: the automatic pipeline, the container image, the package store and the deployment scripts. These are the things you built yourself in Stages 4 to 6.

The backend team’s job is to keep its service *shippable*: dependencies pinned, configuration outside the code, a health check that tells the truth. Each change goes first to **staging**, a copy of production with no real users, and then to production. The public name, the certificates and the front door (Stage 7) are usually shared, and the platform team runs them for everyone.

The website ships the moment it is ready. The phone app ships differently. It goes through the app store’s review, which can take days, and reaches users only as they update.

**Launch.** The first version goes to a small group of users first. The team measures it against the success criteria in the problem statement. It is not “done” when the code is merged. It is done when users have the problem solved.

**Run.** Each team runs what it builds. The backend team owns the dashboards and alerts for the API ([Chapter 35](#)). One backend engineer at a time is **on call** for a week and is woken if an alert fires. When something breaks, the on-call engineer’s first job is to stop the harm, not to find the perfect fix. For a serious incident, the SRE joins and coordinates. The team keeps a **runbook** for each alert: step-by-step instructions written in daylight for the person reading

them at 3 a.m. After every serious incident comes a blameless **postmortem**. Both are in [Chapter 37](#).

**Learn.** The product manager reads the usage numbers and the support messages. Follow-up work from the postmortems becomes tickets. Requests from users go into the next round of discovery, and the loop begins again.

## Where Hand-offs Break

Most problems in a team come from the gaps *between* steps and *between* people, not from inside them. One person can do each step well and the team can still fail. The problem statement said one thing and the design assumed another, or the code worked on the author’s laptop and nowhere else.

The table below lists the seams you have seen in this book. For each one it gives the failure that happens there and the practice that protects it.

Seam	What goes wrong there	The practice that protects it	Where in this book
Discover → Plan	a well-built answer to the wrong question	a written problem statement with success criteria and scope	Part I; Part II, Section 1
Plan → Code	engineers build different ideas; the pieces do not fit	a design document; the API contract agreed before building	Part II; section 34.10
Engineer → engineer	code only its author understands	code review, shared conventions, tests	Part III; Chapter 17
Laptop → server	“it works on my machine”	pinned dependencies, configuration outside the code, containers	Stages 2–4
Build → release	what runs in production is not what was tested	one versioned package, promoted unchanged	Chapters 18, 19 and 23
Ship → Run	the system is handed to people who cannot see inside it	health checks, logs, metrics, runbooks	Chapters 25, 35 and 37
Backend ↔ clients	a change breaks the website, or old copies of the app	a versioned contract; changes that only add	section 34.10
Today → next year	nobody remembers why it was built this way	decision records and a changelog	Chapter 39

Seam	What goes wrong there	The practice that protects it	Where in this book
Run → Learn	the same failure happens again	blameless postmortems, with follow-up work that gets done	Chapter 37

This table is the real reason the book is full of *practices*. Each one exists because some hand-off failed often enough, at enough companies, that people found a way to protect it. When a practice feels like bureaucracy, find the hand-off it protects. Then decide whether your team actually has that hand-off. A practice without a hand-off to protect usually *is* bureaucracy.

## A Change to a Live System

After launch, almost all of a team’s work is changing the live system. Here is one change, followed through the loop: **letting a user share a note with a colleague**.

**Day 1 — Learn → Discover.** Many users have asked for sharing, and the support messages show it. The product manager writes a short problem statement: who needs it, why, and what “done” looks like. Editing a shared note is out of scope, and so is sharing with people outside the company.

**Day 2 — Plan.** The designer sketches the “share” flow. The tech lead writes a one-page design with three parts:

- a new database table recording which notes are shared with whom;
- two new API requests, one to share a note and one to list what has been shared with you;
- a change to the rule for who may read a note.

The security engineer reviews it within the hour, because a shared note must never be visible to anyone it was not shared with (Chapter 38). The web and mobile engineers agree the additions to the contract.

**Days 3–6 — Code and Ship, in small steps.** The backend engineer does the work in stages:

1. First, a database change that only *adds* a table, so the code already running is not affected (Chapter 33).
2. Then the new requests, with their tests. One test checks that a user who was not given access gets “not found”, exactly as if the note did not exist.
3. Each step goes in as its own small pull request, which is reviewed, checked, merged and released on its own.

The new feature reaches production switched off. It is hidden behind a setting called a **feature**

**flag**, so code can be released before users see it.

**Days 4–8 — the clients.** The web and mobile engineers build the screens against the agreed contract. The website change goes live the day it is finished. The app update goes to store review and reaches users over the next few weeks. Meanwhile, older copies of the app know nothing about sharing, and they must keep working. That is why the contract only gained new requests, and nothing existing changed.

**Day 9 — Run.** The flag is turned on for one user in twenty. The team watches the dashboards for errors and slow requests. Two days later, it is turned on for everyone.

**The weeks after — Learn.** The product manager sees how many people share notes. Support notices a new request: people want colleagues to be able to *edit* shared notes. That becomes the next trip around the loop.

Look at what happened. The change went through every step of the loop and involved seven people, yet no single step was big. That is what a healthy change loop looks like. The steps are small and safe, the hand-offs are clear, and the system is never broken in between.

## The Rituals, and What They Are For

Teams have regular meetings and habits. From outside they can look like overhead. Each one, though, serves a hand-off or the Learn arrow:



Ritual	How often	What it is for on the map
Planning	every week or two	choosing which pieces of work come next (Plan → Code)
Stand-up	daily, a few minutes	finding out who is stuck (engineer ↔ engineer)
Design review	before any large change	checking a design before code exists (Plan → Code)
Code review	every change	a second pair of eyes on everything (engineer → engineer)
Release	many times a day, ideally	a change reaching users (Code → Ship)
On-call hand-over	weekly	telling the next person what is fragile (Run → Run)
Postmortem	after every serious incident	turning a failure into lessons (Run → Learn)
Retrospective	every few weeks	looking at how the team works, not what it built (Learn, applied to the team itself)

This book does not teach how to run these meetings well; that belongs to books about teams and process. You do now know where each one sits on the map, and what goes wrong when it is skipped.

## The Same Map, for Any Application

The Notes API is a backend service. Here is the same loop for four other common kinds of software:

Step	Website	Phone app	Data pipeline	Library or installed software
Discover	users and their problem	the same, plus where and when people use their phones	the “users” are the people who need the numbers	the users are other developers
Plan	screens and flows, as well as the API	also: working offline, small screens, slow networks	where the data comes from, how often, how it is checked	the public interface, the hardest part to change later
Code	code for the browser and the backend	code for the phone and the backend	the steps that clean and combine data, and checks on the data itself	the library itself, its tests and examples
Ship	every user gets the new version at once	store review; users update slowly	each scheduled run is, in effect, a release	publish a numbered release
Run	everything in this book, plus errors in users’ browsers	crash reports from devices you cannot reach	“is the data fresh and correct?”	you do not run it; bug reports and support
Learn	how people use the pages	store reviews and crash rates	are the numbers used, and trusted?	issues, questions, downloads

What never changes is even more useful than what does:

- **The loop.** Every kind of software is discovered, planned, coded, shipped, run and learned from.
- **The workshop.** A code repository, automatic checks, versioned releases, monitoring and written decisions appear everywhere, under different names.
- **Contracts between pieces.** Wherever two teams or two programs meet, there is an agreement about what one gives the other, and changing it safely follows the same rules.
- **Observing what is real.** Every team needs to see what its software is actually doing in the hands of real users.

That is why what you learned from one small API carries over to software you have never seen.

## Where Specialists Sit

Every specialty in software is a close-up of one region of the map:

Specialty	Its region of the map
Frontend and mobile	the client, in depth
Backend	the program, the data and the helpers, plus the contract every client depends on
Platform and DevOps	the workshop, and Ship
Site reliability	Run
Data	the data, and Learn
Security	a thread through every step and every piece
Product and design	Discover and Plan

Specialists go deep, and they should. The map is what lets them talk to one another. The people who end up leading technical work are usually those who carry the whole map, and who can talk with each role about its own region. Leading technical work is mostly a matter of managing the seams.

Backend is an especially good place to stand while you learn the whole map. The program sits in the middle of all the pieces. Its contract is where the clients, the data and the platform meet, and its failures show up in every other role's work.

## Using the Map from Now On

The map is not only for reading this book. Use it at work, in four situations:

- **Joining a team.** First place the team on the map. Which regions does it own? What does it receive, and from whom? What does it hand over, and to whom? Then use the questions in [Appendix E](#) to learn the system itself.
- **Meeting a new tool or practice.** Ask the four questions from the opening chapter: when, where, who and why. A tool you cannot place is a tool you do not yet understand.
- **In a meeting that goes nowhere.** Ask which step of the loop the discussion belongs to. Many unproductive meetings are two people in different steps talking past each other. For example, one is still deciding *whether* to build something (Discover) while the other is arguing about *how* (Plan).
- **Learning something new.** Go deep on one region at a time, and keep the map in view while

you do. Depth attached to the map becomes understanding. Depth without it becomes another pile of disconnected facts, and that pile is where this book began.

## Where to Go Next

This book went deep on one region of the map, the backend, and walked the whole loop around it. Here is where to go from here. The directions are ordered by how directly they build on what you now have. Each one comes with a first project built on the Notes API, because extending a system you understand teaches faster than starting from nothing.

**Go deeper where you stand.** Most backend work is data work. Learn how PostgreSQL stores, indexes and plans (*Use The Index, Luke*, then *Designing Data-Intensive Applications*, both in [Appendix D](#)). *First project*: load a million notes, find the three slowest queries with `pg_stat_statements`, and make each one ten times faster, measuring before and after (section 36.2).

**Own the client.** Every API exists for its clients, and building one shows you your API from the other side: the error you thought was clear, the pagination that is awkward to use. Learn enough HTML, CSS and JavaScript to write a small web page that calls an API (the MDN Web Docs are the reference). *First project*: a one-page client for the Notes API, with login, a list, and search. You will meet CORS, where to keep a token in a browser, and why `openapi.yaml` matters to the people on the other side.

**Hand over the workshop.** You ran everything yourself on one server. The next step is to let a platform run parts of it. Chapter 43 introduced both halves. **Kubernetes** runs containers across a fleet from a declaration of what you want, and **Terraform** declares the machines, networks and databases themselves. *First project*: run the Notes API and its worker in a local Kubernetes cluster (kind or minikube), with the migration as a Job that runs before the Deployment. Then write the Terraform for the server and firewall of Chapters 20 to 22, and destroy and recreate them.

**Use a cloud provider's managed pieces.** Replace one piece you ran by hand with a managed service: a managed PostgreSQL, with backups and failover you no longer have to operate; object storage for files; a managed queue. Each one trades control and money for time. *First project*: move the database to a managed PostgreSQL, and write down exactly what you no longer have to do, and what you can no longer do.

**Split a service, only when there is a reason.** Microservices solve problems of *teams* and of scale, and they add every problem of Chapter 40. Learn them by doing one split for a real reason. *First project*: make the email sender a separate service with its own small API and its own deployment, and list everything that got harder: tracing a request, deploying the two

together, testing across the boundary.

**Strengthen the foundations under all of it.** The book touched algorithms, operating systems and networks wherever the Notes API needed them. Depth there pays off everywhere, and especially in interviews and in the hardest production problems. For algorithms and data structures, *Grokking Algorithms* (Bhargava) is a gentle start and *The Algorithm Design Manual* (Skiena) goes much further. For operating systems and networks, see *The Linux Programming Interface* and *Computer Networking: A Top-Down Approach*, also in Appendix D.

Whichever direction you choose, keep going around the loop: decide what to build and why, plan it, build it in small steps with tests, ship it, run it and watch it, and learn from what you see. The tools will change many times during your career. The loop will not.



# Appendix A — Python Environment Setup

---

This appendix provides a complete guide to setting up a Python development environment from scratch, for readers who are new to Python or need to set up a fresh machine.

## A.1 Installing Python

### macOS

The recommended approach is using **pyenv**, a tool that manages multiple Python versions side by side.

```
Install Homebrew (macOS package manager) if not already installed
/bin/bash -c "$(curl -fsSL
↳ https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

Install pyenv, plus the libraries Python is built against (without
them, pyenv install fails or produces a Python missing ssl or sqlite3)
brew install pyenv openssl readline sqlite3 xz zlib tcl-tk

Add pyenv to your shell (bash)
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bash_profile
echo 'export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bash_profile
echo 'eval "$(pyenv init -)"' >> ~/.bash_profile
source ~/.bash_profile

For zsh (the default shell on modern macOS):
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.zshrc
echo 'export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.zshrc
echo 'eval "$(pyenv init -)"' >> ~/.zshrc
source ~/.zshrc
```

```

Install Python 3.11 (the version used throughout this book)
pyenv install 3.11.8

Set it as the default Python version
pyenv global 3.11.8

Verify the installation
python --version
Python 3.11.8

```

## Ubuntu / Debian Linux

```

Install required build dependencies for compiling Python from source
sudo apt-get update
sudo apt-get install -y \
 build-essential \
 libssl-dev \
 zlib1g-dev \
 libbz2-dev \
 libreadline-dev \
 libsqlite3-dev \
 libncursesw5-dev \
 xz-utils \
 tk-dev \
 libffi-dev \
 liblzma-dev

Install pyenv
curl https://pyenv.run | bash

Add to ~/.bashrc
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
echo 'export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(pyenv init -)"' >> ~/.bashrc
source ~/.bashrc

Install Python
pyenv install 3.11.8
pyenv global 3.11.8
python --version

```

## Windows (WSL2 recommended)



The smoothest development experience on Windows is Windows Subsystem for Linux (WSL2), which gives you a full Ubuntu environment:

```
In PowerShell (run as Administrator):
wsl --install
Restart when prompted

After restart, WSL2 opens an Ubuntu terminal
Follow the Ubuntu setup instructions above
```

## A.2 Virtual Environments

A **virtual environment** is an isolated Python environment with its own installed packages, separate from the system Python. Always use a virtual environment for each project to prevent dependency conflicts.

```
Create a virtual environment in the project directory
(named venv, as in Chapter 6)
cd notes-api
python -m venv venv

Activate the virtual environment
source venv/bin/activate # Linux / macOS

After activation, the shell prompt shows (venv):
(venv) user@machine:~/notes-api$

Install project dependencies
pip install -r requirements.txt

Verify installation
pip list

Deactivate when done
deactivate
```

(On Windows, in PowerShell, the activation command is `venv\Scripts\activate`.)

The virtual environment creates a directory `venv/` in the project folder containing:

- A link to the Python interpreter (a copy, on Windows)
- A `site-packages/` directory for installed packages

- Scripts for activation and deactivation

Add it to `.gitignore` — never commit virtual environments to git.

```
.gitignore
venv/
.venv/
__pycache__/
*.pyc
.env
```

## A.3 pip: Installing Packages

pip is Python’s package installer. It downloads packages from **PyPI** (the Python Package Index, at [pypi.org](https://pypi.org)).

```
Install a single package
pip install flask

Install a specific version
pip install flask==3.0.0

Install from a requirements file
pip install -r requirements.txt

Upgrade a package
pip install --upgrade flask

Uninstall a package
pip uninstall flask

List installed packages
pip list

Show info about an installed package
pip show flask

Save current environment to requirements.txt — only in an environment
that holds nothing but the production dependencies (Chapter 17)
pip freeze > requirements.txt
```

**Requirements files.** The Notes API’s two files, as the chapters built them (Chapters 12, 17, and

18):

```
requirements.txt – production dependencies
blinker==1.7.0
click==8.1.7
Flask==3.0.2
gunicorn==22.0.0
itsdangerous==2.1.2
Jinja2==3.1.3
MarkupSafe==2.1.5
packaging==24.0
psycpg2-binary==2.9.9
python-dotenv==1.0.1
Werkzeug==3.0.1
```

```
requirements-dev.txt – development and test dependencies
Everything the application itself needs
-r requirements.txt
pytest==8.1.1
pytest-cov==5.0.0
black==24.3.0
ruff==0.3.5
mypy==1.9.0
pip-audit==2.7.3
```

(Later chapters add optional packages when you adopt their features: `prometheus-flask-exporter` in Chapter 35, `redis` in Chapter 36, `bcrypt` in Chapter 38 — pin each as you add it. And `pip-audit` will report that Werkzeug 3.0.1 has a known vulnerability, as Chapter 17 showed: upgrading pins is routine maintenance.)

## A.4 The Project Structure

The Notes API uses this directory structure:

```
notes-api/
├── .github/
│ └── workflows/
│ └── build.yml # CI: test, build, smoke-test, publish, deploy
├── app/
│ ├── __init__.py # create_app() factory
│ ├── auth.py # Passwords, login tokens, protect() (Chapter 38)
│ └── config.py # All environment variables, read and validated
```

```

├── email.py # The only module that talks to the email service
├── jobs.py # The job queue: enqueue, claim, retry (Chapter 41)
├── database.py # open_db(), get_db(), close_db(), sql(), for_update(), row_to_note()
├── errors.py # JSON error handlers (the handling boundary)
├── logging_config.py # Logging configuration
├── middleware.py # Before/after request hooks
├── migrate.py # The migration runner (python -m app.migrate)
├── startup.py # Startup checks
├── tasks.py # What each kind of job does
├── validation.py # The Validator (Chapter 31)
├── worker.py # The worker process (python -m app.worker)
├── routes/
│ ├── __init__.py
│ ├── auth.py # /auth/register, /auth/login, /auth/session
│ ├── notes.py # /notes/ endpoints, including a note's tags
│ ├── tags.py # /tags/<name>/notes
│ └── health.py # /health and /ping
├── migrations/ # The schema, as numbered changes (Chapter 33)
│ ├── postgresql/ # 0001_create_notes.sql, 0002_add_updated_at.sql, 0003_add_tags.sql, ...
│ └── sqlite/ # the same, in SQLite's dialect (removed in Chapter 39)
├── tests/
│ ├── __init__.py
│ ├── conftest.py # Pytest fixtures (including logged-in clients)
│ ├── test_auth.py # Register, login, logout, tokens
│ ├── test_authorization.py # One user can never reach another's notes
│ ├── test_email_client.py # app/email.py against a misbehaving local server
│ ├── test_email_note.py # POST /notes/<id>/email queues a job
│ ├── test_worker.py # The worker: retries, leases, SKIP LOCKED
│ ├── test_errors.py # Error handler tests
│ ├── test_health.py # Health check tests
│ ├── test_migrations.py # Migration runner tests
│ ├── test_notes.py # Note CRUD tests
│ └── test_tags.py # Tags: joins, constraints, cascades
├── .dockerignore # Files kept out of the image
├── .env.example # Environment variable documentation
├── .gitignore
├── Dockerfile # Container definition
├── docker-compose.yml # Local development stack
├── gunicorn.conf.py # Gunicorn configuration
├── Makefile # Build automation
├── pyproject.toml # Tool configuration (pytest, black, ruff, mypy)
├── requirements.txt # Production dependencies
├── requirements-dev.txt # Development dependencies
└── run.py # Development-server entry point

```

(The migrations are the only definition of the schema: a new database is built by running all of them with `python -m app.migrate`, and the application refuses to start while any is pending.)

## A.5 Common pip Issues and Fixes

### **pip: command not found**

Usually one of two things. On many systems only `pip3` exists, or `pip` belongs to a different

Python than the one you run — `python3 -m pip` always uses the right one. Or the virtual environment is not activated:

```
source venv/bin/activate
```

(On recent Ubuntu and macOS Python installations, installing into the *system* Python is refused with “externally-managed-environment” — PEP 668. That is the system telling you to use a virtual environment.)

#### Package installation fails with compilation error

The package requires a system library that is not installed. For `psycopg2`:

```
Ubuntu/Debian: a compiler, Python's headers, and PostgreSQL's client library
sudo apt-get install build-essential libpq-dev python3-dev

macOS: PostgreSQL's client library
brew install libpq
```

Or use the binary wheel (no compilation needed):

```
pip install psycopg2-binary # Pre-compiled, includes libpq
```

#### ModuleNotFoundError even though the package is installed

The package was installed for a different Python than the one you are running — typically, you are outside the virtual environment. Check, and install with the interpreter you actually use:

```
which python # Should show: ../notes-api/venv/bin/python
python -m pip install -r requirements.txt
```



# Appendix B — Glossary

---

This glossary defines the technical terms introduced throughout the book, in alphabetical order.

**Accept queue:** The kernel data structure that holds fully-established TCP connections waiting to be retrieved by the application via `accept()`. If it fills, Linux silently drops new connection attempts, and clients see slow connects or timeouts.

**ACME:** Automatic Certificate Management Environment. The protocol used by Let's Encrypt to automate certificate issuance and renewal. Defines the HTTP-01 and DNS-01 challenge types.

**Advisory lock:** A lock on a number the application chooses, taken and released through the database (PostgreSQL's `pg_advisory_lock` family). Used to make processes take turns at something the database itself does not lock, such as running migrations.

**API (Application Programming Interface):** A defined set of rules for how software components communicate. In this book, the Notes API provides an HTTP interface for CRUD operations on notes.

**Application factory:** A design pattern where the Flask app is created inside a function (`create_app()`) rather than at module level, enabling deferred configuration and easier testing.

**Authentication:** Establishing who is making a request. The Notes API authenticates with login tokens obtained by email and password (Chapter 38).

**Authorization:** Deciding what an authenticated user may do. In the Notes API, a note is visible only to its owner, enforced in every query.

**Autocommit:** A PostgreSQL mode where each SQL statement is immediately committed as its own transaction. Not the default; `psycopg2` begins transactions implicitly.

**Backlog:** The maximum size of the accept queue, passed as the argument to `socket.listen(backlog)`. Connection attempts beyond this limit are dropped (clients retry and eventually time out).

**bcrypt:** A deliberately slow, salted password-hashing function with an adjustable cost factor.

Fast hashes such as SHA-256 must never be used for passwords.

**Blueprint:** A Flask mechanism for grouping related routes, before/after request hooks, and error handlers. Blueprints are registered on the app with an optional URL prefix.

**CIDR notation:** A way of expressing IP address ranges. `10.0.0.0/8` means all IP addresses from `10.0.0.0` to `10.255.255.255`. The `/8` is the number of fixed prefix bits.

**Circuit breaker:** A reliability pattern that stops calling a failing service and returns immediate errors, giving the service time to recover.

**CNAME record:** A DNS record type that maps a hostname to another hostname (an alias). Cannot be used at the apex of a domain.

**Constraint:** A rule the database enforces on every write, whichever code makes it: NOT NULL, UNIQUE, CHECK, PRIMARY KEY, and foreign keys. Validation serves the client; constraints protect the data.

**Container:** An isolated execution environment that shares the host operating system kernel but has its own filesystem, network, and process space. Docker is the most common container runtime.

**Content negotiation:** The process where a client specifies acceptable response formats (via the Accept header) and the server chooses the best matching format.

**Context local:** A variable that appears global but stores different values for different execution contexts (threads, asyncio tasks). Flask's `request` and `g` are context locals.

**Cookie:** A small piece of data that the server asks the browser to store and send back with subsequent requests to the same domain. Used for sessions.

**CRUD:** Create, Read, Update, Delete — the four basic operations on persistent data.

**Deadlock:** Two transactions each waiting for a lock the other holds. PostgreSQL detects it and aborts one; the cure is to take locks in a consistent order, keep transactions short, and retry.

**DMA (Direct Memory Access):** Hardware mechanism that allows devices (like NICs) to read/write system memory directly without CPU involvement, reducing interrupt overhead.

**DNS (Domain Name System):** The global distributed database that maps human-readable domain names (`api.notes.example.com`) to IP addresses.

**Docker image:** A read-only template containing the filesystem layers that make up a container. Built from a Dockerfile.

**Dockerfile:** A text file containing instructions to build a Docker image. Each instruction that changes files (RUN, COPY, ADD) creates a layer.

**ETag:** An HTTP response header containing a version identifier for the resource. Clients send it back in `If-None-Match`; if it still matches, the server answers 304 Not Modified and the client reuses its cached copy.



**Endpoint:** In Flask, a named reference to a handler function. Every route has an endpoint name; `url_for("notes.list_notes")` builds the URL for the `list_notes` endpoint in the `notes` Blueprint.

**Environment variable:** A key-value pair stored in a process's environment, inherited by child processes. The primary mechanism for passing configuration (including secrets) to applications.

**epoll:** A Linux kernel mechanism for efficiently monitoring many file descriptors simultaneously for I/O readiness. Used by Nginx and async Python frameworks.

**File descriptor (fd):** An integer that the kernel assigns to an open file, socket, or other I/O resource. Processes use file descriptors to reference open resources.

**Flask:** A lightweight Python web framework that provides URL routing, request/response handling, and an extensible middleware system.

**Floating IP (cloud):** DigitalOcean's former name for a Reserved IP (see there).

**Graceful degradation:** Returning a partial or degraded response when a dependency fails, rather than returning a complete error.

**Graceful shutdown:** Allowing in-progress requests to complete before the server process exits, instead of terminating them abruptly.

**Foreign key:** A column whose value must match the key of a row in another table (REFERENCES `notes(id)`). It is how a relationship is stored, and ON DELETE decides what happens to it when the referenced row is deleted.

**g:** Flask's per-request namespace. Data stored in `g` is available throughout the current request (before-request hooks, handler, after-request hooks) and discarded at the end.

**GIL (Global Interpreter Lock):** A mutex in CPython that prevents multiple Python threads from executing Python bytecode simultaneously. Limits true parallelism in CPU-bound Python code.

**Gunicorn:** A Python WSGI HTTP server that implements the pre-fork worker model. The production server for the Notes API.

**HSTS (HTTP Strict Transport Security):** An HTTP response header that tells browsers to only connect to this domain via HTTPS for a specified duration.

**HTTP (HyperText Transfer Protocol):** The request-response protocol used for web communication. Defines methods (GET, POST, PUT, DELETE), status codes, and headers.

**HTTPS:** HTTP over TLS. All communication is encrypted and the server's identity is verified.

**Idempotency key:** A unique value sent with a request (often in an Idempotency-Key header) and repeated on every retry of it, so that the receiving service performs the action only once however many times it is asked.

**Idempotent:** An operation that produces the same result whether performed once or many times. GET, PUT, and DELETE are idempotent by definition in HTTP; POST is not.

**Image layer:** A read-only filesystem diff that Docker stacks to create container filesystems. Layers are cached and shared between images.

**Index (database):** A data structure that allows the database to find rows matching a WHERE clause without scanning the entire table. Critical for query performance.

**Inode:** A data structure in the Linux filesystem that stores metadata about a file (permissions, owner, size, timestamps, disk block locations) but not the filename.

**iptables:** The Linux kernel's packet filtering framework. Docker uses it to implement port forwarding (DNAT rules).

**ISO 8601:** The international standard for representing dates and times: YYYY-MM-DDTHH:MM:SS+HH:MM. The standard format for timestamps in JSON APIs.

**Isolation level:** How much a transaction sees of other transactions running at the same time: Read Committed (PostgreSQL's default), Repeatable Read, or Serializable. Stronger levels prevent more races by aborting conflicting transactions.

**Join:** Combining rows from several tables in one query where they match (JOIN ... ON). An inner join keeps only rows with a partner; a LEFT JOIN keeps every row of the left table, filling the missing side with NULL.

**JSON (JavaScript Object Notation):** A text-based data format using {} for objects, [] for arrays, "" for strings. The standard format for REST API request and response bodies.

**jsonify():** Flask's function for creating a JSON HTTP response. Serializes a Python dict to JSON and sets Content-Type: application/json.

**JWT:** JSON Web Token: a signed token that carries claims (such as a user id and an expiry) inside it, so it can be checked without a database, but cannot easily be revoked before it expires.

**Kernel space:** The region of memory where the operating system kernel runs. Code here has full hardware access. Isolated from user space.

**Lease:** A claim on a piece of work that expires by itself. A worker that takes a job holds it until locked\_until; if the worker crashes, the lease runs out and another worker takes the job.

**Let's Encrypt:** A free, automated Certificate Authority that issues TLS certificates via the ACME protocol.

**Load balancer:** A server that distributes incoming requests across multiple application servers. Used for horizontal scaling.

**Managed database:** A database service where the cloud provider handles installation, backups, updates, and failover. The Notes API uses DigitalOcean Managed PostgreSQL.

**Migration:** A numbered, versioned file that changes a database's schema, stored in the

repository and applied in order by a migration runner, which records in the database which ones have run.

**MIME type** (or media type): A two-part identifier for content formats. `application/json` means JSON data; `text/html` means HTML.

**Mode switch**: The CPU operation of transitioning from user mode to kernel mode (or vice versa), required for every system call. Costs ~100ns.

**MVCC** (Multi-Version Concurrency Control): PostgreSQL’s mechanism for allowing concurrent reads and writes without blocking each other, by maintaining multiple versions of rows.

**N+1 query problem**: Running one query for a list and then one more query for each item in it. Invisible locally, slow across a network; fixed by fetching the related rows for all items at once.

**NAPI** (New API): Linux’s interrupt mitigation mechanism for network I/O. Switches from interrupt-driven to polling mode under heavy load to improve throughput.

**NAT** (Network Address Translation): A technique that rewrites IP addresses in packet headers. Home routers use NAT to share a single public IP among multiple devices.

**Nginx**: A high-performance web server used in this book as a reverse proxy and TLS terminator (it can also serve static files, which the Notes API does not have).

**OOM killer**: The Linux kernel’s mechanism for terminating processes when the system runs out of memory, freeing memory for critical processes.

**Optimistic concurrency**: Detecting a conflicting change at write time instead of locking in advance: the update succeeds only if the row still has the version the change was based on (`WHERE version = ?`). Prevents lost updates.

**ORM**: Object-relational mapper (SQLAlchemy, Django’s ORM): a library that maps tables to classes and generates SQL from code. Convenient, but it hides the SQL it runs, including N+1 queries.

**OWASP**: Open Web Application Security Project. A non-profit that publishes security guidelines including the OWASP Top 10 list of critical web application security risks.

**Parameterized query**: A database query where variable values are passed separately from the SQL text, preventing SQL injection.

**PID 1**: The first process started when the Linux kernel boots. In a Docker container, PID 1 receives signals sent to the container and is responsible for passing them to child processes. The kernel ignores any signal PID 1 has no handler for — which is why a shell as PID 1 does not stop on `SIGTERM`.

**Port**: A 16-bit number (0-65535) that identifies a specific service on a host. Port 443 is HTTPS; port 5432 is PostgreSQL; port 5000 is the default Flask development server.

**Postmortem:** A written analysis of an incident, conducted after resolution, identifying root causes and action items to prevent recurrence. Blameless in good engineering cultures.

**PostgreSQL:** An open-source, production-grade relational database. The database used by the Notes API in production.

**Pre-fork model:** A process model where a master process forks worker processes before any connections arrive. Gunicorn’s default model.

**Prometheus:** An open-source monitoring system that collects metrics by scraping HTTP endpoints.

**ProxyFix:** A Werkzeug middleware that reads `X-Forwarded-For` and `X-Forwarded-Proto` headers from a trusted proxy (Nginx) to correctly set `request.remote_addr` and `request.scheme`.

**psycopg2:** The long-established PostgreSQL database driver for Python, used in this book (its successor is `psycopg`, version 3).

**Race condition:** A bug that occurs when the outcome depends on the relative timing of two or more concurrent operations.

**Rate limiting:** Restricting the number of requests a client can make in a given time window, to protect against abuse and ensure fair resource sharing.

**Read timeout:** How long a client waits for an answer after its request has been sent. When it expires, the outcome is unknown: the other side may have completed the action. Distinct from a connect timeout, which expires before anything was sent.

**Recursive resolver:** A DNS server that performs the full resolution chain (root → TLD → authoritative) on behalf of clients. Google’s `8.8.8.8` is a recursive resolver.

**Request context:** Flask’s per-request state container. Makes `request` and `session` available as apparent globals within the duration of a request (`g` and `current_app` belong to the application context, which Flask pushes alongside it).

**Reserved IP (cloud):** A public IP address you own independently of any server, assignable to one server at a time, so DNS records survive a server replacement (DigitalOcean’s name, formerly Floating IP; Elastic IP on AWS).

**REST (Representational State Transfer):** An architectural style for HTTP APIs where URLs identify resources and HTTP methods identify operations.

**Rollback:** Undoing an uncommitted database transaction, reverting all changes to the state before the transaction began.

**Runbook:** A step-by-step guide for responding to a specific operational situation or incident.

**Salt:** A random value mixed into each password before hashing, so identical passwords produce different hashes and precomputed tables are useless. `bcrypt` stores it inside the hash.

**SIGTERM:** The “polite shutdown” signal in Linux. Unicorn intercepts it to begin graceful shutdown. Contrast with SIGKILL (immediate, forceful termination).

**sk\_buff:** The Linux kernel’s data structure representing a network packet, from the NIC up to the socket’s receive queue (the application gets copies of the data through `recv()`).

**SKIP LOCKED:** A PostgreSQL option (`SELECT ... FOR UPDATE SKIP LOCKED`) that makes a query pass over rows other transactions have locked, instead of waiting for them. It lets many workers take jobs from one table at once.

**Socket:** An endpoint for network communication, represented by a file descriptor. Sockets can be TCP (connection-oriented, reliable) or UDP (connectionless).

**SQL injection:** An attack where user-supplied input is interpreted as SQL code, allowing an attacker to execute arbitrary queries. Prevented by parameterized queries.

**SSL/TLS:** TLS (Transport Layer Security) is the cryptographic protocol that secures HTTPS. SSL is the older predecessor; the terms are often used interchangeably, but SSL itself is obsolete and insecure, and no longer used.

**Staging environment:** A production-like environment for testing code before it reaches production.

**Stateless application:** An application that stores no per-session state locally. All state is in a shared database. Stateless applications can be horizontally scaled.

**strace:** A Linux tool that traces the system calls made by a process. Useful for understanding low-level behavior.

**systemd:** The init system and service manager used by Ubuntu (and most modern Linux distributions). Manages service startup, restart, logging, and dependencies.

**TCP (Transmission Control Protocol):** A connection-oriented, reliable transport protocol. Establishes connections via the three-way handshake (SYN, SYN-ACK, ACK).

**TLS (Transport Layer Security):** The cryptographic protocol providing encrypted communication over TCP. Used by HTTPS.

**Transaction:** A group of database operations that succeed or fail as one unit: all of them are committed, or none is.

**Transactional outbox:** Writing a message or job into a table in the same transaction as the change that causes it, so that both happen or neither does; the work is sent or done afterwards.

**TTL (Time to Live):** In DNS, the number of seconds resolvers may cache a record before re-querying. In other contexts, the expiration time for a cached value.

**URL (Uniform Resource Locator):** The address of a resource on the web. `https://api.notes.example.com/notes/42` identifies the note with ID 42.

**URL converter:** A Werkzeug component that constrains what a URL variable segment may contain and converts the matched string to a Python type.

**User space:** The region of memory where application code runs. Cannot directly access hardware; must use system calls to request kernel services.

**venv:** Python’s built-in module for creating virtual environments (`python -m venv .venv`).

**VPC (Virtual Private Cloud):** An isolated private network within a cloud provider. Resources in the same VPC can communicate over private IP addresses without traversing the public internet.

**WAL (Write-Ahead Log):** PostgreSQL’s durability mechanism. Changes are written to the WAL before being applied to data files. Enables crash recovery and replication.

**Werkzeug:** The WSGI utility library that Flask is built on. Provides URL routing (`Map`, `Rule`), request/response objects, and the development server (which parses HTTP itself; in production, Gunicorn does the parsing).

**Worker:** A process that runs background work outside any HTTP request, typically by taking jobs from a queue.

**WSGI (Web Server Gateway Interface):** The Python standard (PEP 3333) that defines how web servers (Gunicorn) communicate with web applications (Flask). An application is a callable (`environ, start_response`) → `iterable`.

**XSS (Cross-Site Scripting):** An attack where malicious scripts are injected into web pages viewed by other users. Prevented by escaping user-supplied content before rendering as HTML.

**Zero-copy:** A technique that avoids copying data between kernel and user-space buffers — for example `sendfile()`, which sends a file’s contents to a socket entirely inside the kernel.

# Appendix C — Command Cheat Sheet

---

This appendix collects the most frequently used commands from throughout the book, organized by tool.

## C.1 Python and pip

```
Create a virtual environment
python -m venv .venv

Activate the virtual environment
source .venv/bin/activate # Linux/macOS
venv\Scripts\activate # Windows

Install dependencies (requirements-dev.txt includes requirements.txt)
pip install -r requirements-dev.txt

Install a specific package
pip install flask==3.0.2

Upgrade a package
pip install --upgrade flask

Freeze current environment (only one without development tools)
pip freeze > requirements.txt

Audit for security vulnerabilities
pip-audit

Run the development server
python run.py
flask --app "app:create_app()" run --debug # the same, with auto-reload
(On macOS, port 5000 may be taken by AirPlay Receiver: use PORT=5001)
```

```
Open Flask shell (interactive Python with app context)
flask --app run shell

List all registered routes
flask --app run routes
```

## C.2 pytest

```
Run all tests
pytest

Run with verbose output
pytest -v

Run a specific test file
pytest tests/test_notes.py

Run a specific test by name (tests inside a class need the class too)
pytest tests/test_notes.py::TestCreateNote::test_create_note_returns_201

Run tests matching a keyword
pytest -k "create"

Show test output (don't capture stdout)
pytest -s

Stop on first failure
pytest -x

Run in parallel (requires pytest-xdist, not in requirements-dev.txt)
pytest -n auto

Generate coverage report (pytest-cov is in requirements-dev.txt)
pytest --cov=app --cov-report=html

Run only tests without a given marker (markers are defined by you,
e.g. @pytest.mark.slow, and registered in pyproject.toml)
pytest -m "not slow"
```



## C.3 Docker

```

Build an image
docker build -t notes-api:latest .
docker build -t notes-api:sha-a1b2c3d --no-cache .

Run a container as on the server: loopback-only port (Docker bypasses
UFW), restart policy, and time for a graceful shutdown
docker run -d \
 --name notes-api \
 --restart unless-stopped \
 --stop-timeout 35 \
 --env-file /path/to/notes-api.env \
 -p 127.0.0.1:5000:5000 \
 notes-api:sha-a1b2c3d

List running containers
docker ps

List all containers (including stopped)
docker ps -a

View container logs
docker logs notes-api
docker logs -f notes-api # Follow (stream) logs
docker logs --tail 50 notes-api # Last 50 lines
docker logs --since "1h" notes-api # Logs from the last hour

Execute a command inside a container
docker exec notes-api python -c "print('hello')"
docker exec -it notes-api /bin/sh # Interactive shell

Stop and remove a container
docker stop notes-api
docker rm notes-api
docker stop notes-api && docker rm notes-api

Remove all stopped containers
docker container prune

Inspect a container
docker inspect notes-api

```

```

View container resource usage
docker stats notes-api

Pull an image from a registry
docker pull ghcr.io/username/notes-api:sha-a1b2c3d

Push an image to a registry
docker push ghcr.io/username/notes-api:sha-a1b2c3d

Tag an image
docker tag notes-api:sha-a1b2c3d notes-api:latest

Remove an image
docker rmi notes-api:sha-a1b2c3d

Remove unused images
docker image prune

Build and push (common CI pattern)
docker build -t ghcr.io/username/notes-api:sha-$(git rev-parse --short HEAD) .
docker push ghcr.io/username/notes-api:sha-$(git rev-parse --short HEAD)

```

## C.4 Docker Compose

```

Compose v2 is a docker subcommand: "docker compose", not the old
"docker-compose" program (end-of-life since 2023)

Start all services (defined in docker-compose.yml)
docker compose up -d

Stop all services
docker compose down

Stop and remove volumes (delete database data)
docker compose down -v

View logs
docker compose logs -f

View logs for a specific service
docker compose logs -f app

```

```

Rebuild images and restart
docker compose up -d --build

Execute a command in a running service
docker compose exec app flask --app "app:create_app()" routes

Scale a service (horizontal scaling locally). Only works for a service
without a fixed host port: three copies cannot all publish port 5000
docker compose up -d --scale app=3

```

## C.5 Gunicorn

```

Start Gunicorn with a config file
gunicorn --config gunicorn.conf.py "app:create_app()"

Start Gunicorn with command-line options
gunicorn \
 --workers 2 \
 --bind 0.0.0.0:5000 \
 --timeout 30 \
 --access-logfile - \
 "app:create_app()"

Graceful restart of the workers (new workers start, old ones finish
their requests): send SIGHUP to the master – PID 1 in the container
docker kill --signal=HUP notes-api

Gunicorn's processes, seen from the host
docker top notes-api

```

## C.6 systemd

```

(For this book's setup, notes-api is a container, not a systemd service:
use the Docker commands above. These apply to any systemd service – nginx,
docker – and to notes-api if you chose the systemd unit of Chapter 24.)

Start a service
sudo systemctl start notes-api

Stop a service

```

```

sudo systemctl stop notes-api

Restart a service
sudo systemctl restart notes-api

Reload configuration without restart – only for services that support
it (nginx does); reload-or-restart falls back to a restart
sudo systemctl reload nginx
sudo systemctl reload-or-restart notes-api

Enable service to start on boot
sudo systemctl enable notes-api

Disable service from starting on boot
sudo systemctl disable notes-api

Check service status
sudo systemctl status notes-api

Reload systemd configuration (after editing unit files)
sudo systemctl daemon-reload

View service logs (via journald)
journalctl -u notes-api
journalctl -u notes-api -f # Follow (stream) logs
journalctl -u notes-api --since "1h ago" # Last hour
journalctl -u notes-api -n 50 # Last 50 lines

View boot logs
journalctl -b # This boot
journalctl -b -1 # Previous boot

```

## C.7 Nginx

```

Test configuration syntax
sudo nginx -t

Reload configuration (applies new config without dropping connections)
sudo systemctl reload nginx

Restart Nginx service
sudo systemctl restart nginx

```

```

View Nginx access log
tail -f /var/log/nginx/notes-api-access.log

View Nginx error log (the site's own, from Chapter 28)
tail -f /var/log/nginx/notes-api-error.log

Count requests by status code
awk '{print $9}' /var/log/nginx/notes-api-access.log | sort | uniq -c | sort -rn

Renew TLS certificate manually
sudo certbot renew --dry-run # Test renewal (no actual renewal)
sudo certbot renew # Perform renewal

Check certificate expiry
echo | openssl s_client -connect api.notes.example.com:443 2>/dev/null | \
 openssl x509 -noout -dates

```

## C.8 PostgreSQL

```

Connect to PostgreSQL interactively (managed databases require TLS)
psql "postgresql://user:password@host:25060/database?sslmode=require"

Execute a SQL command and exit
psql "$DATABASE_URL" --command="SELECT COUNT(*) FROM notes;"

Apply every pending migration / list them / record existing ones (Chapter 33)
python -m app.migrate
python -m app.migrate --status
python -m app.migrate --baseline 0002

Run the worker by hand; list the queue (Chapter 41)
python -m app.worker
psql "$DATABASE_URL" -c "SELECT status, count(*) FROM jobs GROUP BY status;"

Create a backup (pg_dump must be at least the server's major version)
pg_dump "$DATABASE_URL" --format=custom --file=backup.dump

Restore from backup
pg_restore --dbname="$TARGET_DATABASE_URL" --no-owner backup.dump

```

Inside `psql`, the backslash commands:

Command	Does
\l	List databases
\c database	Connect to a database
\dt	List tables
\d tablename	Describe a table
\q	Quit

(A # after a backslash command is *not* a comment in psql — it would be passed as an argument. psql comments start with --.)

## C.9 SSH and Server Management

```
SSH to the server
ssh deploy@203.0.113.42

SSH with a specific key (e.g. the CI key from Chapter 23)
ssh -i ~/.ssh/notes-api-ci deploy@203.0.113.42

Copy files to the server
scp localfile.txt deploy@203.0.113.42:/remote/path/

Copy files from the server
scp deploy@203.0.113.42:/remote/path/file.txt .

View disk usage
df -h

View memory usage
free -h

View CPU and process usage
top
htop # More readable (install with: apt-get install htop)

View listening sockets and their processes (process names need sudo)
sudo ss -tlnp

Check UFW firewall status
sudo ufw status verbose
```

```
View system logs
journalctl -n 50

Check server uptime
uptime
```

## C.10 DNS Debugging

```
Query a DNS record
dig api.notes.example.com A

Query a specific resolver
dig @8.8.8.8 api.notes.example.com A

Short output (just the answer)
dig +short api.notes.example.com A

Check MX records (mail servers – set on the mail domain, not the API name)
dig example.com MX

Trace the full resolution chain
dig +trace api.notes.example.com A

Reverse DNS lookup (IP to hostname)
dig -x 203.0.113.42

Check if DNS has propagated globally
(web tool, not command line)
https://dnschecker.org
```

## C.11 curl: Testing the API

```
GET request
curl https://api.notes.example.com/notes/

GET with verbose output (shows headers)
curl -v https://api.notes.example.com/notes/

POST with JSON body
```

```

curl -X POST https://api.notes.example.com/notes/ \
 -H "Content-Type: application/json" \
 -d '{"content": "Buy groceries"}'

POST with pretty JSON output (requires jq)
curl -s -X POST https://api.notes.example.com/notes/ \
 -H "Content-Type: application/json" \
 -d '{"content": "Buy groceries"}' | jq .

PUT (update)
curl -X PUT https://api.notes.example.com/notes/42 \
 -H "Content-Type: application/json" \
 -d '{"content": "Updated content"}'

DELETE
curl -X DELETE https://api.notes.example.com/notes/42

Log in (Chapter 38), then send the token with every request
curl -X POST https://api.notes.example.com/auth/login \
 -H "Content-Type: application/json" \
 -d '{"email": "ana@example.com", "password": "correct horse battery"}'
curl https://api.notes.example.com/notes/ \
 -H "Authorization: Bearer ns_..." # the "token" from the login response

Show only status code
curl -s -o /dev/null -w "%{http_code}" https://api.notes.example.com/health

Test with a local server (bypass TLS)
curl http://localhost:5000/notes/

Time the request
curl -w "Total: %{time_total}s\n" -s -o /dev/null
↪ https://api.notes.example.com/notes/

```



# Appendix D — Further Reading

---

The following resources go deeper into the topics covered in this book.

## D.1 Python and Flask

### Python Documentation

- [docs.python.org](https://docs.python.org) — The official Python documentation. Particularly valuable: the tutorial, the library reference, and the glossary.

### Flask Documentation

- [flask.palletsprojects.com](https://flask.palletsprojects.com) — Comprehensive Flask documentation including a quickstart, tutorial, and API reference.

### Werkzeug Documentation

- [werkzeug.palletsprojects.com](https://werkzeug.palletsprojects.com) — Flask’s underlying toolkit. Understanding Werkzeug’s routing (Map, Rule) and request/response objects gives a deeper understanding of Flask.

### Miguel Grinberg’s Flask Mega-Tutorial

- [blog.miguelgrinberg.com](https://blog.miguelgrinberg.com) — A comprehensive, regularly updated tutorial building a full Flask application with authentication, database migrations, and deployment.

### Two Scoops of Django (Greenfield, Greenfield)

- A Django book, but its chapters on project structure, settings management, and testing best practices apply equally to Flask.

## D.2 Databases and SQL

### Use The Index, Luke

- [use-the-index-luke.com](http://use-the-index-luke.com) — Free online guide to database indexing. The most accessible explanation of how indexes work and when to use them.

### PostgreSQL Documentation

- [postgresql.org/docs](http://postgresql.org/docs) — Comprehensive documentation for PostgreSQL. The chapter on performance tips and the EXPLAIN guide are particularly valuable.

### Designing Data-Intensive Applications (Kleppmann)

- O'Reilly, 2017 — A deep exploration of how databases, message queues, and distributed systems work. The chapters on transactions and replication are relevant to production database use.

### Database Internals (Petrov)

- O'Reilly, 2019 — A detailed exploration of how databases store and retrieve data. More advanced but invaluable for understanding why databases behave the way they do.

## D.3 Networking and HTTP

### HTTP: The Definitive Guide (Gourley, Totty, Sayer, Aggarwal, Reddy)

- O'Reilly, 2002 — Comprehensive coverage of the HTTP protocol. Despite its age, the fundamental coverage is still accurate.

### Computer Networking: A Top-Down Approach (Kurose, Ross)

- Pearson, multiple editions — The standard university textbook for networking. Excellent coverage of TCP/IP, DNS, and HTTP.

### High Performance Browser Networking (Grigorik)

- O'Reilly, 2013 — Free at [hpbnc.co](http://hpbnc.org). Deep coverage of network protocols (TCP, TLS, HTTP/1.1, HTTP/2) with performance implications.

### The Linux Programming Interface (Kerrisk)

- No Starch Press, 2010 — Comprehensive guide to Linux system programming, including sockets, processes, and signals. The definitive reference for understanding system calls.

## D.4 Production Operations

### The Site Reliability Engineering Book (Beyer, Jones, Petoff, Murphy)

- Google’s SRE book. Free at [sre.google/books](https://sre.google/books). Defines SLOs, error budgets, the four golden signals, and operational best practices at scale.

### The Site Reliability Workbook (Beyer, Murphy, Rensin, Kawahara, Thorne)

- Google’s follow-up. More practical, with worked examples of SLO implementation.

### The Art of Monitoring (Turnbull)

- Leanpub. Practical guide to building a monitoring system including metrics, logging, and alerting.

### Release It! (Nygard)

- Pragmatic Bookshelf, second edition 2018. Covers stability patterns (circuit breakers, time-outs, bulkheads) that make distributed systems resilient.

### The Twelve-Factor App

- [12factor.net](https://12factor.net) — Free. A methodology for building software-as-a-service applications with explicit focus on environment parity, configuration, and disposability.

## D.5 Security

### OWASP Top 10

- [owasp.org/www-project-top-ten](https://owasp.org/www-project-top-ten) — The definitive list of the most critical web application security risks, with descriptions and mitigations.

### The Web Application Hacker’s Handbook (Stuttard, Pinto)

- Wiley, 2011 — A thorough exploration of web application security from an attacker’s perspective. Understanding how attacks work helps you defend against them.

### Hacking: The Art of Exploitation (Erickson)

- No Starch Press, 2008 — Focuses on lower-level exploitation (buffer overflows, shellcode), but provides invaluable intuition for why security matters.

### The Tangled Web (Zalewski)

- No Starch Press, 2011 — A thorough exploration of browser security mechanisms (same-origin policy, cookies, CSP). Essential for understanding XSS and CSRF.

## D.6 Docker and Containers

### Docker Documentation

- docs.docker.com — Comprehensive documentation for Docker including Dockerfile reference, networking, and best practices.

### Container Security (Rice)

- O'Reilly, 2020 — Covers container security from image scanning to runtime protection.

### Kubernetes in Action (Lukša)

- Manning — The next step after Docker Compose. When your application needs to run across multiple servers with automatic scaling and self-healing, Kubernetes is the answer.

## D.7 Performance

### Systems Performance (Gregg)

- Addison-Wesley, second edition 2020 — Comprehensive guide to performance analysis at the system level (CPU, memory, disk, network). The chapter on application performance profiling is especially valuable.

### Python High Performance (Lanaro)

- Packt, 2017 — Profiling Python code, using NumPy for numerical performance, Cython for compiled extensions, and asyncio for concurrent I/O.

### Brendan Gregg's website (brendangregg.com)

- Articles, tools, and talks on Linux performance analysis (perf, flame graphs, BPF), by the author of *Systems Performance* and *BPF Performance Tools*. Advanced but irreplaceable for diagnosing production performance problems.

## D.8 Command Line and Linux

### The Linux Command Line (Shotts)

- No Starch Press, free at [linuxcommand.org](http://linuxcommand.org) — Comprehensive introduction to the bash shell and Linux command line. Essential reading for anyone working on Linux servers.

**UNIX and Linux System Administration Handbook** (Nemeth, Snyder, Hein, Whaley, Mackin)

- Addison-Wesley, fifth edition 2017 — The comprehensive reference for Linux (and Unix) system administration.

## D.9 Online Resources

**Stack Overflow** ([stackoverflow.com](http://stackoverflow.com)) — The best resource for specific technical questions. When you encounter a confusing error message, searching Stack Overflow is usually the fastest path to an answer.

**The Pragmatic Engineer** (newsletter, [blog.pragmaticengineer.com](http://blog.pragmaticengineer.com)) — A newsletter covering software engineering practices, career development, and industry trends.

**Hacker News** ([news.ycombinator.com](http://news.ycombinator.com)) — Technology news aggregator. The comments on system design, production incidents, and technical architecture discussions are often more valuable than the linked articles.

**DigitalOcean Tutorials** ([digitalocean.com/community/tutorials](http://digitalocean.com/community/tutorials)) — High-quality tutorials covering all aspects of server administration, Nginx configuration, PostgreSQL setup, and more. Written for the exact infrastructure used in this book.

**Mozilla Developer Network** ([developer.mozilla.org](http://developer.mozilla.org)) — The authoritative reference for HTTP headers, status codes, and web standards. When in doubt about what an HTTP header does, MDN has the answer.



# Appendix E — Walking Into an Unfamiliar System

---

For most of your career you will *join* systems you did not build — switching jobs, teams, or projects, and inheriting code someone else wrote. The instinct to read it top to bottom is a trap; large systems are too big for that, and code shows you *what* without *why*. A faster path is a fixed set of questions — the same ones this book answered for the Notes API — that recover any system’s *shape* quickly. They work on any stack, because every running system, whatever its language or scale, has the same skeleton: a process, a network boundary, a request path, a data store, a deployment, and a way of being observed.

This is that checklist. Ask these, roughly in this order.

## **Orient (first hour)**

1. **What is it, and why does it exist?** Read the README and any docs. What problem does it solve, and who uses it? (You are reconstructing the problem statement — the framing work of Part I, *Product Discovery*.)
2. **How does it start as a process?** Find the entry point (`run.py`, `main`, a container CMD) and what supervises it — `systemd`, a container runtime, an orchestrator. The start command tells you more than any single file.
3. **Can I run it locally?** Get it running on your own machine. Nothing teaches a system faster than booting it; the setup steps reveal its real prerequisites and configuration.

## **Trace (first day)**

4. **Where is the network boundary?** What port does it listen on, behind what proxy or load balancer, reachable at what public name? (Chapters 3 and 26–28.)
5. **What is the request path?** Pick one real request and trace it end to end: routing → handler → data access → response. Find the route table; it is the system’s map of what it can do. (Chapters 29–34.)

6. **Where is the data?** Which database(s)? Dump and read the schema — keys, foreign keys, constraints ([Chapter 6](#)). Where does the connection config live?
7. **What does it depend on, and how is it configured?** Read the dependency manifest and the `.env.example / config`. What external services does it call, and what changes between local and production? (Chapters 12–14 and 39.)

### Operate (first week)

8. **How is it deployed?** Find the build and deploy scripts or the CI pipeline. How does a commit become running production? (Chapters 17–25.)
9. **How do I see it?** Where are the logs, metrics, dashboards, and health checks? ([Chapter 35](#).) If you cannot observe it, you cannot operate it.
10. **How does it fail, and who is on call?** Look for runbooks, past incidents or postmortems, and alert definitions. ([Chapter 37](#).)
11. **Why is it the way it is?** Read any decision records (ADRs — [Chapter 39](#)) and skim the git history. The constraints and rejected alternatives explain the odd-looking parts.

You will not get clean answers to all of these — and the *gaps* are themselves information (no runbooks, no decision records, undocumented config). The goal is not mastery on day one; it is to build the skeleton fast, so every new detail has somewhere to attach. Trace one request, run it yourself, and read the schema: those three alone will orient you in almost any system you are ever handed.



# Appendix F — The Mental-Model Spine

---

This is the whole book compressed to its load-bearing ideas — dense enough to re-read in one sitting to refresh the model. If a line here does not ring a bell, the part behind it is where to look.

**The spine, in one breath:** *Code becomes a process; the process binds a port; traffic reaches that port through networks and infrastructure; the program receives input; a code path is selected; logic runs; data is accessed; a response is produced; and the whole system is observed, scaled, secured, and sustained — then, under pressure, distributed.*

**Before the first line of code (Parts I–III of the book):** discover the problem before building a solution — talk to users, test the riskiest assumption cheaply, and write down what “solved” means (Product Discovery); turn the answer into a plan with a scope, milestones, risks, and an honest estimate (Project Planning); and write code that others can read, test, and change — naming, small functions, tests, reviews, and practices that grow with the team (Writing Good Code). The spine below is the rest of the journey, the running system itself, in the eleven stages of Part IV.

**Stage by stage:**

- **1 — Locally:** a command goes shell → fork/exec → a process with memory and file descriptors. The runtime initializes, imports, and builds the app. Network capability is socket → bind → listen → accept. Routes are a table mapping method+path to a handler. A request runs accept → read → parse → route → logic → data → serialize → send. Local dependencies: runtime, libraries (in a venv), a database, env vars.
- **2 — Why local isn’t enough:** “works on my machine” hides environment drift, hidden dependencies, hardcoded config, loopback-only networking, and the absence of restart, monitoring, HTTPS, and auth.
- **3 — Portable:** pin dependencies; externalize config into validated env vars; declare the runtime contract; structure the app (factory + blueprints, high cohesion / low coupling); log structurally and handle errors at a boundary.

- **4 — Packaged:** a build produces an artifact; a Docker image layers base + deps + code; a registry stores versioned images (tag by git hash, never just `latest`).
- **5 — Environment:** a server is CPU/memory/disk + an OS + a network identity; set up users, directories, and a firewall; know public vs private IP.
- **6 — Deploy & keep alive:** pull the image, run it with Gunicorn bound to `0.0.0.0` *inside* the container but published only on the host's `127.0.0.1` (behind Nginx), and supervise it (Docker's restart policy, with systemd starting Docker), with logs to stdout.
- **7 — Reachable:** packets hit the NIC → kernel → port-to-process; DNS maps name to IP; a reverse proxy (Nginx) routes and terminates TLS.
- **8 — Input arrives:** the client builds a request → DNS → TCP handshake → packets → kernel socket → the application's buffer.
- **9 — In-program:** validate untrusted input; route to the handler; run business logic inside database transactions; build the response (status, headers, JSON).
- **10 — Production reality:** observe it (logs, metrics, traces; the four golden signals; health checks); scale it (vertical, then horizontal; pooling, caching, rate limiting); make it reliable (failure modes, retries with backoff, circuit breakers, graceful shutdown, tested backups, runbooks, postmortems); secure it (HTTPS, authn/authz, secrets, OWASP, validation); and keep it legible (decisions as ADRs, delivery as versioning + changelog).
- **11 — Beyond one service:** distribute only under proven pressure (the network is unreliable; partial failure is normal; replication and partitioning; CAP; sagas; idempotency; consensus). Decouple in time with events (queues, at-least-once + idempotent consumers, pub/sub, backpressure). Separate the analytical plane from the operational one (warehouse/lake, ELT, star schemas). Automate fleet operations with orchestration (Kubernetes = desired state + a control loop = systemd for a fleet; infrastructure as code; serverless to own less).

### The principles that recurred everywhere:

- Trace it end to end; localize a failure to a *layer* before you debug.
- The network is unreliable and partial failure is normal — hence timeouts, retries, and **idempotency** wherever a retry can happen.
- High cohesion, low coupling; deep modules; information hiding — complexity is the real enemy.
- Externalize configuration; keep one source of truth per fact; make the implicit explicit (dependencies, contracts, decisions).
- You cannot operate what you cannot see — observe before you scale.
- Fail fast at the boundaries; handle errors only where you can act.

- Simplicity is the default; reach for distribution and orchestration only when a single machine genuinely cannot cope.
- Documentation is a deliverable, and the *why* is the most perishable knowledge in a project.

If you can narrate the spine from memory — command to process to production and beyond — and name the principle behind each move, your mental model is intact. Everything else is detail you can re-derive or look up.



# Appendix G — Python Essentials Used in This Book

---

This book assumes you can already write basic Python: variables, `if` and `for`, functions, lists and dictionaries. Its code also uses a set of features that a first course often skips. This appendix collects them, each with a short example and a pointer to where the book uses it. Read it once before Part IV, or come back when a listing uses something unfamiliar.

## G.1 Modules, Packages, and Running Code

A **module** is one `.py` file. A **package** is a directory of modules with an `__init__.py` file, which Python runs when the package is first imported. The Notes API is the package `app/` (Chapter 15).

```
app/routes/notes.py imports from its sibling modules by their full path
from app.database import get_db, sql
from app.validation import Validator
```

Every module has a variable `__name__`: the module's import name ("`app.routes.notes`"), or "`__main__`" when the file is the program being run. That gives the common guard:

```
def main() -> int:
 ...
 return 0

if __name__ == "__main__": # only when run as a program, not when imported
 sys.exit(main()) # the return value becomes the exit status (Chapter 1)
```

`python -m app.migrate` runs a module *inside* a package as a program. Because Python starts

from the current directory, `from app.x import y` works, which it would not with `python app/migrate.py`.

## G.2 Functions: Arguments and Unpacking

**Keyword-only arguments.** Parameters after a bare `*` must be named by the caller. The book uses this for options that would be unreadable as bare values:

```
def _access(cursor, note_id: int, *, lock: bool = False) -> str | None: ...

_access(cursor, 42, lock=True) # clear
_access(cursor, 42, True) # TypeError: lock must be named
```

**\*args and \*\*kwargs** collect any extra positional and keyword arguments, into a tuple and a dictionary. A decorator uses them to pass arguments through without knowing what they are (G.3).

**Unpacking** does the reverse: it spreads a sequence or a dictionary out.

```
params = [user_id, after]
cursor.execute(query, (*params, limit + 1)) # a tuple: (user_id, after, 21)

defaults = {"level": "INFO", "format": "human"}
settings = {**defaults, "format": "json"} # a copy, with one key changed
```

**Default values are evaluated once**, when the function is defined. Never use a mutable default such as `def f(items=[])`: every call would share one list. Use `None` and create the list inside.

A **lambda** is a one-expression function without a name, handy as an argument: `sorted(notes, key=lambda note: note["id"])`.

A **closure** is a function defined inside another, which remembers the outer function's variables. The book's test fixtures return closures as *factories* (Chapter 17):

```
def make_counter():
 count = 0

 def next_value() -> int:
 nonlocal count # assign to the outer variable, not a new local one
 count += 1
 return count

 return next_value

counter = make_counter()
counter(), counter() # (1, 2)
```

## G.3 Decorators

A **decorator** is a function that takes a function and returns a replacement. @name above a def is shorthand for `func = name(func)`. Flask's `@app.route("/notes/")` registers the function in the URL map and returns it unchanged (section 32.1). A decorator that *wraps* a function looks like this one from Chapter 38:

```
from functools import wraps

def login_required(view):
 @wraps(view) # keep the original's name and docstring (Flask needs the name)
 def wrapped(*args, **kwargs):
 # A 401 response if the request has no valid token; otherwise None,
 # and the view runs with every argument passed through
 return authenticate_request() or view(*args, **kwargs)

 return wrapped
```

## G.4 Type Hints

Type hints describe what a function expects and returns. Python does not check them when the code runs. Tools do: mypy (Chapter 17) reads them and reports mistakes before the code runs.

```
def to_iso(value) -> str: ... # returns a string
def find(note_id: int) -> dict | None: ... # a dict, or None
tags: list[str] = []
counts: dict[str, int] = {}
```

X | None (Python 3.10 and later) means “an X or None”. When a function can return None, the caller must handle that case, and mypy will say so if it does not.

## G.5 Strings

**f-strings** put values into text: `f"Note {note_id} not found"`. Inside the braces, any expression works, and a format can follow a colon: `f"{elapsed_ms:.1f} ms"`.

**Triple-quoted strings** span several lines. The book writes SQL in them. Never put *values* into SQL with an f-string: pass them as parameters (`?` or `%s`), so the database driver quotes them safely (Chapters 31 and 38). The book’s f-strings in SQL only insert fixed text, such as a list of column names or a number of placeholders.

**Raw strings**, `r"..."`, keep backslashes as they are, which regular expressions need: `re.compile(r"[a-z0-9]+(?:-[a-z0-9]+)*")`.

**Bytes are not text.** `"é".encode()` gives the bytes `b'\xc3\xa9'`, and `bytes.decode()` turns them back. Hashing, network code and `bcrypt` work on bytes (Chapter 38).

## G.6 Comprehensions and Built-ins

A **comprehension** builds a list, dictionary or set from a loop in one expression:

```
names = [row["name"] for row in rows]
by_id = {note["id"]: note for note in notes}
long_notes = [note for note in notes if len(note["content"]) > 100]
```

Built-ins the book uses often:



Built-in	Example	Note
<code>dict.get</code>	<code>data.get("limit", 20)</code>	a default instead of a <code>KeyError</code>
<code>isinstance</code>	<code>isinstance(value, str)</code>	check a type; JSON input can be anything
<code>sorted</code>	<code>sorted(names, key=str.lower)</code>	a new sorted list
<code>any, all</code>	<code>all(isinstance(t, str) for t in tags)</code>	stop at the first answer
<code>len, min, max</code>	<code>max(1, min(100, limit))</code>	clamp a value into a range

The **walrus operator**, `:=`, assigns inside an expression. It is handy for a loop that runs until a function returns `None`, as in the companion repository's tests:

```
while (job := jobs.claim(conn)) is not None:
 worker.process(conn, job)
```

## G.7 Exceptions

```
try:
 response = requests.post(url, json=body, timeout=(2, 5))
except requests.ReadTimeout as error:
 raise EmailOutcomeUnknownError("no answer in time") from error
else:
 log_success(response) # only when no exception happened
finally:
 cleanup() # always, with or without an exception
```

- **Your own exceptions** are classes that inherit from `Exception`: `class EmailRejectedError(Exception): pass`. Their names end in `Error` by convention.
- **`raise ... from error`** keeps the original exception attached, so the traceback shows both: what went wrong, and what it meant for your code.
- **Catch only what you can handle**, and only where you can act on it. Everything else goes up to one handling boundary, which logs it once (section 16.5).

## G.8 Context Managers: with

A `with` block guarantees that clean-up happens, even when an exception leaves the block:

```

with open("config.json") as f: # the file is closed at the end of the block
 config = json.load(f)

with app.app_context(): # Flask's application context, in scripts and tests
 db = get_db()

with pytest.raises(ValueError): # the test fails unless the block raises
 parse("not a number")

```

## G.9 Classes, Briefly

The book's own code is mostly functions. It uses classes where a library expects them, or where an object must carry state:

```

class RequestIDFilter(logging.Filter): # inherits from logging.Filter
 def filter(self, record) -> bool: # overrides the method the library calls
 record.request_id = g.get("request_id") # (simplified; Chapter 35)
 return True

class Worker: # Chapter 41
 def __init__(self): # runs when Worker() creates an object
 self.stopping = False # self is the object itself

 def stop(self, signum, frame): # called on SIGTERM
 self.stopping = True

```

`super().__init__()` calls the parent class's version of a method, when a subclass extends it rather than replacing it.

## G.10 Generators and `yield`

A function that contains `yield` is a **generator**: it pauses at each `yield` and resumes on the next request for a value. `pytest` uses this for fixtures that must clean up (Chapter 17):

```

@pytest.fixture
def conn():
 connection = open_db()
 yield connection # the test runs here
 connection.close() # then this runs, even if the test failed

```

## G.11 The Standard Library the Book Leans On

Module	Used for	Chapters
<code>os</code>	environment variables ( <code>os.getenv</code> ), files, processes	1, 13
<code>sys</code>	the exit status, <code>sys.argv</code> , standard error	1, 33
<code>pathlib</code>	paths as objects: <code>Path("migrations") / name</code>	7, 33
<code>json</code>	JSON text to Python values and back	5, 34
<code>logging</code>	structured, levelled log output	16, 35
<code>datetime</code>	times. Always timezone-aware: <code>datetime.now(timezone.utc)</code>	16, 38
<code>re</code>	regular expressions for validation	31, 33
<code>secrets</code>	tokens and keys that must be unguessable	13, 38
<code>hashlib</code>	SHA-256 of login tokens	38
<code>uuid</code>	unique ids: request ids, idempotency keys	35, 37, 41
<code>time</code>	measuring durations, waiting ( <code>time.perf_counter</code> , <code>time.sleep</code> )	16, 41
<code>subprocess</code>	running another program	1, 8
<code>signal</code>	reacting to SIGTERM for a graceful stop	41
<code>sqlite3</code>	the embedded database, before PostgreSQL	2, 12, 33

Two traps worth remembering. A **naïve** `datetime` (one without a time zone) means a different moment on every server: compare only timezone-aware ones. And `random` is predictable: anything secret comes from `secrets`.

## G.12 Async, Briefly

`async def` defines a **coroutine**, and `await` pauses it until a slow operation, such as a network call, finishes, while other coroutines run. It lets one process wait on thousands of connections at once. The Notes API does not use it: each Gunicorn worker handles one request at a time with ordinary functions, and the slowest call, sending email, happens in the separate worker process of Chapter 41. Section 36.8 explains when `async` is worth its extra complexity.

*This book covered the journey from deciding what to build, through planning it and writing it well, to a single Python file becoming a running production system — and the thinking that*

*stretches beyond it. The appendices provide the vocabulary to understand what you've built, the commands to operate it, a way to find your feet in any system you inherit, a one-sitting refresher of the whole model, and the Python it was all written in.*

*Good luck building, deploying, and operating your systems.*



## Code that works is the floor, not the goal.

Most developers learn to write code on their own machine, and then discover that nobody ever showed them the rest: how to decide what is worth building, how to plan it, how to write it so it can change safely, and how to take it all the way to a running, reliable production system.

This book is that missing map. It covers the whole lifecycle in one place, and it follows a real system through every layer, from the command that starts a process to the packet that reaches it from across the internet.

- I Product Discovery
- II Project Planning
- III Writing Good Code
- IV Building Real Applications